Ambersong

Firmware Gospel

What the two boards run, and how to build, update, repair and change it

Release	Version 1 (tag `v.1`), declared 2026-11-10
Hardware revision	A
Firmware, main board	`v.1.5.20260927T141322Z`
Firmware, audio board	`v.1.4.20260926T153301Z`
This PDF exported	2026-11-11
Author	Zeroshin

Ambersong is a 1960s LLOYDS TM-838N AM/SW/FM tube radio rebuilt in a wooden cabinet with two ESP32 boards. This book documents their firmware to the best of the authors' knowledge.

Colophon

Item	Value
Release	Version 1, declared 2026-11-10
Exported	2026-11-11
Hardware revision	A
Firmware version	Main board (ESP32-S3) `v.1.5.20260927T141322Z`. Audio board (ESP32) `v.1.4.20260926T153301Z`.
Author	Zeroshin
Licences	This document: `CC-BY-SA-4.0`. Hardware design (KiCad, STL and SketchUp files): `CERN-OHL-S-2.0`. Firmware: `GPL-3.0-or-later`. Photos and renders: `CC-BY-NC-ND-4.0`. The running hare on the dial: all rights reserved.
Exported from commit	`9de2478`

Callouts in this document:

> ⚠ DANGER Hazard that will kill or seriously injure, its consequence, and how to avoid it.

> ⚠ WARNING Hazard that could kill or seriously injure, its consequence, and how to avoid it.

> NOTICE Risk of damage to the radio or to what it is connected to, and how to avoid it. No injury.

> Note A fact not to miss. It is not a hazard.

The other book, the Markdown editions of both books and every source file are in the Ambersong repository: start with its `README.md`.

Contents

About this book 1
- What this book covers 1
- How the chapters are laid out 1
- The recovery ladder 2
- Conventions 3
- Your mileage may vary 3

1. Rescue: when the radio misbehaves 3
- 1.1 First things first 4
- 1.2 The console without a cable 4
- 1.3 The portal will not open 5
- 1.4 Getting back to the house network 6
- 1.5 The needle says FAULT, or points to the wrong place 6
- 1.6 The frequency on the dial is wrong 7
- 1.7 No sound, the wrong source, or Bluetooth misbehaving 8
- 1.8 The console says “A32 silent” 9
- 1.9 An update went wrong 9
- 1.10 The clock shows the wrong time 11
- 1.11 Settings look lost, or SETTINGS LOCKED 12
- 1.12 When you need the USB cable 12
- 1.13 What the lights and pills mean 12

2. The machine at a glance 14
- 2.1 What the machine is 14
- 2.2 How the needle follows the knob 15
- 2.3 Two boards, and why two 16
- 2.4 What runs where 18
- 2.5 How information moves 20
- 2.6 The source tree 23
- 2.7 Words used everywhere 24
- 2.8 Where to read what 25

3. Building, flashing and updating 25
- 3.1 Building and flashing 25
- 3.9 The first boot 26
- 3.10 Updating over the air 28
- 3.11 Trial, confirmation and rollback 32
- 3.12 Walk-through: updating both boards 36
- 3.13 Versions, and where to read them 37
- 3.14 Build and update constants 38
- 3.15 When it goes wrong 39
- 3.16 Design choices 41
- 3.17 Tried and rejected 42
- 3.18 Known limits 43
- 3.19 Changing it safely 44

4. The main board: start-up, clock, lamps and time 46

4.1 What you see 46
4.2 Start-up, step by step 47
4.3 Tasks and cores: the rules a modifier must keep 50
4.4 The main loop 51
4.5 Amplifier sense and “radio live” 52
4.6 The clock display 53
4.7 Brightness and panel lamps 55
4.8 During an upload 56
4.9 Time keeping 57
4.10 Health reporting 59
4.11 The watchdog 60
4.12 Settings, console keys and constants 61
4.13 When it goes wrong 65
4.14 Design choices 66
4.15 Tried and rejected 68
4.16 Known limits 69
4.17 Changing it safely 70
5. The dial needle 73
5.1 What you see 73
5.2 The chain, layers and tasks 74
5.3 Units and the position frame 76
5.4 Driving the motor 78
5.5 The step emitter 79
5.6 How the needle moves 80
5.7 The supervisor 82
5.8 Reading the index switch 83
5.9 Homing 84
5.10 Sweep, tracking, parking 87
5.11 Staying right 88
5.12 Calibrating the index band and setting the soft limits 92
5.13 When the angle sensor drops out 95
5.14 Memory across restarts 95
5.15 Stops, hunting, uploads 96
5.16 Portal and console 98
5.17 Settings and constants 100
5.18 Physical values of this machine 103
5.19 When it goes wrong 108
5.20 Design choices 111
5.21 Tried and rejected 114
5.22 Known limits 116
5.23 Changing it safely 118
6. Tuning and the dial’s self-calibration 120
6.1 What you see 121
6.2 Radio terms you need 122
6.3 Units and tasks 124

6.4 From shaft angle to megahertz 125
6.5 The calibration curve 127
6.6 The RDA5807M as an instrument 128
6.7 One measurement 131
6.8 The automatic measurement 136
6.9 Buttons and keys 137
6.10 Calibrating from scratch 138
6.11 Settings and constants 139
6.12 When it goes wrong 142
6.13 Design choices 143
6.14 Tried and rejected 147
6.15 Known limits 149
6.16 Changing it safely 151
7. Settings 153
7.1 What a setting is and where you change it 153
7.2 Where settings are stored 154
7.3 How a change is saved (the main board) 156
7.4 The audio board's settings and the main board's mirror 161
7.5 Firmware updates and stored settings 164
7.6 Backup and restore: the settings file 167
7.7 When it goes wrong 172
7.8 Design choices 175
7.9 Tried and rejected 177
7.10 Known limits 178
7.11 Changing it safely 180
8. Network, web portal and console 186
8.1 Reaching the radio 187
8.2 First-time setup 187
8.3 How the radio stays on the network 191
8.4 Transmit power 197
8.5 Network name and network time 199
8.6 Accounts, sign-in and security 199
8.7 How the portal works inside 202
8.8 What the page shows 208
8.9 The console, over USB and the web 209
8.10 Updating from the portal 212
8.11 When it goes wrong 212
8.12 Design choices 215
8.13 Tried and rejected 218
8.14 Known limits 219
8.15 Changing it safely 221
9. The audio board and the link between the boards 224
9.1 Two computers and the wire between them 225
9.2 How the audio board starts and runs 226
9.3 Reading the audio board's boot report 228

9.4 Awake and asleep 229
9.5 The link, and how to read its health 231
9.6 Frames, the decoder and versions 232
9.7 Handshake and heartbeat 236
9.8 Updating the audio board, and its trial 239
9.9 The battery clock on the audio board 240
9.10 When it goes wrong 242
9.11 Design choices 245
9.12 Tried and rejected 247
9.13 Known limits 248
9.14 Changing it safely 249
10. Sound and Bluetooth 256
10.1 What you hear 256
10.2 The audio path 258
10.3 The audio interface (I2S) 259
10.4 Inside the audio board 262
10.5 Volume, knob, fades, mute and gains 267
10.6 Changing source 270
10.7 Bluetooth: who may connect, and when 270
10.8 Memory: the audio board's tightest limit 275
10.9 What the audio board reports 276
10.10 Pops: what was found, and what is still open 277
10.11 When it goes wrong 279
10.12 Design choices 281
10.13 Tried and rejected 282
10.14 Known limits 285
10.15 Changing it safely 286
11. Build scripts 289
12. Known issues, ideas and untested paths 290
12.1 Index by symptom 290
12.2 Tuning 292
12.3 Needle 294
12.4 Settings, portal and console 295
12.5 The link and the audio board 297
12.6 Sound 300
12.7 Main board, clock, display and build 301
12.8 Texts that say something false 302
12.9 Ideas for improvement 303
12.10 Untested paths 305
12.11 Code tidy list 306
Appendix A. Settings reference 309
A.1 Audio and Bluetooth (the audio board) 309
A.2 Display and lights 312
A.3 Needle and tuning 313
A.4 Network, accounts and stored state 317

A.5 Constants of the settings system 318
A.6 Experiments: actions, never saved 320
Appendix B. Network and portal constants, HTTP endpoints, state fields .. 321
B.1 Network constants 321
B.2 Portal constants 322
B.3 Page and console constants 324
B.4 HTTP endpoints 324
B.5 State fields 327
Appendix C. Portal actions 328
C.1 Bluetooth 329
C.2 Needle 329
C.3 Tuning and the dial 330
C.4 Display 330
C.5 Audio and the audio board 331
C.6 Audio experiments 331
C.7 System and network 332
Appendix D. Console keys 332
D.1 Main board 332
D.2 Audio board 335
Appendix E. Portal map: tabs, cards, buttons and the calls they make 336
E.1 Before the tabs 336
E.2 The tabs, in order 336
E.3 Controls and their calls 337
E.4 What has no control on the page 338
Appendix F. Link reference: constants, messages and payloads 338
F.1 The frame 338
F.2 Constants 339
F.3 Messages 341
F.4 Payloads 343
F.5 What the link prints 344
Appendix G. Audio constants and experiments 344
G.1 Compiled constants 345
G.2 Experiments 346
Appendix H. The pop hunt in detail 348
H.1 What a pop is 348
H.2 What the pops were not 349
H.3 Was it the data line? 352
H.4 Was it the clocks? What cut the pops 353
H.5 What did drive 3 cost? 355
H.6 Where it stands 355
H.7 What is still open 356
H.8 How to count pops 356
Appendix I. Pins as the firmware uses them 358
I.1 Main board (S3) 358
I.2 Audio board (A32) 360

Index 361

Figures

Figure 1 The tuning knob, the two sensors, and the needle: the firmware is the only link between knob and needle 15
Figure 2 Both boards, their cores and tasks, the link between them, and the audio path 18
Figure 3 The tuning chain: from the knob's angle to the frequency shown and to the needle's motor 20
Figure 4 What the two boards tell each other over the link 21
Figure 5 Updating the main board over the air 29
Figure 6 Updating the audio board: the main board relays the file over the link, one acknowledged kilobyte ata time 30
Figure 7 The life of a new program on either board: on trial, then confirmed or rolled back 32
Figure 8 The S3′s start-up, step by step 48
Figure 9 Where the time comes from, and how it reaches the clock digits . 57
Figure 10 The dial's travel, measured in motor half-steps 76
Figure 11 Homing, phase by phase 84
Figure 12 The re-index ladder and the routine index check 90
Figure 13 The tube set mixes the station with its own oscillator; the oscillator leaks to the RDA5807M, and the firmware adds the IF back 122
Figure 14 From the shaft's angle to the frequency shown, with the outage path 125
Figure 15 The stages of one measurement 131
Figure 16 The checks a measurement must pass before it reaches the storing rules 132
Figure 17 The storing rules 134
Figure 18 How a change on the main board reaches flash 158
Figure 19 How the main board keeps its copy of the audio board's settings 163
Figure 20 What happens to an uploaded settings file 169
Figure 21 First-time setup, from a blank board to the house network 189
Figure 22 How the S3 moves between joining, the house network, the rescue access point, a forced access pointand radio silence 191
Figure 23 How a phone that joins the rescue access point is led to the portal's sign-in page 193
Figure 24 Where the portal's parts run on the main board, and how a request reaches the network code 203
Figure 25 A page load is two requests; then one state poll every 5 seconds, backing off when polls fail 205
Figure 26 What each board owns, and what crosses the link 225

Figure 27 The audio board's two states, and what moves it between them 230
Figure 28 One frame on the link, byte by byte 233
Figure 29 The frame decoder: one state per field, back to the hunt after every frame 235
Figure 30 The handshake, then the heartbeat 237
Figure 31 Changing the protocol version: the only order that works over the air 254
Figure 32 The audio path: RADIO and BT go through the A32 and the DAC; AUX goes straight to the amplifier'sinput panel, which adds up its inputs 258
Figure 33 One pass of the audio task: the source handshake, the samples from the chosen source, the processingcommon to every source, and the write to the DAC 265
Figure 34 The Bluetooth states, what moves between them, and the lamp pattern of each 272

Tables

Table 1 How the chapters are laid out 1
Table 2 How the chapters are laid out 2
Table 3 The recovery ladder 3
Table 4 The console without a cable 4
Table 5 The needle says FAULT, or points to the wrong place 6
Table 6 An update went wrong 10
Table 7 The clock shows the wrong time 11
Table 8 Settings look lost, or SETTINGS LOCKED 12
Table 9 What the lights and pills mean 13
Table 10 What the machine is 14
Table 11 Two boards, and why two 16
Table 12 What runs where 19
Table 13 The source tree 23
Table 14 Words used everywhere 24
Table 15 Where to read what 25
Table 16 Updating over the air 28
Table 17 How each board names the state 33
Table 18 What earns confirmation 33
Table 19 Versions, and where to read them 37
Table 20 Versions, and where to read them 38
Table 21 Build and update constants 38
Table 22 When it goes wrong 39
Table 23 What you see 47
Table 24 Start-up, step by step 48
Table 25 Multiplexing 54

Table 26 Brightness and panel lamps 56
Table 27 Health reporting 59
Table 28 Health reporting 60
Table 29 Stored settings 62
Table 30 Console keys 63
Table 31 Constants 64
Table 32 Peripherals the S3 firmware takes 65
Table 33 When it goes wrong 65
Table 34 Changing it safely 72
Table 35 The chain, layers and tasks 75
Table 36 Units and the position frame 77
Table 37 Driving the motor 79
Table 38 How the needle moves 82
Table 39 Homing 85
Table 40 Homing 85
Table 41 Homing 86
Table 42 The re-index ladder 91
Table 43 The index band 92
Table 44 Memory across restarts 95
Table 45 Portal and console 99
Table 46 Portal and console 100
Table 47 Settings and constants 101
Table 48 Settings and constants 101
Table 49 Settings and constants 102
Table 50 The motor and its drive 104
Table 51 The travel, the index and the soft limits 107
Table 52 The tuner and the dial 108
Table 53 When it goes wrong 109
Table 54 Tried and rejected 114
Table 55 Units and tasks 124
Table 56 Units and tasks 124
Table 57 From shaft angle to megahertz 127
Table 58 The calibration curve 127
Table 59 The calibration curve 128
Table 60 The RDA5807M as an instrument 129
Table 61 The RDA5807M as an instrument 129
Table 62 One measurement 133
Table 63 One measurement 133
Table 64 One measurement 136
Table 65 Buttons and keys 137
Table 66 Settings and constants 140
Table 67 Settings and constants 140
Table 68 When it goes wrong 142
Table 69 Design choices 144
Table 70 Design choices 145

Table 71 What a setting is and where you change it 154
Table 72 Where settings are stored 155
Table 73 Where settings are stored 156
Table 74 The rules 158
Table 75 The buttons that force a save 159
Table 76 Why “Save settings now” says no 160
Table 77 Version history 165
Table 78 The format, and the download 168
Table 79 Uploading: the rules 170
Table 80 Worked example 172
Table 81 When it goes wrong 172
Table 82 The settings table 180
Table 83 Two cores 182
Table 84 How to test a change 186
Table 85 Reaching the radio 187
Table 86 The builder checklist 188
Table 87 The rescue access point 192
Table 88 Worked example: the router is replaced 195
Table 89 Transmit power 198
Table 90 Roles 200
Table 91 Tasks and cores 204
Table 92 The console, over USB and the web 210
Table 93 The console, over USB and the web 211
Table 94 When it goes wrong 212
Table 95 How to test a change 223
Table 96 Two computers and the wire between them 226
Table 97 Start-up, step by step 227
Table 98 Reading the audio board’s boot report 229
Table 99 Awake and asleep 231
Table 100 The link, and how to read its health 232
Table 101 The link, and how to read its health 232
Table 102 The frame 234
Table 103 Handshake and heartbeat 238
Table 104 Handshake and heartbeat 238
Table 105 Updating the audio board, and its trial 239
Table 106 When it goes wrong 242
Table 107 Where the code is 252
Table 108 Where the code is 252
Table 109 What you hear 257
Table 110 The configuration 260
Table 111 Tasks and cores 262
Table 112 Buffers 262
Table 113 The volume law 268
Table 114 Changing source 270
Table 115 The policy 271

Table 116 The states 273
Table 117 The blue lamp 274
Table 118 Memory: the audio board's tightest limit 275
Table 119 Memory: the audio board's tightest limit 276
Table 120 What the audio board reports 276
Table 121 Pops: what was found, and what is still open 278
Table 122 When it goes wrong 279
Table 123 Tried and rejected 282
Table 124 Where things are 289
Table 125 Build scripts 290
Table 126 Index by symptom 290
Table 127 Tuning 293
Table 128 Needle 294
Table 129 Settings, portal and console 295
Table 130 The link and the audio board 297
Table 131 Sound 301
Table 132 Main board, clock, display and build 302
Table 133 Texts that say something false 303
Table 134 Untested paths 305
Table 135 Code tidy list 306
Table 136 Code tidy list 308
Table 137 Settings reference 309
Table 138 A.1 Audio and Bluetooth (the audio board) 310
Table 139 A.1 Audio and Bluetooth (the audio board) 311
Table 140 A.1 Audio and Bluetooth (the audio board) 312
Table 141 A.2 Display and lights 312
Table 142 A.2 Display and lights 313
Table 143 A.3.1 The needle 313
Table 144 A.3.2 The tuning 315
Table 145 A.4 Network, accounts and stored state 318
Table 146 A.5 Constants of the settings system 318
Table 147 A.6 Experiments: actions, never saved 320
Table 148 B.1 Network constants 321
Table 149 B.2 Portal constants 323
Table 150 B.3 Page and console constants 324
Table 151 B.4 HTTP endpoints 324
Table 152 B.5 State fields 328
Table 153 C.1 Bluetooth 329
Table 154 C.2 Needle 329
Table 155 C.3 Tuning and the dial 330
Table 156 C.4 Display 330
Table 157 C.5 Audio and the audio board 331
Table 158 C.6 Audio experiments 331
Table 159 C.7 System and network 332
Table 160 D.1 Main board 333

Table 161 D.2 Audio board 335
Table 162 E.1 Before the tabs 336
Table 163 E.2 The tabs, in order 336
Table 164 E.3 Controls and their calls 337
Table 165 E.4 What has no control on the page 338
Table 166 F.1 The frame 338
Table 167 F.1 The frame 339
Table 168 F.2 Constants 339
Table 169 F.3 Messages 341
Table 170 F.4 Payloads 343
Table 171 F.5 What the link prints 344
Table 172 G.1 Compiled constants 345
Table 173 G.2 Experiments 347
Table 174 G.2 Experiments 347
Table 175 G.2 Experiments 348
Table 176 H.1 What a pop is 349
Table 177 H.2 What the pops were not 350
Table 178 H.2 What the pops were not 351
Table 179 H.3 Was it the data line? 352
Table 180 H.4 Was it the clocks? What cut the pops 353
Table 181 H.4 Was it the clocks? What cut the pops 354
Table 182 H.8 How to count pops 357
Table 183 I.1 Main board (S3) 358
Table 184 I.1 Main board (S3) 359
Table 185 I.2 Audio board (A32) 360
Table 186 I.2 Audio board (A32) 361

About this book

Ambersong is a LLOYDS TM-838N tube (valve) radio from the early 1960s, rebuilt into a new cabinet with two small computers inside it. This book is the Firmware Gospel: the account of the software those two computers run. It says what the software does, how it does it, and why each choice was made. With it, someone who has never seen the machine can understand it, recover it, build something like it, and change it safely.

What this book covers

The firmware of the two boards, and nothing else. What is physically in the machine — the boards, the parts, the wiring, the values — is the subject of the companion book, the Hardware Bible. This book points to it as "Hardware Bible, chapter 8", and states in place any hardware fact it needs.

One exception. The firmware holds the physical values of this particular mechanism, so this book carries them, in chapter 5 (p. 73). They are the needle's travel, its index switch, the motor's speed limits, the tuner's travel and the printed dial.

The code is what runs. A comment in the code states what was meant when it was written. Where a comment and the code disagree, the code is what the radio does.

The versions described. This book describes the main board's firmware at release v.1.0.5 and the audio board's at release v.1.0.4. The two boards are updated separately, so their numbers need not match. Chapter 3 shows where to read the version each board is running.

How the chapters are laid out

This page and chapters 1 (p. 3) and 2 are the way in. They are this page; a rescue chapter for when something is wrong; and the whole machine at a glance. Chapters 3 to 10 each cover one part of the firmware. Each opens with what you see or hear, readable on its own, then explains how it works and lists its settings. Each then ends with the same tail, in this order, where there is something to say:

Table 1. How the chapters are laid out

Section	What it holds
When it goes wrong	A table: what you see, what happened, what the firmware does, what you do.
Design choices	One line each: the choice, then why.
Tried and rejected	One line each: approaches tried on this machine and dropped.
Known limits	What the firmware does not do, and paths never exercised on the real radio.
Changing it safely	What must stay true, the traps, and how to test a change.

The grey bands. Some passages sit in a grey band headed For firmware changes. They hold what only someone changing the firmware needs. That means code maps, function and file names, internal workings, the Tried and rejected lists and every Changing it safely section. To repair or use the radio, you can skip every band.

Chapter 11 lists the build scripts that ship with the firmware. Chapter 12 opens with an index of known issues by symptom. It then gathers the issues that were left as they are, grouped by area. For each, it gives what you would see, why it was left, and a starting point for a fix. The appendices are reference tables.

Table 2. How the chapters are laid out

Ch.	Title
—	About this book
1	Rescue: when the radio misbehaves
2	The machine at a glance
3	Building, flashing and updating
4	The main board: start-up, clock, lamps and time
5	The dial needle
6	Tuning and the dial’s self-calibration
7	Settings
8	Network, web portal and console
9	The audio board and the link
10	Sound and Bluetooth
11	Build scripts
12	Known issues, ideas and untested paths
A	Settings reference
B	Network and portal constants, HTTP endpoints
C	Portal actions
D	Console keys, both boards
E	Portal map: tabs, cards and buttons
F	Link reference: messages and payloads
G	Audio constants and experiments
H	The pop hunt in detail
I	Pin use by the firmware

The recovery ladder

This book sorts every fault by what it costs to recover from it:

Table 3. The recovery ladder

Class	What it costs
NOTE	Nothing: it heals by itself, or it is only cosmetic.
DEFECT	A portal reboot, a console reset, or the front power switch.
BLOCKER	The radio is unusable until a full power cycle (pulling the plug), or the harm survives one: the needle driven into a mechanical stop; a calibration or the settings corrupted and saved; a wrong calibration sample stored; a board that will not start, or that only the USB cable can reach.

Chapter 1 gives the steps of the ladder in the order to try them, cheapest first.

Conventions

- The two boards. The S3 is the main board, an ESP32-S3: the clock display, the needle, the panel lamps, WiFi and the web portal. The A32 is the audio board, a classic ESP32: the sound, Bluetooth and the battery-backed clock. Chapter 2 explains why there are two.
- Exact words. Text in `code` is exactly what you type or read: a console key, a setting's name, a message the radio prints. Portal buttons, tabs and pills are in bold, spelt as on the page. Console keys are case-sensitive. The key tables write a pair in two ways. Two keys side by side, `s S`, do the same thing. Two keys with a slash, `n` / `N`, are two different commands, described in the same order: "Jog −20 / +20" means `n` jogs −20 and `N` jogs +20. Keys in separate rows, such as `y` and `Y`, are different commands too.
- Version stamps. Every build is stamped `v.1.YYYYMMDDTHHMMSS (hash)`: the release number, the time it was built, and the git commit it was built from. `-dirty` is added if the code had uncommitted changes. Chapter 3 explains the stamp.

Your mileage may vary

Every Tried and rejected list records what was tried here, what happened here, and why it was dropped here. It never says "this does not work". A retired approach was a dead end in this particular machine, not necessarily in yours. Likewise, every design choice in this book is a choice made for this radio, not a law. If you are building your own, question both.

1. Rescue: when the radio misbehaves

This chapter is for the moment something is wrong and you remember nothing about how the machine works. Find your symptom, and try the steps in order, cheapest first. Stop as soon as the radio behaves. Each section ends with the chapter that tells the full story. The last section is one table of every light and portal pill, with its meaning.

1.1 First things first

The machine has two small computers. The S3, the main board, runs the clock display, the dial needle, the panel lamps and the WiFi web page, called the portal. The A32, the audio board, runs the sound, Bluetooth and the battery-backed clock chip. Both are powered whenever the set is plugged in. The front switch only turns the amplifier on and off, so it restarts neither computer. A full power cycle therefore means pulling the plug.

The recovery ladder, cheapest first:

1. ☐ Wait. Most faults heal by themselves.
2. ☐ Press a portal button.
3. ☐ Reboot one board from the portal's System tab.
4. ☐ Switch the front switch off, then on. This also re-homes any needle with no reference point.
5. ☐ Unplug the set, then plug it back in.
6. ☐ Use the USB cable (section 1.12 (p. 12)) as a last resort.

> Note Park the needle before you unplug: switch the amplifier off and let the needle fall to the low end of the dial before you pull the plug. Power-up assumes the needle is already there (chapter 5 (p. 73)).

> Note Download the settings file before you change any calibration, with Download on the System tab's Settings file card. It is the only way to undo the change (chapter 7 (p. 153)).

1.2 The console without a cable

The S3 has a text console. It prints what it is doing, and every key you type is a command. You do not need a cable to reach it:

1. ☐ Sign in to the portal as the administrator.
2. ☐ Open the Console tab.
3. ☐ Send one key at a time. Each character is its own command, so the portal refuses longer input, except while a prompt awaits a whole line.

Lines from the audio board arrive in the same console, prefixed `[A32]`. Three keys are enough to start:

Table 4. The console without a cable

Key	What it prints
`s`	The status: amplifier, source, needle, index switch, angle sensor, tuning chain, portal counters. Also the link to the audio board (`link : alive` or `SILENT`, frames received, checksum errors, frames of the wrong protocol version). Then the last reset reason, the uptime, and the `image` line, which tells whether a new firmware is on trial or was rolled back (section 1.9 (p. 9)). It ends with the list of commands.
`i`	The network: joined, or `OWN ACCESS POINT` the network's name, the radio's address and signal; portal sessions; network time fresh or stale; access-point clients; and what the WiFi driver really holds.
`?`	The list of every console key.

Attaching to the console does not restart the S3, so you never see its start-up banner. Press `s` instead.

Every console key is in Appendix D (p. 332). The console itself is explained in chapter 8 (p. 186).

1.3 The portal will not open

1. ☐ Try both addresses: the radio's address on your house network, and `http://ambersong.local/`. A page that says "no answer from the radio." retries by itself after 2, 4 and 8 seconds, then every 15 seconds.
2. ☐ Wait for the needle. The portal is slow, or does not answer at all, while the needle moves.
3. ☐ Stop all traffic. About twenty to forty quick connections use up the radio's connection records. It then stops answering, although it still answers a ping. Close every browser tab open on the radio, wait one to two minutes, and try once.
4. ☐ Look for the rescue access point. When the S3 cannot reach the house network, it raises its own WiFi network, named **`Ambersong`**, and serves the same portal at **`http://192.168.4.1/`**. It comes up by itself:

   - at once, and for good, on a board with no house network stored;
   - 20 seconds after a failed attempt to join the house network;
   - 20 seconds after the house network connection drops.

   It asks for no password. Join it; on most phones the portal's sign-in page then opens by itself. If it does not, open `http://192.168.4.1/`. At most two devices can join at once. A blank board, with no house network stored, transmits at its lowest power, so stand close.

   > Note The rescue network is open: what you type on it, passwords included, crosses the air unencrypted until the radio goes back to the house network.

5. ☐ If signing in fails: a session ends after 5 minutes without use. After five wrong passwords, a device is locked out for 1 minute, then 2, then 4, and so on up to 64 minutes. A lost administrator password cannot be reset from the portal. The console key `~`, typed over the USB cable, erases all accounts and restores the shipped placeholder account.
6. ☐ Force the access point: the radio has no configuration buttons. From a portal that still answers, press Raise the rescue access point (System tab), or send the console key `Y`. The access point holds for 10 minutes, longer while someone is connected to it. Then the S3 goes back to the house network by itself.
7. ☐ If nothing works but the radio plays: nothing watches the portal or the WiFi software. A hang there leaves the radio, the clock and the needle working, and the portal dead. Unplug the set. If no access point appears afterwards and the house network is not joined either, you need the USB cable (section 1.12 (p. 12)).

Explained in chapter 8 (p. 186).

1.4 Getting back to the house network

- After a forced access point. Wait out the 10 minutes, or disconnect your phone from `Ambersong`. The S3 retries the house network every 2 minutes, but never while someone is connected to its access point. After WiFi off for 2 minutes (System tab), it comes back by itself.
- After a new router or a new password.
  1. ☐ Join the rescue access point and open the portal.
  2. ☐ On the System tab's Network card, type the WiFi name and password. A blank password keeps the stored one.
  3. ☐ Press Save network. The button stays disabled until the form has been filled from the radio.
  4. ☐ Put your phone back on the house network once the S3 joins it, about three seconds later.
- Without a browser. Over the USB cable, console key `y`, then the network name, a space and the password, on one line.

> **Note** The `y` prompt echoes the password into the console log.

Explained in chapter 8 (p. 186).

1.5 The needle says FAULT, or points to the wrong place

First, check that it is really wrong. Several things the needle does are normal. It rests at the low end of the dial when the amplifier is off, or when the source is not RADIO. It also rests there when the link to the audio board is down, because the S3 then takes the source as AUX. It stays pinned at one end when the knob is past the printed scale. It stops, with the clock dark, while the S3 is being updated. Two minutes after the amplifier goes off, it quietly re-checks its reference point. All of that is normal.

The needle's pills, at the top of the portal:

Table 5. The needle says FAULT, or points to the wrong place

Pill	What it means	What you do
NEEDLE OFF - re-index pending (try n/3)	The needle lost steps: its motor missed some, so its count no longer matches where it is. It finds its reference point again by itself once the knob has been still for 3 seconds.	Nothing.
NEEDLE RE-INDEXING	That repair is running.	Nothing.
NEEDLE FAULT	Three automatic repairs failed in a row, or a homing started by hand or at power-up failed. The motor is released and the needle stays where it is. The reason is printed on the console.	Clear the FAULT (below).
NEEDLE HUNTING	The needle keeps stepping back and forth without getting anywhere. A warning only.	STOP on the Needle tab, or console `x`.

Clearing a FAULT, cheapest first:

1. ☐ Press Home or Re-index on the Needle tab.
2. ☐ Send console `H`.
3. ☐ Switch the amplifier off, then on. The front switch starts homing whenever the needle has no reference point.

Homing can fail again with `index never found in either direction`, or with a message that begins `sensor never`. The console then suggests checking the index switch, its magnet, its pull-up resistor and the motor. That is hardware: see Hardware Bible, chapter 8.

Off by the same amount at every station, with no pill.

1. Press Re-index.
2. If it is still off, compare the frequency shown on the portal's Now tab with the station you hear.
3. The frequency is right: the needle's mapping onto the glass is off. Needle Limit ◂ 20 and 20 ▸ (Needle tab) slide where the needle points for every frequency.
4. The frequency is wrong: go to section 1.6 (p. 7).

Explained in chapter 5 (p. 73).

1.6 The frequency on the dial is wrong

The knob's angle becomes megahertz through a curve fitted to up to twelve samples. Three are hand marks that you type. Nine are RDA samples: a small receiver chip inside the cabinet (an RDA5807M) measures them by listening to the tube set's own local oscillator. A wrong sample, once stored, is saved, so download the settings file first (section 1.1 (p. 4)). All the buttons below are on the Needle tab.

1. Measure. Switch the amplifier on, select RADIO, hold the knob still, and press Measure this dial position. The result is stored only if it passes every check. A refusal says why, for example "Move the dial 0.3 MHz", "NOTHING FOUND … nudge it", or "the dial calibrates only while you listen to the radio …". When all nine RDA slots are full, a new sample replaces the most crowded one, or is not stored at all if it "would add the least coverage". Hand marks are never replaced. The radio also measures by itself, at most once every five minutes, while it plays the radio and the knob has been still for 8 seconds.
2. Mark a station, or slide the whole dial. Tune to a station you know, press Mark station A here (or B, or C), and type its frequency. Dial −0.1 MHz and Dial +0.1 MHz move every reading together. That hand correction is never erased by itself: Reset the dial correction to 0 is the only way back.
3. Remove one bad sample. Console `S` lists every sample with its residual, its distance from the curve. Residuals mean something from four samples on. Press Drop one sample and give its slot number.

4. The tuning line says “tuner ends NOT measured”. Turn the knob fully down and press Tuner = low end turn it fully up and press Tuner = high end.
5. Every new sample is off by the same amount since the tube set was realigned. Its intermediate frequency (IF) has changed. Set Tube set IF, then drop or re-measure the old samples.

To start over, Clear all marks returns to the stored straight line. It keeps the hand correction.

Explained in chapter 6 (p. 120).

1.7 No sound, the wrong source, or Bluetooth misbehaving

No sound, cheapest first. On the portal’s Now tab:

1. Is the amplifier on? The audio board plays nothing while it is off.
2. Is the source the one you expect? AUX never passes through the audio board, so the audio board is silent on AUX by design.
3. Mute and volume. A volume set from the portal holds until the front knob moves.
4. Is the **`link`** dot green? The audio board goes quiet within 2 seconds of losing the main board (section 1.8 (p. 9)).
5. Is an audio-board update running? The sound stops for about a minute while it writes.
6. Were experiment buttons pressed on the System tab? They are never saved, and Reboot the audio board undoes them.

Still silent. If the console shows `[A32] proto self-test FAILED - muted, BT closed, awaiting OTA` every 10 seconds, see section 1.9 (p. 9). Otherwise press Reboot the audio board: it clears a stuck audio task.

The wrong source. Only the audio board reads the front source selector, every 50 ms. The main board learns the source from it four times a second, and shows AUX while the link is down.

Bluetooth is not visible.

- A phone can connect only on the BT source, with the amplifier on and the audio board awake.
- A phone that has paired before must reconnect from the phone. The radio never calls a phone.
- A new phone needs the pairing window. Open it with the front pair button (on BT only), or with Pair on the Now tab. It lasts 90 seconds (setting `lookTimeoutS`), with the blue lamp blinking regularly. Opening it first drops any phone already connected.
- With `connectable` off (Bluetooth tab), Bluetooth is closed.

Bluetooth is visible when it should not be. The radio should appear in a phone’s list of new devices only during the pairing window; anything else is a fault. Reboot the audio board rewrites the visibility at start-up. Forget every pairing (Bluetooth tab) removes all stored phones.

Explained in chapter 10 (p. 256) the audio board's awake and asleep states in chapter 9 (p. 224).

1.8 The console says "A32 silent"

`[WARN] A32 silent. Clock and needle keep running.` means the main board has heard nothing valid from the audio board for 2 seconds. The clock keeps time and the needle parks. The main board calls the audio board every second, and prints `[PASS] A32 up: proto vN, firmware ...` when it answers. Most silences heal by themselves.

1. Read the link line of console `s: link : SILENT rx N crc N wrong-version N.`
   - A wrong-version count above 0, with the pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both, means the two programs speak different versions of the link protocol. Until they match, the main board cannot pass anything to the audio board, so the order matters. First update the main board, over its own WiFi, with a build whose protocol version matches the audio board's. Only then update the audio board, through the main board's portal, if it needs a new program too. If the audio board is still on trial after its own update, unplugging the set also works: it rolls back to its previous program.
   - A receive count (`rx`) that does not move points at the wiring or the power of the link (Hardware Bible, chapter 10 (p. 256)).
   - Receive and checksum (`crc`) counts climbing together point at the quality of the signal.
2. Right after an audio-board update: a new program that never completes the handshake with the main board stays on trial (section 1.9 (p. 9)). Unplug the set, and it rolls back to the previous program.
3. The pill AUDIO BOARD SENDS NO STATE, or `Flash BOTH MCUs` on the console: the two boards are talking, but the audio board's status reports do not fit this build. The sound is not affected. Flash both boards.
4. Reboot the audio board is sent over the link, so it cannot reach a silent audio board. A frozen audio board restarts itself within 15 seconds.

Explained in chapter 9 (p. 224).

1.9 An update went wrong

Every new program runs on trial. After an update over the air, a board confirms its new program only after about a minute of healthy running:

- the main board, 70 to 90 seconds after start-up, once its network and portal are up. It prints `image confirmed (a minute of running, network and portal up).`
- the audio board, one minute after its first handshake with the main board.

If the board resets for any reason before that — a crash, or a 15-second hang — it goes back to its previous program by itself.

Table 6. An update went wrong

What you see	What it means	What you do
The page reported a failure	The board's restart can beat its answer to the page.	Read the version shown as running on the System tab (main board), or `[PASS] A32 up: ...` on the console (audio board).
`this firmware is still on trial - retry in a minute` (HTTP 409), or `A32: still on trial - retry in 1 min`	That board is in its trial minute.	Wait for `image confirmed`, then upload again.
`- an earlier update was ROLLED BACK` at the end of the `image` line of `s`, or of the `[A32] boot:` line	The new program failed its trial. The board runs its previous one.	Read the reset reason, fix the build, upload again.
`[FAIL] proto self-test - this image will NOT be confirmed`	A broken main-board build is running.	If it came over the air: Reboot the main board, and it rolls back. If it was flashed by USB: upload a good build.
The new program misbehaves, still on trial	It is not confirmed yet.	Reboot that board from the portal: it rolls back. During the trial, Reboot the main board goes through at once, even over an unsaved change, and the previous program comes back.
`NOT saved yet - this firmware is on trial after an update ...`	The main board writes no settings during its trial.	Nothing. Changes apply at once and are written once the program is confirmed, about a minute later.
The new program misbehaves after `image confirmed`	It is permanent now.	Upload a good program through the portal.

An interrupted upload changes nothing. The main board gives up after 15 seconds without data, the audio board after 20 seconds, and the old program stays. The main board saves nothing during its trial, so a rollback finds the settings exactly as the previous program left them. A change made during the trial is lost with it. Other upload messages, such as "the A32 is not answering", "did not confirm" or a wrong chip, are in chapter 3 (p. 25)'s When it goes wrong table.

Explained in chapter 3 (p. 25).

1.10 The clock shows the wrong time

Table 7. The clock shows the wrong time

What you see	What it means	What you do
Blank digits	No valid time. Normal for a second or two after the main board starts.	If it lasts, the clock chip has no valid time and there is no network time. Join the house network, or set the time with console `W`: local time, `YYYY-MM-DD HH:MM:SS`.
`1017` or `0879`	A frequency (101.7 MHz, 87.9 MHz), not a time.	Choose what the digits show with `showTuning` on the Display tab.
`8888`, or dark while playing	A display test.	Normal on the Display tab, or console `9`.
Off by whole hours	The time zone. It is a POSIX time-zone rule on the Network card. The default is eastern North America, `EST5EDT,M3.2.0,M11.1.0`.	Set your own rule.
Off by minutes	The clock chip keeps the time; network time corrects it every hour while the `time` dot is green.	Without a network, set it with `W`.
12-hour or 24-hour, not as wanted	The `hour12` setting.	Change it on the Display tab.
Pill BATTERY CLOCK LOST ITS TIME - check its battery	The clock chip answered with no valid time. Most likely its backup battery or the module is failing.	Set the time, from the network or with `W`. The pill clears at the next good reading. Then check the battery.
Pill BATTERY CLOCK NOT ANSWERING	The audio board is up, but the clock chip did not answer.	Hardware: see Hardware Bible, chapter 4.

> Note The `W` prompt always uses the eastern rule: it converts what you type with the fixed rule `EST5EDT,M3.2.0,M11.1.0` and leaves that rule in force until your own rule is applied again, at the next restart, or when you save the Network card.

Explained in chapter 4 (p. 46).

1.11 Settings look lost, or SETTINGS LOCKED

Table 8. Settings look lost, or SETTINGS LOCKED

What you see	What it means	What you do
SETTINGS LOCKED - newer version stored	This program is older than the one that saved the settings. It runs on defaults, writes nothing, and refuses the download.	Flash the newer program again, or upload a complete settings file taken earlier.
Audio settings back to defaults after an audio-board update	The update changed their layout.	Upload a settings file exported before the update.
Audio rows show `n/a`, or edits are refused with "the audio board is not answering - its settings cannot be changed now"	The main board holds no current copy of the audio board's settings: the audio board has not answered since the main board started, or has gone silent.	Wait for the handshake, or press Re-read the A32 (Audio tab).
Needle limits back at ±300	The stored pair was corrupt and was cleared.	Home the needle, calibrate the index, and set the limits again (chapter 5 (p. 73)).
Mute off after a restart	By design.	Nothing.
A restore did not bring back the accounts or the WiFi network	They are not in the settings file. A restore never touches them.	Change them on the portal (section 1.4 (p. 6), chapter 8 (p. 186)).

Explained in chapter 7 (p. 153).

1.12 When you need the USB cable

Few faults need it:

- no rescue access point and no house network;
- a lost administrator password (console key ~);
- a confirmed audio-board program that hangs at every start, before the link comes up;
- too little free memory on the audio board for an update.

Where. Each board has its own USB-C socket on the back panel (Hardware Bible, chapter 4).

> ⚠ DANGER Read the Hardware Bible's safety chapter first (chapter 2, and the earth and grounds in chapter 5 (p. 73)). The USB ground is the machine's DC ground, which sits on mains earth. It is not the tube radio's chassis, which sits on mains neutral.

Before you write anything to a board, identify it by its MAC address, and never let PlatformIO (the build and upload tool, chapter 3 (p. 25)) choose the port by itself. The main board's console is its native USB port, at 115200 baud.

Explained in chapter 3 (p. 25) (identifying and flashing) and chapter 8 (p. 186) (the console).

1.13 What the lights and pills mean

The portal's pills are the small labels in its header. Green dots show what is well; red pills appear only when something needs attention.

Table 9. What the lights and pills mean

Where	What you see	What it means	See
Pill `link`	green dot	The main board and the audio board have shaken hands.	1.8
Pill `needle`	green dot	The needle has found its reference point.	1.5
Pill `time`	green dot	Network time was received within the last 4 hours.	1.10
Clock pill	`--:--`	No valid time.	1.10
Network pill	network name and signal in dBm	Joined to the house network.	1.4
Network pill	`access point`	Running the rescue access point.	1.3
Red pill	NEEDLE OFF - re-index pending (try n/3)	Steps lost; it repairs itself.	1.5
Red pill	NEEDLE RE-INDEXING	The repair is running.	1.5
Red pill	NEEDLE FAULT	Homing failed; the motor is released.	1.5
Red pill	NEEDLE HUNTING	The needle keeps stepping without getting anywhere.	1.5
Red pill	AUDIO BOARD SENDS NO STATE	The boards are talking, but the audio board's status reports do not fit this build. Flash both boards. If the audio board stops talking altogether, the `link` pill turns red instead.	1.8
Red pill	BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both	The two programs do not match. Update the main board first, to the audio board's protocol version; then, if it needs a new program, the audio board through it.	1.8
Red pill	SETTINGS LOCKED - newer version stored	Settings saved by a newer program.	1.11
Red pill	UPDATING	An upload is running.	1.9
Red pill	BATTERY CLOCK LOST ITS TIME - check its battery	The clock chip has no valid time.	1.10
Red pill	BATTERY CLOCK NOT ANSWERING	The clock chip does not answer.	1.10
Clock digits	blank	No valid time, or a main-board update is writing.	1.10
Clock digits	`8888`	Display test.	1.10
Panel lamps	dark	Amplifier off, or a main-board update is writing.	—
Panel lamps	full	RADIO, knob moving.	—
Panel lamps	half	RADIO, knob still for 5 seconds.	—
Panel lamps	low	AUX or BT (or the link is down).	—
Blue lamp	dark	Not on BT, or the audio board is asleep.	1.7
Blue lamp	fast breath for 20 seconds, then slow breath	BT source, waiting for a known phone to reconnect.	1.7
Blue lamp	regular blink	Pairing window open: visible to new phones.	1.7
Blue lamp	double flash, then steady bright	A phone has connected.	1.7
Blue lamp	steady dim	Hung up from the portal.	1.7

The panel lamps are explained in chapter 4 (p. 46), the blue lamp in chapter 10 (p. 256).

2. The machine at a glance

This chapter is the map. It names the parts, shows where each piece of firmware runs and how information moves between them. It also lists every source file, and tells you which chapter to open for what. Read it once before the chapters that follow. Everything here is explained in full later; each section says where.

2.1 What the machine is

Ambersong is a cabinet radio built around a tube radio, the LLOYDS TM-838N, which still does the receiving. Its original tuning knob still turns the tube set's own tuning capacitor. Around it, two small computers add what the old set never had:

- a four-digit amber clock behind the dial glass, called the Leditron
- a dial needle moved by a stepper motor instead of a cord;
- panel lamps that follow what the listener does;
- Bluetooth music from a phone;
- a battery-backed clock;
- a password-protected web page, the portal, that shows and changes everything.

The radio has no configuration buttons. Once the cabinet is closed, the portal is the only way in. What is physically fitted and wired is the subject of the Hardware Bible.

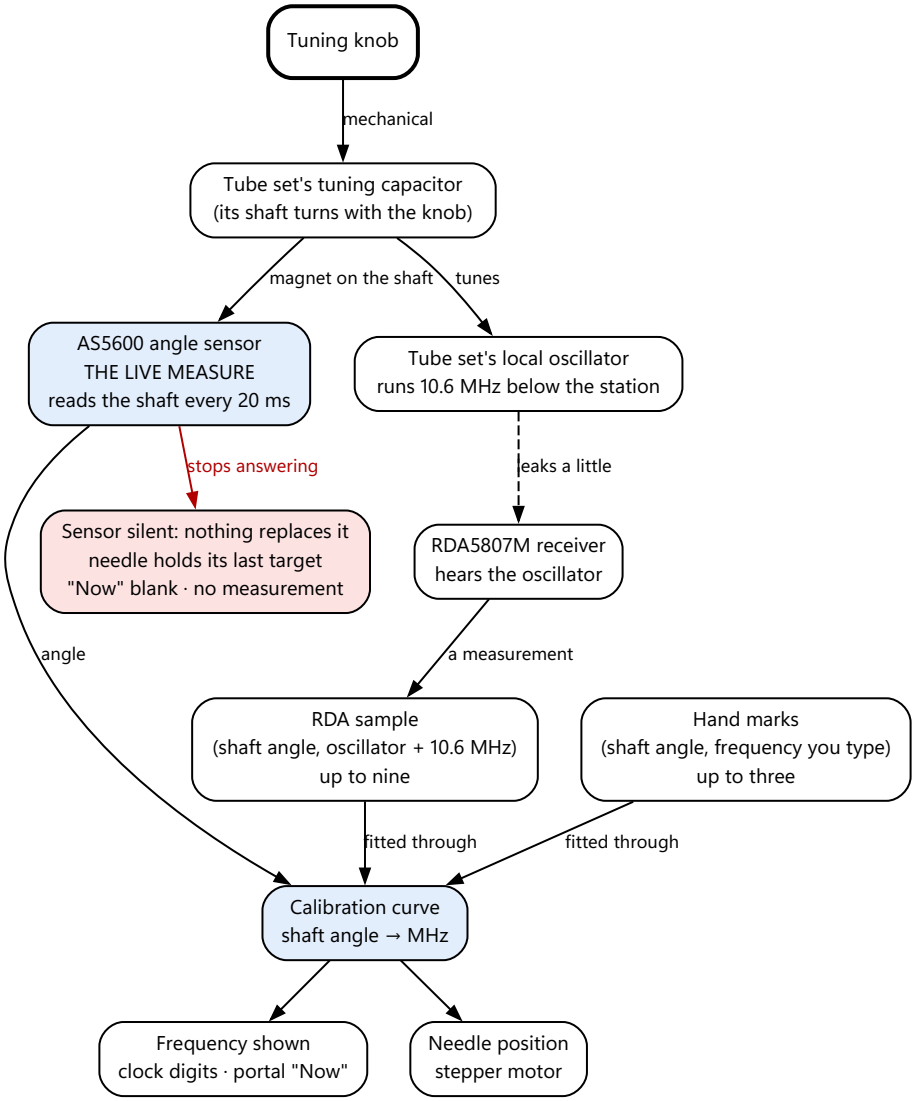
What a listener meets, from the front:

Table 10. What the machine is

Control	What it does	Chapter
Front switch	Turns the amplifier on and off. It does not start the computers: they run whenever the set is plugged in.	4
Source selector	Picks RADIO (the tube set, digitised and played back), BT (a phone) or AUX (an input that never passes through the audio board; the amplifier sums its inputs).	10
Tuning knob	Tunes the tube set. The needle follows it electrically (section 2.2 (p. 15)).	5, 6
Volume knob	Read as a position sensor; it sets the digital volume.	10
Pair button and blue lamp	Open, and show, the Bluetooth pairing window.	10

2.2 How the needle follows the knob

Figure 1. The tuning knob, the two sensors, and the needle: the firmware is the only link between knob and needle



Nothing mechanical joins the tuning knob to the needle. The knob turns the tube set's ganged tuning capacitor; a stepper motor moves the needle. The needle follows the firmware, and the firmware is the only link between the two. Two sensors on the main board play two different parts:

- The AS5600 is the live measure. This magnetic angle sensor reads the angle of the tuning capacitor's shaft every 20 ms. It tells the firmware that the tuning changed, and where the shaft is. Through the calibration curve, that angle gives every frequency the machine shows, on the clock digits and on the portal. It also gives every position the needle takes while it follows the tuning.
- The RDA5807M calibrates the curve. This small FM receiver chip listens to the tube set's local oscillator, the signal the set mixes with the station to

receive it. The oscillator runs below the station, by the set's intermediate frequency (IF) of 10.6 MHz. So the firmware adds 10.6 MHz to the oscillator's frequency to get the station, and stores the pair (shaft angle, station) as a calibration sample. The curve is fitted through these samples and the hand marks you type. A measurement changes the curve. It is never shown as the frequency, and it never moves the needle by itself.

- If the AS5600 stops answering, nothing takes its place. The shaft count freezes. The needle stops following the knob and holds its last target. The portal's Now frequency goes blank. A clock set to show the tuning all the time keeps the last value, and no measurement is started or stored. When the sensor answers again, the angle is placed back on the right turn and everything follows again.

Chapter 5 covers the needle. Chapter 6 covers the curve and how the RDA5807M measures.

2.3 Two boards, and why two

Table 11. Two boards, and why two

	The S3 (main board)	The A32 (audio board)
Chip	ESP32-S3	classic ESP32 (an ESP32-WROOM-32)
Looks after	the Leditron, the panel lamps, the needle and its motor, the tuning-angle sensor (AS5600), the RDA5807M used as a dial instrument, amplifier sensing, WiFi, the portal, network time, firmware updates for both boards	all the audio: the tube set's audio in (ADC), the audio out (DAC), Bluetooth, the source selector, the volume knob, the pair button and blue lamp, the DS3231 clock chip
Source folder	`src/s3/`	`src/a32/`
Keeps in its flash	settings for the display, lamps, needle and tuning; the WiFi network; the portal accounts	settings for the audio and Bluetooth; the volume knob's calibration

Why two. Music from a phone needs Bluetooth Classic (the A2DP profile), and the ESP32-S3 does not have it. So the audio runs on a classic ESP32, and everything else on the S3. The split also keeps two strict timings apart. On the A32, Bluetooth audio runs through the classic ESP32′s legacy I2S audio driver. On the S3, the needle's step emitter, the task that sends the motor its steps, runs at priority 19 and must not be starved. Section 2.4 explains tasks and priorities. Neither tolerates the other's timing.

What follows from the split. The A32 has no road to the outside except through the S3. It has no web portal of its own, and there is no reset wire from the S3 to it. Its firmware updates are relayed by the S3 over the link, the serial line between the two boards. The clock chip sits on the A32, so the time of day crosses the link too.

> FOR FIRMWARE CHANGES
>
> One project, two programs. A single PlatformIO project builds both. The pin map and the protocol between the boards are one set of files

> compiled into both programs, so the two cannot disagree about them by accident.

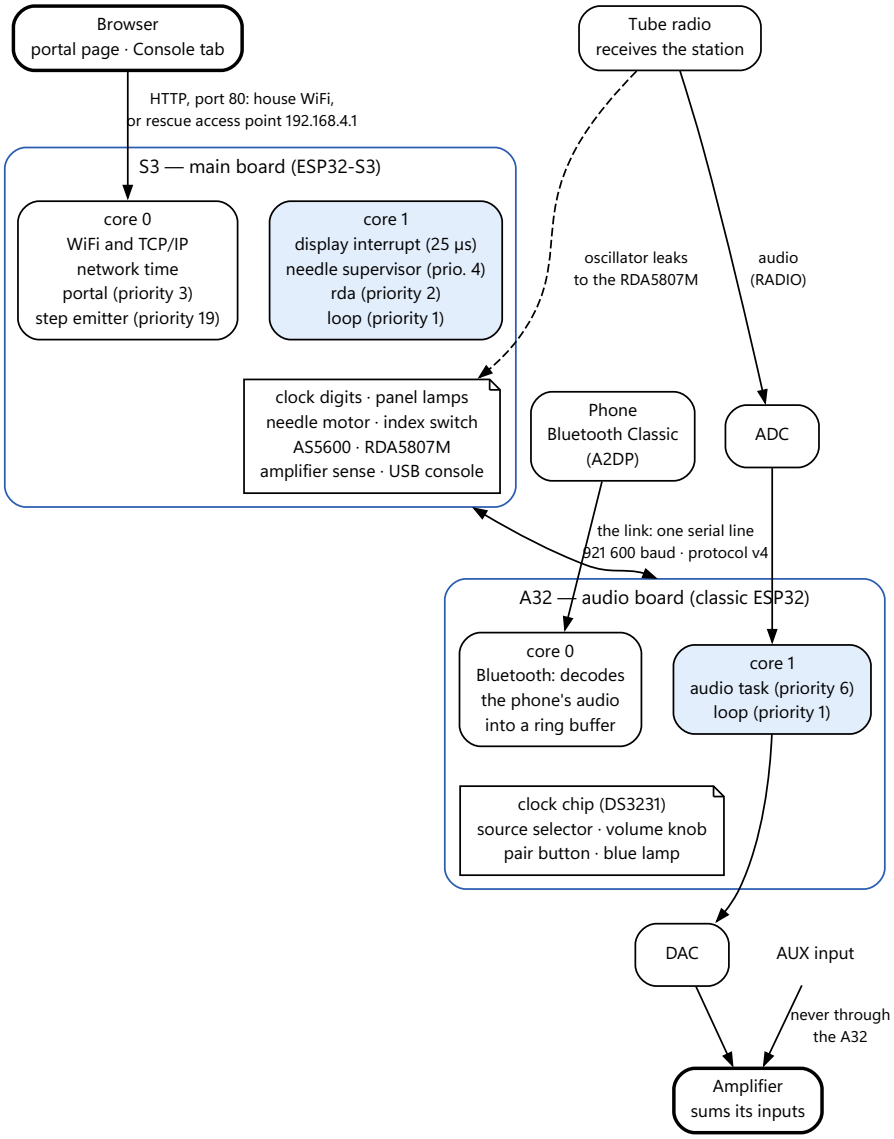
Each board looks after itself.

- Both run a 15-second task watchdog, which turns a hang into a restart.
- Both run a program freshly updated over the air on trial. They go back to the previous program by themselves if it resets before it has proved itself (chapter 3 (p. 25)).
- Each has its own rule for the other's silence. The A32 goes quiet — no sound, Bluetooth closed — within 2 seconds of hearing nothing from the S3. The S3 keeps its clock and needle running, and keeps calling.

Chapter 4 covers the S3′s start-up and watchdog; chapter 9 (p. 224) the A32 and the link.

2.4 What runs where

Figure 2. Both boards, their cores and tasks, the link between them, and the audio path



A task is one thread of the firmware, scheduled by the chip's real-time system. Each ESP32 has two cores, 0 and 1. A task with a higher priority runs first on its core. The task watchdog restarts the board when a watched task stops reporting in for 15 seconds.

> FOR FIRMWARE CHANGES
>
> The table lists every task on both boards: its core, its priority, how often it runs, and whether the watchdog watches it.

Table 12. What runs where

Board	Task	Core	Priority	Runs	What it does	Watched
S3	display interrupt	1	interrupt	every 25 µs	Multiplexes the four clock digits. Register writes only; it must never block.	—
S3	`loopTask` (the main loop)	1	1	pass after pass, with no pause between them	The link to the A32, amplifier sense, what the digits show, brightness, panel lamps, time, the network state machine, the console, the RDA and re-index triggers.	yes
S3	`needle` (supervisor)	1	4	every 5 ms	Plans the needle's moves. Reads the AS5600 angle every 20 ms, and its status every 500 ms.	yes
S3	`rda`	1	2	every 100 ms when idle; at least 300 ms per point during a sweep	Runs the RDA5807M measurements.	no
S3	`nstep` (step emitter)	0	19	1 kHz control tick	Emits the motor's steps. Spins between steps while the needle moves; sleeps when idle.	no
S3	`portal`	0	3	one pass, then sleeps 1 ms	The web server: accounts, every page request, firmware uploads, the web console, the portal's buttons.	no
S3	WiFi, TCP/IP, system timers	0 (WiFi)	system	—	The network stack. Network time arrives here.	no
A32	audio task	1	6	no pause: paced by the audio hardware, one block of 256 frames (about 5.8 ms) per pass	Reads or fetches one block, applies gain, fade and volume, writes it to the DAC.	no
A32	Bluetooth	0	its own	as the phone's audio arrives, in bursts	Decodes the phone's audio and puts the samples in a ring buffer.	no
A32	`loop` (the main loop)	1	1	pass after pass	The link to the S3, sleep and wake, the selector and volume knob every 50 ms, the Bluetooth states and blue lamp, the STATE message every 250 ms, receiving firmware updates, the USB console.	yes

Stacks: S3 main loop 8192 bytes, `needle` 4096, `nstep` 4096, `portal` 8192, `rda` 3072; A32 audio task 4096. On the A32 the watchdog also watches the idle task of core 0.

On the S3, the timing-critical work lives on core 1: the display interrupt, the main loop and the needle supervisor. WiFi, the portal and the needle's step emitter share core 0. The step emitter spins at priority 19 while the needle moves, so the portal is slow during a move. That is known and accepted: a needle stutter while the portal is in use is more

forgivable than a stalled link or a glitched clock. The rules a modifier must keep are in chapter 4 (p. 46).

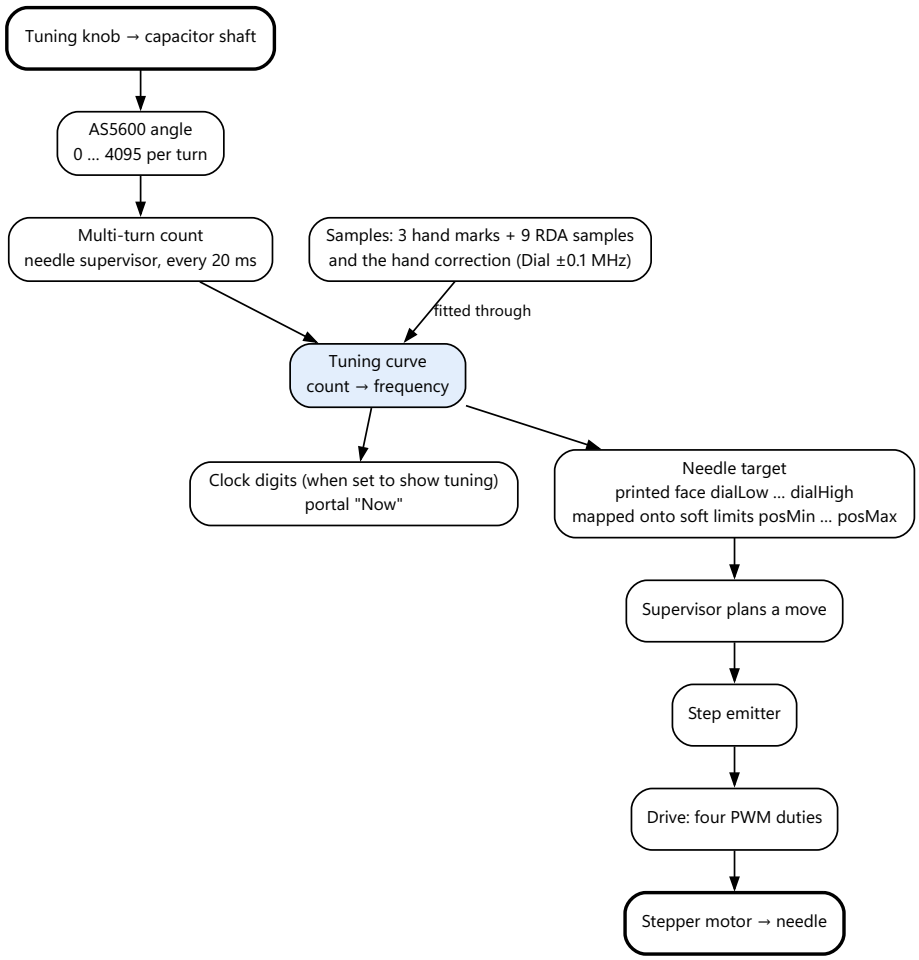
On the A32, Bluetooth owns core 0, and the audio task runs on core 1 at priority 6, above the main loop, with no delay of its own. The audio path is set up once at start-up and never reinstalled. The Bluetooth library hands the decoded samples to the firmware instead of driving the audio output itself. Chapter 10 explains why.

The one radio signal that matters to the firmware is the tube set's local oscillator, because the RDA5807M listens for it. The oscillator leaks a little from the tube set to the RDA5807M. The station is the oscillator's frequency plus the IF, 10.6 MHz; the firmware's IF setting defaults to 10.60 MHz (chapter 6 (p. 120)).

2.5 How information moves

Tuning: from the knob to the needle and to megahertz.

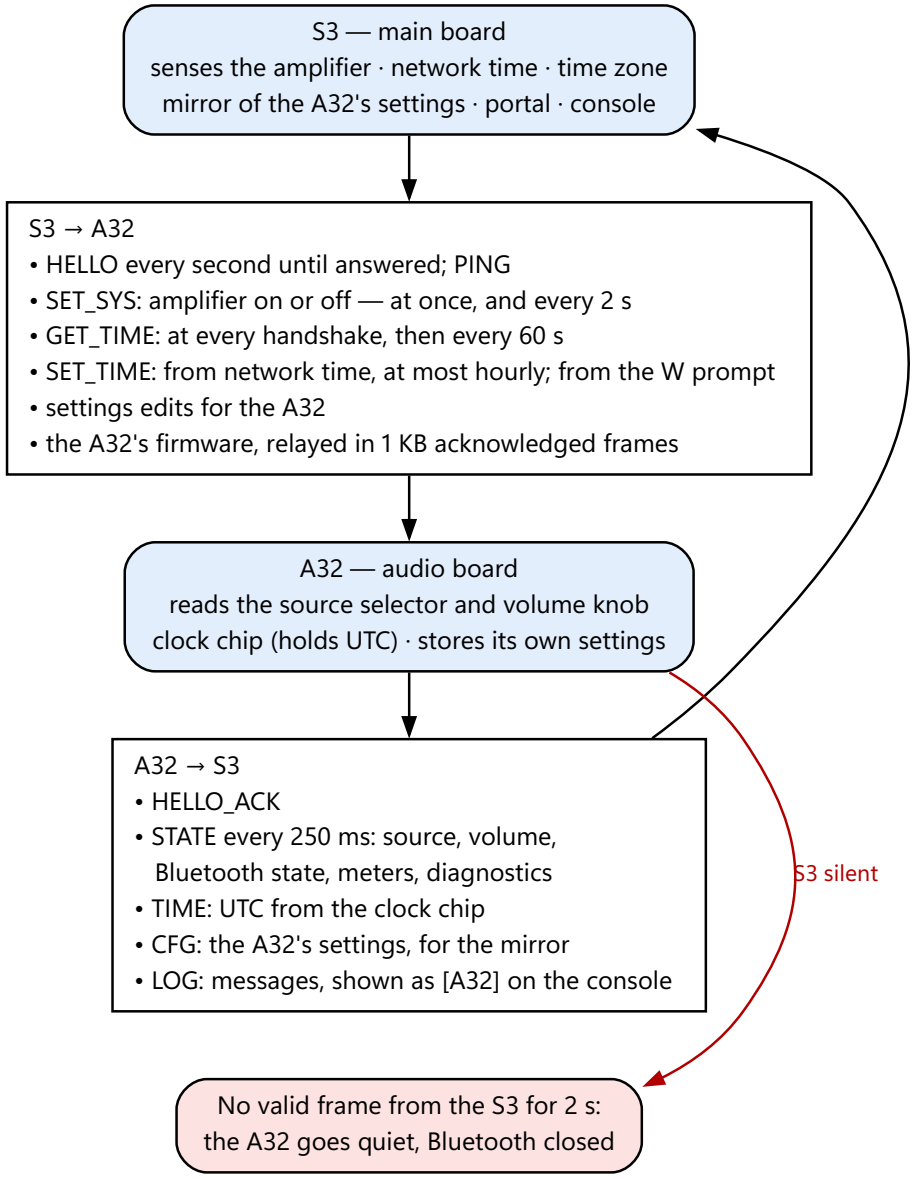
Figure 3. The tuning chain: from the knob's angle to the frequency shown and to the needle's motor



The needle has no end switches. One index switch, part-way along the travel, is its only reference. The needle homes on it once at power-up, and corrects its step count every time it crosses it. The tuning curve is fitted to up to twelve samples: three hand marks and nine RDA samples. The radio takes an RDA sample by itself at most every five minutes, while you listen to the radio and the knob is still. Once the nine are full, a new measurement replaces the most crowded automatic one, so the samples stay spread across the dial. Hand marks are never replaced. A hand correction (the portal's Dial ±0.1 MHz) slides the whole curve and is never erased automatically. Chapters 5 and 6.

Between the two boards.

Figure 4. What the two boards tell each other over the link



- The front switch. The S3 reads the amplifier's state, debounced over 50 ms. A change sets the clock's brightness (full or standby), the panel lamps,

and the needle's job. The needle follows the knob while the radio is live — amplifier on and source RADIO. Otherwise it falls to the low end of the dial and parks. The S3 tells the A32 at once, and again every 2 seconds. The A32 is awake only while the amplifier is on and the S3 is talking to it (chapters 4 (p. 46) and 9).

- The source. Only the A32 reads the selector, every 50 ms. On a change it fades the old source out (150 ms) and the new one in (1 second), and opens or closes Bluetooth. It reports the source to the S3 in its STATE message, its status report, four times a second. The S3 uses that report for the needle, the lamps and the tuning readout. While the link is down, the S3 takes the source as AUX (chapters 4 (p. 46) and 10).
- Time. The DS3231 clock chip on the A32 holds UTC. The S3 asks for it at every handshake and every 60 seconds. It keeps its own software clock between answers, so a slow link never makes the display stutter. When the S3 is on a network, network time corrects the S3 and, once an hour, the DS3231. The time zone is applied only for display, on the S3. If the DS3231 answers with no valid time, or does not answer, a portal pill says so (chapters 4 (p. 46) and 9).
- Settings. Each board stores its own in flash. The S3 keeps a read-only copy, a mirror, of the A32′s settings, so the portal can show them. It drops that copy when the A32 goes silent, and refuses edits to the A32′s settings until the next handshake. A change on the portal applies at once and is written about two seconds later, once the needle is still. A change to an A32 setting travels over the link, and the A32 saves it. Neither board writes its settings while a freshly updated program is on trial (chapter 3 (p. 25)). On both boards, only a write that reached flash counts as saved. The whole set can be downloaded and uploaded as one text file, which is the backup. The WiFi network and the portal accounts are stored apart, and are not in that file (chapter 7 (p. 153)).
- Updates. The browser uploads a program to the S3. For itself, the S3 writes it into its other program slot and restarts. Meanwhile the needle stops and the clock goes dark, while the sound carries on. For the A32, the S3 relays the file over the link in 1 KB acknowledged frames, and the sound stops while the A32 writes. Either board then runs the new program on trial until it confirms it, about a minute later. The two programs must share one protocol version. If they do not, a portal pill says so, and the main board must be updated first (chapter 3 (p. 25), section 3.13 (p. 37) chapter 9 (p. 224)).
- The console. Everything the S3 prints goes both to its USB port and into a 16 KB ring that the portal's Console tab reads. Keys typed there are fed back as if typed on USB. The A32′s messages reach the same console through the link, prefixed `[A32]` (chapter 8 (p. 186)).

2.6 The source tree

FOR FIRMWARE CHANGES

The firmware lives in the folder `firmware/`. Besides a README, a `.gitignore` and the Firmware Gospel itself, these are all its files. `src/s3/` builds into the S3′s program and `src/a32/` into the A32′s, while `include/` is compiled into both.

Table 13. The source tree

File	What it holds	Chapter
`platformio.ini`	The two build environments, `s3` and `a32` the pinned platform and the pinned Bluetooth library.	3
`include/pins.h`	Every pin number either program uses.	App. I
`include/proto.h`	The protocol between the boards: `PROTO_VERSION`, the message types, the payload structures, the frame encoder and decoder.	9, App. F
`include/link.h`	The link over the serial line, the 2-second silence rule, and the protocol self-test.	9
`src/s3/main.cpp`	The S3′s start-up and main loop: the settings and their storage, the tuning-curve fit, the handling of the A32′s messages, the A32 update relay, the portal's live state, the RDA and re-index triggers, the trial and the watchdog, the console keys and prompts.	4
`src/s3/needle.cpp`, `needle.h`	The needle's two tasks: homing, tracking, parking, sweeps, index sensing, the index calibration, the AS5600 reads, and the memory kept across a software restart.	5
`src/s3/drive.cpp`, `drive.h`	Turns a needle position into four PWM duties.	5
`src/s3/display.cpp`, `display.h`	The Leditron's multiplexing interrupt.	4
`src/s3/panel.cpp`, `panel.h`	The panel lamps' PWM and fades.	4
`src/s3/rda.cpp`, `rda.h`	The RDA5807M as a dial instrument, and its task.	6
`src/s3/net.cpp`, `net.h`	The WiFi state machine, the rescue access point and its captive-portal name server, the transmit-power ladder, network time, the `ambersong.local` name.	8
`src/s3/portal.cpp`, `portal.h`	The web server task: accounts, sessions, every endpoint, firmware uploads.	8
`src/s3/console.cpp`, `console.h`	The console's copy of every printed line, to USB and to the ring the portal reads.	8
`src/s3/settings_api.h`	The settings-table row type and the calls the portal uses, so the portal never touches the settings directly.	7
`src/s3/settings_table.h`	The table of every setting, the portal's actions, the settings-file export and import.	7, 8
`src/s3/page_gz.h`	The portal page, compressed. It is made from `data/portal.html` at every S3 build (rewritten only when the page changed) and kept in git. Never edit it by hand.	8
`src/a32/main.cpp`	The A32′s start-up and main loop: the link, sleep and wake, the selector and volume knob, the Bluetooth policy and states, its settings, the update receiver, the trial and the watchdog, the boot report.	9, 10

File	What it holds	Chapter
`src/a32/audio.cpp, audio.h`	The audio engine: I2S set up once, the audio task, gains, volume, fades, mute, the Bluetooth ring buffer.	10
`src/a32/btled.cpp, btled.h`	The blue lamp's patterns.	10
`src/a32/rtc.h`	Reading and writing the DS3231 clock chip.	9
`data/portal.html`	The portal page's source: one file, with its style and script inline.	8
`scripts/version.py`	Stamps each build with `FW_VERSION` and `FW_COMMIT`.	3
`scripts/page.py`	Checks the page's script, compresses the page, and writes `page_gz.h`.	8

2.7 Words used everywhere

Table 14. Words used everywhere

Word	Meaning
S3, A32	The main board (ESP32-S3) and the audio board (classic ESP32).
the link	The serial line between the two boards (chapter 9 (p. 224)).
portal	The S3′s web page. Its Console tab is a copy of the S3′s console (chapter 8 (p. 186)).
pill	A small status label in the portal's header (chapter 1 (p. 3), section 1.13 (p. 12)).
console	The S3′s text console: every key is one command (Appendix D (p. 332)).
rescue access point	The S3′s own WiFi network, `Ambersong`, raised when it cannot reach the house network. It is open, with no password, and most phones open its sign-in page by themselves (chapter 8 (p. 186)).
amp on, radio live	The amplifier is powered; the amplifier is on and the source is RADIO (chapter 4 (p. 46)).
homing, index, re-index	Finding the needle's reference switch; that switch; homing again to repair lost steps (chapter 5 (p. 73)).
hs	A half-step of the needle's motor, the unit of needle position (chapter 5 (p. 73)).
soft limits	`posMin` and `posMax`, the ends of the needle's working travel, set by hand (chapter 5 (p. 73)).
local oscillator, IF	The tube set's own oscillator, which runs below the station; the intermediate frequency, 10.6 MHz, the gap between them (chapter 6 (p. 120)).
hand mark, RDA sample	A typed (angle, frequency) pair; a measured one (chapter 6 (p. 120)).
asleep, awake	The A32′s two states: silent and closed to phones, or playing (chapter 9 (p. 224)).
on trial, confirmed, rolled back	A new program not yet proved; proved and permanent; thrown away by the bootloader for the previous one (chapter 3 (p. 25)).
NVS	The ESP32′s key-value store in flash, where settings live (chapter 7 (p. 153)).

2.8 Where to read what

Table 15. Where to read what

If you want to…	Read
Fix a misbehaving radio tonight	Chapter 1
Build both programs, flash them, update them over the air (the A32 through the S3′s relay), read the versions; trial and rollback	Chapter 3
Understand the S3′s start-up, its tasks, the clock display, the panel lamps, time keeping and the watchdog	Chapter 4
Change how the needle moves, homes or recovers	Chapter 5
Find the physical values of this machine that the firmware holds: needle travel, index, motor speed limits, tuner travel, the printed dial	Chapter 5
Understand how the knob's angle becomes a frequency, and how the dial calibrates itself	Chapter 6
Add or change a setting, or understand the settings file	Chapter 7, Appendix A (p. 309)
Work on WiFi, the portal, its security, or the console	Chapter 8, Appendices B to E
Understand the A32′s start-up and the link protocol	Chapter 9, Appendix F (p. 338)
Work on the sound, the sources or Bluetooth, or read the story of the pops	Chapter 10, Appendices G and H
Know which build scripts ship with the firmware	Chapter 11
Find the known issues left as they are, and starting points for new work	Chapter 12
Know what is soldered, wired or fitted	The Hardware Bible

3. Building, flashing and updating

The firmware is one PlatformIO project that builds two programs: one for the S3, the main board, and one for the A32, the audio board. This chapter tells you how to build both on a clean PC. It shows how to put them on the boards the first time over a USB cable, and how to update them afterwards without a cable, through the portal. It explains the version numbers each program carries, so that you can always tell which program runs on which board and which source it was built from. And it is the one place in this book that explains how a freshly updated program is put on trial, then confirmed or rolled back every other chapter points here.

3.1 Building and flashing

The firmware lives in the folder `firmware/` and is one PlatformIO project with two build environments, one per board: `s3` for the main board, `a32` for the audio board. The two share one pin map and one link protocol, compiled into both programs, so the two boards cannot disagree about them by accident. Build both environments and judge a build by its exit code, never by what was printed on the way. A blank board needs one flash over a USB cable; after that, both boards take their updates over the air through the portal (section 3.10 (p. 28)).

⚠ WARNING A workbench with more than one ESP32 board makes it easy to write to the wrong one: boards of the same kind share the same USB vendor and product numbers, and PlatformIO's automatic port detection will happily pick the wrong port. Read a board's MAC address before you flash it (`pio device list` shows it directly for a board on the chip's own USB port; for a board behind a USB-to-serial bridge, ask the chip with `esptool.py --port COMx read_mac`). Keep a written list of your boards' MAC addresses and compare the target against it before every write. Pass `--upload-port` every time; never let PlatformIO choose the port. The same rule applies to an update over the air: before it, check the board's MAC address in the PC's ARP table (`arp -a`) against the network address the browser is about to use.

⚠ DANGER The tube radio inside this machine is tied to the mains. Read the Hardware Bible's safety chapter (chapter 2 (p. 14), and the earth and grounds in chapter 5 (p. 73)) before you connect any cable between the machine and a mains-powered PC.

Note Never flash a board while someone is listening to it or looking at it. A new program pushed into a machine under observation spoils the observation, and the person may not know why the behaviour changed.

```
pio run -e s3                                          # build
the main board
pio run -e a32                                          # build
the audio board
pio device list                                         # find
your S3 by SER=<its MAC>
pio run -e s3  -t upload --upload-port COMx             # the
port you found, never automatic
esptool.py --port COMy read_mac                         #
confirm the A32 is the board you mean
pio run -e a32 -t upload --upload-port COMy
pio device monitor -p COMx -b 115200                    # the
S3's console
```

3.9 The first boot

What the S3 does at start-up, step by step, is in chapter 4 (p. 46), section 4.2 (p. 47). Here is what a builder meets, in order, on a factory-fresh pair of boards:

1. No stored settings. The compiled defaults apply (Appendix A (p. 309) lists them).
2. No stored WiFi network. The S3 raises its rescue access point at once: network name `Ambersong`, on channel 1, an open network with no password. Join it; on most phones the portal's sign-in page opens by itself. Otherwise open `http://192.168.4.1/`, or `http://ambersong.`

`local/`. Set your house network from the portal's System tab, Network card, or with the console key `y`. If a stored network cannot be joined within 20 s, the rescue access point comes up and the house network is retried every two minutes. The transmit power starts at its lowest rung, 2 dBm, and climbs on failed joins; the ladder and its 15 dBm ceiling are in chapter 8 (p. 186), section 8.4 (p. 197).

3. The placeholder portal account. The firmware ships with a public placeholder account. While the stored account still matches it, the portal refuses everything except changing the password. Only `/api/boot`, `/api/state`, `/api/passwd` and `/api/logout` are let through. Every other request gets HTTP 403 `mustchg`, and the page shows only the change-password form. Both update endpoints refuse too. Change the password first. The console key `~` erases all accounts and restores the placeholder. An administrator can type it in the portal's Console tab; if nobody can sign in any more, the USB cable is the only way to type it. That is by design: there is no front-panel reset for a forgotten password (chapter 8 (p. 186)).
4. The clock. The battery clock on the A32 keeps time, and network time corrects it once the S3 is online. The console key `w` sets it by hand. That prompt reads your input in a fixed eastern North American time zone, whatever the portal's time-zone setting (chapter 4 (p. 46), section 4.9 (p. 57)). A new clock module may come up with no valid time. The portal then shows BATTERY CLOCK LOST ITS TIME - check its battery until the clock reports a valid time again. It is asked every minute.
5. Needle calibration. The needle has no end switches. Calibrate in this order:

    1. ☐ Run the band calibration (`k`) with the needle on its index sensor. It saves its result by itself.
    2. ☐ Set the soft limits (`m`, `M`).
    3. ☐ Set the tuner ends (`c`, `C`). The firmware also uses them at every boot to pick the turn of the tuning shaft.
    4. ☐ Set the dial marks.

    Chapter 5 explains each step.
6. The dial's self-calibration. With the set plugged in but its front switch off, sweep and mark the fixed spurs (`q`, `o`). With the front switch off, the amplifier, and with it the tube set, is off, so its oscillator is not running, and every peak the sweep finds is a fixed spur. Then switch the set on, select RADIO, and measure three positions spread across the dial (`v`, or Measure this dial position on the portal). From then on the radio keeps its own calibration up to date. Chapter 6 explains the method.

The A32 at first boot arms its 15-second watchdog as the very first step. It then runs its own link self-test: a failure mutes it and keeps it off Bluetooth, but the board stays reachable for an update. It loads its settings, reads the clock chip, and waits for the S3's HELLO message (chapter 9 (p. 224)).

A program written by USB is never on trial, on either board. The S3's image line then reads `not tracked (USB flash)`, and both boards save their settings normally from their first boot.

3.10 Updating over the air

The portal's System tab has two file pickers, one per board. Choose the matching `firmware.bin` and upload it. Behind them are two HTTP endpoints, both taking a multipart file upload of `firmware.bin`:

Table 16. Updating over the air

Endpoint	Target
`POST /api/ota/s3`	the S3 itself
`POST /api/ota/a32`	the A32, relayed by the S3

To script an update, sign in with `POST /api/login` (form fields `u` and `p` the answer sets a session cookie), then post the file to the endpoint with that cookie. Keep the credentials in environment variables, never in the script.

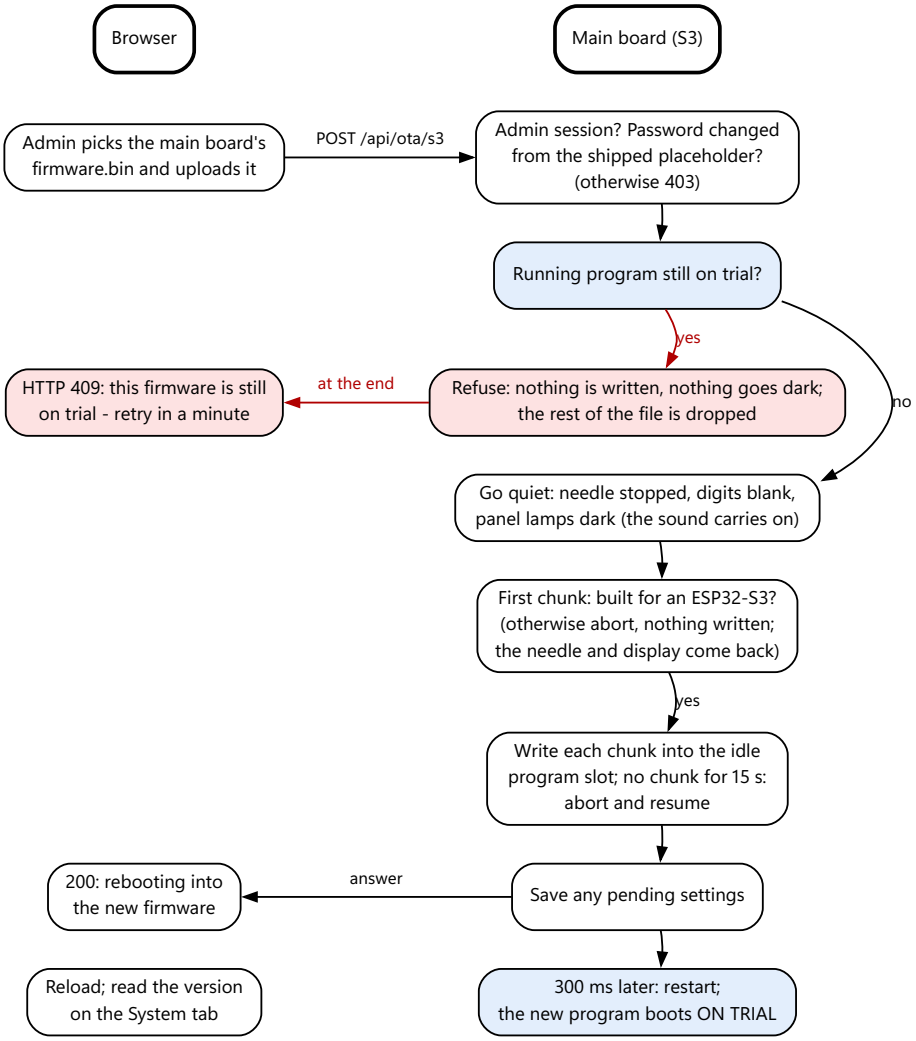
Do not trust the HTTP answer of an update that restarts the board. Several transfers reported a failure and had in fact landed. Confirm by reading the version after the restart (section 3.13 (p. 37)).

3.10.1 What both endpoints check first

- Administrators only. Another account gets `403 admin only`.
- Not with the placeholder password. The shipped account gets `403 change the default password first`.
- The right chip. Bytes 12 and 13 of an ESP32 program image hold the chip number: `0x0009` for an ESP32-S3, `0x0000` for a classic ESP32. On the first chunk, each endpoint refuses an image built for the other chip, before anything is written. The two pickers sit side by side and the two files have the same name, so this mistake is easy to make.
- The upload watchdog. If no chunk arrives from the browser for 15 s, the S3 aborts the update and goes back to work. The web server does not reliably report a connection that died mid-upload, and a dead transfer once left the machine muted for four minutes.

3.10.2 Updating the S3

Figure 5. Updating the main board over the air



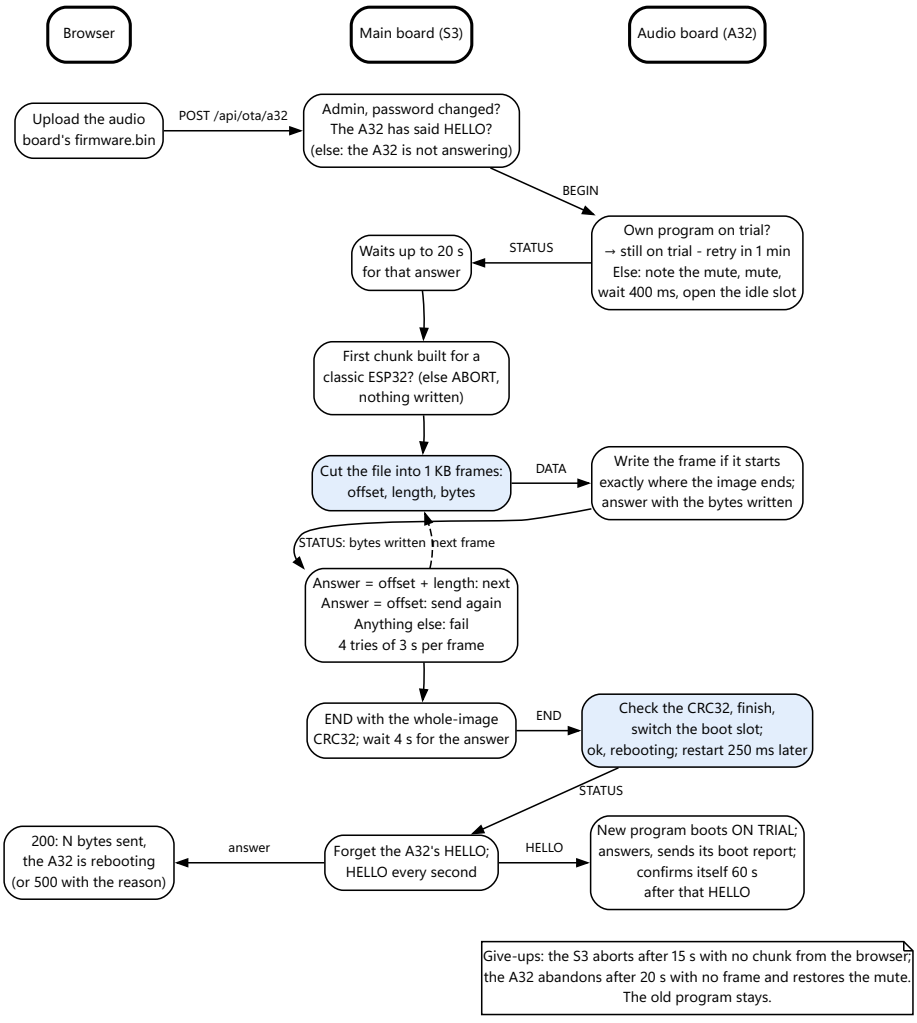
1. Is the running program still on trial? If it is, the update is refused (section 3.11.4 (p. 34)). Nothing is stopped or darkened, and the bytes that keep arriving are dropped. The console prints `portal: S3 update refused - this image is still on trial.` At the end the page gets HTTP 409 with `{"e":"this firmware is still on trial - retry in a minute"}`. The browser still sends the whole file before that answer comes back.
2. Otherwise the S3 streams the file into its idle program slot. While it writes, it stops the needle where it is, blanks the clock digits and darkens the panel lamps. Writing flash suspends the processor's flash cache, which would make those animations stutter; fifteen dark seconds read better than fifteen ugly ones. The sound keeps playing, because it lives on the A32.
3. On success the S3 clears its "update in progress" flag and saves any pending settings (a failed save is reported as a warning). It answers `200`

`rebooting into the new firmware`, waits 300 ms and restarts. The new program boots on trial (section 3.11 (p. 32)). The needle remembers its position across the restart and homes from there (chapter 5 (p. 73)).

4. If the upload fails and the S3 does not restart, the needle and the display come back as they were (chapter 4 (p. 46), section 4.8 (p. 56)). That happens on a wrong chip, an aborted or failed write, or 15 s without data.

3.10.3 Updating the A32 through the S3

Figure 6. Updating the audio board: the main board relays the file over the link, one acknowledged kilobyte ata time



The A32 cannot be reached by WiFi. The file goes to the S3 over WiFi. The S3 relays it to the A32 over the link, the serial line between the boards (921 600 baud, framed and checksummed; chapter 9 (p. 224)). The S3 stores nothing: it is a pipe.

1. Begin. The S3 refuses with `the A32 is not answering` if the A32 has not said HELLO. The A32 refuses with `still on trial - retry in 1 min` if its own running program is still on trial; the portal shows `A32: still on trial - retry in 1 min (err 0)`. Otherwise the A32 remembers your

mute setting and mutes. It waits 400 ms for the fade and for its audio buffers to drain (chapter 9 (p. 224) explains the figure), then opens its idle slot. Opening erases nothing; the slot is erased a sector at a time as the data is written. The S3 waits up to 20 s for this answer. The S3 does not stop its needle or darken its display, because it is not writing its own flash; the sound stops because the A32 is.

2. The chip check. It runs on the first chunk, after the begin, so the A32 has already muted. A file built for the S3 is aborted with `that image is not for the audio board (wrong chip) - nothing was written`, and the abort restores the A32′s mute. A file that is not an ESP32 image at all (its first byte is not 0xE9) passes this check and is refused by the A32′s updater instead (error 8, magic byte).
3. Data. The S3 cuts the upload into 1 KB frames, each carrying its offset and its length. The A32 writes a frame only if it starts exactly where the image so far ends. It acknowledges every frame with the number of bytes it has actually written. The S3 moves on when the acknowledgement equals offset plus length. It sends the frame again when the acknowledgement equals the offset (a duplicate or a late acknowledgement lands here harmlessly). It gives up on anything else (`A32 is at X, we are at Y`). Each frame gets 4 tries of 3 s; four silences give `no answer at N bytes`.
4. End. The S3 sends the CRC32 checksum of the whole image. Neither the size nor the checksum is known at the start, because the upload is streamed. The A32 checks it (`crc mismatch` if it differs), finishes the update, switches its boot slot, answers `ok, rebooting` and restarts 250 ms later. The S3 waits 4 s for that answer. If it does not come, the portal says `the A32 did not confirm the update - check its version once it reconnects`. The A32 commits before it answers, so a lost answer can hide a good update. On success the page shows `<N> bytes sent, the A32 is rebooting` on a failure, HTTP 500 with the reason.
5. After the end, the S3 forgets the A32′s HELLO, so the next handshake shows the new version. The new program boots on trial (section 3.11 (p. 32)).
6. If frames stop arriving for 20 s, the A32 gives up by itself (`the S3 stopped sending`) and restores your mute setting, not “unmuted”. A begin that arrives while an update is already running is refused (`begin refused`), and the running update is given up the same way.

A failed relay keeps the old program. The reason reaches the page, including the A32 updater’s error number. The numbers are: 1 write, 2 erase, 3 read, 4 space, 5 size, 6 stream, 7 MD5, 8 magic byte, 9 activate, 10 no partition, 11 bad argument, 12 abort.

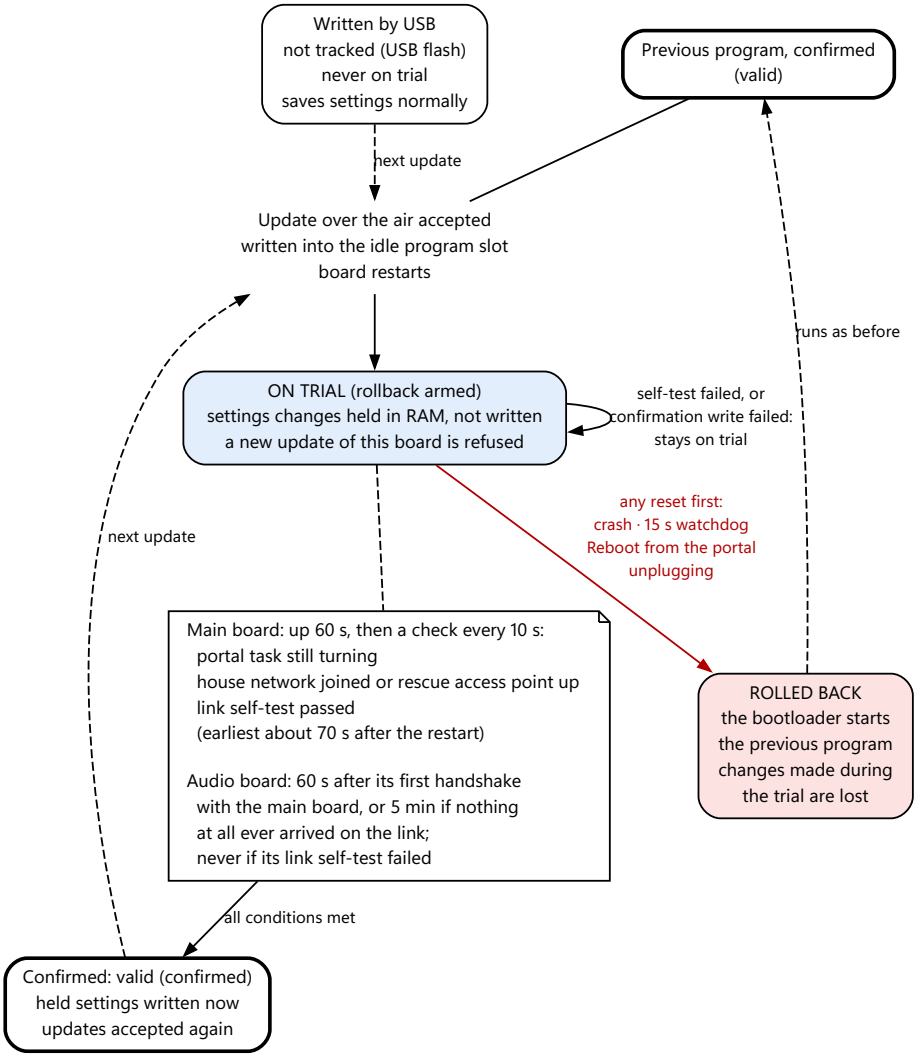
The relay is slow on purpose: about 16 KB/s; a full A32 program of 1.2 MB took 76 s. Each frame waits for its acknowledgement, and each acknowledgement waits for a flash write. A transfer that cannot tell whether it arrived is not worth having faster.

3.11 Trial, confirmation and rollback

Rollback means this: a freshly updated program runs on trial, and unless it confirms itself, the bootloader goes back to the previous program at the next reset. Both boards work this way, and this section is the one place in the book that tells the whole story.

The bootloader marks every new program valid before the program's own start-up runs, by default. A build that crashed or hung at boot would then loop on itself until someone used the USB cable. Both programs instead tell the core "this firmware confirms itself". A program written over the air therefore boots on trial. If the board resets for any reason before the program is confirmed, the bootloader marks it invalid and starts the previous one, which is still in the other slot.

Figure 7. The life of a new program on either board: on trial, then confirmed or rolled back



3.11.1 How each board names the state

The S3 prints an image line at the end of its start-up and in the console's `s` status: `image : <state>, watchdog on|OFF`. The A32 puts the same state in its boot report (section 3.11.7).

Table 17. How each board names the state

State	Meaning
`ON TRIAL (rollback armed)`	A new program sent over the air, not yet confirmed.
`valid`	A confirmed program.
`valid (confirmed)`	Shown after the program confirmed itself during this run.
`NEW - this bootloader has no rollback`	The bootloader did not arm the trial.
`not tracked (USB flash)`	A program written by USB. It is never on trial.

Whenever the bootloader has marked an earlier program invalid, the line ends with `- an earlier update was ROLLED BACK`.

3.11.2 What earns confirmation

Table 18. What earns confirmation

	S3 (main board)	A32 (audio board)
When it checks	Not before 60 s of running, then once every 10 s. The first check only takes a reading, so the earliest confirmation comes about 70 s after start-up; expect 70 to 90 s.	60 s after its first handshake with the S3.
What it needs	The portal task still turning (its pass count has moved since the previous check); a network up (the house network joined, or the rescue access point up); a passed link self-test.	A handshake with the S3 and a passed link self-test. If after 5 minutes nothing at all has ever arrived on the link — no valid frame and no damaged one — it confirms anyway.
What it never waits for	The A32 and the needle.	—
What it prints	`image confirmed (a minute of running, network and portal up).`	`[A32] image confirmed (a minute of running with the S3)`, or `(five minutes running, the S3 never spoke)`
If the confirmation itself fails	`image confirm FAILED (err N) - retrying`, again 10 s later. Until it succeeds, a reset still rolls back and updates are still refused.	The same message and the same retry.

Why the S3 asks only for a way in. Confirmation asks for what the next update needs: a portal and a network. A broken or absent audio board must not roll back a good main board, and a homing fault is about the mechanism, not the build.

Why the A32 waits a minute after the handshake. The handshake arrives about a second after start-up, before the A32 has woken, opened Bluetooth or played anything. Confirming there would accept a build that crashes a moment later.

Why the five-minute fallback. It keeps a good A32 program from being thrown away just because the S3 was absent. It applies only when nothing at all arrived. A program whose link is broken in a way that still produces bytes stays on trial: damaged frames, or valid frames that never include a HELLO. The next reset then takes the old program back.

3.11.3 What ends a trial in a rollback

Any reset before confirmation:

- a crash;
- a hang caught by the 15-second task watchdog (chapter 4 (p. 46), section 4.11 (p. 60) for the S3; chapter 9 (p. 224) for the A32), or by the S3′s interrupt watchdog;
- Reboot the main board or Reboot the audio board on the portal (section 3.11.6 (p. 35));
- a power cut, or unplugging the set.

The rolled-back board runs its previous program exactly as before. A crash, a watchdog reset and a reboot count as software restarts. After a rollback on the S3 caused by one of them, the needle homes from the position it remembered. After a power cut it is assumed at the low stop (chapter 4 (p. 46), section 4.10 (p. 59) chapter 5 (p. 73)).

3.11.4 No second update during a trial

While a board's program is on trial and not yet confirmed, a new update of that board is refused. The new program would be written into the other slot, which holds the very program a rollback returns to. A second bad build would leave nothing to fall back on.

- The S3 decides on the first chunk. It darkens nothing, and the needle keeps running. The answer is HTTP 409 `this firmware is still on trial - retry in a minute`, but only after the browser has sent the whole file.
- The A32 refuses at the begin, before anything is muted: `still on trial - retry in 1 min`.
- An A32 update is not affected by the S3′s trial.

Wait for the `image confirmed` line, then upload again.

3.11.5 No settings are saved during a trial

On the S3, settings changed during the trial are applied at once, kept in RAM and marked unsaved. The ordinary two-second save writes them on the first pass after the program is confirmed. The portal's Save answers `NOT saved yet - this firmware is on trial after an update; settings are written once it is confirmed (about a minute)`. The console prints `save held - this firmware is on trial; settings are written once it is confirmed`. The WiFi network and the portal accounts live in their own namespaces and are not held: they are saved at once, trial or not.

On the A32, a change sent from the portal is applied at once and kept in RAM. The moment the program is confirmed, the A32 writes it and sends `settings changed during the trial are now saved`. The S3′s console prints that as `[A32] ...` next to the confirmation line (it can come one line before it). If that write fails, the A32′s ordinary save retry takes over and says so (chapter 7 (p. 153)).

Why. A new program may change the layout of the stored settings. On the S3 it migrates the stored settings when it starts. Had it written them and then been rolled back, the previous program would find a layout newer than it knows, and lock its settings (SETTINGS LOCKED, chapter 7 (p. 153)). The A32 keeps a stored settings structure only if its size matches exactly, so a rolled-back A32 would run that structure on its defaults. With nothing written until confirmation, a rolled-back program always finds its settings exactly as it left them. So a settings layout change is safe over the air on both boards, with no special step. Downloading a settings file before any update is still a good habit.

The cost. A change made during the trial minute is lost if the trial ends in a reset: a rollback, a reboot of that board, or a power cut.

3.11.6 A reboot during a trial is a rollback

The S3. An ordinary portal reboot first saves the settings, and refuses if the save fails (chapter 8). During the trial there is nothing it may save. So Reboot the main board skips the save and the needle stop. It answers `rebooting - this firmware was still on trial, so the previous firmware comes back`, and restarts at once. The bootloader then starts the previous program. This button is the way to take back an S3 update that misbehaves in its first minute. Refusing would take away the one button that undoes a bad update.

The A32. Reboot the audio board during the trial forces a save that writes nothing, and the restart is a rollback like any other reset.

3.11.7 Reading the outcome without a cable

The S3′s image line, in the console's `s` status (readable on the portal's Console tab), for example `image : ON TRIAL (rollback armed), watchdog on`. The status also gives the last reset reason (chapter 4 (p. 46), section 4.10 (p. 59)).

The A32′s boot report, sent once per A32 start-up at the first HELLO from the S3, and printed on the S3′s console:

```
[A32] boot: commit <hash>, reset reason N, setup X ms, image
<state>, watchdog on|OFF
```

with `- an earlier update was ROLLED BACK` appended when the bootloader has marked a slot invalid. Reset reason 6 is the task watchdog. Chapter 9 explains every field. The report is not repeated if the S3 restarts and handshakes again.

A failed link self-test. Each board tests its own frame encoder and decoder at start-up. Only a broken build can fail it. Neither board halts:

- The S3 prints `[FAIL] proto self-test - this image will NOT be confirmed;` and `reboot to roll back, or upload a good build.` It then starts as usual: display, needle, link, WiFi and portal. The program is simply never confirmed. Sent over the air, it stays on trial, so a portal reboot rolls it back (S3 updates are refused meanwhile). Written by USB, it is not on trial and cannot roll back; upload a good build through the portal.
- The A32 stays muted and closed to Bluetooth, but keeps its link and its update receiver, and says `proto self-test FAILED - muted, BT closed, awaiting OTA` every 10 s. It is never confirmed, so the next reset takes the previous program back.

3.12 Walk-through: updating both boards

This is the whole round trip, with no USB cable connected. When a release changes only one board, update only that board. It assumes the two boards share one protocol version. If the portal shows BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both, the relay in step 6 cannot reach the audio board: follow section 3.13 (p. 37) instead. A release that itself changes the protocol version needs the opposite order, audio board first (chapter 9 (p. 224), section 9.14.5).

1. ☐ Build both environments. Check both exit codes (section 3.1 (p. 25)). Note the commit you built from.
2. ☐ Sign in to the portal as the administrator and open the System tab. Its "running" line shows the S3's current `v.1.<stamp> (<hash>).`
3. ☐ Choose `.pio/build/s3/firmware.bin` in the main board's picker, and upload. The clock goes blank, the panel lamps go dark and the needle stops; the sound keeps playing.
4. ☐ Reload the page (sign in again if it asks). Check that the "running" line shows the new stamp and hash. On the Console tab, `s` ends with `image : ON TRIAL (rollback armed), watchdog on.`
5. ☐ Leave the S3 alone for its trial minute. Do not upload to it again; it will be refused. If the new program misbehaves, press Reboot the main board to bring the previous program back with its settings as they were. Otherwise wait for `image confirmed (a minute of running, network and portal up).`
6. ☐ Choose `.pio/build/a32/firmware.bin` in the audio board's picker, and upload. The sound stops; the S3's clock, lamps and needle carry on. The relay takes over a minute for a full program.
7. ☐ Watch the S3's console for three lines: the A32's boot report, the new handshake (`[PASS] A32 up: proto v4, firmware v.1.<stamp>`), then `[A32] image confirmed (a minute of running with the S3)` about a minute later.

8. ☐ If either board's image line or boot report reads `- an earlier update was ROLLED BACK`, read the reset reason, fix the build, and upload again.

A build that printed `-dirty` names code that no commit records; an upload inside the trial minute is refused with HTTP 409 `this firmware is still on trial - retry in a minute`, and nothing is written. Settings changed during the trial minute are held in RAM and written only at confirmation (section 3.11.5); if the audio board had any, `[A32] settings changed during the trial are now saved` appears next to its confirmation line.

3.13 Versions, and where to read them

The firmware carries four independent numbers. Do not confuse them.

Table 19. Versions, and where to read them

Number	Set in	What it means	Changes when
`FW_VERSION`, for example `v.1.20260926T101500`	`scripts/version.py`	Which binary: MAJOR plus the build time.	Every build.
`FW_COMMIT`, for example `a1b2c3d` or `a1b2c3d-dirty`	`scripts/version.py`	Which source it was built from.	Every commit, or uncommitted changes.
`PROTO_VERSION`, 4	`include/proto.h`	Compatibility on the link between the two boards.	A protocol change that breaks compatibility.
`SETTINGS_VERSION`, 7	`src/s3/main.cpp`	The layout of the S3's stored settings.	A settings layout change (append-only, with a migration; chapter 7 (p. 153)).

MAJOR means only the release number. It is `1` from the v.1.0 release, and it changes only when a new release is declared. A protocol break is `PROTO_VERSION`'s job, not MAJOR's.

Releases. Each release is marked in the firmware repository by an annotated git tag. The tag names a commit; the version stamp still carries only MAJOR, so every build reads `v.1.<stamp>`.

This book describes the pair on the radio: the S3 at `v.1.0.5` (`v.1.20260927T101323`) and the A32 at `v.1.0.4` (`v.1.20260926T113302`), both sent over the air and confirmed. When a release changes only one board, the two boards run builds from different commits. That is normal, and the handshake is what tells them apart.

When the two boards run different protocol versions. At the handshake each board sends its `PROTO_VERSION` and `FW_VERSION`. But the link drops every frame whose protocol number is not its own, before anything reads it, and counts it. So boards with different protocol versions never complete a handshake: the link simply looks silent, like a broken cable. The count is what tells the two apart:

- the `link` line of the console's `s` shows a `wrong-version` count above 0, followed by `<- the boards run different protocol versions: flash both`

- while the link is down, the portal shows the pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both
- `GET /api/state` carries the count as `linkver`.

What to do when they differ. “Flash both” needs the right order, because the relay to the audio board refuses (`the A32 is not answering`) while the versions differ:

1. ☐ Update the main board first, over its own WiFi, with a build whose protocol version matches the audio board’s. The main board’s own update does not use the link, so the mismatch does not block it.
2. ☐ Once the versions match and the handshake comes back, update the audio board through the main board’s portal, if it needs a new program too.

If the audio board is still on trial after its own update, unplugging the set also works: it rolls back to its previous program. Once the audio board has confirmed its program, the mismatch survives a power cycle. To move both boards to a new protocol version on purpose, chapter 9 (p. 224), section 9.14.5 gives the one order that works over the air.

Fields added at the end of the A32′s status report do not change the protocol version. An older peer’s report then has the wrong length and is rejected. So a half-updated machine loses its telemetry but keeps its sound (chapter 4 (p. 46), section 4.10 (p. 59) chapter 9 (p. 224) has the full story).

Where to read each board’s version. Each board shows its version in different places:

Table 20. Versions, and where to read them

Board	Where
S3	The portal’s System tab, line “running”: `v.1.<stamp> (<hash>)` (the schema field `fw`, from `GET /api/schema`, or inside `GET /api/boot` whenever the browser’s cached schema is out of date, which it always is after an update). The console banner, `firmware v.1.<stamp> (<hash>)`. The second line of a downloaded settings file, `# firmware v.1.<stamp> (<hash>)` the first is `# Ambersong settings`.
A32	Only on the S3′s console, which the portal shows: `[PASS] A32 up: proto vN, firmware <version>` at each handshake, and the boot report, `[A32] boot: commit <hash>, ...`. No portal field carries the A32′s version.

> FOR FIRMWARE CHANGES
>
> The portal’s cached settings schema is keyed on `FW_VERSION` plus the administrator flag, so every new S3 build makes browsers fetch a fresh schema.

3.14 Build and update constants

Nothing in this chapter is a user setting. These are the values that govern building and updating.

> FOR FIRMWARE CHANGES
>
> Table 21. Build and update constants

Name	Value	Meaning	Where
`MAJOR`	1	The release number.	`scripts/version.py`, by hand
`FW_VERSION`	`v.MAJOR.YYYYMMDDTHHMMSS`, at most 23 characters	Which binary.	`scripts/version.py`, every build
`FW_COMMIT`	7-character short hash, plus `-dirty` or `nogit`	Which source.	`scripts/version.py`, every build
`PROTO_VERSION`	4	Link compatibility.	`include/proto.h`, by hand
`PROTO_VERSION_LEN`	24 bytes	The version field in the handshake. Changing it is a protocol change.	`include/proto.h`
`PROTO_MAX_PAYLOAD`	1088 bytes	Largest frame payload: a 1024-byte update chunk plus its header.	`include/proto.h`
Page compression	gzip level 9, `mtime=0`	The portal page in program memory.	`scripts/page.py`
Upload watchdog	15 s	Abort an upload with no chunk for this long.	S3 portal
S3 restart after an update	300 ms	Time for the HTTP answer to leave.	S3 portal
A32 begin answer wait	20 s	The A32 preparing its idle slot.	S3 relay
A32 fade wait before writing	400 ms	Mute fade plus audio buffer drain.	A32
Relay frame	1024 bytes	Update data per frame.	S3 relay
Relay tries per frame	4 × 3 s	Resend budget.	S3 relay
A32 end answer wait	4 s	Waiting for `ok, rebooting`.	S3 relay
A32 relay give-up	20 s	No frame for this long ends the A32′s update.	A32
A32 restart after the end	250 ms	—	A32
A32 trial confirmation	60 s after the handshake, or 300 s with no traffic from the S3; retry every 10 s	When a relayed program becomes permanent.	A32
A32 task watchdog	15 s	A frozen main loop resets the chip.	A32
S3 trial confirmation	not before 60 s, then a check every 10 s; earliest about 70 s	When an S3 program becomes permanent.	S3
S3 task watchdog	15 s, on the main loop and the needle supervisor	A hang becomes a reset.	S3
Settings writes during a trial	none, on either board	Changes wait in RAM until confirmation.	S3 and A32
S3 portal reboot during a trial	at once, no save attempted (200 ms for the answer)	A reboot of a trial program is a rollback.	S3 portal

3.15 When it goes wrong

Table 22. When it goes wrong

What you see	What happened	What the firmware does	What you do
A build reports success but the exit code is not 0	It did not build.	—	Trust the exit code.
An upload is refused for the wrong chip	The file in that picker was built for the other board.	Refuses before writing. For the A32: `that image is not for the audio board (wrong chip) - nothing was written.`	Pick the other file.
An upload stops halfway	The browser's connection died.	Aborts after 15 s without a chunk; the S3 goes back to work.	Upload again.
An S3 update fails without a restart	The write failed.	Reports it; the needle and display come back as they were.	Upload again.
The page reports a failure, yet the board restarted	The restart beat the answer.	—	Read the version on the System tab after the restart.
`this firmware is still on trial - retry in a minute` (HTTP 409)	The running S3 program is still on trial.	Drops the upload; darkens nothing.	Wait for `image confirmed`, then upload again.
`A32: still on trial - retry in 1 min`	The running A32 program is still on trial.	Refuses before muting.	Wait a minute.
`the A32 is not answering`	The A32 has not said HELLO.	Refuses to begin.	Wait for the handshake (chapter 9 (p. 224)).
The relay stops mid-stream; the sound stays off until the A32 gives up 20 s later, and the A32 keeps its old program	The relay stopped mid-stream. The S3 sends no abort for a failed frame.	The A32 gives up after 20 s and restores your mute setting.	Upload again.
`the A32 did not confirm the update - check its version once it reconnects`	The A32's final answer was lost. It commits before it answers, so the update may well have landed.	Re-handshakes.	Read `[PASS] A32 up: ...` on the console.
`- an earlier update was ROLLED BACK`	A new program crashed or hung before it was confirmed.	The bootloader started the previous program.	Read the reset reason, fix the build, upload again.
A new S3 program misbehaves, still on trial, portal reachable	—	Nothing by itself.	Reboot the main board: it rolls back at once.
A new S3 program never confirms	Its portal task stalled, or no network came up.	Stays on trial; S3 uploads are refused.	Any reset returns the previous program. With the portal unreachable that means unplugging the set, because the front switch neither resets the S3 nor removes its power.
`[FAIL] proto self-test - this image will NOT be confirmed`	A broken S3 build.	Boots as usual; never confirms.	Sent over the air: Reboot the main board rolls it back. Written by USB: upload a good build through the portal.
A relayed A32 program crashes or hangs at start-up	—	The A32's watchdog or the crash resets it; the	Nothing; fix the build.

What you see	What happened	What the firmware does	What you do
		bootloader starts the previous program; the boot report says `ROLLED BACK`.	
A relayed A32 program runs but never completes a handshake	Bytes arrive from the S3, but no HELLO.	Stays on trial, never confirmed.	A reset returns the previous program. The S3 cannot reset the A32 without the link, so unplug the set: the front switch does not remove the A32's power.
`image confirm FAILED (err N) – retrying`	The confirmation write failed.	Retries every 10 s; updates refused meanwhile.	Nothing.
Settings changed during a trial are gone	The trial ended in a reset: rollback, a reboot of that board, unplugging.	Nothing was written; the stored settings are those from before the update.	Set them again once a program is confirmed.
SETTINGS LOCKED after a deliberate downgrade	The older program does not know the stored settings version.	Runs on defaults; writes nothing.	Upload a complete settings file (chapter 7 (p. 153)).

3.16 Design choices

- The platform pinned to one release. The A32's audio needs the legacy I2S driver, which the Bluetooth library drives directly only on that release's core.
- The Bluetooth library pinned to one commit. The A32 overrides its internals, and an unpinned dependency changes whenever the `.pio` folder is rebuilt.
- One project, two environments, one shared **`include/`**. The pin map and the protocol must be the same file for both boards by construction; a pin map kept in more than one place drifted apart five times during bring-up.
- The version stamped at build time from the build PC's clock. The useful question is "which binary", and the boards run builds from different moments because the S3 updates the A32.
- A separate commit stamp, never inside the version. It would not fit the 24-byte handshake field, and widening it is a protocol change.
- The portal page compressed at build time and served from program memory. Uncompressed page loads took 0.7 to 3.6 s on a server that handles one connection at a time. Every browser accepts gzip. The chip's ROM has a decompressor but no compressor, so compressing per request would only move the cost.
- A zero gzip timestamp. An unchanged page gives byte-identical data, so the ETag holds across rebuilds and a reload costs one empty "304 Not Modified".
- The build fails if the page's script does not parse. A duplicated declaration once shipped a dead page; a syntax error is the one class of bug a machine can catch for free.

- No filesystem for the page. Nothing to upload separately, nothing to fall out of step.
- Two program slots on the A32. Without a second slot there is no update over the air, and the A32 is inside the cabinet.
- The A32 updated through the S3, one acknowledged 1 KB frame at a time, a whole-image CRC32, and a final answer required. The A32 has no network and no reset wire from the S3. A transfer that cannot tell whether it arrived is worse than a slow one.
- The chip checked in the image header before anything is written. The two pickers sit side by side and both files are called `firmware.bin`.
- The A32 confirms a minute after the handshake (or after five minutes with no traffic at all), refuses updates on trial, and runs a 15-second watchdog. An update that cannot boot must not need the USB cable. The watchdog turns a hang into the reset that triggers the rollback.
- The S3 gets the same treatment: trial, refusal on trial, a self-test that degrades instead of halting, and a 15-second watchdog. Otherwise an S3 build that crashed, hung or could not serve the portal needed the USB cable.
- No settings written on either board during a trial. A rolled-back program must never find a layout newer than it knows, and the defence must not depend on a person remembering a procedure.
- A portal reboot during the S3's trial goes through at once. There is nothing it may save, and refusing would take away the one button that undoes a bad update.
- Every board identified by its MAC address, the port always given, and no flashing under observation. Boards of one kind share USB numbers on the bench; an update during a listening or measuring session spoils it.

3.17 Tried and rejected

FOR FIRMWARE CHANGES

- One 3 MB program slot on the A32, as the bench suite used: no room for an update over the air.
- An unpinned Bluetooth library: worked until the visibility rule came to depend on its internals.
- Sending the portal page uncompressed: page loads of 0.7 to 3.6 s.
- Waiting 5 s for the A32 to answer the begin: the update never started, consistently. Now 20 s.
- A 250 ms fade wait before the A32 writes flash: too little margin once the audio buffers were counted. Now 400 ms.
- Trusting the HTTP answer of an update: several updates reported failure and had landed.
- Treating the relay's silent end as success: a lost final answer hid whether the A32 had taken the update, and the S3 went on showing the old A32 version.

- Confirming an A32 program at the first handshake: the handshake lands before Bluetooth starts, so a program that crashed later was already confirmed.
- Judging a build by searching its output for **SUCCESS**: hid a linker failure.
- A version stamp without a commit: matching a build to its source meant comparing build and commit times.
- Letting the Arduino core confirm every new S3 program: a build that crashed or hung at boot looped on itself, and only the USB cable could fix it.
- Saving settings during the S3′s trial, with a settings file kept as the key: the defence was a procedure to remember. Replaced by holding every write.
- Saving settings during the A32′s trial, with advice to export a settings file first: replaced by the same hold.

3.18 Known limits

- The A32′s version is in no portal field. It is readable only on the S3′s console, at each handshake and in the boot report.
- A relay that fails on a frame sends the A32 no abort. It stays muted until its own 20-second give-up (chapter 12 (p. 290)).
- An A32 program built with the wrong **PROTO_VERSION** confirms itself after five minutes. Frames of another protocol version count only as wrong-version frames. The five-minute fallback does not look at those, so to the A32 the S3 looks absent. The link stays down until both boards share a version; the S3 shows the wrong-version count and pill (chapter 12 (p. 290)).
- **– an earlier update was ROLLED BACK** stays in every image line and boot report until the next update replaces the invalid program, so an old rollback can look like a new one (chapter 12 (p. 290)).
- The version stamp uses the build PC's local time with no time zone. Builds made in different zones, or across a daylight-saving change, cannot be ordered by stamp alone.
- A partition-table change always needs a USB cable.
- The downgrade lock has never been exercised. Since settings are held during the trial, a rollback can no longer cause it; only a deliberate downgrade can.
- Rollback has been seen working on both boards, with a deliberately hanging program and with a portal reboot during the S3′s trial. Never exercised on the radio: an S3 program that runs but never earns confirmation; a failed link self-test on either board; a confirmation write that fails; a reset during either board's trial with a setting changed in it.

3.19 Changing it safely

FOR FIRMWARE CHANGES

What must stay true

- The pinned platform and library commit stay pinned. If you move either, do it on purpose, rebuild both boards, and re-test the A32′s sound (radio, Bluetooth and AUX) and its Bluetooth visibility in every mode.
- `FW_VERSION` fits in `PROTO_VERSION_LEN - 1` = 23 characters. The commit hash never goes into it.
- A change that breaks the link's compatibility bumps `PROTO_VERSION`. Appending fields to the status report `ProtoState` does not. MAJOR is not touched for either.
- NVS stays at 0x9000 in both partition tables, so stored settings survive a partition change.
- Edit `data/portal.html`, never `src/s3/page_gz.h`.
- Nothing secret goes into the source: WiFi credentials and accounts live only in NVS.
- Both update handlers (`hOtaUpload()`/`hOtaEnd()` and `hOtaA32Upload()`/`hOtaA32End()` in `src/s3/portal.cpp`) keep their own administrator and placeholder-password checks — they are the only handlers that do not go through `requireLogin()` — and the chip check on the first chunk.
- Any change to a trial keeps two properties: a program is confirmed only after it has shown that it runs, not merely that it booted; and an update is refused while the running program is on trial. On the S3, confirmation never waits for the A32 or the needle, and is never given to a program whose link self-test failed; the same holds for the A32′s self-test.
- Both programs keep `verifyRollbackLater()` returning true, and the task watchdog stays the first thing `setup()` does, so a hang anywhere in the start-up is caught.
- Nothing reaches the S3′s stored settings while its program is on trial: every write goes through `settingsWrite()` in `src/s3/main.cpp`, which refuses on trial. A new path that writes the `amb3` namespace directly would reopen the rollback trap.
- The portal's reboot (`hReboot()`) never waits for, or refuses on, a save while the S3 is on trial.
- The same on the A32: nothing reaches namespace `amb` while its program is on trial. Every write goes through `settingsFlush()` in `src/a32/main.cpp`, which refuses on trial, and `confirmImage()` writes what was held.
- On the A32, the refusal of `MSG_OTA_BEGIN` while on trial, in its message handler, is the only guard: the Arduino `Update` class writes the slot

directly and does not go through the framework's own "refuse while pending" check.

Where the code is. The S3′s trial and confirmation are in `src/s3/main.cpp` (`s3ImageOnTrial()`, `confirmTick()`, `imageReport()`, constant `CONFIRM_AFTER_MS`); its updates in `src/s3/portal.cpp` the relay in `src/s3/main.cpp` (`a32OtaBegin()`, `a32OtaChunk()`, `a32OtaEnd()`). The A32′s side is the `MSG_OTA_*` cases of its message handler, `confirmTick()`, `confirmImage()` and `bootReport()` in `src/a32/main.cpp`.

Traps

- `extra_scripts` in an environment replaces the common list. Keep `version.py` listed in the `s3` environment.
- `pio device list` shows a USB bridge's serial number, not the ESP32′s MAC address. Use `esptool.py read_mac` for boards behind a bridge.
- The S3′s native USB console does not reset the board when a monitor opens. The banner is gone by the time you look.
- `-dirty` looks only at tracked files. A new, untracked source file that is compiled in does not mark the build dirty. Commit before building a release.
- A setting changed during the S3′s trial minute is only in RAM. It is written once the program is confirmed, and lost if the trial ends in a reset.
- A second S3 upload inside the trial minute is refused with HTTP 409 only after the whole file has been sent. Wait for `image confirmed` first.

How to test a change here

1. Build both environments and check both exit codes.
2. Update the S3 over the air. Confirm with the System tab's "running" line (version and commit), not the upload's answer. The image line must read `ON TRIAL (rollback armed), watchdog on`.
3. Try a second S3 update inside the trial: it must be refused with HTTP 409. Change a setting and press Save: it must answer that the settings are held until confirmation. Then wait for `image confirmed (a minute of running, network and portal up)`. If you changed anything about the trial, also check the reboot rollback: send a program, press Reboot the main board during its trial, and check that the S3 comes back on the previous commit with `ROLLED BACK` in its image line.
4. Update the A32 over the air. On the S3′s console, look for the boot report (`image ON TRIAL (rollback armed), watchdog on`), the new version at the handshake, and `image confirmed (a minute of running with the S3)` about a minute later.
5. Try a second A32 update inside that minute: it must be refused. Change an audio-board setting in that minute (for example the Bluetooth lamp's `btLedOff`, then put it back): at the confirmation the

console must show `[A32] settings changed during the trial are now saved.`
6. If you changed anything about the trial or the watchdog, repeat the rollback test with a deliberately hanging program (its main loop spinning for ever), sent over the air with the USB cables unplugged, and watched through its rollback. Never commit that program.
7. Download a settings file and check that its second line, `# firmware ...`, names the new build and commit.

4. The main board: start-up, clock, lamps and time

The S3, an ESP32-S3 module, is the main board of Ambersong. It owns everything a person sees move or light up on the front of the radio: the four-digit amber clock behind the dial glass (the Leditron), the panel lamps behind the dial, and the dial needle. It also reads the tuning knob, senses whether the amplifier is on, and runs the WiFi, the portal and the firmware updates of both boards. This chapter covers how the S3 starts, how its work is shared out, the clock display, the panel lamps, time keeping, the health it reports and its watchdog. The needle has chapter 5 (p. 73), tuning chapter 6 (p. 120), the network and portal chapter 8 (p. 186).

4.1 What you see

- The clock. Four digits show local time as HH:MM, in 24-hour form by default. A leading zero in the hour is blanked, so 09:05 reads " 9:05". The colon between the digits is not under firmware control: it is always lit (Hardware Bible, chapter 7).
- Two clock brightnesses. Full while the amplifier is on, a lower "standby" level while it is off. The change never snaps: the clock holds for a quarter of a second, then eases to the new level over a second and a half.
- An optional frequency readout on the same four digits. `1017` means 101.7 MHz, because the display has no decimal point. It can be off (the default), always on, or shown only while the tuning knob is being turned.
- Blank digits when the machine does not know the time. This is deliberate: a blank clock is better than a plausible wrong one. After every start of the S3 the clock is blank for the second or two it takes to get the time from the audio board.
- Panel lamps that follow what you do. They are dark with the amplifier off. On the RADIO source they come up to full while the knob is turned, and ease down to half five seconds after the knob stops. On the AUX and Bluetooth sources they sit at a separate level.

- During an update of the S3 the clock goes blank, the panel lamps go dark and the needle stops. The radio keeps playing, because the sound lives on the A32. Everything comes back when the S3 restarts.

The S3 never reads the source selector, the volume knob or the Bluetooth button, and never drives the blue lamp: those belong to the A32. It learns the source, the Bluetooth state, the volume and the selector's raw reading (for diagnosis) from the status report the A32 sends four times a second. No firmware on either board drives a fan.

Terms used in this chapter (chapter 2 (p. 14), section 2.4 (p. 18) explains tasks, cores and priorities):

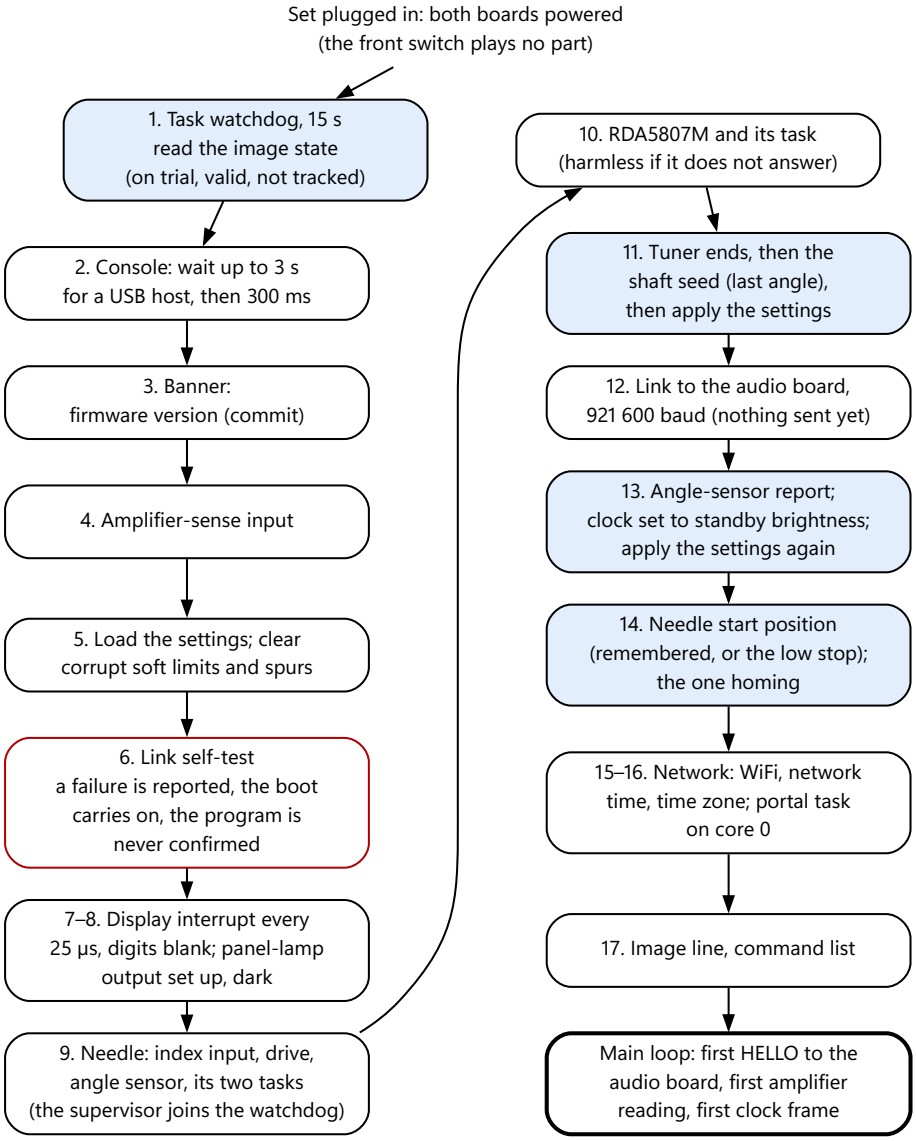
Table 23. What you see

Term	Meaning
Interrupt routine	A function the hardware calls on a timer. It pre-empts every task on its core.
IRAM	The chip's internal instruction RAM. Code placed there keeps running while the external flash chip is busy, for example during a settings save or a firmware write.
LEDC	The ESP32's PWM peripheral ("LED control"). Used here for the panel lamps and the needle motor's coils.
Handshake	The HELLO / HELLO_ACK exchange that opens the link with the A32 (chapter 9 (p. 224)).
Epoch	Seconds since 1970-01-01 00:00 UTC (Unix time).
Software clock	The S3's own count of the time, re-set from the battery clock or network time (section 4.9 (p. 57)).

4.2 Start-up, step by step

An S3 start-up is not the moment a person turns the radio on. Both boards are powered whenever the set is plugged in, whatever the front switch says (Hardware Bible, chapter 5). The S3 starts when the set is plugged in, or after a restart. The front switch brings the amplifier up, and that is the moment a person turns the radio on; the S3 sees it as an amplifier change (section 4.5 (p. 52)).

Figure 8. The S3′s start-up, step by step



The start-up runs on core 1, in this order:

Table 24. Start-up, step by step

Step	What happens	Why it sits here
1	The task watchdog, 15 s, panic on, and the main loop placed under it. Then the S3 reads the state of the program it is running from: on trial, valid, or not tracked (chapter 3 (p. 25), section 3.11 (p. 32)).	First, so that a hang anywhere below resets the chip.
2	The console. It waits up to 3 s for a USB host to open the port, then 300 ms more. With no cable attached this costs the full 3.3 s at every start.	The wait keeps the start-up report whole when a cable is attached. It is a known delay, not yet shortened (chapter 12 (p. 290)). The whole

Step	What happens	Why it sits here
		start-up must finish within 15 s, because the watchdog is already watching.
3	The banner: the name and `firmware <FW_VERSION> (<FW_COMMIT>)`, between two rules of `=`.	—
4	The amplifier-sense input, a plain input with no internal pull resistor.	—
5	The settings are loaded from flash (chapter 7 (p. 153)). Stored soft limits found corrupt are cleared, and so is a stored spur list longer than 16 entries or with any entry outside 77.2 to 97.2 MHz (chapters 5 (p. 73) and 6).	—
6	The link self-test. It encodes a status frame, decodes it, compares the two, then corrupts one byte and checks that the checksum rejects it: a checksum never seen to fail is not known to work. A failure is reported and the start-up carries on; such a program is never confirmed (chapter 3 (p. 25), section 3.11.7 (p. 35)).	—
7	The display. Seven segment outputs and four digit outputs, all low; hardware timer 0 starts its interrupt every 25 µs (section 4.6 (p. 53)). The digits stay blank until the main loop draws the first frame.	Started from core 1, so the interrupt lives on core 1, away from WiFi on core 0.
8	The panel lamps: LEDC channel 1, 1 kHz, 8 bits, dark.	—
9	The needle: the index input (its debounce seeded from a live read), the motor drive (microstepping at `microFast`), the angle sensor's bus with its first status and angle reads, a warning if the sensor's gain is near its ceiling (checked against the compiled defaults, since the stored settings are not applied yet), and the needle's two tasks, `nstep` and `needle` (chapter 5 (p. 73)). The `needle` task puts itself under the watchdog.	After step 1, so the watchdog exists when the supervisor joins it.
10	The RDA5807M receiver: its bus, the chip set-up, and the `rda` task. Harmless if the chip does not answer (chapter 6 (p. 120)).	—
11	The tuning shaft: the measured tuner ends first, then the shaft seed from the last stored angle, then all the settings are pushed to every module.	The seed uses the tuner ends to decide which turn of the shaft the knob is on (chapter 6 (p. 120)).
12	The link to the A32 on its serial port, at 921 600 baud, 8N1, no flow control, with its pins given explicitly. Nothing is sent yet; the main loop sends the first HELLO.	—
13	The angle-sensor report (does it answer, does it see its magnet, its automatic gain). Then the clock is set at once to its standby brightness, and the settings are pushed again.	The clock starts dim and eases up once the main loop has read the amplifier as on.
14	The one homing. The needle's start position is set from the reason for the reset: after a software-type reset, the position remembered in RAM; after a power-up, the low stop (section 4.10 (p. 59) chapter 5 (p. 73)). Then homing starts.	After step 13's settings, which give the needle its geometry.
15	The network: the WiFi network, network time server and time zone are loaded from their own store; the time zone is applied; the S3 starts joining the house network or raises the rescue access point (chapter 8 (p. 186)).	—
16	The portal: the web server and its task on core 0 (chapter 8 (p. 186)).	—

Step	What happens	Why it sits here
17	The image line, `image : <state>, watchdog on|OFF`, with `- an earlier update was ROLLED BACK` when the bootloader has thrown an earlier program away; then the list of console keys.	—

FOR FIRMWARE CHANGES

Pushing the settings to the modules sets the clock brightness target and the panel levels and timings. It sets the needle's limits, motion profiles, microstepping and geometry (taking back what the needle accepted). It also sets the index band, the tuner ends, the tuning curve, the printed dial, the RDA5807M's list of fixed spurs, and the WiFi transmit power.

Four orderings matter, and only the order of the code enforces them:

1. the watchdog before everything (step 1);
2. the tuner ends before the shaft seed (step 11);
3. standby brightness before the first amplifier reading (step 13);
4. the needle's geometry before its start position (step 14).

When homing finishes, the main loop sees the needle's "homed" flag change and gives the needle its job: follow the knob or park (chapter 5 (p. 73)). A sweep in progress is not interrupted.

4.3 Tasks and cores: the rules a modifier must keep

The S3′s tasks, their cores, priorities, periods and stacks are in one table in chapter 2 (p. 14), section 2.4 (p. 18). What follows is what that layout means for anyone changing the code.

FOR FIRMWARE CHANGES

- On core 1, the main loop never yields. It has no delay in its normal path, so the idle task of core 1 never runs, and nothing at priority 0 ever gets core 1. A task at priority 1 on core 1 shares it with the main loop only by time slicing. Periodic work belongs in the main loop, or in a task at priority 2 or higher that yields. The watchdog does not check core 1′s idle task in this build's configuration, so this causes no resets.
- On core 0, the step emitter **`nstep`** spins at priority 19 while the needle moves, so the portal (priority 3) and WiFi get core 0 only between steps. Portal requests can be slow while the needle moves; this is known and accepted. Core 0′s idle task cannot run during a move either, so the step emitter takes it off the watchdog. The watchdog watches the main loop and the `needle` supervisor, and not core 0′s idle task, the step emitter or the portal (section 4.11 (p. 60)).
- Why the step emitter is on core 0. On core 1 it would starve the main loop (the link and the panel) for the length of every sweep, and it would compete with the display interrupt. A needle stutter while the portal is in use is more forgivable than a stalled link or a glitched clock.

- Why the portal is on core 0. Core 1 belongs to the display interrupt and the needle.
- What can hold the main loop. Three console prompts wait inside it and keep the watchdog fed while they wait: `w` (the clock) until 40 s after the last key typed, `y` (the WiFi network) until 60 s after the last key typed, and `l` (the live index monitor) for up to 60 s. Each key typed at `w` or `y` starts its wait again, so those two can hold the loop longer. The `i` report's WiFi scan holds it for a few seconds; a settings save for milliseconds. The `needle` and `rda` tasks pre-empt it.

4.4 The main loop

The main loop runs on core 1 and does the S3's routine work: the link, the network, the settings save, the clock, the panel lamps and the console (chapter 2 (p. 14), section 2.4 (p. 18)).

FOR FIRMWARE CHANGES

Every pass runs these steps, in this order, with no delay between passes:

1. Receive from the A32. Complete frames go to the message handler (chapter 9 (p. 224)).
2. The network state machine and network-time bookkeeping (chapter 8 (p. 186)). Then, while the running program is on trial, the check that confirms it (chapter 3 (p. 25), section 3.11.2 (p. 33)).
3. The learned WiFi transmit power: if the network module's value differs from the stored one, it is copied into the settings and marked changed. It is learned by the network module and stepped only from the console; it is not a portal row and not in the settings file (chapter 8 (p. 186)).
4. The RDA sampler and the needle's re-index triggers (chapters 5 (p. 73) and 6).
5. The settings save: settings unchanged for 2 s are written if it is safe: the needle still, no calibration running, no upload active. They are never written while the running program is on trial (chapter 7 (p. 153)).
6. The clock brightness fade advances one step (section 4.7 (p. 55)).
7. Amplifier sense (section 4.5 (p. 52)): debounce; on a change, retarget the clock brightness, perhaps home or sweep the needle, re-apply the needle's job, and tell the A32 at once. Otherwise tell the A32 the amplifier state every 2 s while the handshake stands.
8. The link keepalive. HELLO every second until the A32 answers; after that a PING every second and a request for the time every 60 s. A time request that gets no answer within 5 s, while the A32 is otherwise answering, marks the battery clock as not answering (section 4.10 (p. 59)). If the link has been silent for more than 2 s, the handshake is dropped and the copy of the A32's status and settings

is forgotten (chapter 7 (p. 153)). The console prints `[WARN] A32 silent. Clock and needle keep running.`

9. The needle follows the source and the amplifier. On a change of source, or when homing finishes, the needle's job is re-applied (not during an upload). If an upload ended without a restart and the needle's stop has landed, an unhomed needle is homed, or the job re-applied (chapter 5 (p. 73)).
10. The band calibration: if one finished, its result is copied into the settings, the soft limits are shifted with it, and the settings are saved (chapter 5 (p. 73)).
11. The panel lamps are given the amplifier state, the source and "knob moving", or driven dark during an upload (section 4.7 (p. 55)).
12. What the digits show is decided every 200 ms, except during an upload (section 4.6.2 (p. 54)).
13. Network time is written through to the battery clock at most once an hour (section 4.9 (p. 57)).
14. The shaft position: every 10 s, if the angle sensor answers and the shaft count changed, the count is stored as the last angle. The next start-up can then tell which turn of the shaft the knob is on. Only a live count is stored. Before the sensor's first good reading the count is a placeholder, and saving it would seat the next start-up on the wrong turn (chapter 6 (p. 120)).
15. The console: one key is taken and dispatched (Appendix D (p. 332)).

While the link is down, the S3 takes the source as AUX everywhere. With the amplifier on, this parks the needle and sets the panel lamps to their "other source" level. It also stops the "while tuning" readout.

4.5 Amplifier sense and "radio live"

The S3 reads its amplifier-sense input. Low means the amplifier is powered. The input is plain; the pull-up and the detector that pulls it low are outside the board (Hardware Bible, chapter 9).

- Debounce. A new reading is accepted only once it has differed from the accepted state for more than 50 ms. The first accepted reading after start-up makes the state known; before that the console reports the amplifier as "unknown".
- On an accepted change, the console prints `amp ON` or `amp OFF`, and the clock brightness is retargeted (`brightOn` or `brightOff`). If the amplifier has just come on, this is not the first reading after start-up, and no upload is running:
  - a needle with no zero starts homing, including a needle in FAULT: switching the amplifier on is a person asking for another try (chapter 5 (p. 73));

    - otherwise, if `sweepOn` is set, the needle makes its decorative end-to-end sweep.

  Then the needle's job is re-applied, and the new amplifier state is sent to the A32 at once.
- The front switch does not home a needle that has its zero. Homing happens once, at S3 start-up; after that the needle knows where it is (chapter 5 (p. 73)). The one exception is the bullet above: a needle with no zero, FAULT included, starts homing on every amplifier-on edge.
- The A32 is told the amplifier state every 2 s. It treats 2 s without hearing from the S3 as "amplifier down" and goes quiet (chapter 9 (p. 224)). That is why any console routine that holds the main loop must keep sending PING.
- Two minutes after the amplifier goes off, once per off period, the needle quietly re-homes and returns (chapter 5 (p. 73)).

"Radio live" has one definition: the amplifier is on and the source is RADIO, with the source taken as AUX when no valid status report from the A32 is held. It decides whether the needle follows the knob or parks, whether the "while tuning" readout may show, and whether an RDA measurement may run. It is a listening policy, not a statement about the tube set. The selector only tells the A32 which sound to play. The tube set runs whenever the amplifier does, whatever the selector says (Hardware Bible, chapter 5 (p. 73)). The needle follows and the dial calibrates only while the radio is what you are listening to, by choice.

4.6 The clock display

4.6.1 Multiplexing

The display has four seven-segment digits. Only one digit is lit at any instant. Each digit gets a slot of 2500 µs; four slots make a 10 ms frame, so the whole display refreshes at 100 Hz. Hardware timer 0 calls the display's interrupt routine every 25 µs; each call is one tick. A prescaler of 80 on the 80 MHz clock gives 1 µs counts, and the alarm reloads at 25. 25 µs is the greatest common divisor of 825 and 1675 µs, so both phases below fall on whole ticks.

> FOR FIRMWARE CHANGES
> One slot is 100 ticks:

Table 25. Multiplexing

Ticks	What happens
0 to 32 (825 µs)	Everything dark: the dead time. It is never shortened.
33	The slot is timestamped, and its deviation from 2500 µs recorded if it is the worst so far. Then, if the lit window L is not zero, the digit is selected first, then that digit's segments are lit.
33 + L	The segments are cleared first, then the digit. Releasing the digit first would let the outgoing digit briefly show the incoming pattern.
99	Unconditional blank: all segments and the digit cleared, the tick counter reset, and the next digit chosen. This runs whatever L is.

The dead time gives the outgoing digit's switch time to turn fully off before the next digit is enabled; without it the display ghosts. On this machine the outgoing digit's high-side switch takes about 825 µs to stop conducting once its select is released. That figure was measured on this machine, and it is the firmware's own value: the Hardware Bible (chapter 7) records the circuit that causes the slow turn-off, not the timing. Another build of the display must measure its own.

Polarity. The routine sets a digit-select output high to turn its digit on, and a segment output high to light a segment, through the driver stages (Hardware Bible, chapter 7).

Brightness changes only the lit window L, never the slot. L is the part of each slot during which the digit is lit. L = brightness × 67 ÷ 255 ticks, at least 1 tick when the brightness is above 0, and at most 66. So the frame rate never changes and dimming cannot flicker. 255 is the maximum; there is no headroom above it. 0 is dark, but the display keeps refreshing.

> Worked example. At 255, L = 66 ticks = 1650 µs lit in each 2500 µs slot: each digit is lit 16.5 % of the time. At the standby default of 110, L = 110 × 67 ÷ 255 = 28 ticks = 700 µs, or 7 % of the time.

The font has the digits 0 to 9 only, in the standard layout (bit 0 = segment A at the top, then B, C, D, E, F, and bit 6 = G in the centre). Any other value blanks that digit. The digits are numbered from the right: digit 1 is the rightmost (units of minutes), digit 4 the leftmost (tens of hours). New digits are handed to the routine all four at once, with interrupts disabled, so it never shows half of an update. In 12-hour form the hour is shown modulo 12, with 0 shown as 12; there is no AM/PM indicator.

Timing health. The worst slot deviation is shown by the console `s` (`display : brightness N worst slot error N us`) and in the portal field `slot`. The console key `Z` zeroes it.

4.6.2 What the digits show

Every 200 ms the main loop decides, in this order:

1. A test pattern, if one is set: `8888` (console `8`, or the portal's display test), or all blank (console `b`). Console `9` returns to normal. A display you can

command is the difference between “the clock is broken” and “the clock has no time yet”.

2. During an S3 update, nothing is decided: the digits were blanked when the upload began (section 4.8 (p. 56)).
3. The frequency readout, if `showTuning` asks for it:

   - 1, “tuning always”: always, whatever the amplifier and the source. This is an operator override and the calibration instrument; calibrating the shaft against the printed dial is done standing at the set, often with the amplifier off.
   - 2, “tuning while tuning”: only while the radio is live and the knob moved within the last `tuneHoldMs` (4 s by default). The time of the last turn is not cleared while the radio is not live. Otherwise a 250 ms glitch of the source selector, or a 2 s link hiccup, would wipe a readout the listener is in the middle of reading.

   The frequency comes from the tuning curve (chapter 6 (p. 120)) and is clamped to the printed dial, `dialLow` to `dialHigh` (87.9 to 107.9 MHz by default). Mode 2 then snaps to the nearest North American FM channel; the channels sit on odd tenths of a MHz (87.9 + 0.2 k). The snap uses the unrounded value and never leaves the printed dial. Mode 1 is not snapped: it is the instrument, and rounding would hide the tenth of a MHz being measured. Leading zeros are not blanked, so 87.9 MHz reads `0879`. The portal shows the true, unclamped frequency, with “(past the printed face)” when it is off the dial; only the clock digits clamp.
4. No valid time: all four digits blank.
5. The clock: local time from the software clock, through the time-zone rule (section 4.9 (p. 57)), shown with the `hour12` and `blankLeadZero` settings.

4.7 Brightness and panel lamps

The clock brightness has two targets: `brightOn` (255 by default) while the amplifier is on, `brightOff` (110) while it is off.

- A new target starts a move from the current level, so a change of mind mid-fade never jumps.
- The level holds for `dispDwellMs` (250 ms), then moves along a smoothstep curve, $k = p^2(3 - 2p)$, over `dispFadeMs` (1500 ms). Smoothstep has zero slope at both ends, so the level starts slowly, speeds up and settles.
- Any change of setting retargets the brightness, so a new `brightOn` from the portal eases in.
- Only the start-up sets a level at once (section 4.2 (p. 47), step 13).
- The console keys `-` and `=` (or `+`) change `brightOn` by 15 and write it straight to the display, bypassing the fade (section 4.16 (p. 69)).

The panel lamps are one PWM output, at 1 kHz with 8-bit duty. A higher duty makes the panel lamps brighter (Hardware Bible, chapter 9). The 1 kHz frequency is a firmware choice.

FOR FIRMWARE CHANGES

The panel lamps' output is LEDC channel 1. LEDC channel 0 is left unused on the S3, for two reasons. The A32 uses its channel 0 for the blue lamp, and keeping them distinct makes the two programs read alike. And in this Arduino core, channels 0 and 1 share LEDC timer 0, so anything put on channel 0 at another frequency would change the panel lamps' frequency too.

Every pass of the main loop gives the lamps three inputs: the amplifier state, the source, and whether the knob is moving. The target level comes from those three alone, so there is no internal state to fall out of step:

Table 26. Brightness and panel lamps

Condition	Target
Amplifier off	0 (dark)
Amplifier on, source RADIO, knob moved within `panelIdleMs` (5 s)	`panelTuning` (255)
Amplifier on, source RADIO, knob idle for 5 s	`panelIdle` (128)
Amplifier on, source AUX or Bluetooth (or the link down)	`panelOther` (96)
S3 update in progress	0 (the main loop passes "amplifier off")

"Knob moving" is the needle module's tuning-activity flag. Its deadband must be wider than the angle sensor's own jitter, or the lamps flash to full with nobody touching the knob (chapter 5 (p. 73)).

Motion. A new target starts a move from the current level. The level holds for `panelDwellMs` (250 ms), then follows the same smoothstep curve over `panelFadeMs` (900 ms). The hold is what makes the machine seem to decide rather than react. The console `s` shows the panel's current level and target (`panel : current -> target`).

4.8 During an upload

When an update of the S3's own program starts (chapter 3 (p. 25), section 3.10.2 (p. 29)), the S3 goes quiet: the needle stops and the display is blanked. While it is quiet:

- the digits stay blank;
- the panel lamps are driven dark;
- amplifier and source changes do not move the needle;
- no settings are saved (chapter 7 (p. 153)).

Why. A firmware write is a long flash operation, and the machine cannot absorb one while it is moving or lit. The display timing test found flash traffic to be the one load that disturbs the display. It concluded that an upload should stop and darken the machine rather than let it look broken.

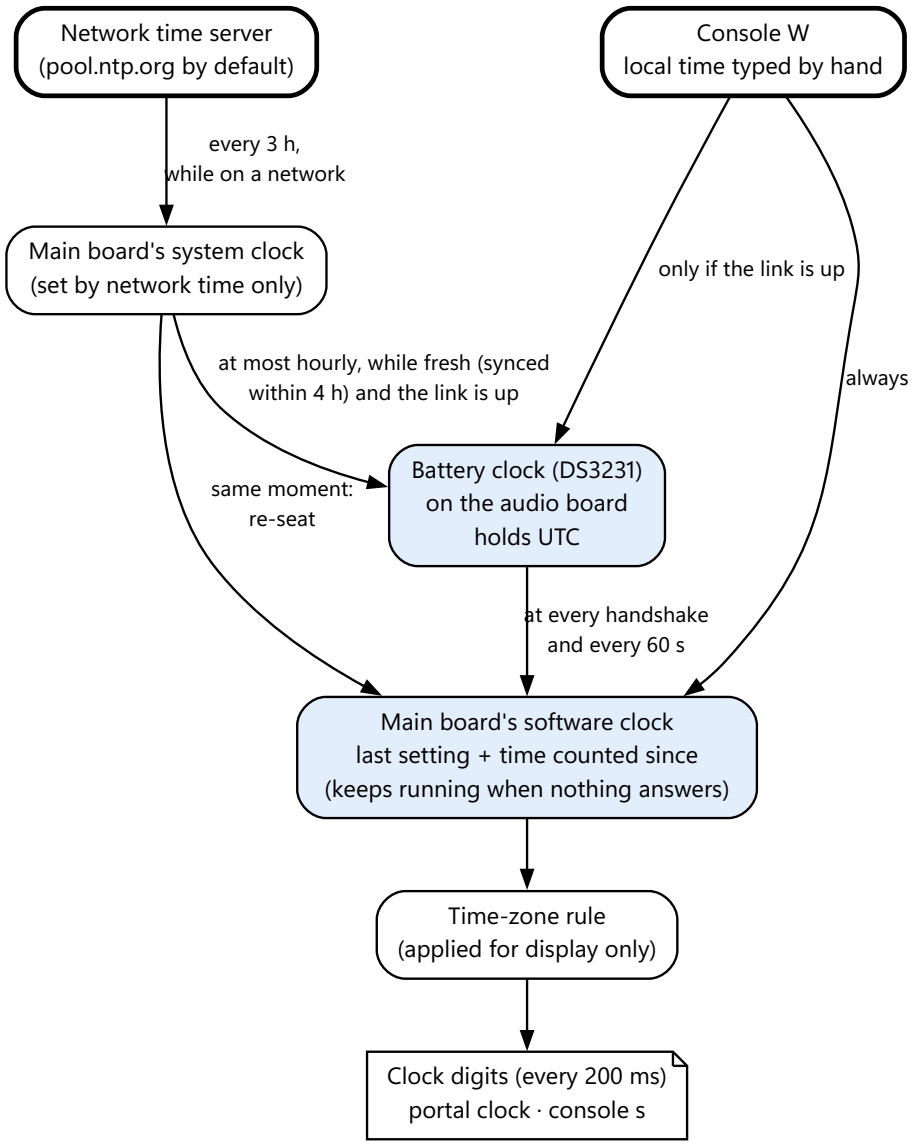
A successful upload ends in a restart, which brings everything back. An upload that ends without a restart (refused, aborted, failed) wakes the S3 up

again. Once the needle's stop has landed, the main loop homes an unhomed needle or re-applies its job (chapter 5 (p. 73)).

An S3 upload refused because the running program is on trial is refused before anything goes quiet, so the needle keeps running and the clock stays lit. An update of the A32, relayed through the S3, does not quiet the S3 at all (chapter 3 (p. 25), section 3.10.3 (p. 30)).

4.9 Time keeping

Figure 9. Where the time comes from, and how it reaches the clock digits



The battery clock is on the A32. The DS3231 clock chip is reachable only from the A32, so the time must cross the link. Its module carries a backup battery, which is what lets it keep time through a power cut (Hardware Bible, chapter 4); this book calls it the battery clock, as the portal does.

The S3 keeps its own software clock:

```
now = epoch at the last setting + (milliseconds since then) ÷
1000        (0 while there is no valid time)
```

It keeps counting when the A32 or the network is absent. A slow or missing link never makes the display stutter; the clock simply stops being corrected.

Three sources of time:

1. The battery clock, through the A32. At every handshake, and every 60 s after, the S3 asks the A32 for the time. The A32 answers with the time in UTC, a validity flag, and the chip's temperature in quarter degrees. It sets the flag only if all three hold: the time is plausible (after 1600000000, that is September 2020); the chip's status register answered; and the chip's oscillator-stopped flag is clear. A clock that has stopped is not trusted (chapter 9 (p. 224)). The S3 accepts a valid, plausible time and re-sets its software clock. An invalid one prints `[WARN] the DS3231 has no valid time yet.` and leaves the software clock alone, still running if it was valid. Either way, the answer also sets the battery clock's health (section 4.10 (p. 59)). The temperature is not used.
2. Network time (NTP), only while the S3 is on a network. The stored server defaults to `pool.ntp.org`, with `time.nist.gov` as the second. The S3 re-synchronises every 3 hours. Network time counts as fresh only if a synchronisation actually happened within the last 4 hours (the 3-hour interval plus an hour's grace). Network time disciplines the battery clock, not the other way round. While the handshake stands and network time is fresh, the S3 sends the network time to the battery clock at most once an hour (at once the first time). It also re-sets its software clock. The console prints `clock disciplined from NTP.` The write goes through to the battery clock so that the correction survives the next power cut. Without a network, which is most of the time, the battery clock is the authority.
3. By hand, the console key **w** (also from the portal's Console tab). It asks for local time as `YYYY-MM-DD HH:MM:SS`, converts it to UTC, refuses anything before 1600000000, and re-sets the software clock. Then it looks at the link:

   - if the handshake stands, it sends the time to the battery clock and prints `clock set, and pushed to the DS3231.`
   - if the A32 is not answering, it sends nothing and says so: `clock set on this board only - the audio board is not answering, so the battery clock was NOT updated. Set it again once the link is back.`

   Writing the battery clock clears its oscillator-stopped flag. Unreadable input changes nothing. The prompt gives up 40 s after the last key typed.

The battery clock always holds UTC. Local time is applied only for display. Keeping local time in a real-time clock makes the hour after a daylight-saving change ambiguous and the hour before it happen twice.

The time zone is a POSIX time-zone rule, stored with the network settings. It is applied once at start-up, and again whenever it is changed on the portal. The default is `EST5EDT,M3.2.0,M11.1.0` (eastern North America: UTC−5, daylight time from the second Sunday of March to the first Sunday of November). Builders in another zone: set your own rule on the portal (chapter 8 (p. 186)).

> Note The `W` prompt's zone is fixed. It sets the rule `EST5EDT,M3.2.0,M11.1.0` before converting, whatever the portal says, and leaves it in force afterwards. After a `W` on a set configured for another zone, the display shows eastern North American time until the zone is saved again on the portal or the S3 restarts. A note in the code there reads "BUILDERS, PERSONALISE THIS". It is left as it is on purpose.

Who reads which clock. The clock digits, the portal's `clock` field ("HH:MM" local, or "--:--") and the console `s` all read the software clock. The S3′s system clock, which network time sets, is used only as the source of network time.

4.10 Health reporting

Uptime comes from a 64-bit microsecond timer. The portal shows it as `up` ("Nd HH:MM:SS"); the console `s` shows `uptime N s`.

The reset reason. The console `s` prints the last reset reason: POWERON, EXT (the reset pin), SW (software restart), PANIC (crash), TASK WATCHDOG, INTERRUPT WATCHDOG, OTHER WATCHDOG, BROWNOUT, deep sleep, or unknown. The S3 is on native USB and does not reset when a terminal attaches, so its start-up banner is almost never seen. A reboot loop would be invisible unless the reason could be read afterwards. The reason is not a portal field; the portal reaches it through its Console tab.

The reset reason also decides where the needle is believed to be at start-up (chapter 5 (p. 73)):

Table 27. Health reporting

Reset reason	Needle position at start-up
SW, PANIC, interrupt watchdog, task watchdog, other watchdog (software-type)	Restored from a checksummed record in RAM that survives these resets, if the record is whole and sane; otherwise the low stop.
POWERON, BROWNOUT, EXT, deep sleep, unknown	Assumed at the low stop.

Console prefixes. The S3 marks its lines `[PASS]`, `[WARN]` and `[FAIL]` where they apply. The A32′s log lines arrive over the link and print as `[A32] ...`.

Portal health fields in this area (chapter 8 (p. 186) lists every field): `amp link` (the handshake stands); `hello astate` (a valid status report from the A32 is held; the page blanks audio readings when it is 0); `linkrx`, `linktx`, `linkcrc` (a receive count at 0 points at wiring or power; receive and checksum counts climbing together point at signalling); `linkver` (frames dropped for another protocol version); `rtc` (the battery clock's health, below); `slot` (the display's

worst slot error); `heap up clock`. While no valid status report is held, values from the A32 are zeroed, or set to a marker where 0 is a real value. A missing key would make the page read "-", and a stale one would make it read a lie.

The battery clock's health. The S3 keeps one number, published as the field `rtc`:

Table 28. Health reporting

`rtc`	Set when	Portal pill
0	Fine, or not asked yet. Set by every answer that carries a valid, plausible time.	none
1	The battery clock answered with no valid time: its oscillator stopped, or it was never set.	BATTERY CLOCK LOST ITS TIME - check its battery
2	The battery clock did not answer at all: a 60-second time request got no answer within 5 s while the handshake stands (the A32 sends nothing when its own read of the chip fails).	BATTERY CLOCK NOT ANSWERING

The value changes only on those events: a later valid time clears it to 0, and a link drop does not clear it. The pill changes nothing in how the clock behaves; it only makes the fault visible without a cable.

Version skew comes in two kinds, and they look different:

- Another protocol version. The link drops such frames and counts them. The link looks silent: there is no handshake, so the A32 sleeps and the radio and Bluetooth are silent (AUX never passes through the A32, so it still plays). The count and the pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both tell it apart from a broken cable (chapter 3 (p. 25), section 3.13 (p. 37) chapter 9 (p. 224)).
- The same protocol version, but a different status layout. A status report that passes its checksum but has the wrong length means the two builds disagree on it. The S3 drops its copy of the A32′s status and prints once: `the A32's STATE frame does not match this build. Flash BOTH MCUs. Audio is unaffected - only telemetry stops.` The sound is not affected.

A short A32 restart. A restart of the A32 can be shorter than the 2-second silence rule. It then shows as the A32′s millisecond counter going backwards in its status reports (with a guard against that counter's own 49.7-day wrap). The S3 prints `[WARN] the A32 restarted - handshaking again.` and shakes hands again, which re-reads the A32′s version, time and settings.

4.11 The watchdog

The task watchdog resets the chip when a task it watches has not reported in ("fed" it) for too long. The first lines of the start-up set it to 15 s with panic on. It watches two tasks:

- The main loop, which includes the start-up itself. The pinned Arduino core feeds the watchdog once before every pass, so any single pass that takes longer than 15 s resets the S3, and so does a start-up that takes longer than 15 s in total.

- The needle supervisor, **`needle`**. It joins the watchdog when it starts and feeds it at the top of every pass. Every branch of a pass sleeps 5 to 20 ms, so 15 s trips only on a real hang (chapter 5 (p. 73)).

It does not watch three things:

- the portal task: a flash write legitimately blocks it, and the portal's own upload watchdog already ends a stalled upload;
- the step emitter: it spins by design;
- core 0's idle task: the step emitter removes it (section 4.3 (p. 50)).

Core 1's idle task is not checked in this build's configuration. The image line reports whether the main loop's subscription worked: `watchdog on` or `watchdog OFF`.

> FOR FIRMWARE CHANGES
>
> Console prompts feed the watchdog. `y` (60 s after the last key typed), `W` (40 s after the last key typed) and `l` (60 s) wait inside the main loop for longer than 15 s. Each feeds the watchdog on every pass of its waiting loop. The `i` report's WiFi scan holds the loop for a few seconds without feeding it, well under 15 s. Any new routine that waits inside the main loop must do the same.

After a watchdog reset the console `s` shows `last reset : *** TASK WATCHDOG ***`. The needle treats it like a software restart and homes from its remembered position (chapter 5 (p. 73)). A watchdog reset of a program still on trial is a rollback (chapter 3 (p. 25), section 3.11 (p. 32)).

The pinned core also runs an interrupt watchdog of 300 ms on both cores, and a brownout detector; both reset the chip.

A failed link self-test does not stop the S3. It reports the failure and starts as usual; only the program's confirmation is withheld (chapter 3 (p. 25), section 3.11.7 (p. 35)).

4.12 Settings, console keys and constants

4.12.1 Stored settings

These live in the S3's stored settings (chapter 7 (p. 153)) and are edited on the portal. "Admin" means only the administrator account can change it.

Table 29. Stored settings

Setting	Default	Unit	Range	What it does	Where you change it
`brightOn`	255	level	0–255	Clock brightness, amplifier on. 255 is the maximum.	Display tab; console `-` `=`
`brightOff`	110	level	0–255	Clock brightness, amplifier off.	Display tab
`hour12`	0	on/off	0–1	12-hour clock, with no AM/PM indicator.	Display tab (admin)
`blankLeadZero`	1	on/off	0–1	Blank a leading zero in the hour.	Display tab (admin)
`dispDwellMs`	250	ms	0–5000	Hold before the clock brightness moves.	Display tab (admin)
`dispFadeMs`	1500	ms	0–10000	Eased travel between the two brightnesses.	Display tab (admin)
`showTuning`	0	choice	0 clock always, 1 tuning always, 2 tuning while tuning	What the digits show.	Display tab (admin); console `f`
`tuneHoldMs`	4000	ms	500–20000	How long the mode-2 readout stays after the knob stops.	Display tab (admin)
`panelTuning`	255	level	0–255	Panel lamps while tuning on RADIO.	Lights tab
`panelIdle`	128	level	0–255	Panel lamps on RADIO, knob idle.	Lights tab
`panelOther`	96	level	0–255	Panel lamps on AUX or Bluetooth.	Lights tab
`panelIdleMs`	5000	ms	0–60000	Idle time before the panel lamps dim to `panelIdle`.	Lights tab (admin)
`panelFadeMs`	900	ms	0–5000	Eased travel (0 is treated as 1).	Lights tab (admin)
`panelDwellMs`	250	ms	0–5000	Hold before the panel lamps move.	Lights tab (admin)
`sweepOn`	0	on/off	0–1	A decorative end-to-end needle sweep each time the amplifier comes on.	Needle tab (admin)

The printed dial, `dialLow` and `dialHigh`, which also bounds the readout, is in chapter 6 (p. 120). The blue lamp's levels on the Lights tab are A32 settings that the S3 only mirrors and forwards (chapter 10 (p. 256)). The time zone and network time server are network settings, stored apart (chapter 8 (p. 186)).

4.12.2 Console keys

Table 30. Console keys

Key	What it does
`8`	Show 8888 (all segments).
`b`	Blank the display.
`9`	Back to normal.
`-` / `=` (or `+`)	`brightOn` −15 / +15, written straight to the display (no fade), and saved.
`f`	Cycle `showTuning` 0 → 1 → 2, and save.
`W`	Set the clock by hand (local time, fixed zone; section 4.9 (p. 57)). Pushed to the battery clock only while the A32 answers, and it says which.
`Z`	Zero the step-jitter and display slot-error counters.
`s`	The status page: among the rest, the link's `wrong-version` count, the last reset reason and the image line. It ends with the command list.
`?`	The command list.
`l`	Live index monitor for up to 60 s; keeps the link alive and feeds the watchdog (chapter 5 (p. 73)).

Every key of both boards is in Appendix D (p. 332).

4.12.3 Constants

```
FOR FIRMWARE CHANGES
```

Table 31. Constants

Name	Value	Meaning
`TICK_US`	25 µs	The display interrupt's period.
`DEAD_TICKS`	33 (825 µs)	Dark time at the start of each slot; measured on this machine.
`SLOT_TICKS`	100 (2500 µs)	One digit's slot; four slots give 100 Hz.
`LIT_MAX`	67; effective maximum 66	Longest lit window; brightness is clamped to 66 so the turn-off lands inside the slot.
Display timer	hardware timer 0, prescaler 80	1 µs ticks.
`FONT`	0x3F 0x06 0x5B 0x4F 0x66 0x6D 0x7D 0x07 0x7F 0x6F	Digits 0–9, bit 0 = A … bit 6 = G.
Panel PWM	LEDC channel 1, 1000 Hz, 8 bits	—
Amplifier debounce	50 ms	—
Display decision	every 200 ms	What the digits show.
Amplifier state to the A32	every 2000 ms	Matches the A32's 2-second silence rule.
HELLO / PING	every 1000 ms	—
Time request	every 60 000 ms	Battery clock resync.
Battery clock answer wait	5000 ms	No answer this long after a 60-second request sets `rtc` to 2.
Network time write-through	every 3 600 000 ms	Hourly.
`NTP_FRESH_MS`	4 h	Network time is fresh within this of the last synchronisation.
Plausible epoch	1600000000	Earlier times are refused.
USB wait at start-up	3000 + 300 ms	Keeps the start-up report whole when a cable is attached (section 4.2 (p. 47), step 2).
`w` / `y` / `l` timeouts	40 s / 60 s / 60 s	Console prompts. `w` and `y` count from the last key typed.
`WDT_TIMEOUT_S`	15 s	Task watchdog on the main loop and the `needle` task, panic on.
`CONFIRM_AFTER_MS`	60 000 ms	No trial confirmation before this uptime (chapter 3 (p. 25)).
Confirmation retry	10 000 ms	One confirmation try every 10 s after that.

Framework settings this chapter relies on (the pinned core's configuration, not set by this project): the main loop on core 1 with an 8192-byte stack; a 1000 Hz FreeRTOS tick; WiFi pinned to core 0; the interrupt watchdog at 300 ms on both cores; the brownout detector on; print-and-reboot on panic; bootloader application rollback enabled; network time interval 3 h.

4.12.4 Peripherals the S3 firmware takes

FOR FIRMWARE CHANGES

Table 32. Peripherals the S3 firmware takes

Peripheral	User
Hardware timer 0	The display interrupt.
LEDC channel 0	Unused on purpose: the A32 uses its channel 0 for the blue lamp. Also, in this core, channels 0 and 1 share LEDC timer 0: anything put on channel 0 at another frequency would change the panel lamps'.
LEDC channel 1 (timer 0)	Panel lamps.
LEDC channels 2–5 (timers 1–2)	The needle motor's coils (chapter 5 (p. 73)).
I2C bus 0 (`Wire`)	The angle sensor (chapter 5 (p. 73)).
I2C bus 1 (`Wire1`)	The RDA5807M (chapter 6 (p. 120)).
`Serial2`	The link to the A32 (chapter 9 (p. 224)).
RAM not cleared at reset	The needle's position record (chapter 5 (p. 73)).

Which pin the firmware drives or reads for each of these is in Appendix I (p. 358) what each pin is wired to is in the Hardware Bible, chapter 4.

4.13 When it goes wrong

Table 33. When it goes wrong

What you see	What happened	What the S3 does	What you do
`[WARN] A32 silent. Clock and needle keep running.`	Nothing valid from the A32 for more than 2 s.	Drops the handshake; takes the source as AUX (needle parks, panel lamps to `panelOther`, mode-2 readout off); the software clock keeps counting; no battery-clock resync and no network-time write-through (both need the handshake); forgets its copy of the A32's settings, so audio-board edits get HTTP 409 "the audio board is not answering - its settings cannot be changed now" and a settings download writes `n/a` for them; `W` sets this board only, and says so; HELLO every second.	Nothing: it heals when the A32 answers, which also fetches its settings again (chapter 1 (p. 3), section 1.8 (p. 9)).
`[WARN] the A32 restarted – handshaking again.`	The A32 restarted in under 2 s.	Shakes hands again; re-reads version, time and settings.	Nothing.
`wrong-version N` on `S`, pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both	The boards run different protocol versions.	The link looks silent: no handshake, so the A32 sleeps and the radio and Bluetooth are silent (AUX still plays). Once the A32 has confirmed its program, this survives a power cycle.	Update the S3 first, over its own WiFi, to the A32's protocol version; then the A32 through it if needed (chapter 3 (p. 25), section 3.13 (p. 37)).
`the A32's STATE frame does not match this build. Flash BOTH MCUs. Audio is unaffected – only telemetry stops.`	Incompatible builds of the same protocol version.	Drops the A32's status; telemetry only. The sound is not affected.	Update both boards.
`[WARN] the DS3231 has no valid time yet.` every minute, pill BATTERY CLOCK	The battery clock's oscillator stopped, or it was never set, and there is no network time.	Keeps its software clock; digits blank if it never had a valid time.	Join a network (it heals by itself), or set the time with `W` either write clears

What you see	What happened	What the S3 does	What you do
LOST ITS TIME - check its battery			the flag. Then check the module's battery.
Pill BATTERY CLOCK NOT ANSWERING	The battery clock does not answer; the A32 logs its own read failure.	Keeps its software clock.	Hardware: Hardware Bible, chapter 4.
Clock a little off	Drift of the software clock.	Re-set from the battery clock every 60 s.	Nothing.
—	The amplifier-sense reading bounced.	50 ms debounce.	Nothing.
Clock blank, lamps dark, needle still	An S3 update is writing.	No saves; amplifier changes ignored; wakes up again if the upload ends without a restart.	Wait.
`s` shows PANIC	A crash.	Reboots; the needle position is restored from RAM.	Nothing; if it repeats, it is a defect to find.
`s` shows INTERRUPT WATCHDOG	The 300 ms interrupt watchdog fired.	Reboots; the needle position is restored.	Nothing.
`s` shows `*** TASK WATCHDOG ***`	The main loop or the needle supervisor hung for 15 s.	Reboots; the needle position is restored. A program on trial rolls back.	Nothing.
The portal is dead, but the radio, clock and needle work	The portal task or the WiFi stack hung. Nothing watches them.	Nothing. Never seen on the radio.	Unplug the set (a USB console still works meanwhile).
Trouble with a new S3 program while it is on trial	See chapter 3 (p. 25), section 3.15 (p. 39).	—	—
A new S3 program ran healthily for a minute, confirmed itself, but is wrong	It earned confirmation, so it is permanent.	Nothing.	Upload a good program through the portal.
`s` shows BROWNOUT	The supply dipped.	Reset; the needle is assumed at the low stop.	Nothing.
The A32 goes quiet while a console prompt waits	`W` (40 s) or `y` (60 s) holds the main loop: no amplifier state, no PING.	The A32 treats the silence as "amplifier down" after 2 s. The prompts feed the watchdog, so they never trip it.	It ends when the prompt ends. Accepted for `y`.
The clock at the wrong brightness after – or = with the amplifier off	Those keys write `brightOn` straight to the display.	Shows `brightOn` until the next retarget.	Switch the amplifier, or change any setting.
After `W`, the old time comes back	`W` was typed while the A32 was not answering.	Set the S3 only, and said so; the next handshake brought the battery clock's time back.	Type `W` again once the link is back.

4.14 Design choices

- Two boards, and the S3 owns everything visible. The A32 must carry the Bluetooth audio (chapter 2 (p. 14), section 2.3 (p. 16)). Everything a person sees move or light up went to the S3. That also keeps the priority-19 step emitter away from the audio timing.
- The display interrupt, the needle supervisor and the main loop on core 1; WiFi, the portal and the step emitter on core 0. Settled by a timing test. WiFi with four concurrent multi-megabyte HTTP streams was imperceptible on the display, and so was a moving stepper on top.

Deliberate flash-bus thrashing was the only load that visibly degraded it, and it was still bearable. The same test set three rules the firmware keeps: debounce settings writes, darken the machine during an upload, never read or write flash in a loop.

- Portal requests may starve while the needle moves. The step emitter's timing wins over web responsiveness.
- The display interrupt lives in IRAM and only writes registers. Flash traffic is the one load that disturbs the display, and IRAM code does not wait for flash.
- 825 µs of dead time in firmware, instead of a hardware change. Without it the display ghosts, and the resulting brightness was judged fine by eye. Worth revisiting only if the display is ever judged too dim.
- Brightness shortens the lit window, never the slot. A constant frame rate means dimming cannot flicker.
- An unconditional blank at the end of every slot, and the lit window clamped to 66 ticks. At brightness 255 the turn-off once fell on tick 100, which never comes because the slot rolls over at 99. The segments stayed lit and the display read 88:88. The clamp fixes the arithmetic; the blank means no future arithmetic can bring the bug back.
- Two clock brightnesses, both adjustable. The colon is lit independently of the firmware and cannot dim with the digits, which is why `brightOff` defaults to a modest 110. The colon's brightness is not a firmware matter.
- Eased changes (a hold, then smoothstep) for both clock and panel lamps. A linear ramp reads as a setting being changed, and an instant step as a glitch. The intent is lamps that let go like a capacitor emptying, and a clock that does not snap to standby.
- Panel rules: dark with the amplifier off, full while tuning on RADIO, an idle level after 5 s, its own level on AUX and Bluetooth, all adjustable. A panel-lamp fault pulse was decided against.
- Three readout modes; mode 2 only while the radio is live and snapped to the channel grid; mode 1 neither gated nor snapped. The readout should appear under the hand while tuning, and only for a source that is playing. But the instrument must stay available with the amplifier off. Turning up from 91.3 should go to 91.5, not 91.4.
- Time lives on the A32; the S3 keeps a software clock re-set from the battery clock; network time writes through hourly; the battery clock holds UTC. The clock chip is on the A32′s side and the display on the S3′s. The software clock keeps the display smooth whatever the link does. The write-through makes a correction survive a power cut.
- A battery clock whose oscillator stopped is not trusted, and the portal says so. A stopped clock can hold a wrong but plausible date. A stopped oscillator most likely means a failing battery or module.
- **`w`** claims the push to the battery clock only when the A32 answers. Otherwise the next handshake quietly brought the old time back.

- Network time counts as fresh only within 4 h of an actual synchronisation. The old test stayed true for ever after one synchronisation, so a free-running clock was written to the battery clock hourly.
- The `w` prompt's zone stays fixed, with a note for builders. A builder elsewhere personalises it for their own device.
- Uptime from the 64-bit microsecond timer. The 32-bit millisecond count rolls over every 49.7 days.
- The status page and the command list on demand, not only at start-up. On native USB nobody sees the start-up banner.
- The `y` prompt may hold the main loop until 60 s after the last key typed. Realistically it does no meaningful harm.
- A task watchdog on the main loop and the needle supervisor only. A flash write legitimately blocks the portal task, and the step emitter spins by design.

4.15 Tried and rejected

FOR FIRMWARE CHANGES

- "The clock never dims": one brightness, so the always-lit colon matched the digits. A dimmer clock with the amplifier off was wanted.
- Shortening the dead time with a hardware change: the firmware dead time gave an acceptable brightness with no rework of a built board. Kept as the answer if the display is ever too dim.
- Longer digit slots to win back lit time: 4 ms per digit gives a 62.5 Hz frame, judged marginal for flicker. Not built.
- Letting the firmware dim the colon by moving it to a free driver channel: closed as a non-issue.
- "Brightness 240 is the maximum": a workaround for the tick-100 bug, not a limit. Removed with the fix.
- Forcing the time zone on every redraw: it would have overridden the zone set on the portal. Now set once.
- Uptime from the millisecond counter: rolled over every 49.7 days.
- "Network time is fresh if the clock reads past 2020": true for ever after one synchronisation.
- Trusting the battery clock on plausibility alone: a stopped clock kept a wrong but plausible date and was adopted.
- A start-up-only help list: nobody sees the banner on native USB, and the list drifted from the keys. One list now, printed by `s` and `?`.
- Clearing the mode-2 readout hold when the radio is not live: a selector glitch or a link hiccup wiped a readout being read.
- Snapping the readout with an "even tenth goes up" rule: it showed 91.5 for a true 91.36. Replaced by snapping from the unrounded value.
- Halting the S3 for ever on a failed link self-test: a board that halts before WiFi can only be fixed by USB. Replaced by carrying on unconfirmed.

- **w** claiming the push to the battery clock whatever the link: with the A32 silent the claim was false.
- Looking silent on a protocol mismatch: a half-updated pair read `SILENT rx 0`, exactly like a broken cable. Replaced by counting and showing the dropped frames.

4.16 Known limits

- About 3.3 s of start-up delay without a USB host, waiting for the console port (chapter 12 (p. 290)).
- The clock is blank for a moment after every S3 start-up, until the A32's first time answer, and stays blank if neither the battery clock nor network time has a valid time.
- Network time cannot reach the display while the link is down. The write-through needs the handshake, and nothing else re-sets the software clock from network time (chapter 12 (p. 290)).
- The software clock is exact only between re-settings. It is re-set at a whole second, so it can lag the battery clock by up to about a second. The display shows minutes, so this is not visible. Only an A32 silence longer than 49.7 days would break its arithmetic.
- **w** uses a fixed zone and leaves it in force (section 4.9 (p. 57)).
- A **w** typed while the A32 is silent sets this board only. If the battery clock holds a valid time, the next handshake puts that time back on the display.
- 12-hour mode has no AM/PM indicator. The font has digits only, so the display cannot show letters or error codes.
- The readout below 100 MHz shows a leading zero (`0879`).
- Readout mode 1 does not check that the angle sensor answers; with a dead sensor the digits show a frequency from the last count.
- **-** and **=** bypass the fade: the clock can sit at the wrong level, or step back briefly before easing.
- Portal requests can be starved while the needle moves.
- The **y** prompt holds the main loop until 60 s after the last key typed, so the A32 goes quiet meanwhile. It also echoes what is typed, the WiFi password included, into the console the portal reads (chapter 8 (p. 186)).
- With the link down, the panel lamps treat the source as AUX, so with the amplifier on they sit at `panelOther`.
- The battery clock's temperature arrives with every time answer and is not used.
- A settings change made in the trial minute of a new S3 program is lost if the trial ends in a reset (chapter 3 (p. 25), section 3.11.5 (p. 34)).
- Exercised on the radio: a hung S3 program reset by the task watchdog and rolled back; the `rtc` and `linkver` fields at 0 on a healthy pair. Never exercised on the radio: a hang of the `needle` task; a hung portal task; the battery-clock pills in a real fault; the protocol-version pill; the HTTP 409 on an audio-board edit while the A32 is silent.

4.17 Changing it safely

FOR FIRMWARE CHANGES

What must stay true

1. The display interrupt (**`onTick()`** in **`src/s3/display.cpp`**) stays **`IRAM_ATTR`**, writes registers only, touches no flash and calls nothing but **`esp_timer_get_time()`**. Keep `Display::begin()` in `setup()`, so the timer interrupt is allocated on core 1.
2. Never shorten **`DEAD_TICKS`**. Keep the turn-off strictly inside the slot. Keep the unconditional blank at tick 99. Only one digit select may ever be set.
3. All seven segment pins stay below GPIO 32. The interrupt clears segments through the low output register only (`segAllMask`, built with `1UL << pin`). Digit pins may be anywhere; `begin()` splits them between the low and high output registers (`GPIO_OUT_W1TS/W1TC`, `GPIO_OUT1_W1TS/W1TC`). Three of the four digit pins are above GPIO 32 today.
4. **`loop()`** must not block. A routine that must wait inside it keeps sending `MSG_PING` (as `limitMonitor()` does), or the A32 decides the amplifier is down. It also returns within 15 s or calls `wdtFeed()` while it waits, as the `y`, `W` and `l` routines do, or the watchdog resets the S3. `portalStateJson()` and anything the portal task calls must not block or write flash.
5. Core 1 below priority 2 belongs to **`loop()`**, which never yields. A new task at priority 0 on core 1 will never run. Put periodic work in `loop()`, or give the task priority 2 or more and make it yield. Tasks on core 0 must yield at least `vTaskDelay(1)`: the step emitter spins at 19.
6. No settings writes while the needle moves, a calibration runs or an upload is active, and none while the program is on trial. Change `cfg`, call `settingsTouch()`, and let `settingsFlush()` write (chapter 7 (p. 153)). Never add a write path that bypasses `settingsWrite()`. The reason is measured: a flash write disables the flash cache, which stalls all code not in IRAM, the step emitter included. During the first settings migration the step jitter reached 383 µs and the display's worst slot error 1257 µs; with writes kept away from motion they fell to 49 µs and 8 µs. The display interrupt survives only because it lives in IRAM. A new stored field is appended, never inserted, with `SETTINGS_VERSION` bumped and a migration added (chapter 7 (p. 153)).
7. Push settings to modules only through **`applySettings()`**. It also writes back what a module accepted (the needle's soft limits, for example), so `cfg` always holds what is running.
8. LEDC: channel 1 is the panel lamps, on LEDC timer 0; channels 2–5 are the needle motor. In the pinned core a channel's timer is

`(channel / 2) % 4`, so channels 0 and 1 share timer 0. Do not use channel 0 on the S3 at another frequency: it would change the panel lamps'. Hardware timer 0 belongs to the display.

9. One definition of "radio live" (`radioLive()`) and one definition of the frequency (`Needle::tuneFreq10f()` / `tuneFreq10()`). Do not rebuild either locally; six copies of the mapping once existed.
10. The battery clock stores UTC. Local time is applied only for display, with the time-zone rule from the network module.
11. A pin moves in **`include/pins.h`** and nowhere else, and what it is wired to is the Hardware Bible's business.
12. The toolchain is pinned. Identify a board before flashing it over USB, and never let the uploader pick a port by itself (chapter 3 (p. 25), section 3.1 (p. 25)).

The watchdog subscriptions (`esp_task_wdt_init(WDT_TIMEOUT_S, true)` and `enableLoopWDT()` first in `setup()` `esp_task_wdt_add(NULL)` in `needleTask()` the removal of core 0's idle task in `stepTask()`) and the trial rules of chapter 3 (p. 25), section 3.19 (p. 44) must hold together.

Traps that have bitten this project

- The turn-off on tick 100 that never happened: the 88:88 display.
- A busy-spinning task on a core whose idle task is watched: a boot loop that looked like a power problem (the step emitter).
- The native-USB console does not reset on attach, so anything printed only at start-up is never seen.
- A serial tool that toggles DTR/RTS resets the board: "uptime 1 s on every reading" was the reading tool resetting it.
- A word typed in the portal's console used to run as a string of one-key commands. Now one key per send, unless a prompt wants a whole line (chapter 8 (p. 186)).
- `litTicks` starts at `LIT_MAX` (67) in `display.cpp`, one tick past the clamp, until the first `setBrightness()` at start-up step 13. It is harmless only because the frame is blank then and tick 99 blanks anyway. Do not show digits before a brightness has been set.

Finding your way in **`src/s3/main.cpp`** (about 3400 lines, in file order)

Table 34. Changing it safely

Section (main functions)	What it does	Chapter
Header, includes, `FW_VERSION` / `FW_COMMIT` fallbacks	—	3
`SETTINGS_MAGIC`, `SETTINGS_VERSION`, `SettingsV1`, `SettingsV2`, `Settings`	The one stored settings structure and its history.	7
Globals: `cfg`, `gTouchGen` / `gSavedGen`, `gLink`, `a32` (the A32's status), `a32cfg` (its settings), `haveState`, `peerHello`, `ampOn`, `ampKnown`, `epochAtSync`, `millisAtSync`, `timeValid`	The S3's shared state.	4, 7, 9
`dispTest`, `brightnessTarget()`, `brightnessNow()`, `updateBrightness()`	Eased clock brightness.	4
`refuse()`, `captureLimit()`, `purgeCorruptLimits()`	Refusal flag; soft-limit capture.	5, 7
`setTuneModel()`, `fitTuneCurve()`, `solveLS()`, `purgeCorruptSpurs()` and helpers	The tuning curve.	6
`applySettings()`	Pushes `cfg` into every module.	4, all
`settingsLoad()`, `settingsDump()`, `settingsTouch()`, `settingsWrite()`, `writeSafe()`, `settingsFlush()` and siblings	Load, migrate and save the settings.	7
`nowEpoch()`	The software clock.	4
`otaWaitAck()`, `otaSendFrame()`, `a32OtaBegin/Chunk/End/SetError/Abort()`	The A32 update relay.	3, 9
`#include "settings_table.h"`	The settings table, actions, settings-file round trip.	7, 8
`onMessage()`	Every frame from the A32: HELLO_ACK, STATE, TIME, CFG, OTA_STATUS, LOG.	4, 9
`srcName()`, `btName()`, `portalRdaJson()`, `portalStateJson()`	The JSON the portal polls.	8
`verifyRollbackLater()`, `s3ImageOnTrial()`, `wdtFeed()`, `imageReport()`, `confirmTick()`	Trial, confirmation, watchdog.	3, 4
`otaQuiet`, `otaResume`, `portalNeedleResume()`, `portalOtaQuiet()`	Dark and still during an upload; resume if it ends without a restart.	4, 8
`radioLive()`, `applyNeedleMode()`	"Radio live"; follow the knob or park.	4, 5
`sampleStart()`, `sampleStore()`, `sampleTick()`, `autoSampleTick()`, `needleRecoverTick()`, `idleIndexTick()`	The RDA sampler and the re-index triggers.	5, 6
`updateDisplay()`	What the digits show.	4
`drainLine()`, `setWifiInteractive()` (`y`), `setClockInteractive()` (`w`), `limitMonitor()` (`l`), `commandList()`, `status()` (`s`)	Console prompts and the status page.	4, 8
`setup()`, `loop()`	Start-up and the main loop, with the console key dispatcher at its end.	4

The display's interface is in `src/s3/display.h` (`begin()`, `setBrightness()` / `brightness()`, `showDigits()`, `showTime()`, `showNumber()`, `blank()`, `worstSlotErrorUs()`, `resetSlotStats()`); the panel lamps' in `src/s3/panel.h` (`begin()`, `update()`, the setters that only `applySettings()` calls, and `level()` / `target()`).

How to test a change, without instruments

- `8`, `b`, `9`: all segments, blank, normal.
- Brightness 255 must not show 88:88 (the tick-100 regression test).

- `Z`, then watch `slot` on the portal or in `s` under load: WiFi traffic, a sweep with `r`, a settings save. After the settings-save guard the worst slot error measured 8 µs; the display timing test treated anything under 100 µs as invisible. `Z` exists so a clean measurement can be taken after a one-off event such as a start-up save. The bench tool behind that test no longer builds; the `slot` figure is the instrument now.
- Switch the amplifier: the clock should hold 250 ms, then ease for 1.5 s; the panel lamps should follow their table.
- `f` to cycle the readout modes; turn the knob with the amplifier on and the source on RADIO for mode 2.
- `W` to set the clock; check the time in `s` and on the portal.
- `s` for everything at once: amplifier, source, needle, panel level and target, display brightness and slot error, time, link counters, last reset and uptime.
- After an S3 update, the trial checks of chapter 3 (p. 25), section 3.19 (p. 44). A change to the watchdog deserves the rollback tests there: a portal reboot during the trial, and a deliberately hanging program, never committed.

5. The dial needle

The needle on the glass dial is not pulled by a dial cord. A small stepper motor moves it, and the main board (the ESP32-S3) decides where it goes. Nothing mechanical joins the tuning knob to the needle: the knob turns the tube radio's tuning capacitor, a magnetic angle sensor reads that capacitor's shaft, and the firmware turns the shaft angle into a frequency and the frequency into a needle position. Chapter 2, section 2.2 (p. 15), shows the whole tie, and chapter 6 (p. 120) explains the tuning half of that chain. This chapter is the needle half.

What matters most is repeatability: the same knob position must always put the needle in the same place on the glass.

Specific to this build — adapt. The needle, its stepper motor, its gearing, its index switch and its travel are this cabinet's mechanism; yours will differ. The methods carry over; the numbers do not. The numbers that belong to this machine are gathered in section 5.18 (p. 103).

5.1 What you see

1. The radio is plugged in. The main board starts, and the needle finds its reference point ("homes"). It then makes one flourish: a run to the low end of the dial, a run to the high end, then straight to where it belongs — the station if the amplifier is on and the source is RADIO, otherwise the

low end. From plug-in to “homed and following the knob” took about 12 seconds when it was timed, on an earlier firmware.
2. You switch on with the source on RADIO. The needle leaves the low end and swings to the station like a meter movement: a small overshoot and a short settle. If the `sweepOn` setting is on, it first makes one full end-to-end sweep.
3. You turn the tuning knob. The needle follows continuously. It ignores changes smaller than two motor half-steps, so it does not chase sensor noise.
4. You switch the amplifier off, or leave RADIO. After one second without movement, the needle falls to the low end: a fast start and a long, slowing finish, like a drive winding down.
5. Two minutes after the amplifier goes off. The needle quietly re-checks its reference point and returns to the low end, so the next listening session starts from a checked zero.
6. You tune past the printed scale. The needle stays at that end of the scale.
7. A firmware update reaches the main board. The needle stops where it is and the clock display goes dark. The radio keeps playing, because the sound runs on the other board. After the restart the needle remembers where it was and homes from there.

> NOTICE Unplug only with the needle parked. The needle has no end switches. If the radio loses power while the needle is above the index (near 102.5 MHz), the next start may first search the wrong way, toward the high mechanical stop. The search has a step limit, but nothing physical stops it. Switch the amplifier off and wait for the needle to reach the low end before you pull the plug.

There are no end switches. One magnetic switch, the index, sits part-way along the travel and is the only position reference. Everything else is kept by counting motor steps, and the count is corrected every time the needle passes the index. Three firmware mechanisms keep the needle off the mechanical stops in place of switches: soft limits checked before every single step, a step limit on every phase of homing, and a known starting belief for the homing search (section 5.14 (p. 95)).

Where the motor, the index switch and the angle sensor sit and how they are wired is in the Hardware Bible, chapter 3 (p. 25), section 3.3, and chapter 8 (p. 186). This chapter states the few hardware facts it needs in place.

5.2 The chain, layers and tasks

The chain. Chapter 2, section 2.5 (p. 20), draws the whole tuning chain. The needle’s half of it, from the shaft angle to the motor:

1. The angle sensor (an AS5600) gives 0 to 4095 per turn of the capacitor’s shaft. The supervisor reads it every 20 ms and keeps a running multi-turn count (chapter 6 (p. 120)).

2. For the needle only, that count is smoothed by a low-pass filter (section 5.10 (p. 87)).
3. The tuning curve turns the count into a frequency (chapter 6 (p. 120)).
4. The printed face turns the frequency into a needle position: the frequencies printed at the two ends of the glass (`dialLow`, `dialHigh`) are mapped onto the needle's two soft limits (`posMin`, `posMax`).
5. The needle supervisor plans a move: which motion law, which microstep (section 5.6 (p. 80)).
6. The step emitter emits one step at a time and checks the soft limits before each (section 5.5 (p. 79)).
7. The drive layer turns the position into four PWM duties for the motor's Darlington driver (section 5.4 (p. 78)).

A bring-up tool is a jog or a held coil test (section 5.16 (p. 98)): something that drives the motor directly, for setting up a mechanism.

FOR FIRMWARE CHANGES

The needle reads the tuning curve through two calls in `src/s3/needle.cpp`, `tuneFreq10f()` and `tuneFreq10Raw()` everything about the curve itself is chapter 6 (p. 120). The pins the firmware drives are the defines `S3_STEPPER_I1` to `S3_STEPPER_I4`, `S3_INDEX_HALL`, `S3_AS5600_SDA` and `S3_AS5600_SCL` (Appendix I (p. 358)).

The layers.

Table 35. The chain, layers and tasks

Layer	Where	Job
Policy	`src/s3/main.cpp`	When to home, track, park or sweep; the recovery and idle-check timers; capturing the soft limits; collecting the index calibration's result; saving the shaft count; pausing and resuming for an update; the console keys.
Needle	`src/s3/needle.cpp`, `needle.h`	The two needle tasks: homing, tracking, parking, sweeps, reading the index, the index calibration, the AS5600 reads, the memory kept across a restart.
Drive	`src/s3/drive.cpp`, `drive.h`	Turns a position into four PWM duties. Nothing above this layer knows which driver chip is used.
Portal actions	`src/s3/settings_table.h`	The Needle tab's buttons.
Update hooks	`src/s3/portal.cpp`	Stops the needle for an upload; resumes it if the upload does not end in a restart.

The tasks. The needle runs on two tasks, both started by `Needle::begin()`. Chapter 2, section 2.4 (p. 18), has them in the table of every task; what matters here:

- The step emitter, **`nstep`**, on core 0 at priority 19. It runs a 1 ms control tick. While the needle moves it spins between steps; it yields one tick when idle, or when the next event is more than 2 ms away.

- The supervisor, **`needle`**, on core 1 at priority 4. One pass every 5 ms (every 20 ms while a bring-up tool holds the needle). It reads the AS5600 angle every 20 ms and its status every 500 ms.

Two other tasks call into the needle: the main loop, which runs the policy and the console, and the portal task, which runs the Needle tab's buttons.

Why the emitter is on core 0. Core 1 carries the clock display's 25 µs interrupt and the main loop, which services the link to the audio board. The emitter spins between steps, so on core 1 it would starve the main loop for the length of every sweep. On core 0 it shares time with WiFi and the portal instead, so the portal can be slow while the needle moves.

Why core 0's idle task is not watched. The emitter starves core 0's idle task on purpose while it moves. With the idle task still watched, the task watchdog reset the chip in a boot loop that looked like a power problem. So `stepTask()` removes core 0's idle task from the task watchdog. The cost: nothing catches a task on core 0 that hangs for good. If that ever matters, watch the emitter task itself; do not put the idle task back.

The supervisor is watched. The task watchdog watches the supervisor with a 15-second timeout (chapter 4 (p. 46)). It feeds the watchdog at the top of every pass, and every branch of a pass sleeps 5 to 20 ms, so the watchdog trips only when the supervisor has truly stopped. A stopped supervisor would leave the needle deaf to every request; the watchdog restarts the main board instead, and the needle's memory carries its place across the restart (section 5.14 (p. 95)). The step emitter is not watched, because it spins by design; neither is the portal task.

5.3 Units and the position frame

Figure 10. The dial's travel, measured in motor half-steps



travel 1869 half-steps (hs) · + is toward high FM

The unit is the half-step, written hs: one half-step of the stepper motor. Every geometry number in this chapter — soft limits, the index band, step limits, drift — is in half-steps, counted from the homed zero. On this machine one half-step moves the motor's output shaft by 0.088 degree. Speeds are in half-steps per second (hs/s) and accelerations in hs/s².

Table 36. Units and the position frame

On this dial	Value
Travel between the two mechanical stops	1869 hs (about 164 degrees)
Index position	1195 hs above the low stop, 674 hs below the high stop — near 102.5 MHz on the glass
Zero	where the index switches on as the needle moves up the dial
Direction	+ is toward high FM
Soft limits (the needle's working ends)	−1156 hs at the low end, +414 hs at the high end
Printed scale	87.9 MHz at the low soft limit, 107.9 MHz at the high one

The soft limits are where the needle is allowed to go. They sit inside the mechanical stops, and the firmware checks them before every single step. The low soft limit is also the park position, where the needle rests when the radio is off.

Because the index is near 102.5 MHz, an evening spent below that frequency never crosses it. That is why the needle also re-checks the index by itself, two minutes after the amplifier goes off.

Two units you meet on the portal and in the settings:

- Frequencies are in tenths of a MHz inside the firmware: 1079 means 107.9 MHz.
- The published position (the portal's `pos`) is in whole half-steps. A movement smaller than one half-step does not show in it.

FOR FIRMWARE CHANGES

How it works inside.

- Fine units. Internally a position is counted in fine units: 256 fine units make one half-step, and 2048 make one electrical revolution of the motor, which is 8 half-steps. The low 11 bits of a fine position are therefore the motor's electrical angle, so the step path needs only a mask and table lookups, never a division. Settings in hs/s are converted to fine units per second.
- Position and target (`gPos`, `gTarget`) are 32-bit signed fine positions. They are 32-bit on purpose: the processor has no atomic 64-bit load, and these are read on one core while written on the other. The whole travel, 1869 hs, is about 478 000 fine units (positions from about −306 000 to about +173 000), far inside 32 bits. Arithmetic that multiplies a position widens it to 64 bits where it happens.
- Zero is the index switch's forward ON edge: the point where it switches on when the needle approaches it moving in the + direction.
- 1. is toward high FM.

> On this machine, advancing the motor's electrical angle moves the needle toward low FM. That inversion lives in one place only, `Drive::setInvert(true)`, called from `Needle::begin()`. It flips the electrical angle, not the position, so everything above the drive layer can say "position increases toward high FM" and mean it.

5.4 Driving the motor

The motor is a small unipolar stepper: each of its two phases is a winding split in two halves, and each half conducts one way only. Four inputs of a Darlington driver array switch the four half-windings. The drive layer, `src/s3/drive.cpp`, turns a fine position into four PWM duty cycles, one per input.

Why PWM: microstepping. Microstepping places the rotor between the motor's natural step positions by giving the coils partial currents. It is the only way to microstep a unipolar motor through a plain Darlington array. Plain half-stepping chatters visibly at the low speeds a needle uses: its two-coils-on positions pull 41 % harder than its one-coil-on positions. At high speed the advantage vanishes: the winding cannot follow the sine fast enough, so the current falls back into half-steps while the emitter pays many times the step rate. The needle never needs that speed.

The PWM. Four PWM channels of the chip's LED controller (LEDC), channels 2, 3, 4 and 5, at 20 kHz, above hearing.

> FOR FIRMWARE CHANGES
>
> `Drive::begin()` asks for 11-bit resolution and steps down one bit at a time, to 8 bits, until the chip accepts. If nothing is accepted it prints `[FAIL] LEDC would not accept 20 kHz at ANY resolution`, and the four outputs do nothing. What was achieved is kept and shown by the console `s`, which prints `*** LEDC REFUSED ***` when nothing was accepted. 11 bits at 20 kHz only just fits the 80 MHz clock the PWM runs from. The negotiation exists because the chip once refused 11 bits, and said so only by its setup call returning 0: the emitter stepped faithfully while the motor never moved. Check what a configuration call returns. Channel 1 is the main board's panel lighting and channel 0 the audio board's Bluetooth LED; the numbers are kept distinct across both firmwares.

The waveform. A 2048-entry table holds max(sin, 0), scaled 0 to 4096. The four outputs are:

- IN1 = max(cos, 0)
- IN2 = max(sin, 0)
- IN3 = max(−cos, 0)
- IN4 = max(−sin, 0)

A half-winding conducts one way only, so a signed sine wave becomes two rectified ones on opposite half-windings. One electrical revolution is 8 half-steps; at each half-step position the duties are:

Table 37. Driving the motor

Electrical angle	Half-step	IN1	IN2	IN3	IN4
0°	0	1	0	0	0
45°	1	0.707	0.707	0	0
90°	2	0	1	0	0
135°	3	0	0.707	0.707	0
180°	4	0	0	1	0
225°	5	0	0	0.707	0.707
270°	6	0	0	0	1
315°	7	0.707	0	0	0.707

(1 is full duty, 4096 in the table.) At 45 degrees IN1 and IN2 each sit at 0.707: the current vector keeps a constant size, which is why microstepping is smoother than half-stepping. Between these rows the microsteps follow the same sine and cosine. IN1 with IN3 form one phase and IN2 with IN4 the other, in the turning order IN1, IN2, IN3, IN4; that pairing is a physical value of this machine (section 5.18 (p. 103)). The wiring of the four inputs to the driver and the motor is in the Hardware Bible, chapter 8, and the cable in chapter 10 (p. 256).

> FOR FIRMWARE CHANGES
>
> Modes. `DRV_WAVE`, `DRV_FULL`, `DRV_HALF` and `DRV_MICRO`. Wave, full and half all read one 8-entry half-step table; the mode picks the step size and where the steps sit (full steps sit half a step over). In `DRV_MICRO` the divisor is rounded down to a power of two, at most 64, so every microstep is a whole number of fine units. `Drive::snap()` puts a position on the current mode's lattice of steps.
>
> Writing and releasing. `Drive::apply()` writes a channel only when its duty changed. `Drive::release()` sets all four duties to 0 and leaves the coils slack.
>
> Duty. `Needle::begin()` sets full duty (`Drive::setDutyPct(100)`). The motor is a 5 V part on a 5 V supply (Hardware Bible, chapter 8), so full duty is safe. The duty stays a parameter because it is the only thing that would stand between a 5 V motor and a higher supply.

5.5 The step emitter

The step emitter is the part of the firmware that moves the motor one step at a time and checks the soft limits before each step (section 5.2 (p. 74)).

FOR FIRMWARE CHANGES

`stepTask()` in `src/s3/needle.cpp` is the only code that emits a step in normal operation. Each loop:

1. Fold in any pending correction. The supervisor never writes the position itself. It leaves a correction in one shared word, `gCorrectHs` the emitter takes it with an atomic exchange (reading and zeroing in one step) and adds it, times 256, to the position.
2. Stand aside for a bring-up tool. If one holds the drive, sleep one tick and skip the rest.
3. Every 1 ms control tick: if a stop or abort is pending, the speed becomes 0 and the move is done. Otherwise, if a constant ("manual") speed is set, use it. Otherwise run the motion law (section 5.6 (p. 80)). When a move has been finished for 400 ms, release the coils: holding costs current and heat, and the needle has no load to hold. This mechanism stays where it is with the coils unpowered (section 5.18 (p. 103)).
4. While moving: the time to the next step is one step divided by the speed, clamped between 20 µs (a ceiling of 50 000 steps per second) and 1 second. The deadline is pulled in on every pass as the motion accelerates. Without that, a move leaving rest slowly schedules its second step from its slowest speed and waits: on the bench rig this gave one step, then a 25-second wait, while reporting 2000 hs/s.
5. At the deadline: if the soft limits allow a step in that direction, the position moves one step, the coils are written, and the step counter goes up. If not, the needle is at a soft limit and the move stops there. The worst lateness is kept (the portal's `jit`). A late emitter never bursts to catch up; it re-synchronises.

The count cannot drift. Every step moves the position by exactly one whole step unit. There is no fractional accumulator anywhere in the step path, so a million steps out and a million back land on the number they started from. Any difference between the count and the needle is therefore the motor, never the arithmetic.

FOR FIRMWARE CHANGES

Soft limits, step by step. `stepAllowed()` refuses a + step at or above `posMax` and a − step at or below `posMin`, but only while the limits are armed. They are disarmed while homing, because zero is not yet known.

5.6 How the needle moves

Every move follows a motion law: a rule that sets the needle's speed from moment to moment, and so how it starts, overshoots and settles. The motion was settled on a separate bench rig, a bare motor with no needle, and moved into the firmware. The rig's results became the settings

themselves; the settings in section 5.17 (p. 100) are the rule. Each move runs one of two laws.

The second-order law: a meter movement. A damped mass on a spring, driven through an actuator with limits: the equation of a moving-coil meter.

> The second-order law
>
> acceleration = wn^2 × error − 2 × `zeta` × `wn` × speed, then limited to | acceleration| ≤ `upAccel` and |speed| ≤ `upVmax`.
>
> Gain ceiling: keep `wn` < 4 × `zeta` × `upAccel` ÷ `upVmax`. At the defaults (0.50, 18000, 1100) the ceiling is 32.7 and `wn` is 9.

- `zeta` below 1 overshoots and settles: it is the overshoot knob.
- `wn` is the ring-down knob. Measured on the rig, and not intuitive: raising `wn` reduces overshoot, because with the speed clamped a stiffer system arrives with less momentum.
- Why the gain ceiling. The law only brakes once the error is small, and it does not know the motor cannot brake harder than `upAccel`. Braking must start before the stopping distance, which gives the ceiling. At start-up the firmware prints `[WARN] wn ... exceeds the gain ceiling ... - expect overshoot and hunting.` when `wn` is over it. Over the ceiling the failure is deceptive. On the rig, `wn` 12 against a ceiling of 8.4 overshot 11 degrees and hunted. It looked like "this profile does not move the motor", because a bare motor asked to reverse at 2000 hs/s only buzzes. Recompute the ceiling whenever `upVmax` or `upAccel` changes.

The decay law: a drive dying away. A planned fall, used only for the park.

> The decay law
>
> speed follows (1 − e^(−t / `riseMs`)) × e^(−t / `fallMs`), scaled so that its total distance is exactly the distance to go.
>
> If the peak would exceed `dnVmax`, the fall time is stretched rather than the peak clipped, because clipping would leave the needle short. A small correction (gain 20 per second, capped at 30 % of `dnVmax`) keeps the needle on the plan.

The decay law has no acceleration limit; `dnAccel` is used only in its arrival test.

Dwell. A pause before a commanded move starts: `dwellUpMs` (100 ms) before a second-order move, `dwellDnMs` (1000 ms) before the park. The pause is most of why the mechanism reads as mechanical. There is no dwell while following the knob. Following issues a new move for every new target, and a pause restarted by every new target made the needle seem to refuse to move until the knob had gone some distance.

Arrival. A move is finished when the needle is within one step of the target and its speed is below max(1.5 × √(2 × acceleration × step), 20 hs/s). A fixed

20 hs/s threshold gave a one-microstep limit cycle: a 2-second move took 21.4 s to settle. A tolerance of half a step (half of the current microstep, not a half-step) can never be met once the position is off the lattice, the set of positions the current microstep can reach.

Which law runs when:

Table 38. How the needle moves

Move	Law	Microstep	Dwell
Homing: search, backoff hops, re-approach hops	second-order	unchanged (whatever was last set)	100 ms before each hop
Sweep legs (both directions)	second-order	`microFast` (1/16)	100 ms
Following the knob (both directions)	second-order	`microSlow` (1/32)	none
Park at `posMin`	decay	`microFast` (1/16)	1000 ms
Index calibration's measuring creeps	constant speed `reapHsps`	`microFast`	none
Jogs (bring-up tools)	constant speed, emitter set aside	half-step or `microFast`	none

So "up" and "down" in the setting names are the names of the two laws, not directions. The second-order limits (`upVmax` 1100) govern every profiled move but the park; the decay limits govern only the park.

> FOR FIRMWARE CHANGES
>
> Microstep changes only at rest. `useMicro()` does nothing if a move is running. Each division has its own lattice of positions; changing it mid-move leaves a target that whole steps can never reach, and the needle jiggles for ever. At 1/32, 1700 hs/s needs 54 400 steps per second, which the 20 µs floor clamps to about 92 % of the command; that is why moves use 1/16 and slow motion 1/32.

5.7 The supervisor

The supervisor is the part of the firmware that decides what the needle does next and plans its moves (section 5.2 (p. 74)). It runs one pass every 5 ms.

> FOR FIRMWARE CHANGES
>
> `needleTask()` does this every 5 ms:
>
> 1. Feed the task watchdog, then carry out any posted stop or calibration abort (section 5.15 (p. 96)).
> 2. Run the hunting detector (section 5.15 (p. 96)).
> 3. Read the index switch, once (section 5.8 (p. 83)).
> 4. Run the band check (section 5.11.2 (p. 89)).
> 5. Update the needle's memory in RTC RAM (section 5.14 (p. 95)).

6. Every 20 ms, read the AS5600 angle; every 25th time, its status.
7. If a bring-up tool holds the needle: sleep 20 ms and start again.
8. If an index calibration is running: advance it one step, sleep 5 ms, start again.
9. By state: a homing state advances homing. SWEEP runs the sweep, or abandons it on a slip. TRACKING plans a new move if the target differs from the position by 2 hs or more. PARKED re-parks if the position is 2 hs or more from `posMin`.
10. Log index edges (console `e`) and score a crossing (section 5.11.1 (p. 88)).
11. Sleep 5 ms.

5.8 Reading the index switch

The index is a Hall-effect switch, passed by a tiny magnet on the needle's mount. Its output is LOW while the magnet is over it and pulled up otherwise, so the firmware reads LOW as ON.

FOR FIRMWARE CHANGES

Read in one place. The switch is read for motion purposes in exactly one place, `sampleIndex()`, once per supervisor pass, before anything branches on state. Homing, the crossing check and the index calibration all read its result — a debounced level and a one-shot "new edge" flag with its snapshot — and never the pin.

The switch is read once per supervisor pass, every 5 ms. Each reading is called a poll.

Debounce: 4 agreeing polls (`IDX_DEBOUNCE_K`). A new level is believed only when four consecutive 5 ms polls agree. The number is derived, not picked:

- To reject glitches of 10 ms or more: $(K - 1) \times 5\ \text{ms} > 10\ \text{ms}$, so $K \geq 4$.
- To still see a fast crossing: the narrowest ON region (reverse ON to reverse OFF, 94 hs) crossed at the fastest speed near it (1700 hs/s, the park) lasts 55.3 ms, about 11 polls. Seeing both edges needs 2K polls, so $K \leq 5$.

$K = 4$ leaves about three polls of margin.

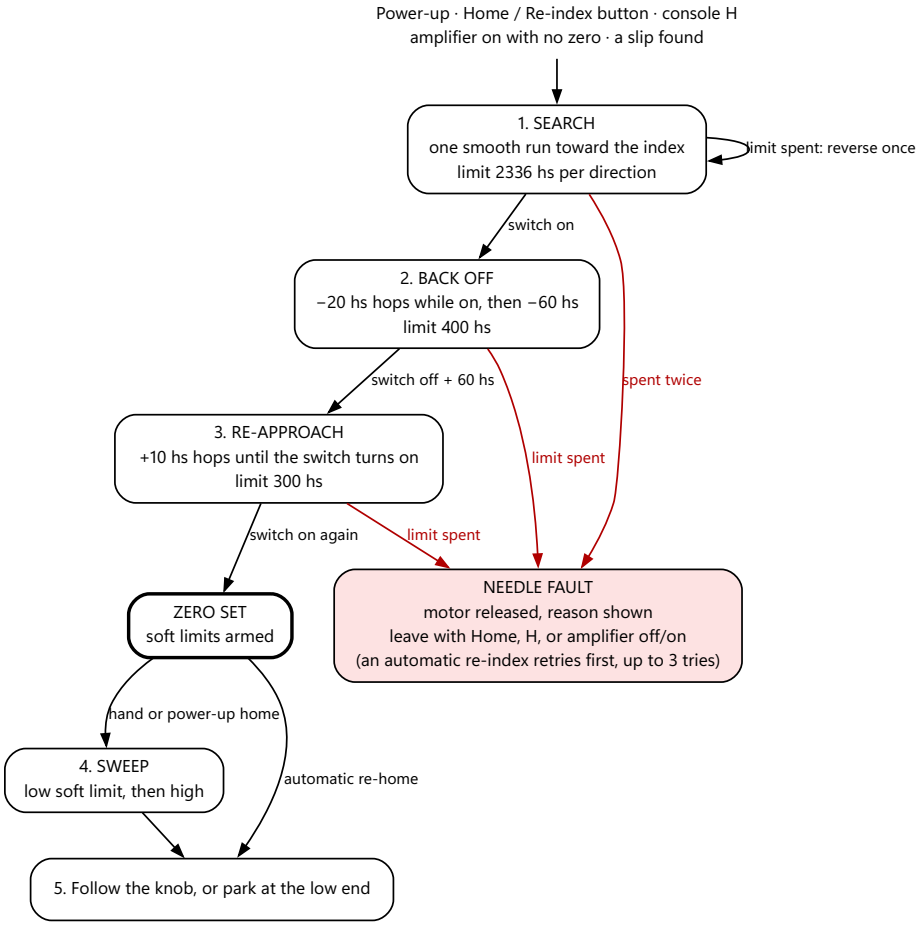
Back-dated, not delayed. When the raw level first disagrees with the confirmed one, the firmware records the position, the speed and the time since the previous poll at that moment. If the run of agreeing polls reaches four, the edge is reported at that first snapshot, not at the poll that confirmed it. The debounce therefore delays knowing about an edge by three polls, but adds no error to where the edge is recorded. What remains is one poll interval of uncertainty: the edge happened somewhere between the previous poll and this one. The recorded interval is clamped to 20 ms, so a scheduling hiccup is not taken for a large lag.

FOR FIRMWARE CHANGES

A true first look. `Needle::begin()` seeds the debounce with a live read, so a magnet already over the switch at power-up reads ON from the first poll, with no false edge. Homing can start within 15 ms of the supervisor starting, and must see the truth from its first look.

5.9 Homing

Figure 11. Homing, phase by phase



Homing finds the index and sets zero on it. It runs at every power-up, when you ask for it, when the amplifier comes on while the needle has no zero, and when the needle finds it has slipped (section 5.11 (p. 88)).

Table 39. Homing

Phase	What the needle does	Step limit
1. Search	Moves toward the index in one smooth run, until the switch turns on.	2336 hs per direction (the whole travel plus 25 %). If it runs out, it reverses once and searches the other way.
2. Back off	Steps down 20 hs at a time while the switch is on; once it turns off, moves a further 60 hs down.	400 hs
3. Re-approach	Creeps up 10 hs at a time, with a short pause before each hop, until the switch turns on again. That edge is zero.	300 hs
4. Sweep	Runs to the low soft limit, then the high one. Skipped when the home was automatic.	—
5. Settle	Follows the knob, or parks at the low end.	—

The final approach always comes from below, so zero is always the same edge of the switch. If the switch is already on when homing starts, the search is skipped and homing begins at the backoff.

Which way the search goes. If the needle has a recorded clue (it saw the index recently), it searches toward the clue. Otherwise, a needle believed above zero searches down, and one believed at or below zero searches up. After a power cut the main board assumes the needle is at the low stop, so it searches up. After a software restart it starts from the position it remembered (section 5.14 (p. 95)).

When the first direction runs out. In the normal case the search limit is never reached: the search stops the moment the switch confirms ON. From the low stop the first pass travels about 1195 hs. If the limit is reached, the console prints `[WARN] index not found in the expected direction - the needle was not where the frame believed. Trying the other way.` and the search reverses, once.

When homing fails — FAULT. If any phase runs out of steps, the needle stops, the motor coils are released, and the portal shows NEEDLE FAULT with the reason:

Table 40. Homing

Reason shown	What it means
index never found in either direction	Both searches ran out of steps without the switch turning on.
sensor never released during backoff	The switch stayed on for the whole 400 hs.
sensor never returned during the re-approach	The switch did not turn on again within 300 hs.

For any of them, check the index switch, its magnet, its pull-up resistor, and the motor. An automatic re-index that runs out of steps does not fault at once; it retries (section 5.11.3 (p. 90)).

Getting out of FAULT. The needle does nothing by itself in FAULT. Any of these starts a fresh home:

- the portal's Home or Re-index button;
- the console key **H**, or **X** then **T**
- switching the amplifier off and on again.

The console keys T and P alone refuse while in FAULT, and say so.

The step limits, and where they come from. Every phase has its own limit, because the soft limits are disarmed while zero is unknown and nothing physical stops the needle. They are properties of the mechanism, never of the soft limits:

Table 41. Homing

Phase	Limit	Derived from
Search, per direction	2336 hs (HOME_BUDGET_HS)	the measured travel, 1869 hs, plus 25 %
Backoff	400 hs (HOME_BACKOFF_BUDGET_HS)	about twice the worst case: the band edge, plus the 60 hs clearance, plus one 20 hs hop
Re-approach	300 hs (HOME_REAPPROACH_BUDGET_HS)	about 2.4 times the roughly 127 hs needed: the 60 hs clearance plus the 67 to 79 hs offset between the switch's two directions

The re-approach speed, and what **reapHsps** really drives. Each 10 hs re-approach hop is an ordinary second-order move under upVmax and upAccel, with the 100 ms dwell before it. The setting reapHsps is not used here, despite its name and its portal label, "Measuring pass speed". It drives exactly two things. The first is the speed of the index calibration's four measuring creeps (section 5.12.1 (p. 92)), which is the measuring pass the label means. The second is the speed of the portal's jog buttons. A 10 hs hop never comes near upVmax the second-order law itself keeps it slow.

The states (Needle::State): IDLE, SEEK_INDEX (search), BACKOFF, REAPPROACH, SWEEP, TRACKING, PARKED, FAULT. A stop sends any state to IDLE; so does a jog or a coil test.

FOR FIRMWARE CHANGES

How zero is set (homingTick()). On the new ON edge, homing computes the shift that makes the back-dated edge read 0 and hands it to the emitter through gCorrectHs. It then waits until the emitter has folded the shift in (gCorrectHs reads 0 again, gZeroPending clears). Only then does it mark the needle homed, arm the soft limits, forget the index clue, note the crossing count as of this home, clear the fault reason and plan the sweep. Without the wait, the crossing check scored the zero edge against the old frame, and the sweep had the frame jump under it mid-move.

Starting a home (startHoming()). It returns nothing when homing started, otherwise a reason, which it also prints. It refuses while an index

> calibration or a bring-up tool holds the needle. Starting a home clears the old fault reason, the drift evidence, the homed flag and a repeating sweep; it keeps the index clue. It does not check whether a home is already running: a second `H`, portal Home or amplifier-on edge during a home restarts homing from where the needle is, with fresh step limits.

5.10 Sweep, tracking, parking

The sweep. After any home that is not an automatic re-index, the needle sweeps: microstep 1/16, a second-order move to `posMin`, then to `posMax`, then on to where it should be. The repeating sweep (console `r`, portal Sweep repeatedly) goes back to `posMin` and continues until stopped.

Where the needle goes next. The needle keeps a "where to go when the busy phase ends": TRACKING (follow the knob) or PARKED. Asking it to track or park sets that:

- if homing, a sweep or a calibration is running, the request only sets where it goes next, and the busy phase finishes first;
- if the needle is homed, it switches at once;
- if it is IDLE and has no zero, it starts a home;
- otherwise (FAULT, or no zero and not IDLE) the request is dropped, and the portal and the console say so.

At power-up the needle goes to PARKED unless told otherwise.

When it follows and when it parks. The main loop asks it to follow while the radio is live — amplifier on and source RADIO — and to park otherwise. That is a listening policy, not a statement about the tube radio, which runs whenever the amplifier does. The needle follows the dial while the radio is what you are listening to (chapter 4 (p. 46)). Nothing is asked while a main-board upload runs or while the needle has no zero. The main loop asks again when the source changes, when the amplifier changes, when the needle becomes homed, after an index calibration, and after an upload that did not end in a restart.

The amplifier-on edge. When the amplifier comes on, the main loop also starts a home if the needle has no zero, or a sweep if `sweepOn` is set. The power-up home always ends in a sweep; **`sweepOn`** adds one on each amplifier-on edge. "No zero" includes FAULT, so every amplifier-on edge sends a faulted needle homing again, up to 2336 hs each way. That is kept on purpose (section 5.20 (p. 111)).

Where to point. The target is the frequency mapped, in a straight line, from the printed face (`dialLow` to `dialHigh`, the frequencies printed at the two needle ends) onto the soft limits (`posMin` to `posMax`), clamped at both ends. It uses the unrounded frequency. Three mappings are kept apart on purpose:

- shaft count to share of the capacitor's travel (the tuner ends, chapter 6 (p. 120));

- shaft count to frequency (the tuning curve, chapter 6 (p. 120));
- frequency to needle (the printed face, here).

> Worked example — this machine's calibration. Soft limits −1156 and +414 hs, printed face 87.9 to 107.9 MHz. The face spans 1570 hs over 200 tenths of a MHz: 7.85 hs per 0.1 MHz. At 100.0 MHz the target is −1156 + (1000 − 879) × 1570 ÷ 200 = −1156 + 950 = −206 hs.

A filtered shaft. The needle follows a smoothed copy of the shaft count: each 20 ms sample moves it 35 % of the way (`TRACK_ACC_ALPHA` = 0.35). A real turn of the knob arrives in about 100 ms; a one-sample spike is cut to about a third. Everything else — the tuning-activity detector, the tuning-curve samples (chapter 6 (p. 120)), every readout — uses the raw count.

Deadband. While following, a new move is planned only when the target differs from the position by 2 hs or more (`TRACK_DEADBAND_HS`). A move is a whole profile (accelerate, decelerate, settle), and one per half-step of sensor noise is audible. PARKED uses the same 2 hs deadband against `posMin`, because a park that settles one half-step short would otherwise be re-commanded for ever.

The moves. Following: second-order, 1/32, no dwell. Parking: decay to `posMin`, 1/16, after a one-second dwell. `posMin` is both the low soft limit and the park position.

5.11 Staying right

Once homed, the needle keeps its place by counting steps from zero. The frame is that count: where the firmware believes zero is, and so where it believes the needle is. If the motor loses steps, the frame is wrong. This section is how the firmware checks the frame and repairs it.

5.11.1 Every crossing is a free measurement

Every confirmed ON edge seen while homed tells the firmware where the needle really is. The supervisor handles each one in five steps:

1. Subtract the sampling lag. The edge happened somewhere in the poll interval before the first disagreeing sample, so on average half an interval earlier: lag = speed × interval ÷ 2. At 1100 hs/s and a 5 ms poll that is about 3 hs.
2. Score only an edge with a known direction. The switch turns on at different places moving up and moving down (its hysteresis). An edge moving + is compared with the forward ON edge (0). An edge moving − is compared with the reverse ON edge, but only once the index calibration has measured it. An edge at rest, or moving − with no band measured, is counted and nothing is concluded.
3. Drift = where it was seen − where it should be. Each scored edge is counted (`gMeas`).

4. |drift| ≤ 40 hs (**CORRECT_MAX_HS**): absorbed. The correction goes to the emitter. The slip flag (the firmware's note that the frame is wrong) is cleared and the index clue (section 5.11.5 (p. 92)) retired: the frame has just been shown right.
5. |drift| > 40 hs: not believed. The number never enters the frame. It becomes a clue to where the index is (expected position + drift), the slip flag is raised, and the console prints a warning. Recovery follows (section 5.11.3 (p. 90)).

Nothing is corrected while an index calibration runs, because it is measuring these same edges. Crossings during a jog are not scored either. A jog runs with the emitter set aside, and the supervisor skips everything after its bring-up branch.

Why 40. It is about half the roughly 79 hs gap between the forward and reverse ON edges — the one systematic error that could pass for drift. A crossing read in the wrong direction is absorbed only if the needle had already drifted about 39 hs or more the other way; with less, it is refused as a slip.

5.11.2 The band check

The crossing check scores ON edges only. It cannot see steps lost after a correct ON edge, while the magnet is still over the switch. That is how this needle once lost steps going up through the index too fast (section 5.20 (p. 111)).

So the firmware also checks the band: the stretch of travel where the switch reads ON, measured by the index calibration (section 5.12.1 (p. 92)). It checks on every pass while the needle is homed, band-calibrated, not calibrating, not in a bring-up tool and not homing. If the switch reads ON while the position is outside the measured band, widened by a margin, the frame is wrong.

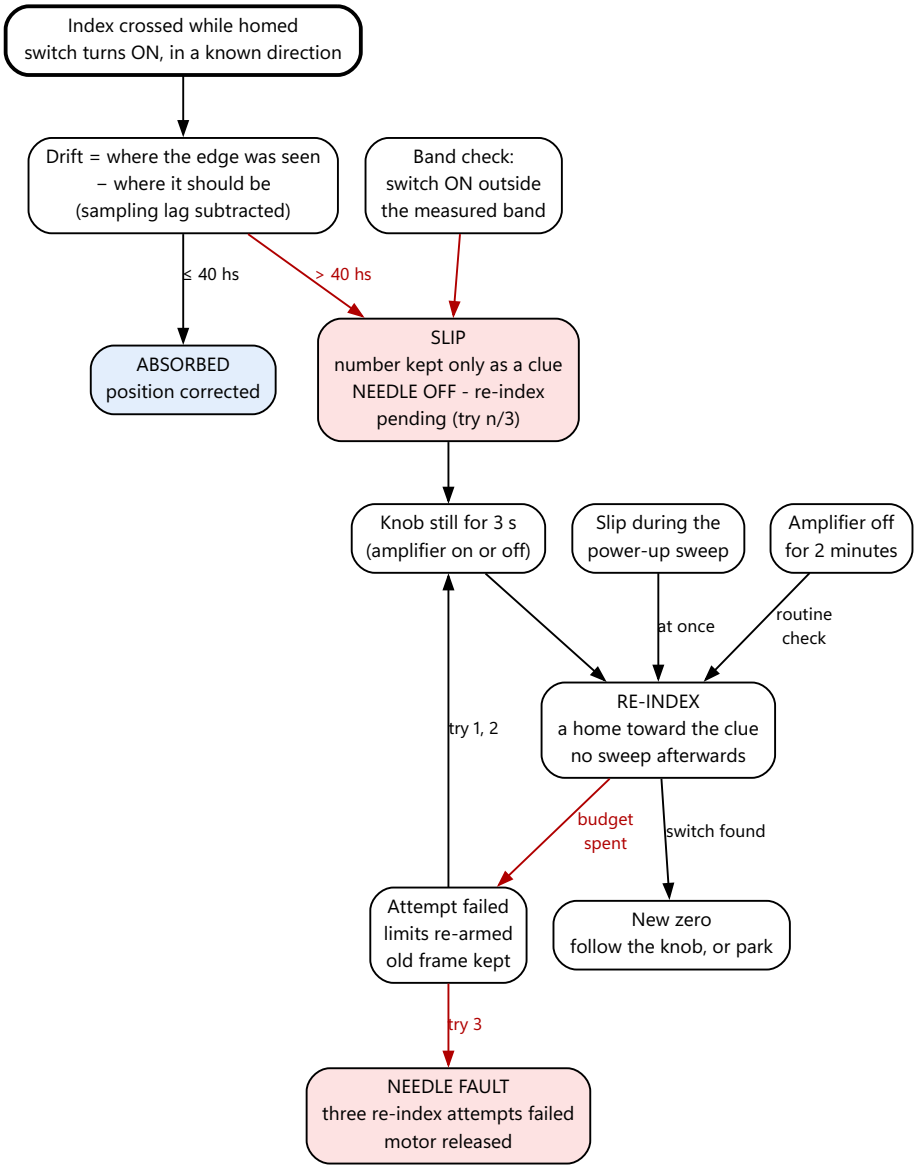
- The band: from the lower of the reverse OFF and forward ON edges, to the higher of the forward OFF and reverse ON edges.
- The margin: 30 hs plus 40 ms of travel at the current speed, because the debounced level lags the needle. A fixed 30 hs was tried first. Going up at 1100 hs/s it fired at 131, 34 hs past the OFF edge at 97 — pure lag — and re-indexed the needle at every start-up.

It fires once per excursion: the clue is set to the current position, the slip flag is raised, a warning is printed, and recovery follows. It does not run before the band has been calibrated: with all four band values at 0, every ordinary crossing would read as a slip and re-index in a loop with no exit.

> Example — this machine's band (forward ON 0, forward OFF 97, reverse ON 79, reverse OFF −15). With the needle at rest, the switch reading ON anywhere outside −45 to 127 hs fires the check. At 1700 hs/s the margin grows to 98 hs.

5.11.3 The re-index ladder

Figure 12. The re-index ladder and the routine index check



A re-index is a home that recovers a frame the needle already has: it searches toward the clue, sets a new zero, and ends without the sweep.

Table 42. The re-index ladder

Drift at a scored crossing	What happens
40 hs or less	Absorbed silently.
More than 40 hs, or the band check fires	Slip flag raised; once the knob has been still for 3 s, a re-index.
A re-index fails	The soft limits are re-armed, the old frame is kept, and it tries again — up to 3 attempts.
The third attempt fails	NEEDLE FAULT, with “three re-index attempts failed. Something is mechanically wrong”.

The trigger. A re-index starts when all of these hold:

- a slip is outstanding;
- no upload is running, and no recovery;
- fewer than 3 attempts have failed;
- the knob has been still for 3 s (`REIDX_STILL_MS`);
- no calibration runs.

The amplifier does not need to be on.

The re-index itself. It refuses during a calibration, a bring-up tool, homing or the sweep. Otherwise it is an ordinary home flagged as automatic. It ends in “where to go next” with no sweep, and a failure is counted instead of faulting. On a failed attempt the limits are re-armed and the homed and slip flags restored, so the trigger comes round again. A wrong frame is then still bounded: the limits are the right distance apart, only in the wrong place.

> FOR FIRMWARE CHANGES
> In the code, the trigger is `needleRecoverTick()` in `src/s3/main.cpp` the re-index is `startReindex()`, then `beginReindex()`.

No zero, no re-index. A re-index recovers a frame that exists. When the needle has no zero — it is in FAULT, or it was stopped before a home finished — the re-index prints `needle: no zero to re-index from - homing instead.` and runs a plain home. That home ends in the sweep like any other, and if it fails it goes straight back to FAULT. In practice only the portal’s Re-index button reaches this case: the automatic triggers all need a homed needle. The portal still answers “re-indexing”.

A slip during the sweep does not wait. The sweep sees the slip, abandons itself, and re-indexes at once.

5.11.4 The routine index check

A park that starts below the index never crosses it. Without a routine check, the needle could not learn of lost steps between listening sessions.

So once the amplifier has been off for 2 minutes (`IDLE_CHECK_MS`), the main loop starts a re-index. It does so once per off period, with the needle homed

and no recovery, calibration, tuning measurement or upload running. The off period's check counts as done only if the re-index actually started.

> FOR FIRMWARE CHANGES
> In the code, the routine check is `idleIndexTick()`.

5.11.5 The index clue

The clue is where the index was last seen, in the frame the position is counting in. A flag says whether it holds a real observation.

- It is replaced by any newer sighting: a slip at a crossing, the band check, or the switch found ON by a homing search.
- It is cleared only when the frame is made or shown true: the re-approach declaring zero, the index calibration shifting the frame, or an absorbed crossing.
- Nothing that merely starts or stops a home touches it. So a re-index cut short by a stop, an upload or a restart still leaves the next home pointed at the index.

> FOR FIRMWARE CHANGES
> In the code, `believeIndexAt()` records a clue and `forgetIndexClue()` clears it.

5.12 Calibrating the index band and setting the soft limits

Both of these are repairs: you need them on a new build, after the soft limits were reset to ±300, or after the index switch or its magnet was moved. Download the settings file first (System tab, Settings file card): it is the only undo (chapter 7 (p. 153)).

5.12.1 The index band

The index calibration measures the four edges of the switch's ON region: on and off, moving up and moving down.

It refuses unless the needle is homed, no slip is outstanding, no homing or sweep is running, and the needle sits on the switch. It says why.

One pass is four constant-speed creeps at `reapHsps` (60 hs/s), each limited to 600 hs:

Table 43. The index band

Creep	Direction	Until the switch turns	Records
1	up (+)	OFF	forward OFF (`idxOffFwd`)
2	down (−)	ON	reverse ON (`idxOnRev`)
3	down (−)	OFF	reverse OFF (`idxOffRev`)
4	up (+)	ON	forward ON (`idxOnFwd`)

The portal runs 3 passes, the console `k 5` (it accepts 1 to 12). The console prints every pass, then each edge's mean and spread. The positions are the

back-dated edge snapshots that homing uses too. At 60 hs/s the sampling lag is about 0.15 hs on average and 0.3 hs at most. That is why these slow edges can serve as the reference that faster crossings are scored against.

Why a constant speed. A move profile is for going somewhere known. A creep moves until the switch says stop and cannot know the distance in advance; a profiled move would mean a fresh dwell and acceleration ramp every few steps. So the emitter has a constant-speed mode that overrides the profile; the jogs use it too.

> FOR FIRMWARE CHANGES
> One step per pass. The calibration is advanced one step per supervisor pass rather than run as one blocking routine, because it is started from a portal button and a blocking call would freeze the portal.

The soft limits stay armed. A creep that reaches a limit fails the calibration with “stopped at a soft limit - the band runs past it; aborted” rather than pushing on.

The result. The frame is shifted so the mean forward ON edge is 0, and the other three edges are stored relative to it. The needle is marked homed, the slip evidence and the clue are cleared, and the crossing count is noted afresh: a frame shift makes every earlier crossing evidence about a frame that no longer exists, as a new home does. The main loop then copies the result into the settings (the `idxOffFwd`, `idxOnRev`, `idxOffRev` rows) and moves both soft limits by the same shift, because they are physical marks on the dial. If the moved pair no longer brackets 0, it falls back to ±300. Then it applies, saves, and sends the needle back to following or parking.

After a real calibration the forward ON edge is 0 and the other three are not. The band counts as calibrated when any of the three stored edges is not zero.

Procedure:

1. ☐ Press Home (or console `H`) and let the home finish. It ends by sweeping away from the index, so the needle is not on the switch afterwards.
2. ☐ Jog the needle back into the band with the jog buttons. About 40 hs above zero is well inside it on this machine. Watch the portal’s live index field until it shows the switch on. (The console’s edge log prints nothing during a jog.)
3. ☐ Press Calibrate the index (or console `K`). The Needle tab shows the progress and the message. Four things stop it: Abort calibration or STOP on the Needle tab, console `A`, or console `X`.
4. ☐ Read the result: forward ON 0, and three non-zero edges in the settings.

5.12.2 The soft limits

The rule. `posMin` ≤ 0 ≤ `posMax`, and `posMin` < `posMax`. The index is a mid-travel mark, so any real pair brackets 0. A pair that fails is refused and the old one kept, and the settings always show the pair the needle runs.

The defaults, ±300 hs (`PROVISIONAL_LIMIT_HS`), are deliberately narrow. With no end switches, a default must be shorter than the travel on both sides. It also must not be too short: the sweep after a home at ±300 must still cross the index band, or the limits could never be captured.

Purge at start-up. Once, after the settings load, a stored pair that is crossed or does not bracket 0 is reset to ±300 and saved. The band values are kept: they are relative to the index.

Capture. Set low limit here and Set high limit here (Needle tab), or console `m` and `M`, store the current position as `posMin` or `posMax`. Capture refuses unless:

- the needle is homed;
- no calibration runs;
- the band is calibrated;
- the index has been crossed and scored since the last home or index calibration (a jog does not count);
- no slip is outstanding.

It then applies the value and checks that it took.

Nudge. Needle Limit ◂ 20 and 20 ▸ (Needle tab) move both limits by the same amount (the action accepts 1 to 200 hs). That slides the position the needle is sent to for every frequency. The press itself does not jog the needle: it changes only the two limits, and it is not a clearance tool. It refuses a result that would not bracket 0.

Procedure (after the index band is calibrated):

1. ☐ Let the needle cross the index once, for example with Sweep once, so a crossing is scored.
2. ☐ Jog the needle to the lowest printed mark of the glass (87.9 MHz on this dial). Jogs are not stopped by the soft limits — that is how a limit is widened — but each press moves at most 200 hs and crossing a limit is announced. A needle driven against a mechanical stop loses steps.
3. ☐ Press Set low limit here (or `m`).
4. ☐ Jog to the highest printed mark (107.9 MHz) and press Set high limit here (or `M`).
5. ☐ Type the frequencies printed at those two marks into `dialLow` and `dialHigh` if they are not already right.

With the provisional ±300 limits the whole printed face maps onto 600 hs, and the needle hugs the middle of the dial. That is not a failed calibration; capture the real limits.

5.13 When the angle sensor drops out

The angle sensor's reading, its outages and how the shaft's turn is chosen belong to chapter 6 (p. 120). What the needle does:

- While the sensor does not answer, the shaft count is frozen and the needle holds its last target. The firmware retries every 2 seconds, and the portal's `i2c` field reads 0.
- When it answers again, the fresh angle is placed back on a turn of the shaft (chapter 6 (p. 120) says which), and the needle follows again. A movement hidden by the outage counts as tuning, so the "knob still" timers, including the 3-second re-index wait, start again, and the console prints how far the shaft moved.

Measure both tuner ends (chapter 6 (p. 120)): with them, the turn after an outage is always the right one; without them, a wrong turn can be saved and inherited by every later start-up.

5.14 Memory across restarts

A software restart does not move the needle, and a power cut leaves it wherever it was. The main board uses this to choose where to believe the needle is at start-up, and so which way homing searches first.

Table 44. Memory across restarts

Reset reason (as console `s` prints it)	Starting belief
SW: a portal reboot, an update, a console restart	The remembered position, if the record is whole and sane.
PANIC (a crash)	The remembered position, as above.
TASK WATCHDOG, INTERRUPT WATCHDOG, OTHER WATCHDOG	The remembered position, as above.
POWERON	The low stop.
BROWNOUT	The low stop. Excluded on purpose: a count kept through a supply sag is not one to steer by.
EXT (the reset pin), deep sleep, unknown	The low stop.
Any of the first three, with the record missing, torn or out of range	The low stop.

When the remembered position is used, the console prints `needle: remembered at N (software restart) - homing from there.`

Homing always runs afterwards. The memory is a better starting belief for the search direction, not a substitute for finding zero.

"The low stop" is a direction hint, not a position. It sets the position to `posMin`, which is negative, so the search goes up. It is wrong by up to the whole travel, and harmless only because the needle is never asked to follow or park while it has no zero. It holds because the needle parks at the low end whenever the amplifier goes off and does not move while unpowered — hence the caution in section 5.1 (p. 73).

Rollback. A watchdog reset counts as a software restart, so a hang ends in a restart that homes from the remembered position. A freshly updated program that crashes or hangs during its trial is rolled back by the

bootloader (chapter 3 (p. 25)), and the previous program reads the same record and homes from there.

> FOR FIRMWARE CHANGES
> How it works inside. The record lives in RTC memory that survives a software reset (`RTC_NOINIT_ATTR`). It holds a magic number `MEM_MAGIC` ("NM01", 0x4E4D3031), the position in fine units, the homed flag, the clue's valid flag, the clue position, and a checksum (FNV-1a over the fields, never over the structure's bytes).
>
> - Writing (`memRecord()`, needle task only, on every pass in which anything changed): magic = 0 first, then the fields, then the checksum, then the magic last. All members are `volatile`, so the stores keep that order, and a reset in the middle leaves magic = 0. Nothing is recorded while a new zero is in flight, and nothing before the start-up position has been set: the needle task starts before `setup()` reaches `bootPosition()`, and would otherwise overwrite the record with the cold position 0.
> - Reading (`bootPosition()`): the record is read once and invalidated before anything else, so it is never used twice. It is used only for the reset reasons above, and only if the magic and the checksum match, both flags are 0 or 1, and both positions are within ±6000 hs (`MEM_POS_SANE_HS`). The position is then snapped to the current microstep lattice and the clue restored. Otherwise `assumeAtLowStop()` runs.
> - The record is kept in fine units so the drive's electrical angle matches where the rotor actually sits. It is read by whichever program starts, so a program with a different record layout must change `MEM_MAGIC`.

Where homing fits in the main board's start-up sequence is in chapter 4 (p. 46).

5.15 Stops, hunting, uploads

5.15.1 Stop and abort

Stop and abort are requests. They can come from the console, from portal buttons and from the upload hook, on either core. They only set a request bit and zero the speed. The emitter stops stepping at once: it checks the bit every control tick and before every step. The supervisor does the rest at the top of its next pass, within 5 ms:

1. abort a calibration;
2. clear a constant speed;
3. end a recovery;
4. end a repeating sweep;
5. set the target to the position;
6. release the coils;

7. release a held coil test (back to microstepping at `microFast`);
8. go to IDLE.

The bits are cleared only after the work. Nothing can set a constant speed while a request is pending.

- A stop during a home leaves the needle IDLE with no zero. It homes again on the next `H`, `T`, `P`, Re-index, amplifier-on edge or upload resume.
- A running jog is not interrupted by a stop.
- After a jog the needle stays IDLE where it was put. Automatic behaviour resumes on `H`, `T`, `P` or a source change.

> FOR FIRMWARE CHANGES
> Who writes the position. The emitter owns the position. The supervisor hands corrections over through `gCorrectHs`: one 32-bit word, one writer, and one reader that only adds it in, with an atomic exchange. A new zero waits for the fold (section 5.9 (p. 84)). `stopPending()` lets the main loop wait for a stop to land before starting the needle again. The places that still write the position directly are listed in section 5.22 (p. 116).

5.15.2 Hunting

Hunting is a needle that keeps stepping without going anywhere. Every 4 seconds (`HUNT_WINDOW_MS`) the firmware checks: if at least 120 steps were emitted (`HUNT_STEPS_MIN`) and the net movement is under 8 hs (`HUNT_NETMOVE_MAX`), the needle is hunting.

- The console prints `[WARN] the needle is HUNTING` when it starts and `the needle has settled` when it ends. The portal shows NEEDLE HUNTING (`hunt`).
- It is a warning only. Nothing is driven differently, and it names no cause.
- Following the knob moves both numbers together, so only travel that cancels itself trips it. It stayed quiet across 41 samples of a settled needle.
- What it cannot see: a needle stuck and not moving, a slow one-way error, or a wrong but steady reading.
- The count is of emitted steps at the current microstep: 120 steps is 3.75 hs at 1/32 and 7.5 hs at 1/16.

It exists because a wobble was once found only by watching the driver board's LEDs. The portal truthfully reported a still needle: the two positions it alternated between rounded to the same half-step.

5.15.3 Uploads and reboots

A flash write stalls every task running from flash, the emitter included. So for an upload of the main board's own program (the update itself is chapter 3 (p. 25)):

- Start. The needle is stopped and the clock display blanked. While the upload runs, nothing asks the needle to follow or park, and the amplifier edge, the recovery trigger and the routine check do nothing.

- Success. The main board restarts, and the needle's memory points the start-up home toward the index (section 5.14 (p. 95)).
- No restart. A refused image (wrong chip), an aborted upload, a failed write, or 15 seconds with no upload activity resumes the needle — but only once the stop has landed. It homes if it has no zero (and is neither recovering nor faulted); otherwise it goes back to following or parking.
- A refused reboot. The portal's reboot saves the settings first. If the save is refused because the needle is moving, it stops the needle and retries for 1.5 seconds. If the save still fails, the reboot is refused and the needle resumes.
- A reboot during a trial skips the save and the stop and restarts at once, because nothing is saved during a trial and the reboot is what rolls the program back (chapter 3 (p. 25)). The needle's memory carries the position across it.
- An upload refused because the program is on trial is refused on its first block, before the needle is stopped: the needle carries on as if nothing had happened.
- An upload to the audio board does not stop the needle.

No setting is written while the needle moves, while a calibration runs or while an upload is active, and none while a program is on trial (chapter 7 (p. 153)).

5.16 Portal and console

The needle's pills (NEEDLE OFF - re-index pending (try n/3), NEEDLE RE-INDEXING, NEEDLE FAULT, NEEDLE HUNTING, and the `needle` dot lit when homed) are in chapter 1 (p. 3), section 1.13 (p. 12).

Needle tab buttons (administrator only):

Table 45. Portal and console

Button	What it does
Home	Starts a home. A refusal is shown.
Re-index	Starts a re-index. Refused during a calibration, a bring-up tool, homing or the sweep. With no zero (FAULT, or a stopped home) it runs a plain home instead.
Sweep once / Sweep repeatedly	One end-to-end sweep, or sweeps until stopped. Refused unless homed.
Track the tuner	Follow the knob. Says so when the request is dropped.
STOP	Stops the needle; also aborts a running calibration.
◂ 100, ◂ 10, 10 ▸, 100 ▸	Jog that many half-steps, microstepped, at `reapHsps`. Clamped to ±200 hs, and says so if clamped. Not stopped by the soft limits. The portal waits for the jog to finish.
Set low limit here / Set high limit here	Capture `posMin` / `posMax` (section 5.12.2 (p. 94)).
Calibrate the index	The index calibration, 3 passes (section 5.12.1 (p. 92)).
Abort calibration	Aborts it.
Tuner = low end / Tuner = high end	Stores the tuner end for the tuning (chapter 6 (p. 120)). Refused if the angle sensor is not answering.
Needle Limit ◂ 20 / 20 ▸	Moves both soft limits by 20 hs (section 5.12.2 (p. 94)).

There is no park button. The tab also holds the tuning-curve buttons (chapter 6 (p. 120)). Its status card shows the position, the state (with the fault reason), the index switch live, and the calibration's progress and message. The Now page shows the position, the state and the index switch too.

Console keys (USB, or the portal's Console tab; every key is in Appendix D (p. 332)):

Table 46. Portal and console

Key	What it does
H	Home.
T / P	Track / park. Prints “NOT tracking” / “NOT parking” when the request is dropped.
x	Stop the needle; also aborts a running calibration and ends a recovery.
r	Repeating full sweep, until x.
k	Index calibration, 5 passes (put the needle on the switch first).
A	Abort a calibration.
m / M	Capture the low / high soft limit here.
n / N	Jog −20 / +20 hs at 60 hs/s, microstepped.
j / J	Jog +200 / −200 hs at 200 hs/s, half-steps.
u / U	Jog +200 / −200 hs at 200 hs/s, microstepped.
1 … 4 / 0	Hold one driver input at full duty / release.
e	Index edge log on or off: every edge, with its back-dated position and direction.
l	Live index monitor, the raw pin, for up to one minute. Stops on any key other than Enter.
c / C	Store the tuner’s low / high end (chapter 6 (p. 120)).
g G	Prints that the end-stop calibration was removed and that m / M replace it.
Z	Clear the jitter counters.
s	Status: position, target, state, drift, crossings, jitter, steps, speed, PWM, and the angle sensor’s chain, with the residue (accumulated count − raw angle) & 4095, which must stay constant. Then the list of commands.
D	Dump all settings.

The S3′s console does not restart when a monitor attaches, so its start-up messages are never seen; press s. The l monitor holds the main loop while it runs, feeding the task watchdog on every pass.

> FOR FIRMWARE CHANGES
>
> For a firmware modifier: the needle’s portal state fields. pos, tgt (half-steps), nstate (state name), homed, drift, driftpend, driftmeas, calbusy, calpct, calmsg, fault, reidx, reidxtry, hunt, idx (the index switch, live, raw), jit, i2c, magnet.

5.17 Settings and constants

All of these are on the portal’s Needle tab (admin only), in the downloadable settings file, and in the console’s settings dump (key D). A change is saved after 2 seconds without further changes, never while the needle moves, and never while a new firmware is on trial (chapter 3 (p. 25)). The portal’s range clamps the value.

Motion

Table 47. Settings and constants

Setting	Default	Range	What it does
`upVmax`	1100 hs/s	100–4000	Top speed of every normal move: following, sweeps, homing. Portal: “Sweep up, top speed”. 1100 is this mechanism’s measured limit. Steps by 50.
`upAccel`	18000 hs/s²	1000–60000	Acceleration limit of normal moves.
`dnVmax`	1700 hs/s	100–4000	Top speed of the fall to the park position.
`dnAccel`	18000 hs/s²	1000–60000	Used only to decide when the park move has arrived.
`wn`	9.0 rad/s	1–21	How quickly the swing settles. Keep it below 4 × `zeta` × `upAccel` ÷ `upVmax`.
`zeta`	0.50	0.1–2.0	How much the needle overshoots (about 5 degrees, about 300 ms of settling, measured on the bench rig).
`riseMs` / `fallMs`	100 / 1000 ms	10–3000 / 10–5000	Shape of the fall to the park position.
`dwellUpMs`	100 ms	0–3000	Pause before a normal move (not while following the knob).
`dwellDnMs`	1000 ms	0–3000	Pause before the fall to the park position.
`microFast`	16	1–32	Microstepping for sweeps, parking, calibration and jogs. Rounded down to a power of two.
`microSlow`	32	1–32	Microstepping while following the knob. The portal labels these two settings “Microstep while moving” (`microFast`) and “Microstep at rest” (`microSlow`); the labels mislead — the parked needle uses `microFast`.
`reapHsps`	60 hs/s	10–500	Speed of the index calibration’s measuring passes and of the portal’s jog buttons. Portal: “Measuring pass speed”. Despite its name, it does not set the homing re-approach speed.
`sweepOn`	off	on/off	A full sweep each time the amplifier comes on.

Where the dial is

Table 48. Settings and constants

Setting	Default	Range	What it does
`posMin`	–300 hs	–3000–3000	Low soft limit, park position, and where `dialLow` is printed. On this machine: –1156.
`posMax`	+300 hs	–3000–3000	High soft limit, and where `dialHigh` is printed. On this machine: +414.
`dialLow`	87.9 MHz	50.0–200.0	Frequency printed at the low end of the glass, mapped onto the low soft limit (`posMin`), not onto the mechanical stop.
`dialHigh`	107.9 MHz	50.0–200.0	Frequency printed at the high end of the glass, mapped onto the high soft limit (`posMax`), not onto the mechanical stop.
`idxOffFwd`, `idxOnRev`, `idxOffRev`	0	–500–500	The index switch’s edges, measured by the index calibration. On this machine: 97, 79 and –15.

The defaults of ±300 hs are safe starting limits for a new build, not this machine’s values. A pair of limits that is crossed, or that does not bracket the index, is refused and the old pair kept.

- Who changes the limits: the capture (`m`, `M`, the portal), the nudge, the index calibration's carry, the purge at start-up, the portal rows and the settings file.
- The ranges of **`posMin`** and **`posMax`** are wide on purpose, because validity is checked separately, not by the range. They are signed on purpose too: a range that started at 0 once clamped a negative value silently and answered "ok".
- The three band rows are rows so that the settings file carries them: a restore onto a blank board had left all four band values at 0, and with them the band check and the scoring of downward crossings switched off.
- **`idxOnFwd`** is stored but is not a row: it is 0 by definition.
- **`dialLow`** and **`dialHigh`** describe this set's printed glass; another set's builder types in their own. Inside the firmware they are in tenths of a MHz (879 and 1079).
- The tuner ends (`calLow`, `calHigh`) and the saved shaft count (`lastAngle`) belong to the tuning, chapter 6 (p. 120).

FOR FIRMWARE CHANGES

Compile-time constants. These change only with a new build.

Table 49. Settings and constants

Name	Value	What it is
`CTRL_US`	1000 µs	Motion control tick.
`FINE_PER_HALFSTEP` / `FINE_PER_EREV`	256 / 2048 fine	One half-step; one electrical revolution.
`DRV_LEDC_FREQ_HZ` / `DRV_LEDC_BITS`	20 000 Hz / 11 bits	Motor PWM; the bits are negotiated down to 8.
LEDC channels	2, 3, 4, 5	The four motor inputs.
Step interval clamp	20 µs to 1 s	50 000 steps per second at most.
Coil release delay	400 ms	After a move ends.
`IDX_DEBOUNCE_K`	4 polls	Index debounce.
Sample interval clamp	20 000 µs	Cap of the lag model.
`CORRECT_MAX_HS`	40 hs	Largest drift absorbed.
`BAND_MARGIN_HS`	30 hs	Band-check margin, plus 40 ms of travel.
`TRACK_DEADBAND_HS`	2 hs	Following and park deadband.
`HOME_BUDGET_HS`	2336 hs	Search limit per direction.
`HOME_BACKOFF_BUDGET_HS`	400 hs	Backoff limit.
`HOME_REAPPROACH_BUDGET_HS`	300 hs	Re-approach limit.
Backoff hop / clearance	20 / 60 hs	In homing.
Re-approach hop	10 hs	In homing.
`AUTO_TRIES_MAX`	3	Failed automatic re-indexes before FAULT.
`BAND_BUDGET`	600 hs	Limit of each index-calibration creep.
Index calibration passes	portal 3, console 5, 1 to 12 accepted	
`HUNT_WINDOW_MS` / `HUNT_STEPS_MIN` / `HUNT_NETMOVE_MAX`	4000 ms / 120 steps / 8 hs	Hunting detector.
`MEM_MAGIC`	0x4E4D3031	Memory record; change it if the record changes.

Name	Value	What it is
`MEM_POS_SANE_HS`	6000 hs	Memory record's sanity bound.
AS5600 poll / status	20 ms / every 25th read	
Bus retry	2000 ms	After an angle-sensor bus failure.
`TUNE_DEADBAND`	12 counts	Knob-movement detector (chapter 6 (p. 120)).
Tuning-active hold	400 ms	How long after a knob movement the knob counts as moving.
`TRACK_ACC_ALPHA`	0.35 per 20 ms	The needle's shaft filter.
`SEAT_MARGIN`	300 counts	Turn window around the tuner ends (chapter 6 (p. 120)).
Decay correction gain	20 per second, capped at 0.3 × `dnVmax`	Keeps the park on its plan.
Jog speed clamp	5 to 900 hs/s	
Portal jog clamp	±200 hs	
Nudge range	±1 to 200 hs	
`PROVISIONAL_LIMIT_HS`	300 hs	Default and purge soft limits.
`REIDX_STILL_MS`	3000 ms	Knob still before an automatic re-index.
`IDLE_CHECK_MS`	120 000 ms	Amplifier off before the routine index check.
`lastAngle` save period	10 000 ms	Shaft count saved when it changed (chapter 6 (p. 120)).
`HALFSTEPS_PER_REV`	4096 hs	Defined, used nowhere.
Placeholders at start-up	limits ±300, dial 88.0 to 108.0, a flat tuning curve at 88.1 MHz	Overwritten by the stored settings before any move.

5.18 Physical values of this machine

No other device will be built exactly like this one, so this mechanism's physical values are kept here, in the firmware book, rather than in the Hardware Bible. The Hardware Bible records what is wired to what and how the mechanism is built. It does not record how this particular mechanism behaves: which way the motor turns the needle, how far the needle can travel, where the index magnet sits in that travel, how fast the gearbox will follow. The firmware depends on all of these, and another builder will not share them.

How to read the tables:

- Compiled: written into the source; changes only with a new build. Setting: a stored setting with a compiled default. Measured: the firmware measures it on the machine and stores it; only its compiled default is this machine's.
- Depends on it: the firmware values derived from a physical value. Change the physical value and re-derive those (section 5.23 (p. 118)).
- Positions are in half-steps from the homed zero, the index's forward ON edge; shaft angles are AS5600 counts, 4096 per turn.

5.18.1 The motor and its drive

Table 50. The motor and its drive

Physical value	This machine	Held as	Another builder
Direction: advancing the motor's electrical angle moves the needle toward low FM	inverted; measured on this machine	the inversion switched on at start-up; compiled	Jog `u` (+200 hs). If the needle moves toward low FM with the inversion off, turn it on, and the other way round. Every position above the drive layer assumes + is toward high FM.
Coil pairing: IN1 and IN3 are the two halves of one phase, IN2 and IN4 of the other, turning order IN1, IN2, IN3, IN4	as stated; the needle has always moved microstepped in both directions on it	the half-step table and the phase offsets of the drive; compiled	Hold each input with `1` … `4`, then step with `j`/`J` and `u`/`U`. A needle that buzzes, stalls or moves backward on some steps has another pairing: change the table, not the wiring.
The motor is a 5 V part, so full PWM duty on its 5 V supply is safe	5 V, 100 % duty	the duty set at start-up; compiled	If your motor's rated voltage is below its supply, lower the duty before anything moves. The bench rig worked out, but never soak-tested, the duty that gives the same coil current as 5 V continuous through this kind of driver: about 100 × 4.8 ÷ (V − 0.2) %, so 41 % at 12 V. Check the temperature before trusting it.
The needle holds its place with the coils unpowered	holds	coil release 400 ms after every move; compiled	If your needle drifts or falls when released, remove the release (and accept the heat), or re-reference more often. A needle that moved while released would show as drift at the next index crossing.
The motor and gearbox do not lose steps at the settled motion	no lost steps: a 30-minute edge log showed the trip points repeating within ±4 hs over dozens of crossings; steps were lost only when driven past a physical stop	the whole open-loop position model	Log `e` over a long `r` sweep: the index edges must repeat. If they wander, slow the motion before anything else.
The speed the mechanism follows going up through the index	1100 hs/s never lost steps; 1200 to 1400 often did; 1500 always did (every power-up sweep then landed about 400 hs, about 5 MHz, low)	`upVmax` default 1100; setting	Sweep `upVmax` downward from high, watching `last drift` over repeated `r` sweeps, and keep a margin below the first failure. `dnVmax` (1700) governs only the park, and is the fastest the needle ever crosses the index. Depends on both: `IDX_DEBOUNCE_K`.
Gear ratio 64:1 (4096 hs per output turn)	4096, recorded as measured on this motor	`HALFSTEPS_PER_REV` compiled, used nowhere	Nothing to do: the firmware counts half-steps from the index and never needs the ratio. Many motors of this family are about 63.68:1 rather than 64:1.

The motion feel — `wn` 9, `zeta` 0.50, `upAccel` and `dnAccel` 18000, `riseMs` 100, `fallMs` 1000, the dwells, 1/16 moving and 1/32 following — is a matter of taste, settled on the bench rig. It is not a physical value, but it was judged on

this motor and needle, and a lighter or heavier pointer may want it re-tuned by eye.

5.18.2 The travel, the index and the soft limits

Table 51. The travel, the index and the soft limits

Physical value	This machine	Held as	Another builder
Travel between the two mechanical stops	1869 hs (about 164 degrees of the motor's output shaft, at 0.088 degree per hs)	`HOME_BUDGET_HS` = 2336 (1869 + 25 %); compiled	Jog from stop to stop and read `pos`. Depends on it: `HOME_BUDGET_HS`, `MEM_POS_SANE_HS` (6000: the travel plus a whole search, with room).
Where the index sits in the travel	1195 hs above the low stop, 674 hs below the high stop; near 102.5 MHz on the glass, so listening below that never crosses it	the homing limits; compiled	Read `pos` at each stop after a home. Depends on it: the backoff and re-approach limits (each runs one way, and neither may reach a stop), `PROVISIONAL_LIMIT_HS`. The index must be mid-travel for anything in this chapter to work.
The index band: the four switching edges	forward ON 0, forward OFF 97, reverse ON 79, reverse OFF −15	`idxOffFwd`, `idxOnRev`, `idxOffRev` forward ON 0 by definition; measured	Run the index calibration with the needle on the switch (section 5.12.1 (p. 92)). Nothing needs re-deriving by hand except the constants in the next three rows.
Offset between the ON edges seen moving + and moving −	about 79 hs (earlier estimates said 67)	`CORRECT_MAX_HS` = 40; compiled	Keep `CORRECT_MAX_HS` at about half your offset, so a crossing read in the wrong direction is refused as a slip unless the needle had already drifted that far the other way.
Narrowest ON region (reverse ON to reverse OFF)	94 hs: about 55 ms, 11 polls, crossed at 1700 hs/s	`IDX_DEBOUNCE_K` = 4; compiled	Re-derive K: (K − 1) × 5 ms above the glitch you reject, and 2 × K polls inside the narrowest region at your fastest speed near the index (section 5.8 (p. 83)).
How far outside the band the switch may still read ON	30 hs plus 40 ms of travel, chosen from the debounce lag at speed plus room	`BAND_MARGIN_HS` = 30; compiled	Re-check against your band and speeds: too small raises false slips, too large misses real ones.
Homing hops and limits	backoff hops 20 hs, clearance 60 hs, limit 400; re-approach hops 10 hs, limit 300	compiled	Re-derive from your band width; each limit must stay shorter than the distance to the stop it faces.
Provisional soft limits	±300 hs: under the travel on both sides (1195 and 674), and above half the band, so the sweep after a home crosses the band	`PROVISIONAL_LIMIT_HS` = 300; the `posMin`/`posMax` defaults	Keep it below the shorter side of your travel and above half your band width.
The soft limits themselves	−1156 / +414 hs, moved 20 hs inward from −1176 / +434	`posMin`, `posMax` measured (captured)	Capture your own, after a home, an index calibration and one scored crossing (section 5.12.2 (p. 94)).
The needle is at the low stop after a power-up	low stop: the needle parks at `posMin` when the amplifier goes off and holds unpowered	the starting belief after a power-on; compiled	Only a direction hint: if your set is often unplugged while playing, the first search may go the wrong way first, within its limit.
A software reset leaves the rotor where it was, to within a detent	yes, to within a detent, across a portal reboot, an update and a watchdog rollback	the memory record (section 5.14 (p. 95)); compiled	Nothing to do unless your driver moves the rotor at reset.

5.18.3 The tuner and the dial

Table 52. The tuner and the dial

Physical value	This machine	Held as	Another builder
The tuner's whole travel lies inside one shaft turn	about 2200 counts of 4096 (2197 measured), for about 20 MHz	the turn choice, and `SMP_MAX_DRIFT` = 20 (about 0.2 MHz), both chapter 6 (p. 120) compiled, checked at run time	Measure both tuner ends (`c`/`C`, or Tuner = low end / high end). If the span is a whole turn or more, the firmware falls back to the nearest turn, which is right only for movement under half a turn.
The tuner's ends repeat	within 300 counts: a margin chosen with the rule above, not a measurement of repeatability	`SEAT_MARGIN` = 300; compiled	Keep the measured span plus twice the margin under 4096.
The tuner's ends	measured: `calLow` 1024, `calHigh` −1173 (the count runs down as the frequency rises, which is allowed); compiled placeholders 0 / 6023	`calLow`, `calHigh` measured	Measure both ends. The 6023 placeholder is left from an older belief that the shaft turned about 1.47 times; its span is wider than a turn, so it also switches off the measured-ends turn choice until real ends are stored. Nothing reads the placeholder as a measurement (chapter 7 (p. 153)).
What the tuner reaches before any calibration	88.1 MHz at the low end, 107.9 MHz at the high end	`bandLow` 881, `bandHigh` 1079; compiled defaults	Calibrate the tuning curve (chapter 6 (p. 120)).
The frequencies printed at the two needle ends	87.9 MHz low, 107.9 MHz high, read off this set's glass	`dialLow` 879, `dialHigh` 1079; setting	Type in what your glass prints at each end.

What another builder does first, in order:

1. ☐ Set the direction.
2. ☐ Check the coil pairing and the duty.
3. ☐ Find the stall speed and set `upVmax` below it.
4. ☐ Jog to both stops and read the travel and the index position.
5. ☐ If yours differ, re-derive the homing limits, `CORRECT_MAX_HS`, `IDX_DEBOUNCE_K` and `PROVISIONAL_LIMIT_HS`, and rebuild.
6. ☐ Home.
7. ☐ Run the index calibration.
8. ☐ Capture the soft limits.
9. ☐ Measure the tuner ends.
10. ☐ Type in the printed dial.
11. ☐ Calibrate the tuning curve (chapter 6 (p. 120)).

5.19 When it goes wrong

The last column is the fault's class on the recovery ladder (NOTE, DEFECT, BLOCKER), set out in About this book.

Table 53. When it goes wrong

What you see	What happened	What the needle does	What you do	Class
Nothing	It lost a few steps (40 hs or less).	Corrects itself at the next pass over the index.	Nothing.	NOTE
NEEDLE OFF on the portal	It lost more than 40 hs, found at a crossing.	Re-homes by itself after 3 seconds with the knob still.	Nothing.	NOTE
NEEDLE OFF on the portal	It lost steps while the magnet was still over the switch; the band check saw it.	As above.	Nothing.	NOTE
A sweep stops halfway at power-up	It slipped during the sweep.	Re-homes at once.	Nothing.	NOTE
Nothing	It lost steps while parked or parking, below the index.	The routine check 2 minutes after switch-off finds it.	Nothing.	NOTE
NEEDLE OFF - re-index pending (try n/3), with n rising	An automatic re-index failed (first or second time).	Re-arms the limits, keeps the old frame, tries again.	Nothing.	NOTE
NEEDLE FAULT after an automatic re-home	Three automatic re-homes in a row failed.	Stops, releases the motor.	Home it by hand (section 5.9 (p. 84)). If it fails again, check the index switch and the motor.	DEFECT
NEEDLE FAULT at power-up or after Home	A homing phase ran out of steps.	Stops at once.	As above; the console suggests checking the switch, its magnet, its pull-up and the motor.	DEFECT
Re-index on a needle with no zero answers “re-indexing”	There is no frame to recover.	Runs a plain home; a failure returns to FAULT, never to a guessed frame.	As above.	NOTE
`[WARN] index not found in the expected direction ...`	The first search went the wrong way.	Searches the other way, once.	Nothing, if the needle survives it.	BLOCKER if the needle is driven against a stop: the limits are off, up to 2336 hs. Made unlikely by the clue and the memory, not impossible.
The first search after a power cut or brownout goes up and away	The needle was above the index when power was lost.	Assumes the low stop, searches up first.	Nothing, if it survives the wrong-way pass. Park before	As above; a brownout is accepted.

What you see	What happened	What the needle does	What you do	Class
			unplugging (section 5.1 (p. 73)).	
A wrong-way first search after a software restart	The memory record was torn or missing.	Assumes the low stop.	As above.	NOTE when the needle was low; as above otherwise.
NEEDLE HUNTING on the portal	The needle keeps moving back and forth around one position.	Warns only.	Find the cause; the console key x stops it.	NOTE
Needle holds still while you tune	The angle sensor stopped answering.	Holds its last target (section 5.13 (p. 95)).	See chapter 6 (p. 120), section 6.12 (p. 142).	NOTE
Tuning wrong after a long angle-sensor outage, or from start-up	The shaft was placed on the wrong turn.	—	Measure both tuner ends (chapter 6 (p. 120)). With them, the right turn is always chosen.	NOTE with the ends measured; without them a wrong turn can be saved and inherited by every start-up: BLOCKER, cured by measuring the ends.
Typed soft limits rejected	The pair was crossed or did not bracket the index.	Keeps the old pair; the console warns.	Enter a valid pair.	NOTE
Soft limits back at ±300 hs	They were found corrupt in memory and reset.	Works inside the safe defaults.	Home, run the index calibration, capture the limits again (section 5.12 (p. 92)).	BLOCKER (guarded: the needle stays off the stops, but the limits must be captured again)
“stopped at a soft limit - the band runs past it; aborted”	An index-calibration creep reached a soft limit.	The calibration fails and says so.	Move the limit, or accept.	NOTE
Needle pinned at one end while tuning	You are past the printed scale.	Holds at the soft limit.	Nothing.	NOTE
Needle frozen, clock display dark	A main-board update is running.	Resumes when it ends.	Wait.	by design
Needle resumes after an update that did not finish	The upload was refused, aborted, failed, or silent for 15 s.	Resumes once the stop has landed.	Nothing.	NOTE
A portal reboot refused	The settings save failed.	The needle resumes.	Retry later.	NOTE
A reboot during a trial	—	Restarts at once without saving; the previous program	Nothing.	NOTE

What you see	What happened	What the needle does	What you do	Class
		comes back and homes from the remembered position.		
A home after a crash, watchdog or software restart	The main board restarted mid-motion.	Homes from the remembered position.	Nothing.	NOTE
The needle never moves; `s` shows `*** LEDC REFUSED ***`	The chip accepted no PWM resolution at 20 kHz (`[FAIL] LEDC would not accept 20 kHz at ANY resolution`).	Nothing moves.	A firmware change. Never seen.	BLOCKER
A restart and a home; `s` shows the reset reason TASK WATCHDOG	The supervisor hung; the task watchdog reset the board.	Homes from the remembered position.	Nothing. Never seen.	NOTE
Needle frozen, or the portal dead, for good	A task on core 0 (the emitter or the portal) hung; nothing watches them.	Nothing.	Power cycle.	BLOCKER; an accepted cost

5.20 Design choices

- The needle follows frequency through the printed face, not the capacitor's travel. The capacitor is one ganged assembly shared with AM and SW and turns wider than the printed FM scale; following its travel compressed and offset every station.
- The motion comes from the bench rig: a second-order law for every move but the park, a decay law for the park, a dwell before commanded moves, microstep by activity. A trapezoid reads as a machine; a meter movement overshoots slightly and settles. `wn` 9 and `zeta` 0.50 gave about 5 degrees of overshoot and a 300 ms ring-down (475 ms at 6 and 0.60); the fall gave a 1-second hesitation, a peak at about 15 % of travel, a long trickle, and landed exactly. At 1700 hs/s, 1/8 and 1/16 delivered 100 % of the steps and 1/32 92 %; the worst step jitter was 5 to 10 µs (no motor attached). The firmware before the rig swept a trapezoid at 850 hs/s, about 2.8 s for a full sweep, far below what the motor could do.
- Following uses 1/32; everything else 1/16. 1/32 is smoother exactly where smoothness shows, but the 20 µs floor clamps it at speed.
- **`upVmax`** is 1100, not the rig's 1700. Faster, the needle stopped following through the index while the count went on, and every power-up sweep landed about 400 hs (about 5 MHz) low: 1500 failed every time, 1200 to 1400 often, 1100 never.
- No end switches: one mid-travel index, soft limits, step limits and a starting belief. This motor does not lose steps at the settled configuration: one full sweep came back to the index 17 hs out, repeated returns agreed within about 12 hs with gear lash, and a 30-minute edge log showed the trip points within ±4 hs; its one jump (about 215 hs) came when the needle was commanded into a stop.

- Home once, at power-up; the front switch does not home a needle that has its zero. The main board starts only when the set is plugged in; afterwards every crossing corrects the count, and the run from the low end to the station is itself a sweep, so `sweepOn` is pure theatre.
- A needle with no zero homes on the amplifier-on edge, FAULT included. Kept on purpose, as one of the ways a person gets the needle out of FAULT.
- Zero is one edge approached one way: the forward ON edge, after a backoff below the band. A Hall switch turns off at a weaker field than it turns on, so the ON region shifts with the direction of travel; a one-way approach cancels that, and gearbox backlash with it.
- Zero comes from the slow re-approach, not from a crossing at speed. Reading the index inside the step emitter for precision at speed would gain single half-steps, at the cost of moving the sensor read across a task boundary on a mechanism with no end switches; a reading at speed only decides between absorbing and re-indexing anyway.
- Every crossing re-references, but only an edge of known direction is scored, with the sampling lag subtracted. The two directions trip about 79 hs apart, twice what may be absorbed. A crossing read at 1500 hs/s was up to 7.5 hs late, always in the direction of travel; with the lag subtracted, the first start-up read `last drift 4` against 7 to 13 before.
- Absorb up to 40 hs; above that, re-index — never trust the bigger number. A re-index uses the number only as a direction hint and takes the new zero from the switch. Each rung of the ladder uses a better measurement than the one below, not a bigger dose of the same one.
- Three failed attempts, then FAULT; a failed attempt re-arms the soft limits. A needle that slips straight back out is mechanical, and repeating would grind; bounded-and-wrong costs accuracy, not gearbox.
- An automatic re-index ends without the sweep. The sweep is theatre; a repair should happen quietly between one thing and the next. A re-index takes about ten seconds end to end.
- The band check: ON outside the measured band means the frame is wrong. The crossing check cannot see steps lost after a correct ON edge.
- A slip during the power-up sweep ends the sweep at once and re-indexes, with no 3-second wait. The sweep drives to the limits in a frame the slip has just shown wrong.
- The automatic re-index waits for 3 seconds of still knob, amplifier on or off. It waits for exactly what the display calls “the hand has left the knob”; with the set off, nobody is looking at the dial.
- A routine index check 2 minutes after the amplifier goes off. Parking never crosses the index from below.
- The index clue is kept until a better one replaces it. A re-index cut short used to leave no clue, and the next home searched by the sign of a frame just proved wrong, with the limits off and a 2336 hs limit.

- The starting belief: the low stop after power-on, the remembered position after a software reset, brownout excluded. “Believe the low stop, search up” ran away from the index after every reboot made with the needle above it.
- Homing limits are properties of the mechanism, not of the soft limits. Limits from the soft-limit span gave a 750 hs search with the ±300 defaults, against a needle about 1195 hs below the index: it ran out, tried the other way into the low stop, and faulted. Every phase has one, because a magnet already over the switch starts at the backoff.
- Soft limits must bracket the index; defaults ±300; purge at start-up; carry with the band shift. A stored pair of −1837 to −117 put the index outside the range, killed re-referencing and drove the park into a stop. Re-setting the limits by hand after every index calibration was the step every earlier corruption came from.
- The index calibration measures from the switch outward and keeps the soft limits armed; there is no end-stop calibration. The operator places the needle and the machine measures back to the index, so nothing is driven into a stop while being calibrated.
- The needle follows the unrounded frequency, with a 2 hs deadband, from a filtered shaft, with no dwell while following. A pointer on a glass dial is an analogue indicator; following the rounded tenth of a MHz made the target hop 7.85 hs on sensor noise. After the fix, 68 of 71 samples of a resting needle sat at one position; the filter removed the remaining 2 to 3 hs flips.
- The shaft’s turn is chosen by the measured tuner ends (chapter 6 (p. 120)).
- Stop and abort are requests; only the emitter writes the position; a new zero waits for the fold. An in-place stop from core 0 raced homing and calibration on core 1, and corrections written between a read and a clear were lost.
- The emitter runs on core 0, core 0’s idle task is off the watchdog, and nothing is written to flash while the needle moves. A settings write stalled the emitter to 383 µs of step jitter, against 49 µs with the guard. For uploads, a dark and still needle is preferred to a stuttering one. A slow portal while the needle moves is an accepted cost.
- Jogs are limited by their step count (±200 hs), not by the soft limits. A needle sitting at a limit must be able to go past it, or the limits could never be widened.
- After a jog the needle stays where it was put. A hand command the machine overrides a moment later is worse than no command.
- FAULT is left to a person. If the firmware cannot fix it, a power cycle will not either. A panel-lamp pulse to signal FAULT was proposed and left out.
- Refusals say why, and failures are visible. Every failure the machine can have must move some published field, at a resolution that would show it.
- `idxOnFwd` is not a settings row. It is 0 by definition; an edited value moved both soft limits physically, into a stop.

- A re-index with no zero runs a plain home. A failed re-index hands back the frame it had; on a needle that never had one, that frame was unmeasured.
- **`upVmax`** changes only through its settings row. The console keys that stepped it had no ceiling.
- Asking to track or park names where the needle goes next; it never cuts a busy phase short. Done the other way, the power-up sweep was overwritten before it started.
- An interrupted upload resumes the needle only after the stop has landed; a refused reboot resumes it too. A resume issued in the 5 ms before the stop lands would be undone by it.

5.21 Tried and rejected

FOR FIRMWARE CHANGES

These were dead ends on this machine. They may not be dead ends on yours.

- Two end-stop switches, homing against them. Replaced by one mid-travel index (the hardware change is in the Hardware Bible, chapter 8).
- Plain on/off half-stepping (and a stepping library's pin order). Here, half-step chatter was visible at needle speeds. Replaced by sine microstepping through PWM.
- An 850 hs/s trapezoid profile. Here it read as a machine.
- Other motion profiles on the rig. Measured on a 170-degree move at 2000 hs/s and 6000 hs/s², judged on taste and on this motor:

  Table 54. Tried and rejected

Profile	Character	Overshoot	Settled
Trapezoid	constant acceleration, cruise, constant braking; reads as a machine	0.2°	1.62 s
S-curve, planned in time	no corner anywhere	0.1°	2.06 s
Second-order (kept)	the meter movement	6.0°	about 4.3 s at the rig's gains of the day
Exponential	decisive start, soft landing, never overshoots; servo-like	0°	3.08 s
Spring-with-friction fall	a mass on a return spring against friction	0°	4.59 s

  A jerk-limited chase overshot 33 degrees before the s-curve was re-planned in time. The spring fall could stop short of zero (at one setting it died 19.8 degrees short), the signature of a worn old set; a fall that always lands was preferred. A random "wobble" needed microstepping to show at all, had to stay under the gear lash or the needle buzzed instead of breathing, and was measured not viable on this gearbox.
- Homing on every amplifier power-up. Replaced by homing once.

- A cold start belief of position 0. It searched down into the stop the needle sat on. Replaced by “the low stop”, now itself limited to cold starts.
- A start-up position saved to flash. Planned as a switch replacement, never built; the RTC memory does the job for software restarts.
- Homing limits from the soft-limit span (see Design choices).
- ±967 default soft limits, an estimate about 293 hs past the measured high stop. A purge that fell back to them, combined with a newly working sweep, would have driven the needle nearly 300 hs into the high stop on its first start-up.
- The end-stop calibration (`g`/`G`, and two portal buttons). It checked nothing before it drove and disarmed the limits: run the wrong one and it crept 4000 hs (about 66 s) into a stop, then left the frame 4000 hs out. Every attempt that drove outward against a guessed limit lost about 300 hs.
- A seek-speed setting. The search only has to find the switch.
- Two reads of the index microseconds apart, and before them an edge detector that froze during jogs and calibration and invented a crossing when released. Replaced by the 4-poll back-dated debounce.
- Writing zero straight into the position at the re-approach. Replaced by a back-dated shift through the emitter, made to wait for the fold: without the wait, a crossing was scored against the old frame (−1226 hs, “Something slipped”) and the sweep had the frame jump 1226 hs under it.
- A 64-bit position. Torn reads across cores.
- Read-then-clear of the correction word. It lost corrections; now an atomic exchange.
- Limit-capture guards on |last drift| > 12 and on a crossing count taken at the start of homing. The first refused on a drift already corrected and its advice re-created the trigger; the second was satisfied by homing’s own crossing.
- Following the capacitor’s travel, then the rounded frequency.
- Accumulating the shaft count across a sensor gap (it injected a whole revolution), then dropping the movement, then the nearest turn (chapter 6 (p. 120)).
- Absorbing more than 40 hs; requiring several crossings to agree (the index is crossed about twice per power cycle, hours apart); a separate “full search” rung (the search already tries the other way).
- A slip over 40 hs flagged and left alone. A one-way ratchet: the needle was 394 hs out after 16 h 40 min of ordinary use, about 24 hs an hour, with the console normally unplugged.
- Taking a crossing’s direction from the sign of the speed. At rest, which is after every completed move, it guessed “forward”. Scoring downward crossings before the band was measured was dropped with it.

- Crossing lag absorbed as drift, from a crossing read that also took the edge log's print time into the recorded position. The frame breathed by about a needle width depending on which way it last crossed.
- A band result kept only in RAM. The next settings change zeroed it while the frame kept its shift; every limit captured afterwards was about 640 hs out, which is how the pair −1837 to −117 came about.
- A jog bounded by the soft limits. A needle at `posMax` refused every forward press.
- The index calibration disarming the soft limits, a leftover from when it could run with no zero.
- Stop and abort carried out in place, from any core.
- A dwell before every tracking move. It read as a threshold; it was a timer, restarted by every new target.
- A panel-lamp pulse for FAULT.
- A real re-index on a needle with no zero. It handed back a frame that had never been measured, armed the limits in it, and tracked or parked in it until the retries ran out.
- Two console keys that stepped **`upVmax`**, the comma (`,`) and the full stop (`.`). The full-stop key had no ceiling.
- Soft limits checked once per move, keyed to the logical direction, in the earliest bench firmware. With the motor's direction inverted, a move toward one stop consulted the other stop's switch and would have pushed through it. Replaced by a check before every step, in the frame the inversion has already corrected.
- The bench rig itself (a bare motor, driver and ESP32, no needle). It produced the motion numbers; it is retired and not published. Section 5.23 says how to rebuild one.

5.22 Known limits

Seen working on the radio: the portal Re-index (about 10 s, back to the same position); the band-check re-index; the routine check after switch-off; the memory after a portal reboot, after an update, and after a watchdog reset that rolled a program back; the hunting detector.

Never exercised on the radio: a slip over 40 hs found by a crossing (rather than the band check) driving the ladder; the three-failure FAULT; any homing FAULT since the step limits were fixed; Re-index on a needle with no zero; the memory after a crash, and the detection of a torn record; the measured-ends turn choice after an outage of more than half a turn; a portal stop or abort landing in the middle of an index calibration.

Known limits and notes (chapter 12 (p. 290)) has the details and starting points for a fix):

1. Homing runs with the soft limits off, under a 2336 hs limit per direction. A wrong starting belief — a cold start with the needle left high, a

brownout, a torn record — can drive toward the far stop before it reverses. In practice: switch the amplifier off, so the needle parks, before unplugging.
2. FAULT has no local sign. Only the portal and the console show it.
3. The portal's needle buttons run on core 0, writing needle state from the portal task while the needle task runs on core 1; only stop and abort are safe across cores.
4. The position is still written outside the emitter by the index calibration's start and finish, which break the ownership rule and could race the emitter; and, safely by construction, by a microstep change (at rest), by the start-up position (before the needle runs), and by the coil release and the jog (emitter set aside). Section 5.23 names them.
5. A second home during a home restarts it from wherever the needle is, with fresh limits.
6. The portal answers "re-indexing" when Re-index runs a plain home, and its refusal during homing or the sweep says "a calibration or a bring-up tool has the needle".
7. A portal jog holds the portal for its whole length: up to 20 s at the minimum `reapHsps`.
8. The gain-ceiling warning checks only the compiled defaults, once at start-up.
9. The power-up home always ends in a full sweep `sweepOn` governs only the amplifier-on sweep.
10. Whether the soft-limit margin is enough is open. The limits were moved 20 hs inward at each end. Before, `posMin` left 19 hs between the park position and the low stop, and the needle drifted about 24 hs an hour. The feared loop — a park in a wrong frame lands past the stop, loses steps, and makes the next park worse — was an analysis, not an observation. After the change: 2 hs of drift in 3 h 20 min, then 2 hs at 1 day 20 h with no re-index needed. Consistent with the loop being real and broken, not proof: nothing counted the switch-offs in those windows.
11. **`CORRECT_MAX_HS`**, **`IDX_DEBOUNCE_K`**, the band margin, the homing limits and the other values of section 5.18 (p. 103) belong to this mechanism. A different magnet, switch position, travel or faster motion needs them re-measured and re-derived.
12. A settings write can land during a move's dwell or a portal jog (the speed reads 0 then) and make the jog stutter.
13. **`HALFSTEPS_PER_REV`** is defined and used nowhere.
14. The re-approach speed is not a setting.
15. A stop during a home leaves the needle IDLE with no zero until the next `H`, `T`, `P`, Re-index, amplifier-on edge or upload resume.
16. The **`upVmax`** row accepts up to 4000 hs/s, far past the 1100 this mechanism follows through the index.

5.23 Changing it safely

FOR FIRMWARE CHANGES

5.23.1 What must stay true

1. `posMin` ≤ 0 ≤ `posMax` and `posMin` < `posMax`, always. `setGeometry()` enforces it; every writer goes through `applySettings()` and its write-back.
2. Only the emitter writes `gPos` in operation. Everyone else hands a correction through `gCorrectHs`. A new zero is not used until the emitter has folded it (`gZeroPending`). Known exceptions: `calStartBand()` and `calFinishBand()`, which break the rule and could race the emitter; and `useMicro()` (at rest), `assumeAtLowStop()` and `bootPosition()` (in `setup()`), `coilRelease()` and `jogRaw()` (emitter set aside), which are safe by construction.
3. The index pin is read for motion decisions only in `sampleIndex()`, once per pass.
4. Every homing phase has a constant limit derived from the mechanism, never from the soft limits.
5. `PROVISIONAL_LIMIT_HS` stays above about half the index band's width, so the sweep after a home crosses the band.
6. `CORRECT_MAX_HS` stays below half the forward/reverse ON offset.
7. The debounce must fit: (K − 1) × 5 ms above the glitch you reject, and 2 × K polls inside the narrowest ON region at the fastest speed near the index. Raise `upVmax` or `dnVmax`, or change the magnet, and recompute K.
8. `wn` < 4 × `zeta` × `upAccel` ÷ `upVmax`.
9. Arrival: a tolerance of one step; a speed threshold that scales with the square root of the step.
10. Never change the microstep division during a move; snap both position and target.
11. Keep the deadline pull-in in `stepTask()`.
12. At 1/32, speeds above about 1560 hs/s are clamped by the 20 µs step floor.
13. No flash writes while the needle moves (`writeSafe()`); uploads stop it.
14. Settings are append-only: bump `SETTINGS_VERSION` and add a migration case (chapter 7 (p. 153)).
15. Change `MEM_MAGIC` whenever the memory record (`NeedleMemory`) changes.
16. The direction inversion lives only in `Drive::setInvert()`.
17. LEDC channels 2 to 5 are the stepper's, in both firmwares' numbering.

18. `assumeAtLowStop()` is a direction hint; nothing may read its position as true.
19. The index clue changes only through `believeIndexAt()` and `forgetIndexClue()`, and is cleared only when the frame is made or shown true.
20. Stop and abort stay requests; anything that sets a speed checks for a pending request.
21. A refusal returns a reason; never report success for a request that was dropped.
22. A re-index needs a zero. Anything that can reach `beginReindex()` on a needle that has none must go through `startReindex()`, which homes instead.
23. The physical values of section 5.18 (p. 103) are this machine's. Change one only from a measurement, and re-derive every constant its row says depends on it.

5.23.2 Traps

- "Up" and "down" in setting names are the two laws, not directions (section 5.6 (p. 80)).
- `reapHsps` does not set the homing re-approach, whatever its name and its label "Measuring pass speed" suggest; it sets the index calibration's creep and the portal jog.
- `gCross` counts edges; `gMeas` counts edges a conclusion was drawn from.
- `gDrift` is history; `gDriftPending` means "still wrong".
- The hunting count is emitted steps, not half-steps.
- `pos` is published in whole half-steps; a smaller oscillation is invisible in it. Watch `hunt`, `jit` and the driver board's LEDs.
- A jog runs with the emitter set aside: its crossings are not scored or logged, and it does not satisfy the limit-capture guard.
- After a correct index calibration `idxOnFwd` is 0 and the other three are not.
- The S3 console is native USB and does not restart when a monitor attaches: anything printed only at start-up is never seen. Use `s`.
- A stop is carried out up to 5 ms later; the state read right after `stop()` may still show the old one.
- With the provisional ±300 limits the needle hugs the middle of the dial. That is not a failed calibration.
- A home ends by sweeping away from the index. To run the index calibration, jog back into the band first.
- A tuner end can be negative, and the "low" end can be the larger count (`calLow` 1024, `calHigh` −1173 on this set). The turn choice takes the ends in either order.

- A gain over the ceiling looks like a profile that does not move the motor.
- Two individually correct changes can be jointly worse than the bug: a purge falling back to old wide defaults, plus a newly working sweep, would have driven the needle into a stop. Neither half showed it alone.

5.23.3 How to test a change

- On the radio, with the console open: `e` (every index edge with position and direction), `l` (the raw switch), `s` (position, target, state, drift, crossings, jitter, steps, speed, PWM, and the angle sensor's residue, which must stay constant), `r` (a repeating sweep, to watch `last drift` over many crossings), `z` (clear the jitter counters), `D` (settings). On the portal: Re-index to exercise recovery, and the state fields `drift`, `driftpend`, `driftmeas`, `reidx`, `reidxtry`, `hunt`, `fault`, `jit`.
- Build both firmware programs with the pinned toolchain and check the build's exit code, not its output (chapter 3 (p. 25)).
- Flash only a board you have positively identified, and not while someone is listening or watching the dial. Uploading through the portal is the normal route (chapter 3 (p. 25)).

5.23.4 Building a bench rig

Motion and drive changes were measured on a bare-motor bench rig before they reached the needle. That rig is retired. A new one needs only a motor, its driver board and an ESP32.

- Tape a paper pointer to the shaft, and make every test end where it began, so any pointer error is lost steps.
- Run every return leg at a rate already known to be safe, so the return can never be what lost the steps.
- Tests worth rebuilding: a rate ladder (where the motor starts to buzz); an out-and-back at a fixed rate over many cycles (the real pull-out test); an unramped start (pull-in); an acceleration ladder; a many-turn run for the gear ratio; a thermal soak; and a backlash measurement (do that one early).
- Measure rates at the division you will move at. At 1/32, rates above about 1560 hs/s are clamped by the 20 µs floor: 1700 hs/s delivers about 92 %.

6. Tuning and the dial's self-calibration

You tune the radio with its original knob, which turns the tube set's own tuning capacitor. The main board reads the angle of that capacitor's shaft and turns it into a frequency in megahertz (MHz). That frequency drives the needle on the glass, the clock digits when they are set to show tuning, and

the Now readout on the portal. The relation between shaft angle and frequency is a curve. It is not known in advance, and it can drift, so the machine measures it itself: a small FM receiver chip inside the cabinet listens to the tube set's own oscillator and works out where the set is really tuned.

Chapter 2, section 2.2 (p. 15) explains how the two sensors share the work: the angle sensor is the live measure, and the receiver chip only calibrates the curve. Chapter 5 is the needle half of the chain. This chapter is the tuning half.

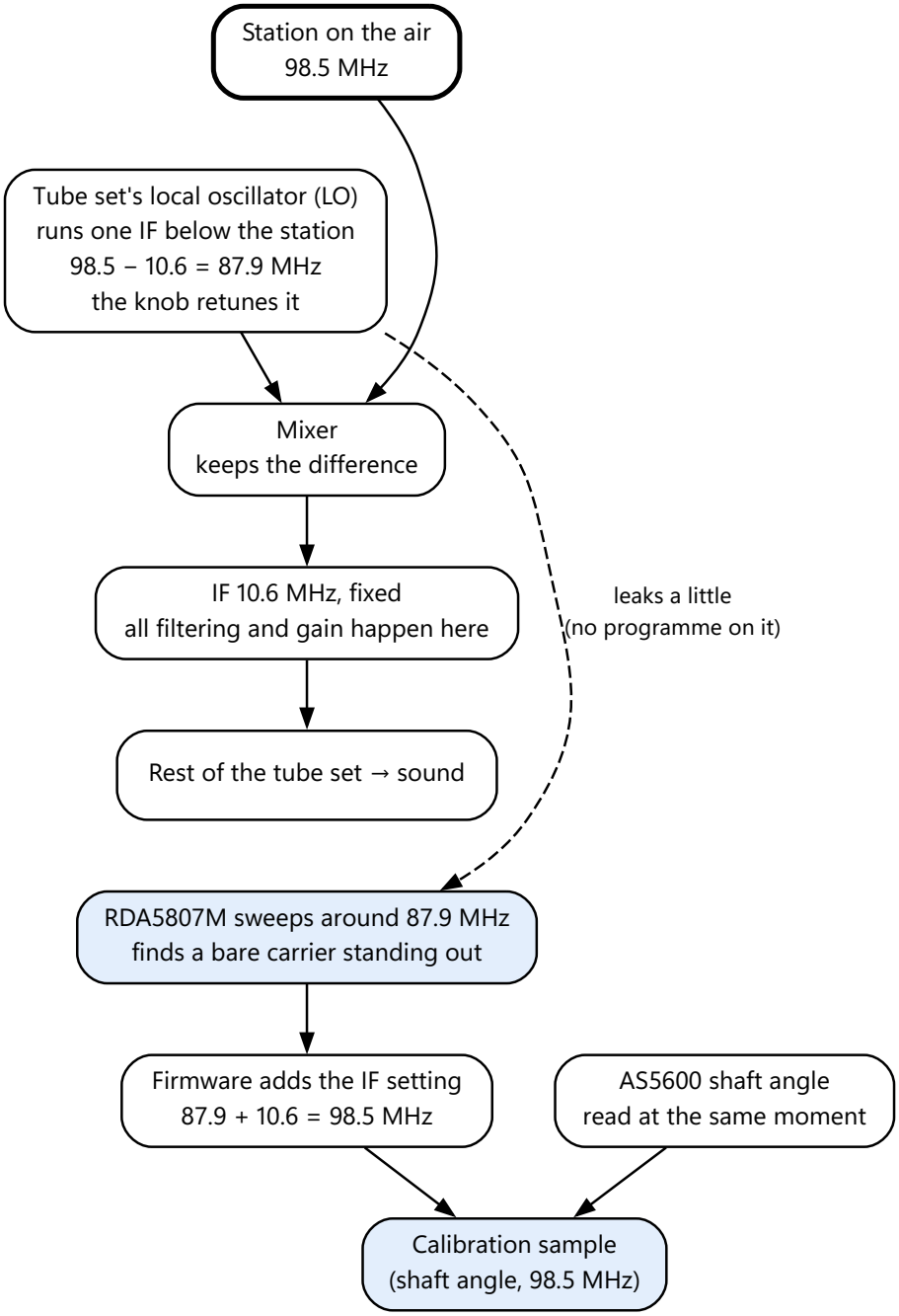
Specific to this build — adapt. The tuning chain is fitted to this LLOYDS TM-838N: its dial, its intermediate frequency and its local oscillator. Your set will differ. The method carries over.

6.1 What you see

1. You turn the knob. The needle follows, the portal's Now shows the frequency to a tenth of a MHz, and the clock digits show it too when set to (section 6.4 (p. 125)).
2. You listen to the radio and leave the knob alone. After 8 seconds of stillness the radio measures the dial by itself, at most once every five minutes. The outcome appears on the portal's Needle tab, under last measurement. For example: `RE-MEASURED 106.1 MHz (LO 95.50, +36 over local) -> slot 4; 6 stored; curve QUADRATIC`. Section 6.8 works through this very line.
3. You press Measure this dial position. The same measurement, at once, with a slightly lower bar for what counts as found. It takes about 15 seconds when the curve already predicts well, up to about 85 seconds when it has to search the whole range. A refusal always says why.
4. The measurements accumulate. Nine slots hold measured samples. A measurement near an existing sample replaces it, so the positions you actually listen to keep being re-measured. Once all nine are full, a new position replaces the most crowded sample, so the nine stay spread across the dial.
5. You can correct by hand. You can type up to three stations you know by ear (the hand marks A, B and C), slide the whole curve in 0.1 MHz steps (the dial correction), reset that correction to 0, and remove any single sample. Nothing automatic ever changes a hand mark or the dial correction.
6. You tune past the printed scale. The clock digits stop at the end of the scale. The portal's Now keeps the true value and adds "(past the printed face)".

6.2 Radio terms you need

Figure 13. The tube set mixes the station with its own oscillator; the oscillator leaks to the RDA5807M, and the firmware adds the IF back



Superheterodyne (“superhet”). The tube set does not amplify the station it receives directly. It generates a signal of its own, the local oscillator (LO), and mixes the incoming station with it. The mixer produces the difference between the two frequencies. That difference is fixed by design and is called the intermediate frequency (IF). All the set’s filtering and amplification

happen at the IF. Turning the knob retunes the LO; the IF never moves. So at any knob position the LO and the station differ by exactly the IF.

Low-side injection. The LO can run above the station (“high-side”) or below it (“low-side”). This set’s FM oscillator runs below the station, so the firmware computes station = LO + IF. This was measured on the bench (section 6.13 (p. 143)), and the Hardware Bible records it (chapter 13).

The IF value. This set’s FM IF is 10.6 MHz. It was set when a professional tuned the FM path, and the RDA5807M’s readings of the oscillator agree with it. The 10.7 MHz on the set’s drawing is the design value. The firmware’s IF is a setting, Tube set IF, with a default of 10.60 MHz (section 6.11 (p. 139)).

Why the LO can be heard. A set of this age has nothing that stops its oscillator from radiating a little. A receiver a few centimetres away hears it as a bare carrier: a signal with nothing broadcast on it. The RDA5807M’s antenna is a 30 cm wire inside the tube radio’s cage, bent over the radio’s FM oscillator (Hardware Bible, chapter 8, section 8.5).

Worked example. The set is tuned to a station at 98.5 MHz. With an IF of 10.6 MHz the LO runs at 98.5 − 10.6 = 87.9 MHz. The RDA5807M, sweeping around 87.9 MHz, finds a carrier standing out there. The firmware adds the IF back: 87.9 + 10.6 = 98.5 MHz. It reads the shaft angle at the same moment and stores the pair (angle, 98.5 MHz) as a sample.

RSSI. “Received signal strength indicator”: the strength number the RDA5807M reports for the frequency it is tuned to, from 0 to 127 (bits 15:9 of its register 0x0B).

Spur, fixed feature, notch. A spur is a signal the RDA5807M hears that is not the tube set’s oscillator: interference made inside the cabinet itself. It stays at the same frequency whatever the knob does. The firmware calls a learned spur a fixed feature, and around each one it cuts a notch: the search may not choose any frequency within ±0.2 MHz of a fixed feature. On this radio the spurs form a comb of seven peaks, at 79.0, 81.8, 84.6, 87.4, 90.3, 93.1 and 95.9 MHz, 2.7 to 2.8 MHz apart, present with the set switched on and off. One of them, 79.0 MHz, reads louder than the oscillator (section 6.15 (p. 149)).

Stereo pilot. An FM broadcast in stereo carries a 19 kHz “pilot” tone. A bare oscillator never carries one. So when the RDA5807M’s stereo indicator (register 0x0A, bit 10) is set on a strong signal, that signal is a broadcast and not the LO. The indicator can only reject: a mono or weak broadcast reads the same as a bare carrier.

Floor, local floor, margin. The floor of a sweep is the median RSSI of all its points. The local floor of one point is the median of up to 16 neighbours within ±0.8 MHz of it. The margin of a point is its RSSI minus its local floor. The firmware judges a peak by its margin, never by its height.

6.3 Units and tasks

Several units are mixed on purpose. In the code, the end of a variable's name tells its unit.

Table 55. Units and tasks

Name ending	Unit	Example
`f10`, `...10`	tenths of a MHz	985 = 98.5 MHz
`f20`, `...20`	twentieths of a MHz (50 kHz steps)	1971 = 98.55 MHz; IF 212 = 10.60 MHz
`acc`, `tuneP[]`, `calLow`, `calHigh`	angle sensor accumulated counts: 4096 per shaft turn, counted across turns	—
permille	0 to 1000 across `calLow` … `calHigh`	shown only by the console `s`

On this radio the count falls as the frequency rises. The firmware never relies on that direction: every test compares sizes or the signs of slopes, never order.

FOR FIRMWARE CHANGES

The tuning work runs in four places on the main board, all but the portal on core 1. Chapter 2, section 2.4 (p. 18) has the full task table.

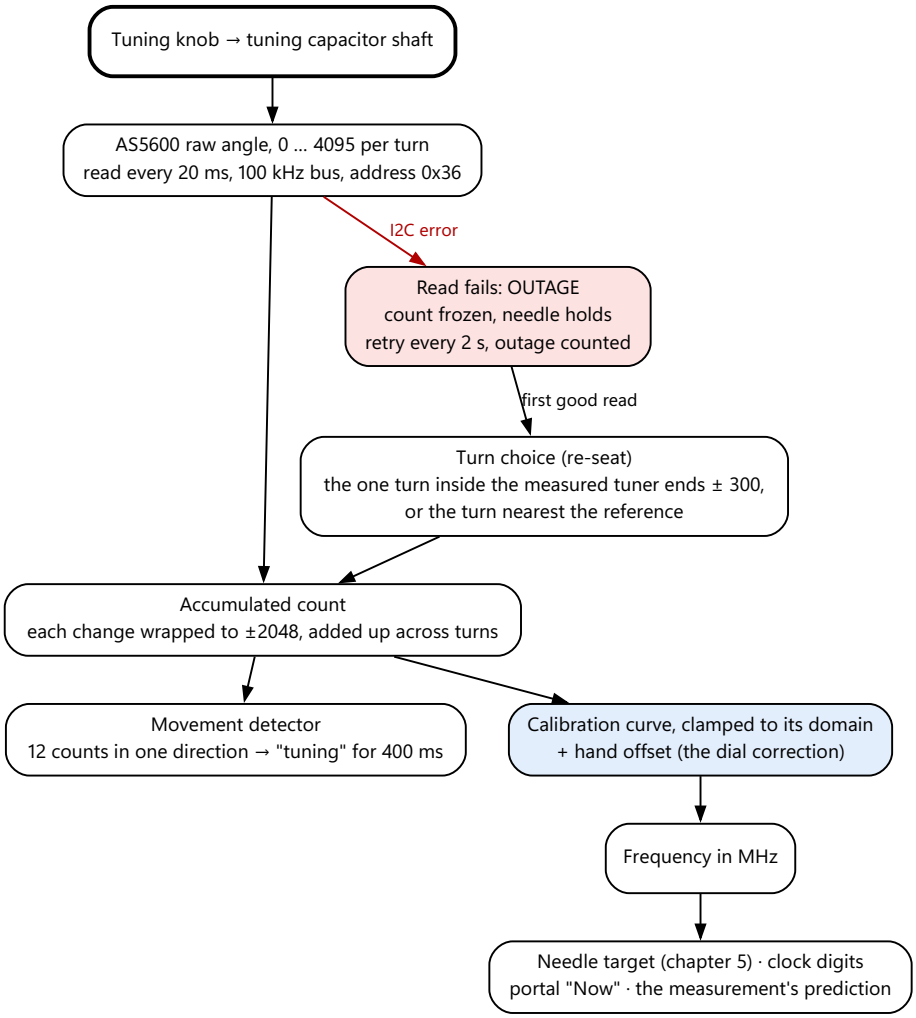
Table 56. Units and tasks

Work	Runs in	Priority	Rate
Angle sensor read	the needle task	4	every 20 ms
RDA5807M sweeps and fine passes	its own task, `rda`	2	looks for a request every 100 ms; each point takes at least 300 ms
Measurement stages, automatic trigger, console keys, saving `lastAngle`	the main loop	1	every pass; `lastAngle` checked every 10 s
Portal actions (measure, marks, settings)	the portal task, core 0	3	on request

The RDA task outranks the main loop on the same core. A path in it that never pauses starves the main loop, and a main loop starved for more than 2 seconds mutes the audio (the link to the audio board times out). So every path in the RDA task pauses: 300 ms per measured point, 10 ms per failed point, 100 ms while idle. The main loop is also under a 15-second watchdog (chapter 4 (p. 46)); the RDA task is not watched.

6.4 From shaft angle to megahertz

Figure 14. From the shaft's angle to the frequency shown, with the outage path



1. Read.

Every 20 ms the firmware reads the 12-bit raw angle from the AS5600, a magnetic angle sensor on the tuning capacitor's shaft, at I2C address 0x36 on the main board's first I2C bus. I2C is the two-wire bus the main board uses to talk to its sensors. The firmware runs this one at 100 kHz, not 400 kHz: slower edges radiate less, and a hand-turned knob needs no speed. The difference from the previous read is wrapped to ±2048 counts and added to the accumulated count, which carries on across shaft turns.

The firmware only ever reads the sensor: the raw angle, its status, its automatic gain (AGC) and the magnetic field strength. It writes none of its configuration and never touches its one-time programmable range, which can be burned once only, ever. The sensor runs at its power-on settings, and all scaling is done in firmware. The full 4096 counts per turn are plenty: the tuner's roughly 2200 counts across 20 MHz are about 10 kHz per count, against channels 200 kHz apart. The status is kept as three separate facts

(magnet detected, too weak, too strong) beside the gain and the field strength. That is because "no magnet" covers three conditions that need different fixes.

2. Which turn.

The AS5600 knows the angle within one turn only. At start-up, and after an outage, the firmware must choose the turn:

- If both tuner ends were measured, and the span between them plus 300 counts on each side is narrower than one turn, exactly one turn puts the angle inside that window. That turn is taken, whatever moved meanwhile.
- Otherwise (ends not measured, the window a whole turn or wider, or the angle outside it) the turn nearest a reference is taken: the saved `lastAngle` at start-up, the last count after an outage. That is right for any movement under half a turn.

The main loop saves the live count as `lastAngle` every 10 seconds, only when it changed and only while the sensor answers. A placeholder count from before the first good read is never saved. At start-up the measured tuner ends are loaded before the first reading is placed on its turn, so the ends can choose the turn (chapter 4 (p. 46) gives the whole start-up order).

On this tuner the whole travel lies inside one shaft turn: about 2200 counts (2197 measured), some 190 degrees. The firmware relies on that and on the ends repeating within 300 counts; it checks the first at run time, and a measured span too wide for one turn falls back to the nearest turn.

3. Outages.

When a read fails, the sensor is left alone for 2 seconds before the next try, the outage counter goes up once, and the sensor is reported down (`i2c 0` in the portal's state). While it is down, the count is frozen and the needle holds its last target. On the first good read the angle is re-seated on a turn by the rule above, never simply added across the gap. A movement hidden by the outage counts as tuning, and the console prints how far the shaft moved. The sensor is reported up again only after the re-seat. Chapter 2, section 2.2 (p. 15) says what each surface does meanwhile.

4. "The knob is being turned."

The firmware treats the knob as being turned for 400 ms after the last detected movement. A movement is detected when changes in the same direction add up to 12 counts (the deadband). A reversal resets the sum, so sensor jitter never trips it, and a slow turn still counts. This drives the tuning readout on the clock digits and every "knob still" timer. Those timers include the 3-second wait before a needle re-index (chapter 5 (p. 73)) and the 8 seconds before an automatic measurement.

5. The curve.

The count is clamped to the range the curve was fitted over (its domain), the curve is evaluated in tenths of a MHz, and the hand offset (the dial correction) is added. A curve value that is not a number, or lies outside 10 to 300 MHz before the offset is added, gives 88.1 MHz plus the offset instead. One version of the result is rounded to the nearest tenth, another is not; the table below says which surface shows which.

6. Who uses which count.

The needle follows a filtered copy of the angle (chapter 5 (p. 73)). The measurements, the readouts and the movement detector all use the raw accumulated count.

Where the frequency is shown.

Table 57. From shaft angle to megahertz

Surface	What it shows
Clock digits, `showTuning` = 1 ("Tuning always")	The frequency rounded to the nearest tenth, held inside `dialLow` … `dialHigh`. Shown whatever the amplifier and source. The panel has no decimal point: `1017` means 101.7 MHz.
Clock digits, `showTuning` = 2 ("Tuning while tuning")	Only while the amplifier is on and the source is RADIO, and for `tuneHoldMs` after the last movement. The unrounded frequency, held inside the printed face, then snapped to the nearest odd tenth (87.9, 88.1, … the grid of FM channels in the Americas), never off the face.
Portal Now	The frequency to a tenth, not held inside the face; "(past the printed face)" is added outside `dialLow` … `dialHigh`. Shown whenever the angle sensor answers.
Portal Needle tab	last measurement (the last outcome sentence, or "measuring N % …"), tuning curve (how many samples, the model, the hand offset), tuner reaches (the curve's frequencies at the two ends of its domain).

Chapter 4 describes the clock readout's modes in full.

6.5 The calibration curve

The curve is fitted through samples: pairs of (shaft count, frequency in tenths of a MHz). There are twelve slots. Slots 0 to 2 are the hand marks A, B and C slots 3 to 11 are RDA samples. The fit treats them alike. Every change to any setting refits the curve, so a stored sample and the running curve can never disagree.

Table 58. The calibration curve

Samples	Curve
Fewer than 2	The stored straight line: from `bandLow` (88.1 MHz) at `calLow` to `bandHigh` (107.9 MHz) at `calHigh`.
2	A least-squares straight line.
3 or more	A quadratic (a gentle curve), accepted only if its slope has the same sign at both ends of the domain: a tuner never turns back. Otherwise, or if the samples are degenerate, a least-squares straight line.

Least squares means the curve that makes the sum of the squared errors at the samples as small as possible.

The domain. If both tuner ends are measured, the domain is exactly `calLow` … `calHigh`, widened only to include a sample outside it. If not, it is the

samples' own span padded by 30 % on each side (at least 100 counts), and the curve's description says "tuner ends NOT measured". Until the ends are measured, `calLow` and `calHigh` hold the placeholders 0 and 6023, which mean "not measured".

Guards. At both ends of the domain the curve, without the hand offset, must lie between 50.0 and 200.0 MHz, and the two ends must be at least 2.0 MHz apart. If either test fails, the stored straight line is installed instead.

The tuning sentence. The model actually running is recorded, never guessed from the number of samples. One sentence describes it, "N marks, MODEL, reaches X - Y MHz", and the console, the portal's card and every tuning message print that same sentence. These words in it mean something is wrong:

Table 59. The calibration curve

Words in the sentence	Meaning	What you do
"QUADRATIC REFUSED: turns back on itself"	The quadratic would reverse inside the tuner's travel; a straight line is running.	Find the bad sample (console `s` residuals) and drop it.
"the marks are degenerate"	The samples cannot define a quadratic; a straight line is running.	Drop the bad sample.
"MARKS REFUSED (...)"	The fit failed a guard; the stored straight line is running. The portal's message is red.	Re-mark or drop the bad sample.
"tuner ends NOT measured"	The domain is a guess padded around the samples.	Press Tuner = low end and Tuner = high end (section 6.10 (p. 138)).

Typing `calLow` or `calHigh` into its portal row returns the sentence as a warning when something is wrong (the fit refused or demoted, or a tuner end not measured). A single field edit can demote the curve and throw marks away, and the answer says so instead of a plain "ok".

> Note The reach can reassure falsely. The stored straight line always reports the most reassuring reach, 88.1 to 107.9 MHz. Read the model in the sentence, not the reach, to know whether your samples are in use.

6.6 The RDA5807M as an instrument

The RDA5807M is a single-chip FM receiver. Here it plays no sound: it is a measuring instrument. It sits on the main board's second I2C bus, which the firmware runs at 100 kHz, and answers at address 0x11; the firmware addresses it there only. The chip, its supply and its antenna are in the Hardware Bible, chapter 8, section 8.5.

> FOR FIRMWARE CHANGES
>
> Set-up. At start-up and at every re-probe the firmware resets and enables the chip, then writes:

Table 60. The RDA5807M as an instrument

Setting	Value	Why
Register 0x02	DHIZ, DMUTE, NEW_METHOD, ENABLE	NEW_METHOD is a sensitivity mode of the chip; the bench test ran with it.
MONO (in 0x02)	cleared	The stereo indicator needs the stereo decoder running.
AFC (automatic frequency control), register 0x04	off	AFC would let the chip slide toward a louder neighbour and report its frequency.
Band, register 0x03	band 2, 76 to 108 MHz, 100 kHz spacing	The LO window starts at 77.2 MHz, below the ordinary 87–108 band.
Low-noise amplifier, register 0x05	both input ports, 2.7 mA	The chip's most sensitive settings. The bench test used the defaults (1.8 mA, one port) and still read the LO at 59 against a floor of 44, but inside the cabinet that margin might not survive, and these cost nothing. Both ports work whichever way the module's antenna pad is routed.
Volume, register 0x05	0	An instrument, not a radio.

The chip is set up at start-up and at every re-probe. Every one of these set-up writes must succeed, and a final read of register 0x0A must answer, or the chip is reported absent. A re-probe runs the whole set-up again, not just a check for an answer: a chip that came back may have browned out and reset to its power-on defaults, AFC on among them.

One point. The firmware tunes the chip to a channel, on the 100 kHz grid (channel = tenths of a MHz − 760) or the 50 kHz grid (channel = twentieths − 1520), waits up to 60 ms for the chip's "tune complete" flag (STC, register 0x0A bit 14), then waits 300 ms, the dwell, whatever the caller asked. The RSSI reading needs about 300 ms to settle after a tune: on the bench a 40 ms dwell hid the oscillator completely. Then it reads the RSSI and the stereo flag. The stereo flag is read only after the tune has completed, because its power-on value is 1 and an early read says "stereo" for everything. A failed read is a failure: never a zero, and never "mono".

A sweep is a run of points 100 kHz apart, each kept as a bin. It is clamped to the LO window, 77.2 to 97.2 MHz, a compiled range. A sweep is refused if one is already running or asked for, if the chip is absent and a re-probe fails, or if nothing is left after the clamp. A failed point is recorded as RSSI 0 and counted. At the end, the sweep's floor is the median of all its bins.

Table 61. The RDA5807M as an instrument

Sweep	Points	Time
Narrow, ±1.5 MHz around a prediction	31	about 10 s
The whole window	201	about 60 to 70 s
Fine pass, ±0.25 MHz in 50 kHz steps	11	about 3.5 s

The candidate. Over the last sweep's bins, skipping any bin within ±0.2 MHz of a fixed feature and any bin whose stereo flag is set above the floor + 3, the firmware picks the bin with the largest margin over its local floor. The local floor uses up to 8 bins each side.

The fine pass pins the candidate down: eleven points 50 kHz apart, ±0.25 MHz around it. It skips any point within ±0.2 MHz of a fixed feature, and any point whose stereo flag is set above the last sweep's floor + 3. It keeps the highest reading: the coarse sweep has already decided this is the oscillator, and a local floor over eleven neighbouring points would measure the peak against itself. If every point was skipped or read 0, the pass found nothing. The fine pass is not clamped to the LO window (only its centre must be inside it), so it can reach 0.25 MHz past either end. Sweeps and fine passes share one request slot, so only one run is ever in progress.

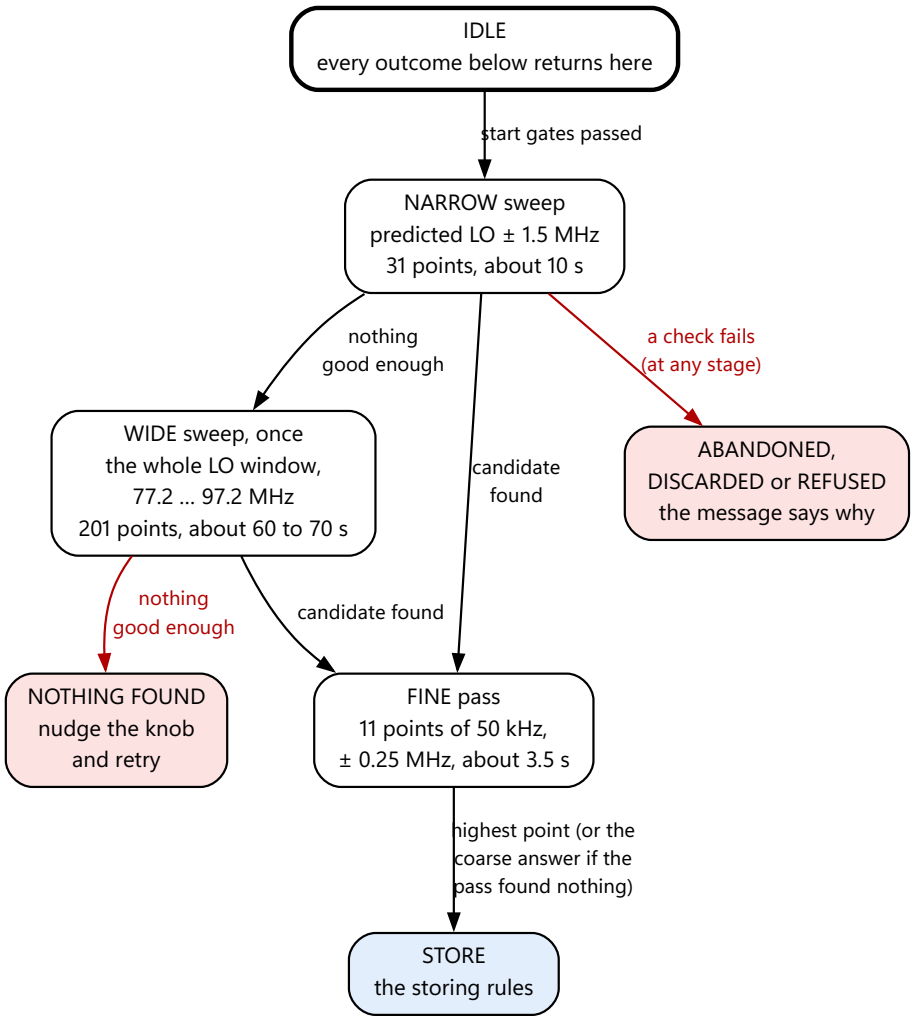
Both halves stay readable. The coarse bins and the fine pass's eleven points stay readable until the next sweep starts; only a sweep clears them. In the fine list, a point skipped as a fixed feature, or whose read failed, shows RSSI 0; a point skipped as a broadcast keeps the RSSI it read. A fine pass that loses the chip keeps no points.

Losing the chip. Five failed points in a row end the run. The chip is marked lost, and the run publishes nothing (no bins, floor 0). A re-probe tries the chip again at most once every 10 seconds, never while a run holds the bus, and runs the whole set-up if it answers. Every sweep request, fine pass request, measurement start and automatic trigger calls the re-probe. The firmware also counts every failed transfer of the last run, in a row or not.

Fixed features. The list lives in the settings, up to 16 entries, empty by default. The RDA task holds a copy, refreshed whenever settings change. It is learned by hand with the console key o, which marks the current best candidate of the last sweep as a fixed feature and saves the list. Do it with the set powered but switched off: the mains cord in, so both boards and the RDA5807M run, and the front switch off. The tube set runs only while the amplifier does, so its oscillator is then not running, and every peak is a spur (section 6.10 (p. 138), step 2). The console o or the portal's Forget the fixed features clears the list, and the clear is saved too. At start-up a stored list longer than 16, or with any entry outside 77.2 to 97.2 MHz, is cleared whole.

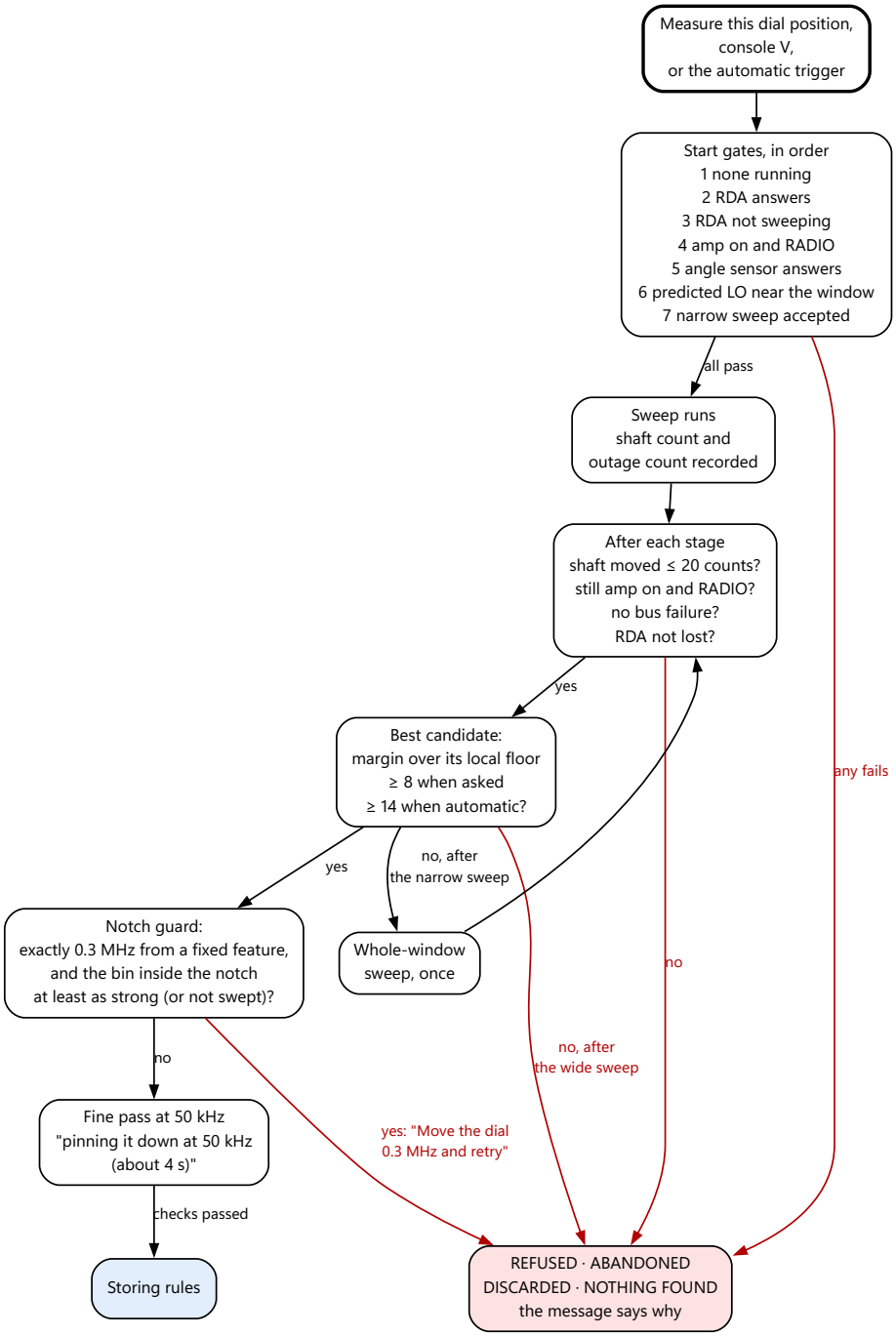
6.7 One measurement

Figure 15. The stages of one measurement



A measurement is started by a person (Measure this dial position on the portal, or the console `v`) or by the automatic trigger (section 6.8 (p. 136)). It moves through stages: a narrow sweep around the predicted LO; if that finds nothing good enough, one wide sweep of the whole window; then a fine pass at 50 kHz; then storing. Each stage is judged once its sweep has finished.

Figure 16. The checks a measurement must pass before it reaches the storing rules



Start gates, in order. Each refusal says why.

Table 62. One measurement

#	Gate	Refused when
1	No measurement already running	one is running
2	The RDA5807M is present, or answers a re-probe	it does not; the message tells “stopped answering after boot” from “not answering”
3	The RDA5807M is not sweeping	it is
4	The amplifier is on and the audio board reports the source RADIO	“the dial calibrates only while you listen to the radio - switch the amp on and select RADIO”
5	The angle sensor answers	it does not
6	±1.5 MHz around the predicted LO touches the LO window	“the dial is outside the range the oscillator can be measured over”
7	The narrow sweep request is accepted	it is refused

The predicted LO is the curve’s frequency at the current shaft position minus the IF.

Gate 4 is a listening policy, not a statement about the oscillator. The tube set runs whenever the amplifier does; the source selector only tells the audio board which sound to play. The dial calibrates, and the needle follows (chapter 5 (p. 73)), only while the radio is what you are listening to, by choice.

Only once all seven pass does the firmware record the shaft count, the angle sensor’s outage count and the stage.

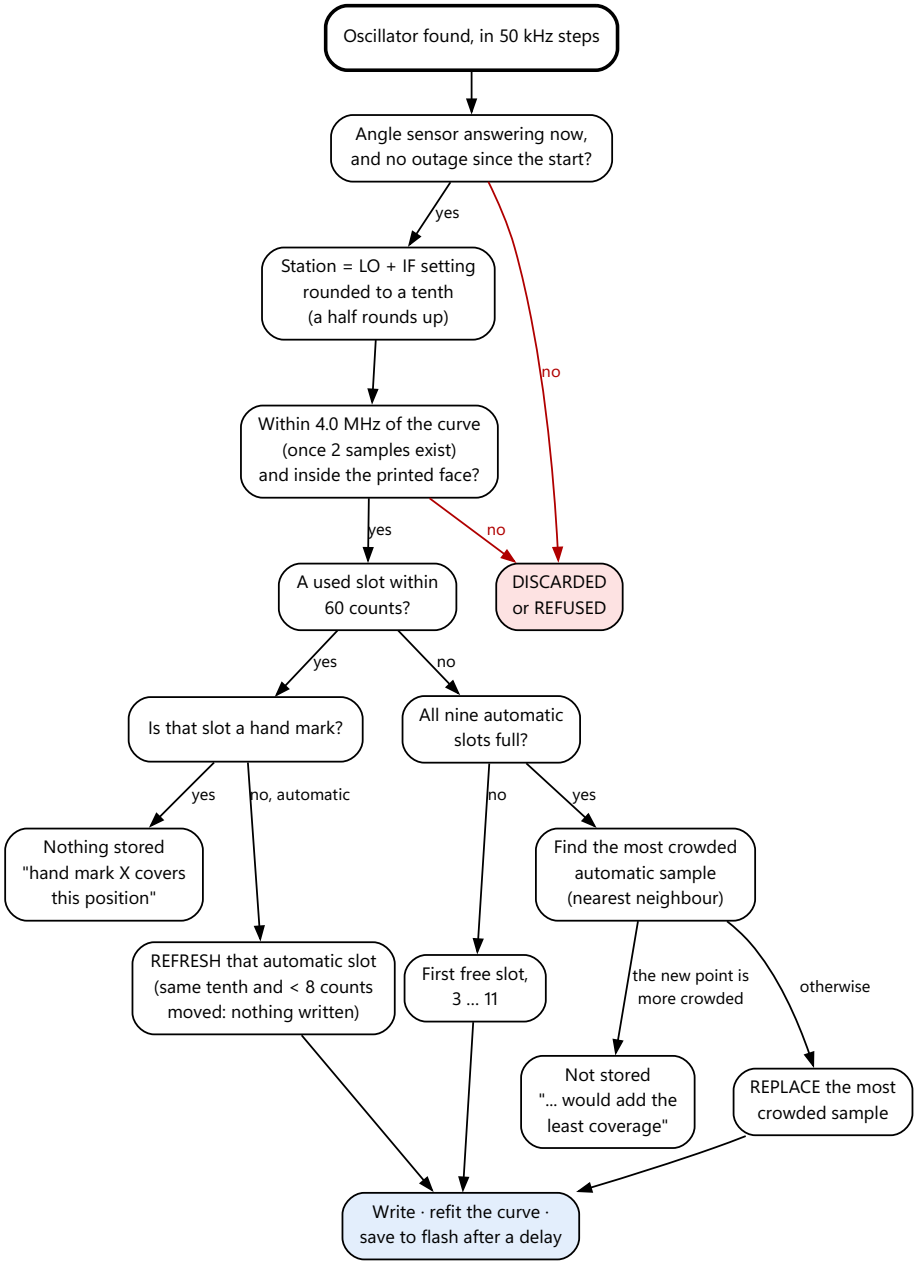
Checks after every stage, before anything is kept.

Table 63. One measurement

#	Check	Outcome
1	The shaft moved no more than 20 counts since the start (about 0.2 MHz on this tuner)	otherwise ABANDONED, “hold it still”
2	The amplifier is still on and the source still RADIO	otherwise DISCARDED
3	No bus failure during the run (unless the chip was lost: checks 4 and 5)	otherwise DISCARDED, “press measure again” (asked for) or “it will retry by itself” (automatic)
4	Fine stage: the chip was lost	ABANDONED. Otherwise the fine pass’s answer is stored, or the coarse answer if the fine pass found nothing
5	Sweep stage: the chip was lost	ABANDONED, naming the connector, “not a fixed feature”
6	The best candidate has a margin of at least 8 (asked for) or 14 (automatic)	then the notch guard, below
7	Nothing good enough after the narrow sweep	one whole-window sweep, stage WIDE
8	Nothing good enough after the wide sweep	“NOTHING FOUND in the whole window … nudge it and retry”

The notch guard. If the candidate is exactly 0.3 MHz from a fixed feature, the firmware looks at the bin one step toward the feature, inside the notch. If that bin is at least as strong as the candidate, or was not swept, the real peak may be hidden in the notch, and the measurement is REFUSED: “Move the dial 0.3 MHz and retry”. Otherwise the fine pass starts (“pinning it down at 50 kHz (about 4 s)”). If the fine pass is refused, the coarse answer is stored at once.

Figure 17. The storing rules



Storing.

1. The angle sensor answers now and has had no outage since the start, or DISCARDED.
2. Station = LO + IF, added in twentieths of a MHz, then rounded to a tenth (a half rounds up).
3. Disagreement gate, once at least 2 samples exist (hand marks included): REFUSED if the answer is more than 4.0 MHz from what the curve says at the current shaft position.
4. REFUSED if the answer is outside the printed face, `dialLow` … `dialHigh`.

5. Slot choice. The first used slot within 60 counts of the start position is "this position". If it is a hand mark, nothing is stored ("hand mark X covers this position"). If it is an automatic slot, that slot is refreshed. Otherwise the first free slot from 3 to 11 is used.
6. A full table keeps the samples spread. If slots 3 to 11 are all used and none is within 60 counts, the firmware finds the most crowded automatic sample. For each of slots 3 to 11 it takes the distance, in shaft counts, to that sample's nearest neighbour. Neighbours are every other stored sample, the hand marks included, and the new point. The smallest distance marks the most crowded sample; on a tie, the lowest slot. Then:
   - if the new point's own nearest neighbour among the stored samples is closer than that distance, the new point is itself the most crowded and would add the least. Nothing is stored: "measured f, not stored: every sample slot is full and this position would add the least coverage";
   - otherwise the new point replaces the most crowded sample, in its slot.

   Hand marks are never replaced, but they count as neighbours.
7. Nothing changed, nothing written. A refresh that gives the same tenth with less than 8 counts of movement writes nothing, and says so: "slot 3 still reads 98.6 - nothing changed, nothing written".
8. The sample is written, the curve refitted, and the settings saved to flash after a short delay. The hand offset is never touched by a sample.
9. The report: "MEASURED (or RE-MEASURED) f MHz (LO x, +m over local) -> slot n; k stored; curve: model". A replacement reads "MEASURED (replacing the most crowded sample)", and the console adds which frequency it replaced. Every accepted sample shows its evidence, the oscillator's frequency and the margin, on the console and on the portal.

Every outcome, stored or not, is printed on the console and kept as the portal's last measurement.

Why the two rules together. The automatic measurement happens where the knob rests, and the knob rests where you listen. The 60-count refresh therefore keeps re-measuring the positions actually used, which is where drift matters. The replacement rule handles a tenth listening position once all nine slots are taken: it gives up the sample that adds the least coverage, so the nine keep spanning as much of the dial as they can.

Worked example of a replacement. Hand marks sit at counts 100, 900 and 1700. Automatic samples at 200, 300, 500, 700, 1100, 1300, 1500, 1900 and 2100 fill slots 3 to 11, in that order. A measurement at count 1000 finds nothing within 60 counts.

Table 64. One measurement

Slot	Sample at	Nearest neighbour	Distance
3	200	100 (mark)	100
4	300	200	100
5	500	300	200
6	700	900 (mark)	200
7	1100	1000 (the new point)	100
8	1300	1100	200
9	1500	1300	200
10	1900	1700 (mark)	200
11	2100	1900	200

The smallest distance is 100, first reached by slot 3. The new point's own nearest neighbour is 900 or 1100, 100 counts away: not closer than 100. So slot 3 gives up its sample at 200 to the new point at 1000.

6.8 The automatic measurement

On every pass of the main loop, the firmware starts an automatic measurement when all of these hold:

- the knob has been still for 8 seconds
- no measurement is running;
- 5 minutes have passed since the last automatic attempt, successful or not (the first attempt after start-up needs no wait);
- the amplifier is on and the source is RADIO;
- the angle sensor answers;
- the RDA5807M is present (one re-probe allowed) and not sweeping;
- the predicted LO is near enough the LO window (start gate 6).

It does not look at what the needle is doing. There is no switch to turn it off. An automatic measurement needs a margin of 14, against 8 when a person asks.

A real one, worked through. The radio was playing, with the needle tracking 105.8 MHz, and the automatic measurement reported: `RE-MEASURED 106.1 MHz (LO 95.50, +36 over local) -> slot 4; 6 stored; curve QUADRATIC`. The record gives the start and the result; the middle steps follow from the code, taking the learned fixed features to be the seven-peak comb.

1. Trigger. Knob still for 8 seconds, 5 minutes since the last attempt, RADIO playing.
2. Prediction. The curve says 105.8 MHz (1058 tenths). The IF in tenths is (212 + 1) ÷ 2 = 106. Predicted LO = 1058 − 106 = 952, that is 95.2 MHz. ±1.5 MHz around it, 93.7 to 96.7 MHz, overlaps the window 77.2 to 97.2.
3. Narrow sweep, 93.7 to 96.7 MHz, 31 bins. The fixed feature at 95.9 notches 95.7 to 96.1 out of the search.

4. Candidate. The bin at 95.5 stands 36 above its local floor. 36 ≥ 14: accepted. It is 0.4 MHz from 95.9, not 0.3, so the notch guard does not apply.
5. Fine pass around 95.5: 95.25 to 95.75 in 50 kHz steps. 95.70 and 95.75 lie within 0.2 MHz of 95.9 and are skipped. The highest reading is at 95.50 (1910 twentieths).
6. Station. 1910 + 212 = 2122 twentieths = 106.10 MHz; (2122 + 1) ÷ 2 = 1061 → 106.1 MHz.
7. Gates. 106.1 − 105.8 = 0.3 MHz, within 4.0. Inside 87.9 to 107.9. An automatic slot, 4, lies within 60 counts: refresh. The value or the position changed, so it is written.
8. The hand offset is left alone. Refit: still a quadratic.

The rounding. Had the fine pass found the LO at 95.05 MHz (1901), the station would be 1901 + 212 = 2113 twentieths, 105.65 MHz, stored as (2113 + 1) ÷ 2 = 1057, 105.7 MHz. An LO at 95.10 also stores 105.7. Half of the 50 kHz resolution is lost on storage (section 6.15 (p. 149)).

6.9 Buttons and keys

The portal buttons are on the Needle tab, admin only. The console keys also work from the portal's Console tab. Appendix C (p. 328) lists every portal action, Appendix D (p. 332) every console key.

Table 65. Buttons and keys

Portal button (action)	Console key	What it does
Measure this dial position (`rda.sample`)	`V`	One measurement at the current position, stored if it passes every gate (section 6.7 (p. 131)).
Mark station A/B/C here (`tune.markA/B/C`); asks for the MHz	—	Stores (current shaft count, typed frequency) in slot 0, 1 or 2. Refused outside 87.0 to 108.5 MHz, while the angle sensor is down, or within 60 counts of another hand mark. Removes any automatic sample within 60 counts. Leaves the hand offset alone.
Clear all marks (`tune.clear`)	—	Empties all 12 slots: back to the stored straight line. Keeps the hand offset, and says so when it is not 0.
Dial −0.1 MHz / Dial +0.1 MHz (`tune.nudge`)	—	Adds to the hand offset, which slides the whole curve. ±0.1 to ±5.0 MHz per request, ±20 MHz in total. Reports the unsnapped frequency now tuned.
Reset the dial correction to 0 (`tune.nudgeZero`)	—	Sets the hand offset to 0 and refits. Apart from restoring a settings file, the only way the hand offset is erased. Answers "the dial correction is already 0" when there is nothing to do.
Drop one sample (slot 0-11) (`tune.drop`); asks for the slot	—	Empties one slot, hand marks included. Leaves the hand offset alone.
Tuner = low end / Tuner = high end	`c` / `C`	Stores the current shaft count as `calLow` / `calHigh` and records that end as measured. Refused while the angle sensor is down. Refits.

Portal button (action)	Console key	What it does
(`needle.tuneLow/High`)		
Forget the fixed features (`rda.spurClear`)	`O`	Empties the fixed-feature list and saves.
—	`o`	Marks the last sweep's best candidate as a fixed feature and saves. Refused if already known or the list is full (16).
—	`R`	The RDA5807M's state, its bus, the window, and where the LO should be.
—	`q` / `v`	A whole-window sweep (about a minute) / a narrow sweep at the prediction (about 10 s). Not a measurement: nothing is stored. A refused sweep says "sweep refused - see the RDA state in 's' (running, absent, or lost)".
—	`a`	Prints the last sweep, bin by bin, then the fine pass's points under "fine pass (50 kHz; 0 = skipped)", with the chosen one marked "<<< chosen".
—	`s`	Status: the tuning chain, the tuning sentence, the hand offset, each sample with the curve's value there and the residual (the difference), and the RDA5807M's state with the best candidate.
—	`D`	The settings dump, which includes the IF as `ifOffset=` (chapter 7 (p. 153)).
Tube set IF (a setting row)	—	The IF the firmware adds to the LO (section 6.11 (p. 139)).

FOR FIRMWARE CHANGES

> For developers: **`GET /api/rda`** (admin). The last measurement as JSON: `n`, `floor`, `if10`, `bins` as "f10 rssi flag;", `fine` as "f20 rssi;" (the fine pass's points, in twentieths of a MHz), and `spurs`. The bin flags: `.` ordinary, `^` above the floor + 5, `S` stereo pilot decoded, `X` fixed feature.

6.10 Calibrating from scratch

Chapter 3 gives the whole first-time order. This is the tuning part of it, done after the needle is calibrated (chapter 5 (p. 73)).

1. ☐ The tuner ends. Turn the knob to its low physical stop and press Tuner = low end (or `c`), then to its high stop and press Tuner = high end (or `C`). The console `s` prints both, signed, and the span. A negative value is not an error: on this radio the count falls as the frequency rises. Judge the span against the mechanism, not against the placeholder 6023: a tuning capacitor sweeps about half a turn (on this radio about 2200 counts, some 190 degrees of shaft). A span short by about a whole turn (4096) is the sign of a lost revolution; the `residue` in the `s` tuning chain is the check.
2. ☐ The fixed features. Leave the set powered but switched off: the mains cord in and the front switch off. The boards and the RDA5807M keep running; the amplifier, and with it the tube set and its oscillator, is off. Sweep the whole window (`q`, about a minute), print it (`a`), and press `o`

once per peak: each press marks the strongest remaining candidate. Stop when what is left is noise. On this radio that was seven presses.

3. ☐ Three spread positions. Switch the set on and select RADIO. Turn the knob to a position low on the dial, press Measure this dial position, and wait: about 15 seconds when the prediction is close, up to about 85 seconds when it has to sweep the whole window. Repeat in the middle and near the top. Three samples turn the stored straight line into a quadratic. A “NOTHING FOUND” means the oscillator sits under a notch: nudge the knob and try again.
4. ☐ Check it without going in a circle. Tune to a station you know that is not one of the three positions, and read the portal’s Now. A quadratic passes through its three samples whatever they are, so only a fourth position proves the curve.
5. After that, nothing is needed. The automatic measurement re-measures wherever the knob rests while the radio plays, and fills the other automatic slots. Hand marks and the dial correction are optional, and only a person’s hand ever changes them.

If the tube set’s IF is not 10.60 MHz, set Tube set IF before step 3: every sample is the oscillator plus that value.

6.11 Settings and constants

Saved settings. All are on the portal’s Needle tab (admin) unless stated, and in the settings file.

Table 66. Settings and constants

Setting	Default	Range	What it does
`ifOffset` ("Tube set IF")	10.60 MHz	10.00 to 11.50 MHz, step 0.05	The tube set's IF: station = LO + this. Stored in 50 kHz steps (default 212).
`calLow` / `calHigh`	0 / 6023 (placeholders, "not measured")	-20000 to 20000 counts	Shaft count at the tuner's low / high mechanical end. Sets the curve's domain, the stored line's slope, and the turn choice. Set by Tuner = low/high end, `c` / `C`, the portal rows or the file; typing a value counts as measuring that end.
`tunerEndsSet`	0	0 to 3	Which ends were really measured: bit 0 low, bit 1 high.
`bandLow` / `bandHigh`	88.1 / 107.9 MHz	settings file only	The two ends of the stored straight line used below 2 samples.
`tuneUsed`, `tuneP[i]`, `tuneF[i]`	empty	12 slots	The samples: 0 to 2 hand marks, 3 to 11 RDA samples. Written by the mark buttons, the measurements, drop, clear and the file.
`tuneOffset10` (the dial correction)	0	±20 MHz in total	The hand offset added to the whole curve. Moved only by Dial ±0.1 MHz, reset only by Reset the dial correction to 0 or a file restore. Nothing automatic touches it.
`spurUsed`, `spur[16]`	empty	77.2 to 97.2 MHz (LO)	The learned fixed features. Console `o` / `O`, Forget the fixed features, the file.
`showTuning`	0	0 clock, 1 tuning always, 2 tuning while tuning	The clock digits' readout mode (chapter 4 (p. 46)). Display tab, console `f`.
`tuneHoldMs`	4000 ms	500 to 20000, step 250	How long mode 2 keeps the frequency after the last movement. Display tab.
`lastAngle`	0	—	The last live shaft count, for the turn choice at start-up. Automatic, checked every 10 s.

`dialLow` and `dialHigh` (87.9 and 107.9 MHz on this dial) are in chapter 5 (p. 73), section 5.17 (p. 100). For tuning they also hold the clock digits inside the printed face and bound which samples are accepted.

In flash and in the settings file. All tuning state is part of the main board's one settings structure (chapter 7 (p. 153) covers its format and versions). In the settings file the samples are lines `tuneMark<i>=acc,f10` and the fixed features `spur<i>=f10`. A restore applies each of the two lists whole or not at all. While a freshly updated main-board firmware is on trial, nothing is written to flash: a sample stored then waits in memory until the firmware is confirmed (chapter 3 (p. 25)).

FOR FIRMWARE CHANGES

Compiled constants.

Table 67. Settings and constants

Constant	Value	File	Meaning
`TUNE_MARKS`	12	main.cpp	Sample slots. Part of the settings layout.
`CFG_SPURS`	16	main.cpp	Fixed-feature slots in the settings layout.

Constant	Value	File	Meaning
`Rda::MAX_SPURS`	16	rda.h	Fixed features the RDA task holds. Must equal `CFG_SPURS` the build stops otherwise.
`Rda::MAX_BINS`	210	rda.h	Bins of one sweep (201 plus slack).
`FINE_POINTS`	11	rda.cpp	The fine pass's points kept for inspection.
`Rda::LO_WINDOW_LO10` / `HI10`	772 / 972	rda.h	The LO window, 77.2 to 97.2 MHz: a dial of 87.9 to 107.9 MHz minus the design IF of 10.7 MHz. Compiled; not derived from the IF setting or the dial. At 10.6 MHz it covers stations 87.8 to 107.8 MHz.
`Rda::IF_OFFSET20_NOMINAL`	214	rda.h	The design IF, 10.70 MHz. Used by no code.
`LOST_AFTER`	5	rda.cpp	Failed points in a row that mark the chip lost.
`PROBE_GAP_MS`	10000	rda.cpp	Shortest time between re-probes.
dwell	300 ms minimum	rda.cpp	Wait after each tune before reading RSSI.
"tune complete" wait	60 ms	rda.cpp	Longest wait for the chip's flag.
fine pass	±5 points of 50 kHz	rda.cpp	±0.25 MHz.
notch	±2 tenths; ±4 twentieths in the fine pass	rda.cpp	±0.2 MHz around each fixed feature.
stereo honoured	RSSI > floor + 3	rda.cpp	Below that the flag is ignored.
local floor window	±8 bins	rda.cpp	Median of up to 16 neighbours.
`SMP_MIN_MARGIN`	8	main.cpp	Margin needed when a person asked.
`SMP_AUTO_MARGIN`	14	main.cpp	Margin needed for an automatic sample.
`SMP_MAX_DRIFT`	20 counts	main.cpp	Shaft movement allowed during a measurement.
`SMP_MAX_DISAGREE10`	40 (4.0 MHz)	main.cpp	Largest gap allowed between the answer and the curve.
`SMP_TRUST_AFTER`	2	main.cpp	Samples needed before the 4.0 MHz gate applies.
narrow half-width	15 tenths (±1.5 MHz)	main.cpp	Also used by start gate 6 and the console `v`.
"same position"	60 counts	main.cpp	Refresh and duplicate radius, for samples and hand marks.
"nothing changed"	same tenth and < 8 counts	main.cpp	Saves flash wear.
`AUTO_GAP_MS`	300000 (5 min)	main.cpp	Between automatic attempts.
`AUTO_STILL_MS`	8000	main.cpp	Knob still before an automatic attempt.
`TUNE_DEADBAND`	12 counts	needle.cpp	Movement detector threshold.
movement window	400 ms	needle.cpp	How long the knob counts as "being turned".
`SEAT_MARGIN`	300 counts	needle.cpp	Added to each measured end for the turn choice.
angle read period	20 ms	needle.cpp	
fit guards	ends within 50.0 to 200.0 MHz; span ≥ 2.0 MHz; quadratic never turns back	main.cpp	
unmeasured-ends pad	30 % of the samples' span, at least 100 counts	main.cpp	

Constant	Value	File	Meaning
fallback frequency	88.1 MHz + offset; valid range 10 to 300 MHz, tested on the curve before the offset	needle.cpp	
hand mark range	87.0 to 108.5 MHz	settings_table.h	
nudge	±0.1 to ±5.0 MHz per request, ±20 MHz total	settings_table.h	
panel grid	87.9 + 0.2 k MHz (odd tenths)	main.cpp	FM channels of the Americas.

The source files are under `src/s3/`.

6.12 When it goes wrong

Table 68. When it goes wrong

What you see	What happened	What the firmware does	What you do
Every measurement refused; the RDA state says “NOT FOUND on the bus”	The RDA5807M did not answer at start-up, or one of its set-up writes failed.	Refuses every request with a reason; re-probes at most every 10 s.	Fix the connection; the next request re-probes. A failed set-up write heals by itself on a re-probe.
“ABANDONED … check its connector”	The chip stopped answering during a run (5 failed points in a row).	Ends the run, keeps nothing, marks the chip lost.	The same; it heals on a re-probe.
DISCARDED, “press measure again” or “it will retry by itself”	Scattered bus errors on the RDA5807M’s bus during the run.	Keeps nothing.	Press measure again; the automatic measurement retries in 5 minutes.
ABANDONED, “hold it still”	The knob moved more than 20 counts during the measurement.	Keeps nothing.	Hold still and retry.
DISCARDED after an angle sensor outage	The angle sensor dropped out during the measurement, or is down now.	Keeps nothing.	Retry.
DISCARDED	The amplifier was switched off, or the source changed, during the measurement.	Keeps nothing.	Nothing.
“NOTHING FOUND … nudge it”	The LO sits under a notch.	Tried narrow, then wide.	Move the dial a little. Normal at about one position in six.
REFUSED, “Move the dial 0.3 MHz”	The LO sits beside a notch; the peak may be hidden in it.	Keeps nothing.	Move 0.3 MHz and retry.
REFUSED, “a different signal”	The answer is more than 4.0 MHz from the curve (2 or more samples).	Keeps nothing.	Nothing needed.
REFUSED	The answer is outside the printed face.	Keeps nothing.	Nothing.
“… would add the least coverage”, or a sample replaced	All nine automatic slots are full and the position is new.	Replaces the most crowded sample, or stores nothing.	Nothing: normal operation.
Nothing at first; console `s` shows a large residual on one sample	A wrong sample passed every gate.	Nothing: it is not detected. Stored and fitted.	Console `s` residuals (meaningful from 4 samples); Drop one sample. A later

What you see	What happened	What the firmware does	What you do
			automatic sample at that spot also refreshes it.
Every reading off by the same amount	A hand correction no longer fits the samples.	Nothing, by design.	Reset the dial correction to 0, or nudge it back.
“QUADRATIC REFUSED” or “the marks are degenerate”	The fit turns back, or the samples cannot define it.	Runs a least-squares straight line.	Drop the bad slot.
“MARKS REFUSED (…)”, red message	The fit leaves 50 to 200 MHz, or spans less than 2 MHz.	Runs the stored straight line.	Re-mark or drop the bad sample.
“tuner ends NOT measured” on every surface	`tunerEndsSet` is not 3.	Pads the samples’ span for the domain.	Tuner = low end / Tuner = high end.
Fixed features gone after start-up, or a settings file’s list refused	The stored or uploaded list was corrupt.	Clears the stored list; refuses a bad file list whole.	Re-learn with the set off (section 6.10 (p. 138), step 2).
Nothing at first	A fixed feature was learned with the set on (o does not check), and notches the LO out.	Nothing: it is not detected.	o or Forget the fixed features, then re-learn with the set off.
Every new sample off by the same amount	The IF moved (the IF transformers were realigned).	Nothing: it is not detected.	Re-measure the IF, set Tube set IF, then drop or re-measure the old samples.
Nothing: the audio no longer mutes during a measurement	Failed points in the RDA task could starve the main loop and mute the audio.	Pauses 10 ms after each failed point; five failures in a row end the run.	Nothing.
The needle holds still while you tune; the portal’s Now is blank	The angle sensor stopped answering.	Holds; retries every 2 s; re-seats the turn when it answers.	Nothing if it comes back; otherwise its wiring and cable (Hardware Bible, chapters 8 and 10).

Harm that survives a restart (BLOCKER). A wrong sample that passes every gate, a bad sample left behind a refused fit, a fixed feature learned with the set on, and a changed IF are all saved. The other rows recover by themselves, or would be blockers only without the firmware’s handling (a wrong sample stored after a bus error, an outage, a lost chip, a change of source, a set-up write that failed, an answer far from the curve, or a peak hidden beside a notch).

6.13 Design choices

- Read the tube set’s own oscillator. The LO frequency is the dial position at any position, with no station to identify: it works on a dead channel, in a basement, on a poor antenna. It wants a short, deliberately poor antenna inside the cabinet, and that selectivity helps separate the oscillator from broadcasts.
- Station = LO + IF. Measured on the bench: a 6 MHz move of the dial moved the carrier 6 MHz, in step, below the station. Low-side also puts the whole dial’s LO, 77.2 to 97.2 MHz, inside the chip’s band. The bench probe was a

stand-alone ESP32 with one RDA5807M, connected to nothing in the radio; its record, computed with the design IF of 10.7 MHz:

Table 69. Design choices

Dial	Unit	Bare carrier found	Unit	RSSI at 87.80	at 87.90	at 81.70
92.5	MHz	81.8	MHz	41	40	59
98.5	MHz	87.8	MHz	59 (+18)	59 (+19)	40 (−19)

Neither carrier had a stereo pilot or RDS (a broadcast's digital data), and both sat below the broadcast band. A closed-loop read then gave 98.60 MHz for a dial verified at 98.5: one 100 kHz step, before any fine pass existed.

- The IF is a setting in 50 kHz steps, default 10.60 MHz. The fitted IF is 10.6, not the 10.7 design value; six stations identified by ear gave 10.60 MHz five times and 10.65 once. The IF could move again if the IF coils are realigned, so it must be correctable without a reflash. Twentieths, because 10.65 cannot be written in tenths.
- The RDA5807M has its own I2C bus. Its addresses are fixed in silicon, and the angle sensor's bus stays the tuning sensor's alone.
- Band 2, AFC off, MONO cleared, "tune complete" polled, dwell ≥ 300 ms. Band 2 because the window starts below 87 MHz. AFC off so the chip cannot slide to a louder neighbour. MONO cleared for the stereo indicator. A 40 ms dwell read every channel at 12 to 23 instead of 40 to 63 and hid the LO completely. That confident false negative also made a freshly fitted antenna look useless. A raw register read of 49 on a channel the sweep had just called 15 caught it.
- The chip counts as present only when its whole set-up landed. One failed AFC write once left a receiver free to slide while every measurement counted as clean.
- A lost chip is detected and heals. Before, a chip that stopped answering stayed "present", the measurement blamed a fixed feature, and back-to-back failures starved the main loop long enough to mute the audio.
- Judge a peak by its margin over the local floor; never pick the strongest. The 79.0 MHz spur reads 63 against the LO's 56 to 59, and was the tallest feature at five dial positions; "the strongest bare carrier" once reported 89.70 for a dial at 98.5. A fixed threshold fails at the bottom of the dial. In a blind test the bench read 107.3 MHz at +10 and 88.5 MHz at only +6, where a fixed +7 reported "nothing rose" (98.5 and 107.3 had given +15 to +18). The window's low end sits at the edge of the chip's band, where it is least sensitive, and low-side injection makes that the bottom of the dial. The station there was also received more weakly. Both blind readings were right.
- The stereo flag rejects only above the floor + 3. The first inspectable sweep showed the flag on a bin at RSSI 36 against a floor of 42: a pilot "decoded" out of noise. A real broadcast stands well above the floor. As a hard gate it would one day have thrown away the oscillator.

- Fixed features are learned by hand, with the set off, never compiled in. With the oscillator off, every peak is a spur. A compiled list would claim, on a freshly erased machine, features it never measured, and blind the search there.
- No confirmation by movement. The firmware never checks that the peak moved with the dial; the learned fixed features, the margin and the 4.0 MHz gate do that job.
- Narrow, then wide once, then 50 kHz. The narrow sweep answers in about 10 s when the curve is roughly right; the wide sweep rescues a bad prediction or an LO under a notch; 50 kHz decides which side of a channel boundary the capacitor sits on. A failed fine pass keeps the coarse answer.
- Samples are stored, not coefficients, and in shaft counts, not permille. Samples can be refitted with a better model without a migration, and no float goes into flash. Permille depends on the tuner ends, so re-measuring them would change the meaning of every sample.
- A quadratic least-squares curve, with guards. The dial is curved: the local slope was 0.251 tenths of a MHz per permille between 91.3 and 98.5, and 0.288 between 98.5 and 107.3. A two-point line was exact at its marks and about 0.5 MHz out mid-band; the straight line over the tuner's travel was 1.4 MHz out at the bottom, 0.7 at the top and 0.1 in the middle. Three samples (91.3, 100.7, 107.4) predicted 98.5 at a position nobody had measured, confirmed by ear. A five-sample fit:

  Table 70. Design choices

Shaft count	Measured (MHz)	Curve (MHz)	Error
−805	107.1	107.19	+0.09
−694	105.8	105.67	−0.13
−317	100.7	100.75	+0.05
−134	98.5	98.49	−0.01
502	91.3	91.30	−0.00

  Worst error 0.13 MHz, about two-thirds of a 0.2 MHz channel. So where the error passes 0.1 MHz, the panel's snapped readout can show the neighbouring channel. A line through the same points was 2.4 times worse. The curve bends by about 0.54 MHz across the travel.
- Slots 0 to 2 are written by hand alone. When samples could refresh hand marks, parking at the ends replaced all three (88.5 → 88.1, 98.5 → 98.4, 107.3 → 107.9).
- Refresh a nearby sample; write nothing when nothing changed; demand more when nobody is watching. Drift is the reason the feature exists, so re-measuring must replace the old answer. A sampler writing every five minutes would reach about 100,000 flash writes a year. Observed margins ran +18 to +28, so +14 for automatic samples costs nothing real.
- A full table replaces its most crowded sample. With a refusal, the calibration stopped learning new positions for good once nine were taken. Spread, not recency: samples bunched in one part of the dial extrapolate

badly to its ends, and dropping the oldest would pull every sample into the favourite stations.

- Measure only while the radio is being listened to; a timid trigger; always armed. Each condition is a reason not to measure: a wrong sample costs more than a missed one. And the knob rests where the listener listens.
- The hand offset is yours, and only you erase it. It is a calibration made by ear; whether it still fits is your judgement, and there is a reset button. Two writers on one value would mean your correction and the machine's silently fighting. It is shown on the console `s` and the Needle tab, so a stale one cannot hide.
- Acceptance: 4.0 MHz after two samples, the printed face, a single-bin margin. 4.0 MHz is loose on purpose: before calibration the straight line was 1.4 MHz out at the bottom, and a tight gate would refuse the measurements that fix it. Before two samples the curve is an unchecked default.
- The 50 kHz half-step is rounded away on storage, and kept that way. Changing it needs a new settings version; nothing on the radio showed a need (section 6.15 (p. 149)).
- A fine pass landing on a rounding boundary beside a notch is left alone. The value it gives records where the oscillator is at that position.
- The notch guard. 100.7 MHz needs an LO of 90.1, inside the 90.3 notch; the search took 90.0 and stored 100.6, twice. A guard on distance alone refused 98.5, a hand mark, so the guard also looks at the bin inside the notch.
- Any bus failure during a measurement discards it. A failed point reads as RSSI 0, which drags local floors down until noise scores like the oscillator.
- No sample from a frozen or re-seated angle sensor. While the sensor is down its count is frozen, so a knob turned then passes the drift check and the frequency would be stored against the old position.
- Nothing is stored if the set left RADIO during the measurement. A measurement lasts 10 to 85 s; switching off or changing source meanwhile is ordinary use.
- The shaft's turn is chosen by the measured tuner ends. "Nearest to the last count" chose the wrong turn after more than half a turn of movement during an outage, saved it as `lastAngle`, and every later start-up inherited it.
- The dial means where the capacitor is; samples are stored unsnapped. You tune for the clearest sound, which need not be the channel centre. On a poor antenna the set sat about 0.15 MHz high of 98.5 and received it through FM capture (the stronger station wins the receiver). The RDA5807M read 98.65 MHz, and was right to. The panel, snapping, shows 98.7; the portal keeps the true value.
- The readout rules. Mode 2 snaps from the unrounded value: snapping a value already rounded to a tenth put a quarter of the dial one channel wrong (from 91.3, turning up must give 91.5, not 91.4). The portal never

clamps: past the printed face you should see that the tuning went beyond the needle.
- Everything reachable without the USB cable. The USB ground is the machine's DC-side ground, which is on mains earth (Hardware Bible, chapter 5); a cable to a computer adds a second ground path. And once the cabinet is shut, the cable is gone.
- Both halves of a measurement stay readable. The coarse sweep says where the oscillator is, the fine pass says exactly where; each is the evidence for its half of the answer.
- The two fixed-feature list sizes are tied at build time. Had the RDA task's list been smaller, learned features would have been dropped in silence.

What the bench probe taught. The stand-alone probe answered three questions:

- the oscillator can be heard (a bare carrier moved one-for-one with the dial);
- the injection is low-side;
- the chip can read the dial (a closed-loop read one 100 kHz step off, then two blind reads, both right).

It also showed the oscillator cannot be told by loudness: a fixed spur at 79.0 MHz was louder at every dial position, and only moving the dial told them apart. The firmware uses that lesson by learning what does not move (the fixed features, with the set off) and masking it; it does not itself check that a peak moves between two sweeps. Its other lessons are built in: judge a rise against the local floor, not a fixed threshold; dwell 300 ms, not 40; coarse 100 kHz steps to find the oscillator, fine 50 kHz steps to pin it down. A whole evening of "nothing on the bus" on that rig turned out to be one flaky jumper wire; the diagnostics had said the pin map and the chip were fine.

6.14 Tried and rejected

FOR FIRMWARE CHANGES

- Mapping real stations and snapping to the nearest. It needs an outdoor antenna, the opposite of the short inside wire that reading the oscillator wants.
- Comparing the RDA5807M's audio with the tube set's. The most work for the least gain: the chip has no digital audio output, so its audio would need an analog path into a spare converter, and the comparison would have to run on the audio board, which holds the tube set's audio, far from the main board where every constant it would correct lives.
- A straight line from `bandLow` to `bandHigh`, typed by hand. Right only if the needle reached both ends; it read 104.6 at 107.3. It survives only as the fallback below two samples.

- Two-point “Mark station A / B”, a line through two marks kept in memory. Exact at the marks, about 0.5 MHz out mid-band on this curved dial.
- “Tuned to this station NOW”, shifting the line to one point. Dropped with the line; the dial correction does that job on the curve.
- Samples in permille. Re-measuring the tuner ends silently redefined every sample.
- Earlier fit domains: the samples’ own span (froze the ends of the dial); the placeholder ends (2.7 times the real travel, which pulled the curve’s turning point inside and refused good marks); twice the samples’ span (cut inside the real travel); a 30 % pad over measured ends (it reported “reaches 85.96 - 112.43 MHz”).
- High-side injection. Assumed at first: at dial 98.5 the LO would have been at 109.2 MHz, outside the window swept, so the first sweeps found nothing; the top of the dial would have needed an LO near 118.6 MHz, beyond the chip’s band.
- The IF as a compiled 10.7 MHz. Every sample stored 0.1 MHz high.
- A 40 ms dwell, band 0, the strongest carrier, a fixed threshold. Each hid the LO or picked a spur.
- AFC at its default (on), MONO forced. Every sweep was taken by a receiver free to slide.
- The stereo flag as a hard gate. The first inspectable sweep showed it set 6 below the floor.
- Automatic learning of fixed features (“anything that never moves is a spur”), and compiling this cabinet’s comb into the firmware.
- Confirming the LO by a second sweep at another position, expecting it to move with the dial. Never built; the fixed features, the margin and the 4.0 MHz gate do the work.
- A calibration session: one wide sweep, then five narrow ones as you move across the dial. Each press measures one position instead.
- “Require a coherent multi-bin peak”, “reject outside 87.0 to 108.5”, “a couple of MHz” of disagreement. Shipped instead as a single-bin margin, the printed face and 4.0 MHz.
- Measuring whenever the amplifier is on, Bluetooth included; an on/off switch for the sampler; measuring only while the needle is idle.
- The RADIO gate worded as a hardware fact (“the oscillator is not running”). The tube set runs whenever the amplifier does, so the words were wrong; it is a listening policy.
- Refusing a second sample at the same position. Replaced by the refresh.
- A full table refusing every new position (“drop one (tune.drop) to make room”). The calibration stopped learning until someone dropped a sample by hand.

- Samples allowed to refresh hand marks, and **v** allowed to claim slots 0 to 2.
- Samples zeroing the hand offset: first every sample, then a requested sample, a hand mark, a drop and a clear. That erased a correction set by ear, unasked.
- The 50 kHz path through the 100 kHz tuning call (a “fine” flag). It could never address 50 kHz.
- Fine passes clearing the sweep’s bins. A refined measurement left nothing to inspect.
- The notch guard on distance alone. It refused 98.5, a hand mark.
- A flag raised before the thing it announces. Twice: the measurement’s stage raised before its sweep was requested, and “sweeping” reporting false for the 100 ms the RDA task takes to notice a request. Both let a stage judge the previous position’s bins: 91.4 MHz was stored while the set was on 98.5. The fit’s guard caught it (“QUADRATIC REFUSED: turns back on itself”); a strong 88.5 broadcast was blamed first.
- A sweep of an absent chip with no pause. About 10 s of main-loop starvation per sweep.
- Across an angle sensor outage: movement added up across the gap (once injected a whole revolution); then movement dropped (shifted the frame until restart); then the nearest turn (wrong beyond half a turn).
- The readout gated on the magnet-detect bit. It hid a good reading.
- A movement detector applied to one 20 ms read. The 12-count deadband became a speed floor of about 53 degrees per second: a slow turn never registered, and the needle tracked while the panel did not.
- The stand-alone bench probe itself: a tool to reach the first release, now retired and not published; its results are in section 6.13 (p. 143).

6.15 Known limits

- The very top of the dial cannot be measured. The LO window is compiled as 77.2 to 97.2 MHz, the printed dial minus the design IF of 10.7 MHz. At the fitted 10.6 MHz, 107.9 MHz needs an LO at 97.3 MHz, one step outside. So measurements cover stations 87.8 to 107.8 MHz. At the very top, a measurement finds nothing or only the edge of the peak. Another IF setting moves the unreachable band. The curve still extrapolates over the last tenth, and the dial reads true up to 107.3 MHz, the highest station receivable where this radio lives. Widening the window would let in the fixed features just above 97.2 MHz (98.7, 101.6 and 104.4 MHz), which would then have to be learned like the others.
- 50 kHz is lost on storage. Samples are stored in tenths: an odd-twentieth result is stored 50 kHz high, about +25 kHz on average, at most half a channel step.

- The notches cost about a sixth of the window. “NOTHING FOUND” at roughly one position in six is normal.
- The comb’s peaks sit at multiples of 2.8224 MHz, the audio board’s I2S clock rate (the clocks of its digital audio link, which run even while that board sleeps).
- No automatic detection of a wrong sample that passes every gate. The `s` residuals mean nothing at exactly three samples: a quadratic passes through all three.
- A mono broadcast near the LO is told apart only by its margin, the notches and the 4.0 MHz gate.
- Crowding is judged by shaft counts only. A stale sample in a sparse part of the dial is kept while a fresh one in a crowded part is given up; a stale sample is replaced only when you tune near it again.
- Fixed-feature learning is console-only (`o`, also through the Console tab). `o` does not check that the set is off, and marks any peak with a positive margin: you judge from `a` when to stop.
- The fine pass’s list does not mark a point rejected as a broadcast. Such a point keeps its RSSI while the legend says “0 = skipped”; the `fine` list in `/api/rda` has the same gap.
- The sweep’s results are not locked between tasks. A console dump, or `/api/rda`, during a sweep can show a mix of two sweeps, and a status line read mid-rewrite can mix old and new text. The fixed-feature list is replaced in an order that keeps a reader inside valid entries; at worst one sweep would judge one frequency against the old or new list. None of this has been seen to matter.
- The outage message always names the tuner’s ends. “AS5600 gap - angle re-seated on the turn inside the tuner’s ends” prints even when the ends were not measured and the nearest turn was used.
- Accepted race: a hand mark and an automatic sample landing at the same instant, and settings written from both cores without a lock.
- Hard-coded for this radio’s region and dial: the odd-tenth panel grid, the 50 to 200 MHz guard, the 88.1 MHz fallback.
- The hand offset sits on top of the measured curve. A non-zero offset makes every readout differ, on purpose, from what the RDA5807M measured; the prediction, the 4.0 MHz gate and the `s` residuals include it. The portal state carries it as `toff`.
- Drop one sample also accepts slots 0 to 2, so it removes a hand mark too. It is the only way to remove one hand mark without clearing all of them.
- Early refreshes moved several samples by a tenth or two (107.4 → 107.2, 98.7 → 98.4, 105.8 → 105.7), but those first readings predate AFC off and the fine pass, so they are not drift.
- Never exercised on the radio: the discard after a bus error, the lost path, a re-probe after a real loss, the full-table replacement (the table has never been full), and a measurement showing both the coarse and the fine lists.

6.16 Changing it safely

FOR FIRMWARE CHANGES

What must stay true.

1. One evaluator. Only `Needle::tuneFreq10Raw()` and its wrappers `tuneFreq10()` (rounded) and `tuneFreq10f()` (unrounded) turn a shaft count into a frequency.
2. One fit. `fitTuneCurve()` in main.cpp, called from `applySettings()`. Hand marks and samples share it. It solves the normal equations by Gaussian elimination with partial pivoting (`solveLS()`), on counts centred on their mean.
3. **`cfg`** is the origin of the samples and the fixed-feature list. Modules hold copies pushed by `applySettings()` (`Rda::spurSet()`); never write the RDA module's copy directly. `spurSet()` publishes the count last when the list grows and first when it shrinks.
4. Settings are append-only: bump `SETTINGS_VERSION` and add a migration (chapter 7 (p. 153)). `TUNE_MARKS` and `CFG_SPURS` are layout, not tuning knobs. `Rda::MAX_SPURS` must equal `CFG_SPURS` a `static_assert` in main.cpp stops the build otherwise.
5. A failed I2C read is never a value, and a run with any failure (`runFailures()`) is never stored.
6. Raise a flag only after the thing it announces: `gRunning` before `gWant` is cleared; the sampler stage only after the request was accepted; `sweeping()` is `gRunning || gWant`.
7. **`rdaTask`** outranks **`loop()`**: every path must block (`pointFailed()` yields 10 ms), and nothing may make `loop()` wait more than 2 s. `loop()` is also under the 15 s task watchdog.
8. Add the IF in twentieths (`cfg.ifOffset20`), then round. `ifOff10()` is for predictions and printouts only.
9. Slots 0 to 2 are written only by `tune.markA/B/C`, `tune.drop`, `tune.clear` and a file restore. The full-table replacement in `sampleStore()` looks at slots 3 to 11 only.
10. The hand offset `tuneOffset10` is written only by `tune.nudge`, `tune.nudgeZero` and a file restore.
11. Refusals go through `refuse()` or `smpSay()`, so the portal shows them in the refusal colour. `smpSay()` also sets `gSmpNote`, the portal's last measurement.
12. `i2cOk()` turns true only after the re-seat (`seatTurn()`); captures refuse while it is false.
13. A dwell of at least 300 ms, whatever the caller asks.
14. Only a sweep (`requestSweep()`) clears the bins and the fine points; a refinement (`requestRefine()`) replaces only the fine points.

Where things live. `readAs5600()`, `readStatus()`, `seatTurn()`, `seedAccumulator()`, `tuningActive()` and `encoderGaps()` are in src/s3/

needle.cpp. `initChip()`, `reprobe()`, `tuneTo()`, `tuneTo20()`, `bestCandidate()`, `isSpur()` / `isSpur20()`, `markLost()` and `rdaTask` are in src/s3/rda.cpp. `sampleStart()`, `sampleTick()`, `sampleStore()`, `autoSampleTick()`, `radioLive()`, `loInWindow()`, `besideSpur()`, `tuneStateLine()`, `tuneModelName()`, `fitRefused()`, `fitDemoted()`, `installStoredLine()`, `purgeCorruptSpurs()`, `portalStateJson()`, `portalRdaJson()` and `updateDisplay()` are in src/s3/main.cpp; `tuneTroubleNote()` is in src/s3/settings_table.h. The start-up order is in chapter 4 (p. 46).

Traps.

- The count falls as frequency rises on this radio. Compare sizes and slope signs, never order. `calLow`, `calHigh`, `tuneP[]` and `lastAngle` are signed and legitimately negative: print them signed (printed unsigned, a measured end of −1173 once read as 4294966123).
- Every notch blinds ±0.2 MHz, and there are seven here. Widening `isSpur()` blinds more of the dial.
- `tuneStateLine()` and `tuneModelName()` are the only sources of the tuning sentence. A refusal is the flag `fitRefused()`, never a word parsed from the sentence.
- The stored straight line always reports the reassuring reach 88.1 to 107.9 (section 6.5 (p. 127)).
- The portal is the only interface once the cabinet is closed. Anything printed only on the console has not been reported.
- A new IF setting does not move the LO window, which is compiled (`Rda::LO_WINDOW_LO10` / `HI10`). `purgeCorruptSpurs()` checks the stored list against that window, so it must follow any new window.
- A judgement that cannot be observed either works or silently throws the signal away, and nothing says which. Keep every judgement visible in `/api/rda` and the console `a`: that is how the stereo flag's fault was found.
- `radioLive()` is a listening policy, not a test of whether the oscillator runs. Do not reuse it where the oscillator's state matters.
- `tuneTo()` still takes a `fine` flag that every caller sets to false; setting it true only re-addresses the same 100 kHz points. The 50 kHz path is `tuneTo20()`.

How to test a change.

- `GET /api/rda` after a measurement, or the console `a`: every coarse bin with its judgement, then the fine pass's points.
- The console `s`: the tuning chain (the residue `(acc - raw) & 4095` must never change while the machine runs), the tuning sentence, and the per-sample residuals (meaningful from 4 samples).
- A fit test that is not circular: predict at a position that is not a sample, and confirm by ear.

- The fixed-feature baseline: the set powered but switched off (cord in, front switch off), `q`, then `a`. Every peak is a spur.
- The full-table rule: with all nine automatic slots full (console `s` lists them), measure a new position; the answer names the replacement, or says the new point would add the least coverage. Work the expected slot out by hand first (section 6.7 (p. 131), Storing, step 6).
- The untested paths: on the bench, break the RDA5807M's bus during a sweep and watch for the discard and the "ABANDONED" line, then restore it and watch for the re-probe.
- Build both boards with the pinned toolchain, and update the main board through the portal (chapter 3 (p. 25)).
- Test without a USB cable to a mains-earthed computer: the cable adds a second ground path.

7. Settings

A setting is any adjustable value that must survive a power cut: the volume law, the gains, the clock and panel brightness, the needle's motion, its soft limits and calibrations, the tuning curve, the Bluetooth policy, the lamp levels. Each of the two boards keeps its own, in its own flash, and the main board shows both sets on the web portal as if they were one.

This chapter explains where settings live, when a change reaches flash, what an update or a rollback does to them, and how to back them up and restore them with the settings file. That file is the only undo. Appendix A (p. 309) lists every setting of both boards, with its default and range.

7.1 What a setting is and where you change it

The S3 (the main board) owns everything about the display, the lights, the needle and the tuning. The A32 (the audio board) owns everything about sound and Bluetooth. A person meets settings in four places:

Table 71. What a setting is and where you change it

Where	What it changes	Who may use it
The web portal, tabs Audio, Display, Lights, Needle and Bluetooth	Every setting that is one value with one range, shown as a slider, a box, a checkbox or a drop-down. Each tab also has buttons that change settings as a side effect (for example Set low limit here).	A guest: volume, mute and the five brightness levels (`brightOn`, `brightOff`, `panelTuning`, `panelIdle`, `panelOther`) — these settings; the guest's Bluetooth actions are separate (chapter 8 (p. 186), section 8.6.1 (p. 199)). The administrator: everything.
The settings file, on the System tab: Download, and Upload what is below	Every portal setting, plus a few values that are not one number (the tuning samples, the fixed features, the fallback band).	The administrator only.
The USB consoles of both boards	A few values, by single keys. The cabinet is shut in normal use, so this is a bench tool. The main board's console is also reachable from the portal's Console tab (chapter 8 (p. 186)).	Whoever holds the cable.
The front volume knob	The audio board's volume, directly.	Anyone.

The firmware also changes some settings by itself: the saved shaft angle, the learned WiFi transmit power, the automatic tuning samples, the index calibration's results, and the two clean-ups at start-up (section 7.3.3 (p. 159)).

Why the guest gets those seven. The portal's two roles are split by damage, not by secrecy. A guest sees every value, but may move only what is obvious and reversible. Everything else changes what the machine is.

When a change takes effect. At once. It is written to flash about two seconds later, once the needle is standing still (section 7.3 (p. 156)). Just after a firmware update of either board, the new firmware runs on trial until it has confirmed itself, about a minute later (chapter 3 (p. 25), section 3.11 (p. 32)). Until then the board that was updated writes nothing, and the change waits in memory.

7.2 Where settings are stored

Both boards keep settings in NVS (non-volatile storage), the small key-value store that the chip's software framework keeps in a flash partition. A namespace groups keys; a value can be a number, a string, or a blob (a block of bytes). Neither board keeps settings in a file system.

Table 72. Where settings are stored

Board	Namespace	Key	Holds	In the settings file?
S3	`amb3`	`cfg`	All the main board's settings, as one 220-byte blob.	Yes, except the few values listed in section 7.6.1 (p. 167).
S3	`net`	`ssid`, `pass`, `ntp`, `tz`	The house WiFi network's name and passphrase, the time server, the time zone.	No. Changed on the portal's Network card; the main board's console `y` sets the WiFi name and passphrase only (chapter 8 (p. 186)).
S3	`auth`	`users`	The portal accounts: names, salted SHA-256 password hashes, administrator flags.	No. Changed on the portal's account cards (chapter 8 (p. 186)).
A32	`amb`	`audio`	The audio settings, one 15-byte blob.	Yes.
A32	`amb`	`bt`	The Bluetooth settings, one 12-byte blob.	Yes.
A32	`amb`	`potMin`, `potMid`, `potMax`	The volume knob's calibration, one 16-bit number each.	As a comment only; never read back.

NVS sits at the same flash address on both boards, so stored settings survive a change of partition table. What a USB upload, a full chip erase or an update over the air does to the flash is in chapter 3 (p. 25), section 3.5. The needle's remembered position is not a setting: it lives in memory that survives a software restart but not a power cut (chapter 5 (p. 73), section 5.14 (p. 95)).

> Note Set your own time zone. The compiled network defaults include a fixed time zone. Change it for your build (chapter 8 (p. 186)).

What survives what.

Table 73. Where settings are stored

What happens	Main-board settings	Audio-board settings
Power cut, or a restart, more than 2 s after the last change, with the needle still	Kept.	Kept, but mute is always off after a start, and the knob's first reading sets the volume (section 7.4.3 (p. 162)).
Power cut within 2 s of a change, or while the needle is still moving	That change is lost; the old value is intact.	Within 2 s of a change: that change is lost; the old value is intact. The needle does not hold the audio board's save.
Power cut during the flash write itself	The firmware relies on the ESP-IDF flash storage (NVS) keeping the old value when a write is interrupted. That is the library's behaviour, not this firmware's.	The same.
An update over the air of that board	Kept. Nothing is written during the new firmware's trial minute; a change made in that minute is lost if the power goes before confirmation.	The same. On the audio board a restart in that minute also loses it.
An update that adds main-board settings	Kept; the stored settings are converted, and the new ones start at their defaults (section 7.5.1 (p. 164)).	—
An update that changes the size of the audio or Bluetooth structure	—	That structure returns to its defaults. The knob calibration survives. Download the settings file first (section 7.5.5 (p. 167)).
A rollback after an update	The previous firmware finds its own settings, untouched.	The same.
Older firmware flashed over newer settings	Locked: the machine runs on defaults and writes nothing (section 7.5.4 (p. 166)).	No lock; a structure of the wrong size returns to its defaults.

At start-up, the main board:

1. reads its blob (or converts an older one, or uses the defaults, or locks);
2. runs two clean-ups once (section 7.5.3 (p. 165));
3. sets the tuner ends before it reads the saved shaft angle, so that the angle picks the right turn;
4. applies everything.

The audio board's settings reach the main board later, after the two boards greet each other on the link (section 7.4.4 (p. 163)). The audio board, at its own start-up:

1. asks whether its own firmware is on trial;
2. loads its defaults, its knob calibration and its two blobs;
3. turns mute off, and applies them.

Its first knob reading then sets the volume.

7.3 How a change is saved (the main board)

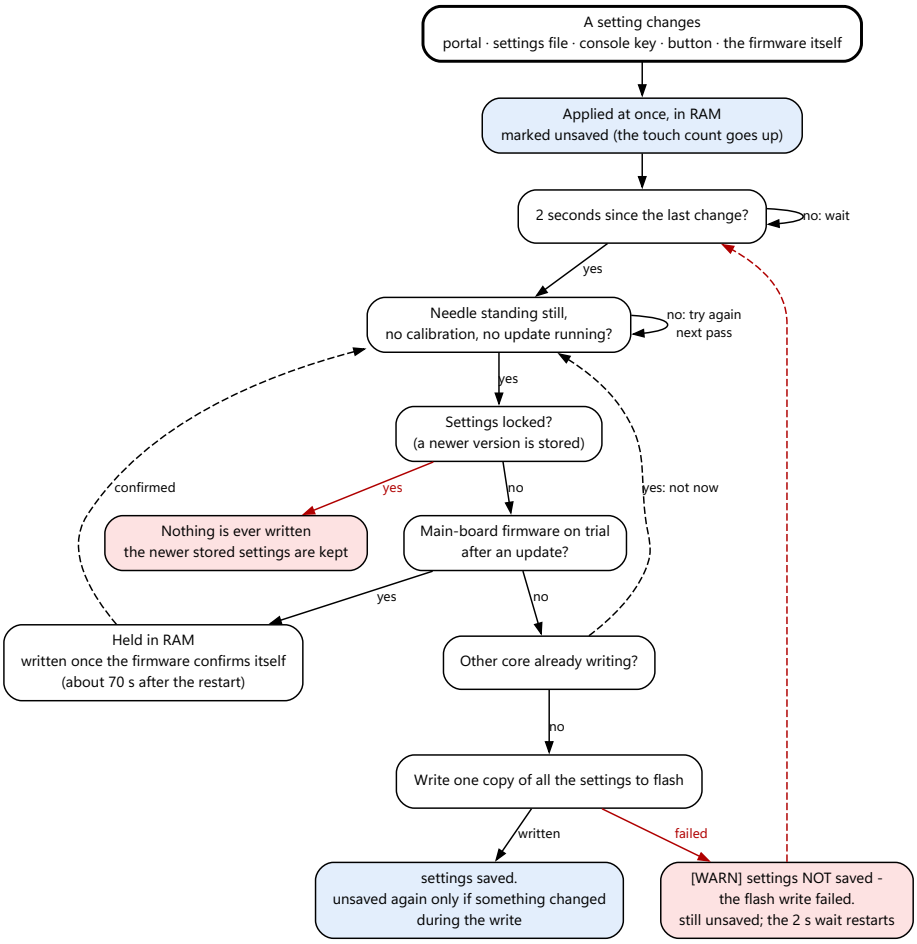
7.3.1 The rules

1. A change applies at once, in memory, and is marked unsaved.
2. It is written two seconds after the last change. Each new change restarts the wait. This wait (a debounce) exists because flash traffic is the one

load that visibly disturbs the clock display. Without it, a slider dragged across its range would write on every pixel.

3. It is written only while the needle stands still: its speed exactly zero, no needle calibration running, and no main-board update being received. Writing to flash stops the chip's flash cache, and with it every piece of code that does not run from internal RAM, including the needle's step emitter. Measured during a write without this guard: 383 µs of step jitter and a display timing error of 1257 µs, against 49 µs and 8 µs with it.
4. Nothing is written while a newly updated main-board firmware is on trial. Changes wait in memory, still unsaved, and the ordinary two-second save writes them on the first pass after the firmware has confirmed itself, about 70 s after its restart. A rolled-back firmware therefore never meets settings written by the firmware that replaced it. Chapter 3, section 3.11.5 (p. 34), tells the whole story.
5. Nothing is written while the settings are locked (section 7.5.4 (p. 166)).
6. A failed write is retried. The settings stay unsaved, the two-second wait restarts, and the console prints `[WARN] settings NOT saved - the flash write failed.` A good write prints `settings saved.`
7. A change that lands during a write stays unsaved, whichever core made it, and is written on the next pass.

Figure 18. How a change on the main board reaches flash



Worked example: dragging a Lights slider.

Table 74. The rules

Time	Unit	What happens	Unsaved?
0.0	s	The portal sets `panelIdle` to 140. Applied at once.	yes
0.4	s	The slider moves again, to 150. The two-second wait restarts.	yes
2.4	s	Two seconds of quiet, but the needle is following the knob: wait.	yes
3.1	s	The needle stops. One copy of all the settings is written; `settings saved.`	no

7.3.2 The buttons that force a save

Table 75. The buttons that force a save

Button	What it does
Save settings now (System tab, Machine card)	Writes at once, retrying until nothing is left unsaved. Answers “saved”, or the reason it could not (section 7.3.4 (p. 159)), shown as a refusal.
Reboot the main board (asks to confirm)	While the main board’s firmware is on trial: restarts at once and answers `rebooting - this firmware was still on trial, so the previous firmware comes back`. Nothing was saved during the trial, so there is nothing to wait for, and this button stays the one way to undo a bad update. Otherwise: saves first. If the save is refused and the settings are not locked, it stops the needle and retries every 100 ms, up to 15 times. Still refused: the needle resumes and the answer is HTTP 409 `not rebooting - <why>`. While locked, the reboot goes ahead, since nothing would be written anyway.
The end of a main-board update	Saves, then restarts even if the save failed; the failure is printed on the USB console only. The new firmware then boots on trial. An upload sent while the running firmware is itself still on trial is refused before any of this (chapter 3 (p. 25), section 3.11.4 (p. 34)).

7.3.3 What the main board saves by itself

- The shaft angle (`lastAngle`), every 10 s when it has changed, and only while the angle sensor answers. It lets the next start-up choose the right turn of the tuning shaft (chapter 6 (p. 120)).
- The learned WiFi transmit power (`wifiTxQ`), whenever the WiFi driver’s value differs from the stored one (chapter 8 (p. 186), section 8.4 (p. 197)).
- Automatic tuning samples, in slots 3 to 11 (chapter 6 (p. 120)). They never touch the hand marks or the hand offset.
- The index calibration’s result: the band, and the soft limits moved into the new frame (chapter 5 (p. 73), section 5.12 (p. 92)).
- The two clean-ups at start-up (section 7.5.3 (p. 165)).

All of these follow the same rules: two seconds of quiet, the needle still, not on trial.

7.3.4 Why “Save settings now” says no

A refused save gives one of five reasons:

Table 76. Why “Save settings now” says no

Reason	What to do
The stored settings are from a newer firmware.	See section 7.5.4 (p. 166).
The firmware is on trial; settings are written once it is confirmed. The portal shows `NOT saved yet - this firmware is on trial after an update; settings are written once it is confirmed (about a minute)` the console shows `save held - this firmware is on trial; settings are written once it is confirmed.`	Wait about a minute.
The needle is moving, or a calibration or an update is running.	Wait for the needle to stop.
The flash write failed.	It retries by itself.
The settings kept changing while being written.	The ordinary save writes them once the changes stop.

7.3.5 How it works inside

FOR FIRMWARE CHANGES

For a firmware modifier. All of this is in `src/s3/main.cpp`.

- **`settingsTouch()`** means “something changed”. It records the time (`dirtyAt`) and atomically adds one to the touch count `gTouchGen`. Every writer calls it after the change.
- **`gSavedGen`** is the touch count that the last good write copied. **`settingsDirty()`** is `gTouchGen != gSavedGen`. “Unsaved” is worked out from the two counts, not stored as a flag, so a change that lands during a write, on either core, is never marked saved by that write.
- **`writeSafe()`** is true only when the needle’s speed is exactly 0, no needle calibration runs and no main-board update is in progress.
- **`settingsWrite()`** is the only function that writes the `amb3` blob:
  1. refuse if the downgrade lock is set;
  2. refuse, silently, while this firmware is on trial (`s3ImageOnTrial()`);
  3. one writer at a time: an atomic exchange on a `busy` flag. The loser sets `gWriteBusy` and returns false (“not now”), because the portal’s reboot (core 0) and the main loop (core 1) could both open the same storage handle;
  4. read `gTouchGen` before copying;
  5. copy the settings into a static 220-byte snapshot;
  6. open `amb3` and write `cfg` from the snapshot, checking both results;
  7. on failure: stay unsaved, restart the two-second wait, print the warning;
  8. on success: `gSavedGen` = the count read in step 4; print `settings saved.`
- **`settingsFlush()`** runs on every pass of the main loop: if unsaved, and 2000 ms since the last touch, and `writeSafe()`, then `settingsWrite()`.
- **`settingsForceFlush()`** (“Save settings now”; the portal calls it as `settingsFlushNow()`): nothing unsaved returns true. Locked, on trial,

or not `writeSafe()` return false. Otherwise it writes until nothing is left, up to 40 attempts: if the other core is writing it waits 25 ms and tries again; a real flash failure returns false; still unsaved after 40 attempts returns false. Its callers act on "true" at once, so true must mean "nothing is unsaved".
- **`gSaveWhy`** holds the reason for the last refusal, and `settingsSaveWhy()` hands it to the portal.

In the worked example above, the touch count goes from 41 to 42 at 0.4 s while `gSavedGen` is 40; the write at 3.1 s reads 42 before copying and sets `gSavedGen` to 42.

7.4 The audio board's settings and the main board's mirror

The audio board owns and saves its settings. The main board keeps a mirror, a copy it reads from the audio board, so that the portal shows what is actually in the machine rather than what it last asked for. This section is the one place in the book that sets out how the audio board saves.

7.4.1 What the audio board stores and how it loads

It stores three things in the `amb` namespace: the audio structure, the Bluetooth structure, and the three knob-calibration readings `potMin`, `potMid`, `potMax`.

At start-up it fills the compiled defaults, reads the three knob keys (defaults 60, 0 meaning "centre not measured", and 3990), then reads each blob only if its stored length equals the structure's size exactly. There is no conversion on this board: any change to either structure means that structure's defaults. Last, mute is forced off: mute is never restored. A saved mute once made the set start in silence with nothing on the front to say why.

7.4.2 How the audio board saves

- The same two seconds. A change marks the settings unsaved; two seconds after the last change the board writes both blobs and the three knob keys.
- Every result is checked: opening the store, and the byte count of each write. Only when all of them succeed is the change counted as saved, and `settings saved.` printed. On any failure the change stays unsaved, the two-second wait restarts, and `settings NOT saved - the flash write failed; retrying` is printed on the audio board's USB console and sent to the main board, whose console prints it as `[A32] ...`.
- No motion guard. The audio board drives no motor.
- Nothing is written while a newly updated audio-board firmware is on trial. Changes made meanwhile — from the portal, a knob calibration or the audio board's console — apply at once and wait in memory, still unsaved. The firmware confirms itself a minute after the main board's first greeting, or after five minutes if the main board never spoke, and never while its link self-test has failed (chapter 3 (p. 25), section 3.11.2). At that moment it

writes what was held. When that write succeeds it sends `settings changed during the trial are now saved`, which the main board's console prints next to `[A32] image confirmed (...)`. If that write fails, the ordinary failure path above takes over.

- The hold is silent. While it lasts the portal answers "ok" to an audio-board edit as usual: the main board keeps no flag for the audio board's trial, and sees it only as text in the audio board's start-up report (chapter 9 (p. 224)). Only the log line at confirmation shows that anything waited.
- Why the hold. An audio-board update that changes the size of either structure cannot save its new layout before it has proved itself, so a rollback always finds blobs the previous firmware can read. The cost: a change made during the audio board's trial is lost if the power goes, or the audio board restarts, before confirmation.
- Reboot the audio board forces a save before the restart. If that write fails, the board restarts anyway and the change is lost. During a trial the forced save writes nothing, by design: a restart on trial is a rollback, so there is nothing to keep.
- During an audio-board update, mute is forced on and restored afterwards to what it was.
- Bounds. When it applies them, the audio board clamps the volume law to 1.0–4.0, the balance to −100…+100 and every gain to −60…+30 dB. Fade times are not clamped there; a fade time of 0 acts as 1 ms. The main board, the only remote sender, clamps every value first.
- Who writes: the main board's audio and Bluetooth messages (portal edits and file uploads), a knob calibration, and the audio board's console keys `+`, `-`, `m`, `t`, `T`, `p`, `P` and `c`.

7.4.3 The volume knob and the stored volume

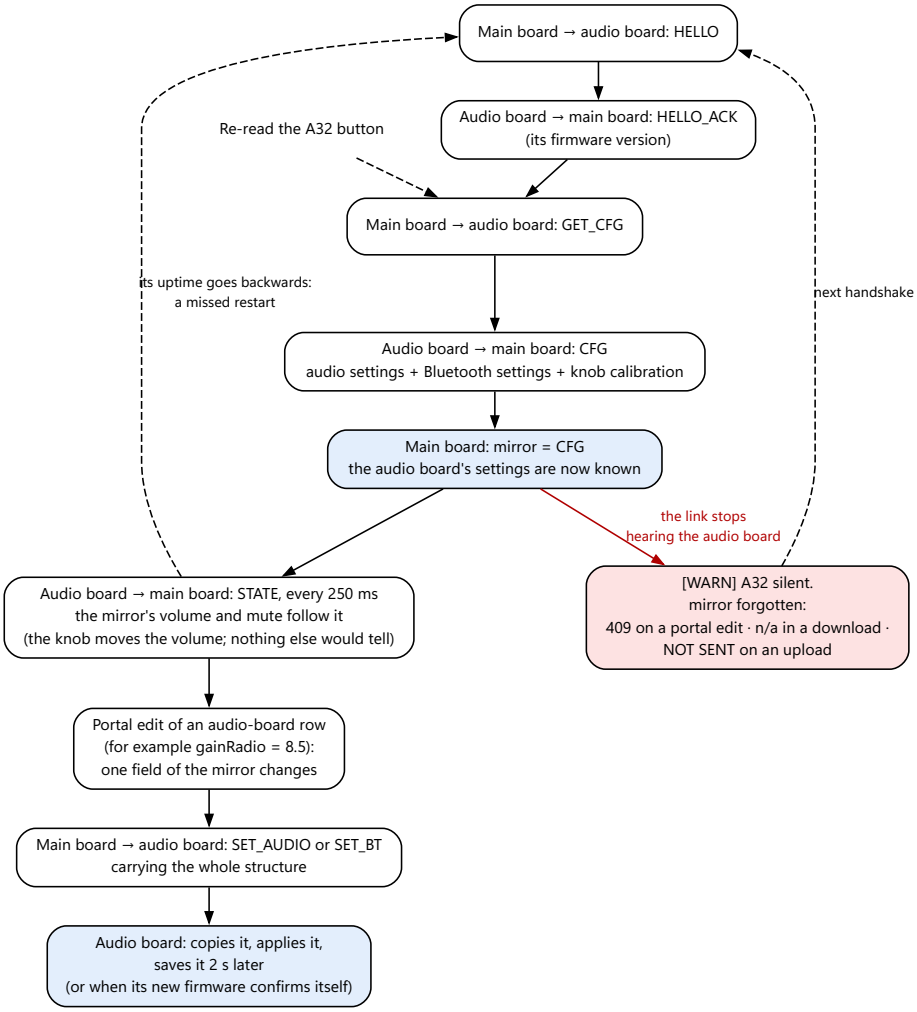
The audio board reads the knob every 50 ms, maps the reading through its three-point calibration to the volume scale 0–255, and writes the volume without marking the settings unsaved. The first reading after every audio-board start always applies; later readings apply only when the raw reading has moved by at least 24 counts. So:

- a volume set from the portal holds until the knob is next moved;
- the knob's position wins at every start;
- the knob-set volume reaches flash only when something else causes a save.

Chapter 10 describes the knob in full; Appendix A (p. 309), section A (p. 309).1, lists its calibration values.

7.4.4 The mirror

Figure 19. How the main board keeps its copy of the audio board's settings



- At every greeting (the handshake, chapter 9 (p. 224)), the main board asks the audio board for its settings and copies the answer: audio, Bluetooth and knob calibration. A board that restarted brings its own values back, rather than the main board's copy overwriting them. When the audio board's uptime in a status report goes backwards, the main board notices a restart the link missed and greets it again, which refreshes the mirror.
- Status every 250 ms. The audio board's status report carries its volume and mute, and the mirror follows them. The knob moves the volume; nothing else would tell the main board.
- An edit sends the whole structure. A change to one audio-board row edits that field of the mirror and sends the whole audio or Bluetooth structure. The audio board copies it, applies it and saves it two seconds later (or at confirmation, while it is on trial).
- "Do not write a mirror we have never read." Until the first answer arrives the mirror is all zeros. Every audio-board row is refused in that window: a push would have sent volume 0, gain 0, every lamp off and "not

connectable", and the audio board would have saved them. The portal answers such an edit with HTTP 409, `the audio board is not answering – its settings cannot be changed now`.
- Re-read the A32 (Audio tab) asks for the audio board's settings on demand.
- The knob calibration is not a row. The Audio tab's buttons Pot: minimum, Pot: centre and Pot: maximum tell the audio board to measure the knob itself (the median of nine readings), because the reading and the thing being calibrated must not live on opposite sides of a link.
- Boards running different link versions do not look like a dead wire: the main board counts the frames it drops and shows BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both (chapter 9 (p. 224)).

7.4.5 When the audio board falls silent

When the link, having greeted the audio board, stops hearing it, the main board forgets its copy of the audio board's settings, with the rest of its audio-board state, and prints `[WARN] A32 silent.` From then on, until the next greeting fetches the settings afresh:

- a portal edit of an audio-board row is refused with HTTP 409, `the audio board is not answering – its settings cannot be changed now`
- a Download writes `n/a` for every audio-board row;
- an Upload names the audio-board rows under "NOT SENT - the audio board is not answering".

The same applies when the audio board has not answered since the main board started.

Why. A copy that counted as known after the audio board went quiet took portal edits into the mirror only. Those edits were overwritten when the audio board came back, while the portal said "ok". And a download wrote the stale copy.

7.5 Firmware updates and stored settings

7.5.1 How the main board converts older settings

The main board's settings blob begins with a header: a magic number, `0xA838`, that marks a blob with a header, and the settings version, now 7. The structure follows three rules:

- Never reorder or remove a field. Add new fields at the end only.
- Every addition raises the settings version and adds a conversion step (a migration).
- An older blob is copied into the current structure, so its values survive and the new fields keep their defaults.

Loading, step by step:

1. No blob: `no stored settings – defaults in use.` The defaults run, and are written on the first change.

2. Read as many bytes as were stored, up to the current size, into a zeroed buffer. Size decides only how much may be read, never which version the blob is.
3. Fewer than 100 bytes (the size of the oldest layout): a warning, and the defaults.
4. No magic number: the blob is version 1, the layout from before the header existed (its first two bytes are the two brightness values). It is copied field by field.
5. Version 7: a straight copy, `settings loaded.`
6. Version 6, 5, 4 or 3: the old blob is copied up to where the first field that version did not have begins. The cut is the field's position, not the blob's length, because a new field can sit in the old structure's trailing padding (version 3 did exactly that). Converting version 4 also works out which tuner ends were measured (below).
7. Version 2: field by field, from the kept version-2 layout.
8. The magic number, but any other version: this firmware is older than the one that saved the blob. Defaults in memory, and the downgrade lock is set (section 7.5.4 (p. 166)).

Every conversion marks the settings unsaved, so the converted blob is written back as version 7 about two seconds later, once the needle is still and the firmware is confirmed. Each conversion prints one console line saying what the new fields start as.

7.5.2 Version history

Table 77. Version history

Version	What it added
1	The layout without a header: the one that holds the first real needle calibration. 38 of its 39 fields carry across. The one that does not, `seekHsps`, was a deleted setting and is dropped. The index search only has to find the sensor, and the slow re-approach defines zero, so its speed never mattered. A setting that implies it matters is worse than none.
2	The magic number and version header; the layout that shipped the web portal.
3	`tuneHoldMs`, placed in version 2's trailing padding (still 100 bytes).
4	`dialLow`, `dialHigh`: the first addition that made the structure longer.
5	The tuning samples and their mask (`tuneUsed`), the hand offset `tuneOffset10`, and `tunerEndsSet`. A version-4 blob whose tuner ends differ from the placeholders 0 and 6023 is taken as proof of a hand measurement and gets `tunerEndsSet` = 3 (both ends).
6	`wifiTxQ`, and the fixed-feature list (`spurUsed` and 16 slots).
7	`ifOffset` (stored as `ifOffset20`), default 10.60 MHz.

The conversion from 6 to 7 changes behaviour on the first start. Every station the firmware computes moves down 0.1 MHz (what the readout shows and where the needle points, not the sound), because the firmware used to assume an IF of 10.70 MHz and now uses 10.60 MHz, the tube set's fitted IF (Hardware Bible, chapter 13; chapter 6 explains the IF).

7.5.3 The two clean-ups at start-up

After loading, two checks run once:

- Soft limits. A stored pair that does not bracket the index at 0 (`posMin` above 0, `posMax` below 0, or crossed) cannot be real. It is replaced by the provisional pair −300…+300 half-steps and saved, and the console asks for a home, an index calibration and new limits (chapter 5 (p. 73), section 5.12 (p. 92)). The index band's values are kept, because they are measured from the index.
- Fixed features. A count above 16, or any entry outside the window the tuning chip searches (77.2 to 97.2 MHz), empties the list and saves the empty list. A truncated list is not a list.

Both run at start-up only, never while settings are being edited.

7.5.4 The downgrade lock

When the main board finds the magic number with a version it does not know, the firmware is older than the one that wrote the blob. It locks the settings. While locked:

- the machine runs on defaults in memory: provisional soft limits, the default tuning line, the default brightness, and so on;
- nothing is written, so the newer settings on flash are never overwritten;
- Download is refused, with HTTP 409 and a sentence that the page shows instead of downloading. A file of defaults would look complete, and uploading it back would lift the lock and write defaults over the real calibration;
- the portal shows the pill SETTINGS LOCKED - newer version stored
- Save settings now answers with the lock's reason.

Two ways out:

1. Flash the newer firmware again. It reads its own settings normally.
2. Upload a complete settings file, taken earlier. Complete means: all 38 main-board rows present and accepted; a tuning block that committed (`tuneUsed` present, the mark lines matching the mask, the offset readable); and a fixed-feature block that committed (`spurUsed` present, slots 0 to N−1 exactly). Audio-board rows, `bandLow`, `bandHigh` and `tunerEndsSet` are not required. Keys this firmware does not know (from a newer firmware's file) are refused and named, but do not block. On success the console prints `complete settings file uploaded – the newer stored settings will now be replaced.` and the next save writes version 7 over the newer blob. Otherwise the upload's note says `SETTINGS STILL LOCKED – a complete file is needed (N of 38 settings, no tuning marks, no fixed features); nothing was saved.` The values that did arrive are still applied in memory.

Why a single line cannot lift the lock. While locked, the settings in memory are defaults. Lifting the lock on one line such as `volume=120` wrote the defaults for everything else over the newer blob two seconds later.

A rollback cannot cause the lock: a new firmware writes nothing during its trial, so if it is rolled back the previous one finds its own settings (chapter 3

(p. 25), section 3.11.5 (p. 34)). Flashing an older firmware on purpose, by cable or over the air, after a newer one has confirmed itself and saved, can.

7.5.5 Updates that change the audio board's settings

The audio board has no conversion. An update that changes the size of the audio or the Bluetooth structure returns every setting in that structure to its default on the first start of the new firmware; the knob calibration survives. Download the settings file before such an update and upload it afterwards. The audio rows are sent once the main board has read the mirror. If the update came over the air, the upload applies at once but is written only when the new audio-board firmware confirms itself, about a minute after its first greeting. Should it be rolled back instead, the previous firmware finds its own blobs untouched.

7.6 Backup and restore: the settings file

The settings file is the backup, and the only undo. It is plain text, one `key=value` per line, so that it stays readable and repairable by hand when neither this firmware nor a browser exists.

7.6.1 The format, and the download

Download on the System tab's Settings file card (administrator only) saves the file as `ambersong.txt`. Here is its shape (the values are illustrative):

```
# Ambersong settings
# firmware v.1.20260925T184108 (1a2b3c4)
# settings version 7
# saved 2026-09-25T18:45:00
# the audio board had not answered - its settings are n/a
volume=40
...
gainRadio=8.000
ifOffset=10.600
...
connectable=1
bandLow=881
bandHigh=1079
tuneOffset10=0
tuneUsed=7
tunerEndsSet=3
tuneMark0=-817,913
tuneMark1=120,985
tuneMark2=1043,1073
spurUsed=2
spur0=790
spur1=818
# potcal 61..1974..3988
```

Table 78. The format, and the download

Line	What it is
`# firmware ...`	The build stamp and commit of the firmware that wrote the file (chapter 3 (p. 25)).
`# settings version 7`	A comment only; not checked on upload.
`# saved ...`	Local time, written only when the clock is valid.
`# the audio board had not answered ...`	Written only while the main board holds no copy of the audio board's settings.
`volume=40` … `connectable=1`	One line per portal setting, 58 lines, in the portal's order. Settings stored as decimals, or stored scaled (gains, the volume law, the IF), are written with 3 decimals; every other setting as a whole number. Audio-board rows read `n/a` while the main board holds no copy.
`bandLow`, `bandHigh`	The fallback tuning line, in tenths of a MHz (chapter 6 (p. 120)).
`tuneOffset10`, `tuneUsed`, `tunerEndsSet`, `tuneMark<i>`	The tuning calibration: the hand offset, the mask of used slots, which tuner ends were measured, and one `<shaft count>,<frequency × 10>` line per used slot. `tuneUsed` is always written, even when 0.
`spurUsed`, `spur<i>`	The fixed features: their count, then each one's oscillator frequency × 10, slots 0 to N−1. `spurUsed` is always written, even when 0.
`# potcal ...`	The knob calibration (minimum, centre, maximum), written only while the main board holds a copy; never read back.

Why **`tuneUsed`** and **`spurUsed`** are always written. On upload, "replace the marks" and "replace the list" hang on these keys. A backup taken before any marking can then undo the marking. Without the zero line it would look like a file from before those features existed, and would leave the marks alone.

Why **`n/a`** while the audio board is silent. A file of the mirror's zeros, uploaded later, would push volume 0, gains 0, every lamp off and "not connectable" to the audio board, which would save them.

Not in the file: the header's magic number and version (the version is a comment); `lastAngle` (live state); `idxOnFwd` (zero by definition); `wifiTxQ` (the learned WiFi power; console only); the audio board's unused `autoConnect` field; the WiFi network and time settings; the portal accounts; the knob calibration (comment only); and every experiment (Appendix A (p. 309), section A (p. 309).6).

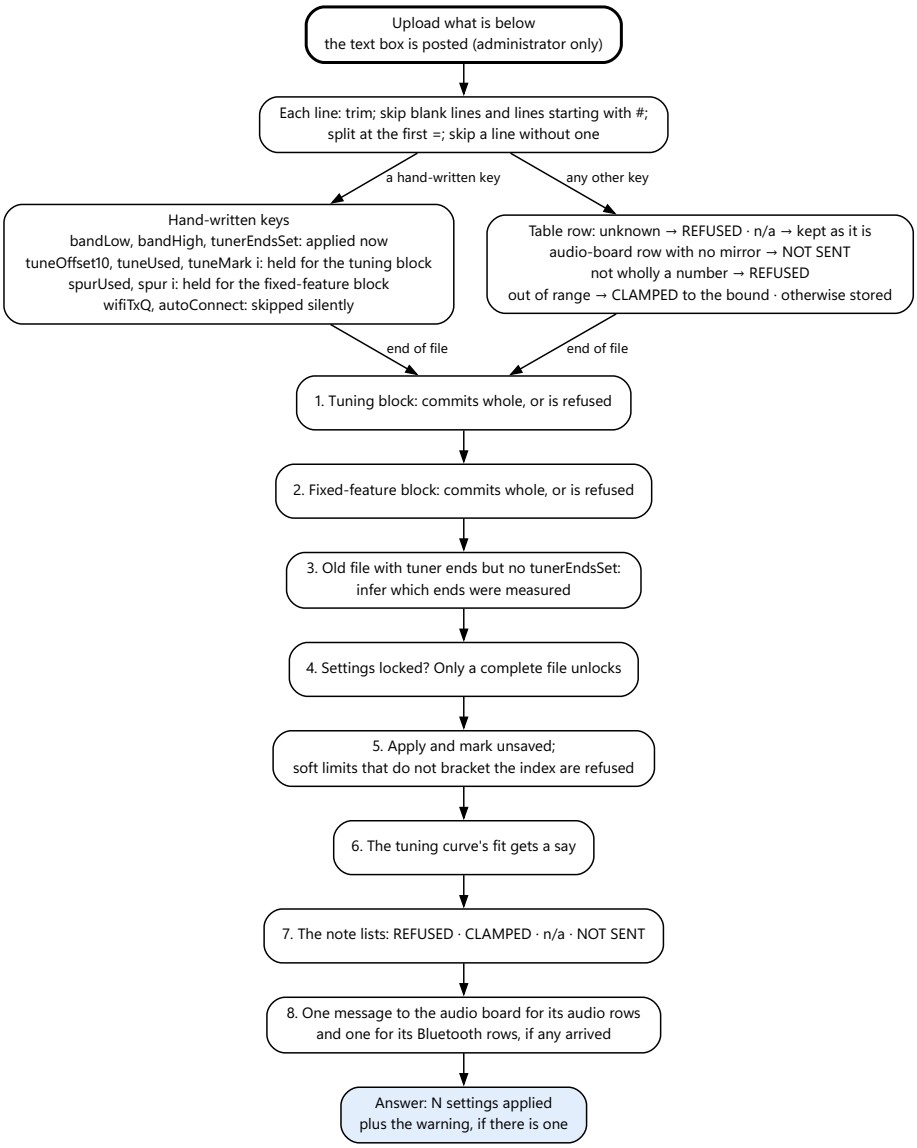
Accounts and the WiFi passphrase are left out on purpose. The file is a backup that gets copied, pasted and kept anywhere; the house network's passphrase and the portal's password hashes stay on the machine. A restore does not restore accounts. (Forgotten password: chapter 8 (p. 186).)

7.6.2 Uploading: the rules

Paste the file into the System tab's text box and press Upload what is below (administrator only; the page waits up to 20 s for the answer).

Note An upload is a merge, not a replace. A key absent from the file keeps its current value. Leaving a line out does not return a setting to its default. The only exceptions are the tuning block and the fixed-feature block, which replace the stored marks and list as a whole when they commit.

Figure 20. What happens to an uploaded settings file



Each line is trimmed. Empty lines and lines that start with `#` are skipped; a `# comment` after a value makes the value unreadable. The line is split at its first `=` a line without one is skipped.

Hand-written keys (the ones that are not portal rows) follow their own rules:

Table 79. Uploading: the rules

Key	Rule	On a bad value
`tunerEndUsed`	Whole number 0 to 3, applied at once.	REFUSED and named.
`bandLow`, `bandHigh`	Whole number 500 to 2000 (tenths of a MHz), applied at once.	REFUSED and named (refused, not clamped).
`tuneOffset10`	Whole number -200 to 200 (tenths of a MHz), held with the marks.	The whole tuning block is refused: `TUNING CALIBRATION REFUSED - the hand-offset line is unreadable; the stored calibration was kept.`
`tuneUsed`	A whole number, in decimal or as `0x` hexadecimal (a console dump once printed it that way), 0 to 0xFFFF; held.	Ignored, with no note of its own, and every mark line after it is skipped. With a `tuneOffset10` line in the file (every downloaded file has one), the block is then refused with a misleading “the file announced 0 and 0 parsed”; without one, nothing is said (chapter 12 (p. 290)).
`tuneMark<i>`	Only after a `tuneUsed` line. `i` is a whole decimal 0 to 11 (`tuneMarkA` is not accepted); the value is `<shaft count>,<frequency × 10>`, the count within ±1 000 000 and the frequency 870 to 1085; held.	Skipped without a word; the mask check then refuses the block.
`spurUsed`	Whole number 0 to 16, held.	Spoils the fixed-feature block.
`spur<i>`	Only after `spurUsed`. `i` 0 to 15; the value 772 to 972 (tenths of a MHz, the oscillator window); no slot twice; held.	Spoils the fixed-feature block.
`wifiTxQ`, `autoConnect`	Retired keys: skipped whatever the value, not counted and not named, so a file saved by an older firmware uploads cleanly.	—

A key starting with `tuneMark` or `spur` always belongs to its block; it is never treated as unknown.

Why **`tuneMarkA`** is refused. The console dump labels the hand slots A, B and C. A loose reading once took `tuneMarkA` as slot 0, so mark A could silently receive another mark’s numbers with the mask still agreeing.

Portal rows, checked in this order:

1. An unknown key: REFUSED.
2. The value `n/a` (any case): noted under “n/a in the file - kept as they are”.
3. An audio-board row while the main board holds no copy: noted under “NOT SENT - the audio board is not answering”.
4. A value that is not wholly a number: REFUSED. Blanks around it are trimmed, and one decimal comma is read as the point; `107,3` is 107.3.
5. Otherwise the value is bounded to the row’s range. If that changed it by more than 0.0005, it is noted CLAMPED and stored at the bound.

Rows are written straight into the settings or the mirror, with no message per line.

At the end of the file, in this order:

1. The tuning block. It commits only if `tuneUsed` was seen, the marks that parsed match the mask exactly, and any offset line was readable. Then all 12 slots, the mask and (if present) the offset replace the stored ones

together. Otherwise: `TUNING MARKS REFUSED - the file announced N and M parsed; the stored calibration was kept`. A file with no `tuneUsed` line (older than version 5) leaves the marks alone.

2. The fixed-feature block. It commits only if `spurUsed` was seen, no line spoiled it, slots 0 to N−1 all arrived, and no two entries are closer than 0.3 MHz (the matcher's ±0.2 MHz would count them as one). Otherwise `FIXED-FEATURE LIST REFUSED - the stored one was kept`.
3. Which tuner ends were measured, worked out only for a file that has `calLow` or `calHigh` but no `tunerEndsSet` (files older than version 5). For each end, a value equal to the old placeholder (0 or 6023) means "not measured", and anything else "measured". This is a convention of the file, not a claim about the shaft: a measured end that happened to equal a placeholder exactly would be read as unmeasured.
4. The downgrade lock check (section 7.5.4 (p. 166)).
5. If any main-board value changed, everything is applied and marked unsaved. If the soft limits that came out differ from what the file asked, because the pair did not bracket the index: `SOFT LIMITS REFUSED (a..b does not bracket the index) - kept c..d`.
6. The tuning curve gets a say. Unless the tuning block was refused, and when marks are stored: if the resulting curve is refused or demoted, `after restore: <state>` is added. `calLow` and `calHigh` alone can do this, since they set the curve's domain (chapter 6 (p. 120)).
7. The four lists: `REFUSED (unreadable or unknown): ...`, `CLAMPED to its range: ...`, `n/a in the file - kept as they are: ...`, `NOT SENT - the audio board is not answering: ...`. Each names up to five keys, each cut to 20 characters, then `, ...`. They come last so that the block refusals are never crowded out of the note, which holds 512 bytes.
8. One message to the audio board for its audio rows and one for its Bluetooth rows, if any landed.

The answer. The count is of accepted items: each accepted line, plus one per committed block. The page shows `N settings applied` when there is any note it shows the note as a warning instead, not in the success colour, and then reloads the values. A file whose values are valid but wrong is applied and saved: the upload is your act.

7.6.3 Worked example

With the audio board answering and no lock, this upload:

```
volume=999
posMax=abc
wn=10,5
nosuchkey=1
tuneUsed=7
tuneMark0=-817,913
tuneMark1=120,985
```

gives:

Table 80. Worked example

Line	Result
`volume=999`	CLAMPED to 255, and sent to the audio board once, at the end.
`posMax=abc`	REFUSED: not a number.
`wn=10,5`	Accepted as 10.5 (the decimal comma).
`nosuchkey=1`	REFUSED: unknown.
`tuneUsed=7` and two marks	The mask announces three marks and two parse, so the whole tuning block is refused; the stored marks stay.

The answer counts 2 settings applied, and the page shows the warning `TUNING MARKS REFUSED - the file announced 3 and 2 parsed; the stored calibration was kept; REFUSED (unreadable or unknown): posMax, nosuchkey; CLAMPED to its range: volume.` `wn` is saved about two seconds later, once the needle is still.

7.6.4 Good habits

- Download the settings file before any update.
- Before an update that changes the audio board's structures, download first and upload afterwards (section 7.5.5 (p. 167)).
- Keep the files: one taken before any tuning marks were made can undo the marking.

7.7 When it goes wrong

The last column is the fault's class on the recovery ladder (NOTE, DEFECT, BLOCKER), set out in About this book.

Table 81. When it goes wrong

What you see	What happened	What the firmware does	What you do	Class
`[WARN] settings NOT saved - the flash write failed.` Save and Reboot report it	A main-board flash write failed.	Stays unsaved, retries after 2 s.	Nothing.	NOTE
`[A32] settings NOT saved - the flash write failed; retrying`	An audio-board flash write failed.	Stays unsaved, retries after 2 s. A forced save before a commanded restart is not retried: the board restarts anyway.	Nothing, except before a restart.	NOTE; never exercised on the radio
A setting back at its old value after a power cut	The power went within 2 s of the change.	The old value is intact.	Make the change again.	NOTE
Changes gone after a power cut just after a main-board update	The power went during the trial minute; nothing is written then.	The flash still holds the settings from before the update.	Make the changes again.	NOTE
Audio changes gone after a power cut, or an audio-board restart, just after an audio-board update	The same, on the audio board; a restart on trial is also a rollback.	The flash still holds the audio settings from before the update.	Make the changes again.	NOTE

What you see	What happened	What the firmware does	What you do	Class
Some values of an upload present, others not, after a power cut	A background save caught a half-applied upload, and the power went within about two seconds (an accepted race, section 7.11.2).	Half the upload is on flash.	Upload the file again.	BLOCKER, accepted
SETTINGS LOCKED - newer version stored Download refused	Older firmware over newer settings.	Defaults in memory, nothing written.	Flash the newer firmware, or upload a complete file (section 7.5.4 (p. 166)).	DEFECT
A warning at start-up, and defaults	The stored blob is shorter than 100 bytes.	Defaults, no lock; saved on the first change. Never seen.	Upload a settings file.	BLOCKER
Settings that make no sense after a start-up, such as the needle's motion	The header was damaged so the magic number no longer matches.	Reads the blob as version 1 and saves it back as version 7.	Upload a settings file.	BLOCKER (in theory)
Soft limits back at ±300 hs; the console asks for a recalibration	The stored pair did not bracket the index.	Replaced and saved at start-up.	Home, calibrate the index, set the limits (chapter 5 (p. 73), section 5.12 (p. 92)).	BLOCKER (guarded: the needle stays off the stops, but the limits must be captured again)
The fixed-feature list empty	A count or an entry was out of range at start-up.	Emptied and saved.	Learn them again with the tube set switched off (console ∘ chapter 6 (p. 120)).	DEFECT
The machine behaves wrongly after an upload	The file held valid but wrong values.	Applied and saved: it is the user's act.	Upload a good file.	DEFECT
REFUSED notes, or a block refused	The file was truncated or damaged.	What arrived is merged; the tuning and fixed-feature blocks are refused unless complete; everything refused is named.	Upload the file again.	NOTE
`SOFT LIMITS REFUSED (a..b does not bracket the index) - kept c..d`	The file's soft-limit pair was invalid.	Keeps the old pair.	Correct the file.	NOTE
`the audio board is not answering - its settings cannot be changed now` `n/a` in a	The audio board has not answered since the main board started.	Refuses audio-board rows.	Restore the link; the mirror arrives at the greeting	NOTE; never exercised

What you see	What happened	What the firmware does	What you do	Class
download; NOT SENT on an upload			(chapter 9 (p. 224)).	on the radio
`[WARN] A32 silent.`, then the same as above	The audio board fell silent after it had answered.	Forgets the mirror.	Restore the link; the next greeting fetches the settings again.	NOTE; never exercised on the radio
Nothing	The audio board restarted.	Greets it again; the mirror is refreshed from the audio board's flash.	Nothing.	NOTE
Audio and Bluetooth settings back at their defaults after an audio-board update	The update changed the size of a structure.	Uses that structure's defaults; saves them on the next change (after confirmation, for an update over the air). The knob calibration survives.	Upload a file downloaded before the update.	BLOCKER by design; avoidable by downloading first
Audio rows shown as `n/a` audio-board changes do not hold	The two boards run builds of the same protocol version that do not match (a half-flashed pair).	The audio board ignores the settings messages; the main board never gets the audio settings; the radio plays on the audio board's stored settings.	Flash both boards.	BLOCKER: the audio board's settings cannot be changed until a firmware update
BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both console `s` shows `wrong-version N` no radio or Bluetooth sound	The two boards run different protocol versions.	Every frame is dropped and counted: no handshake, so the audio board sleeps: no radio or Bluetooth sound (AUX does not pass through the audio board).	Update the main board first, over its own WiFi, to the audio board's protocol version; the reverse order cannot work (chapter 9 (p. 224), section 9.6.3 (p. 236)).	BLOCKER: survives a power cycle once the audio board has confirmed its program
Reboot the main board answers `not rebooting - <why>`	A change was unsaved while the needle moved.	Stopped the needle, retried for 1.5 s, refused, resumed the needle.	Try again.	NOTE
A change gone after a main-board update	It was unsaved and could not be written when the update ended.	Restarted anyway; warning on the USB console only.	Make the change again.	NOTE
Nothing	A main-board update that raised the settings version was rolled back.	The previous firmware reads its own blob; no lock.	—	NOTE

What you see	What happened	What the firmware does	What you do	Class
Nothing	An audio-board update that changed a structure was rolled back.	The previous firmware reads its own blobs.	—	NOTE
Nothing	Flash wear.	Writes happen only on a change, two seconds after it, never during motion. The most frequent writer is the shaft angle, at most every 10 s and only after the dial moved. The storage spreads its entries across its pages. At a handful of writes per listening session, wear is not a practical concern.	—	NOTE

7.8 Design choices

- One table describes every setting and generates the portal's widgets, the bounding and both directions of the file. Four hand-kept copies drift apart; one table makes every setting affordable.
- Roles split by damage, not secrecy. A guest sees every value and moves only the obvious and reversible ones.
- The machine bounds every value; the browser's limits are only a suggestion. One bounding rule serves the portal and the file, and refuses NaN. A `posMax` of 30000 would drive the needle into its stop, and a second copy of the rule once let the portal store a choice of 99.
- Numbers are read whole; one decimal comma counts as the point; anything else is refused. The old reader took "107,3" as 107 and "" as 0, and the clamp then stored a bound: an IF of 10.00 MHz, a soft limit at the index.
- Refused and clamped lines are named, in their own lists, last, and in a separate warning. A partial apply reported as "41 settings applied" in green is false reassurance.
- The main board's settings are versioned, add-at-the-end only, converted on load, and identified by their header, never their size. Throwing everything away on a layout change is right for a bench rig and wrong for a machine somebody has spent an evening calibrating. And versions 1, 2 and 3 were all 100 bytes.
- Tuning samples are stored, not curve coefficients, and the curve is refitted whenever settings are applied. Samples describe themselves and can be refitted by a better model without a conversion.
- Writes wait two seconds and never happen while the needle moves. A flash write stops everything outside internal RAM.
- "Unsaved" comes from two counts; one writer at a time; the blob is written from a snapshot. A stored flag cleared around the write lost changes that landed during it.

- "Save settings now" tells the truth, and a reboot refuses to run over an unsaved change. "Saved" followed by "rebooting" over a refused save was a data-loss path.
- Neither board writes settings while its new firmware is on trial, and a reboot during the trial goes through at once. A firmware that changes the layout would otherwise save it within seconds; rolled back, the previous firmware would find settings it cannot read. A reboot on trial is a rollback, so refusing it would only take away the one button that undoes a bad update.
- A blob from a newer firmware locks all writes, refuses the download, and only a complete file lifts the lock. Before the lock, the first automatic save wrote defaults over the newer blob; a one-line upload then did the same.
- The audio board owns and saves its settings; the main board mirrors them, refreshes the mirror at every greeting and never pushes a mirror it has not read. A restarted audio board must keep its own values; an unread mirror is all zeros.
- When the audio board falls silent, the main board forgets the mirror. Edits made into a stale mirror were silently lost.
- The audio board counts a save only when every write reached flash, and retries a failure. A flag dropped before the write said "saved" when nothing was.
- **`n/a`** for the audio board's rows when it has not answered. A file of zeros would silence the radio and switch off every lamp, and the audio board would save it.
- Mute is never restored; the volume is, but the knob's first reading wins. A saved mute made the set start in silence with nothing on the front to say why.
- The audio board saves before a commanded restart, except on trial. "Reboot the audio board" within two seconds of a change used to lose it.
- The tuning marks, their mask and the hand offset commit as one unit; the two count keys are always written. Wiping the marks on the first `tuneUsed` line let a truncated upload erase the calibration with its only backup; an offset belongs to the curve it corrected.
- The hand offset is never erased automatically. It is the user's calibration, made by ear (chapter 6 (p. 120)).
- **`idxOnFwd`** is not a row: it is zero by definition, and an edited value once moved both soft limits into a stop. The other three index-band values are rows, so the file carries them: a restore onto blank flash left them at zero, and every index crossing then read as a slip.
- **`bandLow`**/**`bandHigh`** are not rows but stay in the file. Nothing reads them once two marks exist, but with no marks they are the live tuning, so a backup must keep them.
- The WiFi transmit power is learned state, stored but neither a row nor a file line (chapter 8 (p. 186)).

- `autoConnect` is no longer a row, but stays in the audio board's structure, unread. A row that does nothing is a false control; removing the field would change the structure's size and reset every Bluetooth setting.
- The IF is a setting, in 50 kHz steps. Readings of 10.60 and once 10.65 MHz cannot be written in tenths, and the IF moves if the IF coils are realigned.
- The settings file is plain text. A restore is the only undo, so the backup must outlive the tools that wrote it.
- Experiments are actions, never saved (Appendix A (p. 309), section A (p. 309).6).
- Credentials and accounts stay out of the file. The file gets copied, pasted and kept anywhere; losing the accounts on a restore is the accepted trade.
- No lock around the settings across the two cores. The races left are microseconds wide, and the change would touch every writer.

7.9 Tried and rejected

FOR FIRMWARE CHANGES

These were dead ends on this machine. They may not be dead ends on yours.

- Rejecting a stored blob on any change of size. Any layout change silently replaced a calibrated machine's settings with defaults.
- Deciding the version by the blob's size. Versions 1, 2 and 3 were all 100 bytes, so the version-1 branch caught everything. Found by review, not by failure.
- Reading only a blob of exactly the current size. It would have discarded every convertible blob from version 4 on.
- Cutting the old blob at its stored length. A new field can sit in the old structure's padding.
- A stored "unsaved" flag, cleared before and then after the write. Changes that landed during a write were marked saved and lost.
- A save that returned nothing and printed "saved" regardless.
- Checking for corrupt soft limits while applying settings. It fired mid-edit and overwrote what had just been typed.
- Soft-limit ranges of −3000…0 and 0…3000. They assumed a centred index; with clamping, the high limit was silently unsettable while it was negative.
- Two copies of the bounding rule. The second guarded only the file.
- `bandLow`/`bandHigh` as editable "Tuner reaches" rows. The curve ignored them, so corrections typed there were discarded.
- `idxOnFwd` as a row. An edit moved both soft limits into a stop.
- `wifiTxQ` as a row in dBm with a scale of 0.25. Two spellings of one key; a pasted dump line would have read as four times the power.
- `wifiTxQ` in the settings file, pushed to the WiFi driver on upload. It is learned state that nobody sets.

- The **`autoConnect`** row (“Chase the phone (not advised)”). Stored, shown, downloaded and uploaded, and read by nothing.
- The console keys comma (**`,`**) and full stop (**`.`**) for the needle’s top speed. The full stop had no ceiling and bypassed the row’s bounds; the portal row does the job inside its range.
- Counting the mirror as known for the whole start once the audio board had answered. A silent audio board took edits into the mirror only, and a download wrote the stale copy.
- The audio board clearing its unsaved flag before writing, and ignoring the results. A failed write said “saved”.
- Saving during a main-board trial, with a written warning for whoever raises the settings version. A rollback after such a save left the previous firmware locked on defaults.
- Saving during an audio-board trial, with advice to download the file before any update that changes a structure. A rollback would have left the previous firmware on defaults for that structure.
- Zeroing the hand offset on a hand mark, a drop, a clear or a requested sample. It erased the user’s correction unasked.
- Wiping the marks on seeing **`tuneUsed`**, and applying **`tuneOffset10`** at once. A truncated file erased the calibration, or slid an offset onto the wrong curve.
- A lenient whole-number reader for the hand-written keys. It answers 0 for garbage, and 0 is the destructive value for `tuneUsed`, `spurUsed` and `tunerEndsSet`.
- One bit for both tuner ends. One measured end declared both measured, and the curve then ran over a range far wider than the shaft.
- Downloading zeros for a silent audio board.
- Lifting the downgrade lock on any upload.
- Saving the audio board’s mute. The set started in silence.
- Refusals inside the success message. They were painted green.

7.10 Known limits

Seen working on the radio: the download; the upload; every conversion up to version 7; the main board’s trial hold (a portal reboot on trial rolled back with nothing saved); the audio board’s trial hold (a change made on trial was written only at confirmation, with its log line).

Never exercised on the radio: the `n/a` download and the 409 refusal of audio-board rows (they need the audio board silent); the downgrade lock, and with it the complete-file unlock, the NOT SENT note and the refused download; a failed audio-board save; an audio-board rollback with a change held in memory; a failed write at the audio board’s confirmation. Chapter 12 suggests how to test the silent audio board and the lock.

Known limits (chapter 12 (p. 290) has the details and starting points for a fix):

1. The audio board has no conversion. An update that changes the audio or Bluetooth structure resets that structure to its defaults on its first start (section 7.5.5 (p. 167)). A rollback no longer adds to this, but the forward update still does.
2. A change made during either board's trial minute exists only in memory until that firmware confirms itself; a power cut in that minute loses it, and on the audio board so does any restart. The audio board's hold is silent: the portal answers "ok" and nothing marks the change as waiting.
3. The audio board's save before a commanded restart is not retried. If that write fails, the change is lost.
4. An audio-board edit made in the first 2 s after the audio board falls silent is accepted, sent to nobody, and replaced by the audio board's own settings when it comes back: the refusal starts only once the silence has cleared the mirror.
5. Changes made on the audio board's console (the volume law, the knob calibration) do not refresh the main board's mirror. The next portal edit of any audio row sends the mirror's older volume law back. Re-read the A32 refreshes it. Bench only.
6. The knob-set volume is never marked unsaved, so it reaches flash only when something else is saved; the knob's first reading replaces it at start-up anyway.
7. An unreadable **`tuneUsed`** line has no note of its own (section 7.6.2 (p. 168)). The retired keys are skipped without a note, by design.
8. Merge, not replace: an upload cannot return a setting to its default by leaving it out, except the two blocks, through their count keys.
9. The file has no checksum, and its version line is only a comment. Old files work key by key.
10. "Save settings now" does not re-check that the needle is still between its retries a write could start as the needle starts moving. The window is tiny.
11. The page has no "unsaved" indicator. The portal's state carries the lock, not the unsaved state.
12. No lock around the settings across the two cores (accepted; section 7.11.2).
13. The console dump **`D`** is only partly a settings file. Its first lines carry several `k=v` pairs each, which the upload refuses; its tuning, IF (`ifOffset=`) and fixed-feature lines match the file, and its `wifiTxQ` line is ignored.
14. The main board's brightness keys – and + bypass the bounding rule, with their own floor and ceiling.
15. The **`upVmax`** range reaches 4000 hs/s, far past the 1100 hs/s at which this needle still follows through the index (chapter 5 (p. 73)).
16. When an upload has a warning, the page shows only the warning, not the "N settings applied" count.

7.11 Changing it safely

FOR FIRMWARE CHANGES

The main board's settings code is in `src/s3/main.cpp` (the structure, loading, saving), `src/s3/settings_table.h` (the table, the file, the portal actions) and `src/s3/settings_api.h` (the row type); the audio board's is in `src/a32/main.cpp` the shared structures are in `include/proto.h`.

7.11.1 The settings table

Every portal setting is one row of the array `gSet[]` in `settings_table.h`. The row type `SettingDesc`, declared in `settings_api.h`:

Table 82. The settings table

Field	Meaning
`key`	The stable machine name, used by the file and the portal's data. It must never change.
`label`, `unit`	What a person reads.
`tab`	Which portal tab (`TAB_AUDIO` … `TAB_BT` `TAB_SYSTEM` has no rows). Row order is screen order and file order.
`store`	How the bytes are stored: `SU8`, `SU16`, `SI16`, `SI32`, `SF32` (unsigned or signed 8-, 16-, 32-bit whole number; 32-bit float). It must match the C type of the field `ptr` points at; nothing checks this.
`kind`	How it is shown: `K_INT`, `K_BOOL`, `K_FLOAT`, `K_ENUM` (a choice; the choices are one comma-separated string, and the value is the index).
`lo`, `hi`, `step`	The slider's range and the hard clamp, in shown units.
`scale`	Stored value × scale = shown value. Gains are stored in tenths of a dB (scale 0.1), the volume law as gamma × 10 (0.1), the IF in 50 kHz steps (0.05). Whole numbers travel safely in a structure copied byte for byte across the link; the person still reads "+8.0 dB".
`ptr`	The address of the live field, inside `cfg` (the main board's settings) or `a32cfg` (the mirror).
`owner`	Who applies it: `OWN_S3`, `OWN_A32_AUDIO`, `OWN_A32_BT`.
`admin`	1 = administrator only.

There are 58 rows: 38 owned by the main board, 10 by the audio board's audio side, 10 by its Bluetooth side. From this one table the firmware generates the description the browser draws (`/api/schema`), the current values (`/api/values`), the bounding done by `/api/set`, and both directions of the file. The page knows nothing about any particular setting. The browser caches the description against a tag made of the firmware version and the administrator bit, so it is fetched again only after a firmware change or a change of role. `settings_table.h` is included once, by `main.cpp`, after `cfg` and `a32cfg` are defined, because the rows point straight into them; `portal.cpp` sees only `settings_api.h`.

Reading, bounding and writing one value.

- **`settingGet(i)`** copies the stored bytes with `memcpy`, converts to float and multiplies by `scale`. It never casts and dereferences the pointer, because the mirror is a packed structure (no padding) and some 16-bit members sit at odd addresses (`bt.lookTimeoutS` is at offset 17).

- **`settingBound(d, v)`** is the only place a value is bounded; the portal and the file both call it. It refuses a value that is not finite (NaN passes every comparison, and `"nan"` parses), forces a yes/no to 0 or 1, clamps a choice to its first…last, and clamps everything else to `lo`…`hi`.
- **`settingSet(i, shown)`** bounds the value; refuses an audio-board row while the mirror has never been read (`haveCfg` false); converts to stored units with `lroundf` compares with the old bytes and writes with `memcpy` returns false if nothing changed. For `calLow` or `calHigh` it sets that end's bit in `tunerEndsSet` (typing a tuner end counts as measuring it). Then it applies: a main-board row calls `applySettings()` and `settingsTouch()` an audio row sends the whole audio structure (`MSG_SET_AUDIO`); a Bluetooth row the whole Bluetooth structure (`MSG_SET_BT`). Nothing in `settingSet` writes flash.
- **`POST /api/set`** (`hSet()` in `portal.cpp`): sign-in required; 404 for an unknown key; 403 when a guest touches an administrator row; 409 for an audio-board row while the mirror is not known; the value must be wholly a number (`argNumber()`), else 400. It calls `settingSet`, reads the value back and answers `{"ok":1,"v":<actual>}`, plus `"clamped":1` when the stored value differs from the request by more than 0.0005, plus a `"warn"` with the tuning curve's state for `calLow` or `calHigh`. The page snaps the widget to the returned value.
- **`applySettings()`** (`main.cpp`) pushes every main-board value into its module: the display's brightness target, the panel levels and timings, the needle's speeds, response, envelope, dwell, microstepping, measuring speed, soft limits and index band, the tuner calibration, a refit of the tuning curve, the printed dial, the fixed-feature list, and the WiFi transmit power. It then writes back the soft limits that `Needle::setGeometry()` actually accepted, so the settings always equal what the needle uses. It runs dozens of times a session, from both cores, so it must stay cheap, and running it twice must change nothing.
- The file: `settingsToText()` builds it and `GET /api/settings.txt` serves it; `POST /api/settings.txt` calls `settingsFromText()`. The answer is `{"ok":1,"m":"N settings applied"}`, or `{"ok":1,"m":...,"warn":<note>}`.
- The audio board: `settingsDefaults()` fills the audio structure `cfgA` (`ProtoAudio`) and the Bluetooth structure `cfgB` (`ProtoBtCfg`); `settingsLoad()`, `settingsTouch()`, `settingsFlush(force)` and `settingsApply()` do what section 7.4 (p. 161) describes; `gImgOnTrial` and `gImgConfirmed` gate the flush; `confirmImage()` writes what was held; `updateVolumePot()` reads the knob. The mirror on the main board is `ProtoCfgAll a32cfg`: `ProtoAudio audio; ProtoBtCfg bt; uint16 potMin, potMid, potMax`.

7.11.2 Two cores

The main board changes settings from two tasks:

Table 83. Two cores

Core	Task	What it writes
0	The portal task (`serverTask` in `portal.cpp`)	`/api/set`, the file upload, the actions (`tune.mark*`, `tune.clear`, `tune.drop`, `tune.nudge`, `tune.nudgeZero`, `needle.limitLow/High`, `needle.nudge`, `needle.tuneLow/High`, `rda.spurClear`, `sys.save`), the reboot and update-end saves. Each writes `cfg` or `a32cfg` and calls `applySettings()` from core 0.
1	The main loop	`settingsFlush()`, the shaft-angle save, the `wifiTxQ` copy, tuning samples, index-calibration results, console keys, and the link handler writing `a32cfg` (the settings answer, and volume and mute from the status report).

What protects the data: the touch counts (a change on either core during a write is never marked saved); the single-writer flag; the snapshot (the blob written is one consistent copy); "touch after the change" everywhere; and the multi-field units (the tuning marks with their mask and offset; the fixed-feature list), which are read into local variables and committed in one step. `Link::send` may be called from both cores: the UART driver takes the whole buffer under one lock.

What is not protected, by decision: there is no lock around `cfg` itself. The races left are microseconds wide: an upload half-applied at the instant a background save takes its snapshot, followed by a power cut within about two seconds; a hand mark and an automatic sample in the same microsecond; a soft-limit edit against a concurrent `applySettings()`.

7.11.3 What must stay true

1. The main board's `Settings` structure only grows at the end. Every addition raises `SETTINGS_VERSION` and adds a cut-at-the-new-field conversion. The version comes from the header; size only limits the read.
2. Every `gSet` row's `store` code matches the C type of the field it points at. All access goes through `memcpy` (the audio structures are packed).
3. `settingBound()` is the only bounding rule. Never add a second copy in a caller.
4. `settingsTouch()` is called after every change to `cfg` that must persist, from either core. A change without it is lost at power-off.
5. `settingsWrite()` is the only writer of `amb3/cfg`. Nothing writes during motion, a calibration or an update (`writeSafe()`).
6. Audio-board rows are never pushed before the mirror has been read; never export zeros for them.
7. A multi-field calibration commits as a unit or not at all. Count keys (`tuneUsed`, `spurUsed`) are written even when zero.

8. Refusals reach the page in the `warn` field, never in the success message.
9. Hand-parsed keys use `strtol`/`strtof` and require the whole field to be consumed; never `toInt()` or `toFloat()`.
10. `applySettings()` stays cheap, gives the same result when run twice, and writes back the accepted soft limits.

7.11.4 Traps

- Version 1 is recognised by the absence of the magic number. Do not add a size test.
- A new field may land in trailing padding: the conversion's cut must be `offsetof` the new field.
- **`CFG_SPURS`** (16) is frozen into the layout. `Rda::MAX_SPURS` must equal it; a `static_assert` in `main.cpp` stops the build otherwise.
- The main loop copies the WiFi driver's transmit power into **`cfg.wifiTxQ`** on every pass. Any path that sets `wifiTxQ` must also push it to the driver (`applySettings()` does, through `Net::setTxPower()`), or it is reverted at once.
- A retired key must stay in the upload's skip list (`wifiTxQ`, `autoConnect`), or every old file reports it as REFUSED.
- **`calLow`**/**`calHigh`** are rows and also feed the curve fit. Writing them sets `tunerEndsSet` bits (`settingSet`) or infers them (file). The values 0 and 6023 mean "not measured".
- **`posMin`**/**`posMax`** are validated by **`Needle::setGeometry()`**, not by the row's range.
- The portal task runs on core 0 anything that busy-waits there freezes the portal.
- **`settingsDump()`** is written by hand it is not generated from the table.
- In the file, **`tuneUsed`** must come before its **`tuneMark`** lines and `spurUsed` before its `spur` lines.
- A version raise and the rollback. This trap is closed on the main board because a firmware on trial writes nothing, so a converted blob reaches flash only after the new firmware has confirmed itself. Keep the trial check first in `settingsWrite()` any other path that writes the `amb3` blob would reopen the trap. On the audio board, keep the trial check in its `settingsFlush()` ahead of the write, never write the `amb` namespace from anywhere else, and keep the flush in `confirmImage()`, or the changes held during a trial are written only by the next change.

7.11.5 How to add a main-board setting

1. Decide what it is. A value that persists, is one number and has one range becomes a row. An experiment becomes an action (Appendix A (p. 309), section A (p. 309).6). A list, a learned value, or several fields that must commit together becomes a hand-written key (step 8).

2. Add the field at the end of `struct Settings` in `src/s3/main.cpp`, with its default as the initializer. That default is what every existing machine gets on its first start with the new firmware. Never insert, reorder, widen or remove. (To grow the fixed-feature list, add a second array.)
3. Raise **`SETTINGS_VERSION`** (7 to 8).
4. Add a conversion for the old version in `settingsLoad()`, next to the others:

```
if (ver == 7) {
  size_t cut = offsetof(Settings, newField);
  if (got < cut) cut = got;
  memcpy(&cfg, blob, cut);
  cfg.magic = SETTINGS_MAGIC; cfg.version =
SETTINGS_VERSION;
  gTouchGen++; dirtyAt = millis();      // boot,
single-threaded
  Con.println(F("  settings MIGRATED from version 7 -
<newField> starts at <default>."));
  return;
}
```

   The older conversions need no change: their cuts stop earlier, so the new field keeps its default. Never decide on size.
5. Add a row to `gSet[]` in `settings_table.h` with `S_(...)` (or `E_(...)` for a choice): a key that will never change, label, unit, tab, a `store` code matching the field's exact C type, kind, `lo`/`hi`/`step` in shown units, `scale`, `&cfg.newField`, `OWN_S3`, and `admin` = 1 unless the value is obvious and reversible. The row's position is its position on screen and in the file.
6. Use it in **`applySettings()`**: push it to its module. `settingSet()` already calls `applySettings()` for every main-board row.
7. Add it to **`settingsDump()`** by hand.
8. For a hand-written key instead of a row: write it in `settingsToText()` after the rows; read it in `settingsFromText()` before the table lookup (`settingFind()`), with whole-field parsing; decide whether a bad value is refused (and named with `noteRefusedKey()`) or ignored; for a multi-field unit, read into local variables and commit at the end; write a count key even when zero; add it to the dump byte for byte as the file writes it.
9. Decide how the downgrade lock treats it. A new row is required by the complete-file check automatically (every `OWN_S3` row). A hand-written key is required only if you add it to that check. A file from the previous version lacks the new row and cannot lift a lock.
10. Nothing in the page or **`portal.cpp`** changes: the description is generated, and its cache tag changes with the firmware version.
11. Build both programs and test (section 7.11.7).

7.11.6 How to add an audio-board setting

1. Add the field at the end of `ProtoAudio` or `ProtoBtCfg` in `include/proto.h`. This changes a structure that travels on the link: a board running the other build rejects the audio, Bluetooth and settings messages on their length, so flash both boards. `static_assert`s check that the structure still fits the link's payload limit (1088 bytes).
2. Do not raise **`PROTO_VERSION`** for an added field. The rule is written in `include/proto.h` ("WHEN PROTO_VERSION CHANGES"): a structure that changes size needs no raise, because the receiver refuses a payload of the wrong length, so a mismatched pair loses only that message type and the link stays up. Raise it only for a change the length check cannot see (same size, different layout or meaning). A raise drops every frame of the other version, which costs the greeting and with it the route for updating the audio board through the main board. Update the audio board first, through the main board, while the two still match (chapter 9 (p. 224)).
3. Every setting in that structure resets to its default on the first start, because the stored blob no longer matches the structure's size (section 7.5.5 (p. 167)). Download a settings file before the update and upload it after.
4. Give it a default in `settingsDefaults()` and apply it in `settingsApply()` (both in `src/a32/main.cpp`). Add any clamp the audio board itself needs when applying.
5. Add a row to `gSet[]` pointing at `&a32cfg.audio.newField` or `&a32cfg.bt.newField`, owner `OWN_A32_AUDIO` or `OWN_A32_BT`, with a `store` code matching the field.

7.11.7 How to test a change

Without the real machine where possible. Steps 2, 3 and 4 are also good checks after a restore.

Table 84. How to test a change

#	Test	What to do	What to expect
1	Build	Build both programs.	Both succeed (check the exit code, chapter 3 (p. 25)).
2	Round trip	Download the file; upload it unchanged.	`N settings applied`, no warning.
3	Refusals	Upload `posMax=abc`, `nosuchkey=1`, `volume=999`, `wn=10,5`.	REFUSED `posMax`, `nosuchkey` CLAMPED `volume` `wn` accepted as 10.5.
4	Whole blocks	`tuneUsed=0x7` with only two `tuneMark` lines; the same for `spurUsed`.	`TUNING MARKS REFUSED ... announced 3 and 2 parsed`, stored marks unchanged; the fixed-feature list likewise.
5	Soft limits	Upload `posMin=100`.	`SOFT LIMITS REFUSED`.
6	Save path	Change a needle value while the needle moves and press Save settings now.	The motion refusal; once it stops, "saved". The console prints `settings saved.` about two seconds after a change.
7	Conversion	Flash the new build over a board holding the previous version; compare `D` dumps before and after.	The `MIGRATED` line. Over the air: no `settings saved.` until the console prints `image confirmed`.
8	Main-board trial hold	Right after a main-board update over the air, change a setting and press Save settings now then Reboot the main board.	`NOT saved yet - this firmware is on trial ...` then `rebooting - this firmware was still on trial, so the previous firmware comes back`.
9	Audio-board trial hold	Right after an audio-board update over the air, change an audio-board row from the portal (a Bluetooth lamp level is harmless).	"ok"; no `settings saved.` on the audio board's USB console (if connected); a minute after the greeting, `[A32] settings changed during the trial are now saved` next to `[A32] image confirmed (a minute of running with the S3)`. Put the value back afterwards.
10	Retired keys	Upload `wifiTxQ=60` and `autoConnect=1`.	No REFUSED note and no change.
11	Downgrade lock (a bench board only)	Flash an older build by cable over a newer blob.	The pill; a 409 on Download; a partial upload that keeps the lock; a complete upload that lifts it.
12	Silent audio board	Keep the link down when the main board starts, or cut it after the greeting.	`n/a` in the download, NOT SENT on upload, a 409 on a portal edit of an audio row.

On the real machine, flash only as chapter 3 (p. 25) says: identify the board first, never while someone is listening or looking, and with the pinned toolchain.

8. Network, web portal and console

When the cabinet is closed, the web portal is the only way into the machine. The radio has no configuration buttons. The main board (the S3) joins the house WiFi and serves a password-protected web page that shows everything the machine knows: a live dashboard, every setting and action, the settings file, the user accounts, the network settings, firmware updates

for both boards, and a live copy of the S3′s console that you can also type into.

If the S3 cannot reach the house network, it raises its own open WiFi network, the rescue access point, and serves the same portal there while it keeps trying to go home. Nothing ordinary should ever need the USB cable; the cable stays the last way in when everything else has failed. All the code in this chapter runs on the S3.

8.1 Reaching the radio

Table 85. Reaching the radio

Where you are	Address
On the house network	The address the router gave the radio, or `http://ambersong.local/`
On the rescue access point `Ambersong`	`http://192.168.4.1/`, or `http://ambersong.local/`. Most phones open the sign-in page by themselves.
With the USB cable	The S3′s console, on its native USB port at 115200 baud (section 8.9 (p. 209))

The portal is plain HTTP on port 80. Sign in with an account (section 8.6 (p. 199)). The S3 also takes its time of day from the internet and corrects the audio board's battery clock with it (chapter 4 (p. 46), section 4.9 (p. 57)).

Chapter 1, sections 1.3 (p. 5) and 1.4, has the short recovery steps when the portal will not open or the radio has lost its network. This chapter explains them.

8.2 First-time setup

8.2.1 The builder checklist

The firmware ships with four values that are not yours. Set them before anything else.

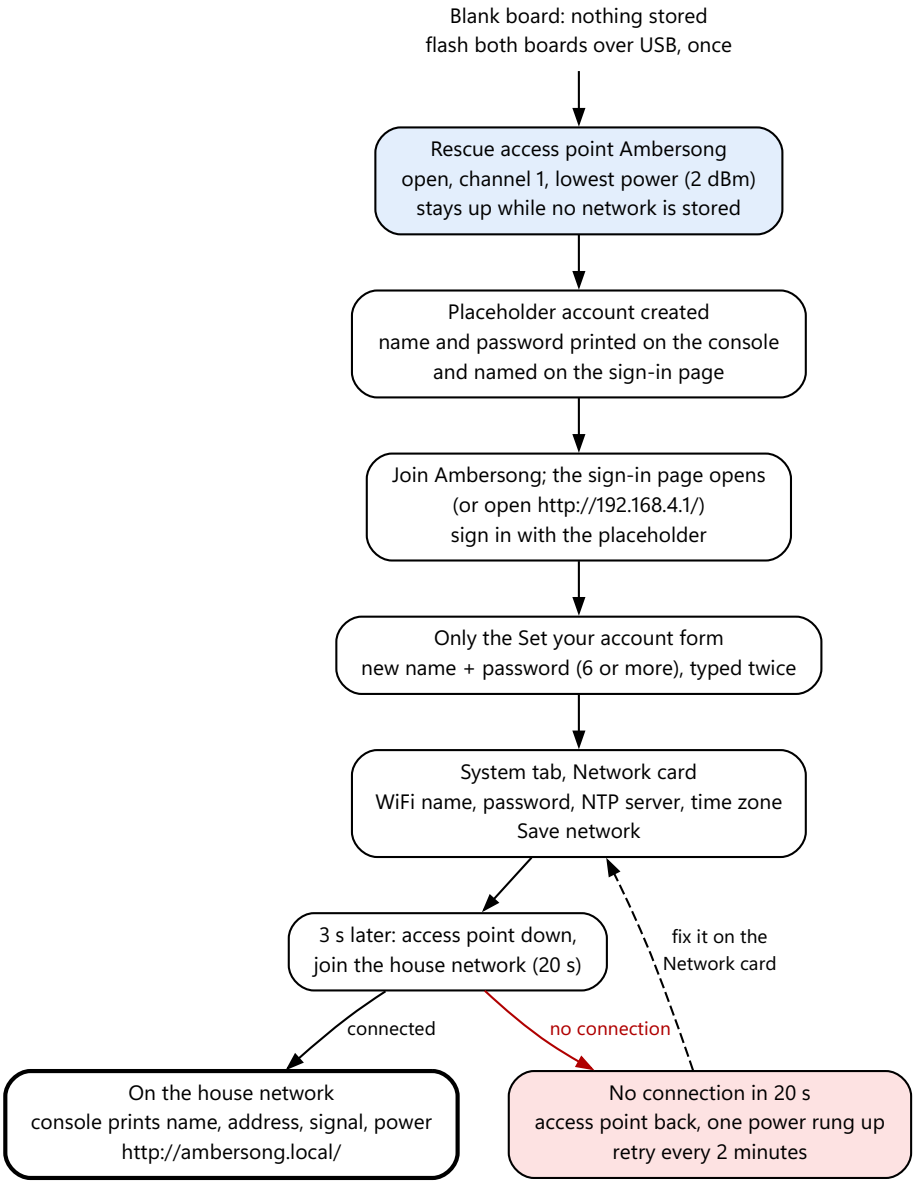
Table 86. The builder checklist

What	Why	Where
Your portal account	A blank board creates a placeholder account whose name and password are public: the console prints them and the sign-in page names them. Until the account stops being the placeholder, the portal refuses everything but the Set your account form, and that form asks you to replace both the name and the password (section 8.6.3 (p. 200)).	On the portal, at the first sign-in (section 8.2.2 (p. 189)). Do not compile your own credentials into the image; the first-login form is the place for them.
Your time zone	The default is `EST5EDT,M3.2.0,M11.1.0`: eastern North America, UTC−5, daylight time from the second Sunday of March to the first Sunday of November.	The Network card on the System tab (a POSIX TZ rule), or the default in `loadPrefs()` in `src/s3/net.cpp`.
Your time server	The default primary server is `pool.ntp.org` `time.nist.gov` is the fixed second.	The Network card, or the same default in `loadPrefs()`.
The **w** prompt's time zone	The console's `w` clock prompt uses a fixed `EST5EDT` rule, whatever zone the portal holds, and leaves that rule in force afterwards (chapter 4 (p. 46), section 4.9 (p. 57)).	`setClockInteractive()` in `src/s3/main.cpp`: put your own POSIX TZ string there, or read the one the network code holds.

The rescue access point has no WiFi password, by design, so there is nothing to change there. What that costs is in section 8.6.7 (p. 201).

8.2.2 From a blank board, step by step

Figure 21. First-time setup, from a blank board to the house network



A blank board has an empty NVS ("non-volatile storage", the ESP32's key-value store in flash). This chapter uses two of its areas: `net` (the WiFi network, the time server and the time zone) and `auth` (the portal accounts). The machine's own settings are stored elsewhere (chapter 7 (p. 153)).

1. Flash both boards over USB, once (chapter 3 (p. 25), section 3.1 (p. 25)). After that every update can go over the air, the audio board's included. The audio board has no reset line from the S3, so after its first flash it is reached only through the S3's relay.
2. The S3 raises its rescue access point. With no network name stored, it skips the join and raises the network `Ambersong` on channel 1 at once. The network is open: joining it asks for no password. With no network

configured the retry loop never runs, so the access point stays up for as long as the board is powered.

3. The portal creates the placeholder account. It finds no account stored and creates the shipped placeholder administrator account. The console prints its name and password, and the sign-in form's "set your account" page names them too.
4. Sign in and replace the placeholder. Join `Ambersong`. On most phones the sign-in page opens by itself (section 8.3.3 (p. 193)); otherwise open `http://192.168.4.1/`. Sign in with the placeholder. The portal now refuses everything except four addresses (section 8.6.3 (p. 200)), and the page shows only a Set your account form. Choose a new name and a password of at least six characters, typed twice. Only then does the rest of the portal open. The form insists on a name other than the placeholder's, because an account that keeps the shipped name keeps half the shipped credentials. That rule is the page's: the radio itself would accept a new password alone. After the change the page drops its cached copy of the page layout (which carries the old name) and reloads.
5. Give it the house network. On the System tab, Network card, fill in the WiFi name, the WiFi password, the NTP server and the time zone. Press Save network. The S3 stores them, and 3 seconds later it drops the access point and joins the house network by itself.
6. On success the console prints the network name, the address the router gave the radio, the received signal and the transmit power that worked. From then on the radio is at its house address or at `http://ambersong.local/`.
7. On failure (no connection within 20 seconds) the rescue access point comes back, the transmit power climbs one rung (section 8.4 (p. 197)), and the house network is retried every 2 minutes.

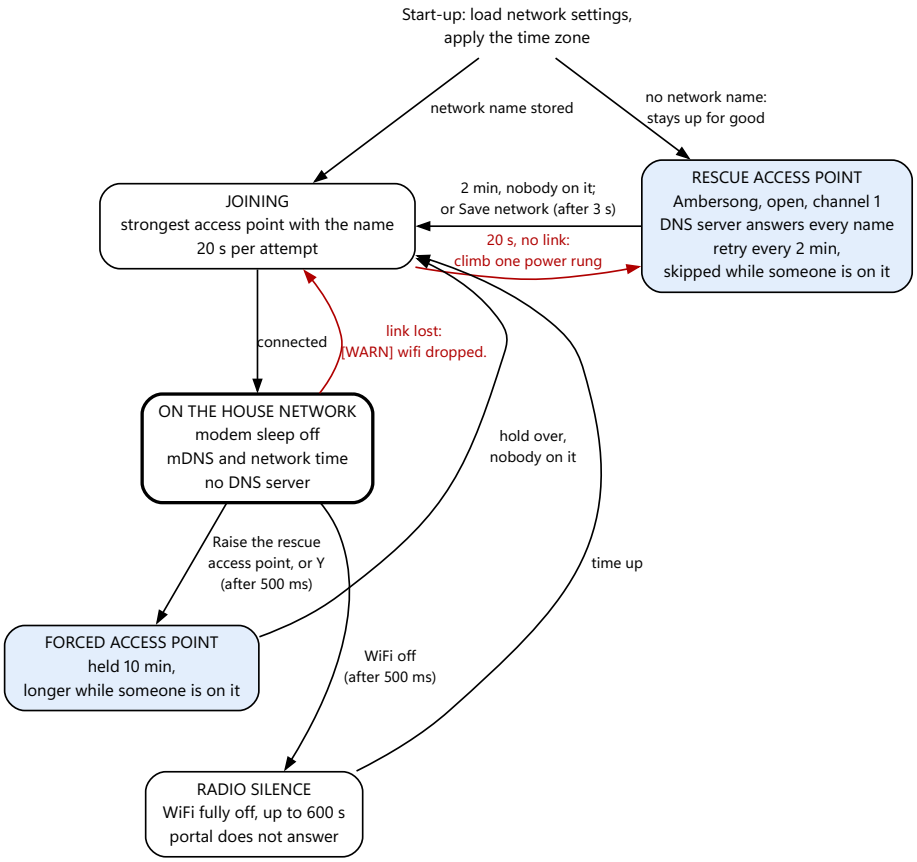
Without a browser, the console key `y` sets the network over the USB cable: type the name, a space and the password, on one line (section 8.9 (p. 209)).

The rest of first-time setup (homing, the index band, the dial samples) is in chapters 5 (p. 73) and 6.

> Note A blank board's access point transmits at the lowest power, 2 dBm. The transmit-power ladder climbs only on failed joins, and a blank board has no network to fail on. On this machine an access point at 2 dBm was visible at 44 % but could not be joined. Stand close; if it still fails, set the network with `y` over the cable you just flashed with, or step the power with `B` (section 8.4 (p. 197)).

8.3 How the radio stays on the network

Figure 22. How the S3 moves between joining, the house network, the rescue access point, a forced access pointand radio silence



Two terms first. The S3 is a station (STA) when it is a client of the house router, and an access point (AP) when it is its own small network. It can be both at once on its one radio (AP+STA), but that is the slow way (section 8.3.1 (p. 191)).

The rule behind all of it: the radio must never become unreachable. A failed join ends on the rescue access point; a forced access point ends; radio silence ends. Nothing reachable from the portal is unbounded.

8.3.1 Joining the house network

- It picks the strongest access point carrying the network's name, after scanning every channel, not the first one heard. A house with a mesh node or an extender has several.
- It takes a lone access point (one with nobody connected) down for the length of the attempt. One radio cannot beacon an access point on channel 1 and at the same time hold the router's tight authentication round trip on another channel. Once, every retry for half an hour failed that way: the router was found, the authentication timed out. If somebody

is connected to the access point, it stays up and the join runs in AP+STA, the slow way.
- It gives each attempt 20 seconds. Connected: the access point (if up) goes down, modem sleep is turned off, and the S3 starts its network name and network time (section 8.5 (p. 199)). Not connected after 20 s: the transmit power climbs one rung, the console prints `join failed at X dBm – trying Y dBm next time.`, and the rescue access point comes up.
- Modem sleep stays off. With power save on, the radio listens only at each beacon from the router, and connections took 0.5 to 1.0 s to open and page loads 4 to 19 s. With it off, a page took 1.1 s instead of 9.0 s. It costs about 80 mA, which does not matter on a set powered from the mains. The S3 turns it off after it connects (asked before, the request is ignored) and asserts it again every 5 seconds, because it once came back by itself after an update.
- A lost link prints `[WARN] wifi dropped.` and starts a new join at once.

8.3.2 The rescue access point

Table 87. The rescue access point

Property	Value
Name	`Ambersong`
Security	Open: no WiFi password
Channel	1
Hidden	no
Devices at once	2 at most: “a rescue hatch, not a hotspot”
Address	`http://192.168.4.1/` (the ESP32’s default access-point address); also `http://ambersong.local/`

When it comes up. At once and for good on a board with no network stored; 20 seconds after a failed join; 20 seconds after the house link drops (the rejoin fails first); and on demand (section 8.3.4 (p. 194)).

Going home. While the access point is up and a network name is stored, the S3 tries the house network again every 2 minutes. If anyone is connected to the access point, the attempt is skipped: a retry would tear the access point down under the person using it to fix the settings. When nobody is connected, the access point disappears for up to 20 seconds during the attempt; someone trying to join at that moment must wait.

What the console says. When it raises the access point, the S3 asks the WiFi driver what it really holds and prints either `raised and the driver confirms it` or `raised but THE DRIVER DOES NOT CONFIRM IT`, then a line `open, no password  http://192.168.4.1/  channel 1  bssid …` (the BSSID is the access point’s own hardware address). If all three attempts fail: `[FAIL] the access point will not start. Set the network over the cable with 'y', or reboot.`

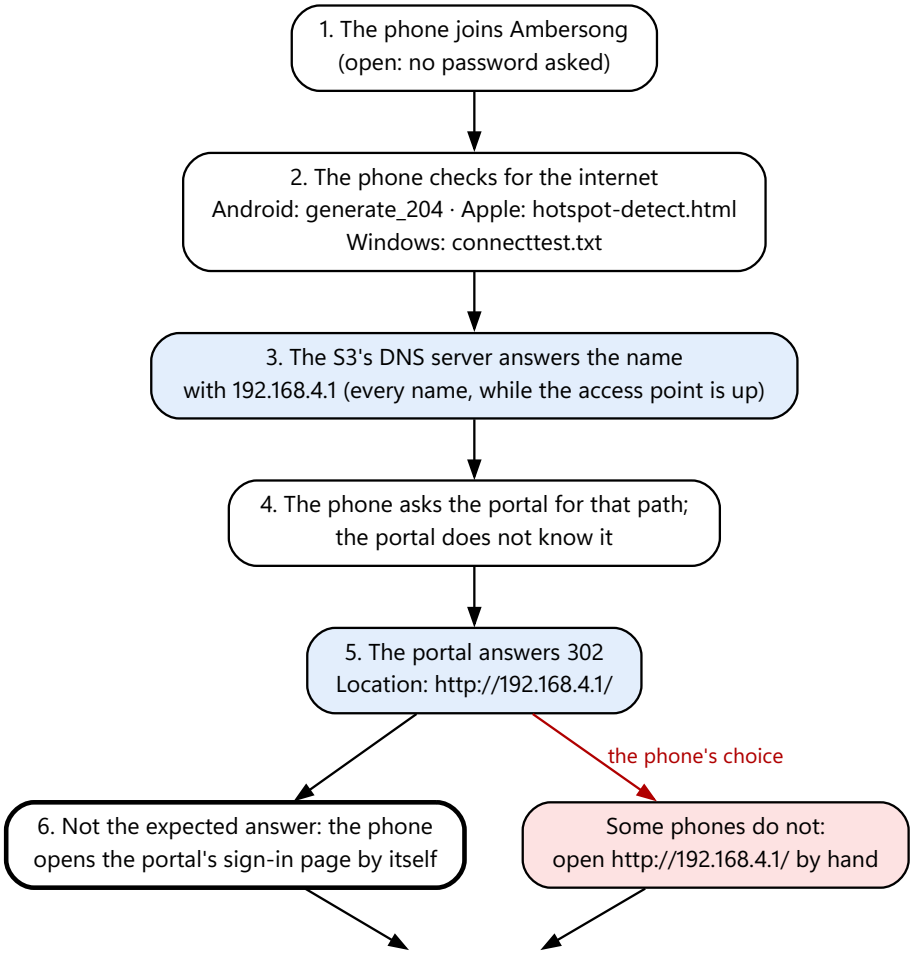
What it hears. The S3 logs what happens on its access point: `[ap] started`, `[ap] stopped`, `[ap] probe request heard, N dBm`, `[ap] a device`

`ASSOCIATED` and `[ap] a device left`. A count of joined devices cannot tell "nobody tried" from "somebody tried and failed"; these lines can. A probe request heard while a phone scans nearby proves that the S3's receiver works on channel 1.

The cost of an open access point is in section 8.6.7 (p. 201).

8.3.3 The captive portal

Figure 23. How a phone that joins the rescue access point is led to the portal's sign-in page



A phone that joins a WiFi network checks whether it reaches the internet by fetching a known address. Android asks for `generate_204`, Apple devices for `hotspot-detect.html`, Windows for `connecttest.txt`, and so on. When the answer is not the one it expects, the phone concludes that the network wants a sign-in and opens that page by itself. Two pieces make the S3 answer that way:

- A DNS server (the service that turns names into addresses). While the access point is up, it answers every name with the access point's own

address, 192.168.4.1. It starts on the first pass after the access point comes up and prints `captive portal: every name now points at this radio.`. It stops as soon as the access point goes down (the radio went home, a join attempt took the lone access point down, or radio silence began). It never runs on the house network.
- A catch-all redirect. The portal answers any path it does not know with a redirect (HTTP 302). On the access point the redirect names `http://192.168.4.1/` in full, because the phone's check arrives under someone else's host name and a bare `/` would send it back to that host. On the house network it names `/`.

So the phone's check reaches the S3, asks for a path the portal does not know, is redirected to `http://192.168.4.1/`, and the phone shows the portal's page. Whether it opens the page is the phone's choice some do not, and then you type the address. The portal's sign-in still guards every action.

8.3.4 Forcing the access point

The rescue hatch can be tested on purpose: the button Raise the rescue access point (System tab), or the console key `Y`. The answer reads `raising the access point - join "Ambersong", then http://192.168.4.1/.` The driver's confirmation appears on the console only, as `access point is up.` or `[WARN] the access point did NOT come up - the driver holds no AP.`

A forced access point holds for 10 minutes, and longer while somebody is connected to it (the retry skips while a client is on it and looks again every 2 minutes). Then the S3 goes home by itself. The hold is bounded on purpose: an unbounded hold, reachable from the portal, would strand the machine off the house network until a power cycle.

8.3.5 Radio silence

WiFi off for 2 minutes (System tab, action `sys.quiet`) turns the S3′s WiFi fully off for a number of seconds: 120 from the button, at most 600. The answer is `wifi off - it will return by itself.` When the time is up the console prints `radio silence over - rejoining.` and the S3 rejoins by itself. It exists to hear whether the S3′s own 2.4 GHz transmitter is what breaks up the audio board's Bluetooth sound. During radio silence the portal does not answer.

8.3.6 Worked example: the router is replaced

Table 88. Worked example: the router is replaced

Time	What happens
00:00	The link drops: `[WARN] wifi dropped.`. The S3 tries to join for 20 s and fails.
00:20	The transmit power climbs one rung, and `Ambersong` comes up.
02:20	Nobody is connected. The access point goes down and the S3 tries the house network for 20 s. It fails and climbs another rung.
about 02:40	The access point is back.
03:00	A phone joins `Ambersong`. From now on every 2-minute retry is skipped. The sign-in page opens on the phone by itself (or open `http://192.168.4.1/`).
later	The user signs in, types the new network's name and password on the Network card, and presses Save network.
3 s later	The S3 joins the new network, turns modem sleep off, starts its network name and network time, and drops the access point. The rung that connected is kept.

8.3.7 How it works inside

This section is for anyone changing the network code; it names the functions and variables behind sections 8.3.1 (p. 191) to 8.3.5.

FOR FIRMWARE CHANGES

The state machine is `Net::loop()` in `src/s3/net.cpp`, called from the main loop on core 1, right after the link poll, on every pass. Its state is a handful of file-level variables: `gSta` (on the house network), `gApUp` (our access point is up), `joining` and `joinStart`, `lastTry` (the last attempt to go home), `pendingJoin` (a scheduled rejoin), `gForced` and `forcedAt` (a forced access point), `quietUntil` (radio silence), the request flags `gWantAp` and `gWantQuiet`, `gTxQ` (the transmit power in quarter-dBm), and the captive portal's DNS server `gDns` with its flag `gDnsUp`.

`begin()` loads the `net` area (`ssid`, `pass`, `ntp`, `tz`), applies the time zone at once (so local time is right with no network at all), registers the WiFi event handler, calls `WiFi.persistent(false)` (the Arduino library must not keep a second copy of the credentials), sets the host name `ambersong`, and starts a join.

`startJoin()` raises the access point instead if no name is stored. Otherwise it takes a lone access point down with `WiFi.softAPdisconnect(false)`, sets `WIFI_ALL_CHANNEL_SCAN` and `WIFI_CONNECT_AP_BY_SIGNAL`, sets the mode (`WIFI_AP_STA` if the access point stayed up, else `WIFI_STA`), re-applies the transmit power (every mode change resets it), calls `WiFi.begin()` and starts the 20-second clock.

`loop()`, in this priority order on every pass:

1. The DNS server follows the access point. While `gApUp` is set it is started if not running, then answers one waiting request; once `gApUp` clears it is stopped. This runs first, before any step below can return.

2. A radio-silence request more than 500 ms old: carry it out, and return.
3. A force-AP request more than 500 ms old: carry it out, then print whether the driver really holds an access point.
4. Radio silence running: return until it ends; then rejoin.
5. A scheduled rejoin now due: drop the station if joined, stamp `lastTry`, start a join.
6. Joining. Connected: `gSta` set; if the access point was up, `softAPdisconnect(true)` then `WiFi.mode(WIFI_STA)`, a change from AP+STA that does not pass through "no mode"; modem sleep off with `WiFi.setSleep(WIFI_PS_NONE)` print the link, start mDNS and network time. Twenty seconds without a link (`JOIN_MS`): climb one rung, raise the access point, stamp `lastTry`.
7. On the access point, with a name stored, more than 2 minutes since `lastTry` (`RETRY_MS`) and not inside a forced hold: clear `gForced`, stamp `lastTry` skip the attempt if anyone is connected, otherwise start a join.
8. Was on the house network and lost it: print the warning and start a join.
9. On the house network, every 5 seconds: mark network time fresh if it has synchronised (chapter 4 (p. 46), section 4.9 (p. 57)), and assert `WIFI_PS_NONE` again.

`startAp()`, the rescue access point:

- `WiFi.disconnect(false, false)` (stop chasing the router, keep the radio on), 50 ms, `WiFi.mode(WIFI_AP_STA)`, 100 ms. It never passes through **`WIFI_MODE_NULL`**. In the Arduino core, going to "no mode" de-initialises the WiFi driver, and the next mode change re-initialises it onto network-interface objects the library never destroyed (arduino-esp32 issue #7232). The symptom is what this machine did for weeks: the access point's start event fires, `softAP()` returns true, and nothing is on the air.
- Up to 3 attempts of `WiFi.softAP()` with the name `Ambersong`, no passphrase, channel 1, not hidden, at most 2 clients, 400 ms apart. The passphrase argument is `nullptr`, which makes the network open. A **`true`** is not believed unless `apReallyUp()` agrees: it asks the driver whether its mode includes AP and whether its AP configuration holds a non-empty name. `softAP()`'s return cannot answer that, because the library skips the driver call when the new configuration is byte-identical to the old one and returns true anyway.
- `WiFi.enableSTA(false)` (AP+STA to AP, with no de-initialisation on this path), then re-apply the transmit power.
- `esp_wifi_set_event_mask(0)`, so the driver reports probe requests (masked by default in ESP-IDF).

- Print what the driver holds, then the `open, no password` line, and start mDNS.

The redirect is the portal's handler for unknown paths (`server.onNotFound` in `Portal::begin()`, `src/s3/portal.cpp`). It names `/` when `Net::isSta()` is true, and otherwise `http://` plus `WiFi.softAPIP()`. The DNS server is the Arduino `DNSServer`, started as `gDns.start(53, "*", WiFi.softAPIP())`.

`forceAp()` cancels any pending join, sets `gForced`, stamps `forcedAt` and **`lastTry`**, and raises the access point. Without the `lastTry` stamp the very next pass saw a retry overdue since boot, and the forced access point lived about one loop tick. The hold is `FORCE_HOLD_MS`.

`quiet(seconds)` caps the time at 600 s, clears all state, calls `WiFi.disconnect(true, false)` and `WiFi.mode(WIFI_OFF)`. Unlike every other path in this file it does go through the driver's de-initialisation; that is its purpose, and it has never failed.

Portal requests only ask (section 8.7.1 (p. 202)): `Net::requestForceAp()` and `Net::requestQuiet()` set a flag and a timestamp, and `loop()` acts more than 500 ms later. A network change from the portal (`Net::setWifi()`) sets `pendingJoin` 3 s ahead.

8.3.8 Two diagnostics

The console key `i` prints the network state and two reports (section 8.9 (p. 209)):

- What the driver really holds: the mode, the transmit power as the driver reports it, the country and channel plan, the access point's configuration read back, the beacon interval, the radio modes, the channel, and the access point's address and client count. It exists because, for months, "`softAP()` returned true" was the only evidence ever held that the access point transmitted. It adds `*** LOW - the transmitter is turned down ***` to any power under 10 dBm, although the ladder's lower rungs are normal (chapter 12 (p. 290)).
- A scan of every channel, only when the S3 is not on the house network. It lists every network heard, marks those that carry the configured name, and gives channel, signal, security and BSSID. It tells apart three failures that look alike from the inside: "I cannot find the router", "I found it and it refuses me", and "there are three and I picked the far one". The scan holds the main loop for a few seconds.

8.4 Transmit power

The S3 learns the WiFi transmit power it needs, on a ladder of six rungs:

Table 89. Transmit power

Rung	Stored value (quarter-dBm)	Transmit power (dBm)
1 (floor and default)	8	2.0
2	20	5.0
3	28	7.0
4	34	8.5
5	44	11.0
6 (ceiling)	60	15.0

A failed join climbs one rung, and whichever rung connects is kept. After the top rung the ladder wraps back to the bottom. After the first success the walk normally never happens again.

Why a ladder. A join can fail for two opposite reasons: too much power (on this board the transmitter then radiated nothing at all) or too little (a distant router). From the inside the firmware cannot tell which, so it walks every rung. It starts at the bottom, because 2 dBm is ample for a router a few metres away.

The 15 dBm ceiling is a firmware choice, measured on this machine: a forced access point at 20 dBm was seen by no device for 90 seconds; at 15 dBm two devices saw it within 22 seconds, and a PC joined it and signed in to the portal. The ceiling is held in the firmware because the fault is believed to be a firmware problem. The 2 dBm floor is also a firmware value; the board once connected at 2 dBm when every higher rung then in use radiated nothing.

Where it is stored. Not with the network settings but in the machine's settings, as `wifiTxQ` (default 8), saved with the rest (chapter 7 (p. 153), section 7.3.3 (p. 159)). It is not a portal setting and not in the settings file; an old file's `wifiTxQ=` line is ignored on upload, so old backups still load cleanly. Only the console reaches it:

- `B` steps it by hand along 60, 44, 34, 28, 20, 8, cycling, marks it for saving (it is written about 2 s later) and prints what the driver applied. `B` keeps its own place in that list from boot, so its first press after a boot always gives 44 (11 dBm), whatever the current rung.
- `D` prints it among the settings, in raw quarter-dBm, with a `#` line giving the dBm.

The rescue access point transmits at the learned rung. That is why a blank board's access point runs at 2 dBm (section 8.2.2 (p. 189)).

> FOR FIRMWARE CHANGES
>
> How it works inside. `applyTxPower()` in `src/s3/net.cpp` clamps `gTxQ` to 8..60 and hands it to `esp_wifi_set_max_tx_power()`. It is called after every WiFi mode change, because the driver does not carry the power across one. `TX_LADDER` holds the rungs; `ladderNext()` returns the rung above the rung nearest to the current value. At boot `applySettings()`

> pushes `cfg.wifiTxQ` into the network code; the main loop copies `Net::txPower()` back into the settings on every pass and marks them for saving, so the learned rung survives a power cut and the network code wins while it is hunting.

8.5 Network name and network time

The network name (mDNS). The radio answers as `ambersong.local`, advertising the service `_http._tcp` on port 80. mDNS (multicast DNS) lets devices on the same network find it by that name without a DNS server. It starts when the S3 joins the house network or raises its access point. The name keeps answering after the S3 comes home from its rescue access point: in a test, it answered within 18 s of the return and on every check after. No restart is needed.

> FOR FIRMWARE CHANGES
> Inside, `startMdns()` does nothing while its flag `mdnsUp` is set. A lost link and radio silence clear the flag, so the next start runs `MDNS.begin()` again.

Network time. The S3 starts network time (SNTP) on every successful join, with the stored server and `time.nist.gov` as the second. Saving the Network card stores the server and the time zone, applies the zone at once, and restarts network time if the S3 is on the house network. Everything else about time — what "fresh" means, the hourly correction of the battery clock, the time zone, the `W` prompt, the battery-clock pills — is in chapter 4 (p. 46), sections 4.9 (p. 57) and 4.10.

8.6 Accounts, sign-in and security

The portal is built against one threat: "people in the house, and a guest who is curious". The portal protects the machine from those, and no more. It uses plain HTTP, with no encryption (TLS), on purpose: against that threat model, TLS on an ESP32 costs more than it buys.

8.6.1 Roles

The split is about damage, not secrecy: a guest gets "the music and the lights, not the machine", and sees everything.

Table 90. Roles

What the account can do	A normal account (guest)	The administrator
See every value	yes	yes
Volume and mute	yes	yes
Display brightness and the three panel-lamp levels	yes	yes
Bluetooth: play, pause, next, previous, disconnect, pair	yes	yes
Change their own password	yes	yes
The needle and its calibrations, gain staging, everything else that changes what the machine is	no	yes
The network, accounts, updates, the settings file, reboots	no	yes
The Console tab	no	yes

8.6.2 Accounts and passwords

- Four accounts at most, each in its own slot. Slot 0 is the only administrator. It cannot be deleted or demoted; new accounts are always normal; nothing can promote one. A machine with no administrator would be a machine that needs a cable.
- Names and passwords. Passwords are at least 6 characters. A new user's name is 2 to 16 characters; a renamed account's, 1 to 16.
- Changing a password. Anyone may change their own (the old one is required). The administrator may change any other account's password without knowing the old one, so a guest's "I forgot mine" needs no cable. The page has no button for that; it is `POST /api/passwd` with the target slot `i` (Appendix B (p. 321)). From the page, a forgotten guest password is handled by removing the account and adding it again.
- Passwords are stored hashed, because anyone with a USB cable can read this flash and people reuse passwords on things that matter. Each is one pass of SHA-256 over an 8-byte random salt followed by the password.
- Not in the settings file. Restoring a settings file does not restore accounts, and the WiFi password lives only with the network settings.

8.6.3 The placeholder gate

While the signed-in account is still the shipped placeholder, the portal refuses every request with HTTP 403 `{"e":"mustchg"}`, except four: `/api/boot`, `/api/state`, `/api/passwd` and `/api/logout`. The page then shows only the Set your account form.

"Still the placeholder" means: the name is the placeholder's and the stored password is the placeholder's. So a new password alone clears the gate, and so does a rename alone; the page asks for both (section 8.2.2 (p. 189)). The page learns about the gate from the live state (`mustchg`), not from the cached page layout, because that cache is not refreshed when a password changes.

8.6.4 Sessions

- Kept in memory only. A restart of the main board signs everyone out. The front switch does not restart it; a portal reboot, a crash or unplugging the set does.
- Four at once. A fifth sign-in evicts the stalest session (“never refuse a login”).
- 5 minutes idle, sliding. Every request restarts the 5 minutes. The browser’s cookie lives 24 hours, but the server’s 5-minute rule is what governs.
- The token is 32 hexadecimal characters (128 random bits), in the cookie `amb`.

When a session expires, the next request answers 401, the page stops polling and shows the sign-in form.

8.6.5 Password guessing

Failures are counted per device address; four addresses are tracked. From the 5th failure in a row, that address is locked out for 1 minute, then 2, 4, 8 and so on, doubling up to 64 minutes. While locked, a sign-in answers HTTP 429 with the seconds left. A success clears the record. The code’s reasoning: “Five wrong guesses is a person who has forgotten; twenty is not.”

8.6.6 A lost administrator password

Nothing on the portal can reset slot 0. There is no password reset on the front of the machine: the USB socket is the recovery path. The console key ~ erases all portal accounts and restores the placeholder; then sign in and set your account again (section 8.2.2 (p. 189)). The web console can send ~ too, but only from an administrator session that is still open, so in practice a lost administrator password means the USB cable.

8.6.7 What the portal does not protect

Read this before you put a build like this on a network you share.

- Plain HTTP. Passwords and session cookies cross the WiFi unencrypted. On the house network the router’s WiFi encryption at least covers the radio link.
- The rescue access point is open. On it nothing is encrypted at all. While it is up, the portal sign-in and the house WiFi password typed during a rescue cross the air in clear, and anyone within radio range can capture them. The access point is up only when the house network cannot be reached or when someone raises it by hand, and it closes when the radio gets home. The portal sign-in still guards every action on it. This is the price of a rescue network that needs no password at the moment you need it most.
- Weak password hashing. One SHA-256 pass with an 8-byte salt and no key stretching. The flash is not encrypted, so anyone with the USB cable can read the hashes.

- No CSRF token. Protection against a foreign web page making requests in your name rests on the cookie's `SameSite=Lax`.
- The lockout tracks four addresses. A fifth address takes over slot 0's record and resets it.
- One administrator. Losing its password needs the cable, and ~ erases every account.
- The **y** prompt echoes the WiFi password into the console log, readable by any administrator with the Console tab open, and onto USB.
- The web console has the cable's power, including z (erase the WiFi radio's RF calibration) and ~ (reset the accounts). It is administrator-only.

What is enforced instead: no anonymous access to any API, salted hashes, sessions in memory only, the lockout, and no credentials in the settings file.

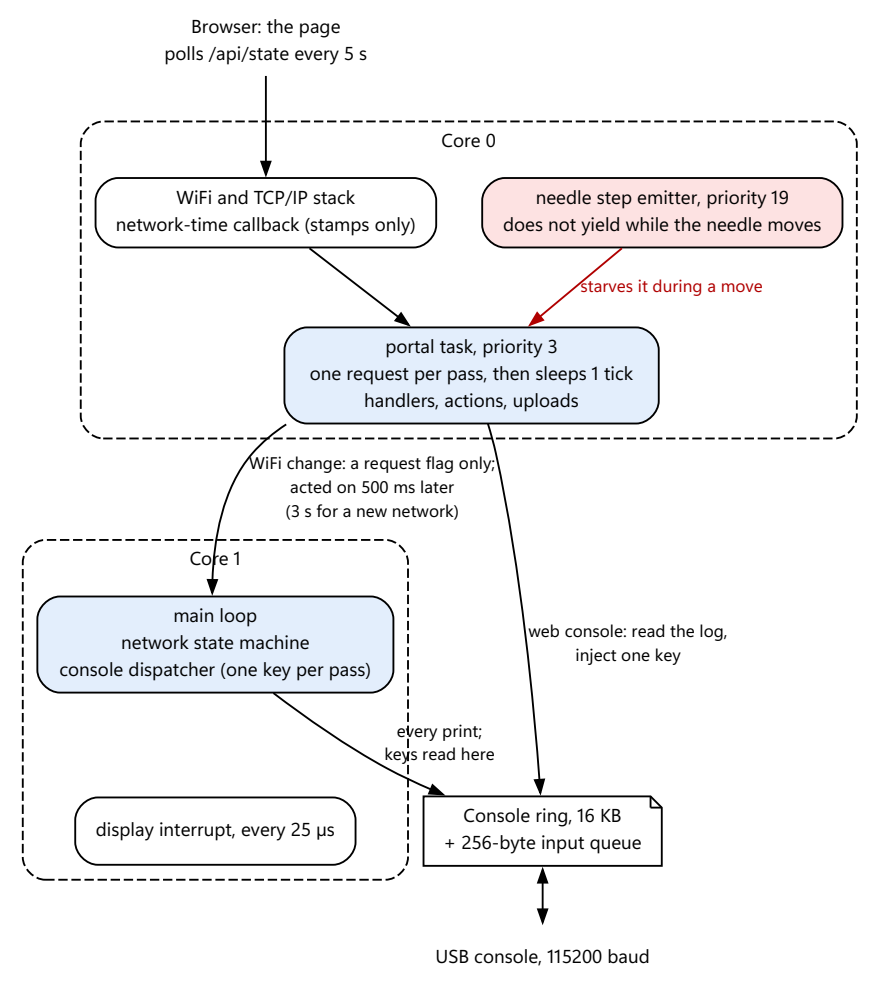
8.7 How the portal works inside

This section explains why the portal behaves as it does: why it slows while the needle moves, why it stops answering when it gets too many connections, and how the page is built. The parts that name tasks, functions and files are for anyone changing the firmware.

8.7.1 Tasks and cores

> FOR FIRMWARE CHANGES

Figure 24. Where the portal's parts run on the main board, and how a request reaches the network code



Chapter 2, section 2.4 (p. 18), has every task in one table; chapter 4 (p. 46), section 4.3 (p. 50), the rules for both cores. For the portal:

Table 91. Tasks and cores

Part	Where it runs
HTTP server (`serverTask()`, `src/s3/portal.cpp`)	Its own task `portal`, core 0, priority 3, 8192-byte stack. One `handleClient()` per pass, then a sleep of one tick (`vTaskDelay(1)`). It serves only while the S3 is on the house network or its access point is up; during radio silence it idles.
Network state machine (`Net::loop()`, `src/s3/net.cpp`)	The main loop, core 1.
Console tee `Con` (`src/s3/console.cpp`)	Any task: a locked ring.
Console dispatcher (`loop()`, `src/s3/main.cpp`)	The main loop, one character per pass.
Network-time callback (`onSntpSync()`, `src/s3/net.cpp`)	The TCP/IP stack's task. It only stamps the time.
Portal actions (`doAction()`, `src/s3/settings_table.h`)	The portal task, per request.
The page	The browser.

Why core 0. Core 1 belongs to the display's multiplexing interrupt and the needle's timing. An early measurement showed that WiFi plus four parallel downloads on core 0 kept the display's worst slot error under 100 µs; the finished firmware measured 22 µs idle and 61 µs under eighteen hammering requests. The asynchronous web server library was never tried on this machine.

Why it sleeps one tick every pass. The portal's loop sleeps one tick each pass so the rest of core 0 gets its turn.

The cost: the portal is starved while the needle moves. The needle's step emitter runs on core 0 at priority 19 and does not yield during a move. The portal was measured unreachable 36 % of the time while the needle tracked the knob, and one 933 kB update took 293 s. This is accepted: nobody should need the portal while tuning. Stop the needle, or wait. Keep the needle at rest during an audio-board update, because the portal task relays it (chapter 12 (p. 290) has a starting point for a fix).

FOR FIRMWARE CHANGES

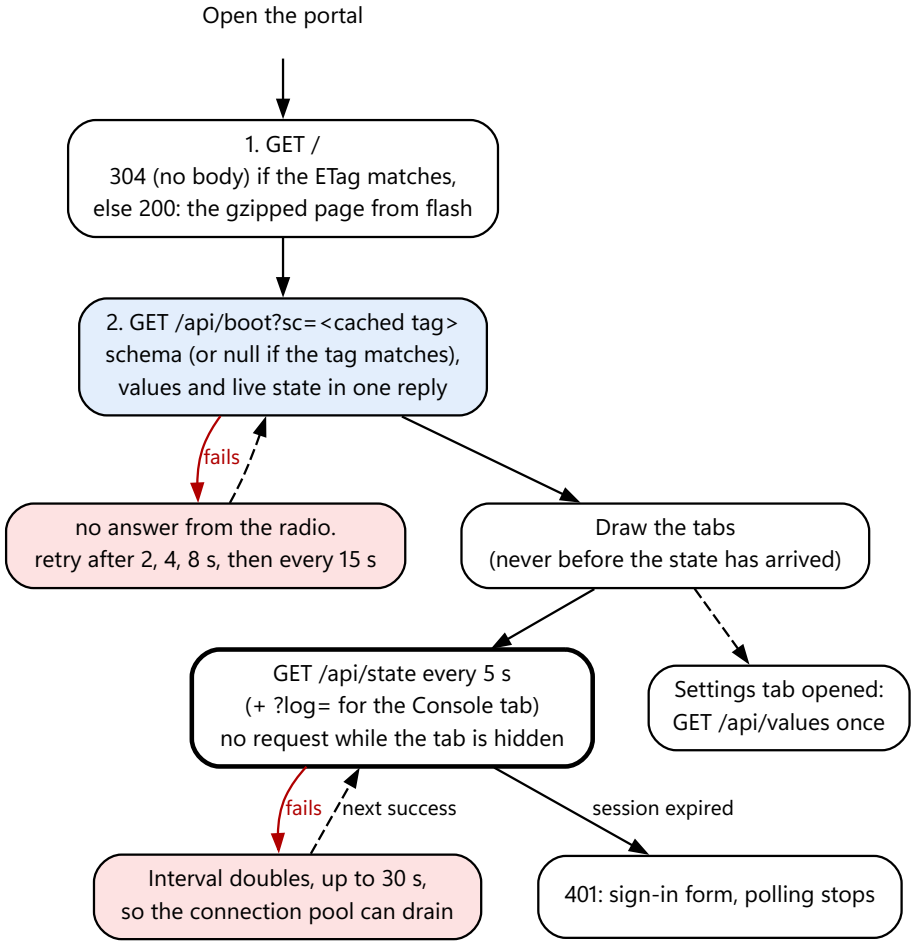
The cross-task rule: a handler never changes the WiFi mode itself. A handler runs on core 0; the network state machine on core 1. When a handler once raised the access point itself, it raced the network loop and tore down the very link its answer still had to travel on. So the two actions that touch the radio, `net.forceAp` and `sys.quiet`, only ask: they set a flag and a timestamp, and the network loop carries the request out more than 500 ms later, so the answer leaves first. A network change from the Network card is deferred the same way, by 3 seconds.

Not under the task watchdog. The portal task is not watched by the S3's 15-second task watchdog (chapter 4 (p. 46), section 4.11 (p. 60)): a flash write legitimately blocks it, and the upload watchdog (section 8.7.3 (p. 206)) covers

a stalled upload. Nothing catches a portal task that hangs for good; unplug the set.

8.7.2 The connection budget

Figure 25. A page load is two requests; then one state poll every 5 seconds, backing off when polls fail



This is the rule that shapes everything the page does. The server is the Arduino core's synchronous `WebServer`, and it handles one connection at a time. Each HTTP connection, once closed, leaves a TCP control block (the network stack's record of a connection) waiting about a minute in the state called TIME_WAIT, against a pool of roughly sixteen. About twenty to forty quick connections use up the pool. The radio then stops accepting connections, while ping stays perfect (ping does not use that pool), and it recovers after a minute or two of silence.

A dashboard polled once a second could never have worked: about sixty outstanding blocks against a pool of sixteen. Worse, the old page kept polling at the same rate while its polls failed, so the pool never drained and the fault looked permanent. The test that settled it was the cheapest one: stop touching the machine, then try once. After 3 minutes without traffic the

first request answered in 1.07 s; 20 back-to-back requests all succeeded; the next 19 all died. Modem sleep, an over-eager socket reaper and core-0 starvation had each been blamed first; each was a real defect worth fixing, and none was the cause. At the new pace, about 60 connections a minute became about 12, and the polls measured afterwards all answered in 0.35 to 3.1 s.

So the page works like this:

- A page load is two requests: the page itself (usually answered 304, “not modified”, with no body), then `/api/boot`, which returns the page layout, the values and the live state in one reply.
- The page layout is cached in the browser. The page describes every setting it draws from a schema sent by the radio (section 8.7.4 (p. 207)). The browser keeps the last schema in `localStorage['amb.sc']` and offers its tag as `/api/boot?sc=<tag>`. The tag is the firmware version plus the administrator bit (`FW_VERSION-0` or `-1`); if it still matches, the radio answers `"sc":null`, and about 11 kB becomes about 3 kB. Every new build changes the tag. The administrator bit is in it so that a guest signing in on a browser an administrator used can never inherit the administrator’s cached page.
- Polling: `/api/state` every 5 seconds. After a failure the interval doubles, up to 30 seconds, so a used-up pool can drain. No request while the browser tab is hidden. Polling stops on a 401, so a half-typed password is not wiped by a redrawn sign-in form.
- The console rides the state poll (`/api/state?log=`), with no connection of its own.
- Settings tabs fetch `/api/values` when opened, not on a timer.
- Every request has a deadline of 12 seconds, covering the body as well as the headers (20 s for a settings upload; none for firmware uploads). “Headers, then stall” is what this machine does when it has run out of control blocks.
- The page never renders nothing. If `/api/boot` fails, it shows `no answer from the radio.` and retries after 2, 4 and 8 seconds, then every 15 seconds. An earlier page simply gave up when its first request timed out, and “it loads but doesn’t show anything” was that line exactly.
- Nothing is drawn before the state has arrived, and drawing never throws. The first page drew before it had any state; the first paint read a field of the still-empty state, threw an error, and everything after it (the first poll, the timer) never ran. The page rendered once, blank, forever. A dashboard that throws is a dashboard that stops.
- Anyone scripting against the radio must stay at about one request every few seconds.

8.7.3 Request handling

The server checks every request the same way. The first three points below are for anyone changing the server; the rest explain answers you may see.

FOR FIRMWARE CHANGES

- Who may ask. Every API handler starts with `requireLogin()`, which answers 401 `{"e":"login"}` when there is no valid session, so a forgotten check fails closed. It also enforces the placeholder gate, in one place: “a rule enforced in fifteen places is a rule with fourteen chances to be forgotten”. Only these are anonymous: `GET /` (the page), `POST /api/login`, `POST /api/logout`, `GET /favicon.ico`, and the redirect for unknown paths. The two firmware-upload handlers do not use `requireLogin()` and repeat the session, administrator and placeholder checks themselves (chapter 3 (p. 25), section 3.10.1 (p. 28)).
- The silent-socket reaper. Browsers open spare connections in advance. `WebServer` accepts one and waits up to 5 seconds (`HTTP_MAX_DATA_WAIT`, fixed inside the library) for a request that never comes, while real requests queue behind it; sign-ins took 15 seconds. So after every `handleClient()`, the server task closes the connection it is holding if it has been open 3 seconds (`IDLE_SOCKET_MS`) with nothing to read. The reaper is off while an upload runs, because an upload has quiet moments. With it (then at 750 ms) a cold sign-in went from 4.8 s to 0.12 s, and three parallel requests from 10, 16 and 19 s to about 1.1 s each. No curl test could have found this, because curl opens exactly one connection.
- The upload watchdog. `otaWatchdog()` runs every pass. If an upload is flagged and nothing has arrived for 15 seconds, it releases the machine: for the S3′s own image it aborts the update and lets the needle and display run again; for the audio board’s relay it aborts the relay. `WebServer` does not reliably report an upload that died with its connection.

- Numbers in requests must be one finite number, the whole string; one decimal comma is read as a point (“107,3” is 107.3). Anything else is refused. Clamping and checks live in the machine, never only in the browser.
- Unknown paths answer 302 (section 8.3.3 (p. 193)). `/favicon.ico` answers 204, no content.
- Counters for “portal dead, ping alive”. The console `s` prints the portal’s `loops` (server task passes), `served` (requests served) and the task’s free stack in bytes. Frozen loops mean the task is starved; climbing loops mean the fault is in accepting connections.

8.7.4 The page: built once, served from flash

The page lives inside the firmware, so it never falls out of step with the program that serves it. How it is built and served matters only to someone changing it.

FOR FIRMWARE CHANGES

The page is one self-contained file, `data/portal.html`: inline CSS, one `<script>` block, no external files. At every build, `scripts/page.py` checks that its script parses, gzips it and writes it into the program as `src/s3/page_gz.h`, with an ETag (a version label that browsers send back to ask "has this changed?"). There is no filesystem: nothing to upload separately and nothing that can fall out of step with the firmware serving it.

Why compress at build time. The uncompressed page took 0.7 to 3.6 s per load on a one-connection server. The ESP32's ROM can decompress but not compress, and compressing at run time would trade one cost for another. The page is about 35 kB of HTML and about 13 kB compressed.

Serving. If the request's `If-None-Match` header carries the ETag, the answer is 304 with no body. Otherwise 200, with the ETag, `Cache-Control: no-cache, must-revalidate` (the browser always asks, so a firmware update is never hidden behind a stale page) and `Content-Encoding: gzip`, straight from flash. The server collects only two request headers, `Cookie` and `If-None-Match`.

The page is driven by the schema. It knows no setting by name except `volume` and `muted` (the Now tab's music card). It draws whatever the schema lists for each setting: key, label, unit, tab, kind, low, high, step, read-only flag and choices. Kind 1 is a checkbox, kind 3 a drop-down, anything else a slider with a number box. Adding a setting is one row in `src/s3/settings_table.h` that row also gives the setting its clamping and its line in the settings file (chapter 7 (p. 153), section 7.11.1). Its scale is why a value can travel between the boards as an integer (tenths of a dB, say) while a person reads "+15.0 dB".

A change posts `/api/set`. The stored value in the reply, possibly clamped, is written back into every widget with that key. Volume and mute follow the physical knob through the live state, except while their slider has focus.

Action buttons are one list in the page (`ACTIONS`): per tab, a label, an action, a value and an administrator-only flag. A value of `?` means "ask": the station marks ask for the frequency, and the drop-sample button for a slot number. A comma typed in the answer becomes a point.

8.8 What the page shows

Tabs, in order: Now (the dashboard and the music card), then the schema's tabs Audio, Display, Lights, Needle, Bluetooth and System, then Console (administrator only), then Account. Appendix E (p. 336) maps every tab, card and button to the call it makes.

Header pills. The header shows the link to the audio board, the needle homed, network time fresh, the clock, the network (its name and signal, or "access point"), the address, and red warnings when something needs attention. Chapter 1, section 1.13 (p. 12), lists every pill and what it means.

Toasts are the message strip. An acknowledgement shows for 2.6 seconds in amber. A refusal or a warning shows for 12 seconds in red. A refusal often arrives on a 200 answer carrying a `warn` field (portal actions, the tuner ends, the settings upload), and the page paints it red. Earlier versions showed several refusals in the success colour; this rule stops that.

The Network card (System tab, administrator):

- Save network stays disabled until the form has been filled from the radio. A failed prefill once let one click save three empty fields and drop the house network.
- A blank password field keeps the stored password. The radio never sends the password back.
- Saving always resends the network name, so changing only the time server or the time zone also makes the radio leave and rejoin its network 3 seconds later.
- The answer is `network saved - the radio joins it in a few seconds`. An empty name is refused: `the network name is empty`.

Firmware uploads show a progress bar. Reboot the main board asks for confirmation first, and on success the page shows only "rebooting", whatever the radio answered (chapter 12 (p. 290)).

8.9 The console, over USB and the web

The S3 has one text console. Every key is a command. The full key list is in Appendix D (p. 332) chapter 1 (p. 3), section 1.2 (p. 4), shows how to start.

Two ways in, one console. The console is the S3′s native USB port, the USB-C socket on the back panel, at 115200 baud. Every byte the S3 prints goes both to the USB port and into a 16 KB ring in memory, which the portal's Console tab reads. Keys typed on the Console tab are queued and read as if typed on USB. The audio board's lines reach the same console through the link, prefixed `[A32]`.

Why a web console. With no cable there is nothing to unplug from a closed cabinet, and no second ground connection between a PC and the machine. The USB socket's ground is the machine's DC-side ground, which sits on mains earth, not on the tube radio's chassis; a PC plugged into it adds a second path to earth (Hardware Bible, chapter 5).

Sending from the web: one key per send. The S3 reads one character per pass of its main loop, and every character is a command. Before this rule a typed word ran as a string of commands: "status" ran `t`, `a` and then a 200-half-step jog that ignores the soft limits, and any word with a `z` erased the RF calibration. The portal now refuses anything longer than one key (`one`

`key at a time - each character is its own command`), except while a prompt is reading a line, when up to 128 characters go through. The portal appends Enter to every send; the console ignores it. After a send, the page polls once early, 900 ms later.

Reading from the web. The page asks for the console's new bytes on its 5-second state poll, at most 4096 bytes each time, oldest first. It keeps the last 60 000 characters and inserts `[... older output was overwritten before it could be shown ...]` or `[... the main board restarted ...]` when it sees a gap.

Line prompts. Two keys read a whole line:

Table 92. The console, over USB and the web

Key	Asks for	Gives up after this long with no key typed	Unit
`y`	`name <space> password` of the house network. Stores them and rejoins 3 s later. A blank line cancels.	60	s
`W`	Local time, `YYYY-MM-DD HH:MM:SS` (chapter 4 (p. 46), section 4.9 (p. 57)).	40	s

Each wait restarts at every character typed, so a prompt can stay open longer while someone types. While a prompt waits, the main loop is held: the clock display freezes, the network state machine does not run, and the S3 sends nothing on the link, so the audio board, which treats two seconds of silence as "amplifier down", mutes. Both prompts keep the S3's watchdog fed, so a prompt left open does not restart the board. When a prompt returns, the rest of the line is discarded, so the tail of an over-long line can never run as one-key commands (where an `m` or a `c` would overwrite a calibration and a `j` would jog 200 half-steps, ignoring the soft limits). The `y` prompt's buffer fits the longest legal pair: a 32-character name, a space and a 63-character password.

> Note `y` echoes what you type, the password included, into the console ring that the Console tab shows, and onto USB.

The live index monitor `ι` prints the index sensor about every 250 ms for up to 60 seconds. It ignores Enter (the portal sends one after every key), stops on any other key, and keeps the audio board awake by pinging it on every pass (chapter 5 (p. 73)).

The network keys belong to this chapter:

Table 93. The console, over USB and the web

Key	What it does
`i`	Joined, or `OWN ACCESS POINT` the network's name, the radio's address and signal; portal up or down, sessions, network time fresh or stale; access-point clients; the driver report; and, only when not on the house network, a scan of every channel (section 8.3.8 (p. 197)). It prints `OWN ACCESS POINT` whenever the S3 is not joined, also while joining or during radio silence (chapter 12 (p. 290)).
`y`	Set the house network (line prompt, above).
`Y`	Raise the rescue access point now; it holds 10 minutes, longer while someone is on it, then goes home.
`~`	Erase all portal accounts and restore the placeholder (section 8.6.6 (p. 201)).
`z`	Erase the WiFi radio's RF calibration in flash and reboot. A transmitter diagnosis; it touches none of the radio's own calibrations.
`B`	Step the transmit power (section 8.4 (p. 197)).

What the console does not show. Anything not written through the console object, including the ESP-IDF framework's own log lines. Those, brownout reports included, go to the S3's UART0, not to its USB console, so silence on the USB console proves nothing about them.

FOR FIRMWARE CHANGES

How it works inside. `Con` (`class ConsoleTee`, `src/s3/console.cpp`) owns the USB port; it is the only file that touches the real `Serial`. The output ring is `LOG_CAP` = 16384 bytes, and `gSeq` counts every byte ever written. Input comes first from a 256-byte injection queue (`IN_CAP`) fed by the portal, then from USB. One spinlock guards both rings and is held only for memory copies; the USB write happens outside it, because it can wait on the host. `write()` reports the full length even when no USB host took anything, so a machine with only the portal attached is not treated as failing.

- Reading: an administrator's page with the Console tab open asks `/api/state?log=<next byte number>`. The reply gains `"log":{"from":F,"next":N,"t":"..."}`, built by `copySince()`: the oldest bytes since `F` first, newlines escaped, other control characters dropped. A `from` older than the ring, or from the future (the S3 rebooted under an open page), resyncs to the oldest byte held.
- Writing: `POST /api/cons` with `k=<text>` the server appends a newline and injects it. More than one character is accepted only while `Con.lineWanted` is true, which only the `y` and `W` prompts set. When a prompt returns, `drainLine()` discards input up to the end of the line, within a 50 ms window.
- Blocking keys: the prompts and `l` call `wdtFeed()` while they wait; `l` also sends a link ping each pass. The `i` scan blocks for a few seconds without feeding, well under the watchdog's 15 seconds.
- One help list, `commandList()`, printed by `?`, at the end of every `s`, and at boot.

8.10 Updating from the portal

Both boards are updated from the System tab's Firmware card: Update the main board and Update the audio board, each a file picker with a progress bar. Chapter 3 is the home of updating: what both upload endpoints check first (section 3.10.1 (p. 28)), the main board's update (3.10.2), the relay to the audio board (3.10.3), the trial and the rollback (3.11), and a full walk-through (3.12).

What to remember from the portal's side:

- The answer can lie. A successful update reboots, and the reboot can beat the HTTP reply, so the page may report failure for an update that worked. Check the running line on the System tab afterwards.
- Only the administrator, and not while the account is still the placeholder.
- A wrong file is refused before anything is written: the two pickers sit side by side and the two builds look alike. For the main board the wrong-chip reason reaches only the console; the page shows "flash" (chapter 12 (p. 290)).
- During an update of the main board the needle stops and the display goes dark; the sound keeps playing. During an update of the audio board the sound stops; the needle and display carry on.
- No second main-board update during the trial minute, and Reboot the main board during that minute rolls back at once (chapter 3 (p. 25), section 3.11 (p. 32)).
- Keep the needle still during an audio-board update (section 8.7.1 (p. 202)).

Which version am I running? The running line on the System tab shows the main board's version and commit stamp, `v.1.<stamp> (<hash>)`. The audio board's version is only on the console. Chapter 3, section 3.13 (p. 37), explains both.

8.11 When it goes wrong

The last column is the fault's class on the recovery ladder (NOTE, DEFECT, BLOCKER), set out in About this book, and whether the path has ever run on the radio.

Table 94. When it goes wrong

What you see	What happened	What the firmware does	What you do	Class
The radio is off the house network; `Ambersong` appears	Wrong WiFi password stored: no connection within 20 s.	Raises the rescue access point; climbs one transmit rung per failure (all six, wrapping); retries every 2 minutes.	Join `Ambersong` (open; the page opens by itself on most phones, otherwise `http://192.168.4.1/`) and fix it on the Network card (type the new password; blank keeps the old), or console `y`.	NOTE: heals once the setting is fixed, no cable. The access point's return after a failed join, on the current

What you see	What happened	What the firmware does	What you do	Class
				retry path, has never run on the radio.
`[WARN] wifi dropped.` the access point appears	The router is off or was replaced.	Rejoins; access point after 20 s; retries every 2 minutes, never while someone is on the access point.	Wait, or fix it through the access point (section 8.3.6 (p. 195)).	NOTE
`the network name is empty`	An empty network name was submitted.	Refuses (HTTP 400). The page cannot save before its form is filled.	—	NOTE
`[FAIL] the access point will not start ...`	The driver did not confirm the access point after 3 attempts.	Nothing more.	USB console: `y`, `i`, `B`, `z` or reboot.	BLOCKER if nothing radiates at all (USB only)
The join fails and the access point is not seen	The transmitter is set above what the board can radiate.	The ladder climbs and wraps to 2 dBm; the top rung is capped at 15 dBm.	Wait for the ladder; or `B` over USB.	NOTE
The radio goes home on its own	A forced access point's hold expired with nobody on it.	Goes home after 10 minutes.	Nothing.	NOTE
The phone joins `Ambersong` but no page opens	The phone decides whether to open the page.	The DNS server and the redirect keep answering.	Open `http://192.168.4.1/` by hand.	NOTE. A phone opening the page by itself has not yet been seen on this radio.
No `captive portal: ...` line on the console	The DNS server failed to start.	Tries again on every pass while the access point is up; the portal still answers at its address.	Open `http://192.168.4.1/` by hand.	NOTE; never seen
A blank board's access point cannot be joined	No network name, so no failed join to climb the ladder: it stays at 2 dBm.	Nothing.	Move closer; or `y` / `B` over USB.	NOTE when moving closer works; BLOCKER only if nothing but the USB console helps
The portal is dead but the radio answers ping	The pool of connection records is used up.	The page backs off to 30 s so the pool drains.	Close every tab on the radio, wait 1 to 2 minutes, try once. `s` shows `loops` and `served`.	NOTE

What you see	What happened	What the firmware does	What you do	Class
The portal is slow while the needle moves	The step emitter holds core 0.	Nothing; accepted.	Stop the needle, or wait.	NOTE
`no answer from the radio.`	`/api/boot` failed.	Retries after 2, 4, 8, then every 15 s.	Wait.	NOTE
The sign-in form comes back	The session expired (401).	Shows the sign-in form; stops polling.	Sign in.	NOTE
Sign-in refused with a wait	Five failures from one device (429).	Locked for 1 to 64 minutes, doubling.	Wait.	NOTE. Never run beyond its first steps.
The administrator's password is lost	—	Nothing on the portal can reset slot 0.	USB console ~ (erases all accounts, restores the placeholder).	BLOCKER for the administration: only the USB console's ~ recovers it; the radio keeps playing
A main-board upload stops part-way	The connection aborted, or 15 s without data.	Aborts; the needle and display come back once the needle's stop has landed; the old program stays in its slot.	Upload again.	NOTE
An audio-board upload stops part-way	As above.	The upload watchdog aborts the relay. A relay failure mid-stream sends no abort, and the audio board stays muted about 20 s until its own timeout (accepted as harmless).	Upload again.	NOTE
The upload is refused at once	The wrong file was picked: its chip number does not match.	Refused before anything is written.	Pick the other file.	NOTE
The page says the update failed, but it worked	The reboot beat the reply.	—	Check running on the System tab.	NOTE
`not rebooting - <why>`	Reboot the main board with an unsaved change that cannot be saved (program confirmed).	Stops the needle, retries the save for 1.5 s; then refuses (409) and lets the needle resume.	Fix the cause, press again.	NOTE
The previous program is back after Reboot the main board (the radio answers `rebooting - this firmware was still on trial, so the previous firmware comes back` the page shows only "rebooting")	Reboot the main board while the new program is on trial.	No save attempted; restarts; the bootloader returns to the previous program.	Upload the new build again if it was good.	NOTE; by design, seen working

What you see	What happened	What the firmware does	What you do	Class
`the audio board is not answering - its settings cannot be changed now`	An audio-board setting was edited while the audio board is silent.	Refuses (409).	Wait for the audio board to answer (chapter 9 (p. 224)).	NOTE; never exercised
The settings download is refused	Settings are locked: the firmware is older than the saved settings.	Refuses (409); the page says why.	Flash the newer firmware (chapter 7 (p. 153), section 7.5.4 (p. 166)).	NOTE; never exercised
`one key at a time - each character is its own command`	A word was typed in the web console with no prompt waiting.	Refuses (400).	Send single keys.	NOTE
`console busy`	The console's input queue is full (256 bytes).	Refuses (503).	Wait.	NOTE
The display freezes, the sound mutes	A `y` or `w` prompt is waiting.	The main loop is held until the line ends, or until 60 s (`y`) or 40 s (`w`) pass with no key typed; the watchdog stays fed, so the S3 does not restart.	Finish the line, or send a blank line.	NOTE; accepted
The portal is gone for up to 10 minutes after WiFi off	Radio silence.	WiFi off, then rejoins by itself.	Wait.	NOTE
`[FAIL] proto self-test - this image will NOT be confirmed; ...`	A broken main-board build.	Boots as usual (display, needle, link, WiFi, portal); never confirms the program.	Sent over the air: Reboot the main board, and the previous program returns (an upload is refused while it is on trial). Flashed by USB: it is not on trial; upload a good build.	DEFECT; never exercised
The image line adds `an earlier update was ROLLED BACK`	The new main-board program hung or crashed on trial; the watchdog or the crash reset it.	The bootloader returns to the previous program.	Upload a fixed program.	NOTE; seen working
A new main-board program runs but never confirms	No network, or the portal task is stuck.	Stays on trial and keeps running; a later reset rolls it back.	Reboot the main board if the portal answers; otherwise unplug and replug.	BLOCKER; never exercised
`this firmware is still on trial - retry in a minute`	A main-board upload was sent during the trial.	Drops the bytes; refuses (409).	Wait for `image confirmed` (about 70 s after boot), then upload again.	NOTE

8.12 Design choices

- A synchronous web server in its own task on core 0. It was the one arrangement measured harmless to the display.
- Plain HTTP, no TLS. Against a household threat model, TLS on an ESP32 costs more than it buys; instead: no anonymous API, salted hashes, sessions in memory, lockout, no credentials in the settings file.

- Roles split by damage, not secrecy. A guest gets the music and the lights, not the machine, and sees everything.
- Sessions expire after 5 idle minutes, sliding.
- No password reset on the front of the machine. The USB console's ~ is the recovery path.
- A public placeholder account with a hard first-login gate. The firmware is published, so no real credentials can be compiled in.
- The gate has no "must change" field in the account record, because the record's size is its length in flash and a new field would discard every stored account; and it is not carried in the cached page layout, where it would never clear.
- One choke point, `requireLogin()`, for sign-in and the gate. One rule in one place cannot be forgotten in the fifteenth handler; the two upload handlers are the only exceptions and repeat the check.
- The page is gzipped at build time, given an ETag and served from flash, with no filesystem. Load time was the page's size; there is no compressor in ROM; a separate filesystem image can fall out of step with the firmware.
- The build fails if the page's script does not parse. A duplicate declaration once shipped and killed the page on the radio.
- The page is generated from one settings table. Adding a setting is one row; four hand-edited places per setting would drift.
- The connection budget: polls every 5 s backing off to 30 s, none while hidden; a two-request page load; the console on the state poll; the favicon answered 204. All because of the pool of about sixteen connection records.
- Silent sockets are closed after 3 s, never during an upload. Speculative browser sockets made 15-second sign-ins.
- Modem sleep off after connecting, and asserted again every 5 s. Measured: page 9.0 s to 1.1 s, state 4.1 s to 0.3 s, connection set-up 0.5–1.0 s to 15–80 ms. The 80 mA it costs does not matter on a mains set.
- The radio must never become unreachable: a rescue access point plus a background retry, reachable from the portal. A fault in the rescue access point is fixed, not recorded as an observation.
- Never take the WiFi driver through "no mode" while the access point or the station must survive, and believe the driver, not `softAP()`'s return. The core's de-initialisation bug and the library's identical-configuration shortcut.
- A forced access point holds 10 minutes, then goes home by itself. An unbounded hold would strand the machine off the house network.
- A lone access point is taken down for a join attempt, and kept if someone is on it. One radio cannot hold the router's authentication while beaconing on another channel.
- The retry waits while anyone is on the access point. Otherwise the settings form dies every two minutes in exactly the case the access point exists for.

- Transmit power is learned on an ascending, wrapping ladder from 2 to 15 dBm, and only the console reaches it. A join can fail from too much power or too little; a settings row once wrote 2.000 where the dump wrote 8, quadrupling the power on a paste.
- Portal handlers only request WiFi changes; the network loop acts 500 ms later (3 s for a network change). No cross-core race, and the answer leaves first.
- Network time is fresh only within 4 hours of a real synchronisation. “Later than 2020” stayed true forever (chapter 4 (p. 46), section 4.9 (p. 57)).
- The time zone is a POSIX rule, applied once. The C library handles daylight saving; the display code used to force the zone on every redraw, which would have overridden the portal’s.
- mDNS is not restarted when the S3 comes home from its access point. Tested: the name answered without it.
- Request numbers are parsed whole, one decimal comma accepted, non-finite values refused. “107,3” was stored as 107, and “nan” passed every clamp (`posMin=nan` would have been a soft limit that does not limit). The clamp is one function used by both the portal and the settings file: “a rule written twice is a rule that will be enforced once”.
- Refusals travel on a 200 with a **`warn`** field and are painted red for 12 s. Refusals shown in the success colour for 2.6 s were missed, repeatedly.
- A reboot refuses rather than lose an unsaved change, and a refused reboot lets the needle resume. “Change something, press Reboot” lost the change.
- The settings download is refused while settings are locked. RAM then holds defaults, and a round trip would overwrite the real calibration.
- One console object for two audiences, **`Con`**, not a redefined **`Serial`**. `Serial` is a core macro, and redefining it depends on include order.
- One key per web-console send, except while a prompt reads a line. Typed words ran as strings of commands.
- A prompt discards the rest of its line, and the **`y`** buffer fits the longest legal pair. An over-long line’s tail ran as one-key commands.
- The live index monitor **`l`** works from the web console.
- The **`w`** prompt keeps its fixed eastern-time rule, with a note for builders. The rule belongs to this build’s location.
- The **`y`** prompt may hold the main loop until 60 s after the last key typed, mute the audio board meanwhile, and echo the password. Judged not to cause meaningful harm in realistic use.
- The rescue access point is open. It is for someone whose radio has lost its network; a passphrase to find and type is one more obstacle at that moment. Its cost is in section 8.6.7 (p. 201).
- The rescue access point is a captive portal. A person rescuing the radio should not have to know `192.168.4.1` either. The DNS server runs only with the access point, so nothing changes on the house network.

- The portal starving while the needle moves is accepted. Nobody should need the portal while tuning; whether to change it is undecided.
- Upload safety: a chip check, a 15-second inactivity watchdog, and the needle stopped and the display darkened only for the S3′s own image. Two side-by-side pickers; dead connections do not always report an abort.
- The S3′s own image runs on trial, and nothing is saved during the trial (chapter 3 (p. 25), section 3.11 (p. 32)).
- An edit of an audio-board setting is refused while the audio board is not answering. Without its answer the edit would go nowhere, read back as 0, and be overwritten when it came back (chapter 7 (p. 153), section 7.4.5 (p. 164)).

8.13 Tried and rejected

FOR FIRMWARE CHANGES

These failed here, on this machine, with this toolchain. They may work elsewhere.

- Once-a-second dashboard polling, at the same rate while failing. It used up the connection pool; the radio stopped accepting connections while ping stayed perfect.
- Four requests per page load (page, schema, values, state). Replaced by `/api/boot`.
- Settings tabs refreshed every 10 s. Replaced by a refresh when the tab opens.
- A page that drew before it had any state, and a start-up that gave up silently when its first request timed out. The first left the page blank forever; the second showed nothing and never retried.
- A separate console poll. Replaced by the log riding on `/api/state`.
- A 750 ms silent-socket reaper. Once power save came back and round trips passed a second, it killed real requests and cut off the only route in.
- Turning modem sleep off before connecting. Ignored, because connecting re-applies power save.
- An uncompressed page, and a 302 for the favicon (the browser downloaded the page twice).
- A stale second copy of the page in the source, unreferenced. Deleted.
- A stack counter named in words that reported bytes, so it over-read the portal task's headroom by four.
- The first access-point start: it ignored `softAP()`'s return, called it in the same breath as the mode change, and pinned no channel. The access point was claimed and never seen.
- **`WiFi.disconnect(true, false)`** at the head of the access-point start, and **`softAPdisconnect(true)`** on the retry path. Both went through "no mode" and de-initialisation; the access point then broadcast nothing.

- Believing **`softAP()`**'s return and printing "is up". For months, nothing was on the air.
- Raising the forced access point inside the HTTP handler, with an immediate "verified" answer. It raced the network loop and tore down the link the answer needed.
- A forced access point that did not stamp its last try. It lived one loop tick.
- Transmit power as a typed setting, 20 dBm as the top rung, and transmit power as a settings-file line. All removed; an old file's line is ignored.
- Console keys **`,`** and **`.`** for the needle's up-leg top speed. **`.`** had no ceiling. The portal's `upVmax` row does the job inside its bounds (chapter 5 (p. 73)).
- A drop-sample prompt that asked for a frequency. It now asks `Which sample slot to drop (0-11)?`.
- "Network time fresh" meaning "the clock is past 2020".
- Real credentials compiled into the image.
- Multi-character web-console sends.
- A second help list printed at boot, which drifted from the real keys.
- A console read that returned the newest bytes: it dropped the head of any burst over 4 KB.
- The library's **`toFloat()`** for request numbers: it read "107,3" as 107 and accepted "nan".
- The **`sys.sddly`** portal action. It sent the radio to full volume. Do not restore it (chapter 10 (p. 256)).
- The "channel lock" theory of the AP+STA join failure. Checked against the ESP-IDF documentation and wrong: in AP+STA the station's channel wins, and the access point moves to it, announcing a channel switch.
- The "core 0 starvation" theory of "portal dead, ping alive". The cause was the connection pool.

8.14 Known limits

Seen working on the radio: the network name after a return from the access point (11 checks out of 11); a portal reboot during a main-board trial, which brought the previous program back; both boards updated over the air, on trial and then confirmed; the open access point and its DNS server from the S3's side (driver confirmed, `open, no password`, `captive portal: every name now points at this radio.`, probe requests heard), with the S3 home by itself about 10 minutes later. An earlier test, on an older retry path, had a PC join the access point and sign in.

Never exercised on the radio:

- the rescue access point coming back after a failed join on the current retry path;

- a phone joining the open access point and opening the sign-in page by itself;
- the placeholder gate and the whole first-time path of section 8.2.2 (p. 189)
- the settings download refused while locked;
- the lockout beyond its first steps, the "full" answer when adding a fifth account, and session eviction with more than four browsers;
- the refusal of an audio-board setting while the audio board is silent, and the two battery-clock pills in a real fault.

Known limits:

1. The security limits of section 8.6.7 (p. 201): plain HTTP, the open access point, one hash pass, no CSRF token, four tracked addresses, one administrator, `y` echoing the password.
2. Whether a phone opens the sign-in page is the phone's choice.
3. The portal is starved while the needle moves.
4. `y` and `w` hold the main loop, and the audio board mutes while they wait.
5. `w` uses a fixed `EST5EDT` rule and ignores the portal's time zone (section 8.2.1 (p. 187)).
6. A change made in the minute a main-board program is on trial is lost if the power goes in that minute.
7. A blank board's access point runs at 2 dBm (section 8.2.2 (p. 189)).
8. While retrying from the access point, the access point disappears for up to 20 seconds every 2 minutes, if nobody is connected to it.
9. Radio silence still goes through the WiFi driver's de-initialisation (it has never failed).
10. The ESP-IDF framework's own log lines never reach the web console.
11. The WiFi workarounds depend on internals of the pinned Arduino core (the soft-AP shortcut, the mode-change branches, `HTTP_MAX_DATA_WAIT`). The toolchain stays pinned; moving it must re-check all of them.

Small issues left as they are (chapter 12 (p. 290) has each, with a starting point for a fix):

- A main-board upload refused during the trial also prints the updater's "No Error" line on the console.
- A main-board upload refused for the wrong chip gives its reason on the console only; the page shows "flash".
- Saving the Network card always rejoins, even for a time-zone change alone.
- A portal-raised access point is confirmed on the console only.
- `b`'s first press after a boot sets 11 dBm, whatever the current rung.
- `i` calls the lower ladder rungs "LOW", and prints `OWN ACCESS POINT` whenever the S3 is not joined.
- Reboot the main board shows only "rebooting" on the page.

8.15 Changing it safely

FOR FIRMWARE CHANGES

8.15.1 What must stay true

1. The radio must never become unreachable. A failed join ends in `startAp()` a forced access point ends; radio silence ends. Nothing reachable from the portal may be unbounded.
2. Never pass the WiFi driver through **`WIFI_MODE_NULL`** while the access point or the station must survive: no `WiFi.disconnect(true, …)` and no `softAPdisconnect(true)` from a single-interface mode. Only `quiet()` does it, on purpose.
3. Believe the driver, not return values (`apReallyUp()`, `driverReport()`).
4. Re-apply the transmit power after every WiFi mode change (`applyTxPower()`).
5. HTTP handlers never change the WiFi mode or tear down the link they request, and `Net::loop()` acts after its grace.
6. Every API handler starts with **`requireLogin()`**. A new two-callback handler must repeat the session, administrator and placeholder checks in both callbacks.
7. Do not add fields to **`User`** (`name[17]`, `salt[8]`, `hash[32]`, `admin`, `used`): its size is the flash record's length, and every stored account would be discarded.
8. Stay inside the connection budget: no new periodic requests; ride on `/api/state`.
9. Refusals reach the page as refusals (`warn` on a 200, or an error status), never as a plain acknowledgement.
10. Clamping and checks live in the machine, not in the browser (`argNumber()`, `settingBound()`, the clamps in `doAction()`).
11. Web console: one key per send unless **`Con.lineWanted`**. A new line prompt sets `lineWanted` while it reads, clears it, and calls `drainLine()` after.
12. A console loop that blocks must ping the audio board (`MSG_PING`), or the audio mutes after 2 s. It must also feed the loop watchdog (`wdtFeed()`) if it can wait longer than the 15-second timeout. A bounded wait well under 15 s (the `i` scan) need not.
13. Everything the S3 prints goes through **`Con`** only `console.cpp` touches `Serial`.
14. The server task yields every pass (`vTaskDelay(1)`).
15. The reaper stays off while an upload runs (**`gOta`**), and `gOta` is cleared before the settings flush on the S3 upload's success path (the save refuses while an upload is flagged).
16. **`mustchg`** stays out of the schema (the schema is cached by tag).

17. While the S3 image is on trial, nothing is written and a reboot is never refused (`settingsWrite()`, `hReboot()`): the reboot is how a bad update is undone.
18. The page draws nothing before it has state, and its paint code never throws on a missing field. A throw in the first paint stops the whole page.
19. An edit of an audio-board row needs the audio board's answer (`a32CfgKnown()` in `hSet()`); a new path that pushes audio-board settings must check it too.
20. The captive portal's DNS server lives and dies with the access point (`gDns` in `Net::loop()`). It must stop the moment `gApUp` clears and must never run on the house network, where it would answer the house's names with 192.168.4.1. A new path that raises or drops the access point must keep `gApUp` in step, because the DNS server follows it.
21. On the access point the catch-all redirect names the access point's address in full (`http://192.168.4.1/`), not `/`: the phone's check arrives under another host name.

8.15.2 Traps

- Edit `data/portal.html`, never `src/s3/page_gz.h`. The header is regenerated on every `pio run -e s3` it is committed, so a page change shows as a large diff there. Commit both.
- Install `esprima` into PlatformIO's own Python (`pip install esprima` in its virtual environment), or the script check silently degrades to a warning and a broken page can ship. The check covers only the first `<script>` block; keep one.
- The page is written without a single backslash (it uses `String.fromCharCode(10)` for a newline): the editing tools have halved these backslashes before. Keep it so, and check escapes in C strings byte by byte (as `jsonEscLog()` was).
- The ETag changes only when the HTML does, and the page is served `no-cache`, so browsers fetch a changed page by themselves; there is nothing to clear by hand.
- `WebServer::client()` returns a copy; stopping the copy works because it shares the socket.
- `i` blocks for seconds when the S3 is not on the house network (it scans). Expected.
- In AP+STA the station's channel wins and the access point moves to it; do not pin the station to channel 1.
- Silence on the USB console proves nothing about ESP-IDF's own errors (they go to UART0).
- The audio-board relay puts about 2.1 kB of frame buffers on the portal task's stack.

8.15.3 How to test a change

1. Build: `pio run -e s3`. Look for `portal script: N bytes, parses cleanly` and `portal page: X bytes -> Y gzipped …, etag ….` Both environments should build with no new warnings.
2. Flash: identify the S3 by its MAC address before any USB flash, and pass the port explicitly (chapter 3 (p. 25), section 3.1 (p. 25)). Before an over-the-air update, confirm that the address you are sending to is this radio.
3. Wireless updates are for a machine that is working, not for one that is being worked on. Three images were once pushed over the air into a machine that could not be observed; one failed part-way and left a build whose own socket reaper then blocked the fix. Have a portal you trust before you let it overwrite firmware.
4. curl proves the machine; only a browser proves the page. Two portal faults were invisible to curl: a page that threw before its first poll, and speculative sockets.
5. Scripts: sign in once with `POST /api/login`, keep the cookie, and stay at one request every few seconds. Keep the credentials in environment variables, never in the script. Updates go as a multipart upload to `/api/ota/s3` or `/api/ota/a32`.
6. Checks used on the radio:

   Table 95. How to test a change

Check	Expected
`GET /api/schema` with no session	401
A normal account sets `volume`, sends `bt.pause`	200
A normal account sets `taper`, sends `needle.home`, fetches the settings file	403
`posMax=99999`	Clamped to the row's maximum
`showTuning=99`	Clamped to 2
`posMin=nan`	Refused; the stored value is unchanged
The settings file	Scaled values in shown units: `gainRadio=15.000`, not 150
"status" typed in the web console	400

7. The rescue access point — with someone at the radio and the USB cable at hand, because a failed access point strands the radio until a cable is attached:

   1. ☐ Press Raise the rescue access point.
   2. ☐ On the Console tab, watch for `raised and the driver confirms it`, `open, no password`, `captive portal: every name now points at this radio.` and `[ap] probe request heard`.

3. ☐ Join from a phone (force a rescan; no password is asked). Expect the sign-in page to open by itself if it does not, open `http://192.168.4.1/`. Sign in.
4. ☐ Expect the radio home about 10 minutes after the last device leaves.
5. ☐ After it goes home, check that the DNS server stopped: a name looked up on the house network must not answer 192.168.4.1.

Still owed: the failed-join path (store a wrong password, someone at the radio) and the phone opening the page by itself.

8. Transmitter questions: `i` (the driver report and a scan), `B` (step the power), `z` (erase the RF calibration and reboot).
9. After any update, confirm the running version on the System tab.

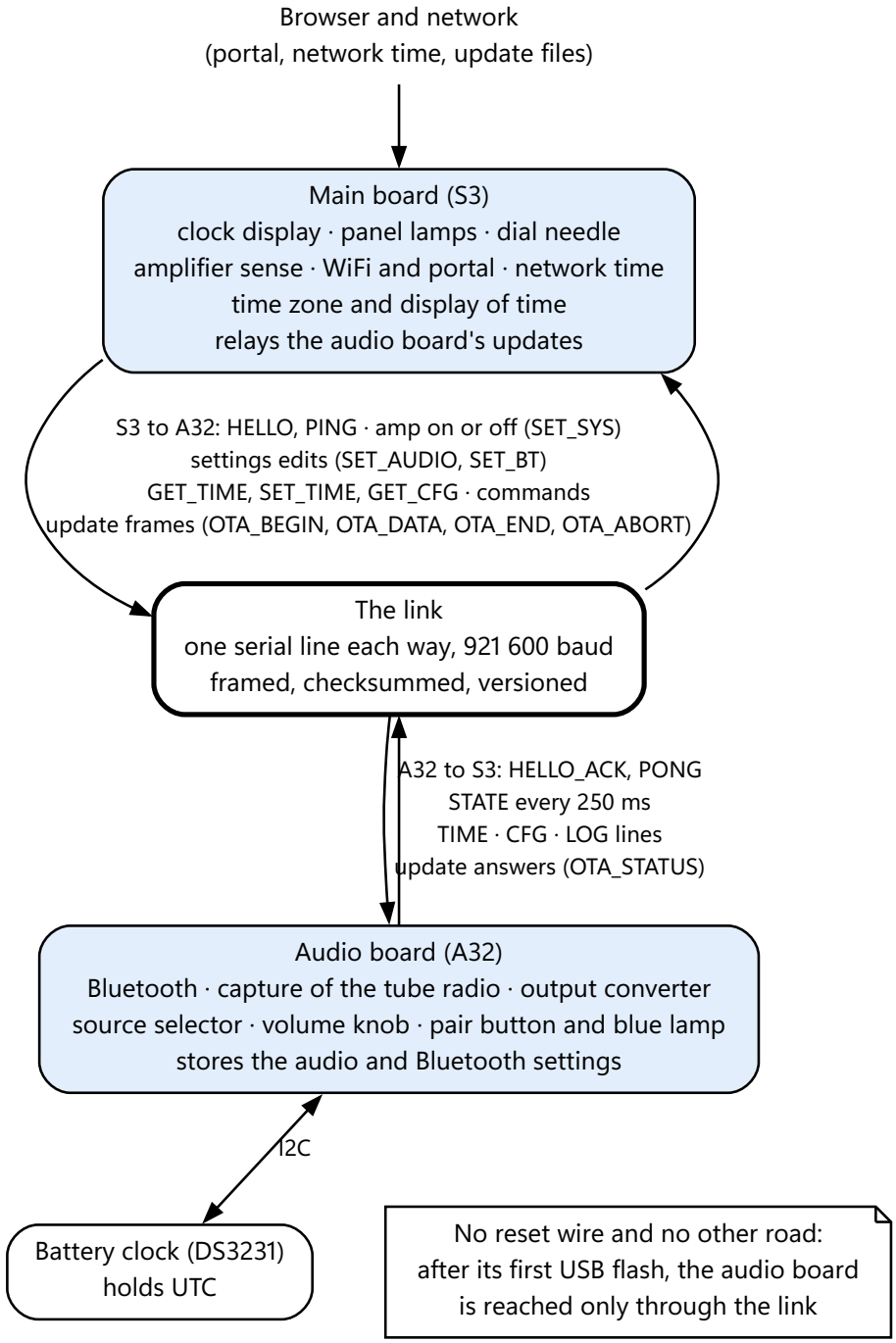
9. The audio board and the link between the boards

Ambersong has two computers. The S3, the main board, runs everything you see; the A32, the audio board, runs everything you hear. They talk over one serial line, the link, and everything that has to cross from one half of the machine to the other goes through it: the time of day, whether the amplifier is on, every audio and Bluetooth setting the portal edits, the A32's live state for the portal, and the A32's own firmware updates. When the link works, the listener sees nothing of it. When the S3 stops talking, the A32 goes silent on purpose within two seconds, so that a dead main board is obvious.

This chapter covers how the A32 starts and runs, the link and its protocol, and how to read the link's health. Updating the A32 and its trial are in chapter 3 (p. 25) time keeping and the battery clock's pills in chapter 4 (p. 46) the A32's settings and the S3's copy of them in chapter 7 (p. 153) the sound itself in chapter 10 (p. 256).

9.1 Two computers and the wire between them

Figure 26. What each board owns, and what crosses the link



The A32 is a classic ESP32, and the S3 an ESP32-S3 (Hardware Bible, chapter 4, section 4.1). The A32 also talks to the real-time clock chip, the DS3231, which this book calls the battery clock.

The table shows what each board owns and how the other board learns it. The `MSG_…` names are the link's messages (section 9.7 (p. 236) Appendix F (p. 338) lists them all).

Table 96. Two computers and the wire between them

Thing	Owner	How the other board learns it
Time of day, in UTC	the battery clock on the A32; network time on the S3 corrects it	`MSG_TIME` (A32 to S3), `MSG_SET_TIME` (S3 to A32)
Time zone, and the display of time	the S3	never sent: everything on the link is UTC
Amplifier on or off	the S3 senses it	`MSG_SET_SYS` (S3 to A32)
Audio and Bluetooth settings	the A32 stores them in its own flash	the S3 keeps a read-only copy, the mirror (`MSG_CFG`), and pushes edits (`MSG_SET_AUDIO`, `MSG_SET_BT`)
Source, volume, Bluetooth state, meters, diagnostics	the A32	`MSG_STATE`, every 250 ms
The A32's firmware image	uploaded to the S3's portal	relayed to the A32 over the link (`MSG_OTA_*`)

The one fact behind most of this chapter. There is no reset wire from the S3 to the A32 (Hardware Bible, chapter 4 (p. 46)), and the A32's firmware has no wireless update of its own. After its first USB flash, the only way to reach the A32 from outside the cabinet is through the link.

What the listener notices:

- A dead S3 is obvious. When the S3 stops talking, the A32 goes silent within 2 seconds: no radio or Bluetooth sound (AUX does not pass through the A32), Bluetooth closed, blue lamp dark (section 9.4 (p. 229)).
- A frozen A32 restarts itself within 15 seconds (section 9.2.3 (p. 228)).
- An A32 update fades the sound out, takes about a minute, and restarts the A32 on the new program, which then runs on trial (section 9.8 (p. 239)).

9.2 How the audio board starts and runs

9.2.1 Start-up, step by step

Each step sits where it does for a reason.

Table 97. Start-up, step by step

Step	What happens	Why here
1. The task watchdog	Set to 15 s, with a restart (panic) on expiry; the main loop's task is put under it.	Start-up runs on that same task, so from this line on any hang, including a Bluetooth start that never returns, ends in a restart. Whether the watchdog took is shown in the boot report (section 9.3 (p. 228)).
2. The program's state	Asks the bootloader's records whether this program is on trial.	The answer becomes the label in the boot report (the labels are in chapter 3 (p. 25), section 3.11.1 (p. 33)). A program flashed by USB is never on trial.
3. The USB console	Opens at 115 200 baud and prints a banner with `FW_VERSION` and `FW_COMMIT`.	This console needs a cable and the cabinet is normally shut, so the lines that matter are also sent to the S3 later.
4. Inputs and settings	Sets up the source selector, the volume knob and the blue lamp; loads the stored settings (chapter 7 (p. 153), section 7.4.1 (p. 161)).	Chapter 10 describes these.
5. The battery clock	Starts its I2C bus and reads it once. The local console prints `DS3231 responding, time valid` or `time INVALID (lost power)`, and the chip's temperature.	Only on the A32′s own console.
6. The protocol self-test	Encodes a status frame, feeds it back byte by byte through a decoder, checks the type, the length and a byte-exact round trip, then flips one bit and requires the checksum to reject the frame.	"A checksum that has never been seen to fail is not known to work."
7. Audio	Starts the audio engine and applies the settings.	Chapter 10.
8. Bluetooth	Configures and starts the Bluetooth audio receiver, applies its transmit power, and starts invisible to phones.	Chapter 10.
9. The link	Registers the message handler, then opens the link with its pins given explicitly.	After Bluetooth: bringing it up earlier was considered and dropped (section 9.11 (p. 245)).
10. Source and mute	Reads the selector once, sets the source, and works out the mute.	This sets the rule that the A32 is asleep, and so muted, until the S3 speaks (section 9.4 (p. 229)).
11. Start-up time	Records how long start-up took, for the boot report.	On this radio it took about 1.1 s.

A failed protocol self-test does not stop the A32. It marks the build as broken (`protoBroken`) and carries on, so the link and the update receiver still come up and a good build can be sent over the air. While the mark is set, the sound stays muted, Bluetooth stays closed, and every 10 s the A32 says `proto self-test FAILED - muted, BT closed, awaiting OTA` on its console and to the S3. Such a build is never confirmed, so one that came over the air is rolled back by the next reset (chapter 3 (p. 25), section 3.11.7). Only a broken build can fail the test.

9.2.2 The main loop

> FOR FIRMWARE CHANGES
>
> The A32′s main loop does, in this order, on every pass:

1. Reads the link.
2. Decides asleep or awake (section 9.4 (p. 229)), before anything that depends on it.
3. The blue lamp.
4. The settings save, two seconds after the last change (chapter 7 (p. 153), section 7.4.2 (p. 161)).
5. Any Bluetooth disconnect still pending.
6. The source selector, every 50 ms.
7. The update give-up: an update with no accepted data for 20 s is abandoned, with the reason `the S3 stopped sending`.
8. The trial check: whether a program on trial has earned confirmation (chapter 3 (p. 25), section 3.11.2 (p. 33)).
9. The `protoBroken` announcement, every 10 s while it applies.
10. The volume knob.
11. The Bluetooth state, and the refresh of its visibility.
12. The pair button.
13. One status message (**`MSG_STATE`**) every 250 ms, sent whether or not the S3 has ever answered.
14. The console lines of the zero-data watch, and the A32′s own USB console keys (chapter 10 (p. 256)).

9.2.3 The watchdog

A task watchdog is a timer that restarts the chip when a watched task stops checking in. The A32′s is set to 15 s, with a restart on expiry, as the very first line of start-up, and it watches the main loop's task. The Arduino core feeds it once per pass of the main loop.

- Start-up must finish in 15 s, because it runs on the watched task.
- 15 s leaves room for the Bluetooth start, and for the longest legitimate wait inside one pass of the loop: the image check at the end of an update.
- The idle check of core 0 moves to 15 s too. The framework already watched core 0′s idle task, at 5 s; the timeout is shared, so setting 15 s changes both.

Without it, a frozen loop or a Bluetooth start that never returned went unnoticed, and the portal's Reboot the audio board, which is a message on the link, cannot be read by a frozen loop.

9.3 Reading the audio board's boot report

The A32′s own console needs a cable. So at every start-up the A32 sends one line, its boot report, to the S3, at the first HELLO (the S3′s greeting, section 9.7 (p. 236)) it receives from it. The S3 prints it on its console (also on the portal's Console tab) with an `[A32]` prefix:

```
[A32] boot: commit <hash>, reset reason 3, setup 1066 ms,
image ON TRIAL (rollback armed), watchdog on
```

Table 98. Reading the audio board's boot report

Field	Meaning
`commit`	The source the running program was built from: the short git hash, with `-dirty` when tracked files had uncommitted changes, or `nogit` (chapter 3 (p. 25), section 3.13 (p. 37)).
`reset reason`	Why the chip last started: 1 power-on, 3 software restart (what an update or Reboot the audio board produces), 4 crash (panic), 6 task watchdog, 9 brown-out. The number is the ESP32 framework's reset-cause code (`esp_reset_reason()`); these five are the ones this book records. The framework has others: the external reset pin, the interrupt watchdog, other watchdogs, deep sleep, unknown. The main board's console `s` prints its own last reset reason by name, from the same list (chapter 4 (p. 46), section 4.10 (p. 59)).
`setup`	How long start-up took, in ms.
`image`	The trial state: `ON TRIAL (rollback armed)`, `valid`, `NEW - this bootloader has no rollback` or `not tracked (USB flash)` it reads `valid (confirmed)` once the program has confirmed itself (chapter 3 (p. 25), section 3.11.1 (p. 33)).
`watchdog`	`on`, or `OFF` if the watchdog did not take.
ending	`- an earlier update was ROLLED BACK`, when the bootloader has marked a program slot invalid.

This line is how a rollback or a watchdog restart becomes visible without the USB cable. A rollback caused by a hanging test program read, in full:

```
[A32] boot: commit <hash>, reset reason 6, ... image valid,
watchdog on - an earlier update was ROLLED BACK
```

Once per A32 start-up. If the S3 restarts later and shakes hands again, the report is not repeated.

The version is not in this line. It follows at the handshake, as `[PASS] A32 up: proto v4, firmware <version>` the two lines together name both the program and its source.

Other lines the A32 sends, all printed by the S3 as `[A32] ...`:

- the confirmation result: `image confirmed (...)`, or `image confirm FAILED (err N) - retrying`
- `settings changed during the trial are now saved` (chapter 3 (p. 25), section 3.11.5 (p. 34));
- `proto self-test FAILED - muted, BT closed, awaiting OTA`, every 10 s (section 9.2.1 (p. 226));
- a failed settings save, printed as `[A32] settings NOT saved - the flash write failed; retrying` (chapter 7 (p. 153), section 7.4.2 (p. 161));
- a Bluetooth disconnect that did not land.

9.4 Awake and asleep

The A32 has one rule, checked on every pass of its main loop:

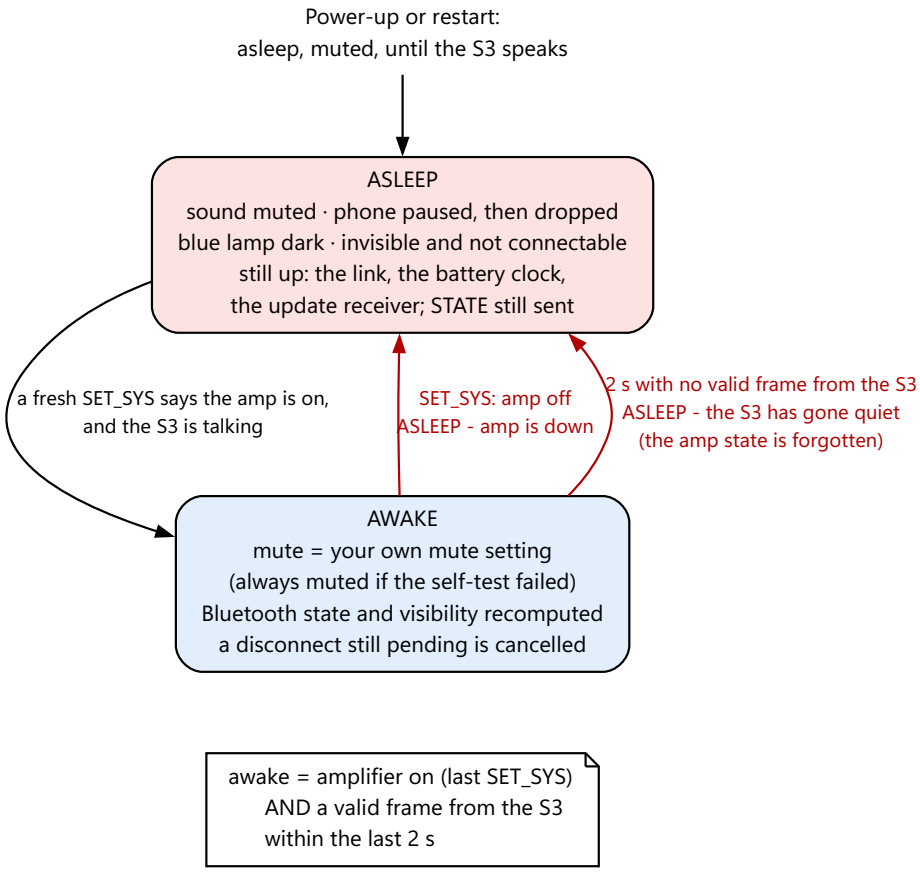
```
awake = amplifier on  AND  a valid frame from the S3 within
the last 2 s
```

- Amplifier on is the last amplifier state the S3 sent (`MSG_SET_SYS`).
- A valid frame from the S3 means HELLO before the handshake, PING and SET_SYS after it (section 9.7 (p. 236)). A frame is one message on the link

(section 9.6 (p. 232)). Frames from the S3 are the only ones that keep the A32 awake.

The A32 acts only on a change between the two states, not on every pass: running the teardown on every pass would send a disconnect for ever.

Figure 27. The audio board's two states, and what moves it between them



Falling asleep (the amplifier switched off, or 2 s without a valid frame from the S3):

1. The A32's console prints `ASLEEP - amp is down` or `ASLEEP - the S3 has gone quiet`.
2. If the S3 is the reason, the A32 forgets the amplifier state. It must not wake on the S3's last "amp on", which may be stale: the amplifier may have been switched off while the S3 was restarting. It stays asleep until a fresh SET_SYS says the amplifier is on.
3. The phone is paused and then disconnected, the sound muted, the blue lamp darkened, and the radio made invisible and not connectable to phones.

Waking (amplifier on and the S3 talking): any disconnect still pending from the last 250 ms is cancelled, the mute is worked out again, and the Bluetooth state and visibility are recomputed.

The mute is a state, not an event. Every place that could change it works it out from one rule:

```
muted = (self-test failed)  OR  (asleep)  OR  (your own mute
setting)
```

An earlier version muted when the A32 fell asleep while four other places unmuted. Turning the source knob while the S3 was dead brought the sound back, and the alarm vanished.

Asleep is not off. The link, the battery clock and the update receiver stay up: they are what the S3 needs from the A32, and the only reason for it to be awake at all. The A32 keeps sending its status. The audio clocks keep running; asleep is a mute, not a stop.

What the listener sees

Table 99. Awake and asleep

Event	What happens
Power-up	A brief silence until the S3 has shaken hands and sent its first “amp on”. Accepted.
The S3 restarts (its own update, a portal reboot, a crash)	Within 2 s: the sound fades out, the phone is dropped, the blue lamp goes dark. When the S3 is back and says the amplifier is on, the A32 wakes. The phone must reconnect.
The link cable fails in one direction	If the S3-to-A32 direction is cut: the same as an S3 restart, for as long as the fault lasts. Either way, the board whose receiving direction is cut reports the link as silent on its console (`link SILENT rx 0` on the S3).

Why. An unreachable S3 is made known: the radio then does not work at all, with no sound, no clock, no lights. (AUX does not pass through the A32, so a source on AUX still plays.) The costs are accepted and written down so that nobody “fixes” them: an S3 restart or a long S3 update silences the radio mid-listen, and every power-up has a short dead moment. The rule earned its place on its first real fault, the day after it was written: one direction of the link failed at a connector, the A32 never learned that the amplifier was on, so it slept, and the whole front of the radio said something was wrong. The firmware before the rule would have played on.

The S3′s side of the bargain. Anything that stops the S3′s main loop must keep the A32 fed. The console prompts read the link while they wait; the live index monitor (console `l`) also sends a PING on each pass. The WiFi prompt `y` can still hold the S3′s loop until 60 s after the last key typed, which puts the A32 to sleep meanwhile; that is left as it is, judged harmless in real use. Chapter 4, section 4.11 (p. 60) lists the prompts that hold the loop.

9.5 The link, and how to read its health

The line. A UART on `Serial2` on both boards, at 921 600 baud, 8 data bits, no parity, one stop bit, no flow control. The firmware uses S3 GPIO13 to send and GPIO14 to receive, and A32 GPIO27 to send and GPIO26 to receive (Appendix I (p. 358)); the cable is in the Hardware Bible, chapter 10, section

10.12. The baud rate is the firmware's own choice, and the cable carries it: every frame is checksummed, and the counters show a clean link (`crc 0`). Each board receives into a 4096-byte buffer.

Where to read it. The S3′s console `s` prints a `link` line; the portal's status carries the same counts.

Table 100. The link, and how to read its health

Status field	Meaning
`linkrx`	Valid frames received from the A32.
`linktx`	Frames sent to the A32.
`linkcrc`	Frames received with a bad checksum.
`linkver`	Frames dropped because they carry another protocol version.
`hello`	The handshake is done.
`astate`	A valid status report from the A32 is in hand.

A healthy pair reads, for example:

```
  link       : alive  rx 489  crc 0  wrong-version 0
```

During a protocol mismatch it reads (the count is an example):

```
  link       : SILENT  rx 0  crc 0  wrong-version 412  <- the
boards run different protocol versions: flash both
```

How to read it

Table 101. The link, and how to read its health

The link line shows	It points at
`alive`, `rx` climbing, `crc 0`, `wrong-version 0`	A healthy link.
`SILENT`, `rx 0`, `wrong-version 0`	The wiring or the power of the link (Hardware Bible, chapter 10).
`rx` climbing and `crc` climbing with it	The quality of the signal.
`wrong-version` climbing	The other board is talking, but was built with another protocol version. The portal shows the pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both while there is no handshake. What to do: section 9.6.3 (p. 236).
`alive`, and `wrong-version` above 0 but not climbing	A mismatch that has been fixed. The count runs from the S3′s start-up, so it stays above zero until the next S3 restart; the pill goes away at the handshake.

Only the S3 shows a mismatch. The A32 counts dropped frames too, but prints the count nowhere; its own USB console `s` shows only `link SILENT rx 0`. The A32′s `s` shows the link, the frames received, and whether it is awake and why.

9.6 Frames, the decoder and versions

This section describes the link's wire format. You need it only to change the protocol or to decode the link with an analyser; section 9.6.3 (p. 236), on the two version numbers, is for everyone.

9.6.1 The frame

FOR FIRMWARE CHANGES

The link is a binary protocol: every message travels as one frame, with a start marker, a version byte, a type, a length, the payload and a checksum.

Figure 28. One frame on the link, byte by byte

Byte	Content	Note	Checksum
0	0xA5	start marker, first byte	not covered
1	0x5A	start marker, second byte	
2	ver	protocol version, 4; any other value: frame dropped	covered by the CRC16
3	type	message id (Appendix F)	
4	len, low byte	payload length, little-endian, at most 1088	
5	len, high byte		
6 …	payload	len bytes (none for an empty message)	
6 + len	CRC, low byte	CRC-16/CCITT-FALSE: poly 0x1021, start 0xFFFF, no final XOR	the checksum itself
7 + len	CRC, high byte		

- The start marker, `0xA5 0x5A`, is not covered by the checksum: it is there to find the start of a frame again after damage, not data.
- The version byte is the protocol version, currently 4. A frame with any other value is dropped by the decoder and counted as wrong-version.
- The type is the message id (Appendix F (p. 338) lists them all).
- The length is the payload's, little-endian, at most 1088 bytes: a 1024-byte update chunk plus room for its header.
- The checksum is a 16-bit CRC with polynomial 0x1021, initial value 0xFFFF and no final XOR, computed bit by bit over the version, type, length and payload, and sent little-endian. This is the variant usually called CRC-16/CCITT-FALSE: the ASCII string `123456789` gives 0x29B1. It is computed bit by bit, not from a table, because "a 512-byte table would buy nothing and cost cache the display ISR wants."
- Payloads are packed structures, copied byte for byte. Both ends are little-endian processors of the same family built by the same compiler, so this shortcut works. If a third kind of device ever joins the link, the encoding must become byte by byte.

- A frame that would not fit is not sent at all. The encoder returns a length of 0 and writes nothing, so “a caller that ignores the return value still cannot emit a torn frame”.

Worked examples (bytes in hexadecimal, as sent):

```
PING, empty payload:         A5 5A 04 03 00 00 61 17
PONG, empty payload:         A5 5A 04 04 00 00 F1 92
TIME, 8-byte payload:        A5 5A 04 21 08 00 | 80 3B B1
6A  01  64  00 00 | 17 8E
                                                 unixUtc
valid tempC4 reserved

1790000000    1     100 = 25.00 °C

(2026-09-21 14:13:20 UTC)
```

Frame sizes. Each frame is 8 bytes of overhead plus its payload:

Table 102. The frame

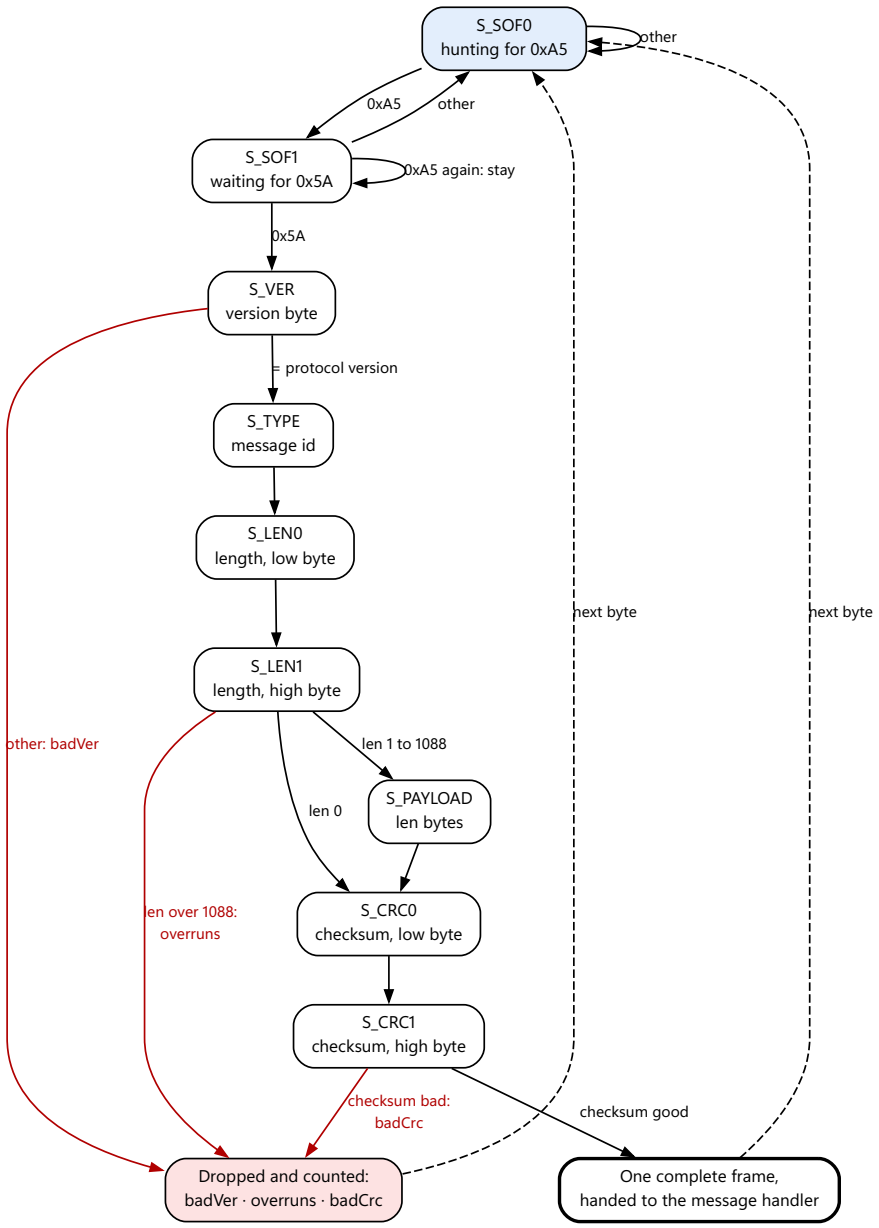
Message	Frame, bytes
HELLO, HELLO_ACK	33
STATE	138
CFG	41
TIME, SET_TIME	16
SET_AUDIO	23
SET_BT	20
SET_SYS	12
OTA_STATUS	46
OTA_DATA	up to 1038
empty messages	8

At 921 600 baud the line carries about 92 kB/s. A full update frame takes about 11.3 ms, and the A32′s status reports cost about 550 bytes per second.

9.6.2 The decoder

The decoder takes one byte at a time, so it can be fed straight from the serial port with no assumption that frames arrive whole.

Figure 29. The frame decoder: one state per field, back to the hunt after every frame



- A second **`0xA5`** while waiting for **`0x5A`** keeps the decoder waiting for `0x5A`: it may be the real start of a frame whose predecessor was cut short.
- After the last checksum byte the decoder always hunts again from the next byte. It does not go back and rescan the bytes inside a frame that failed. A damaged length byte can therefore hide what follows it for a while (section 9.10 (p. 242)).
- A payload is copied into its structure only if its length is exactly that structure's size. The frame has already passed its checksum, so a length that does not match means the two builds disagree about the

structure: a version skew, not noise. A partial copy would be worse than none.
- One-byte commands are checked by length (two bytes for `MSG_CLK_PAIR`), and update data frames are read at fixed offsets (Appendix F (p. 338)).

9.6.3 Version numbers on the link

- The protocol version (`PROTO_VERSION`, 4) is the wire format. It travels as the version byte of every frame, and a frame with another value never gets past the decoder. Version 2 is when the audio settings gained their fades.
- The firmware version (`FW_VERSION`, `v.1.<stamp>`) names the build. It travels in HELLO and HELLO_ACK, so each console can say which build the other board runs. A break in the wire format is the protocol version's job, not the firmware version's.
- The commit stamp (`FW_COMMIT`) is shown next to the version but never sent in the handshake: the handshake's version field is 24 bytes, and widening it would itself be a protocol change.

Chapter 3, section 3.13 (p. 37) has all four version numbers and where to read each.

When the two boards run different protocol versions. Neither board can then pass a single frame to the other. There is no handshake and the A32 sleeps, so neither the radio nor Bluetooth plays; AUX does not pass through the A32 and still does. The main board shows it with the pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both and with `wrong-version` climbing in console `s` (section 9.5 (p. 231)). The pill says "flash both", but only one order works:

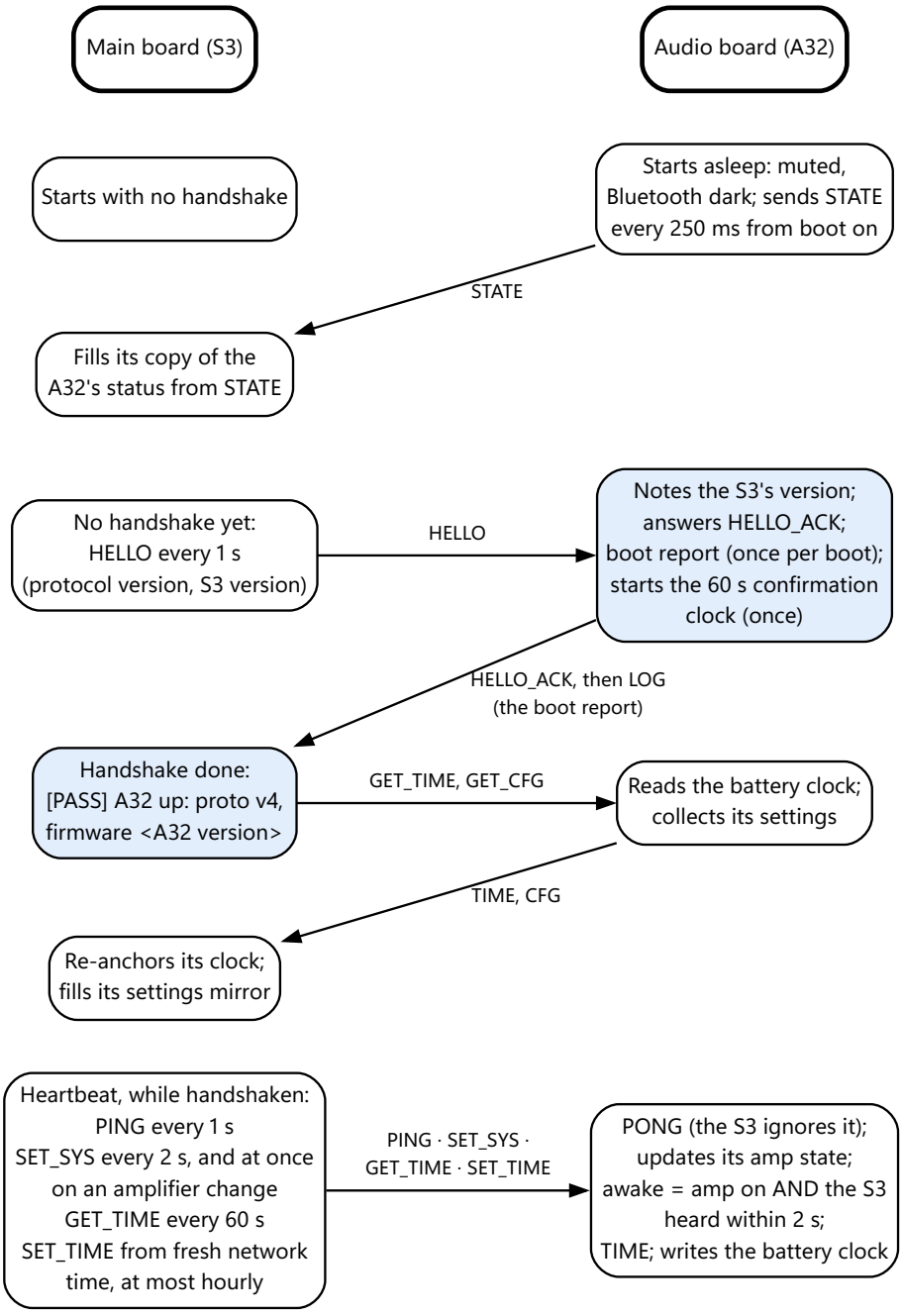
1. ☐ Update the S3 first, over its own WiFi (chapter 3 (p. 25), section 3.10.2 (p. 29)), with a build of the A32's protocol version. Its own update route does not use the link, so it still works.
2. ☐ Once the two versions match, update the A32 through the S3 again.

Updating the A32 first cannot work while the versions differ: the relay through the S3 needs the link, and it refuses with `the A32 is not answering`. If the A32's new program is still on trial, unplugging the set also works: the reset rolls the A32 back to its previous program. No USB cable is needed either way.

9.7 Handshake and heartbeat

The S3 drives the conversation; the A32 answers.

Figure 30. The handshake, then the heartbeat



1. From its start-up on, the A32 sends its status (STATE) every 250 ms, unprompted. The S3 fills its copy of the A32's status from it.
2. Until it gets an answer, the S3 sends HELLO every second, carrying its protocol version and firmware version.
3. The A32 answers HELLO_ACK with its own versions. At the first HELLO of its start-up it also sends its boot report (section 9.3 (p. 228)) and starts the one-minute clock of its trial (chapter 3 (p. 25), section 3.11.2 (p. 33)).
4. On HELLO_ACK the S3 prints `[PASS] A32 up: proto v4, firmware <A32 version>`, and asks for the time (GET_TIME) and the settings (GET_CFG).

The answers, TIME and CFG, set its clock (chapter 4 (p. 46), section 4.9 (p. 57)) and fill its settings mirror (chapter 7 (p. 153), section 7.4.4 (p. 163)).

5. The heartbeat, for as long as the handshake stands:
   - PING every second; the A32 answers PONG, which the S3 ignores;
   - SET_SYS every 2 seconds, and at once when the debounced amplifier state changes;
   - GET_TIME every 60 seconds;
   - SET_TIME when network time is fresh, at most once an hour.

Which frames keep which side alive. The S3 counts the A32 as alive while frames arrive from it, and the A32 sends STATE four times a second, so PONG is not needed. The A32 counts the S3 as alive only on frames from the S3.

When the S3 starts over. It drops the handshake, forgets the A32's status, and goes back to HELLO every second when:

Table 103. Handshake and heartbeat

Cause	The S3's console
Nothing valid has arrived from the A32 for 2 s. The S3 also forgets its copy of the A32's settings (chapter 7 (p. 153), section 7.4.5 (p. 164)).	`[WARN] A32 silent. Clock and needle keep running.`
The A32's millisecond count went backwards in a STATE frame: the A32 restarted faster than the 2-second rule could notice. Values near the count's 49.7-day wrap are ignored, and the count must fall back by more than 5 s, which tolerates reordering.	`[WARN] the A32 restarted - handshaking again.`
An A32 update has ended, successful or not confirmed, so that the version shown is the one the A32 comes back with.	the update's own messages (chapter 3 (p. 25), section 3.10.3 (p. 30))

What the handshake checks. The S3 refuses a HELLO_ACK of the wrong size, with `[FAIL] HELLO_ACK size - version skew`. It also compares the protocol numbers, but a frame with another protocol version never reaches that check; a protocol mismatch shows instead in the wrong-version count (section 9.5 (p. 231)).

FOR FIRMWARE CHANGES

What the S3 does with each message from the A32

Table 104. Handshake and heartbeat

Message	What the S3 does
HELLO_ACK	Checks its size, notes the A32's versions, marks the handshake done, prints, asks for the time and the settings.
STATE	Copies it into its picture of the A32 and runs the restart test. Copies the volume and mute into the settings mirror, because the knob moves the volume and the A32 does not resend its settings when it does.
TIME	Re-sets the S3's clock if the time is valid (chapter 4 (p. 46), section 4.9 (p. 57)).
CFG	Fills the settings mirror; the A32's settings are now known.
OTA_STATUS	Fills the answer the update relay is waiting for (section 9.8 (p. 239)).
LOG	Prints it as `[A32] <text>`.

A status report that does not fit this build. If a STATE frame passes its checksum but has the wrong length, the S3 drops its picture of the A32′s status and prints, once per start-up:

```
[WARN] the A32's STATE frame does not match this build. Flash
BOTH MCUs. Audio is unaffected - only telemetry stops.
```

The portal then shows the pill AUDIO BOARD SENDS NO STATE: the two boards are talking, but the audio board's status reports do not fit this build. Flash both boards. If the audio board stops talking altogether, the `link` pill turns red instead. Showing the last values instead would report a measurement that was not taken.

9.8 Updating the audio board, and its trial

The whole procedure, its messages on the page, its figure and every refusal are in chapter 3 (p. 25), section 3.10.3; the trial, confirmation and rollback of both boards in chapter 3 (p. 25), section 3.11 (p. 32). This section adds only what happens on the link.

FOR FIRMWARE CHANGES

The messages. The S3 relays the uploaded file over the link, 1 KB per frame, storing nothing:

Table 105. Updating the audio board, and its trial

Message	Direction	Carries
`MSG_OTA_BEGIN`	S3 to A32	size and checksum, both 0, and the S3′s own firmware version
`MSG_OTA_DATA`	S3 to A32	the absolute offset, the length, and up to 1024 bytes
`MSG_OTA_END`	S3 to A32	the CRC32 of the whole image
`MSG_OTA_ABORT`	S3 to A32	nothing; sent for a wrong chip, an unanswered END, an aborted upload, or 15 s of upload silence
`MSG_OTA_STATUS`	A32 to S3	the bytes committed so far, the state (1 receiving, 3 ok, 4 failed), the updater's error number, and a short text

- Size and checksum are 0 at the start because the S3 is passing on an upload it has not finished receiving. The whole-image CRC32 travels in END; both sides compute it over exactly the bytes committed. An END with a checksum of 0 skips the comparison.
- Offsets are absolute, and the answer is a byte count. A frame that arrives twice is ignored, one that arrives early is refused, and every answer carries the number of bytes actually committed. The answer carries no frame identity, so the S3 reasons only in byte counts; a late answer from an earlier try reads as "send again", which is harmless.
- The S3 starts over the handshake after every A32 update, so the next HELLO_ACK shows the version the A32 came back with.

- Each frame waits for its answer, and each answer for a flash write, so the line's speed is not the limit: the relay runs at about 16 KB/s.

Why the A32 waits 400 ms before its first flash write. Writing and erasing flash stalls all code that is not in internal RAM, and the audio engine is not; it would tear rather than pause. So the A32 mutes through its normal fade-out first, and the update sounds like a change of source. The mute is seen at the next audio block (at most 5.8 ms); the fade-out takes `fadeOutMs` (150 ms by default); then the output's 8 DMA buffers drain (46.4 ms). That is about 200 ms in all, and 400 ms covers a fade-out of up to about 340 ms. When the wait runs short, the flash write freezes the audio task mid-drain and the output hardware sends exact zeros: a click through the amplifier at the start of the update.

Note The wait is fixed, and `fadeOutMs` is not. The portal accepts a fade-out of up to 5000 ms, and any fade-out above about 340 ms makes every audio-board update start with a click. Set it lower.

FOR FIRMWARE CHANGES

The trial, as it touches the link. The details are in chapter 3 (p. 25), section 3.11 (p. 32) these points matter to anyone changing the link:

- The one-minute clock starts at the first HELLO the new program receives.
- The five-minute fallback confirms a program only if nothing has ever arrived: no valid frame and no damaged one. Frames of another protocol version are counted only as wrong-version, which the fallback does not look at. The procedure for changing the protocol relies on this (section 9.14.5).
- A program whose protocol self-test failed is never confirmed.
- While a program is on trial and not yet confirmed, `MSG_OTA_BEGIN` is refused, before anything is muted. The Arduino updater writes the slot directly and does not go through the framework's own "refuse while pending" check, so this refusal in the A32's message handler is the only guard.

9.9 The battery clock on the audio board

How the S3 keeps time, the three sources of time, the time zone and the battery clock's two portal pills are in chapter 4 (p. 46), sections 4.9 (p. 57) and 4.10. This section covers the A32's side.

The battery clock is a DS3231 module on the A32's own I2C bus (the module, its battery and its wiring: Hardware Bible, chapter 4, section 4.7). The firmware addresses it at 0x68, on GPIO22 (data) and GPIO21 (clock), at 100 kHz, a speed the firmware chose. Starting the bus enables the ESP32's internal pull-ups on those two pins.

Everything in the chip and on the link is UTC. The S3 applies the time zone, because it has network time and the portal.

> FOR FIRMWARE CHANGES
> Reading the clock:
>
> 1. The seven time registers from 0x00 (BCD, 24-hour, the year counted from 2000).
> 2. The status register 0x0F. Its bit 7 is OSF, the oscillator-stopped flag, which the chip sets when its oscillator has stopped, for example after losing both main power and its battery.
> 3. The temperature registers 0x11 and 0x12, in quarter degrees.
> 4. Conversion to a Unix time, with the time zone pinned to UTC around the conversion (the conversion function works in local time).
>
> The read counts as failed only when the time registers themselves give no answer.

The time is valid only when all three hold:

```
valid = (time later than 1600000000, September 2020)  AND
(the status register answered)  AND  (OSF clear)
```

A status register that did not answer counts as not valid. That is the safe direction: the S3 keeps its own clock and only declines to re-set it from this one reading. The first spare byte of the time message carries the OSF bit itself.

Why OSF matters. A DS3231 whose oscillator stopped keeps counting from wherever it was, so after a power loss it reports a date that is plausible and wrong. Plausibility cannot catch that; OSF can. OSF stays set until the time is written, and that is the point: “a clock that stopped and was never set since IS wrong.”

Writing the clock: the time in 24-hour form with the weekday, then the status register is read, changed and written back with OSF cleared. That clear is what makes the time trustworthy again. The first SET_TIME, from network time or typed by hand, clears it, and from then on the chip reports valid again.

The A32′s two handlers.

- GET_TIME: reads the chip and answers TIME. If the chip does not answer, the A32 prints a warning on its own console and sends nothing. Silence is therefore the S3′s only sign of a clock that does not answer, and it shows the pill BATTERY CLOCK NOT ANSWERING (chapter 4 (p. 46), section 4.10 (p. 59)). The GET_TIME sent at the handshake is not timed, so that pill can take up to about a minute after a handshake to appear.
- SET_TIME: writes the time to the chip. The incoming `valid` and `tempC4` fields are ignored.

The S3 uses neither the temperature nor the OSF copy.

9.10 When it goes wrong

The A32 is powered whenever the set is plugged in: its 5 V supply is taken before the front switch (Hardware Bible, chapter 5), so the front switch does not restart it. A “power cycle” of the A32 means unplugging the set.

The last column is the fault’s class on the recovery ladder (NOTE, DEFECT, BLOCKER), set out in About this book.

Table 106. When it goes wrong

What you see	What happened	What the firmware does	What you do	Class
A restart; the boot report shows reset reason 6	The A32’s main loop or start-up froze, on a confirmed program.	The watchdog restarts it after 15 s.	Nothing, if it does not recur. A hang at every start-up, before the link comes up, is a boot loop only USB can fix.	NOTE; BLOCKER if it recurs at every start-up
`– an earlier update was ROLLED BACK`	A new program crashed or hung at start-up, while on trial.	The bootloader starts the previous program.	Nothing needed.	NOTE
After an A32 update the radio stays silent, and no handshake completes	The new program boots and hears frames, but never completes the handshake.	Stays on trial; the A32 is asleep. Nothing on the link can restart it.	Unplug the set: the reset rolls it back.	BLOCKER
`proto self-test FAILED – muted, BT closed, awaiting OTA` every 10 s	A new program failed its protocol self-test.	Muted, Bluetooth closed; the link and the update receiver stay up. Never confirmed, so an upload is refused while it runs.	Sent over the air: Reboot the audio board from the portal, or unplug the set; the reset rolls it back. Flashed by USB (not on trial): upload a good program through the portal.	DEFECT if the link still carries the reboot; BLOCKER (unplug) if not; never exercised on the radio
`that image is not for the audio board (wrong chip) – nothing was written`	The file was built for another chip, such as the S3.	ABORT; the A32 restores your mute setting.	Upload the right file.	NOTE
An update fails with the updater’s error 8 (magic byte)	The file is not an ESP32 program at all.	The A32 gives up and restores your mute setting.	Upload the right file.	NOTE

What you see	What happened	What the firmware does	What you do	Class
`crc mismatch` or `end failed`	Damage in transit.	A frame with a bad checksum is sent again; damage found at the end gives up the update.	Upload again.	NOTE
`the A32 did not confirm the update - check its version once it reconnects`	The answer to END was lost.	ABORT, a new handshake.	Read the version at the next handshake (`[PASS] A32 up: ...`).	NOTE
`A32: still on trial - retry in 1 min (err 0)`	A second update inside the trial minute.	Refused before any mute.	Wait a minute.	NOTE
The upload stops partway; the sound stays off until the A32 gives up	The browser, the WiFi or the S3 stopped mid-stream.	The A32 gives up after 20 s with no data and restores your mute setting; the S3 aborts after 15 s with no upload activity. The old program stays.	Upload again.	NOTE
`image confirm FAILED (err N) - retrying`	The confirmation write failed.	Retries every 10 s; updates refused meanwhile.	Nothing, if a retry lands.	NOTE
A good new program is gone after a power cut or a reboot	A reset during its trial minute.	Rolled back.	Upload it again.	NOTE
Nothing; `crc` counts one more	One damaged frame.	Dropped and counted.	Nothing.	NOTE
The A32 falls asleep for up to about 80 s, then wakes by itself	A damaged length byte (a value up to 1088): the decoder swallows up to 1090 following bytes before it hunts again. From the S3, about 14 bytes per second arrive after the handshake.	Asleep, then awake again.	Nothing.	NOTE
Within 2 s the sound fades out, the phone is dropped, the blue lamp goes dark	The S3 went silent: its restart, its own update, a crash.	Asleep; the amplifier state forgotten; STATE still sent.	Nothing: it wakes when the S3 is back and says the amplifier is on.	NOTE (deliberate)
`[WARN] A32 silent. Clock and needle keep running.` or `[WARN] the A32 restarted - handshaking again.`	The A32 went silent, or restarted quickly.	The S3 drops the handshake and its picture of the A32′s status (and its settings mirror after 2 s of silence); the portal shows the A32′s fields as unknown; the S3′s clock keeps running from its last setting; HELLO every second.	Nothing.	NOTE
`link SILENT` on one or both boards	The link cable is faulty, in one or both directions.	As above, on the side that hears nothing.	Hardware: Hardware Bible, chapter 10, section 10.12.	—
`[WARN] A32 silent` and the `link` pill red, yet the sound plays on; the needle parks	Only the audio-board-to-main-board direction of the link fails: a wiring fault.	After 2 s without a frame the main board drops the handshake, the audio board's state and its copy of the audio-board settings. The source counts as AUX (the needle parks); the clock runs on the main board's own count; audio-board edits are refused;	Nothing in the firmware: everything recovers by itself when the line works again. The fault is in the wiring: Hardware Bible,	NOTE; a wiring fault; never seen on

What you see	What happened	What the firmware does	What you do	Class
		the main board sends only HELLO each second (SET_SYS, PING and GET_TIME stop). The audio board still receives those HELLOs, so it stays awake on the last amplifier state it was told: the sound plays on, and it never learns the amplifier went off.	chapter 10, section 10.12.	the radio
`the A32's STATE frame does not match this build. Flash BOTH MCUs.` pill AUDIO BOARD SENDS NO STATE	The status layout differs between the two builds.	The S3 shows the A32′s status as unknown; warns once. The sound is not affected.	Flash both boards.	NOTE
No radio or Bluetooth sound; `rx 0` with `wrong-version` climbing; pill BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both	The two programs have different protocol versions.	Every frame dropped at the decoder, on both sides: no handshake, the A32 asleep, the relay refuses (`the A32 is not answering`).	Update the S3 over its own WiFi to the A32′s protocol version (section 9.6.3 (p. 236)). If the A32 is still on trial, unplugging the set also works: it rolls back.	BLOCKER (survives a power cycle once the A32 is confirmed)
`[WARN] the DS3231 has no valid time yet.` at each reading; pill BATTERY CLOCK LOST ITS TIME - check its battery	The battery clock's oscillator stopped (OSF set).	The S3 does not re-set its clock from it.	Nothing with a network: the next fresh network time fixes it. Otherwise set the time with the console `W`. If it comes back after every power cut, check the module's battery.	NOTE with a network; DEFECT without
Pill BATTERY CLOCK NOT ANSWERING	The battery clock does not answer; a time request went unanswered for 5 s while the A32 is alive.	A warning on the A32′s console. The S3 still takes network time.	Hardware: Hardware Bible, chapter 4, section 4.7.	—
HTTP 409 `the audio board is not answering - its settings cannot be changed now` `n/a` in a settings download	The S3 has no copy of the A32′s settings: never read, or the A32 went silent.	Refuses the edit.	Wait for the handshake.	NOTE; never exercised on the radio
`[A32] settings NOT saved - the flash write failed; retrying`	An A32 settings write fell short.	Stays unsaved; retried 2 s later.	Nothing, if a retry lands. A change is lost only if the A32 restarts before one does.	NOTE; never exercised on the radio
Settings changed during the A32′s trial are gone	The trial ended in a reset: a rollback, Reboot	Nothing was written during the trial; the previous program finds its	Set them again once a program is confirmed.	NOTE (by design)

What you see	What happened	What the firmware does	What you do	Class
	the audio board, a power cut.	settings as it left them, in the layout it knows.		
Nothing	An A32 update that changed the layout of its stored settings was rolled back.	Nothing was written during its trial, so the previous program loads its own settings normally.	Nothing.	NOTE

9.11 Design choices

- A framed, checksummed, versioned binary protocol, not lines of text. The link carries firmware, so a damaged byte must be caught, not installed; both boards print to their own consoles, and a framed link cannot be confused by a stray log line; and the S3 updates the A32, so the two will sometimes run mismatched builds.
- No way to open the link without naming its pins, and the receive buffer sized before the port opens. The serial port's default pins on the classic ESP32 include the A32's audio frame clock, and the buffer size is ignored once the port is open.
- Packed structures, copied byte for byte. Both ends are the same kind of processor and compiler; the shortcut is recorded, with the condition that ends it.
- The status report grows by appending fields, without a new protocol version. An older peer's report then has the wrong length and is refused, so a half-updated machine loses only its telemetry and keeps its sound. A version change would drop every frame, which costs the handshake, which puts the A32 to sleep, and the A32's update travels over this same link. "The graceful failure is the correct one here, not the loud one."
- A new message id needs no version change; a retired id is never reused. An older peer ignores an id it does not know, and an old S3 that still sends a retired id is ignored, "the safe direction". New diagnostics got their own block, 0x4n, rather than a number that would have worked only by luck of direction.
- The A32 sleeps whenever the amplifier is off or the S3 is silent. An unreachable S3 must be known; the costs are accepted (section 9.4 (p. 229)).
- When the A32 falls asleep because the S3 went quiet, it forgets the amplifier state. The S3's last "amp on" may be stale; waking on it unmuted into an amplifier that might be off.
- A failed protocol self-test degrades the A32 instead of halting it. A halt before the link came up made an updated program that failed it unreachable without opening the cabinet. "A halt protects nothing that a mute does not." The cheapest failure the test guarded, a payload too big for a frame, became a compile-time error.
- Rollback for A32 updates, and a task watchdog on its main loop. The bootloader supported rollback, but the core marked every program valid

before start-up, so a program that crashed or hung at boot needed USB. The watchdog turns a hang into the reset that rollback needs, and restarts a frozen loop that nothing on the link can reach.

- The watchdog is 15 s, with a restart, armed as the first line of start-up. It must cover a Bluetooth start that never returns, and leave room for the image check at the end of an update.
- A trial program is confirmed 60 s after the first HELLO, with a five-minute fallback only when nothing at all was received, a retry every 10 s after a failed confirmation, no update while on trial, and a boot report that names the watchdog state and any earlier rollback. A handshake a second after start-up proves nothing about waking, Bluetooth or playback; a broken link that still produces frames must stay on trial; and writing over the fallback program during a trial would leave nothing to fall back to.
- The link comes up after Bluetooth, as before. The link is serviced only in the main loop, so opening it earlier in start-up would not reach a hang in the Bluetooth start; rollback and the watchdog cover that need.
- The update relay: 1 KB frames, each answered with the committed byte count, absolute offsets, the image streamed with its CRC32 in END, the sound muted and given 400 ms before the first write, and the S3′s needle not stopped. Section 9.8 and chapter 3 (p. 25), section 3.10.3 (p. 30) give the reasons.
- END must be answered; the S3 starts the handshake over after every A32 update and whenever the A32′s millisecond count goes backwards; an unanswered END aborts and says to check the version; the wrong-chip reason reaches the page. A lost END was once reported as success while the A32 kept its old firmware, and the S3 went on showing the old version.
- The END wait reads the clock once per pass. Two readings could straddle the 4 s limit, and the subtraction would wrap to about 49 days.
- A relay that fails partway sends no ABORT. Not considered worth changing: the A32′s own 20 s give-up ends it.
- The decoder does not rescan the inside of a failed frame. The consequence (section 9.10 (p. 242)) was judged acceptable, and has never been seen.
- The battery clock is valid only while its oscillator-stopped flag is clear, and writing the time clears it. After a power loss the chip reports a plausible, wrong date. A set flag is shown on the portal; most probably the battery clock's battery, or the module itself, is at fault.
- UTC everywhere below the S3; the S3 owns the time zone; network time corrects the battery clock, and the battery clock holds the time when there is no network. Local time in the chip would make the hour after a daylight-saving change ambiguous and the hour before it happen twice.
- The A32 owns and saves its settings; only a write that reached flash counts; the S3 keeps a mirror and refuses to push while it has none. Chapter 7, section 7.4 (p. 161) gives the reasons.

- Reboot the audio board saves pending settings first, skipping the 2-second wait. A reboot within two seconds of a change used to restart before the write, and the change came back as the old value.
- The A32 writes no settings while its program is on trial. Chapter 3, section 3.11.5 (p. 34).
- The S3′s WiFi prompt may hold its loop, and so put the A32 to sleep, until 60 s after the last key typed. Judged to cause no meaningful harm in real use.
- The S3 gets the same rollback and watchdog treatment (chapter 3 (p. 25), section 3.11 (p. 32), and chapter 4 (p. 46), section 4.11). A build whose protocol self-test failed is never confirmed, on either board.

9.12 Tried and rejected

FOR FIRMWARE CHANGES

These were dead ends on this machine. They may not be dead ends on yours.

- A text protocol: rejected when the protocol was written, for the reasons in section 9.11 (p. 245).
- Opening the A32′s serial port without naming its pins: it took the audio frame-clock pin, and the sound failed later with no obvious cause. Now impossible.
- 0xA5 0xA5 as the start marker: set aside for 0xA5 0x5A when the protocol was written.
- A table-driven CRC: the bit-by-bit CRC is fast enough, and the table would take cache the display interrupt wants.
- Halting the A32 on a failed self-test: an updated program that failed it was unreachable.
- The self-test as the only guard against oversized payloads: replaced by compile-time checks.
- **`MSG_ADC_CLOCK`** (0x1C): cut the converter's clock pin to silence the input. Retired once the output converter took its clock from the same pin: cutting it became a clock error at the output, not a mute.
- **`MSG_SD_DELAY`** (0x41): a remote write to the audio output's delay register. It sent the radio to full volume (of its four positions, only 0 played correctly), and a build that still carried it was judged unsafe. Its last road, a key on the A32′s USB console, was removed too; only the read-back remains, in the status report and on the A32′s status line (chapter 10 (p. 256)).
- An A32 settings save that ignored its own result: a failed write read as saved.
- An A32 that saved settings during its trial, with advice to export a settings file first: replaced by holding every write until confirmation.

- A settings mirror that outlived the A32's silence: portal edits went nowhere and were overwritten when the A32 came back, and a download wrote the stale copy.
- A protocol mismatch that looked like a dead wire: the dropped frames were counted but shown nowhere, so a half-updated pair read `SILENT, rx 0`, exactly like a broken cable.
- An A32 that plays on when the S3 is silent (the link's health displayed, never acted on): replaced by the awake rule.
- Waking on the S3's last known amplifier state: could wake unmuted into an amplifier that was off.
- A 250 ms wait before the first flash write: no margin while the output chain was briefly 16 buffers, and a risk of a click; 400 ms since, kept when the chain went back to 8.
- A 5 s wait for the answer to BEGIN: failed consistently; 20 s since.
- Ignoring the answer to END: a lost END was reported as "image sent" while the A32 kept its old firmware.
- A 48-character error buffer on the S3: cut the messages short; 96 since.
- Restoring "unmuted" after a failed update: it unmuted a set the user had muted. The user's own mute setting is restored now.
- The core's default of trusting every updated program: a program that crashed at start-up needed USB.
- Confirming a trial program at the first handshake: the handshake came before Bluetooth and playback, so a crash there was already "good". It lasted one afternoon.
- An unconditional five-minute confirmation: restricted the same evening to "nothing heard".
- Bringing the link up before Bluetooth: useless (section 9.11 (p. 245)).
- Battery-clock validity by plausibility alone: a DS3231 that had lost power reported a plausible, wrong date, and the S3 adopted it.
- Reading the Bluetooth transmit power back in every status report: coincided with an A32 restart the moment Bluetooth started page-scanning. The value is now read back once, when it is set.

9.13 Known limits

- Any reset during the trial minute rolls a good program back: a power cut, Reboot the audio board, or a crash in that minute. A settings change made in that minute goes with it.
- A program that failed its protocol self-test is never confirmed. Sent over the air, it stays on trial, refuses any new update, and is rolled back by the next reset. Its own message, "awaiting OTA", is true only for a program flashed by USB, which is not on trial.
- An A32 program built with the wrong protocol version confirms itself after five minutes, because frames of another version count only as wrong-version. Section 9.14.5 relies on this; the flip side is that such a program,

built by mistake, becomes permanent, and the S3 must then be updated to match it. The S3 shows the mismatch.
- A protocol mismatch is shown only on the S3′s side (section 9.5 (p. 231)).
- With the A32 absent from start-up, the S3 has no clock even with network time. The S3 sets its clock from TIME, from `W`, or when it passes network time on to the battery clock, and that last step waits for the handshake.
- An edit to an A32 setting just before the A32 goes silent can still be lost: the refusal starts only once 2 s of silence have cleared the S3′s mirror.
- Setting the A32′s watchdog also changes core 0′s idle check from 5 s to 15 s.
- The S3′s "PROTOCOL MISMATCH" check in its HELLO_ACK handler can never fire; the wrong-version count does that job.
- The A32′s version is in no portal field; it appears only on the S3′s console (`[PASS] A32 up: ...`).
- The battery clock's year is two digits from 2000; the chip's century bit is ignored.
- **`- an earlier update was ROLLED BACK`** stays in every boot report until the next update.
- The A32′s forced save before Reboot the audio board is tried once; if that write fails, the board restarts and the change is lost.
- The S3′s portal task shares core 0 with the needle's step emitter. While the needle moves, the portal is starved, and the relay runs on the portal task: this is the likely cause of four A32 update failures seen during bring-up. Keep the needle at rest during an A32 upload (chapter 8 (p. 186)).
- The S3 does not halt on its own failed self-test either: it runs on, and its program is never confirmed (chapter 3 (p. 25), section 3.11.7 (p. 35)).

Seen working on the radio: the trial state, the refusal of a second update on trial, the confirmation, a real rollback caused by a watchdog restart of a hanging test program (reset reason 6), updates of both boards over the air, each on trial and then confirmed, with the S3′s link line at `crc 0 wrong-version 0`, and the A32′s settings hold during a trial.

Never exercised on the radio: the five-minute "the S3 never spoke" confirmation; the degraded self-test mode, and with it the rule that such a build is never confirmed; a settings download while the A32 is silent (`n/a` rows) and the 409 refusal of an edit then; the wrong-version count and pill with a real mismatch (they have only read 0); the battery-clock pills in a real fault; a failed A32 settings save.

9.14 Changing it safely

> FOR FIRMWARE CHANGES
>
> ### 9.14.1 What must stay true
>
> 1. The protocol and link files are shared by both builds. Any change to a message structure means rebuilding and updating both boards;

know in advance which side loses what while they differ (section 9.14.5).

2. Prefer appending fields (a length mismatch costs only that message) and new message ids (ignored by old peers, no version change). The rule written in the protocol file: a new message id, no change; a structure whose size changes, no change, because a wrong length is refused and the mismatched peer loses only that message type; a change the length check cannot see (same size, different layout or meaning), change `PROTO_VERSION`. Change it only by the procedure in section 9.14.5.
3. Keep every payload structure in the compile-time size checks, and keep the three offset checks if you touch the update data structure (Appendix F (p. 338)).
4. Never reuse a retired id (0x1C, 0x41). Put new S3-to-A32 diagnostics in 0x4n.
5. Open the link with explicit pins, and set the receive buffer before the port is opened.
6. The mute is a state: every new path that changes it goes through `applyMute()`, and `protoBroken || !awake` stays in it.
7. Only frames from the S3 keep the A32 awake. Do not block the S3's main loop for more than about 1.5 s without reading the link and sending a PING.
8. The update receiver stays reachable in every degraded mode (asleep, self-test failed). Add no halt before the link is opened on the A32.
9. `verifyRollbackLater()` keeps returning true, and confirmation happens only after the new program has done its real work (woken, Bluetooth, audio). Do not call `esp_ota_mark_app_valid_cancel_rollback()` earlier. Keep the on-trial refusal in the `MSG_OTA_BEGIN` handler: it is the only thing that stops an update being written over the fallback program.
10. Start-up finishes in under 15 s, and so does any single pass of the main loop (the update's image check, delays).
11. The battery clock: UTC only; valid requires OSF clear; writing the time clears OSF.
12. The S3 refuses portal pushes of A32 settings while it has no mirror (`haveCfg` false), and clears `haveCfg` whenever it declares the A32 silent.
13. A settings save, on either board, counts as done only when the write reached flash.
14. Nothing reaches the A32's stored settings (namespace `amb`) while its program is on trial: every write goes through `settingsFlush()`, which returns early on trial, and `confirmImage()` writes what was

held. A new path that writes `amb` directly would reopen the rollback trap.

9.14.2 Traps

- The default pins. On the classic ESP32, `Serial2`'s default pins are GPIO16 and GPIO17, and the firmware uses GPIO17 as the audio frame clock (`A32_I2S_LRCK`). A bare `Serial2.begin(baud)` would silently take that pin, and the sound would fail much later with no obvious cause. That is why `Link::begin()` has no form without pins.
- The buffer size before **`begin()`**. The core ignores `setRxBufferSize()` once the port is running and keeps its 256-byte default. That is enough for control frames, which is why the first version seemed to work, but 1 KB update frames at 921 600 baud overflow it.
- Two writers on the S3. The S3 writes the link from its main loop on core 1 and from the portal task on core 0 (the update relay). Frames do not interleave because each frame goes out in one `write()` call, and the core's serial write holds a lock for the whole call. Keep it that way.
- A wrong length is refused silently. A structure grown on one side looks like "telemetry stopped", not like a wire error: the checksum passes.
- A packed field cannot bind to a reference; copy through local variables (the STATE builder does).
- **`mktime()`** works in local time. Pin `TZ=UTC0` around it on the A32 (`rtc.h` does).
- The update's answer has no frame identity. Reason about it only in byte counts.
- The 400 ms wait is fixed. If you raise `fadeOutMs` above about 340 ms, or deepen the audio output's DMA chain, raise the wait in the `MSG_OTA_BEGIN` handler to match (section 9.8 (p. 239)).
- A successful relay does not mean the new program stays. It may still roll back within the minute. Check for `[A32] image confirmed` and the version at the next handshake.

9.14.3 Where the code is

Table 107. Where the code is

File	What is in it
`include/proto.h`	Message ids, payload structures, constants, `protoEncode()`, the decoder `ProtoFramer::feed()`, the compile-time checks. Shared by both builds.
`include/link.h`	The `Link` class and `protoSelfTest()`. Shared by both builds.
`include/pins.h`	Every pin either program uses (Appendix I (p. 358)).
`src/a32/main.cpp`	The A32's `setup()` and `loop()`, `onMessage()`, `serviceWake()`, `applyMute()`, `bootReport()`, `confirmTick()`, `confirmImage()`, `settingsFlush()`, `verifyRollbackLater()`.
`src/a32/rtc.h`	`Rtc::begin()`, `Rtc::read()`, `Rtc::write()`.
`src/s3/main.cpp`	The S3's link keepalive block in `loop()`, its `onMessage()`, the relay (`a32OtaBegin()`, `a32OtaChunk()`, `otaSendFrame()`, `a32OtaEnd()`), `nowEpoch()`, `limitMonitor()`, `setClockInteractive()`.
`src/s3/portal.cpp`	`hOtaA32End()`, `otaWatchdog()`, and `hSet()`'s refusal through `a32CfgKnown()`.

The **`Link`** class, a thin wrapper around a hardware serial port and the decoder:

Table 108. Where the code is

Member	What it does
`begin(port, rxPin, txPin, baud)`	Sets a 4096-byte receive buffer, then opens the port. There is no version without pins.
`poll()`	Reads every available byte into the decoder. On each complete, valid frame: stamps the receive time, counts it, calls the handler. Never blocks.
`send(type, payload, len)`	Encodes the frame into a stack buffer and writes it with one `write()` call. Refuses, and counts it, if the payload is too big.
`peerAlive()`	A valid frame within the last 2000 ms.
`notePeerHello()`	Stores the other board's version string and protocol number.
`rxCount()`, `txCount()`, `badCrc()`, `badVer()`	Valid frames in, frames out, frames with a bad checksum, frames with another protocol version.

9.14.4 How to test a change

- At start-up, both boards print `[PASS] protocol v4, state frame 138 bytes, CRC rejects damage`.
- The S3's console `s`: the link line shows alive or SILENT, the frames received, the checksum errors and the wrong-version frames; a healthy pair reads `crc 0 wrong-version 0`. The portal's status: `linkrx`, `linktx`, `linkcrc`, `linkver`, `hello`, `astate`, and `rtc` for the battery clock.
- The A32's own console (USB, on the bench): `s` shows the link, the frames received, and whether it is awake and why.
- After any A32 update, the S3's console should show, in order: `[A32] boot: ... image ON TRIAL (rollback armed), watchdog on`, then `[PASS] A32 up: proto v4, firmware <new>`, then 60 s later `[A32]`

`image confirmed (a minute of running with the S3)`. A second update inside that minute must be refused.

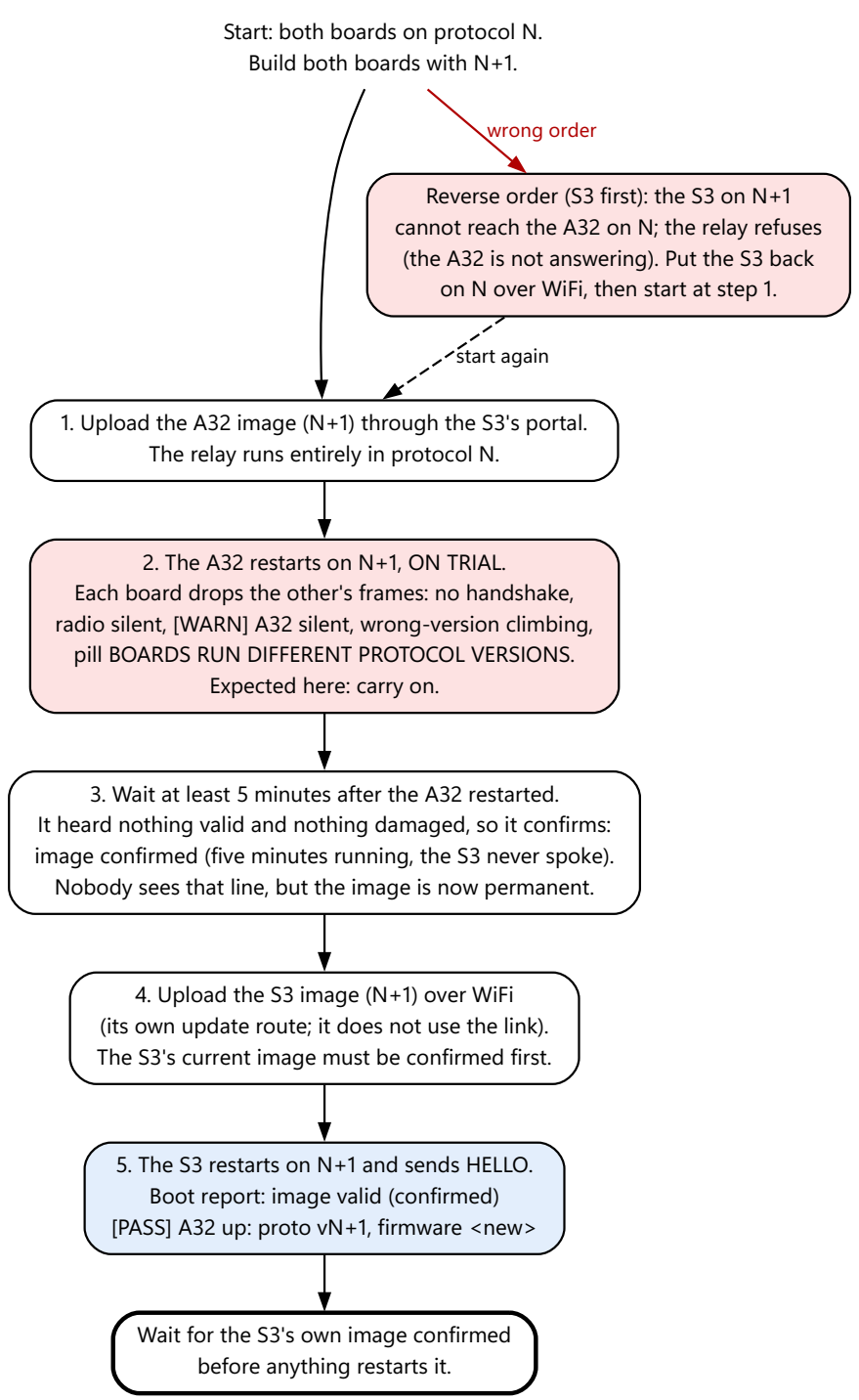
- To check the settings hold, change an audio-board setting in the portal during that minute (for example the blue lamp's `btLedOff`, from 255 to 254, then back): the A32 must write nothing until it confirms, then print `[A32] settings changed during the trial are now saved`.
- The rollback itself needs a deliberately bad program: one whose main loop spins for ever, or one that crashes when it wakes. Relay it, then look for `reset reason 6` and `- an earlier update was ROLLED BACK` in the next boot report. Keep a USB cable at hand in case the bad program does something the watchdog cannot catch.
- Identify a board with certainty before flashing it, by USB or over the air (chapter 3 (p. 25), section 3.1 (p. 25)).

9.14.5 Changing the protocol version

A change of `PROTO_VERSION` is a hard wall: two boards with different values cannot exchange a single frame, including the update frames the A32 depends on. The A32 can be updated only through the S3, while the S3 can be updated over its own WiFi, which does not use the link. So there is exactly one order that works over the air.

Before you start, ask whether the change can be made by appending fields or adding message ids instead. Both keep a half-updated machine playing.

Figure 31. Changing the protocol version: the only order that works over the air



Start with both boards on protocol N. Build both boards with N+1.

1. ☐ Upload the A32 program (N+1) through the S3's portal. The relay runs entirely in protocol N, since both boards are still on N; the content of the image does not matter to it.

2. ☐ Let the A32 restart on N+1, on trial. It drops every frame from the S3 at the decoder and counts them as wrong-version; its counts of valid frames and of damaged frames stay at 0. There is no handshake: the A32 is asleep and the radio silent. The S3 sees nothing valid either: `[WARN] A32 silent`, HELLO every second. Its console `s` shows `wrong-version` climbing (the A32′s STATE frames now carry N+1), and the portal shows BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both. That is expected here: carry on.
3. ☐ Wait at least 5 minutes after the A32 restarted. It finds no HELLO, no valid frame and no damaged one, and confirms: `image confirmed (five minutes running, the S3 never spoke)`. Nobody sees that line, because the S3 drops it, but the program is now permanent.
4. ☐ Upload the S3 program (N+1) over WiFi, by the S3′s own update route. The S3′s current program must already be confirmed: an S3 update sent while the S3 is on trial is refused.
5. ☐ Let the S3 restart on N+1 and send HELLO. The A32 answers. Its boot report, the first it has sent this start-up, should read `image valid (confirmed)`, followed by `[PASS] A32 up: proto vN+1, firmware <new>`.
6. ☐ Wait for the S3′s own **`image confirmed`** before anything restarts it; otherwise it rolls back to N while the A32 stays on N+1. The S3 confirms about 70 s after start-up, once its network and portal are up, and does not need the A32 for it.

Why the five-minute fallback makes this possible. The fallback confirms a program that has heard nothing valid and nothing damaged, and frames of another protocol version count only as wrong-version. So an A32 updated first confirms itself after five minutes of hearing only the old S3, and a later reset (a power cut, say) no longer takes it back to N. If the S3 is updated before those five minutes are up, the A32 instead confirms 60 s after the new S3′s first HELLO, and that works too. The risk on that shorter path is a reset of the A32 before it confirms: it would roll back to N while the S3 is already on N+1, the mismatch would be the other way round, and the relay could not reach the A32. You would then put the S3 back on N over WiFi and start again. Waiting the five minutes removes that case.

> Note The reverse order does not work. An S3 on N+1 cannot reach an A32 on N: the A32 drops its frames, the handshake never happens, and the relay refuses with `the A32 is not answering`. Put the S3 back on N over WiFi, then follow the order above.

10. Sound and Bluetooth

The A32, the audio board, makes the sound of the radio. It is a classic ESP32-WROOM-32, a separate chip from the ESP32-S3 that runs the rest of the radio. It plays the tube radio's audio, digitised and played back, or music streamed from a phone over Bluetooth, and it stays silent on AUX. Phone music needs Bluetooth Classic, which the ESP32-S3 does not have; that is why this job runs on a second chip.

This chapter covers the sound path, the volume, the sources, Bluetooth and its blue lamp, the A32′s tight memory, what it reports, and the pops: loud clicks that took two weeks to tame and that a change to the clock pins cut by about 150 times. The A32′s start-up, its awake and asleep states and the link to the S3 are in chapter 9 (p. 224). Its updates and its trial and rollback are in chapter 3 (p. 25). How it saves its settings is in chapter 7 (p. 153), section 7.4 (p. 161). Appendix A (p. 309) lists every audio and Bluetooth setting, Appendix G (p. 344) the compiled constants and the experiments, and Appendix H (p. 348) the pop hunt in full.

Specific to this build — adapt. The gains, the channel order, the volume curve and the knob calibration here suit this tube set, this amplifier and this speaker set. Adapt them to yours.

How the converters, the volume knob, the source selector, the pair button and the blue lamp are wired is in the Hardware Bible, chapters 6 and 9. This chapter states in place the few hardware facts it needs.

10.1 What you hear

1. Three sources, picked by the front selector. RADIO is the tube radio's audio, digitised and played back. BT is music streamed from a phone. AUX is an analogue input that never passes through the A32, so on AUX the A32 stays silent.
2. Turning the selector fades the old source out quickly (150 ms) and brings the new one in gently (1 s).
3. The front volume knob sets the level along a curve that suits the ear. It acts on RADIO and BT only, never on AUX. Radio and Bluetooth come out equally loud at the same knob position.
4. Bluetooth is invisible unless the selector is on BT and the amplifier is on. A phone that paired before can reconnect by itself, from the phone. A new phone needs the pair button, which makes the radio visible for 90 seconds. The radio never calls a phone.
5. The blue lamp on the front tells the Bluetooth state (section 10.7.6 (p. 274)).
6. When the amplifier is off, or the S3 stops talking, the A32 sleeps: it plays nothing, the lamp goes dark and Bluetooth closes (chapter 9 (p. 224)).

7. During an A32 update the sound stops while the A32 writes its flash. The needle and the clock keep working (chapter 3 (p. 25), section 3.10.3 (p. 30)).
8. A reboot or an update of the A32 makes one pop of its own: the audio clocks stop, the DAC goes to standby, and it restarts.

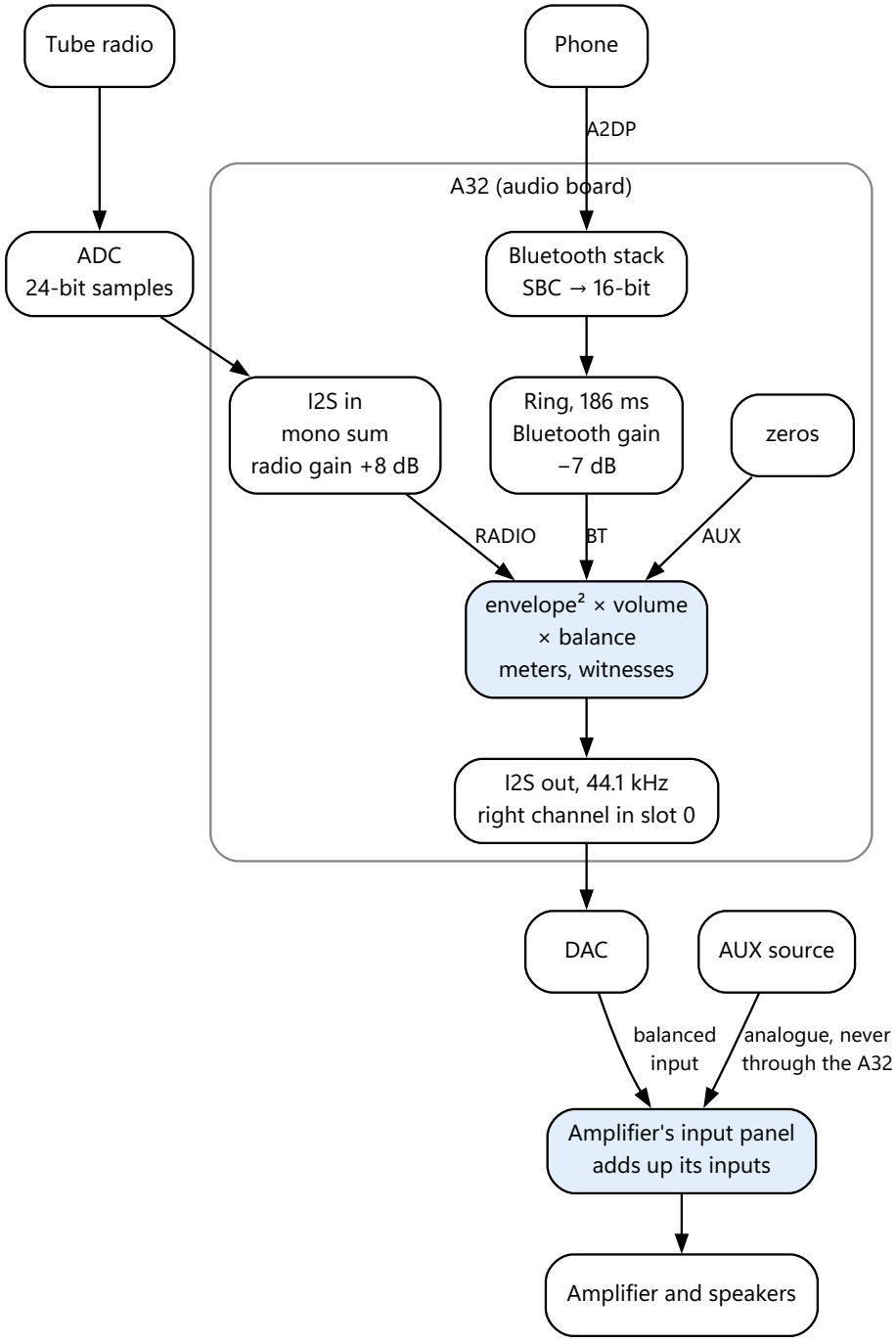
Words used in this chapter. These terms recur from here on.

Table 109. What you hear

Term	Meaning
I2S	A serial bus for audio samples. BCK (bit clock) ticks once per bit. LRCK (word select) toggles once per frame and says which channel the bits belong to. Data lines carry the bits, most significant bit first.
Slot	One channel's share of a frame. Here each frame has two 32-bit slots.
MCLK	The master clock, a faster reference that some converters need. The converters here call their MCLK input SCK.
Full duplex	One I2S peripheral sends (to the DAC) and receives (from the ADC) at the same time, on shared BCK and LRCK.
DAC / ADC	Digital-to-analogue converter (the output) and analogue-to-digital converter (the radio input).
DMA	Direct memory access. The I2S driver keeps a chain of RAM buffers that the hardware plays out or fills by itself; the CPU only keeps the chain topped up. DMA buffers must sit in a special part of internal RAM.
APLL	The ESP32's audio PLL, a clock generator that can make 11.2896 MHz exactly.
A2DP, SBC, PCM	The Bluetooth profile for streaming stereo music. The phone compresses the music with the SBC codec; the Bluetooth library decodes it to 16-bit samples (PCM).
AVRCP	The Bluetooth remote-control profile that goes with A2DP: play, pause, next, and the phone's volume slider.
Bluedroid	The Bluetooth stack inside Espressif's framework. It runs its own task on core 0.
Connectable, discoverable	The two halves of the scan mode. Connectable: a phone that already knows the radio can connect. Discoverable: the radio shows up in a phone's list of new devices.
Bond, pairing	The stored keys that let a phone that paired once reconnect later.
dBFS	Decibels relative to digital full scale. 0 dBFS is the loudest sample possible.
Q16, Q24	Fixed-point numbers. In Q16, 65 536 means 1.0. In Q24, 16 777 216 means 1.0.
RMS, peak	The average power of the signal, and its largest single sample.
Underrun	Bluetooth audio ran out in the middle of a block.
LEDC	The ESP32's PWM peripheral, used here for the blue lamp.

10.2 The audio path

Figure 32. The audio path: RADIO and BT go through the A32 and the DAC; AUX goes straight to the amplifier'sinput panel, which adds up its inputs



- RADIO. The tube radio's audio goes to the ADC, which sends 24-bit samples to the A32 over I2S. The A32 sums the two channels to mono, applies the radio gain, then the fade, the volume and the balance, and sends the result over I2S to the DAC.

- BT. The phone streams SBC-compressed music over A2DP. The Bluetooth stack decodes it into a ring buffer. The A32 takes samples from the ring, applies the Bluetooth gain, and then the same fade, volume and balance.
- AUX. The A32 sends digital zeros to the DAC. AUX audio reaches the amplifier without passing through the A32.

The amplifier adds its inputs together; it does not choose between them (Hardware Bible, chapter 6, section 6.7 (p. 131)). So on AUX the A32's only job is to send zeros. The front selector switches no audio: it only tells the A32 which source you chose.

10.3 The audio interface (I2S)

10.3.1 Installed once, never touched again

The A32 installs its I2S driver once, at start-up. It is never reinstalled, reconfigured, stopped or re-pinned. Changing source is only a choice of where the next block of samples comes from: ordinary code, with no driver calls.

Why. The two paths want different drivers. The Bluetooth library, left to itself, wants a transmit-only, 16-bit driver with no MCLK. The radio path needs full duplex, 32-bit slots and an MCLK for the ADC. Reinstalling the driver at every source change, while the Bluetooth stack still holds a reference to it, is the kind of fault that crashes the chip in ways that are hard to trace. So the firmware installs the superset once and keeps it. This is also why the platform is pinned (chapter 3 (p. 25), section 3.16 (p. 41)): the next Arduino core retires this I2S driver.

10.3.2 The configuration

Both converters follow the A32's clocks as I2S slaves (Hardware Bible, chapter 6, section 6.2). The table gives the I2S driver's settings, field by field, for anyone changing or porting the driver.

> FOR FIRMWARE CHANGES

Table 110. The configuration

Field	Value	Why
Port	`I2S_NUM_0`	
Mode	master, transmit and receive	The ESP32 makes the clocks; both converters follow them.
Sample rate	44 100 Hz	Fixed. There is no resampler.
Bits per sample	32	The ADC sends 24-bit samples in standard I2S format. They arrive most significant bit first at the top of a 32-bit slot, so the raw word is already at full scale.
Channel format	`I2S_CHANNEL_FMT_RIGHT_LEFT`	
Frame format	`I2S_COMM_FORMAT_STAND_I2S`	Standard (Philips) I2S: data starts one bit after the LRCK edge.
Interrupt	level 1	
DMA buffers	8 buffers of 256 frames	46.4 ms of audio per direction (section 10.8 (p. 275)).
`use_apll`	true	See below.
`tx_desc_auto_clear`	true	If the CPU falls behind, the driver sends zeros instead of replaying old audio.
`fixed_mclk`	44 100 × 256 = 11.2896 MHz	BCK is 64 × 44 100 = 2.8224 MHz.

After installing the driver, the firmware sets the pins, sets the clock-pad drive to 3 (section 10.10 (p. 277)) and starts the audio task. The pins: MCLK on GPIO0, BCK on GPIO18, LRCK on GPIO17, data to the DAC on GPIO4, data from the ADC on GPIO19.

Why the APLL stays on. The MCLK on GPIO0 feeds both converters' SCK inputs. The DAC switches off its own internal clock generator when it is given an external SCK. The APLL is the only ESP32 clock that makes 11.2896 MHz exactly, and BCK and LRCK are divided from that same clock, so the ratio SCK:LRCK is exactly 256 by construction. Without the APLL the divider runs from a 160 MHz clock that cannot make 44 100 Hz exactly, a worse clock for both parts. Do not "try" turning it off.

Why MCLK is on GPIO0. The classic ESP32 can put the I2S master clock only on GPIO0, 1 or 3. GPIO1 and 3 are the USB console. So GPIO0 must never be taken away from the I2S peripheral: that would stop the clock of both converters at once.

If installation fails, the A32 prints a `[FAIL]` line on its USB console and does not start the audio task. The radio is then silent, and only the A32's USB console says why.

10.3.3 Keeping the Bluetooth library off I2S

The Bluetooth library used here (ESP32-A2DP) normally owns the I2S driver. Here it must not. Three guards make that structural, and each was added after a real bug.

FOR FIRMWARE CHANGES

The rule behind all three: “make it impossible, do not make it a rule to remember.”

1. The stream-reader flag (`a2dp.set_stream_reader(a2dpStream, false)`). The library hands the decoded samples to the firmware, and the `false` tells it not to write I2S itself.
2. An empty output object (`a2dp.set_output(nullOut)`, a `NullA2dpOutput` whose every method is empty). The flag does not stop everything: the library calls its output’s `set_sample_rate()` whether or not output is enabled, and the library’s default output then reconfigures the driver. The moment a phone started streaming, that call turned the firmware’s driver into a 16-bit transmit-only one. Bluetooth then played at full scale and garbled, ignoring the volume; the channels swapped; and the radio stayed dead afterwards. The empty object has no way to reach I2S at all.
3. A no-op volume control (`a2dp.set_volume_control(&noVol)`, the library’s `A2DPNoVolumeControl`). The library’s AVRCP volume scales the sample buffer in place, one line before it calls the stream reader, and the flag does not stop it. At a phone slider of 1/127 that is a divide by about 2048, with truncation. With the no-op control the library cannot touch the samples. The phone’s volume is still read as information and published as `avrcpVolume`. The firmware’s own curve is the only volume law in the machine.

The firmware uses the library’s plain sink class (through a subclass, section 10.15.5), not the “queued” variant. The plain class starts no I2S task of its own, so the decoded samples arrive directly on the Bluetooth task.

10.3.4 Channel order

The firmware puts the right channel in the first I2S slot (slot 0) and the left in slot 1, the reverse of the usual convention. Every frame it writes is `{right, left}`. It does so because on this machine that order gave correct stereo.

> Note Check the channel order on your build. This is an adjustment for this machine, not a fact about the parts. The amplifier-to-speaker side is correct; verify the A32-and-DAC side with a left/right test track played over Bluetooth.

There is no portal switch to swap the channels; one is listed as a possible improvement (chapter 12 (p. 290)). Until then, swapping them is a firmware change in the audio task, where the Bluetooth path, the balance and the meters all take slot 0 as the right channel.

10.4 Inside the audio board

Sections 10.4.1 and 10.4.2 list the A32's tasks and buffers, for anyone changing the audio code. Section 10.4.3 follows one block of audio through the A32, and explains the meters the portal shows.

10.4.1 Tasks and cores

FOR FIRMWARE CHANGES

Table 111. Tasks and cores

Who	Core	Priority	What it does
Audio task (`audioTask()` in `src/a32/audio.cpp`)	1	6	Reads or fetches one block, processes it, writes it to I2S. Stack 4096 bytes.
Bluetooth task (Bluedroid)	0	the stack's own	Decodes SBC and hands the samples to the ring (`a2dpStream()` → `Audio::pushBt()`).
Arduino main loop (`loop()` in `src/a32/main.cpp`)	1	1	The link to the S3, sleep and wake, source and volume reading every 50 ms, the Bluetooth states, the lamp, the 250 ms state message to the S3, the USB console.

Core 0 belongs to the Bluetooth stack. Sharing a core between SBC decoding and a real-time audio loop invites dropouts, so the audio task sits on core 1.

The audio task never waits on a timer, and must not. Its pacing is free. On RADIO, reading a block waits until a full block has been captured. On BT and AUX, writing a block waits while the DMA chain is full. The audio task also never prints: serial output from a priority-6 real-time task causes the dropouts it would be trying to report. Anything worth saying is announced later from the main loop.

10.4.2 Buffers

Table 112. Buffers

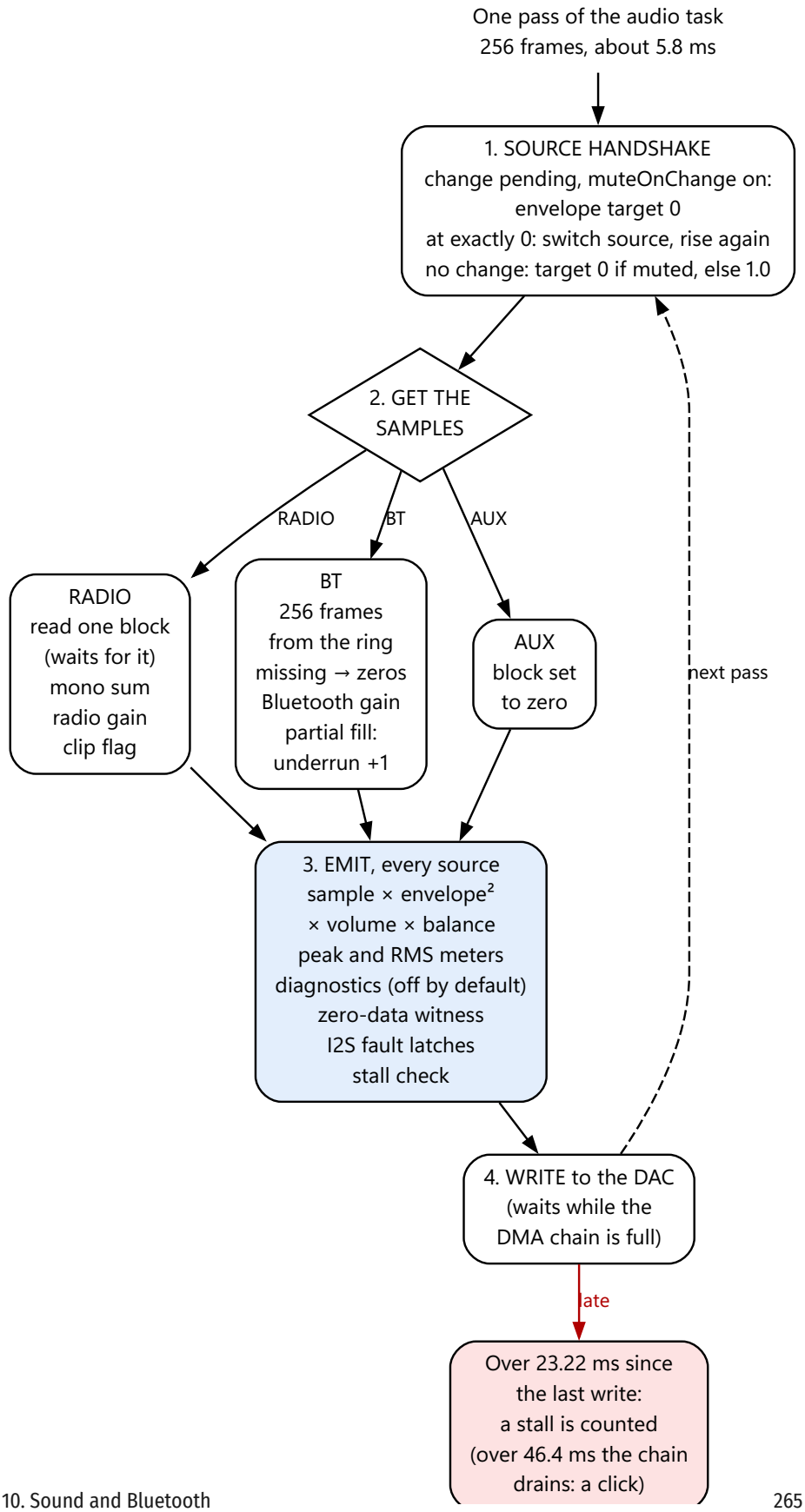
Buffer	Size	Notes
One block (`BLOCK`)	256 frames	One pass of the audio task, about 5.8 ms.
`inBuf`, `outBuf`	2 KB each	256 frames × 2 slots × 4 bytes, static.
`btBuf`	1 KB	
Bluetooth ring (`RING_FRAMES`)	8192 frames of two 16-bit samples, 32 KB, static	Absorbs about 186 ms of jitter. Bluetooth delivers audio in bursts; the radio path is paced by the ADC and needs no ring.
DMA chain	8 × 256 frames per direction, 32 KB in all	46.4 ms per direction (section 10.8 (p. 275)).

The ring needs no lock. It has one writer (the Bluetooth task) and one reader (the audio task). The writer publishes the new head with a release store; the reader reads the head with an acquire load and publishes the new tail with a release store. The chip is an in-order CPU and the ring sits in uncached RAM, so missing barriers had never caused harm; the code adds them anyway, because "correct by luck is not correct". The writer

> must never block the Bluetooth task, so a full ring drops the newest samples instead of waiting.

10.4.3 One block through the audio task

Figure 33. One pass of the audio task: the source handshake, the samples from the chosen source, the processingcommon to every source, and the write to the DAC



Each pass does four things.

1. The source-change handshake.

If a new source is waiting and `muteOnChange` is on, the fade envelope's target goes to 0. When the envelope reaches exactly 0, the task switches to the new source, and the envelope rises again. So a change is: fade out, switch in silence, fade in. With `muteOnChange` off the task switches at once, with no fade. With no change waiting, the target is 0 if muted and 1.0 otherwise.

2. Get the samples.

- RADIO. Read one block. With `monoSum` on (the default), each frame becomes `m = (slot0 + slot1) / 2`, sent to both slots. Both ADC channels carry the same mono radio signal (Hardware Bible, chapter 6, section 6.4), so the sum gains about 3 dB of signal-to-noise: the signal adds up, the converter noise does not. The result is multiplied by the radio gain (`gainRadioQ16`) with saturation; a saturated sample sets the `clip` flag. With `monoSum` off the two slots are kept apart.
- BT. Take 256 frames from the ring; missing frames are filled with zeros. Each 16-bit sample is shifted left 16 bits to full 32-bit scale, then multiplied by the Bluetooth gain (`gainBtQ16`). The underrun counter counts only a partial fill: audio was flowing and ran out mid-block, and the zero-fill made a click. An empty ring means nothing is streaming and is not counted. (A first version counted empty blocks too and ran at a few hundred per second, which made it useless.)
- AUX. The block is set to zero.

3. Emit.

The same code runs for every source.

- Volume, balance and the envelope (section 10.5 (p. 267)). Each sample becomes `sample × envelope² × volume × balance`.
- Meters. The peak meter reads the very end of the chain, so it moves with the knob. It is read and reset by the 250 ms state message, in tenths of a dBFS. It uses 64-bit arithmetic: a 32-bit version could never register the loudest possible sample (negating the most negative 32-bit value gives itself back), so it under-read exactly during a full-scale event. That fix made the meter trustworthy as evidence in the pop hunt. The RMS meter is measured after the source gain and before the volume, so the knob does not move it. That is what let the radio and Bluetooth gains be matched once for every volume. Peaks cannot do that job: FM stations are limited hard and music is not.
- Diagnostics, all off by default, applied in this order: the constant-DC word (replaces every sample), the low-bit output mask, the zero-data floor, the sign-extended tail. Appendix G (p. 344) describes each.
- Zero-data witness. A copy of the DAC's own zero-data detector. The DAC mutes its output by itself after 1024 frames of zero data (Hardware Bible, chapter 6, section 6.5). The witness counts each time the DAC would enter

and leave that mute, on what is actually about to go out. It counts a word as zero when its top 24 bits are zero, on the reading that the DAC resolves 24 bits of a 32-bit slot; the datasheet does not say whether it keeps the bottom 8.
- I2S fault latches. The peripheral's raw interrupt bits that the driver does not service are OR-ed into `i2sSticky`, which is never cleared.
- Stall detector. If more than 23.22 ms (one zero-data window) passed since the previous write, the task counts a stall and records its length and time.

4. Write

the block to I2S, waiting while the DMA chain is full.

10.5 Volume, knob, fades, mute and gains

10.5.1 The knob is a sensor

The front volume knob is a position sensor, not an audio part. It turns a potentiometer whose wiper the A32 reads on GPIO35, over its full 0–3.3 V swing, with the ESP32's own ADC (not the radio's converter) at its widest input range (11 dB attenuation) (Hardware Bible, chapter 9, section 9.4).

Reading. Every 50 ms the A32 makes two throw-away conversions, then takes the median of five. The raw reading is mapped through a three-point calibration (minimum, centre, maximum): each half of the travel is its own straight line, and the centre reading maps to 128 of 255 by construction. The calibration exists because, measured on this set, the knob at its mechanical centre read 215 of 255 with only the two ends calibrated. A deadband of 24 ADC counts (about 0.6 % of travel) stops the volume trembling at rest.

Three rules follow from "the knob is a sensor":

- The first reading after the A32 starts always applies. The knob's position is the stored state.
- The knob does not store the volume. Nothing needs storing (chapter 7 (p. 153), section 7.4.3 (p. 162)).
- The portal can also set the volume. That value holds until the knob moves by more than the deadband. The portal action `sys.pot 0` makes the A32 ignore the knob altogether; it is an experiment and is not kept across a reboot.

Calibrating the knob. On the portal's Audio tab (administrator), turn the knob to its minimum and press Pot: minimum, to its mechanical centre and press Pot: centre, to its maximum and press Pot: maximum (actions `pot.min`, `pot.ctr`, `pot.max`). The A32's USB console keys `p`, `c` and `P` do the same. For the portal buttons the A32 measures the knob itself, with the median of nine readings, because the reading and the thing being calibrated must not sit on opposite sides of a link. The three readings are stored on the A32; Appendix A (p. 309) lists their defaults.

The fallback. If the stored calibration is implausible — the maximum not more than 64 counts above the minimum — the A32 falls back to the compiled defaults 60 and 3990, with no centre point. They are roughly this set's knob ends, held as a starting point until the knob is calibrated.

> ⚠ WARNING Calibrate the knob before anything else. A stored calibration of 930, 927 and 933 was implausible, so the firmware silently used its defaults, which had nothing to do with that knob. The volume was pinned near 18 of 255 (about −62 dB, the DAC near −77 dBFS), with the amplifier turned up to match: every fault then arrives 62 dB louder than it should. That is a safety issue, not a quality one.

10.5.2 The volume law

`volQ16 = (volume / 255)^gamma × 65 536`, where gamma is the setting `taper` divided by 10, clamped to 1.0–4.0.

Table 113. The volume law

Gamma	At half travel	Character
1.0	—	linear amplitude
2.0	−12 dB	
2.5 (default)	−15 dB	
3.0	−18 dB	
about 3.3	—	behaves like a real audio-taper potentiometer

The power is computed only when the volume or gamma changes, once per block, never per sample. The new gain takes effect as a step at a block boundary; it is not ramped. That is harmless for a hand on a knob. The "volume watch" counts these steps (`gsteps`) and records the smallest and largest gain applied, because the volume reading once wandered by itself, between 17 and 20, during the pop hunt.

10.5.3 Balance

`balance` runs from −100 to +100; negative moves the sound to the left. The other side is scaled by `65 536 − |balance| × 655`. At ±100 the far side is about −65 dB, not silent.

10.5.4 Fades

The fade envelope is a Q24 number, stepped every sample and squared before use. A linear amplitude ramp does not sound like a fade: it rushes up and then sits there. Q24 is needed because Q16 cannot express a one-second fade (the step rounds to 1, which gives 1.49 s).

The defaults are 1000 ms in and 150 ms out (`fadeInMs`, `fadeOutMs`). The reasons given are "a gentle arrival, not a jumpscare" and "leaving should be quick - you turned the knob". A value of 0 ms is treated as 1 ms, so a source change can never wait forever for a silence that never comes.

A fade-out longer than about 340 ms makes every A32 update start with a click, because the update's mute wait is fixed (chapter 3 (p. 25), section 3.10.3 (p. 30)).

10.5.5 Mute

Mute is a function of state. The A32 computes

```
mute = protoBroken  OR  not awake  OR  muted
```

and every place that can change any of the three recomputes it. `muted` is the user's mute; awake is chapter 9 (p. 224)'s rule (amplifier on, and the S3 heard within 2 seconds); `protoBroken` means the firmware's link self-test failed at start-up, and such a build stays up, muted and invisible to phones, so that it can be replaced over the air (chapter 3 (p. 25), section 3.11 (p. 32)). The one deliberate exception is the start of an A32 update, which forces the mute directly; an update that gives up restores the user's mute, not "unmuted" (chapter 3 (p. 25), section 3.10.3 (p. 30)).

Why computed: in the first version, falling asleep set the mute and four other places cleared it without looking at sleep. Turning the source knob while asleep, because the S3 had died, then brought the sound back, and the chosen alarm — silence when the S3 is gone — vanished.

Mute is never stored. It is forced off at every start of the A32 (chapter 7 (p. 153), section 7.4.1 (p. 161)). A stored mute once booted the set silent, with nothing on the front of the radio to say why.

Mute is always software. Muting means writing zeros. No A32 pin reaches the DAC's hardware mute input: the DAC module holds it un-muted by itself, and GPIO16 is unused (Hardware Bible, chapter 6, section 6.5 (p. 127)).

The I2S clocks keep running while asleep. Asleep is a mute, not a stop.

10.5.6 Gains

The compiled defaults are +8.0 dB for the radio (`gainRadio`) and −7.0 dB for Bluetooth (`gainBt`). They were measured with the RMS meter:

- Radio. At 0 dB the radio read RMS −29.9 dBFS with peaks to −14.0 dBFS. +8 dB puts its peaks near −6 dBFS and its RMS near −22 dBFS. The old +15 dB clipped at the gain stage, which means at every volume.
- Bluetooth. A −14 dBFS RMS pink-noise reference played from a PC at full volume arrived at about −15 dBFS. −7 dB lands it within about 1 dB of the radio, so changing source does not change loudness.

The A32 clamps any gain to −60 to +30 dB. The radio gain suits the tube radio's output with the tube set's own volume control at about one eighth of its travel (Hardware Bible, chapter 6, section 6.8). A different tube-set level needs a different radio gain. The amplifier's ceiling was set by ear with the A32 at full volume, and these values were kept.

10.6 Changing source

The A32 reads the front selector on GPIO36 every 50 ms. It first makes three throw-away conversions, because GPIO36 reads high for a while after another channel of the ESP32′s own ADC has been sampled (78 counts of residue, measured). Then it takes the median of five.

Table 114. Changing source

Raw reading	Source	Voltage on GPIO36 (Hardware Bible, chapter 9, section 9.3)	Unit
below 424	AUX	0	V
424 to 2470	BT	0.81	V
2471 or above	RADIO	3.3	V

The thresholds live in `pins.h` as `A32_MODE_THRESH_AUX_BT` and `A32_MODE_THRESH_BT_RADIO`.

On a change the A32 does, in order:

1. If the old source was BT, it pauses the phone and then drops it (section 10.7.3 (p. 273)).
2. It hands the new source to the audio task, which ramps across (section 10.4.3 (p. 264)).
3. It recomputes the mute, the Bluetooth state and the scan mode.

The fade exists because a change of source without a ramp is an audible thump through a valve amplifier. The selector is an on-off-on switch whose centre position, off, reads 0 V through 22 kΩ, which is AUX. So a turn from RADIO to BT, or back, can pass through AUX on the way.

At start-up the A32 reads the selector once and sets the source before its main loop first runs. The audio task starts on AUX, at volume 0, muted, until the settings and the first knob reading are applied.

The S3 learns the source from the A32′s state message and shows AUX while the link is down (chapter 9 (p. 224)).

10.7 Bluetooth: who may connect, and when

10.7.1 The policy

Three rules have been in the code from the start:

- a phone that wanders into range must not latch on its own;
- a phone that has connected before may reconnect on demand, from the phone;
- the button is for devices that have not connected before.

They give exactly one arrangement:

Table 115. The policy

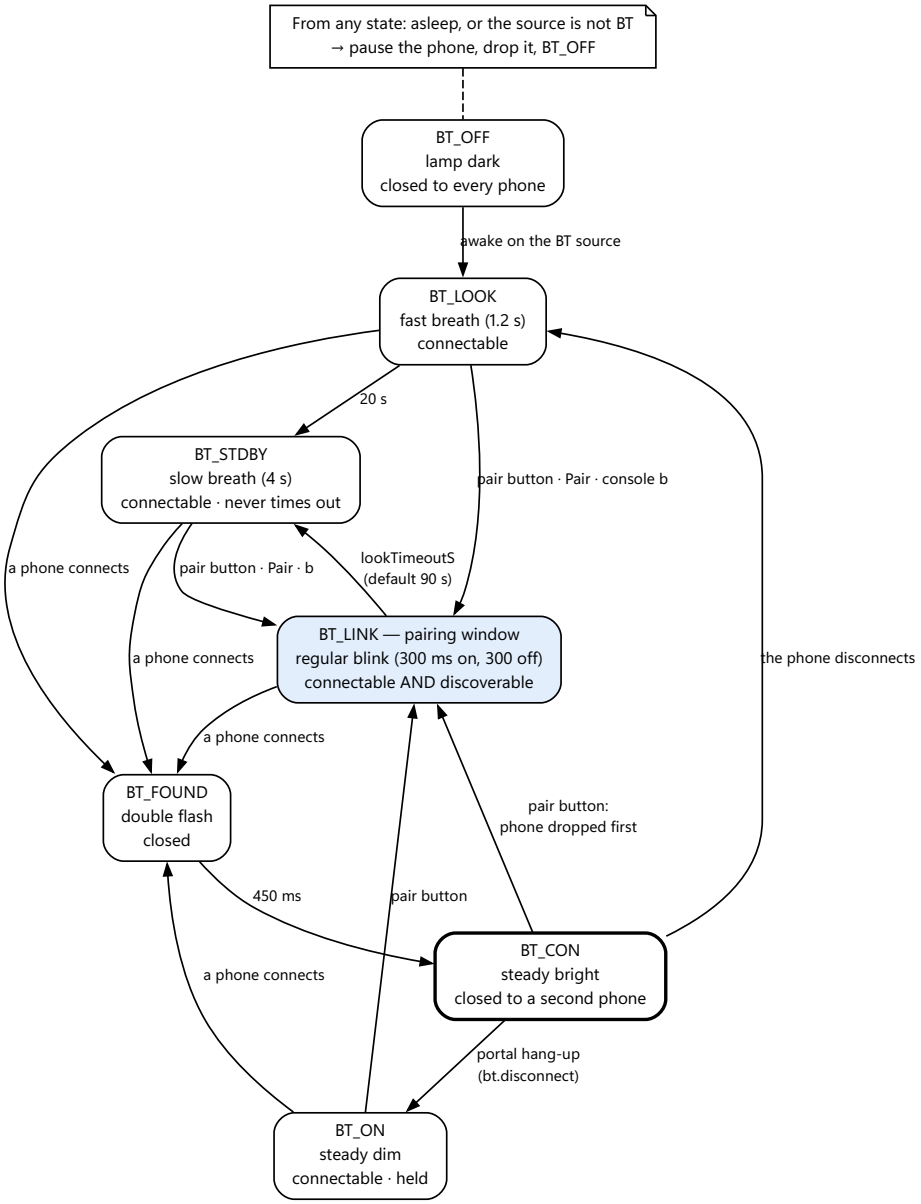
Situation	Connectable	Discoverable
Not on the BT source, or asleep	no	no
On BT, idle	yes	no
Pairing window open (after the button)	yes	yes, until the window closes
A phone is connected	no	no

All of it depends on being awake. Both chips are powered whenever the set is plugged in; without the awake gate, a phone could connect at 3 a.m. and quietly send its audio into a radio that is off. The radio never calls a phone: automatic reconnection is switched off in the library. No second phone can call in behind the first while one is connected.

In full, the A32 is connectable only when all of these hold: the setting `connectable` is on, it is awake, the source is BT, its link self-test passed, and no phone is connected. It is discoverable only when it is connectable and the pairing window is open. With `connectable` off, Bluetooth is closed entirely, the pairing window included.

10.7.2 The states

Figure 34. The Bluetooth states, what moves between them, and the lamp pattern of each



The state (`btState`) is published to the S3 and shown by the blue lamp.

Table 116. The states

State	Lamp	Entered when	Left when
BT_OFF	dark	asleep, or the source is not BT	awake on BT → BT_LOOK
BT_LOOK	fast breath	entering BT, or a connection ended	after 20 s → BT_STDBY
BT_STDBY	slow breath	from BT_LOOK, or when the pairing window closes	held; never times out
BT_LINK	regular blink	the pair button, the portal's Pair (bt.pair), A32 console b	after lookTimeoutS (default 90 s) → BT_STDBY
BT_FOUND	double flash	a phone connected	the lamp moves itself to BT_CON after 450 ms, and the published state follows
BT_CON	steady bright	from BT_FOUND	the phone disconnects → BT_LOOK
BT_ON	steady dim	portal hang-up (bt.disconnect)	held, like BT_STDBY

BT_LOOK, BT_STDBY and BT_ON have the same scan mode: connectable, not discoverable. Only BT_LINK is discoverable. "Looking" is a message from the lamp; the radio never sends a page to a phone.

10.7.3 Letting a phone go

Disconnect entirely, but pause first. Whenever the radio lets a phone go, it sends an AVRCP pause (if pauseOnLeave is on) and drops the phone 250 ms later, without blocking. The pause matters: tearing down A2DP under a phone that thinks it is playing left its media app "playing into nowhere", a real dead end seen on this radio. Leaving BT drops the phone entirely; staying connected while the selector was elsewhere gave "the phone showed connected, play appeared to work, and nothing came out".

A disconnect is only a request the library confirms it later, on the Bluetooth task. The A32 holds the requested state while it waits, so the pair button pressed with a phone connected ends in the pairing window, as it should. If the disconnect has not landed after 3 s, the A32 stops waiting and sends BT: the disconnect did not land - still connected to its USB console and to the S3. The lamp then shows connected.

Stray phones. If a phone is connected while the A32 is asleep or off the BT source, the A32 sends it away by the same pause-then-drop path, at most once every 3 s. The rule is "nothing connects at night", not merely "nothing can".

10.7.4 The pair button and the other controls

The pair button is on GPIO25, active low, with the pin's internal pull-up. No capacitor is drawn on it (Hardware Bible, chapter 9, section 9.2), so the debounce is the firmware's: a press counts on release, and only after the button was held for more than 30 ms. It acts only on the BT source. A press drops whoever is connected first, then opens the pairing window. A second phone must not pair behind the first and leave it unclear which one is in charge.

Portal actions (Appendix C (p. 328)): guest bt.play, bt.pause, bt.next, bt.prev, bt.disconnect (graceful, ends in BT_ON) and bt.pair administrator

`bt.forget`, which removes every stored pairing. `bt.pair` opens the window only on the BT source, but the portal answers "pairing window open" on any source (chapter 12 (p. 290)).

Sample rate. If a phone negotiates a rate other than 44 100 Hz, the A32 only warns on its USB console. There is no resampler, so such a stream plays at the wrong pitch.

10.7.5 Transmit power

At start-up, after Bluetooth starts and before the first scan mode lets the radio transmit (as Espressif's framework requires), the A32 sets the Bluetooth controller's power range: a floor of 0 dBm and a ceiling of `btTxLevel`, which defaults to 0 dBm. The stock range is 0 to +3 dBm. The floor stays at 0 dBm unless the ceiling is set lower, because a low floor lets the controller run near its margin and raises retransmissions.

The portal action `sys.bttx 0..5` (−12 to +3 dBm, down only) changes the ceiling; `sys.bttx 5` restores stock. It is not stored, so every start is at 0 dBm. The controller's own readback (`bttx`, `bttxmin`) is taken once, when the value is set. The ceiling was a pop mitigation: transmit bursts are the largest current step the board makes, and it was kept as it is.

10.7.6 The blue lamp

The lamp's pattern for each state is fixed, implemented as written. It is driven by LEDC channel 0 at 2 kHz, 8-bit.

Table 117. The blue lamp

State	Pattern
OFF	off, and actively driven off
ON	steady, 64/255
LOOK	breath 32 → 255 → 32 over 1200 ms
STDBY	breath 8 → 120 → 8 over 4000 ms
FOUND	150 ms on, 150 off, 150 on, then CON by itself
CON	steady, 180/255
LINK	300 ms on, 300 ms off, repeating

- Active low. The lamp is on GPIO33 with its anode on the supply, so the pin sinks its current (Hardware Bible, chapter 9, section 9.2). A brightness B is written as 255 − B.
- "Off" is driven high, not left as an input, because an input pin's leakage lights the LED faintly in a dark room.
- Breathing uses a raised cosine, because a linear ramp reads to the eye as a fast rise and a long dim tail.
- Each state has a scale of 0–255 (the settings `btLedOff` … `btLedLink`, default 255, "exactly as specified"). OFF ignores its scale.
- FOUND always resolves: the lamp moves itself to CON after 450 ms, and the published state is read back from the lamp, so the two share one clock.

- Full drive. At the default scale of 255 the lamp is driven fully on (the pin held low for the whole PWM period) for 300 ms of every 600 ms in LINK, at the top of every LOOK breath, and for both 150 ms flashes of FOUND. That is the specification, kept on purpose, and judged safe for this lamp's drive. A lower per-state scale lowers the drive.

10.8 Memory: the audio board's tightest limit

> NOTICE Do not spend RAM on the A32 without measuring first. The A32 has about 24 KB of heap really free. Asking for 32 KB more once made it impossible to update, and only a USB cable brought it back.

The figure. About 24 KB is really free: the Bluetooth library's own report of the free heap, printed on the A32′s USB console at start-up, read 23 620 bytes. It was measured once, with that day's build, and has not been re-measured since.

The incident. The DMA chain was once raised from 8 to 16 buffers, to ride out audio-task freezes of tens of milliseconds. The driver allocates the same chain for transmit and receive, so that cost another 32 KB of DMA-capable internal RAM, more than was free. Then:

- every update was refused: the updater returned false with error code 0, which means its own 4 KB allocation had failed, on a chip seconds out of reset;
- phones and a PC could not connect over Bluetooth;
- the chip crashed once.

There is no way to reset or reflash the A32 from the S3: no reset or boot-mode wire runs between them (Hardware Bible, chapter 4). Only a USB flash back to 8 buffers recovered it; an over-the-air update of the very same image, refused six times in a row before, was then accepted. The change had been shipped because the heap figure then in view read about 231 KB and "never moved" (231 748 before, 231 740 after). That was the wrong instrument (below). The conclusion drawn: "a radio that cannot be updated without opening the cabinet is worse than one that clicks when the audio task is starved."

The arithmetic. Each DMA buffer is 256 frames × 8 bytes = 2 KB.

Table 118. Memory: the audio board's tightest limit

Chain	Per direction	Unit	Both directions	Audio held
8 buffers (the firmware's)	16	KB	32 KB	46.4 ms per direction
16 buffers (the incident)	32	KB	64 KB, 32 KB more	twice as much

Every millisecond of chain depth costs about 0.35 KB per direction, about 0.7 KB per millisecond in total, whether it is bought with more buffers or longer ones (the driver caps one buffer at 4092 bytes). Longer buffers differ only in how the memory fragments, not in how much they take. The 32 KB Bluetooth ring is static memory and already inside the budget.

Which number to trust.

Table 119. Memory: the audio board's tightest limit

Number	What it is
The portal's `heap` field	The S3's own free heap. It says nothing about the A32; no A32 heap figure reaches the portal at all.
The A32 console `s` line	`ESP.getFreeHeap()`, the free internal heap in general, which is not what a plain allocation or a DMA allocation can get.
The Bluetooth library's "Available Heap" lines at start-up	The A32's `esp_get_free_heap_size()`. The only place it is printed.
The largest free DMA block	Printed by nothing today.

So the ~24 KB is the only recorded figure. To read the free heap yourself, connect a USB cable to the A32's own USB port (read the Danger in chapter 3 (p. 25), section 3.1 (p. 25) first) and restart it: the Bluetooth library prints its "Available Heap" lines while Bluetooth starts. The largest free DMA block needs a build that prints it. Section 10.15.2 says what to do before spending RAM.

10.9 What the audio board reports

Every 250 ms the A32 sends a state message to the S3, which shows it on the portal. The fields for this chapter:

Table 120. What the audio board reports

Portal key	Meaning
`under`	Bluetooth underruns (partial fills only) since the A32 started.
`ring`	The ring's high-water mark since the last read.
peaks, `clip`	The peak level per channel since the last read, in tenths of a dBFS; the clip flag.
`rmsl`, `rmsr`	RMS after the source gain, before the volume.
`zarm`, `zrel`, `zsince`, `zlong`, `zmute`, `zfloor`	The zero-data witness: entries, releases, time since the last release, longest mute, muted now, floor on.
`i2sraw`	The I2S fault latches, sticky since start-up.
`stalls`, `stallus`, `stlast`, `stlastus`	Audio-task stalls longer than 23.22 ms.
`praw`, `pmin`, `pmax`, `pjump`, `vsteps`, `vjump`, `vjms`, `gsteps`, `gmin`, `gmax`, `upot`	The volume watch: knob readings, their extremes and largest jump, volume steps, applied gain steps and range, knob in use.
`dindrv`, `sddly`, `tail`, `clkdrv`, `mclkdrv`, `bckdrv`, `lrckdrv`	Pad drives and registers, read back from the hardware. `clkdrv` = 255 means the three clock pads disagree.
`dctest`, `dcword`	The constant-DC diagnostic.
`a32ms`	The A32's uptime.
`bttx`, `bttxmin`	The Bluetooth transmit power range as the controller reported it.

Every readback comes from the hardware — the pad, the register, the controller — never from a copy of what the firmware thinks it wrote. That rule paid for itself once: a test script's portal session had expired, its commands were silently refused, and only the pad readback showed that the setting had never moved.

`i2sraw` = `0x092A` is normal. Most of its bits latch in normal running. On this radio it read `0x092A` (2346) at every read, across starts and through the heaviest popping on record: bits 1, 3, 5, 8 and 11, which are `tx_put_data`, `rx_rempty`, `tx_rempty`, `in_done` and `out_done`, most likely set at start-up. `rx_wfull` and `rx_rempty` are expected on BT and AUX, where nothing reads the receive chain and it overflows by construction. So a steady `i2sraw` is not a fault, and it gave no evidence of a transmit fault when pops happened. The A32′s console `s` decodes the bits by name.

Only the A32′s USB console shows the Bluetooth stack's heap figure, the `[ZDD]` lines the A32 prints when the witness sees a release or a stall, and the text of update errors.

10.10 Pops: what was found, and what is still open

Appendix H (p. 348) tells the hunt in full, with every test and number and the method for counting pops. This is the summary.

What a pop is. A loud, short click through the speakers, at irregular times, on RADIO and on BT, never on AUX. Any A32 volume other than 0 produces them.

What it is not. Each of these was excluded by a measurement: the Bluetooth branch (RADIO popped the same way); the S3, its WiFi and the link (the S3 powered off: no change); the firmware's arithmetic (the peak meter never above −118.4 dBFS through 60 s of popping); every linear stage in the firmware, the underrun click included (a phone volume a hair above zero brought the pops back at full loudness); the DAC's zero-data mute (`zrel` did not move through 8 minutes of pops); the volume reading (knob ignored, still popping); audio-task stalls (`stalls` frozen, `under` at 0 during pops); the DAC's clock-error mute (a streamed tone passed through 37 pops almost unchanged); the needle motor, the FM tuner chip, the 5 V supply, the tube set, the box fan, the earth, a mains filter, the room and the amplifier. Mains noise conducted on the same circuit is a contributor, not the cause.

What the pops look like. Pop sizes fell on a ladder of binary weights (¼, ½, ⅝, ¾, 1 and 3/2 of a main size), and the pops survive a constant word on the data line: they do not need the data to change, but a near-zero word never showed them. That reads as the DAC mis-reading a few of the top bits of single words, not as noise added in the analogue path.

What cut them: clock-pad drive 3. At start-up the firmware sets the MCLK (GPIO0), BCK (GPIO18) and LRCK (GPIO17) pads to drive strength 3, the strongest of the ESP32′s four, instead of the power-on 2.

Table 121. Pops: what was found, and what is still open

Clock-pad drive	Condition	Window	Unit	Pops
2	220 Hz tone over A2DP, one-minute alternation	936	s	52 (3.33/min)
3	same run, alternating windows	932	s	0 above −35 dBFS
3	tone at −30 dBFS over A2DP, floor on	45.1	min	1 (0.022/min)
3	constant word `0xFF000000`, A2DP streaming	90.0	min	0 (upper bound about 0.03/min)

The alternation's stratified exact test gave p = 2.8e-16, with all 15 pairs the same way. Drive 3 cut the pops about 150 times in the measured conditions.

What it is. A margin fix, not a root cause. The offered story is a hypothesis; no oscilloscope has ever seen an edge. It runs like this. GPIO0 drives two converter clock inputs down a harness stub. The DAC's fallback clock recovery is off while it has an external SCK. A stiffer edge stops a marginal sampling instant latching the wrong bit. At 354 ns per bit, the question is edge quality, not timing. Drive 3 says the DAC was latching wrong bits on a marginal edge, not why the edge was marginal. The drive setter cannot stop MCLK by accident: GPIO0 is an RTC-capable pad, and the framework writes only the pad's drive field; the pad's function select is a separate bit.

What it cost. At drive 3 the radio path was once heard as distorted, "like people lightly talking through kazoos"; at drive 2 it sounded right but the pops came back "like full-on popcorn". Both were reproduced by changing `sys.clkdrv` and nothing else. Since then drive 3 has been judged to work with no apparent consequences, and there is nothing to gain from drive 2. Drive 3 stays the default. `sys.clkdrv` stays in the firmware so that drive 2 against 3 can be tested again, or drive 2 put back, without a reflash.

Hardware changes along the way (Hardware Bible, chapters 4 and 6): the DAC's master-clock wiring was changed early in the hunt, and the pops went away for nine days; decoupling capacitors were fitted; ferrite cores and new audio cables went on later. After the ferrites, no pops were heard, even before the cable changes.

Still open.

- Drive 3 is a margin fix, not a root cause.
- A burst of 183 pops a minute (932 in 5.1 minutes, drive 2, RADIO, A32 volume 8), 55 times the earlier drive-2 rate: five conditions differed from the earlier runs and none was isolated. The decoupling capacitors fitted shortly before may be part of the cause.
- The "kazoo" distortion was judged by ear only.
- The A32 rebooted once during the hunt.
- No oscilloscope was used. The firmware cannot see a DAC-side clock error; no telemetry exists for it.
- Every measured rate is from the Bluetooth path, because that is what was streamed: a limit of the evidence, not a property of the fault.

- Two tests remain: driving MCLK and the BCK/LRCK pair apart (`sys.clkpair`), and a volume swing (A32 volume 47 against 94 with a tone streaming) to tell a fixed wrong code at the DAC from an error proportional to the sample.

10.11 When it goes wrong

The last column is the fault's class on the recovery ladder (NOTE, DEFECT, BLOCKER), set out in About this book.

Table 122. When it goes wrong

What you see	What happened	What the A32 does	What you do	Class
Updates refused (the updater's error 0), phones cannot connect	RAM spent past the ~24 KB free, for example 16 DMA buffers.	Refuses every update.	Reflash by USB; nothing else reaches it (section 10.8 (p. 275)).	BLOCKER (USB only)
Silence, or the radio at full volume	A future change writes the data-line output delay (or the BCK/LRCK out-delays) with audio live. Nothing in the current firmware can.	Nothing: it cannot see it. 1 or 3: silence; 2: full volume through the amplifier.	Turn the amplifier down, then make the change set the delay back to 0; only 0 plays correctly. The speakers are at risk (section 10.15.3).	BLOCKER (harm)
Wrong or missing sound after `sys.dindrv` 0 or 1	The data-line drive was set to 0 or 1.	The line may mis-decode for long stretches.	Set 2 or 3, or reboot the A32.	DEFECT
Silence from start-up; `[FAIL]` on the A32′s USB console	The I2S driver failed to install.	No audio task.	Reboot the A32; if it persists, reflash.	DEFECT
Silence; the link, the portal and the lamp work	The audio task is blocked for good (no known cause). The A32′s loop watchdog does not watch it.	Nothing.	Reboot the audio board from the portal.	DEFECT
A click; `stalls` rises	The audio task was starved for more than 46.4 ms.	The chain drains; zeros go out.	Nothing.	NOTE
Music at the wrong pitch	The phone negotiated a rate other than 44.1 kHz.	A USB warning only.	Nothing in the firmware; there is no resampler.	NOTE
Rare drops or underruns in a long stream	The phone's clock drifts against the local 44.1 kHz.	The ring slowly fills (drops) or empties (underruns).	Nothing.	NOTE
Silence, lamp dark, Bluetooth closed	The S3 was silent for more than 2 s.	Sleeps: mute, Bluetooth closed, forgets the amplifier state.	Nothing: it wakes when the S3 speaks and says the amplifier is on (chapter 9 (p. 224)).	NOTE (by design)
The volume does not follow the knob's travel	The stored knob calibration is implausible (maximum no more than 64 counts above the minimum).	Falls back to 60 / 3990, no centre.	Recalibrate (section 10.5.1 (p. 267)).	BLOCKER (guarded: the knob runs on 60 / 3990)
The knob maps wrongly	The stored calibration is plausible but wrong.	Nothing.	Recalibrate.	BLOCKER

What you see	What happened	What the A32 does	What you do	Class
`BT: the disconnect did not land - still connected` the lamp shows connected	A disconnect request never landed within 3 s.	Warns on USB and to the S3.	Nothing.	NOTE
A phone connects while the radio is off or on another source, then drops	A phone connected while asleep or off BT.	Pauses and drops it, retried every 3 s.	Nothing.	NOTE
Silent, invisible to phones; `proto self-test FAILED - muted, BT closed, awaiting OTA` every 10 s	The firmware's link self-test failed at start-up.	Muted, Bluetooth closed; the link and the update receiver stay up; never confirmed.	If it arrived over the air: Reboot the audio board, which rolls it back (an update is refused while it is on trial). If flashed by USB: update to a good program (chapter 3 (p. 25), section 3.11 (p. 32)).	DEFECT
Silence for up to 20 s after an update stopped mid-transfer	No frame from the S3 for 20 s.	Gives up and restores the user's mute.	Nothing.	NOTE
Every A32 update starts with a click	`fadeOutMs` is above about 340 ms.	Nothing.	Set it lower (chapter 3 (p. 25), section 3.10.3 (p. 30)).	NOTE
The pair button blinks the lamp, but no phone finds the radio	`connectable` is off.	Bluetooth fully closed.	Turn it on.	NOTE (by design)
Audio settings back at their defaults after an update	The stored settings had another size.	Defaults apply.	Set them again from the portal (chapter 7 (p. 153), section 7.4 (p. 161)).	BLOCKER by design; avoidable by downloading the settings file before the update (chapter 7 (p. 153), sections 7.5.5 (p. 167) and 7.6.4)
`settings NOT saved - the flash write failed; retrying`	A settings write to flash failed.	Keeps the change unsaved, warns, retries 2 s later.	Nothing.	NOTE; never exercised on the radio
A change made during the A32's trial is gone after a restart	The restart rolled the program back.	Held the change in RAM.	Set it again (chapter 3 (p. 25), section 3.11 (p. 32)).	NOTE (by design)

What you see	What happened	What the A32 does	What you do	Class
`the audio board is not answering - its settings cannot be changed now` (HTTP 409)	A portal edit of an audio row while the A32 is silent.	—	Wait for the handshake, then edit again (chapter 7 (p. 153), section 7.4.5 (p. 164)).	NOTE; never exercised on the radio
A pop	Nothing the firmware can see.	Nothing.	Count them (Appendix H (p. 348)); see section 10.10 (p. 277).	NOTE; root cause open

10.12 Design choices

- I2S is installed once, full duplex, and never reconfigured; the Bluetooth library never reaches I2S. The two paths need incompatible drivers, and reinstalling under the Bluetooth stack risks hard faults. The library leaked into I2S through its sample-rate call and into the samples through its volume control, so both doors are shut by empty objects rather than flags.
- APLL on, a fixed 256 × 44 100 Hz MCLK on GPIO0, and GPIO0 never detached from I2S. Both converters take their master clock from GPIO0, and only the APLL makes 11.2896 MHz exactly.
- The radio's two channels are summed to mono. Both carry the same signal; the sum gains about 3 dB of signal-to-noise.
- The right channel goes in slot 0. On this machine that order gave correct stereo; another build must check it (section 10.3.4 (p. 261)).
- Fades are asymmetric, 150 ms out and 1000 ms in, on a squared per-sample Q24 envelope. Leaving a source is a deliberate act; arriving should be gentle. A linear ramp does not sound like a fade.
- The volume curve is a portal setting, gamma 2.5, and the knob is calibrated at three points. The right curve depends on the amplifier and can only be settled by ear. The knob's raw reading is not proportional to rotation, so no curve applied afterwards could fix it; a centre point does.
- The knob is a sensor: the first reading applies, and the volume is not stored. The knob's position is the stored state; coming up at a stored volume the knob disagrees with repeats the silent-boot failure.
- The knob calibration lives on the A32, because the A32 takes the readings: the reading and the thing being calibrated must not sit on opposite sides of a link.
- Radio gain +8 dB, Bluetooth gain −7 dB. Measured with the RMS meter, so both sources are equally loud at every knob position and the radio does not clip before the volume.
- Mute is software only, computed from state, never stored. No pin reaches the DAC's mute input; a mute toggled at events was undone by other code paths; a stored mute booted the set silent.
- Bluetooth is invisible unless awake and on BT; the radio never initiates; only the firmware's own policy writes the scan mode. Those are the three rules, and "nothing connects at night".

- The Bluetooth library is pinned to one commit (chapter 3 (p. 25), section 3.16 (p. 41)). The visibility rule depends on the library's internals, and an unpinned library changes under the build without warning.
- Pause the phone, then disconnect, whenever the radio lets a phone go. A phone torn down mid-play leaves its app playing into nowhere.
- The pair button drops the connected phone first. Otherwise a second phone can pair behind the first and it is no longer obvious which one is in charge.
- The blue lamp is driven fully on in LINK, at the top of LOOK and in FOUND, at the default scale of 255 (section 10.7.6 (p. 274)).
- A Bluetooth transmit ceiling of 0 dBm at start-up. A pop mitigation: transmit bursts are the largest current step the board makes.
- Clock pads at drive 3 from start-up. About 150 times fewer pops in the measured conditions; drive 3 works with no apparent consequences, and there is nothing to gain from drive 2.
- The DMA chain stays at 8 buffers. 16 buffers took 32 KB the chip did not have; updates and Bluetooth failed.
- Nothing in the firmware writes the I2S transmit output delays only the readback `sddly` remains. Why: written with audio live, the data-line delay sent the radio to full volume through the valve amplifier at position 2 and silenced it at 1 and 3; only 0 plays correctly, and the firmware cannot detect a wrong one. The full danger warning is in section 10.15.3.
- Experiments are actions, never stored, and the remote DC word is chosen by index only. A stored experiment gets its result blamed on something else later; nothing louder than −42 dBFS can be sent remotely (Appendix G (p. 344)).
- The A32 sleeps when the amplifier is off, and S3 silence counts as amplifier off (chapter 9 (p. 224)).
- A failed update restores the user's mute, not "unmuted" (chapter 3 (p. 25), section 3.10.3 (p. 30)).
- The retired **`autoConnect`** byte stays in the stored Bluetooth structure, unread; its portal row is gone. It did nothing, since the radio never calls a phone, and removing it would change the structure's size, which the A32 rejects (Appendix A (p. 309)).

10.13 Tried and rejected

FOR FIRMWARE CHANGES

These were dead ends on this radio. They may not be dead ends on yours.

Table 123. Tried and rejected

Approach	The idea	What happened here	What is left in the firmware
Letting the Bluetooth library own I2S	the library's normal way	it reconfigured the driver the moment a phone streamed: full scale, garbled, channels swapped, radio dead	the three guards (section 10.3.3 (p. 260))
Reinstalling the driver at each source change	one driver per path	the kind of fault that crashes the chip in ways hard to trace	installed once
The integer clock divider	MCLK from the 160 MHz clock	cannot make 44 100 Hz exactly	APLL on
Underrun theory	the pop is the ring's zero-fill click	pops stayed full size at a tiny phone volume; a linear stage cannot do that	underrun counter kept (partial fills only)
WiFi or coexistence theory	the S3's WiFi disturbs Bluetooth	S3 powered off: no change	—
Draining the receive half on every source	read the idle receive chain on BT and AUX so it cannot disturb transmit	RADIO, which reads it, popped the same	receive free-runs on BT and AUX by design
ADC master-clock toggle	detach GPIO0 from I2S to silence the ADC	a hazard once the DAC also took its clock from GPIO0; it also had two bugs: a null pin table that selected the built-in DAC pins, so the toggle worked exactly once, and a floating clock input	removed; its message is ignored. To silence the ADC, mute its samples
Data-line framing	activity at the word boundary makes the DAC lose framing	the DAC's fallback clock recovery it blamed was switched off by the clock-wiring change; the pops later came back on constant words	superseded
Output low-bit mask	zero the bottom bits so the data line is still at the boundary	never confirmed by ear; a `k` sweep fixed nothing on RADIO; widths 2 to 15 bits never tested one by one; it cannot touch the next word's sign bit that opens every slot	off; A32 console `k` only
Hardware mute held high on GPIO16	stop a floating mute input	no change; the pin reached nothing	removed
Weaker data-line drive	slow the "aggressor" edge	no result recorded that it helped; 0 and 1 risk mis-decoding	`sys.dindrv`
Data-line output delay (and the BCK/LRCK out-delays)	walk the data edge off the frame clock; once written up as a "free" knob	only 0 plays correctly (section 10.15.3). Keeping it on the USB console behind a warning was rejected too	nothing writes it; readback `sddly` only
Zero-data mute release	the DAC's own analogue mute releases with a step	`zrel` did not move through 8 minutes of pops	witness kept; floor off, `sys.zfloor`
Volume-step theory	block-boundary gain steps from a jittery knob reading	knob ignored, `gsteps` frozen, still popped	volume watch kept
Sign-extended tail	the data line never moves across a word boundary	blind on-off-on-off test: no change in rate or size	off, `sys.tail`
DAC clock-error mute	pops are the DAC muting on a clock error	a streamed tone passed through 37 pops almost unchanged (median −0.3 dB)	—

Approach	The idea	What happened here	What is left in the firmware
One fixed-size sign-bit flip	every pop is the same flip	20 of 31 pops had the opposite polarity ($p \approx 0.15$); sizes spread over 19.1 dB	—
Stall–Bluetooth link	stalls are caused by Bluetooth	13 stalls in about 33 minutes with Bluetooth idle	stall counter kept
Deeper DMA chain (16 buffers)	ride out audio-task stalls	broke updates and Bluetooth (section 10.8 (p. 275))	back to 8. Longer buffers, or a count raised only as far as the largest free DMA block allows, were named but never tried; both cost the same RAM per millisecond
DAC system clock grounded, internal PLL from BCK (TI's advice for clicks on a sibling DAC)	let the DAC make its own clock	that was this radio's wiring before the clock change, when the pops were harder; not re-tried	—
Clock drive split (`sys.clkpair 23`)	MCLK at 2, BCK/LRCK at 3, to keep both good halves	never run; drive 3 was kept	`sys.clkpair`
Clock-pad drive 2	the power-on default	about 150 times more pops in the measured conditions	`sys.clkdrv` can put it back
Storing the mute	restore the user's mute at start-up	booted the set silent, with nothing on the front to say why	mute forced off at start-up
The first fade ramp, in Q16	advanced per sample	finished in 0.18 ms: a hard cut heard as a jump-scare	squared Q24 envelope
A fixed logarithmic volume law	one curve for every amplifier	the right curve depends on the amplifier	gamma setting
Storing the volume	come up at the last volume	repeats the silent-boot failure	the knob is the state
+15 dB radio gain	more radio level	clipped at the gain stage, at every volume	+8 dB
Matching radio and Bluetooth by peaks	equal peaks	FM is limited hard and music is not	RMS meter
Letting the library write the scan mode	the library's normal way	the radio sat connectable and discoverable on RADIO, on AUX and asleep after every hang-up, source change or sleep	policy override (section 10.15.5)
Staying connected outside BT	keep the phone across source changes	"the phone showed connected, play appeared to work, and nothing came out"	pause, then drop
Capping the lamp's drive below full	a gentler drive	rejected; the specification is kept	per-state scale

The constant-DC diagnostic is kept on purpose: `0xFF000000` is the proven popper and `0x00FF0000` its matched silent control.

10.14 Known limits

Seen working on the radio: drive 3 at start-up, read back as 3 from all three pads after a cold power cycle.

Never run on the real radio:

- the channel order from the A32 and the DAC (a left/right test track over Bluetooth);
- Bluetooth invisibility on RADIO, checked with a phone; it must also be run after any library change;
- an A32 settings write that fails, and its retry;
- the portal's refusal of an audio-board edit while the A32 is silent;
- `sys.clkpair`
- the Bluetooth transmit-power A/B: the 0 dBm start-up ceiling is unproven;
- a drive-3 measurement with Bluetooth idle;
- the volume-swing test (section 10.10 (p. 277));
- a phone negotiating 48 kHz;
- `connectable` off;
- a long session measuring clock drift between a phone and the local 44.1 kHz (there is no rate adaptation; the 186 ms ring absorbs only jitter).

Limits by design.

1. No resampler, no equaliser. The path is bit-exact apart from gain, volume, balance and the fades. A digital equaliser on the A32 could only ever cover RADIO and BT, because AUX never passes through the A32; the one point that catches all three sources is analogue, after the amplifier's summing (chapter 12 (p. 290)).
2. The audio task is not watched by the A32's loop watchdog. A wedged audio task leaves the loop, the link and the portal alive, so Reboot the audio board fixes it.
3. The I2S clocks run even while asleep. They may be the source of a fixed comb of spurs the FM tuner chip sees at multiples of 2.8224 MHz (harmonics 28 to 34 match within one sweep step; chapter 12 (p. 290)).
4. No A32 heap figure reaches the portal, and no key prints the reliable one on demand (section 10.8 (p. 275)).
5. **`sys.bttx`** resets at every start, like every experiment, so the start-up value is always 0 dBm.
6. The **`autoConnect`** byte is kept and read by nothing.
7. A reboot or an update of the A32 makes one pop of its own.
8. The stall click. The audio task is sometimes frozen for 23 to 111 ms. A flash write with the cache off is the leading guess. It is not fixed, because a deeper chain costs RAM the chip does not have (section 10.8 (p. 275)).
9. The blue lamp is driven fully on for 300 ms of every 600 ms in LINK, at the top of every LOOK breath and in both FOUND flashes.

Small known issues, left as they are (chapter 12 (p. 290) has the details):

1. The update's mute wait is a fixed 400 ms a fade-out above about 340 ms makes every update start with a click (chapter 3 (p. 25), section 3.10.3 (p. 30)).
2. The portal's **`bt.pair`** sends a message named `MSG_BT_LOOK`, but opens the pairing window (`BT_LINK`). It opens it only on BT, yet the portal answers "pairing window open" on any source.
3. The A32's console **`s`** reads and resets the peak meter (with its clip flag) and the ring high-water mark, so the portal misses those values for one period. It also clears the I2S latches, but the portal reads `i2sSticky`, which nothing clears. Prefer the portal for those values.

10.15 Changing it safely

FOR FIRMWARE CHANGES

10.15.1 What must stay true

1. Never reinstall, reconfigure, **`i2s_set_clk`**, **`i2s_stop`** or re-pin I2S after **`Audio::begin()`**.
2. Keep both **`set_output(nullOut)`** and **`set_volume_control(&noVol)`**, and the stream-reader flag.
3. Never detach GPIO0 from the I2S peripheral. Never turn the APLL off.
4. The audio task never delays, never prints, and blocks only in the I2S calls. **`pushBt()`** never blocks.
5. Mute only through **`applyMute()`**, which sets `Audio::setMute(protoBroken || !awake || cfgA.muted)`. The start of an update is the one deliberate exception.
6. The scan mode is written only by **`applyScanMode()`**. Never call the library's `set_discoverability()`, which writes the mode directly.
7. Do not change the ESP32-A2DP pin without re-testing invisibility with a phone: on RADIO, on AUX, asleep, after a portal hang-up and after a source change, check that the radio does not appear in the phone's list of new devices. Check first that every library path that writes the scan mode still goes through the virtual `set_scan_mode_connectable()` (section 10.15.5).
8. Do not update the platform (chapter 3 (p. 25), section 3.19 (p. 44)).
9. Slot 0 carries the right channel on this machine. On another build, play a left/right test track before trusting it.
10. New **`ProtoState`** fields go at the end, so a version mismatch truncates rather than shifts. `PROTO_VERSION` is deliberately not raised for such an append, because a raised version loses the handshake, and with it the update relay to the A32 (chapter 9 (p. 224)). New message ids are backward compatible.
11. Readbacks come from the hardware, never from a copy.
12. Read-and-reset counters (`getPeaks()`, `getRms()`, `btRingFill()`) should have one reader: the 250 ms state message. Today the console `s` also reads `getPeaks()` and `btRingFill()`. `i2sFaults()` is

read and cleared by the console `s` alone; the state message sends `i2sSticky` instead. `zeroWatch()`, `gainWatch()` and `i2sSticky` are monotonic and safe to read anywhere.

13. **`i2sFaults()`** clears only the bits the driver does not service (masked against `int_ena`). An earlier version cleared everything and could lose the driver's buffer-done events, blocking the audio task forever.

10.15.2 The heap and the update wait

Before spending any RAM on the A32, read `esp_get_free_heap_size()` and `heap_caps_get_largest_free_block(MALLOC_CAP_DMA)` on the A32's USB console. No key prints them today (console `s` prints only `ESP.getFreeHeap()` the library's start-up lines give only the first), so add the print in the same build, and do not trust the ~24 KB for a build that has grown since. Budget about 0.7 KB of DMA RAM per millisecond of chain depth. A +32 KB request was fatal once and only USB recovered it (section 10.8 (p. 275)).

The update wait. The A32 mutes and waits a fixed 400 ms before it writes its flash (chapter 3 (p. 25), section 3.10.3 (p. 30)), because writing flash freezes code that is not in IRAM, the audio task included. If the fade and the DMA chain have not finished by then, the driver swaps in zeros mid-chain: a level step at the DAC, a click. The budget is at most 5.8 ms to see the mute, plus `fadeOutMs` (150 ms by default), plus the chain (46.4 ms): about 202 ms. The wait covers a fade-out of up to about 340 ms; `fadeOutMs` can be set up to 5000 ms. If you raise **`fadeOutMs`** above about 340 ms, or deepen the DMA chain, raise the 400 ms wait in **`OTA_BEGIN`** to match.

10.15.3 Hazards

> ⚠ WARNING Never write the I2S transmit output delays with audio live: one position sends the radio to full volume. The register `tx_sd_out_delay` shifts the data line in time. Swept live on this radio, position 2 sent it to full volume through the valve amplifier; positions 1 and 3 silenced it; only 0 plays correctly. Every I2S transmit out-delay — `tx_sd_out_delay`, `tx_ws_out_delay` and `tx_bck_out_delay` — mis-frames the DAC, and the firmware cannot detect it. The speakers survived, but they are at risk. Nothing in the current firmware writes any of them: the portal action and the A32 console key that once did were removed, and only the readback `sddly` remains. If one is ever needed again: amplifier down, mute, set, then un-mute at a known low level.

- Data-line drive 0 or 1, or any clock drive below 2, can mis-clock for long stretches. The clock setter clamps to 2–3; the data-line setter

(`sys.dindrv`) does not. Use it with someone present and the amplifier down.
- Verify a flash by something only the new build can say (its version string, the A32′s uptime resetting), not by a value the previous build already published.

10.15.4 Traps

- The volume gain steps at block boundaries; it is not ramped.
- `muteOnChange` off switches sources with no ramp: a thump.
- `setFades(0, …)` becomes 1 ms, on purpose.
- Balance ±100 is about −65 dB, not silence.
- The zero-data witness counts the DAC's idea of zero (the top 24 bits), not the firmware's.
- With the sign-extended tail on, a word of 1 to 255 followed by a non-negative word becomes exact zero. It matters only if the tail is ever turned on.

10.15.5 How the scan-mode policy is enforced

`applyScanMode(force)` is the only function that writes the scan mode:

```
connectable  = cfgB.connectable && awake && source == BT
&& !protoBroken && !btConnected
discoverable = connectable && btState == BT_LINK
```

It remembers what it last wrote and returns early unless the result changed or `force` is set.

The library writes the scan mode by itself, on the Bluetooth task, at the pinned version: it calls `set_scan_mode_connectable(true)` on every disconnect and once more when the stack comes up, and `set_scan_mode_connectable(false)` on every connect. Its "true" means connectable and generally discoverable. The stack-up call is only queued when Bluetooth starts, so it lands after the firmware has applied its own mode. Because `applyScanMode()` had cached "invisible" and none of those writes changed the policy, the radio once sat connectable and discoverable on RADIO, on AUX and asleep after every hang-up, source change or sleep.

The fix is a subclass. `PolicyA2dpSink` overrides the library's virtual `set_scan_mode_connectable(bool)` so that it only sets `scanDirty = true`. The main loop then calls `applyScanMode(true)` after `updateBtState()`, which re-derives the mode from the firmware's own state and writes it past the cache. The flag is cleared before the write, so a change that lands during the write is caught on the next pass. `scanDirty` starts true, so the start-up write happens in any case. `onConnState()` sets it again after updating `btConnected`, so at least one pass sees the new connection state. This depends on the library's

internals — that `set_scan_mode_connectable` is virtual and that every scan-mode write goes through it — which is why the library is pinned.

Bluetooth timing races, already handled; do not undo them.

- `btDropping` is set before `a2dp.disconnect()`, because the callback that clears it runs on another task and could otherwise land first. While it is set, the connected branch of `updateBtState()` holds the requested state instead of promoting it back to `BT_FOUND`. `onConnState()` clears it when the disconnect lands; after `BT_DROP_TIMEOUT_MS` (3 s) `updateBtState()` clears it and warns. A stale flag is swept first thing in `updateBtState()`.
- `scanDirty` is cleared before the write; `onConnState()` sets it again after `btConnected`.
- The wake edge cancels a pending sleep disconnect (otherwise the phone is dropped 250 ms after waking and the lamp goes dark while awake) and clears the stray-phone timer (`strayDropAt`).
- `updateBtState()` reads `BtLed::state()` back to publish the lamp's own move from `BT_FOUND` to `BT_CON`.

10.15.6 Where things are

Table 124. Where things are

What	Where
I2S set-up, the clock-pad drive, the audio task, the ring, meters, witnesses, diagnostics	`src/a32/audio.cpp`, `src/a32/audio.h` (`Audio::begin()`, `audioTask()`, `pushBt()`, `popBt()`)
The blue lamp	`src/a32/btled.cpp`, `src/a32/btled.h` (`BtLed::update()`)
The three guards, Bluetooth states, scan mode, disconnects, sleep and wake, source, knob, mute	`src/a32/main.cpp` (`setup()`, `updateBtState()`, `applyScanMode()`, `gracefulDisconnect()`, `serviceDisconnect()`, `updateSource()`, `readSource()`, `updateVolumePot()`, `potRotation()`, `applyMute()`, `applyBtTxPower()`)
Selector thresholds and pins	`include/pins.h`
State message and timing constants	`include/proto.h`

How to count pops, if they come back, is in Appendix H (p. 348).

11. Build scripts

Only two tools ship with the firmware, both build scripts that run before every compile (chapter 3 (p. 25), section 3.1 (p. 25)). They matter only to someone building or changing the firmware.

FOR FIRMWARE CHANGES

Table 125. Build scripts

Script	What it does
`scripts/page.py`	Checks the portal page's script, compresses the page and embeds it in the main board's build.
`scripts/version.py`	Stamps every build of both boards with its version and its commit.

12. Known issues, ideas and untested paths

No firmware is finished; this one has a list of things that were seen, understood and left as they are. Only one of them stops the radio from working: an audio-board program built with the wrong protocol version (section 12.5 (p. 297)). Each is written down so that whoever meets it does not have to rediscover it, and so that whoever wants to fix it knows where to start.

The chapter opens with an index: find what you see, and it sends you to the section that explains it, in this chapter or another. Then come the issues left as they are, grouped by area, one row each: what you see, why it was left, and where a fix would start. The "where a fix starts" column names functions and files; only someone changing the firmware needs it. A starting point is a suggestion, not a design; none has been tried.

The class in the last column is the recovery ladder of About this book:

- NOTE heals by itself or is only cosmetic.
- DEFECT needs a portal reboot, a console reset or the front switch.
- BLOCKER leaves the radio unusable through a power cycle, or reachable only by the USB cable.

Two issues here are BLOCKERs: that program (section 12.5 (p. 297)) and, at worst, a blank board's access point, if nothing but the USB console helps (section 12.4 (p. 295)). The chapter ends with ideas for improvement, the paths never exercised on the radio with a way to test each, and a tidy list for the code.

Everything here was checked against the main board's firmware v.1.0.5 and the audio board's v.1.0.4. The placeholder portal account, the open rescue access point and the default time zone are not faults but things a builder sets: see the builder checklist, chapter 8 (p. 186), section 8.2.1 (p. 187).

12.1 Index by symptom

Find what you see; the second column gives the section that explains it (5.19 is chapter 5 (p. 73), section 5.19; Appendix H (p. 348) is the pop hunt).

Table 126. Index by symptom

What you see	Where
The needle	

What you see	Where
NEEDLE OFF, NEEDLE RE-INDEXING, NEEDLE FAULT, NEEDLE HUNTING	1.5, 5.19
The needle holds still while you tune	5.13, 6.12
The needle pinned at one end; the first search at power-up goes up and away; `*** LEDC REFUSED ***`	5.19
The needle frozen and the clock dark	4.8, 5.19
Re-index answers “re-indexing” but homes; steps lost after raising `upVmax` no warning for a gain that will hunt	12.3
A portal jog stutters	12.4
Tuning and the dial	
A measurement REFUSED, DISCARDED, ABANDONED or NOTHING FOUND; QUADRATIC or MARKS REFUSED; “tuner ends NOT measured”	6.12
Every reading off by the same amount; one sample with a large residual in `s`	6.12
The top of the dial (107.9 MHz) cannot be measured	6.15, 12.2
Samples stored up to 50 kHz high; the fine-pass list shows a rejected broadcast; a garbled status line or a mixed sweep dump; the gap message names the tuner’s ends	12.2
Drop one sample removed a hand mark	12.4
The clock and the display	
The clock blank or `--:--` a battery-clock pill; the old time back after `W`	1.10, 4.13, 9.10
The clock blank with a working network, the audio board absent since start-up	12.5
Eastern North American time after `W`	4.9, 8.2.1
The wrong brightness after `-` or `= 0879` a frequency shown while the angle sensor is dead; 3.3 s before anything starts	4.16, 12.7
The battery clock wrong from the year 2100	12.5
Sound and Bluetooth	
Pops	10.10, Appendix H (p. 348)
A click at the start of every audio-board update; a click with `stalls` rising; silence while the link and the blue lamp work	10.11, 12.6
Full volume or silence after a change to the audio interface	10.11, 10.15.3
Wrong pitch; rare drops; the volume does not follow the knob; a phone dropped; no phone finds the radio	10.11
“pairing window open”, but nothing opens	12.5
Updates and phones refused (memory)	10.8, 10.11
Peak or ring readings missing on the portal after the audio board’s `s`	12.6
One pop at every audio-board restart or update	10.14
The network, the portal and the console	
Off the house network, `Ambersong` appears; a phone joins but no page opens	1.4, 8.11
A blank board’s access point cannot be joined; the portal slow while the needle moves	8.11, 12.4
The portal dead but the radio answers a ping; sign-in refused with a wait; the password lost	8.11
The portal dead while the radio, clock and needle work	4.13
The Network card’s save rejoins; the access point confirmed only on the console; the reboot toast says only “rebooting”; an upload’s count hidden; no “unsaved” sign	12.4
`one key at a time ...`, `console busy` the display freezes and the sound mutes while a prompt waits	8.11
`y` shows the password; `i` says `*** LOW ...` the first `B` gives 11 dBm; `D` cannot be pasted into a file	12.4

What you see	Where
`i` says `OWN ACCESS POINT` while joining; `B`, `o` or `O` say “saved” too early; `q` or `v` refused with no reason	12.8
Updates and versions	
An upload refused (wrong chip, on trial, audio board not answering); the page says it failed, but it worked	3.15
“flash” on the page for a main-board file for the wrong chip; “No Error” after a refusal	12.4
`– an earlier update was ROLLED BACK` at every start-up; the sound off 20 s after a relay stopped	3.15, 12.5
`proto self-test FAILED – muted, BT closed, awaiting OTA` silence and no handshake after an audio-board update	9.10, 12.5
The audio board’s version on no portal page	12.5
The Firmware card says the needle “parks”; two builds cannot be ordered by stamp	12.8, 12.7
Settings	
SETTINGS LOCKED a setting back at its old value after a power cut; the fixed-feature list empty	1.11, 7.7
Audio settings back at their defaults after an update	7.7, 10.11
`the audio board is not answering ...` (HTTP 409); `n/a` in a download	7.7, 9.10
An audio edit lost just before the audio board fell silent; an audio-board console change undone	12.5
“TUNING MARKS REFUSED - the file announced 0 and 0 parsed”, or no word about the marks	12.4
The link between the boards	
`[WARN] A32 silent. link SILENT` AUDIO BOARD SENDS NO STATE asleep for up to 80 s; reset reason 6	1.8, 9.10
`[WARN] A32 silent` and the link pill red, yet the sound plays on	9.10, 12.5
BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both	9.10, 12.5
The audio board says `crc ok`, `[PASS] clock pads`, or `[ZDD] DAC left analogue mute` its help line omits `k` and `g`	12.8, 12.5

12.2 Tuning

Issues in the dial calibration and the tuner chip.

Table 127. Tuning

What you see	Why it was left	Where a fix starts	Class
The top of the dial cannot be measured. The firmware sweeps the oscillator only between 77.2 and 97.2 MHz. At the default IF of 10.60 MHz, 107.9 MHz needs it at 97.3 MHz, one step outside, so a measurement there finds nothing or only the edge of the peak. Every station up to 107.8 MHz is inside.	The highest station receivable where this radio lives is 107.3 MHz, and the dial reads true up to there.	The range is compiled (`Rda::LO_WINDOW_LO10`, `LO_WINDOW_HI10` in `src/s3/rda.h`), from the old nominal IF of 10.7 MHz. Derive it from the dial's printed ends minus the IF setting. Then the fixed features just above 97.2 MHz (98.7, 101.6 and 104.4 MHz) must be learned like the others, and the start-up purge of out-of-range fixed features (`purgeCorruptSpurs()`) must follow the new range. `requestSweep()` clamps every sweep to the range; `requestRefine()` checks only the centre; `sampleStart()` refuses a position well outside.	NOTE
Readings shared between tasks without a lock. A status line read mid-rewrite mixes old and new text; a sweep dump (`a`, `/api/rda`) taken during a sweep mixes two sweeps. The fixed-feature list can be replaced while a sweep reads it: never seen; at worst one sweep judges one frequency against the old or the new list.	Cosmetic or theoretical, and a lock would add a new way for the tuner task and the portal to wait on each other.	One small mutex for the tuner task, or copy on write: build the status text and the bins privately, publish them with one index or pointer swap. For the fixed-feature list, make the count and the array `volatile` and publish with an atomic store, as `pushBt()` in `src/a32/audio.cpp` publishes its ring head.	NOTE
The gap message names the tuner's ends ("AS5600 gap - angle re-seated on the turn inside the tuner's ends") even when the ends are not measured and the nearest turn was used.	Wording only.	Have `seatTurn()` in `src/s3/needle.cpp` report which rule it used, and print "on the nearest turn" when the ends are not measured, or the shaft lies outside them.	NOTE
Samples are stored to 100 kHz. The fine pass measures in 50 kHz steps, but a sample is stored in tenths of a MHz, rounded up: an odd-twentieth result is stored 50 kHz high (an oscillator at 95.05 or at 95.10 MHz stores 105.7 MHz either way, with the default IF). About +25 kHz on average.	At most half a channel step, and the dial reads true.	Store `tuneF` in twentieths and fit the curve on those: a settings version bump, with a conversion that doubles every stored mark (chapter 7 (p. 153), section 7.5 (p. 164)). The rounding is the `station10` line of `sampleStore()`.	NOTE
The fine-pass list does not mark a broadcast. A point rejected as a broadcast keeps its reading, under the legend "fine pass (50 kHz; 0 = skipped)", and looks like a candidate that lost. The `fine` list in `/api/rda` has the same gap.	The choice itself is right; only the printout is incomplete.	Keep a stereo flag per fine point, as the coarse bins do (`binStereo()`), and print "(STEREO - a broadcast)" after such a point; or complete the legend.	NOTE
Fixed features appear at multiples of 2.8224 MHz. One suspect, the audio board's I2S clocks, which run even while it sleeps, has never been tested.	The notches work; the comb costs only part of the window (chapter 6 (p. 120), section 6.15 (p. 149)).	Add an experiment action, reset at start-up like the others, that stops the I2S peripheral for a set time (about 90 s) and restarts it. Sweep inside that time and compare the `a` list with one taken with the clocks running. I2S is set up in `Audio::begin()` and fed by `audioTask()`.	NOTE

12.3 Needle

Issues in the needle's motion and its portal buttons.

Table 128. Needle

What you see	Why it was left	Where a fix starts	Class
A direct write of the position can race a pending correction. Never seen; a hypothesis from reading the code. The step task owns the position and folds corrections in at the top of every pass, but the index calibration's start and finish, the microstep change, the coil release, the jog, the low-stop assumption and the start-up position still write it directly. A correction landing during such a write would be lost, and the needle would run in a frame off by a few to a few hundred half-steps.	Rare, never seen, and it corrects itself: the next index crossing, the idle check or a re-index finds the error.	Make the step task the only writer: add a "set position to" request that `stepTask()` applies at the top of its pass, next to the fold. Simpler: have each writer wait until `gCorrectHs` reads 0, as `homingTick()` does. The writers are `calFinishBand()`, `calStartBand()`, `useMicro()`, `coilRelease()`, `jogRaw()`, `assumeAtLowStop()` and `bootPosition()`, in `src/s3/needle.cpp`.	DEFECT at most
"Measuring pass speed" says less than it does. The row (`reapHsps`, default 60 half-steps/s) sets the index calibration's creep and the portal's jog. It does not set the homing re-approach, which is not a setting.	Judged not worth changing.	Rename the label, for example "Calibration creep and jog speed". The key stays, so settings files keep working.	NOTE
No warning for a needle gain that will hunt. The start-up check of `wn` against its ceiling runs before the stored settings are applied, so it only checks the compiled defaults. A stored or portal value above the ceiling is never warned about.	Judged not needed.	Call the same check (`wnCeiling()`) from `applySettings()` after the needle settings are pushed down; print the warning and return it to the portal as a `warn`.	NOTE
The portal's needle buttons run on the portal task (core 0) and write the needle's state while the needle task runs on core 1; only stop and abort are handed over safely. A jog also holds the portal for its whole length, up to about 20 s at the lowest `reapHsps` nobody gets an answer meanwhile.	A known cross-core class, deferred: the buttons are used by one person at a time, with the needle at rest.	Turn each action into a request that the needle task picks up at the top of its pass, as `Needle::stop()` already works, and answer the portal from its result. The jog becomes a request that returns at once.	DEFECT at most; the jog's hold is a NOTE
Re-index answers "re-indexing" when it homes. On a needle with no zero it runs a plain home. Its refusal always says "a calibration or a bring-up tool has the needle", also when the needle is homing or sweeping.	Not yet addressed; the console says what really happened.	Have `startReindex()` return a reason string, as `startHoming()` does, and a flag saying it homed; then answer "homing - there is no zero to re-index from".	NOTE
`upVmax` accepts speeds that stall the needle: up to 4000 half-steps/s, while this needle stalls through the index far below that.	Not yet addressed. The default, 1100, is the measured limit of this mechanism.	Lower the row's maximum to the measured stall speed (chapter 5 (p. 73), section 5.18 (p. 103)), and note that another mechanism needs its own ceiling.	DEFECT at most: the crossing check finds the lost steps

What you see	Why it was left	Where a fix starts	Class
			and re-indexes

12.4 Settings, portal and console

Issues in the stored settings, the web portal, the network and the main board's console.

Table 129. Settings, portal and console

What you see	Why it was left	Where a fix starts	Class
i calls normal powers "LOW". It adds `*** LOW - the transmitter is turned down ***` to any value under 10 dBm, but the ladder's rungs 8, 20, 28 and 34 (quarter-dBm) are normal.	A bench line; its comment already says to check the value against the rung.	In `Net::driverReport()`, compare the driver's value with the rung asked for; flag only a difference, or a zero.	NOTE
The first **B** after a start-up gives 11 dBm. `B` keeps its own place in its list from start-up, so its first press always sets 44 (11 dBm), whatever the radio was running at, and saves it.	A bench key; the printout shows the new value.	Start from the rung in force (`cfg.wifiTxQ`) and step to the next lower entry.	NOTE: a later press, or the ladder, changes it again
The portal is starved while the needle moves. The step task runs at high priority on core 0 and does not yield during a move. The portal shares that core. An audio-board update is relayed by the portal task: the likely cause of four failed audio-board updates on one early day.	Accepted: the step timing comes first, and the portal is not needed while tuning.	Keep the needle at rest during an audio-board update (the workaround in use). For a fix, emit steps from a hardware timer or the RMT peripheral so the step task can sleep between steps, and measure the step jitter before and after.	NOTE: it ends when the needle stops
A settings write can land while the needle moves. "Still" is judged by the needle's speed, which reads 0 in the pause before a move and during a portal jog, so a write can land then and make the jog stutter. The forced save also retries up to 40 times without asking again.	Not yet addressed; the stutter is brief and the step task catches up.	Judge "still" by the needle's state (no move, no dwell, no jog), not its speed, and call `writeSafe()` again before each retry.	NOTE
No lock around the settings across the two cores. Known races, each of microseconds: a hand mark and an automatic sample at the same instant; an upload half applied when a background save copies the settings, followed by a power cut within about 2 s; a soft-limit edit crossing an apply. None seen.	The risk was accepted rather than rebuild the settings around a lock (chapter 7 (p. 153), section 7.11.2).	One mutex held by every writer of `cfg` and by the copy in `settingsWrite()` or post every portal change to the main loop as a request, so only one core ever writes.	NOTE

What you see	Why it was left	Where a fix starts	Class
No “unsaved” sign on the page, for example during the trial minute after an update, when nothing is saved. The page shows only the lock.	Not yet addressed.	Add an `unsaved` field to the state the page polls, and a pill “CHANGES NOT SAVED YET” naming the reason the firmware already keeps for the Save button.	NOTE
A settings upload with a warning hides the count: the page shows the warning, not “N settings applied”.	Not yet addressed.	Show both: the count, then the warning.	NOTE
An unreadable **`tuneUsed`** line. It is skipped with no note, and so is every mark line after it; the stored marks are kept. In a downloaded file the page then says “TUNING MARKS REFUSED - the file announced 0 and 0 parsed; the stored calibration was kept”, which is misleading. In a file with no `tuneOffset10` line, nothing is said.	Not yet addressed; nothing stored is lost.	Remember that a `tuneUsed` line was seen but unreadable, as the hand-offset line already does (`mkOffBad`), and refuse the block in its own words: “the tuneUsed line could not be read - the file’s samples were not applied”.	NOTE
The dump **`D`** is only partly a settings file. Its first lines carry several pairs each, which an upload refuses; only its tuning, IF (`ifOffset=`) and fixed-feature lines paste as they are, and its `wifiTxQ` line is ignored.	A bench view; the portal’s Download is the settings file.	Print one pair per line, or the same text as the download.	NOTE
Reboot the main board shows only “rebooting”. On trial the radio answers “rebooting - this firmware was still on trial, so the previous firmware comes back”, and the page drops that sentence. A refusal is shown in full.	Not yet addressed.	In the page’s `#rb` handler: `toast(r.m || 'rebooting')`.	NOTE
Two main-board update messages. An upload refused during the trial also prints the updater’s “No Error” on the console. An upload refused for the wrong chip gives its reason on the console only; the page shows “flash”.	Not yet addressed; both refusals work.	Skip `Update.end()` when the upload was refused or never began. Keep a reason string for the main board’s own image, as the relay does with `a32OtaSetError()`, and send it from `hOtaEnd()`.	NOTE
Saving the Network card always rejoins, 3 s later, even for an NTP server or time-zone change alone, because the card always sends the network name.	Not yet addressed; the rejoin takes a few seconds.	Have `Net::setWifi()` skip the rejoin when the name and passphrase are unchanged, or send them only when they changed.	NOTE
A portal-raised access point is confirmed on the console only.	The console is readable on the Console tab.	Carry the access point’s state in the page’s state, so a browser that joins it sees a pill.	NOTE
A blank board’s access point transmits at 2 dBm. The ladder moves only on failed joins, and a board with no network has none. On this machine an access point at 2.0 dBm was visible at 44 % but could not be joined; on another board it may be fine at close range, untested.	On this machine the rung was learned long ago.	Raise the access point at a fixed rung known to be joinable (15 dBm here), or step the ladder while the access point is up and nobody has joined.	NOTE when moving closer works; BLOCKER only if nothing but the USB console

What you see	Why it was left	Where a fix starts	Class
			helps. A blank board has just been flashed by USB, and `y` sets the network over the same cable
`y` echoes the WiFi password, onto USB and into the console ring, which any administrator with the Console tab open can read.	Accepted for this machine; a builder may judge otherwise.	In `setWifiInteractive()`, echo the network name, then a `*` for each character after the space.	NOTE
Drop one sample also drops hand marks. It accepts slots 0 to 2, the hand marks A, B and C.	It is the only way to remove one hand mark without clearing them all, so it may be wanted; the label is what is loose.	Relabel it (“Drop a mark or sample, slot 0-11”), or refuse slots 0 to 2 there and add a separate “Clear mark A/B/C”.	NOTE
`sys.quiet` casts its number unchecked. Nothing in practice: a value of 65 536 or more is undefined in C++; on this compiler 65 536 becomes 0, which turns into the 1 s minimum, and radio silence is capped at 600 s anyway.	Harmless.	Clamp the argument to 0..600 before the cast in `doAction()`.	NOTE

12.5 The link and the audio board

Issues in the link between the boards and in the audio board’s own firmware.

Table 130. The link and the audio board

What you see	Why it was left	Where a fix starts	Class
An audio-board image built with the wrong **`PROTO_VERSION`** confirms itself after five minutes, because frames of another version count only as wrong-version and the S3 looks absent. It becomes permanent; the link stays down, with no radio or Bluetooth sound, until the main board matches. The main	By design: it lets a protocol change roll out one board at a time (chapter 9	If it ever bites: count wrong-version frames on the audio board too, and refuse the five-minute path while they arrive, so the image stays on trial and rolls back at the next reset. That	BLOCKER: radio and Bluetooth stay

What you see	Why it was left	Where a fix starts	Class
board shows it (`wrong-version` in `s`, the pill, `linkver`); the audio board counts it but prints it nowhere, and its `s` shows only `link SILENT rx 0`.	(p. 224), section 9.14.5).	removes the one-board-at-a-time path, so the protocol procedure would need another way.	silent through a power cycle once the audio board has confirmed itself. Recovery: update the main board built with the audio board's version, then both together; no cable
No network time without the audio board. If the audio board is absent or silent from start-up, the main board never takes network time either, because that path writes through to the battery clock and waits for the handshake. The clock stays blank even with a working network.	Accepted: with the audio board down there is no radio or Bluetooth sound either, and the clock comes back with the link.	Let network time set the main board's software clock (`epochAtSync`, `millisAtSync`, `timeValid`) without the handshake, and send the write-through only when the audio board answers. The block is in the main loop, gated on `peerHello`.	NOTE
A one-way link failure leaves the audio board on a stale amplifier state. If only the audio-board-to-main-board direction fails, the main board hears nothing, drops the handshake and from then on sends only HELLO, once a second. The audio board still receives those HELLOs, so for it the main board is alive: it stays awake on the last amplifier state it was told. The sound plays on, and if the amplifier is switched off it never learns it (Bluetooth stays connectable on the Bluetooth source). Everything recovers by itself when the line works again. Never seen on the radio (chapter 9 (p. 224), section 9.10 (p. 242)).	Not yet addressed; it needs a broken wire to happen.	Have the audio board count only `SET_SYS` (not HELLO) as "the main board is talking", or have the main board keep sending `SET_SYS` while it tries to handshake. The sending side is the link keep-alive block of the main board's `loop()` (`SET_SYS` and `PING` go only while handshaken); the waking side is the audio board's `serviceWake()` (awake = amplifier on and the main board alive).	NOTE
`autoConnect` is still in the protocol, stored by the audio board and read by nothing: the radio never starts a Bluetooth	Removing the byte changes	Remove it the next time `ProtoBtCfg` has to change anyway, so the reset is	NOTE

What you see	Why it was left	Where a fix starts	Class
connection itself. Old settings files that carry the key still upload.	the structure's size. That needs no new `PROTO_VERSION`: until both boards carry it, each refuses the other's Bluetooth-settings messages and everything else works. But the audio board then resets its stored Bluetooth settings to defaults, and both boards need the update.	paid once. The audio board has no conversion: any size change puts that structure back to its defaults (chapter 7 (p. 153), section 7.5.5 (p. 167)).	
"pairing window open", and nothing opens. Pair a new device (`bt.pair`) sends a message named `MSG_BT_LOOK`, which opens the pairing window (the state `BT_LINK`, not `BT_LOOK`), and only on the Bluetooth source. The portal answers "pairing window open" on any source.	Renaming a message touches both boards and the protocol header.	Rename it (for example `MSG_BT_PAIR`); the value stays, so `PROTO_VERSION` need not change. Have `bt.pair` refuse with "switch to Bluetooth first" when the audio board's reported source is not Bluetooth.	NOTE
The audio board's help line omits **k** and **g**. Both keys work (Appendix D (p. 332)).	Bench keys, reachable only with a USB cable on the audio board.	Add both to the `keys:` line printed in `setup()`.	NOTE
The audio board's version is on no portal page. It appears on the main board's console at each handshake (`[PASS] A32 up: ...`) and in the boot report.	Not yet addressed.	Add the audio board's version (and its commit, from the boot report) to the page's state, and show it on the System tab beside the main board's. The value is `gLink.peerVersion`.	NOTE
A relay that fails mid-image sends no abort. The audio board stays muted until its own 20 s give-up, then restores its sound and keeps its old firmware. A wrong chip, a dropped connection, a stalled upload and a missing final answer do send the abort.	Judged not a problem.	Call `a32OtaAbort()` in `hOtaA32Upload()` the moment `a32OtaChunk()` fails.	NOTE
An audio edit made just before the audio board falls silent is lost. The refusal starts only once 2 s of silence have cleared the main board's copy; an edit in those 2 s is accepted, sent to nobody, and replaced by the audio board's own settings when it returns.	Not yet addressed; the window is short and needs the audio board to fail at that moment.	Have the audio board acknowledge each settings message, and report the edit as not applied when no acknowledgement arrives.	NOTE: nothing is corrupted

What you see	Why it was left	Where a fix starts	Class
The save before a commanded restart is tried once. When the main board asks the audio board to restart, the audio board forces a save; if that write fails, it restarts anyway and the change is lost.	Not yet addressed; a failed write has never been seen.	Retry the forced save a few times, and send a log line to the main board before restarting if it still fails.	NOTE
Audio-board console changes are undone. The volume law (`t`, `T`) and the knob calibration (`p`, `c`, `P`) set on the audio board's console are saved there, but the main board's copy is not refreshed, and the next portal edit of any audio row sends the older volume law back. Re-read the A32 (`sys.getcfg`) refreshes the copy.	Bench keys, reachable only with a USB cable on the audio board.	Have the audio board send its settings to the main board after any console change, as it answers `MSG_GET_CFG`.	NOTE
A broken audio-board image on trial says the wrong thing. An image that failed its link self-test announces "proto self-test FAILED - muted, BT closed, awaiting OTA", and at start-up "link and OTA kept up so this build can be replaced". On trial, an update is refused; only a reset, which rolls it back, gets out. The words are true only for an image flashed by USB.	The behaviour (never confirmed, rolled back by the next reset) is accepted; only the words are wrong.	When the image is on trial, announce "reboot the audio board to go back to the previous firmware" instead.	NOTE
The battery clock's temperature is sent and not used. Every time message carries it; the audio board prints it once, in its own USB banner; the main board never reads it.	Not yet addressed.	Show it in the portal, or drop the field at the next protocol change that has to happen anyway (see `autoConnect` above).	NOTE
The battery clock is wrong from the year 2100. The audio board reads and writes the year as two digits from 2000 and ignores the chip's century bit.	Not yet addressed.	Read and write the century bit in `Rtc::read()` and `Rtc::write()`, or accept the limit and say so.	NOTE
"ROLLED BACK" stays in every start-up report of both boards until the next update replaces the invalid image, so an old rollback can look like a new one.	Not yet addressed.	Remember in flash that a rollback was reported and print it only at the first start-up after it; or print the invalid image's version. Both boards use `esp_ota_get_last_invalid_partition()`.	NOTE

12.6 Sound

Issues in the sound path.

Table 131. Sound

What you see	Why it was left	Where a fix starts	Class
A click at the start of an audio-board update when the fade-out is longer than about 340 ms. The audio board mutes, then waits a fixed 400 ms before it writes flash, which stalls the audio task. The wait covers the fade-out, one audio block and the 46.4 ms output buffers. The default fade-out is 150 ms; the portal allows up to 5000 ms.	The default is well inside the wait, and an update is rare.	Make the wait follow the setting, the fade-out plus about 60 ms, staying well under the 20 s the main board waits for the audio board's answer; or cap `fadeOutMs` at 340 ms. The wait is the `delay(400)` in the `MSG_OTA_BEGIN` case of the audio board's `onMessage()`.	NOTE
The stall click. The audio task is sometimes frozen for 23 to 111 ms, and a click can be heard. The cause was never measured; the leading guess is a flash write, which turns the flash cache off and stalls every task that runs from flash.	A deeper output chain would ride through it, but costs memory the audio board does not have (chapter 10 (p. 256), section 10.8 (p. 275)).	Measure first. The stall detector already records each stall's length and time (`stalls`, `stlast`); record the time of every flash write on the audio board beside it. If they line up, keep flash writes away from playing moments, as the main board keeps them away from the moving needle.	NOTE: the sound comes back by itself
Silence while everything else works: the audio task is not watched by the watchdog. If it wedged, the main loop, the link and the portal would stay alive with no radio or Bluetooth sound, and nothing would restart the audio board. Never happened.	Not yet addressed.	Subscribe the audio task to the watchdog (`esp_task_wdt_add()`) and feed it once per block. The 15 s timeout is far above the longest stall seen.	DEFECT: Reboot the audio board fixes it
The audio board's **s** steals the portal's readings. It reads the peak meter (with its clip flag) and the ring's high-water mark, and reading resets them, so the portal misses them for one period. It also clears the I2S latches, but the portal reads `i2sSticky`, a copy nothing clears.	A bench key, reachable only with a USB cable on the audio board.	Give the console a read that does not reset (a "peek"), and leave the resetting read to the state message.	NOTE
No live figure for the audio board's free memory. The one figure, about 24 KB, was measured once. `s` prints only `ESP.getFreeHeap()`, which is not what a plain or a DMA allocation can get; the Bluetooth library prints the right figure only at start-up, on USB; the portal's `heap` is the main board's.	Not yet addressed; no fix chosen.	Print `esp_get_free_heap_size()` and `heap_caps_get_largest_free_block(MALLOC_CAP_DMA)` in `s`. Carrying them in `ProtoState` puts them on the portal; a new field goes at the end of the structure (chapter 10 (p. 256), section 10.15 (p. 286)).	NOTE

12.7 Main board, clock, display and build

Issues in the main board's start-up, its clock display and the build.

Table 132. Main board, clock, display and build

What you see	Why it was left	Where a fix starts	Class
About 3.3 s before anything starts. At every start-up the main board waits up to 3 s for its USB console port to open, then 300 ms more, before the display, the needle and the link start. In the radio no USB host is attached, so every start-up pays it all.	It keeps the start-up report whole when a cable is attached.	Shorten the wait, or start the display and the link first and print the banner when the port opens. The web console's ring already keeps lines for a late reader.	NOTE
Readout mode 1 shows a stale frequency. With the readout on "Tuning always", the digits show the frequency of the last angle count even when the sensor has stopped answering. Mode 1 is deliberately not gated on the amplifier and the source; the sensor is a separate matter.	Not yet addressed.	When `Needle::i2cOk()` is false, show the placeholder the clock shows when it has no valid time.	NOTE
The frequency readout shows a leading zero below 100 MHz (`0879` for 87.9 MHz). The clock follows `blankLeadZero` the readout does not.	Not yet addressed.	Pass `true` for the leading-zero blank in the `Display::showNumber()` call of `updateDisplay()`, or follow `blankLeadZero`.	NOTE
The console's – and = (or +) bypass the fade. They change `brightOn` by 15 and write it straight to the display, also while the amplifier is off, so the clock can sit at the wrong level until the next amplifier change, or step back briefly before it eases. They also skip the settings table's bounds, though their own floor and ceiling (0 and 255) match today.	Bench keys; the portal's brightness rows take the normal path.	Route both keys through the settings table's setter for `brightOn`, the portal's path, so the bounds and the fade apply.	NOTE
Version stamps cannot always be ordered. The stamp comes from the build PC's local clock, with no time zone; two builds made in different zones, or across a daylight-saving change, cannot be ordered by stamp alone.	Every build so far was made on one PC.	Stamp in UTC and add a `Z` in `scripts/version.py`. The string must still fit the 24-byte version field of the handshake (chapter 3 (p. 25), section 3.14 (p. 38)).	NOTE

12.8 Texts that say something false

These texts tell a reader something that did not happen. None changes what the radio does. All are NOTE.

Table 133. Texts that say something false

Where you read it	What it says	What is true	Fix
The portal's Firmware card	"The needle parks and the display goes dark while an update writes"	The needle stops where it is; a park is a different move.	"The needle stops and the display goes dark".
Console `B`, `o`, `O`	"saved" ("known, saved", "the clear is saved")	They only mark the settings changed. The write follows about 2 s later once the needle is still, and never while an image is on trial or the settings are locked.	"will be saved".
Console `i`	`OWN ACCESS POINT` and the access point's name whenever the main board is not joined	Also during a join, or radio silence, when no access point is up.	Print the real state: joining, silent, or access point up.
The answer to `POST /api/net`	"network saved - the radio joins it in a few seconds", also for a request with only the NTP server and time zone	Only a direct call to the API sees it, because the page always sends the network name (section 12.4 (p. 295)).	Answer by what was set.
Console `q`, `v`	"sweep refused - see the RDA state in 's' (running, absent, or lost)"	A request lying wholly outside the oscillator window is refused too, and leaves nothing in `s` to see; `v` reaches it when the dial sits outside the window.	Have `requestSweep()` give its reason.
The audio board's console	"[ZDD] DAC left analogue mute"	The code only models that event, from a theory the pop hunt excluded (Appendix H (p. 348)).	"[ZDD] a zero run long enough for the DAC's zero-data mute has ended".
The audio board's console	"OTA wrote N bytes, crc ok"	Printed also when the main board sent no checksum and none was checked.	Print "no crc sent" in that case.
The audio board's start-up line	"[PASS] clock pads at drive N"	PASS whatever the pads read back, also a value other than 3, or 0xFF when the three pads disagree.	Print FAIL unless it reads 3.

12.9 Ideas for improvement

These are not faults: improvements considered after v.1, with a starting point.

12.9.1 Check left and right, and let the portal swap them

The idea. The audio board writes the right speaker's sound into the first I2S slot and the left's into the second, the reverse of the usual order, because that gave correct stereo on this machine. The channel order is a setting of this machine that another builder will need to check (chapter 10 (p. 256), section 10.3.4 (p. 261)), and a portal switch to swap it on the fly would make that easy.

The test still to run. Play a left/right test track over Bluetooth and listen to which speaker speaks. That settles the Bluetooth path only. The radio path copies the converter's two slots straight across, and with mono sum on (the default) both slots carry the same sound, so the radio's order matters only

with mono sum off. The amplifier sends each channel to the right speaker; whether the audio board and the DAC deliver them the right way round has never been tested.

> FOR FIRMWARE CHANGES
>
> Where. `audioTask()` in `src/a32/audio.cpp`: the Bluetooth path writes `.r` into slot 0 and `.l` into slot 1, and the radio path copies the input slots in order. The channel format is set in `Audio::begin()` the order is described at the top of `src/a32/audio.h`.
>
> How.
>
> 1. Add a swap flag to the audio settings (`ProtoAudio` in `include/proto.h`).
> 2. In `audioTask()`, swap each frame's two words after the volume, balance and meter pass and before `i2s_write()`. Balance and the level meters then keep meaning the right and left speakers.
> 3. Add the row to `src/s3/settings_table.h`, and it appears in the portal.
>
> What the new field costs. It changes `ProtoAudio`'s size. A size change needs no new `PROTO_VERSION` (chapter 9 (p. 224), section 9.14.5): a message of the wrong length is refused, so a half-updated pair loses only the messages that carry `ProtoAudio`, and the link stays up. Until both boards carry the new structure, the audio board refuses every `MSG_SET_AUDIO` (volume, balance, fades and the rest do not reach it), and the main board refuses the audio board's `MSG_CFG` (its copy stays unknown, so the portal refuses audio edits). So:
>
> - Flash both boards: the audio board first, through the main board (updates do not use `ProtoAudio`), then the main board.
> - Download a settings file first and upload it afterwards. The audio board accepts a stored structure only if its size matches exactly, so its stored audio settings go back to their defaults once.

12.9.2 An equaliser

There is none; one was considered together with the swap.

> FOR FIRMWARE CHANGES
>
> It would go in the same pass of `audioTask()` that applies volume and balance, and would cover the radio and Bluetooth only: AUX never passes through the audio board. A few biquad filters per channel, with fixed-point coefficients worked out on the main board (or when the setting changes) and sent as settings. The audio board is short of memory and its audio task runs under a tight deadline (chapter 10 (p. 256)), so measure the task's time per block and the free memory before and after. The pop hunt showed how much this chain cares about timing.

12.10 Untested paths

Each of these is in the code and has never run on the radio. Nothing has gone wrong; each needs a fault, a state or a set-up the radio has not had. The class of each path if it fails is in the When it goes wrong table of its chapter. “Test build” means the method of the rollback tests (chapter 3 (p. 25), section 3.19 (p. 44)): a build made to fail in one chosen way, never committed, sent over the air with the USB cables unplugged, and watched through its rollback; one build per path.

Table 134. Untested paths

Area	Path never run on the radio	How to test it
Tuning	The discard of a measurement after a bus error on the tuner chip; the “lost” path; a re-probe after a real loss	On the bench, break the tuner chip’s bus during a sweep and watch the console for the discard and the ABANDONED line; restore it and watch for the re-probe.
Tuning	The replacement of a sample in a full table	Fill the table to twelve samples and take one more.
Tuning	A measurement showing both the coarse and the fine lists	Run `a` straight after a measurement.
Needle	Re-index on a needle with no zero	From the portal: Re-index after a failed home.
Needle	A portal stop or abort in the middle of an index calibration	From the portal, during Calibrate the index.
Needle	A slip of more than 40 half-steps found at a crossing, driving the re-index ladder; the three-failure FAULT; any homing FAULT since the step budgets were fixed	They need lost steps, which a test build can fake by shifting the counted position.
Needle	The position memory after a crash, and the detection of a torn record	The crash: a test build with a deliberate crash.
Needle	The turn chosen by the measured tuner ends after an angle-sensor outage of more than half a turn	—
Settings	The `n/a` rows of a download and the HTTP 409 refusal of audio edits, while the audio board is silent	A test build of the audio board that stops sending for a minute.
Settings	The downgrade lock, its complete-file unlock, the NOT SENT note and the refused download	A test build of the main board with a higher settings version, left to confirm itself and save, then replaced by the release image over the air. A rollback cannot do it: nothing is saved on trial.
Settings	A failed audio-board save and its retry; a failed write at the audio board’s confirmation	Needs a flash fault.
Network	The rescue access point coming back after a failed join, on the current retry path	Needs a network that refuses the radio.
Network	The placeholder gate and the whole first-time path (chapter 8 (p. 186), section 8.2.2 (p. 189))	A spare board with the release image.
Network	The lockout beyond its first steps; the “full” answer to a fifth account; session eviction with more than four browsers	—
Network	A phone joining the open access point and opening the sign-in page by itself. The main board’s side has been seen working.	Raise the access point from the portal, join `Ambersong` from a phone, and expect the sign-in page to open by itself.

Area	Path never run on the radio	How to test it
Link	The audio board's five-minute "the S3 never spoke" confirmation; the wrong-version count and pill with a real mismatch (they have only read 0)	A test build with another `PROTO_VERSION` on the audio board only.
Link	The degraded mode after a failed link self-test, on either board, and the rule that such a build is never confirmed	A test build with the self-test forced to fail.
Link	The two battery-clock pills in a real fault (`rtc` 1 or 2)	A test build of the audio board that reports an invalid time, or does not answer the time request.
Recovery	A main-board image that runs but never earns confirmation (portal or network down); a confirmation write that fails; a reset during either board's trial with a setting changed in it; a hang of the needle supervisor; a hung portal task	One test build per path.
Sound	Bluetooth staying invisible in RADIO mode, checked with a phone; to be repeated after any library update. The check still pending.	With a phone, on RADIO (chapter 10 (p. 256), section 10.7 (p. 270)).
Sound	The channel order	A left/right test track over Bluetooth (section 12.9.1 (p. 303)).
Sound	`sys.clkpair`: the master clock and the frame clocks driven at different strengths	Chapter 10, section 10.10 (p. 277), and Appendix H (p. 348), section H (p. 348).7.
Sound	The volume-swing test: audio-board volume 47 against 94 with a tone streaming, to see whether pops follow the signal level	Chapter 10, section 10.10 (p. 277).
Sound	The Bluetooth transmit-power A/B that would prove the 0 dBm start-up ceiling; a drive-3 pop count with Bluetooth idle	Appendix H (p. 348), sections H (p. 348).7 and H.8.
Sound	A phone negotiating 48 kHz; `connectable` off	Each waits for a listening session with the right phone or set-up (chapter 10 (p. 256), section 10.14 (p. 285)).
Sound	A long session measuring the drift between a phone's clock and the local 44.1 kHz. There is no rate adaptation; the 186 ms ring absorbs only jitter.	A long listening session over Bluetooth.

12.11 Code tidy list

FOR FIRMWARE CHANGES

None of these changes what the radio does; they mislead a reader of the code. Comments: correct them in one comment-only commit, and prove it by stripping the comments from each file and comparing with the previous commit. Unused code: remove it in one commit; the build then proves nothing used it.

Comments that say something the code does not do. The code is right.

Table 135. Code tidy list

Where	What the comment says	Fix
`src/s3/rda.cpp`, the stereo bits (`gBinST`)	"See isStereoBin()"; the function is `binStereo()`.	Rename it in the comment.
`src/s3/rda.cpp`, the broadcast comment in the fine pass of `rdaTask()`	"see candidate()"; it is `bestCandidate()`.	Rename it in the comment.

Where	What the comment says	Fix
`src/s3/rda.cpp`, `requestSweep()`	"three of the five known fixed features" lie outside the window; the window's comment in `src/s3/rda.h` lists four (98.7, 101.6, 104.4 MHz and the 76.2 MHz band edge).	Make the two agree.
`src/s3/needle.cpp`, `readAs5600()`	The gap warning "still says 'nearest turn' either way"; it prints "on the turn inside the tuner's ends" (section 12.2 (p. 292)).	Correct it with that fix.
`src/s3/needle.h`, `setMicro()`	`moving` is the division used for homing; no homing path selects a division, so a home started from tracking runs at `microSlow`. The comment above `coilRelease()` already says the division is set only for a sweep, tracking and a park.	Correct the header, or have homing select `moving` if that was the intent.
`src/s3/needle.cpp`, `calFinishBand()`	`cfg.idx*` "is never written by anything"; the main loop now writes them from `calTakeBand()`.	Put it in the past tense.
`src/s3/needle.cpp`, the band snapshot at the top	`applySettings()` runs `setIndexCal(0,0,0,0)` true only before an index calibration has been saved.	"with the stored values, zero before the first band is saved".
`src/s3/main.cpp`, `updateDisplay()`	"The timezone is set ONCE, by Net::begin()"; it is also set by `Net::setNtp()`, by `startNtp()` in `src/s3/net.cpp`, and by the console `W`.	List the setters.
`src/s3/main.cpp`, the `B` case of the console	Its list has an 80 level "applied as 60"; there is no 80, and after 8 it wraps to 60.	Drop the sentence.
`src/s3/main.cpp`, `settingsDump()`	Its `wifiTxQ` line is "BYTE-FOR-BYTE WHAT settingsToText WRITES"; the settings file no longer carries `wifiTxQ`.	Say the line is for reading only.
`src/s3/main.cpp`, above `settingsDump()`	"one-pair lines from dialLow on"; the `dialLow` line carries two pairs.	"from tuneOffset10 on", or split the line.
`src/s3/net.cpp`, the join-failure comment in `Net::loop()`	"This file opens with 'THE RADIO MUST NEVER BECOME UNREACHABLE'"; that text opens `src/s3/net.h`.	Name `net.h`.
`src/s3/main.cpp` and `src/s3/needle.cpp`: `seatTurn()`'s banner, the `calLow` comment, `SMP_MAX_DRIFT`'s comment	The tuner's travel is "about 2300 counts"; the measured span is 2197, about 2200.	Write about 2200.
`data/portal.html`, the `data-a` click handler	`tune.drop` uses "the same frequency prompt"; it has its own slot-number prompt.	Say so.
`src/a32/audio.cpp`, "THE TWO KNOBS"	The data-line output delay function "remains only for the A32's own USB console"; the setter and its key are gone, only the read-back `sdOutDelayIs()` is left.	Say only the read-back remains.
`src/a32/audio.cpp`, `src/a32/audio.h`, the audio board's key handler	They date the removal of the console key `y` a day earlier than the release that removed it, v.1.0.3.	Give the release, v.1.0.3.
`src/a32/audio.cpp`, above `zeroWatch()`	"Every other counter in this file resets on read"; `btUnderruns()` and `gainWatch()` do not.	Name the ones that do.
`src/a32/main.cpp`, above the `[ZDD]` announcements	Calls "a pop is predicted HERE" in the text below stale; that text is gone.	Drop the reference.

Where	What the comment says	Fix
src/a32/main.cpp, settingsDefaults(), cfgB.autoConnect	The value is “stored and shown”; its portal row was removed, so it is stored and not shown.	Correct it.

Code defined and never used.

Table 136. Code tidy list

Where	What	Fix
src/s3/rda.cpp	R2_MONO.	Remove it.
src/s3/rda.h, src/s3/rda.cpp	Rda::fineRssi(), so gFineR is written and never read.	Remove both, or print it with the fine pass (section 12.2 (p. 292)).
src/s3/rda.cpp	tuneTo()'s fine flag, false at every caller. True would be wrong: the function takes tenths of a MHz, so doubling the channel addresses the same 100 kHz points; the real 50 kHz path is tuneTo20().	Remove the parameter, so nobody can pass true.
src/s3/rda.h	Rda::IF_OFFSET20_NOMINAL, the nominal 10.7 MHz, read by nothing.	Remove it, or keep it and derive the oscillator window from it (section 12.2 (p. 292)).
src/s3/needle.h, src/s3/needle.cpp	Needle::getCalibration(), Needle::getIndexCal(), Needle::state(): no caller.	Remove them.
src/s3/drive.h	HALFSTEPS_PER_REV.	Remove it, or keep it as documentation with a comment that nothing uses it.
src/s3/drive.h, src/s3/drive.cpp	DRV_FULL: Drive::setMode() handles it, nothing selects it.	Remove it, or keep it as documentation.
src/s3/net.h, src/s3/net.cpp	Net::apVerified(), Net::isQuiet(), Net::quietLeft(), Net::ntpResync().	Remove them.
src/s3/main.cpp, the MSG_HELLO_ACK case of onMessage()	The “[WARN] PROTOCOL MISMATCH” check, which can never fire: the decoder drops every frame of another version first, and the wrong-version count does the job. Its comment already calls it “A GUARD THAT CANNOT FIRE TODAY”.	Kept as a guard; only its removal is open.
src/a32/main.cpp	btStreaming: written by the Bluetooth audio-state callback, never read.	Remove it.
src/a32/audio.h, src/a32/audio.cpp, src/a32/rtc.h	Audio::source(), Rtc::present().	Remove them.
include/link.h	txDropped_ (counted, never read), lastRxMs(), peerProto() with peerProto_ (the main board reads the handshake's protoVersion directly).	Remove them, or print txDropped_ in the console s.
include/proto.h	ProtoFramer::overruns: counted, never read.	Remove it, or print it in s beside the other link counters.
platformio.ini, [env:s3] and [env:a32]	The flags AMB_S3=1 and AMB_A32=1: no source file tests them; each board is selected by its own source folder.	Remove them, or use them if shared code ever needs to tell the boards apart.

Appendix A. Settings reference

This appendix lists every stored setting of both boards, then the constants of the settings system, then the experiments, which look like settings but are never saved. How settings are saved, backed up and restored is chapter 7 (p. 153). Where a chapter explains a setting in detail, the row points to it.

How to read the tables.

- Default is the compiled value, with its unit. Where the firmware stores a value scaled, the stored number follows in brackets.
- Range is the hard clamp: a value outside it is stored at the bound, and the answer says CLAMPED (chapter 7 (p. 153), section 7.6.2 (p. 168)). Exceptions are stated.
- Changed by:

  Table 137. Settings reference

Word	Meaning
Portal (all)	On its portal tab, by any signed-in account, guest included.
Portal (admin)	On its portal tab, by the administrator only.
File	A settings-file upload (administrator only).
S3 key `x`	The main board's console key (USB, or the portal's Console tab).
A32 key `x`	The audio board's console key (its own USB cable only).
Knob	The front volume knob.
Button	A portal button, named as on the page.
Auto	The firmware itself.

- Saved in names the board's storage namespace and key (chapter 7 (p. 153), section 7.2 (p. 154)): `amb3/cfg` is the main board's one settings blob; `amb/audio` and `amb/bt` are the audio board's two blobs; `amb/potMin` and its neighbours are single audio-board keys; `net` and `auth` are the main board's network and account stores.
- Every portal row is written in the downloaded settings file and read back on upload.

A.1 Audio and Bluetooth (the audio board)

Audio tab. All are the audio board's, saved in `amb/audio`.

Table 138. A.1 Audio and Bluetooth (the audio board)

Key	Default	Range	Meaning	Changed by
`volume`	40	0–255 (a linear fader)	Master volume; the volume law below is applied on the audio board. The knob's first reading after each audio-board start replaces it (chapter 7 (p. 153), section 7.4.3 (p. 162)).	Portal (all), File, A32 key `+` / `-` (10 per press; `+` also unmutes), Knob
`muted`	1 in the compiled defaults; forced to 0 at every start	0 / 1	Software mute: the audio board sends zeros to the converter. Saved, but never restored. Forced on during an audio-board update and restored afterwards.	Portal (all), File, A32 key `m`
`taper`	2.5 (stored 25)	1.0–4.0, step 0.1	The volume law: gain = (volume ÷ 255) to the power gamma. 1.0 is linear amplitude; 2.0 puts half travel at −12 dB; 2.5 at −15 dB; 3.0 at −18 dB; about 3.3 behaves like a real audio-taper potentiometer.	Portal (admin), File, A32 key `t` / `T` (0.1 per press)
`balance`	0	−100 to +100	Negative leans left.	Portal (admin), File
`gainRadio`	+8.0 dB (stored 80, tenths of a dB)	−20 to +30 dB, step 0.5 (the audio board also clamps −60 to +30)	Makeup gain on the tube-radio input, before the volume.	Portal (admin), File
`gainBt`	−7.0 dB (stored −70)	−20 to +30 dB, step 0.5 (the audio board also clamps −60 to +30)	Bluetooth gain, before the volume, set to match the radio's loudness within about 1 dB.	Portal (admin), File
`monoSum`	1 (on)	0 / 1	Sum left and right on the radio input (about 3 dB less noise).	Portal (admin), File
`muteOnChange`	1 (on)	0 / 1	Fade down and up across a change of source.	Portal (admin), File
`fadeInMs`	1000 ms	0–5000 ms, step 50	Fade-up time on arriving at a source. 0 acts as 1 ms.	Portal (admin), File
`fadeOutMs`	150 ms	0–5000 ms, step 10	Fade-down time on leaving a source. 0 acts as 1 ms. Above about 340 ms, an audio-board update starts with a click (chapter 9 (p. 224)).	Portal (admin), File

Why these defaults.

- **`volume`** 40 and **`balance`** date from an earlier setup of the amplifier's own volume control, which has since been replaced. That control sits on the amplifier's board, reached only from the back. The front knob is the audio board's, not the amplifier's (Hardware Bible, chapter 6, section 6.8). The defaults stay: the knob's first reading overwrites `volume` at start-up anyway, and `balance` is a plain left/right trim. It does nothing on AUX, which never passes through the audio board (Hardware Bible, chapter 6 (p. 120), section 6.7 (p. 131)).

- **`gainRadio`** +8.0 dB was measured with the tube radio's own volume control at about one eighth of its travel, its normal position (Hardware Bible, chapter 6, section 6.8). The previous +15 dB clipped at the gain stage at every volume. A different tube-set level needs a different radio gain.
- **`gainBt`** −7.0 dB was measured the same way. A pink-noise reference at −14 dBFS RMS (dBFS: decibels below digital full scale), a typical streaming loudness, played from a computer at full volume, arrived at about −15 dBFS. −7 dB lands it within about 1 dB of the radio, so a change of source does not change the loudness.
- **`fadeInMs`** 1000 and **`fadeOutMs`** 150: arriving at a source should be gentle, about a second; leaving should be quick, because the listener has just turned the selector.
- **`muteOnChange`** exists because the source selector is an ON-OFF-ON switch whose centre position reads as AUX (Hardware Bible, chapter 9, section 9.3), so a change from RADIO to Bluetooth passes through AUX for a moment. The fade covers that passage.

Chapter 10 explains the volume law, the gains and the fades.

Bluetooth tab. All are the audio board's, administrator only, saved in `amb/bt`.

Table 139. A.1 Audio and Bluetooth (the audio board)

Key	Default	Range	Meaning	Changed by
`connectable`	1 (on)	0 / 1	Let a paired phone reconnect. Even at 1, the audio board is connectable only while it is awake (amplifier on, main board talking) and the source is Bluetooth, so a phone cannot latch on at night to a radio whose amplifier is off. At 0, Bluetooth is closed entirely, the pairing window included (chapter 10 (p. 256)).	Portal (admin), File
`lookTimeoutS`	90 s	15–600 s, step 5	The pairing window, despite the name: the explicit pairing state LINK falls back to STDBY after this. The state LOOK has its own fixed 20 s, which is not a setting (chapter 10 (p. 256)).	Portal (admin), File
`pauseOnLeave`	1 (on)	0 / 1	Send a "pause" to the phone before the disconnect that follows leaving Bluetooth.	Portal (admin), File
`autoConnect`	0	—	Not a setting. A field of the Bluetooth structure that nothing reads: the radio never starts a Bluetooth connection itself. It stays so the structure keeps its size (a size change would reset every Bluetooth setting). Not a portal row; an old file's `autoConnect` line is skipped.	Nothing

The volume knob's calibration. The audio board's, saved as three single keys `amb/potMin`, `amb/potMid`, `amb/potMax`. Not portal rows. Written in the settings file only as the comment `# potcal min..centre..max`, and never read back from it.

Table 140. A.1 Audio and Bluetooth (the audio board)

Key	Default	Range	Meaning	Changed by
`potMin`	60	0–4095 (raw reading)	The knob's raw reading at its minimum.	Pot: minimum, A32 key `p`
`potMid`	0, meaning "not measured"	0–4095	The raw reading at the knob's mechanical centre.	Pot: centre, A32 key `c`
`potMax`	3990	0–4095	The raw reading at its maximum.	Pot: maximum, A32 key `P`

The three points map the knob to the volume, 0–255. If the maximum is not at least 65 above the minimum, the defaults are used. The defaults 60 and 3990 are this machine's knob, read at its two ends; they apply until the knob is calibrated (the knob's wiring is in the Hardware Bible, chapter 9, section 9.4). The Pot buttons are on the Audio tab, administrator only; the audio board measures the knob itself, as the median of nine readings. Chapter 10 explains the knob.

A.2 Display and lights

Display tab. All are the main board's, saved in `amb3/cfg`. Chapter 4 explains them.

Table 141. A.2 Display and lights

Key	Default	Range	Meaning	Changed by
`brightOn`	255	0–255	Clock brightness while the amplifier is on.	Portal (all), File, S3 key `-` and `+` / `=` (15 per press)
`brightOff`	110	0–255	Clock brightness with the amplifier off. Not lower on purpose: the display's colon is lit by its own fixed circuit and cannot dim with the digits (Hardware Bible, chapter 7, section 7.6).	Portal (all), File
`hour12`	0 (off)	0 / 1	12-hour clock, with no AM/PM indicator.	Portal (admin), File
`blankLeadZero`	1 (on)	0 / 1	Blank the hour's leading zero.	Portal (admin), File
`dispDwellMs`	250 ms	0–5000 ms, step 50	Stillness before the clock brightness starts to move.	Portal (admin), File
`dispFadeMs`	1500 ms	0–10000 ms, step 50	Eased (smoothstep) travel between the two brightnesses.	Portal (admin), File
`showTuning`	0	a choice, 0–2	0: the clock always. 1: the tuning readout always. 2: the tuning readout while tuning, then back to the clock.	Portal (admin), File, S3 key `f` / `F` (cycles)
`tuneHoldMs`	4000 ms	500–20000 ms, step 250	In mode 2, how long the readout stays after the dial last moved.	Portal (admin), File

Lights tab. The panel lamps are the main board's, saved in `amb3/cfg`. The Bluetooth lamp's levels are the audio board's, saved in `amb/bt` the main board only mirrors and forwards them.

Table 142. A.2 Display and lights

Key	Default	Range	Meaning	Changed by	Saved in
`panelTuning`	255	0–255	Panel lamp level while tuning.	Portal (all), File	`amb3/cfg`
`panelIdle`	128	0–255	Panel lamp level, radio idle.	Portal (all), File	`amb3/cfg`
`panelOther`	96	0–255	Panel lamp level on AUX or Bluetooth.	Portal (all), File	`amb3/cfg`
`panelIdleMs`	5000 ms	0–60000 ms, step 250	Time without tuning before “idle”.	Portal (admin), File	`amb3/cfg`
`panelFadeMs`	900 ms	0–5000 ms, step 50	Eased panel fade. 0 acts as 1 ms.	Portal (admin), File	`amb3/cfg`
`panelDwellMs`	250 ms	0–5000 ms, step 50	Stillness before the panel lamps move.	Portal (admin), File	`amb3/cfg`
`btLedOff`	255	0–255	Bluetooth lamp brightness scale in state OFF. State OFF ignores its scale (chapter 10 (p. 256)).	Portal (admin), File	`amb/bt`
`btLedOn`	255	0–255	… in state ON, ready.	Portal (admin), File	`amb/bt`
`btLedLook`	255	0–255	… in state LOOK, looking.	Portal (admin), File	`amb/bt`
`btLedStdby`	255	0–255	… in state STDBY, standby.	Portal (admin), File	`amb/bt`
`btLedFound`	255	0–255	… in state FOUND.	Portal (admin), File	`amb/bt`
`btLedCon`	255	0–255	… in state CON, connected.	Portal (admin), File	`amb/bt`
`btLedLink`	255	0–255	… in state LINK, explicit pairing.	Portal (admin), File	`amb/bt`

A.3 Needle and tuning

“hs” is a half-step of the needle’s stepper motor. “Counts” are the tuning shaft’s accumulated angle-sensor readings (4096 per turn, carried across whole turns). Every key here is the main board’s and saved in `amb3/cfg` every portal row here is on the Needle tab, administrator only. The keys that are not portal rows say so.

A.3.1 The needle

The motion defaults are the settings arrived at on a bench test rig; they are this build’s settings. Chapter 5, section 5.17 (p. 100), explains each; section 5.18 (p. 103) holds this machine’s measured values.

Table 143. A.3.1 The needle

Key	Default	Range	Meaning	Changed by
`upVmax`	1100 hs/s	100–4000, step 50	Top speed of every second-order move, in either direction (portal label “Sweep up, top speed”).	Portal (admin), File
`upAccel`	18000 hs/s²	1000–60000, step 500	Acceleration of the same moves.	Portal (admin), File
`dnVmax`	1700 hs/s	100–4000, step 50	Top speed of the decay: the fall to the park position.	Portal (admin), File

Key	Default	Range	Meaning	Changed by
`dnAccel`	18000 hs/s²	1000–60000, step 500	The decay's acceleration figure; the decay law itself has no acceleration limit, so it is used only in the park's arrival test (chapter 5 (p. 73)).	Portal (admin), File
`wn`	9.0 rad/s	1–21, step 0.5	Natural frequency of the second-order ring-down: how quickly the swing settles. Keep it below 4 × `zeta` × `upAccel` ÷ `upVmax`.	Portal (admin), File
`zeta`	0.50	0.1–2.0, step 0.05	Damping of the ring-down: how much the needle overshoots.	Portal (admin), File
`riseMs`	100 ms	10–3000 ms, step 10	The fall's envelope: rise.	Portal (admin), File
`fallMs`	1000 ms	10–5000 ms, step 10	The fall's envelope: fall.	Portal (admin), File
`dwellUpMs`	100 ms	0–3000 ms, step 25	Pause before a second-order move (not while following the knob).	Portal (admin), File
`dwellDnMs`	1000 ms	0–3000 ms, step 25	Pause before the fall to the park position.	Portal (admin), File
`microFast`	16 (1/16 step)	1–32	Microstep divisor for sweeps, parks, the index calibration and jogs, and for the needle sitting at the park (portal label "Microstep while moving"). Rounded down to a power of two.	Portal (admin), File
`microSlow`	32 (1/32 step)	1–32	Microstep divisor while following the tuning knob (portal label "Microstep at rest"). The two portal labels are misleading: "at rest" here means following the knob, and the parked needle uses `microFast` (chapter 5 (p. 73), section 5.17 (p. 100)).	Portal (admin), File
`reapHsps`	60 hs/s	10–500, step 10	Creep speed of the index calibration (portal label "Measuring pass speed"); also the portal's jog speed. Not the homing re-approach.	Portal (admin), File
`sweepOn`	0 (off)	0 / 1	A full end-to-end needle sweep each time the amplifier comes on. The home at the main board's start-up (the set plugged in, or a restart) ends with a full sweep whatever this says (chapter 5 (p. 73)).	Portal (admin), File
`posMin`	−300 hs from the index	−3000 to 3000	Low soft limit: the park position, and where `dialLow` is printed. The pair must bracket the index (0) and not cross; any other pair is refused and the old one kept. The range is wide on purpose, so that a wrong value is refused rather than silently stored at a bound. Reset to −300…+300 at start-up if corrupt. On this machine: −1156.	Portal (admin), File, S3 key `m`, Set low limit here, Needle Limit ◂ 20 / 20 ▸ (moves both), Auto (the index calibration shifts it; the start-up clean-up)
`posMax`	+300 hs	−3000 to 3000	High soft limit: where `dialHigh` is printed. Same rules. On this machine: +414.	Portal (admin), File, S3 key `M`, Set high

Key	Default	Range	Meaning	Changed by
				limit here, Needle Limit ◂ 20 / 20 ▸, Auto
`idxOffFwd`	0 hs	−500 to 500	The index band: the sensor's OFF edge going up, from the index. A measurement, not a preference.	Auto (the index calibration: S3 key `k`, Calibrate the index), Portal (admin), File
`idxOnRev`	0 hs	−500 to 500	The index band: the ON edge going down.	As `idxOffFwd`
`idxOffRev`	0 hs	−500 to 500	The index band: the OFF edge going down.	As `idxOffFwd`
`idxOnFwd`	0	—	Not a row and not in the file. The homing edge of the band, zero by definition. An upload line `idxOnFwd=...` is refused as unknown.	The index calibration (always 0)
`dialLow`	879 (87.9 MHz)	500–2000 (tenths of a MHz)	The frequency printed at the low end of the dial face. The firmware puts it at the low soft limit (`posMin`), not at the mechanical stop.	Portal (admin), File
`dialHigh`	1079 (107.9 MHz)	500–2000	The frequency printed at the high end of the dial face. The firmware puts it at the high soft limit (`posMax`).	Portal (admin), File

Why these defaults.

- **`upVmax`** 1100, not 1700: moving up through the index faster than about 1100 hs/s, the needle mechanism stops following while the firmware keeps counting (measured: always at 1500, often at 1200 to 1400, never at 1100). A physical value of this machine; the range still allows up to 4000.
- **`microFast`** / **`microSlow`**: the firmware makes the microsteps itself, with PWM on the four inputs of the motor's driver, which has no microstepping of its own (Hardware Bible, chapter 8). The row caps the divisor at 32.
- **`posMin`** / **`posMax`** −300 / +300: the provisional soft limits. They assume the needle's real travel extends more than 300 hs on each side of the index, and that the index band is narrower than about 600 hs, so the sweep after a home crosses it.
- **`dialLow`** / **`dialHigh`**: this set's dial face prints 87.9 MHz at its low end and 107.9 MHz at its high end, and the firmware maps them onto the soft limits (`posMin`, `posMax`). Another set's builder types in their own, and sets the soft limits where the glass prints them.

A.3.2 The tuning

Chapter 6 explains every row. `calLow`, `calHigh` and `ifOffset` are portal rows; the keys below them are hand-written keys, in the settings file but not on the portal. Their upload rules are in chapter 7 (p. 153), section 7.6.2 (p. 168).

Table 144. A.3.2 The tuning

Key	Default	Range	Meaning	Changed by
`calLow`	0 (a placeholder “not measured”)	−20000 to 20000 counts	Shaft count at the tuner’s low mechanical end. Typing it sets bit 0 of `tunerEndsSet`.	Portal (admin), File, S3 key `c`, Tuner = low end
`calHigh`	6023 (a placeholder)	−20000 to 20000 counts	Shaft count at the high end. Typing it sets bit 1.	Portal (admin), File, S3 key `C`, Tuner = high end
`ifOffset20` (“Tube set IF”)	10.60 MHz (stored as `ifOffset20` = 212, in 50 kHz steps)	10.00–11.50 MHz, step 0.05	The tube set’s IF. Turns the tuning chip’s reading of the set’s local oscillator into a station frequency. 10.60 MHz is the tube set’s fitted FM IF (Hardware Bible, chapter 13).	Portal (admin), File
`bandLow`	881 (88.1 MHz)	500–2000, else REFUSED	The fallback straight line’s low end, in tenths of a MHz. Nothing reads it once two samples exist.	File only
`bandHigh`	1079 (107.9 MHz)	as `bandLow`	The fallback line’s high end.	File only
`tuneOffset10`	0	−200 to 200 (tenths of a MHz, so ±20 MHz)	The hand offset (the dial correction), sliding the whole tuning curve. Commits with the marks. Never erased automatically.	Dial −0.1 MHz / Dial +0.1 MHz (0.1 to 5 MHz per request, ±20 MHz in total); reset to 0 only by Reset the dial correction to 0 File
`tuneUsed`	0	0–0xFFFF; decimal or `0x` always written	The mask: bit i set means slot i holds a sample.	Mark station A/B/C here, Clear all marks, Drop one sample (slot 0-11), Measure this dial position, S3 key `v`, Auto (the automatic sampler), File
`tuneMark0` … `tuneMark11`	empty	`<shaft count within ±1 000 000>,<frequency × 10, 870–1085>`	The tuning samples. Slots 0 to 2 are the hand marks A, B, C; slots 3 to 11 are automatic samples. Raw shaft counts, so a sample keeps its meaning when the tuner ends are measured again.	As `tuneUsed`
`tunerEndsSet`	0	0–3, else REFUSED	Bit 0: the low tuner end was measured; bit 1: the high end. Only 3 lets the curve use the measured travel. Worked out for files older than version 5 (chapter 7 (p. 153), section 7.6.2 (p. 168)).	S3 keys `c` / `C`, Tuner = low end / high end, typing `calLow` / `calHigh` on the portal, File, the version-4 conversion
`spurUsed`	0	0–16; always written	How many fixed features are stored. Empty by default on purpose: the list is only ever earned.	S3 key `o` (add the strongest candidate), S3 key `O` (clear), Forget the fixed features, Auto (the start-up clean-up), File
`spur0` … `spur15`	empty	772–972 (oscillator frequency × 10); no two	The fixed features: peaks the cabinet itself produces, left out when hunting the tube set’s local oscillator.	As `spurUsed`

Key	Default	Range	Meaning	Changed by
		closer than 0.3 MHz		
`lastAngle`	0	—	Not in the file. The last shaft count, which picks the right turn at start-up.	Auto: every 10 s when it changed, only while the angle sensor answers

Why these defaults. `calLow` / `calHigh` 0 / 6023 are old compiled placeholders meaning “not measured”, not a claim about the shaft. The `spur*` range is the window the tuning chip searches, 77.2 to 97.2 MHz. That is because fixed features are learned at local-oscillator frequencies. The tube set’s FM oscillator runs below the station (Hardware Bible, chapter 13), and the window is the printed dial minus the 10.7 MHz design IF (chapter 6 (p. 120)).

A.4 Network, accounts and stored state

These are stored but are neither portal rows nor lines of the settings file. All are the main board’s.

Table 145. A.4 Network, accounts and stored state

Key	Default	Range	Meaning	Changed by	Saved in
`wifiTxQ`	8 (2.0 dBm)	8–60 (quarter-dBm: 2.0–15.0 dBm)	The learned WiFi transmit power: the rung of the joining ladder that last worked. The ladder tops out at 15 dBm, a firmware choice. Printed by the console dump `D` an old file's `wifiTxQ` line is skipped. Chapter 8, section 8.4 (p. 197).	Auto (copied from the WiFi driver on every pass of the main loop), S3 key `B` (cycles 60, 44, 34, 28, 20, 8; the first press after a start gives 44)	`amb3/cfg`
`ssid`, `pass`	empty	name up to 32 characters, passphrase up to 63	The house WiFi network. Never written in the file.	Portal Network card, S3 key `y`	`net`
`ntp`	`pool.ntp.org`	a host name	The primary time server; `time.nist.gov` is a fixed second.	Portal Network card	`net`
`tz`	`EST5EDT,M3.2.0,M11.1.0`	a POSIX time-zone rule	The local time rule: the default time zone. Change it for your build (chapter 8 (p. 186)).	Portal Network card	`net`
`users`	One administrator account with a compiled default password that the portal forces you to change	up to 4 accounts	The portal accounts, salted and hashed. Never written in the file.	Portal account cards; S3 key `~` erases every account and restores the default administrator (chapter 8 (p. 186))	`auth`
`magic`, `version`	`0xA838`, 7	—	The header of the main board's settings blob. The version is written in the file as a comment.	The firmware	`amb3/cfg`

The network and account stores are written at once, even while a new firmware is on trial (chapter 3 (p. 25), section 3.11.5 (p. 34)).

A.5 Constants of the settings system

FOR FIRMWARE CHANGES

These change only with a new build. The Where column names the source file, function or constant that holds each value.

Table 146. A.5 Constants of the settings system

Name	Value	Where	Meaning
`SETTINGS_MAGIC`	`0xA838`	`src/s3/main.cpp`	Marks a blob that has a header (version 2 and later).
`SETTINGS_VERSION`	7	`src/s3/main.cpp`	The current layout of the main board's settings.
`sizeof(Settings)`	220 bytes	`src/s3/main.cpp`	The size of the main board's blob.

Name	Value	Where	Meaning
Oldest layout	100 bytes	`settingsLoad()`	A shorter blob is refused and the defaults used.
Audio structure	15 bytes	`include/proto.h`	`ProtoAudio`, the `amb/audio` blob.
Bluetooth structure	12 bytes	`include/proto.h`	`ProtoBtCfg`, the `amb/bt` blob.
`TUNE_MARKS`	12	`src/s3/main.cpp`	Tuning sample slots: 3 by hand, 9 automatic.
`CFG_SPURS`	16	`src/s3/main.cpp`	Fixed-feature slots, frozen into the layout. `Rda::MAX_SPURS` must equal it; a `static_assert` checks.
`PROVISIONAL_LIMIT_HS`	300 hs	`src/s3/main.cpp`	The default soft limits, and the ones the start-up clean-up restores, ±.
Main-board save wait	2000 ms	`settingsFlush()`	Quiet time after the last change.
`lastAngle` save period	10 s	the main loop	
“Save settings now” retries	40, 25 ms apart while the other core is writing	`settingsForceFlush()`	
Reboot save retries	15, 100 ms apart	`hReboot()`	None while the firmware is on trial.
Audio-board save wait	2000 ms	`settingsFlush()` (audio board)	
End of the audio board’s trial hold	60 s after the main board’s first greeting, or 5 min if the main board never spoke	`CONFIRM_AFTER_HELLO_MS`, `CONFIRM_AFTER_MS` (audio board)	Held changes are written at confirmation.
Knob reading	every 50 ms, median of five	`updateVolumePot()` (audio board)	
Knob deadband	24 raw counts (about 0.6 % of travel)	`POT_DEADBAND` (audio board)	Smallest knob movement that changes the volume after the first reading.
Knob calibration reading	median of nine	audio board	Taken by the Pot buttons.
Knob calibration fallback	minimum 60, maximum 3990; used when the maximum is not at least 65 above the minimum	audio board	
Volume law clamp on the audio board	10–40 (gamma × 10)	`Audio::setTaper`	Applied when the setting is applied.
Balance clamp on the audio board	−100 to 100	`Audio::setBalance`	
Gain clamp on the audio board	−60 to +30 dB	audio board	Applied to both gains when they are converted. Fade times are not clamped there; 0 acts as 1 ms.
Upload timeout	20 s	`data/portal.html`	
Upload note	512 bytes; each list names 5 keys of up to 20 characters	`settings_table.h`	
Clamp report threshold	0.0005	`hSet()`, `settingsFromText()`	A larger change by the bound is reported as clamped.
Mirror payload limit	1088 bytes	`PROTO_MAX_PAYLOAD`, `include/proto.h`	Checked by `static_assert` for the audio, Bluetooth and combined structures.

Name	Value	Where	Meaning
Portal rows	58: 38 main board, 10 audio, 10 Bluetooth	`gSet[]`	

A.6 Experiments: actions, never saved

These portal actions change behaviour until the next restart, and are never saved. The reason: “an experiment that survives a reboot is one whose result gets attributed to something else a week later. If it turns out to be the fix it becomes a setting then, with a reason attached.” All are administrator only. Those marked “no button” are reached only by posting the action directly to the portal (Appendix C (p. 328)).

Table 147. A.6 Experiments: actions, never saved

Action	Value	What it does	Button	Chapter
`sys.pot`	0 ignore, 1 use	Ignore the front volume knob.	Ignore the volume pot / Volume pot back on	10
`sys.clkdrv`	2–3, clamped	Drive strength of the audio board’s three clock pins (GPIO0, 18 and 17) together. Its start-up value is 3.	no button	10
`sys.clkpair`	22, 23, 32 or 33 only	Master-clock drive, then bit-clock and word-clock drive.	no button	10
`sys.dindrv`	0–3, clamped	Drive strength of the data line.	no button	10
`sys.tail`	0 / 1	Sign-extended tail.	Sign-extended tail ON / off	10
`sys.zfloor`	0 / 1	Zero-data floor.	Zero-data floor ON / off	10
`sys.dctest`	0 / 1	Replace the output with a constant DC word: silent as audio, so anything heard is the fault.	Constant DC test ON / off	10
`sys.dcword`	index 0–6	Pick the DC word, from a fixed list, all at or below −42 dBFS.	no button	10
`sys.bttx`	0–5 (−12 to +3 dBm)	Bluetooth transmit ceiling, down only.	no button	10
`sys.quiet`	seconds; absent or 0 or less means 120; at most 600	WiFi off; it returns by itself.	WiFi off for 2 minutes	8
`disp.test`	0 normal, 1 show 8888, 2 blank	Display test; console keys `8`, `b`, `9` do the same.	Show 8888 / Blank / Normal (Display tab)	4

`sys.sddly`, a former experiment of this kind, sent the radio to full volume and is retired; do not restore it (chapter 10 (p. 256), section 10.15.3).

Appendix B. Network and portal constants, HTTP endpoints, state fields

This appendix is for anyone who changes the network or portal code, or who writes a script that talks to the radio. It lists every network and portal constant, every HTTP endpoint and the network fields of the live state. You do not need it to use or repair the radio: chapter 8 (p. 186) explains each value in context.

Everything here runs on the main board (the S3). “Fixed” means the value is set in the source and changes only with a new build.

B.1 Network constants

In `src/s3/net.cpp` unless stated.

Table 148. B.1 Network constants

Name	Value	Unit	Range	Meaning	Changed by
`ssid`, `pass` (NVS area `net`)	empty	text	name up to 32, password up to 63	The house network	Network card; console
`ntp` (NVS `net`)	`pool.ntp.org`	host name	—	Primary time server; `time.nist.gov` is the fixed second	Network card
`tz` (NVS `net`)	`EST5EDT,M3.2.0,M11.1.0`	POSIX TZ rule	—	Local time rule	Network card
`AP_SSID`	`Ambersong`	—	—	Rescue access point’s name	fixed
Access-point security	open (`nullptr` passphrase in `softAP()`)	—	—	No WiFi password on the rescue access point (chapter 8 (p. 186), section 8.6.7 (p. 201) for the cost)	fixed
Access-point channel / devices / hidden	1 / 2 / no	—	—	“a rescue hatch, not a hotspot”	fixed
Captive-portal DNS server (`gDns`)	port 53, every name (`"*"`) to the access point’s address	—	—	Runs only while the access point is up	fixed
Access-point address	192.168.4.1	—	—	The ESP32’s soft-AP default	—
`softAP()` attempts	3, 400 ms apart; settle 50 ms + 100 ms	—	—	`startAp()`	fixed
`JOIN_MS`	20 000	ms	—	Join timeout before the access point and the next power rung	fixed
`RETRY_MS`	120 000	ms	—	Retry the house network from the access point	fixed
`FORCE_HOLD_MS`	600 000	ms	—	Minimum life of a forced access point	fixed

Name	Value	Unit	Range	Meaning	Changed by
Request grace	500	ms	—	Delay before a portal force-AP or radio-silence request is carried out	fixed
`setWifi` grace	3000	ms	—	Delay before rejoining after a network change	fixed
Radio silence	120 (portal default)	s	1–600	Length of radio silence	`sys.quiet`
`TX_LADDER`	8, 20, 28, 34, 44, 60	quarter-dBm	2.0–15.0 dBm	Transmit-power rungs; the 15 dBm top is a firmware choice	fixed
`cfg.wifiTxQ` (the settings, not the settings file)	8	quarter-dBm	8–60	Learned transmit power	the ladder; console `B`
Scan / choice	all channels / by signal	—	—	Pick the strongest access point with the name	fixed
Power save	`WIFI_PS_NONE`	—	—	Set after connecting, and every 5 s	fixed
Host name / mDNS	`ambersong` / `ambersong.local`, `_http._tcp` on 80	—	—	Name on the network	fixed
`NTP_FRESH_MS`	4	hours	—	Network time counts as fresh this long after a sync	fixed
Network time to the battery clock	1	hour	—	At most this often, while fresh (`src/s3/main.cpp`)	fixed

B.2 Portal constants

In `src/s3/portal.cpp` unless stated.

Table 149. B.2 Portal constants

Name	Value	Unit	Meaning
HTTP port	80	—	
Portal task	core 0, priority 3, 8192	bytes of stack	
`IDLE_SOCKET_MS`	3000	ms	Silent-socket reaper; off during uploads
Upload watchdog	15 000	ms	No upload activity for this long ends the upload
`MAX_USERS`	4	accounts	Slot 0 is the administrator
`MAX_SESS`	4	sessions	The stalest is evicted on a fifth sign-in
`IDLE_MS`	5	minutes	Sliding session idle expiry
Cookie	`amb`, `HttpOnly`, `SameSite=Lax`, `Max-Age` 86400 s, `Path=/`	—	The server's 5-minute rule governs
Session token	32 hexadecimal characters (128 bits from `esp_random()`)	—	
Account record	`User{ name[17], salt[8], hash[32], admin, used }`, one blob, key `users` in NVS area `auth`	—	Password stored as SHA-256 of (8-byte salt + password), one pass
`MAX_BAD`	4	addresses	Device addresses tracked for lockout
Lockout	from the 5th failure: 1, 2, 4 … 64	minutes	Doubling
Password / name	at least 6 / 2–16 (new user), 1–16 (rename)	characters	
Log slice	4096	bytes per poll	Web console read
Console send	1 key, or up to 128 characters while a prompt reads a line	characters	Web console write
Reboot save retries	15 × 100	ms	Before refusing a reboot (none while the image is on trial)
Restart delays	200 (reboot), 300 (S3 update)	ms	Let the answer leave first
Placeholder account	`DEFAULT_USER` / `DEFAULT_PASS`	—	Created on a blank board and by console `~`. This book does not print the values: the console prints them when it creates the account, and the sign-in page names them (chapter 8 (p. 186), section 8.2.2 (p. 189)).

B.3 Page and console constants

Table 150. B.3 Page and console constants

Name	Value	Where
Poll interval	5 s, doubling on failure to 30 s; none while the tab is hidden	`data/portal.html` (`POLL_MIN`, `POLL_MAX`)
Start-up retry	2, 4, 8, then every 15 s	`data/portal.html`
Request deadline	12 s (settings upload 20 s, firmware upload none)	`data/portal.html`
Toast	2.6 s acknowledgement (amber) / 12 s refusal or warning (red)	`data/portal.html`
Console buffer on the page	60 000 characters; early poll 900 ms after a send	`data/portal.html`
Schema cache	`localStorage['amb.sc']`, tag `FW_VERSION-<0/1>`	the page and `schemaTag()`
`LOG_CAP` / `IN_CAP`	16 384 / 256 bytes	`src/s3/console.cpp`
Prompts	`y` 60 s, `W` 40 s (restarted at each character); `drainLine` 50 ms	`src/s3/main.cpp`
`y` buffer	98 bytes: a 32-byte name, a space, a 63-byte password, the terminator	`src/s3/main.cpp`
`l` monitor	60 s at most, a line about every 250 ms	`src/s3/main.cpp`
USB console	115 200 baud, native USB CDC (`ARDUINO_USB_CDC_ON_BOOT=1`)	`platformio.ini`, `setup()`

B.4 HTTP endpoints

All on the S3, port 80, registered in `Portal::begin()`.

- Signed in: a valid session cookie `amb`.
- Gated: refused with 403 `{"e":"mustchg"}` while the account is still the placeholder.
- Admin: the slot-0 account.
- Request bodies are form-encoded unless stated. `{"ok":1,"m":"…"}` is the generic acknowledgement.
- An API request that needs a session and has none answers 401 `{"e":"login"}`. The two upload endpoints answer 403 instead.

Stay at about one request every few seconds (chapter 8 (p. 186), section 8.7.2 (p. 205)).

Table 151. B.4 HTTP endpoints

Method	Path	Who	Parameters	What it does	Answers
GET	/	anyone	header `If-None-Match`	The gzipped page from flash	200 (gzip, ETag, `no-cache, must-revalidate`); 304 if the ETag matches

Method	Path	Who	Parameters	What it does	Answers
GET	`/favicon.ico`	anyone	—	Nothing	204
any	unknown path	anyone	—	Redirect; on the rescue access point this is the captive portal's redirect	302 `Location: /` on the house network; 302 `Location: http://192.168.4.1/` otherwise
POST	`/api/login`	anyone	`u`, `p`	Check the salted hash, create a session, set the cookie	200 "welcome" + `Set-Cookie` 403 `{"e":"bad"}` 429 `{"e":"locked","s":N}`
POST	`/api/logout`	anyone (ends the session if the cookie is valid)	—	End this session, clear the cookie	200 "bye"
GET	`/api/boot`	signed in; allowed while gated	`sc` = cached schema tag	Schema (or `null` if the tag matches), values and state in one reply `{"sc":…,"val":…,"st":…}`	200; 401
GET	`/api/schema`	signed in, gated	—	Schema only: tag, user, admin, fw, tabs, setting rows	200; 401; 403
GET	`/api/values`	signed in, gated	—	`{key: value}` for every setting row	200; 401; 403
POST	`/api/set`	signed in, gated; admin for admin rows	`k` key, `v` value	Parse the whole number (decimal comma accepted), clamp, store, apply	200 `{"ok":1,"v":stored[,"clamped":1][,"warn":…]}` (`warn` only for `calLow`/`calHigh` when the tuning curve is in trouble); 404 `{"e":"nokey"}` 403 `{"e":"admin"}` 409 `{"e":"the audio board is not answering - its settings cannot be changed now"}` for an audio-board row while the A32's settings are not known (chapter 7 (p. 153), section 7.4.5 (p. 164)); 400 `{"e":"not a number"}` (also when `v` is missing)
POST	`/api/act`	signed in, gated; per action (Appendix C (p. 328))	`a` action, `v` optional number	Run the action	200 `{"ok":1,"m":…}` 200 `{"ok":1,"warn":…}` for a refusal, including an unreadable `v` ("REFUSED - not a number: …"); 403 `{"e":"no"}` for an unknown or forbidden action
GET	`/api/state`	signed in; allowed while gated	`log` = next console byte (admin only; ignored otherwise)	Live dashboard plus `net{sta,ssid,ip,rssi,ntp}`, `sess`, `ota`, `mustchg` with `log`, a `log{from,next,t}` slice of up to 4096 bytes	200; 401
GET	`/api/rda`	signed in, gated, admin	—	The last RDA sweep, bin by bin with the judgement made about each bin (the coarse bins are kept after a refinement), the fine pass's 11 points (`fine`), and the fixed-feature list (chapter 6 (p. 120)). It exists so that the judgement can be	200; 403 text "admin only"

Method	Path	Who	Parameters	What it does	Answers
				checked: without it, a wrong judgement would discard the signal silently.	
POST	/api/cons	signed in, gated, admin	k = one key, or up to 128 characters while a prompt reads a line	Inject into the console as if typed, newline appended	200 “sent”; 403 text “admin only”; 400 {"e":"1 to 128 characters"} 400 {"e":"one key at a time - each character is its own command"} 503 {"e":"console busy"}
GET	/api/settings.txt	signed in, gated, admin	—	Download the settings file (ambersong.txt, key=value); audio-board rows read n/a while the A32’s settings are not known (chapter 7 (p. 153))	200 text/plain attachment; 403 text; 409 text “Settings are locked: …”
POST	/api/settings.txt	signed in, gated, admin	raw text/plain body	Apply a settings file	200 {"ok":1,"m":"N settings applied"[,"warn":…]} 403 {"e":"admin"}
GET	/api/users	signed in, gated	—	{"me":…,"u":[{i,n,a}],"max":4} a normal user sees only themself	200
POST	/api/passwd	signed in; allowed while gated	new old (required for your own account); i target slot (admin only); name optional rename	Change the password (new salt) and optionally the name	200 “password changed” / “credentials changed”; 404 {"e":"nouser"} 403 {"e":"admin"} 403 {"e":"oldpw"} 400 {"e":"short"} 400 {"e":"longname"} 409 {"e":"taken"}
POST	/api/user/add	signed in, gated, admin	n (2–16), p (6 or more)	Add a normal user in the first free slot, 1 to 3	200 “user added”; 403; 400 {"e":"name"} / {"e":"short"} 409 {"e":"exists"} 507 {"e":"full"}
POST	/api/user/del	signed in, gated, admin	i (1–3)	Delete the user and drop their sessions at once	200 “user removed”; 403; 400 {"e":"no"}
GET	/api/net	signed in, gated, admin	—	{"ssid":…,"ntp":…,"tz":…}, never the password	200; 403 {"e":"admin"}

Method	Path	Who	Parameters	What it does	Answers
POST	`/api/net`	signed in, gated, admin	`ssid`, `pass` (blank = keep), `ntp`, `tz`	If `ssid` is present: refuse it empty, store, rejoin in 3 s. If `ntp` is present: store the server and the time zone, apply the zone, restart network time if on the house network	200 “network saved - the radio joins it in a few seconds”; 400 `{"e":"the network name is empty"}` 403
POST	`/api/reboot`	signed in, gated, admin	—	If the S3 image is on trial: restart at once, no save (the previous image comes back). Otherwise save the settings; if refused (and not locked), stop the needle and retry 15 × 100 ms; restart	On trial: 200 “rebooting - this firmware was still on trial, so the previous firmware comes back”. Otherwise 200 “rebooting”; 409 `{"e":"not rebooting – <why>"}` (needle resumed). Restart 200 ms after a 200; 403
POST	`/api/ota/s3`	admin session, not the placeholder (checked in both upload callbacks)	multipart file (the page uses field `f`)	Write the S3’s other program slot, reboot (chapter 3 (p. 25), section 3.10.2 (p. 29))	200 “rebooting into the new firmware”, then restart; 500 `{"e":"flash"}` 403 text “admin only” / “change the default password first”; 409 `{"e":"this firmware is still on trial – retry in a minute"}` while the running image is on trial (the bytes are received and dropped; nothing restarts)
POST	`/api/ota/a32`	same	multipart file	Relay the image to the A32 over the link (chapter 3 (p. 25), section 3.10.3 (p. 30))	200 “N bytes sent, the A32 is rebooting”; 500 `{"e":"<reason>"}` 403

Two answers to know about (chapter 12 (p. 290)): `POST /api/net` always answers “network saved - the radio joins it in a few seconds”, even for a request carrying only the time server and zone (the page always sends the network name, so only a direct request sees this); and the page shows only “rebooting” for any successful `POST /api/reboot`, whatever the radio answered.

B.5 State fields

The fields of `GET /api/state` that belong to chapter 8 (p. 186). The other fields belong to the chapters that produce them; chapter 4 (p. 46), section 4.10 (p. 59), lists the main board’s health fields.

Table 152. B.5 State fields

Field	Meaning	Built by
`net.sta`	1 = on the house network	`buildState()`, `src/s3/portal.cpp`
`net.ssid`	The house network's name, or `Ambersong`	`buildState()`
`net.ip`	The radio's address	`buildState()`
`net.rssi`	Received signal in dBm; 0 on the access point	`buildState()`
`net.ntp`	1 = network time fresh	`buildState()`
`sess`	Active sessions	`buildState()`
`ota`	1 = an upload is in progress	`buildState()`
`mustchg`	Set while the signed-in account is still the placeholder; the page then shows only Set your account	`portalStateJson()`, `src/s3/main.cpp`
`rtc`	The battery clock's health: 0 fine or not asked yet, 1 answered with no valid time, 2 not answering (chapter 4 (p. 46), section 4.10 (p. 59))	`portalStateJson()`
`linkver`	The S3's count of link frames dropped for another protocol version (chapter 3 (p. 25), section 3.13 (p. 37) chapter 9 (p. 224))	`portalStateJson()`
`log`	Only with `?log=` from an administrator: `{"from":F,"next":N,"t":"..."}`, up to 4096 console bytes, oldest first	—

Appendix C. Portal actions

Look here when you want to know exactly what a portal button does, who may press it, and which chapter explains it; or when you script an action that has no button.

Every portal button that does something, rather than change a setting, posts `/api/act` with an action name `a` and an optional number `v` (Appendix B (p. 321), section B (p. 321).4). Setting widgets post `/api/set` instead.

- Who: "guest" actions are allowed to every signed-in account; all others need the administrator.
- **v**: absent means 0.
- Answers: 200 `{"ok":1,"m":…}` when the action ran; 200 `{"ok":1,"warn":…}` when it was refused, painted red on the page (an unreadable `v` gives `REFUSED - not a number: …`); 403 `{"e":"no"}` for an unknown action or one the account may not use.
- Button: the label on the page, and its tab. "Script only" means there is no button: post `/api/act` directly, signed in as the administrator.
- The actions run in `doAction()`, `src/s3/settings_table.h`, on the portal task.

Most actions belong to other chapters; the last column says which.

C.1 Bluetooth

Table 153. C.1 Bluetooth

Action	Who	v	What it does	Button	Ch.
`bt.play`	guest	—	Bluetooth play, sent to the audio board	Now: Play	10
`bt.pause`	guest	—	Pause	Now: Pause	10
`bt.next`	guest	—	Next track	Now: ▸▸	10
`bt.prev`	guest	—	Previous track	Now: ◂◂	10
`bt.disconnect`	guest	—	Drop the Bluetooth source	Now: Disconnect Bluetooth: Disconnect	10
`bt.pair`	guest	—	Open the pairing window. The audio board acts only on the Bluetooth source, but the answer is “pairing window open” on any source (chapter 12 (p. 290)).	Now: Pair Bluetooth: Pair a new device	10
`bt.forget`	admin	—	Drop every pairing	Bluetooth: Forget every pairing	10

C.2 Needle

Table 154. C.2 Needle

Action	Who	v	What it does	Button (Needle tab)	Ch.
`needle.stop`	admin	—	Stop the needle; also aborts a running calibration and ends a recovery or a repeating sweep (a running jog is not interrupted)	STOP	5
`needle.home`	admin	—	Home; a refusal says why	Home	5
`needle.sweep`	admin	0 once, non-zero repeat	Sweep the measured travel	Sweep once (0) / repeatedly (1)	5
`needle.reindex`	admin	—	Re-index now	Re-index	5
`needle.track`	admin	—	Follow the tuner	Track the tuner	5
`needle.jog`	admin	half-steps, clamped to ±200 (a clamp is reported)	Jog in micro-step mode	◂ 100, ◂ 10, 10 ▸, 100 ▸ (−100, −10, +10, +100)	5
`needle.limitLow`, `needle.limitHigh`	admin	—	Set a soft limit here	Set low / high limit here	5
`needle.nudge`	admin	half-steps, ±1 to 200	Move both soft limits together	Needle Limit ◂ 20 / 20 ▸ (−20 / +20)	5
`needle.calBand`	admin	passes (default 3)	Characterise the index band	Calibrate the index (3)	5
`needle.calAbort`	admin	—	Abort a calibration	Abort calibration	5

C.3 Tuning and the dial

Table 155. C.3 Tuning and the dial

Action	Who	v	What it does	Button (Needle tab)	Ch.
`needle.tuneLow`, `needle.tuneHigh`	admin	—	Record the tuner's end here	Tuner = low / high end	6
`tune.markA`, `tune.markB`, `tune.markC`	admin	MHz, 87.0 to 108.5	Hand mark at this dial position	Mark station A / B / C here asks "Frequency of the station you are tuned to now, in MHz" (pre-filled 107.3)	6
`tune.nudge`	admin	MHz, ±0.1 to ±5.0	Slide the tuning curve (the hand dial correction)	Dial −0.1 / +0.1 MHz	6
`tune.nudgeZero`	admin	—	Reset the hand dial correction to 0, the only thing that erases it. Answers "dial correction reset to 0", or "the dial correction is already 0".	Reset the dial correction to 0	6
`tune.clear`	admin	—	Clear every mark; the hand dial correction is kept (the answer says so when one is set)	Clear all marks	6
`tune.drop`	admin	slot 0 to 11	Drop one sample; the hand dial correction is kept. Slots 0 to 2 are the hand marks A, B and C, and dropping one removes that mark (chapter 12 (p. 290)).	Drop one sample asks "Which sample slot to drop (0-11)?"	6
`rda.sample`	admin	—	Measure this dial position with the RDA5807M. Refused outside the listening policy: "the dial calibrates only while you listen to the radio - switch the amp on and select RADIO"	Measure this dial position	6
`rda.spurClear`	admin	—	Forget the fixed features	Forget the fixed features	6

A comma typed in a prompt's answer is read as a decimal point.

C.4 Display

Table 156. C.4 Display

Action	Who	v	What it does	Button (Display tab)	Ch.
`disp.test`	admin	0 normal, 1 show 8888, 2 blank	Display test	Show 8888 (1) / Blank (2) / Normal (0)	4

C.5 Audio and the audio board

Table 157. C.5 Audio and the audio board

Action	Who	v	What it does	Button	Ch.
`pot.min`, `pot.ctr`, `pot.max`	admin	—	Record the volume pot's position	Audio: Pot: minimum / centre / maximum	10
`sys.getcfg`	admin	—	Re-read the audio board's settings	Audio: Re-read the A32	9
`sys.rebootA32`	admin	—	Reboot the audio board	System: Reboot the audio board	9
`sys.pot`	admin	0 ignore / 1 use	Ignore the volume pot (not saved)	System: Ignore the volume pot (0) / Volume pot back on (1)	10
`sys.bttx`	admin	0 to 5 (−12 to +3 dBm)	Bluetooth transmit power, down only (not saved)	script only	10

C.6 Audio experiments

None of these is saved. Chapter 10 explains each.

Table 158. C.6 Audio experiments

Action	Who	v	What it does	Button	Ch.
`sys.dctest`	admin	0 or 1	Constant-DC diagnostic on or off	System: Constant DC test ON (1) / off (0)	10
`sys.dcword`	admin	index 0 to 6	Pick one of seven fixed DC words for the diagnostic	script only	10
`sys.zfloor`	admin	0 or 1	Zero-data floor experiment	System: Zero-data floor ON (1) / off (0)	10
`sys.tail`	admin	0 or 1	Sign-extended tail experiment	System: Sign-extended tail ON (1) / off (0)	10
`sys.dindrv`	admin	0 to 3, clamped	Data-pin drive experiment	script only	10
`sys.clkpair`	admin	22, 23, 32 or 33	Clock-pin drive pair	script only	10
`sys.clkdrv`	admin	2 to 3, clamped	Clock-pin drive	script only	10

`sys.sddly` is retired: this portal action sent the radio to full volume. Do not restore it (chapter 10 (p. 256), section 10.15.3).

C.7 System and network

Table 159. C.7 System and network

Action	Who	v	What it does	Button (System tab)	Ch.
`sys.save`	admin	—	Save the settings now; a refusal says why	Save settings now	7
`sys.quiet`	admin	seconds; absent or 0 or less = 120; capped at 600	Ask for radio silence: WiFi fully off, then back by itself. Carried out by the network loop 500 ms later. Answer: “wifi off - it will return by itself”	WiFi off for 2 minutes (120)	8
`net.forceAp`	admin	—	Ask for the rescue access point; carried out 500 ms later; holds 10 minutes. Answer: “raising the access point - join “Ambersong”, then http://192.168.4.1/”. The driver’s confirmation is on the console only.	Raise the rescue access point	8

Rebooting the main board is not an action: Reboot the main board posts `/api/reboot` (Appendix B (p. 321)).

Appendix D. Console keys

Each board has a text console in which every key is one command. This appendix lists every key of both, with where it is explained. Look here when a chapter or a failure table names a key, or when you work at the bench with a USB cable. The keys are case-sensitive. A row reads like this:

- keys separated only by a space (`s S`) do the same thing;
- keys separated by a slash (`m` / `M`) are a pair of different commands, and the slashes in the description give each key’s action in the same order (`m` sets the low limit, `M` the high one);
- keys on rows of their own (`y` and `Y`) are different commands.

The last column gives the sections that explain each key: 5.16 is chapter 5 (p. 73), section 5.16 (p. 98).

D.1 Main board

How to reach it. The main board’s native USB port, the USB-C socket on the back panel, at 115 200 baud; or, with no cable, the portal’s Console tab, signed in as the administrator (chapter 8 (p. 186), section 8.9). Lines from the audio board appear there too, prefixed `[A32]`.

Rules.

- One key per send from the web console. Every character is its own command, so the portal refuses anything longer, except while a line prompt (`y`, `W`) is waiting, when up to 128 characters go through.
- Any key not listed is ignored, silently, including the Enter the portal appends to every send.
- The S3 does not restart when a console attaches, so its start-up banner is never seen: press `s`.
- The web console has the cable's power, including `z` and `~` (chapter 8 (p. 186), section 8.6.7 (p. 201)).

Table 160. D.1 Main board

Key	What it does	Where
Status and help		
`s S`	Status: the amplifier, source, needle, index switch, drive, angle sensor and tuning chain, portal counters, the link (`alive` or `SILENT`, frames received, checksum errors, `wrong-version N`, with "<- the boards run different protocol versions: flash both" when N is not 0), the last reset reason, the uptime and the `image` line (trial and rollback). For the needle: position, target, state, drift, crossings, jitter, steps, speed, PWM, and the angle sensor's residue, which must stay constant. For the tuning: the curve, the hand offset, each sample with its residual, the RDA5807M's state. Ends with the command list.	1.2, 4.12.2, 5.16, 6.9
`?`	The command list (also printed at the end of every `s`, and at start-up).	8.9
Network		
`i I`	Network: joined or `OWN ACCESS POINT`, the network's name, the radio's address and signal; portal up or down, sessions, network time fresh or stale; access-point clients; what the WiFi driver really holds; and, only when not on the house network, a scan of every network heard, which holds the main loop for a few seconds.	8.3.8, 8.9
`y`	Line prompt: the house network's `name <space> password` (gives up 60 s after the last key typed; a blank line cancels). Stores them and rejoins 3 s later. Sets the WiFi name and password only; the NTP server and the time zone change only from the portal's Network card. Echoes what is typed, the password included.	8.9
`Y`	Raise the rescue access point now. It holds 10 minutes, longer while someone is on it, then goes home.	8.3.4, 8.9
`~`	Erase all portal accounts and restore the placeholder account (lost-password recovery).	8.6.6
`z`	Erase the WiFi radio's RF calibration in flash, and reboot. A transmitter diagnosis; it touches none of the radio's own calibrations.	8.9
`B`	Step the WiFi transmit power along 60, 44, 34, 28, 20, 8 quarter-dBm (15 down to 2 dBm), cycling, and print what the driver applied. The first press after a start-up always gives 44. The value is kept with the settings and written about 2 s later, though the key says "saved" at once.	8.4, 12.4
Clock and display		
`W w`	Line prompt: local time, `YYYY-MM-DD HH:MM:SS` (gives up 40 s after the last key typed). Converted with a fixed `EST5EDT` rule, whatever zone the portal holds, which stays in force afterwards. Sets the main board's clock, and the battery clock only while the audio board answers; otherwise it says the battery clock was NOT updated.	4.9
`8`	Show `8888`: all segments.	4.12.2
`b`	Blank the display.	4.12.2
`9`	Back to normal.	4.12.2
`-` / `=` `+`	Clock brightness with the amplifier on (`brightOn`) down / up by 15, written straight to the display with no fade, and saved.	4.7, 12.7

Key	What it does	Where
f F	Cycle the readout (showTuning): clock always, tuning always, tuning while tuning. Saved.	4.6.2
Z	Zero the step-jitter and display slot-error counters.	4.12.2, 5.16
Needle		
H h	Home the needle.	5.9, 5.16
T t	Track the tuner. Prints “NOT tracking” when the request is dropped.	5.10, 5.16
P p	Park. Prints “NOT parking” when the request is dropped.	5.10, 5.16
x X	Stop the needle; also aborts a running calibration. It also ends a recovery.	5.15.1, 5.16
A	Abort a running calibration.	5.15.1, 5.16
r	Sweep the full measured travel, repeating, until x.	5.10, 5.16
k K	Calibrate the index band, 5 passes; put the needle on the switch first. (The portal button uses 3.)	5.12.1
m / M	Set the low / high soft limit at the needle’s position.	5.12.2
n / N	Jog −20 / +20 half-steps at 60 half-steps/s, microstepped.	5.16
j / J	Jog +200 / −200 half-steps at 200 half-steps/s, in half-step mode (a bring-up tool).	5.16
u / U	Jog +200 / −200 half-steps at 200 half-steps/s, microstepped.	5.16
1 … 4 / 0	Hold one driver input at full duty / release the coils.	5.16
e	Index edge log on or off: every edge, with its back-dated position and direction.	5.8, 5.16
l L	Live index monitor: the raw pin about every 250 ms, for up to 60 s. Enter is ignored; any other key stops it. It pings the audio board and feeds the watchdog on every pass.	5.16, 8.9
g G	Print that the end-stop calibration was removed and that m / M replace it. Kept because old instructions still name these keys.	5.16
Tuning		
c / C	Store the tuner’s low / high end: the current angle count. Refused if the angle sensor does not answer.	6.9, 6.10
V	Measure this dial position with the RDA5807M and store it as a sample if it passes every gate.	6.7, 6.9
R	The RDA5807M’s state, its bus, the window, and where the local oscillator should be.	6.9
q	A sweep of the whole window (about a minute). Not a measurement: nothing is stored.	6.9
v	A narrow sweep around the prediction (about 10 s). Nothing is stored.	6.9
a	Print the last sweep, bin by bin (the coarse points, kept after a refinement), then the fine pass’s points at 50 kHz, with the chosen one marked.	6.9
o	Mark the last sweep’s strongest candidate as a fixed feature. Refused if already known, or if the list is full (16). Use it with the tube set switched off: o does not check.	6.9, 6.10
O	Forget every fixed feature.	6.9
Settings		
D	Dump all settings as text, including the IF (ifOffset=) and the learned transmit power (wifiTxQ, which the settings file does not carry). Only partly a settings file.	7.10, 12.4

`o`, `O` and `B` print “saved” when the change is only marked for saving (section 12.8 (p. 302)). The keys `,` and `.`, which once stepped the needle’s top speed, were removed; they now fall under “ignored”.

D.2 Audio board

How to reach it. Only through the audio board’s own USB cable, at 115 200 baud. The cabinet is shut in normal use, so these are bench keys; the portal cannot reach them. The audio board’s important lines also reach the main board’s console, prefixed `[A32]`.

Table 161. D.2 Audio board

Key	What it does	Where
`+` / `-`	Volume up / down by 10. `+` also un-mutes.	10.5, 7.4.2
`m`	Mute.	10.5.5, 7.4.2
`b`	Pair, on the Bluetooth source only, like the pair button: drops the connected phone (pausing it first) and opens the pairing window. On any other source it does nothing but print `not in BT mode`.	10.7.2, 10.7.4
`v`	The front knob on or off.	Appendix G (p. 344), G.2
`p` / `c` / `P`	Store the knob’s minimum / centre / maximum reading: the knob calibration.	10.5.1
`t` / `T`	The volume law (`taper`) down / up by 0.1.	10.5.2
`r`	A raw knob stream at 10 Hz.	Appendix G (p. 344), G.2
`d`	Cycle the constant DC word. The first press selects `kDc[1]` (`0x00FF0000`), because the start-up word is `kDc[0]`.	Appendix G (p. 344), G.2; Appendix H (p. 348)
`k`	Cycle the low-bit output mask: 0, 1, 2, 4, 6, 8, 12, 16 bits.	Appendix G (p. 344), G.2
`g`	Cycle the data-line pad drive, 0 to 3. Drives 0 and 1 may mis-decode for long stretches, as with `sys.dindrv`: only with someone present and the amplifier down.	Appendix G (p. 344), G.2
`s`	Status: the knob reading and its calibration; the I2S fault latches, decoded by name; the link and the awake state; every pad and register, read back from the hardware; the peaks, the ring, the underruns, and `ESP.getFreeHeap()`. It reads and resets the peaks, the ring’s high-water mark and the I2S latches, so prefer the portal for those.	9.5, 10.9, 12.6

Notes.

- The help line the audio board prints at start-up does not list `k` or `g` both work (section 12.5 (p. 297)).
- The volume law (`t`, `T`) and the knob calibration (`p`, `c`, `P`) set here are saved on the audio board, but the main board’s copy is not refreshed: the next portal edit of any audio row sends the older volume law back. Re-read the A32 refreshes the copy (section 12.5 (p. 297)).
- The key `y`, which cycled the data-line output delay, has been removed, because one of its positions sent the radio to full volume (chapter 10 (p. 256), section 10.15.3).

Appendix E. Portal map: tabs, cards, buttons and the calls they make

This is the portal page, tab by tab, with the HTTP call behind every control. Use it to find where a control lives, or which call to make when you script the radio. Chapter 8, sections 8.7 (p. 202) and 8.8, explain how the page works; Appendix B (p. 321) lists every endpoint and Appendix C (p. 328) every action.

Settings widgets are not listed one by one. The page draws them from the schema the radio sends: each setting row becomes a checkbox, a drop-down, or a slider with a number box, on the tab its row names. Every one of them posts `POST /api/set` with its key (`k`) and value (`v`); the value in the reply, possibly clamped, is written back into every widget with that key. The settings themselves are in Appendix A (p. 309).

Who sees what. A normal account (guest) sees every tab except Console, and every value, but its changes are refused except for the guest controls (chapter 8 (p. 186), section 8.6.1 (p. 199)). In the tables, "(admin)" marks what needs the administrator.

E.1 Before the tabs

Table 162. E.1 Before the tabs

Where	Control	Call
Sign-in form	Sign in	`POST /api/login` (`u`, `p`), then `GET /api/boot`
Set your account (shown only while the account is the placeholder)	Save	`POST /api/passwd` (`old` = the placeholder password, `new`, `name`); then drops the cached schema and reloads
The page itself	(loading)	`GET /` (usually 304), then `GET /api/boot?sc=<tag>` then `GET /api/state` every 5 s (chapter 8 (p. 186), section 8.7.2 (p. 205))

E.2 The tabs, in order

Table 163. E.2 The tabs, in order

Tab	What it holds
Now	The live dashboard and the Music card.
Audio	Its settings from the schema; an Actions card (admin).
Display	Its settings from the schema; an Actions card (admin).
Lights	Its settings from the schema.
Needle	Its settings from the schema; an Actions card (admin).
Bluetooth	Its settings from the schema; an Actions card.
System	Its settings from the schema; the Settings file, Network, Firmware, Machine and Actions cards (admin).
Console (admin)	The S3's console: its output, and one key per send.
Account	Change your password; add and remove normal users (admin); sign out.

The settings tabs fetch `GET /api/values` when you open them, not on a timer.

E.3 Controls and their calls

Table 164. E.3 Controls and their calls

Tab, card	Control	Call
Now, Music	Play, Pause, ◂◂, ▸▸, Disconnect, Pair	`/api/act` `bt.play`, `bt.pause`, `bt.prev`, `bt.next`, `bt.disconnect`, `bt.pair`
Now, Music	Volume slider, Mute	`/api/set` `volume`, `muted`. Both follow the physical knob through the live state, except while the slider has focus.
Audio, Actions (admin)	Pot: minimum / centre / maximum Re-read the A32	`/api/act` `pot.min`, `pot.ctr`, `pot.max` `sys.getcfg`
Needle, Actions (admin)	Home Re-index Sweep once / repeatedly Track the tuner STOP	`/api/act` `needle.home` `needle.reindex` `needle.sweep` 0 / 1; `needle.track` `needle.stop`
Needle, Actions (admin)	◂ 100, ◂ 10, 10 ▸, 100 ▸	`/api/act` `needle.jog` −100, −10, +10, +100
Needle, Actions (admin)	Set low / high limit here Needle Limit ◂ 20 / 20 ▸	`/api/act` `needle.limitLow` / `needle.limitHigh` `needle.nudge` −20 / +20
Needle, Actions (admin)	Calibrate the index Abort calibration	`/api/act` `needle.calBand` 3; `needle.calAbort`
Needle, Actions (admin)	Tuner = low / high end	`/api/act` `needle.tuneLow` / `needle.tuneHigh`
Needle, Actions (admin)	Mark station A / B / C here (asks for MHz); Clear all marks	`/api/act` `tune.markA` / `tune.markB` / `tune.markC` `tune.clear`
Needle, Actions (admin)	Dial −0.1 / +0.1 MHz Reset the dial correction to 0	`/api/act` `tune.nudge` −0.1 / +0.1; `tune.nudgeZero`
Needle, Actions (admin)	Measure this dial position Drop one sample (asks for a slot); Forget the fixed features	`/api/act` `rda.sample` `tune.drop` `rda.spurClear`
Display, Actions (admin)	Show 8888 / Blank / Normal	`/api/act` `disp.test` 1 / 2 / 0
Bluetooth, Actions	Pair a new device Disconnect Forget every pairing (admin)	`/api/act` `bt.pair` `bt.disconnect` `bt.forget`
System, Settings file (admin)	Download	`GET /api/settings.txt`. While settings are locked, the page refuses with a toast.
System, Settings file (admin)	Upload what is below (the text box)	`POST /api/settings.txt` (raw text; 20 s deadline)
System, Network (admin)	(the form fills itself)	`GET /api/net`
System, Network (admin)	Save network (disabled until the form is filled)	`POST /api/net` (`ssid`, `pass`, `ntp`, `tz`)
System, Firmware (admin)	Update the main board Update the audio board	`POST /api/ota/s3` / `POST /api/ota/a32`, form field `f`, sent with `XMLHttpRequest` and a progress bar
System, Machine (admin)	Save settings now	`/api/act` `sys.save`
System, Machine (admin)	WiFi off for 2 minutes	`/api/act` `sys.quiet` 120
System, Machine (admin)	Constant DC test ON / off Zero-data floor ON / off Sign-extended tail ON / off	`/api/act` `sys.dctest` 1 / 0; `sys.zfloor` 1 / 0; `sys.tail` 1 / 0
System, Machine (admin)	Ignore the volume pot / Volume pot back on	`/api/act` `sys.pot` 0 / 1
System, Machine (admin)	Reboot the audio board	`/api/act` `sys.rebootA32`

Tab, card	Control	Call
System, Machine (admin)	Reboot the main board (asks to confirm)	`POST /api/reboot`
System, Actions (admin)	Raise the rescue access point	`/api/act net.forceAp`
Console (admin)	Send, or Enter	`POST /api/cons` (`k`); the output arrives through `GET /api/state?log=`
Account	Change it (my password)	`POST /api/passwd` (`old`, `new`)
Account (admin)	Add a normal user Remove	`POST /api/user/add` (`n`, `p`); `POST /api/user/del` (`i`)
Account	Sign out	`POST /api/logout`

E.4 What has no control on the page

Table 165. E.4 What has no control on the page

Call	Reached by
`POST /api/passwd` with a target slot `i` (the administrator sets another account's password without the old one)	A direct request, signed in as the administrator. From the page, remove the account and add it again.
`/api/act sys.bttx`, `sys.dcword`, `sys.dindrv`, `sys.clkpair`, `sys.clkdrv`	A direct request, signed in as the administrator (Appendix C (p. 328)).
`/api/act needle.jog`, `needle.nudge`, `tune.nudge` with other values than the buttons send	A direct request, within the action's bounds (Appendix C (p. 328)).

Appendix F. Link reference: constants, messages and payloads

This appendix is for anyone who changes the link's code or decodes the link with a logic analyser. To use or repair the radio you need only chapter 9 (p. 224), which explains how these values are used.

It lists everything that travels on the link between the main board (S3) and the audio board (A32): the frame, the constants, every message and every payload. None of these values is a portal setting. All are compiled in; changing one means rebuilding, and for those in `include/proto.h`, rebuilding and updating both boards (chapter 9 (p. 224), section 9.14 (p. 249)).

F.1 The frame

Table 166. F.1 The frame

Bytes	Field	Value
0–1	start marker	`0xA5 0x5A` (`PROTO_SOF0`, `PROTO_SOF1`); not covered by the checksum
2	version	`PROTO_VERSION`, 4
3	type	the message id (section F (p. 338).3)
4–5	length	payload length, little-endian, 0 to 1088
6 … 5 + length	payload	the message's structure (section F (p. 338).4)
last 2	checksum	CRC-16/CCITT-FALSE (polynomial 0x1021, initial value 0xFFFF, no final XOR), computed bit by bit over the version, type, length and payload; sent little-endian. The ASCII string `123456789` gives 0x29B1.

A frame is 8 bytes plus its payload. The line carries about 92 kB/s at 921 600 baud, 8N1. Worked examples of whole frames are in chapter 9 (p. 224), section 9.6.1 (p. 233).

Table 167. F.1 The frame

Message	Payload, bytes	Frame, bytes
HELLO, HELLO_ACK	25	33
STATE	130	138
CFG	33	41
TIME, SET_TIME	8	16
SET_AUDIO	15	23
SET_BT	12	20
SET_SYS	4	12
OTA_STATUS	38	46
OTA_DATA	6 + up to 1024	up to 1038
empty messages	0	8

F.2 Constants

Table 168. F.2 Constants

Name	Value	Meaning	Where
`PROTO_VERSION`	4 (0–255)	The version byte in every frame. Change it only by the procedure in chapter 9 (p. 224), section 9.14.5. Version 2 is when the audio settings gained their fades.	`include/proto.h`
`PROTO_SOF0`, `PROTO_SOF1`	0xA5, 0x5A	The start marker.	`include/proto.h`
`PROTO_MAX_PAYLOAD`	1088 bytes	The largest payload, and the size of the decoder's buffer. Must hold every payload structure (checked when compiling).	`include/proto.h`
`PROTO_BAUD`	921 600 baud	Link speed, 8N1.	`include/proto.h`
`PROTO_STATE_MS`	250 ms	The A32's status report period.	`include/proto.h`
`PROTO_SILENCE_MS`	2000 ms	How long without a valid frame before a board counts the other as silent; both boards.	`include/proto.h`
`PROTO_VERSION_LEN`	24 bytes	The version string field in HELLO and OTA_BEGIN.	`include/proto.h`
Receive buffer	4096 bytes	The serial receive buffer, set before the port opens.	`Link::begin()`
HELLO / PING period	1000 ms	HELLO until answered, then PING.	S3 `loop()`
SET_SYS period	2000 ms	The amplifier state to the A32; also sent at once on a change.	S3 `loop()`
GET_TIME period	60 000 ms	Re-setting the S3's clock from the battery clock.	S3 `loop()`
Battery-clock answer wait	5000 ms	A periodic GET_TIME unanswered this long, while handshaken, sets `rtc` = 2.	S3 `loop()`
Network time to the battery clock	at most once per 3 600 000 ms	Only while handshaken and network time is fresh.	S3 `loop()`
Restart-test slack	5000 ms	The A32's millisecond count must fall back by more than this; earlier values from 0xF0000000 up are ignored.	S3 `onMessage()`

Name	Value	Meaning	Where
BEGIN answer wait	20 000 ms	The S3 waits for the A32 to accept an update.	`a32OtaBegin()`
DATA answer wait	3000 ms × 4 tries	Per frame.	`otaSendFrame()`
END answer wait	4000 ms	State 3 or 4 required.	`a32OtaEnd()`
Upload silence	15 000 ms	No upload activity this long: ABORT to the A32.	`otaWatchdog()`, `src/s3/portal.cpp`
Update frame	1024 bytes	Data per `MSG_OTA_DATA`.	S3 `otaBuf`
Mute settle before flash	400 ms	From muting to the first flash write. Must exceed the mute, the fade-out and the output buffers' drain.	A32 `MSG_OTA_BEGIN` handler
A32 update give-up	20 000 ms	Since the last accepted DATA frame.	A32 `loop()`
Restart delay after success	250 ms	Lets the final status go out.	A32 `MSG_OTA_END` handler
`WDT_TIMEOUT_S`	15 s	The A32's task watchdog, with a restart. Must exceed start-up and the longest pass of the main loop.	`src/a32/main.cpp`
`CONFIRM_AFTER_HELLO_MS`	60 000 ms	Trial confirmation, counted from the first HELLO.	`src/a32/main.cpp`
`CONFIRM_AFTER_MS`	300 000 ms	Fallback confirmation when nothing at all was ever received.	`src/a32/main.cpp`
Confirmation retry	10 000 ms	After a failed confirmation.	`confirmTick()`
A32 settings save wait	2000 ms	After the last change; skipped by a forced save (Reboot the audio board, confirmation).	`settingsFlush()`
A32 settings writes on trial	none	Changes wait in RAM; written at confirmation.	`settingsFlush()`
`protoBroken` announcement	10 000 ms	On the console and as a LOG line.	A32 `loop()`
Battery clock	address 0x68; registers 0x00, 0x0F, 0x11; 100 kHz	Chapter 9, section 9.9 (p. 240).	`src/a32/rtc.h`, `include/pins.h`
Battery-clock plausibility floor	1 600 000 000 s (Unix)	September 2020; used on both boards.	`rtc.h`, S3 `onMessage()`
A32 partitions	`min_spiffs.csv`: two 1.875 MB program slots	Two slots are needed for both the relay and the rollback (chapter 3 (p. 25), section 3.16 (p. 41)).	`platformio.ini`
`FW_VERSION`	`v.1.YYYYMMDDTHHMMSS`	Which binary; fits in 24 bytes; stamped at build time (chapter 3 (p. 25), section 3.13 (p. 37)). Sent in HELLO and HELLO_ACK.	build
`FW_COMMIT`	short git hash, for example `a1b2c3d` 7 characters, plus `-dirty` or `nogit`	Which source; stamped at build time. Never sent in the handshake.	build

F.3 Messages

Numbering. The ids come in blocks: 0x0n handshake, 0x1n S3-to-A32 control, 0x2n A32-to-S3 reporting, 0x3n update relay, 0x4n S3-to-A32 diagnostics (opened because the 0x1n block is full at 0x1F). The numbers are explicit “so a mismatch between builds is diagnosable from a capture”. A retired id is never reused. Every handler ignores an id it does not know.

Table 169. F.3 Messages

Id	Name	Direction	Payload	Sent by, when	Receiver does
0x01	`MSG_HELLO`	S3 to A32	`ProtoHello` (25)	S3, every 1 s while not handshaken	Notes the S3′s version; answers HELLO_ACK; sends the boot report (once per start-up); starts the 60 s confirmation clock (once).
0x02	`MSG_HELLO_ACK`	A32 to S3	`ProtoHello` (25)	Answer to HELLO	Marks the handshake done; prints `[PASS] A32 up: proto vN, firmware X` sends GET_TIME and GET_CFG.
0x03	`MSG_PING`	S3 to A32	empty	S3, every 1 s while handshaken; the S3′s live index monitor on each pass	Answers PONG.
0x04	`MSG_PONG`	A32 to S3	empty	Answer to PING	Ignored by the S3.
0x10	`MSG_SET_AUDIO`	S3 to A32	`ProtoAudio` (15)	A portal edit of an A32 audio setting; a settings upload (once, at the end)	Applies; saves 2 s later (on trial: at confirmation).
0x11	`MSG_SET_BT`	S3 to A32	`ProtoBtCfg` (12)	A portal edit of an A32 Bluetooth setting; a settings upload	Applies; saves 2 s later (on trial: at confirmation).
0x12	`MSG_SET_SYS`	S3 to A32	`ProtoSys` (4)	S3, every 2 s while handshaken, and at once when the amplifier state changes	Stores the amplifier state; recomputes visibility.
0x13	`MSG_SET_TIME`	S3 to A32	`ProtoTime` (8)	Fresh network time (at most hourly, handshaken); console `w` (handshaken only)	Writes the battery clock; clears OSF.
0x14	`MSG_GET_TIME`	S3 to A32	empty	At HELLO_ACK, then every 60 s	Reads the battery clock and answers TIME; sends nothing if the chip is silent (the S3 then sets `rtc` = 2).
0x15	`MSG_BT_FORGET`	S3 to A32	empty	Portal action `bt.forget`	Removes every Bluetooth pairing.
0x16	`MSG_BT_LOOK`	S3 to A32	empty	Portal action `bt.pair`	If the source is Bluetooth: drops the phone and opens pairing.
0x17	`MSG_REBOOT`	S3 to A32	empty	Portal action `sys.rebootA32`	Saves pending settings at once (nothing while on trial), then restarts.
0x18	`MSG_CAL_POT`	S3 to A32	uint8: 0 minimum, 1 maximum, 2 centre	Portal actions `pot.min`, `pot.max`, `pot.ctr`	Records that knob calibration point.
0x19	`MSG_BT_CMD`	S3 to A32	uint8: 0 play, 1 pause, 2 next, 3	Portal actions `bt.play`, `bt.pause`, `bt.next`, `bt.prev`, `bt.disconnect`	Sends the command to the phone (AVRCP); a disconnect is graceful.

Id	Name	Direction	Payload	Sent by, when	Receiver does
			previous, 4 disconnect		
0x1A	MSG_GET_CFG	S3 to A32	empty	At HELLO_ACK; portal action sys.getcfg	Answers CFG.
0x1B	MSG_TEST_DC	S3 to A32	uint8 0/1	Portal action sys.dctest	Constant-DC output test on or off.
0x1C	MSG_ADC_CLOCK	—	—	Retired nothing sends it	No handler.
0x1D	MSG_ZERO_FLOOR	S3 to A32	uint8 0/1	Portal action sys.zfloor	Zero-data floor on or off.
0x1E	MSG_USE_POT	S3 to A32	uint8 0/1	Portal action sys.pot	The knob drives the volume, or is ignored.
0x20	MSG_STATE	A32 to S3	ProtoState (130)	A32, every 250 ms, unprompted, from start-up	Fills the S3′s picture of the A32′s status; restart test.
0x21	MSG_TIME	A32 to S3	ProtoTime (8)	Answer to GET_TIME	Re-sets the S3′s clock if valid.
0x22	MSG_LOG	A32 to S3	text, no terminating zero	The boot report; the confirmation result; settings held during the trial now saved; the protoBroken announcement every 10 s; a failed settings save; a Bluetooth disconnect that did not land	Printed as [A32] <text>.
0x23	MSG_CFG	A32 to S3	ProtoCfgAll (33)	Answer to GET_CFG	Fills the settings mirror; the A32′s settings are known.
0x30	MSG_OTA_BEGIN	S3 to A32	ProtoOtaBegin (32)	Start of a relayed update	Refuses on trial; otherwise mutes and starts receiving.
0x31	MSG_OTA_DATA	S3 to A32	6 + length bytes (ProtoOtaData, read at fixed offsets)	Each 1 KB frame of the relay	Commits the bytes if the offset matches; always answers OTA_STATUS.
0x32	MSG_OTA_END	S3 to A32	uint32: the CRC32 of the image	End of the relay	Compares the CRC32, finishes, answers, restarts.
0x33	MSG_OTA_ABORT	S3 to A32	empty	A wrong chip; an unanswered END; an aborted upload; 15 s of upload silence	If an update is running: gives up and restores the user's mute.
0x34	MSG_OTA_STATUS	A32 to S3	ProtoOtaStatus (38)	After every BEGIN, DATA, END and give-up	Fills the relay's answer fields.
0x40	MSG_DIN_DRIVE	S3 to A32	uint8 0..3	Portal action sys.dindrv	Drive strength of the DAC data pad.
0x41	MSG_SD_DELAY	—	—	Retired nothing sends it	No handler.
0x42	MSG_SIGN_TAIL	S3 to A32	uint8 0/1	Portal action sys.tail	Sign-extended tail on or off.
0x43	MSG_CLK_DRIVE	S3 to A32	uint8 2..3	Portal action sys.clkdrv	Drive of all three clock pads.
0x44	MSG_DC_WORD	S3 to A32	uint8 index 0..6	Portal action sys.dcword	Picks one of seven fixed DC words, all at or below −42 dBFS; never a raw word; index 7 and up ignored.
0x45	MSG_BT_TX	S3 to A32	uint8 0..5 (ESP_PWR_LVL_N12 .. P3)	Portal action sys.bttx	Bluetooth transmit power. Values above P3 are ignored, so this path can only turn it down from the default maximum.

Id	Name	Direction	Payload	Sent by, when	Receiver does
0x46	`MSG_CLK_PAIR`	S3 to A32	uint8[2]: MCLK drive, then BCK/LRCK drive, each 2..3	Portal action `sys.clkpair`	Drives the two clock groups separately.

The diagnostics (0x1B, 0x1D, 0x1E, 0x40 to 0x46) belong to the pop hunt; chapter 10 (p. 256) explains them.

F.4 Payloads

All payload structures are packed and copied byte for byte (chapter 9 (p. 224), section 9.6.1 (p. 233)); they live in `include/proto.h`. Sizes are in bytes.

Table 170. F.4 Payloads

Structure	Size	Fields
`ProtoHello`	25	`protoVersion` (u8), `fwVersion[24]`
`ProtoAudio`	15	`volume`, `muted` (the software mute only), `balance`, `gainRadio` and `gainBt` (tenths of a dB), `monoSum`, `muteOnChange`, `fadeInMs`, `fadeOutMs`, `taperX10` (chapter 10 (p. 256) explains each)
`ProtoBtCfg`	12	`connectable`, `autoConnect`, `lookTimeoutS`, `ledBrightness[7]`, `pauseOnLeave`
`ProtoSys`	4	`ampOn` (1 = on), `reserved[3]`
`ProtoTime`	8	`unixUtc` (u32, always UTC), `valid` (u8), `tempC4` (i8, quarter degrees), `reserved[2]` the A32 puts the battery clock's OSF bit in `reserved[0]`
`ProtoCfgAll`	33	`ProtoAudio`, `ProtoBtCfg`, `potMin`, `potMid`, `potMax`
`ProtoState`	130	source, Bluetooth state, volume, muted, peaks left and right, AVRCP volume, clipped, the raw selector reading, `peerName[24]`, underruns, ring-buffer fill, the zero-data watch, the volume watch, pad drives read back, the DC test and its word, **`a32Ms`** (the A32's millisecond count when the frame was built, used for the restart test and to place events in time), the last stall, the Bluetooth transmit power read back, the three clock pads' drives, RMS left and right
`ProtoOtaBegin`	32	`size` and `crc32` (both sent as 0: the image is streamed), `fwVersion[24]` (the S3 fills it with its own version)
`ProtoOtaData`	1030	`offset` (u32, absolute), `len` (u16), `data[1024]` only `len` bytes of data are sent
`ProtoOtaStatus`	38	`received` (bytes committed), `state` (1 receiving, 3 ok, 4 failed; 0 idle and 2 verifying are defined but never sent), `errCode` (the updater's error number), `detail[32]`

The updater's error numbers carried in `errCode` are listed in chapter 3 (p. 25), section 3.10.3 (p. 30).

Checks at compile time.

- Every one of the ten structures above is checked to fit in `PROTO_MAX_PAYLOAD`. "A struct that does not fit is a fact about the SOURCE, so the build is where it belongs." All ten are listed, because "the one nobody thought to check is the one that grows."
- The update data structure's layout is pinned: `offset` at byte 0, `len` at byte 4, `data` at byte 6. The A32 reads DATA frames at those offsets by hand.

F.5 What the link prints

These lines appear on the S3′s console (and its Console tab on the portal). Lines from the A32 itself carry the `[A32]` prefix.

Table 171. F.5 What the link prints

Line	Meaning	Explained in
`[PASS] protocol v4, state frame 138 bytes, CRC rejects damage`	The protocol self-test passed at start-up. Both boards print it, each on its own console.	chapter 9 (p. 224), section 9.14.4
`[PASS] A32 up: proto vN, firmware <version>`	Handshake done; the A32′s version.	chapter 9 (p. 224), section 9.7 (p. 236)
`[A32] boot: commit <hash>, reset reason N, setup X ms, image <state>, watchdog on`	The A32′s boot report, once per A32 start-up.	chapter 9 (p. 224), section 9.3 (p. 228)
`[WARN] A32 silent. Clock and needle keep running.`	Nothing valid from the A32 for 2 s.	chapter 9 (p. 224), sections 9.7 (p. 236) and 9.10
`[WARN] the A32 restarted - handshaking again.`	The A32′s millisecond count went backwards.	chapter 9 (p. 224), section 9.7 (p. 236)
`[FAIL] HELLO_ACK size - version skew`	A HELLO_ACK of the wrong size.	chapter 9 (p. 224), section 9.7 (p. 236)
`[WARN] the A32's STATE frame does not match this build. Flash BOTH MCUs. Audio is unaffected - only telemetry stops.`	A status report of the wrong length.	chapter 9 (p. 224), section 9.7 (p. 236)
`link : ... wrong-version N  <- the boards run different protocol versions: flash both`	Frames of another protocol version (console `s`).	chapter 9 (p. 224), section 9.5 (p. 231)
`[WARN] the DS3231 has no valid time yet.`	The battery clock answered with no valid time.	chapter 4 (p. 46), section 4.9 (p. 57)
`[A32] image confirmed (a minute of running with the S3)` or `(five minutes running, the S3 never spoke)`	The A32′s new program is permanent.	chapter 3 (p. 25), section 3.11 (p. 32)
`[A32] image confirm FAILED (err N) - retrying`	The confirmation write failed.	chapter 3 (p. 25), section 3.11.2 (p. 33)
`[A32] settings changed during the trial are now saved`	Held settings written at confirmation.	chapter 3 (p. 25), section 3.11.5 (p. 34)
`[A32] settings NOT saved - the flash write failed; retrying`	An A32 settings write fell short.	chapter 7 (p. 153), section 7.4.2 (p. 161)
`[A32] proto self-test FAILED - muted, BT closed, awaiting OTA`	A broken A32 build; every 10 s.	chapter 9 (p. 224), section 9.2.1 (p. 226)

Appendix G. Audio constants and experiments

This appendix is mainly for anyone changing the audio board's firmware, or running the pop-hunt experiments again. It lists the numbers compiled into the A32′s audio and Bluetooth firmware, and the experiments that can change its behaviour until the next restart. Chapter 10 explains what each one does. The stored audio and Bluetooth settings, which a user can change and keep, are in Appendix A (p. 309) every A32 console key is in Appendix D (p. 332).

G.1 Compiled constants

Table 172. G.1 Compiled constants

Name	Value	Unit	Meaning	Where
`SAMPLE_RATE`	44 100	Hz	Fixed; there is no resampler.	`audio.cpp`
`BLOCK`	256	frames	One pass of the audio task, about 5.8 ms.	`audio.cpp`
`RING_FRAMES`	8192	frames	The Bluetooth ring: about 186 ms, 32 KB.	`audio.cpp`
`dma_buf_count` / `dma_buf_len`	8 / 256	buffers / frames	46.4 ms per direction, 32 KB for both directions. Do not raise without measuring (chapter 10 (p. 256), section 10.8 (p. 275)).	`Audio::begin()`
MCLK	11.2896	MHz	256 × 44 100, from the APLL.	`Audio::begin()`
Clock-pad drive at start-up	3	0–3	GPIO0, 18 and 17. The remote setting clamps to 2–3.	`Audio::begin()`
Data-pad drive	the power-on default	0–3	Never set at start-up.	—
`tx_sd_out_delay`	never written; read back only (`sddly`)	0–3	Hazard: see chapter 10 (p. 256), section 10.15.3. Nothing can write it.	`sdOutDelayIs()`
Audio task	core 1, priority 6, 4096 B stack			`Audio::begin()`
`ENV_ONE`	2^24	Q24	Envelope unity.	`audio.cpp`
Volume-law clamp	10–40	gamma × 10	Gamma 1.0 to 4.0.	`setTaper()`
Balance step	655	Q16 per unit	±100 leaves about −65 dB.	`audioTask()`
`ZDD_FRAMES`	1024	frames	The DAC's zero-data window, 23.22 ms.	`audio.cpp`
`ZDD_LSB`	`0x00000100`	word	One 24-bit step, −138.5 dBFS: the zero-data floor's word.	`audio.cpp`
`ZDD_STALL_US`	23 220	µs	A gap between writes above this is a stall.	`audio.cpp`
`DEVICE_NAME`	"Ambersong"		The Bluetooth name.	`main.cpp`
`BT_DROP_TIMEOUT_MS`	3000	ms	How long a disconnect may stay in flight; the spacing of stray-phone drops.	`main.cpp`
Pause-to-disconnect gap	250	ms		`gracefulDisconnect(`
`BT_LOOK` → `BT_STDBY`	20 000	ms		`updateBtState()`
`btTxLevel`	`ESP_PWR_LVL_N0` (0 dBm)	level 0–5 = −12 to +3 dBm	The Bluetooth transmit ceiling at start-up; the floor is 0 dBm.	`main.cpp`
`kDc[7]`	`0xFF000000`, `0x00FF0000`, `0xFFFF0000`, `0x00FFFFFF`, `0x000000FF`, `0x00100000`, `0x001FFFFF`	words	The constant-DC diagnostic's words, all at or below −42 dBFS.	`main.cpp`
DC word at start-up	`0xFF000000`		The proven popper (Appendix H (p. 348)).	`audio.cpp`
Update mute wait	400	ms	Before the A32 starts writing its flash (chapter 3 (p. 25), section 3.10.3 (p. 30)).	`onMessage()`

Name	Value	Unit	Meaning	Where
Update give-up	20 000	ms	No frame from the S3.	`loop()`
Source and knob reading	50 / 50	ms		`loop()`
`POT_DEADBAND`	24	ADC counts	About 0.6 % of the knob's travel.	`main.cpp`
Selector thresholds	424 / 2471	raw ADC	Below 424 AUX; 424 to 2470 BT; 2471 and above RADIO.	`pins.h`
`PROTO_STATE_MS`	250	ms	The state message to the S3.	`proto.h`
`PROTO_SILENCE_MS`	2000	ms	The S3 is considered gone, and the A32 sleeps.	`proto.h`
Settings save delay	2000	ms	After the last change (chapter 7 (p. 153), section 7.4.2 (p. 161)).	`settingsFlush()`
Pair button	counted on release, after more than 30 ms held		Only on the BT source.	`loop()`
Blue lamp PWM	LEDC channel 0, 2 kHz, 8-bit			`btled.cpp`
Knob calibration fallback	60 and 3990, no centre	raw ADC	Used when the stored maximum is not more than 64 counts above the minimum.	`potRotation()`

G.2 Experiments

Experiments are actions, not settings: none survives a restart of the A32. The reason: "an experiment that survives a reboot is one whose result gets attributed to something else a week later." Every one is read back from the hardware and shown on the portal (chapter 10 (p. 256), section 10.9 (p. 276)).

From the portal (administrator; every portal action is in Appendix C (p. 328)):

Table 173. G.2 Experiments

Portal action (message)	Effect on the A32	Range	Notes
`sys.clkdrv` (`MSG_CLK_DRIVE`)	Drive of the clock pads GPIO0, 18 and 17 together.	clamped to 2–3	The pop knob; start-up value 3.
`sys.clkpair` (`MSG_CLK_PAIR`)	MCLK drive, then BCK/LRCK drive.	22, 23, 32 or 33 only; others refused	Never run on the radio.
`sys.dindrv` (`MSG_DIN_DRIVE`)	Data-line pad drive.	0–3	0 and 1 may mis-decode for long stretches: use it with someone present and the amplifier down.
`sys.dctest` (`MSG_TEST_DC`)	Replace the output with the DC word.	0/1	Silent as audio; anything heard is the fault.
`sys.dcword` (`MSG_DC_WORD`)	Pick `kDc[index]`.	0–6; others refused	An index, never a raw word, so nothing louder than −42 dBFS can be sent.
`sys.zfloor` (`MSG_ZERO_FLOOR`)	The zero-data floor (below).	0/1	
`sys.tail` (`MSG_SIGN_TAIL`)	The sign-extended tail: the last 8 bits of each word take the sign of the next word.	0/1	Ruled out as a pop fix.
`sys.pot` (`MSG_USE_POT`)	Ignore the front knob (0) or use it (1).	0/1	
`sys.bttx` (`MSG_BT_TX`)	The Bluetooth transmit ceiling.	0–5 (−12 to +3 dBm); others refused	Down only; 5 restores the stock range.
retired `MSG_ADC_CLOCK`	Ignored.		The ADC master-clock toggle, removed.
retired `MSG_SD_DELAY`	Ignored.		The data-line output delay, removed.

From the A32′s USB console only (115 200 baud, on the A32′s own cable):

Table 174. G.2 Experiments

Key	Effect
`d`	Cycle the DC word. The first press selects `kDc[1]` (`0x00FF0000`), because the start-up word is `kDc[0]`.
`k`	Cycle the low-bit output mask: 0, 1, 2, 4, 6, 8, 12, 16 bits. Not listed in the start-up help line.
`g`	Cycle the data-line drive 0–3. Not listed in the start-up help line. Same hazard as `sys.dindrv`.
`v`	The knob on or off.
`r`	A raw knob stream at 10 Hz.

The key `y`, which cycled `tx_sd_out_delay`, has been removed.

The diagnostics, in the order the audio task applies them (all off by default):

Table 175. G.2 Experiments

Diagnostic	What it does	What it was for
Constant DC word	Replaces every sample with a fixed word, silent as audio.	Anything heard is the fault. `0xFF000000` is the proven popper and `0x00FF0000` its matched silent control (Appendix H (p. 348)).
Low-bit output mask	Zeroes the bottom bits of each word.	Keep the data line still at the word boundary. Never confirmed by ear.
Zero-data floor	Keeps the DAC's output stage alive on a constant word, `ZDD_LSB` (one 24-bit step, −138.5 dBFS).	Keep the DAC's own zero-data mute off, on the theory that its release made the pop. Excluded (Appendix H (p. 348)).
Sign-extended tail	Sets the last 8 bits of each word to the sign of the next word.	Stop the data line moving across a word boundary. With it on, a word of 1 to 255 followed by a non-negative word becomes exact zero.

Appendix H. The pop hunt in detail

For about two weeks the radio made pops: loud, short clicks through the speakers, at irregular times, on RADIO and on BT, never on AUX. This appendix keeps every test and every measured number, organised by the question each one answered, so that the next person to hear a click does not start from zero. It ends with the method for counting pops. Chapter 10, section 10.10 (p. 277), has the summary. You need this appendix only if the clicks come back, or before you change anything in the audio clocks.

Every rate here is given with its window. A bare "zero" is never written: zero events in a window is a bound, not a cure.

The raw evidence no longer exists. The recordings and the analysis scripts behind every number here were not kept, and neither were the test tools. Only the numbers survive. Anyone who needs such tools again will have to remake them; section H (p. 348).8 says how they worked.

H.1 What a pop is

What you hear. A loud click, in bursts and alone. Any A32 volume other than 0 produces pops; at a volume of 8 they were plainly audible.

Sizes. Pop sizes fell on a ladder of binary weights: ¼, ½, ⅝, ¾, 1 and 3/2 of a main size (79 events over three soaks, each rung within about 0.2 to 0.4 dB, no clipping). Polarity was mostly mixed, but the 3/2 rung came out negative 9 times in 9. That reads as a few of the top bits of single words coming out wrong with the rest intact: neither a framing slip (it would smear) nor the DAC's own mute (one size, one polarity).

In one evening's recordings the sizes clustered in three families about 1 dB wide: −7.2, −9.2 and −25.2 dBFS at the microphone (31 events). The quiet family's shape correlated only 0.41 to 0.46 with the loud one's, but clips of both were listened to and called pops, so the detector is calibrated by ear. A low correlation is not evidence of "not a pop".

Pops do not need the data to change. On a constant word (`0xFF000000`, 5 minutes, Bluetooth source, tube set out, A32 volume 7, amplifier at maximum) the radio made 6 isolated pops. They fell into two populations. 2 of the 6 matched the recorded step of the switch into the DC word (correlation 0.75 and 0.88) and sat inside a burst of stalls: they were stall clicks. The other 4 were short impulses, not steps, the same sizes as in music. A near-zero word had never popped and this −42 dBFS word did, so whether a pop is visible depends on the word's value. That fits the DAC misreading bits, not noise added in the analogue path.

Not every click is a pop.

Table 176. H.1 What a pop is

Click	Shape	How to tell
A pop	an impulse; it rings	no stall near it
A stall click	a step	the stall counter moved; it correlates with a recorded switch step
A reboot or update click	one click	the A32′s clocks stopped; check the flash log
A crackle storm	dense detections for 35 to 60 s, peaks near −3 dBFS	many detections bunched together, not isolated like a pop or a stall

H.2 What the pops were not

The first observations, on Bluetooth. Music over Bluetooth popped. Pausing the phone stopped the pops. Muting the phone — still streaming, so frames of zeros kept arriving — stopped them. And a phone volume a hair above zero, music inaudible, brought them back at full loudness. The mute test was a clean design: same link, packet rate, decode load, CPU and radio; only the sample values changed. The last observation ruled out every linear operation in the firmware, the underrun click included (its size is exactly the last sample before the gap): turn the phone down 40 dB and every artefact a linear stage can make drops 40 dB too. The underrun theory explained the pause and the mute perfectly, led for an hour, and died on this.

The first round of tests, each a measurement:

Table 177. H.2 What the pops were not

Test	Result	What it excluded
S3 powered off completely, A32 alone, playing	no change	WiFi, 2.4 GHz coexistence, the link, the whole second board
RADIO source	popped the same way as BT	the whole Bluetooth branch: SBC, the ring, underruns, the library's volume
AUX	clean	nothing on its own: AUX writes zeros through the same path, so it only says zeros are safe
DAC mute input held high	no change	one of the DAC's mute routes (the pin was later found to reach nothing)
Peak meter through 60 s of continuous popping on RADIO	never above −118.4 dBFS; `clip` never set	the firmware's arithmetic: the words written were clean
Pop timing	irregular and bursty: under one a second, sometimes 3 to 5 inside a second, sometimes 7 to 8 s apart	anything locked to a firmware period (the 172.3 Hz blocks, the 20 Hz knob reads, the 4 Hz state message)
ADC unpowered; then ADC powered with its I2S cable out	clean both times; with the cable out a 60 Hz hum went too	—
ADC's master clock cut in firmware, ADC still powered and wired	pops and hum gone; restored, both came back	passive loading: the converter had to be running

How could words at −118.4 dBFS, about one bit, make a full-scale bang? Shifting a word by k bit positions multiplies it by 2^k, 6 dB per bit, so reaching full scale needed k of about 16 to 20: the DAC was losing or gaining bit-clock edges mid-frame. And a shift of zero is zero, which explained why silence never popped, without any cause that looked at the content.

When the pops came back after nine quiet days (section H (p. 348).3), the microphone recorder of section H (p. 348).8 replaced the ear. The baseline was 3.00 pops/min, stable over three 10-minute blocks (2.80, 3.20, 3.00). At that time a bad knob calibration had pinned the volume near 18 of 255, with the amplifier turned up to match, so every fault arrived 62 dB louder than it should (chapter 10 (p. 256), section 10.5.1 (p. 267)).

Later exclusions:

Table 178. H.2 What the pops were not

Suspect	Test	Result
The DAC's own zero-data mute releasing with an unramped step	the zero-data witness	`zrel` did not move through 8 minutes of pops
The volume reading wandering (it did wander, between 17 and 20)	the volume watch, then the knob ignored	`gsteps` frozen, still popping
Audio-task stalls	the stall counter	`stalls` frozen and `under` at 0 during pops
The needle motor, the FM tuner chip, the 5 V supply (the set ran from a bench supply), the tube set, the box fan, the earth, a mains filter, the room, the amplifier	—	all excluded; for the amplifier, AUX was clean and the pop follows the amplifier's own volume
The DAC's clock-error mute	its datasheet says a clock error attenuates, then mutes hard for about 104 samples (about 2.4 ms), so the programme must vanish	through 37 recorded pops the streamed tone's level, measured phase-insensitively (band-limited 180–260 Hz, analytic magnitude), changed by a median of −0.3 dB; only 8 of 37 dipped past −10 dB
One fixed-size sign-bit flip	polarity and size of 31 pops	20 of 31 had the opposite polarity ($p \approx 0.15$); sizes spread over 19.1 dB

A first attempt at the clock-error test, with a coherent 6 ms average at exactly 220.0 Hz, read +4 dB: it was measuring the pop's own ring-down.

Mains is a contributor, not the cause. A heat gun on the same circuit made the radio pop from 5 cm or 10 m away; on another circuit it did nothing at any distance, so the route is conducted, not radiated. The baseline survived on a quiet circuit, so two rates were adding up. An inlet filter changed nothing.

Volume is not a variable. A result that "volume 17 brings the pops back" turned out to be one heat-gun burst. Timestamp every event before believing a rate.

Stalls and Bluetooth. Stalls once seemed to track Bluetooth: 0 in 34 minutes with nothing connected, 8 in the 7 minutes after a phone connected. That was coincidence with when streaming happened: later, 13 stalls came in about 33 minutes with the Bluetooth radio idle and the source on RADIO (`stalls` from 1 to 14 after a start), the longest 60 ms against the 46.4 ms chain. That retraction is about the stalls only, not about the pops' relation to Bluetooth: say "the stall correlation", never "the Bluetooth correlation". Stall counts, each since its own start: 25 at 21.6 hours up, and 21 at 41.9 hours up.

Pops and Bluetooth activity. The pops seemed to follow Bluetooth activity: 1 in about 30.7 minutes idle against 12 in about 6 minutes after a phone connected, the same constant word on the wire. That was not a designed A/B, and time confounds it: the phone had simply been reconnected.

H.3 Was it the data line?

The constant-DC words. The constant-DC diagnostic was given a settable word: every slot carries the same fixed word, silent as audio, so anything heard is the fault. Six words, each held at least 30 s on RADIO:

Table 179. H.3 Was it the data line?

Word	Level	Unit	Lowest set bit	Result
`0x00100000`	−66.2	dBFS	20	silent
`0x00FF0000`	−42.2	dBFS	16	silent
`0x000AAAAA`	−69.7	dBFS	1	few pops
`0x00155555`	−63.7	dBFS	0	many pops
`0x001FFFFF`	−60.2	dBFS	0	many pops
`0x000000FF`	−138.5	dBFS	0	many pops

Level, bit count and edge density (the prediction of the day) all failed to order the results. The position of the lowest set bit did, with no exception: `0x00FF0000` was silent and `0x000000FF` popped, the same 8 bits, 96 dB apart. I2S sends the most significant bit first, so bit 0 sits right against the frame-clock edge.

The reading then: activity on the data line at the word boundary made the DAC lose its framing; at that time the DAC recovered its clock from BCK by itself. Real audio has a busy lowest bit at every level, which is why the pops had not followed the music's level. The I2S fault latches were clean in eight one-second windows during popping, so the fault lay after the firmware's write. The low-bit output mask was built as a fix; it was never confirmed by ear, and of its widths only bits 0, 1 and 16 had known results. The ADC master-clock toggle was retired, and the sleep rule added.

The DAC's clock wiring was then changed so that the DAC takes its master clock from GPIO0 (Hardware Bible, chapter 6 (p. 120)). The radio played with no pops at all, and nine quiet days followed. That change also switched off the DAC's own clock recovery from BCK, which the data-line reading had blamed.

When the pops returned, the data-line story fell:

- The sign-extended tail (the last 8 bits of each word set to the sign of the next word, so the data line never moves across a word boundary) was tested blind with the heat gun, on-off-on-off. No change in rate or size, and the rate did not rebound when the tail went back off. The data-line mechanism was dropped.
- Muted with the zero-data floor on (the DAC's output stage kept alive on a constant word) the radio made 0 pops in 3 minutes, against 3.7/min playing. It was read then as "the pop needs changing data". That was over-read: its premise, that a 1-bit word keeps the DAC's zero-data mute off, was only the firmware's model of the DAC. The better reading is that the

DAC slips through its clocks whatever the data, and a slip is audible only when the word is not tiny.
- Popping on a constant word (section H (p. 348).1) settled it: the pops do not depend on the data changing.
- An A/B of **`0xFF000000`** against **`0x00FF0000`** (12 cycles of one-minute windows, Bluetooth with no phone connected, drive 2) was inconclusive: 1 against 0 events, p = 0.489, under the rule fixed before the run that fewer than 6 events is inconclusive. The chain was alive: all 23 switch thumps were detected within 0.06 s of their stamps.

The pops of the first round, which the clock-wiring change fixed, and the later ones, with sizes on a ladder, were read as two different faults.

The data-line output delay. A register that shifts the data line in time (`tx_sd_out_delay`) was added and swept live. At position 2 it sent the radio to full volume through the valve amplifier; positions 1 and 3 silenced it. The speakers survived. Nothing in the firmware writes it any more; chapter 10, section 10.15.3, carries the warning.

H.4 Was it the clocks? What cut the pops

The suspect. `setClockDrive()` sets the drive strength of the three clock pads together: MCLK (GPIO0), BCK (GPIO18) and LRCK (GPIO17). The ESP32 has four drive strengths; the power-on value is 2.

A first alternation was inconclusive, not negative. One morning, 30 pairs of one-minute windows, drive 2 against 3: the fault was dormant, 3 loud pops in the hour (drive 2: 3, drive 3: 0, p = 0.125; at a −45 dBFS threshold, 10 against 6, p = 0.22). The rate changes by the hour: earlier, four 2-minute quarters had been fooled by one burst, a p = 0.005 that vanished pair by pair.

The decisive A/B. One-minute alternation, 30 windows, a 220 Hz tone streamed over A2DP, the recorder counting, no stalls in the run:

Table 180. H.4 Was it the clocks? What cut the pops

Drive	Window	Unit	Isolated pops	Rate
2	936	s	52	3.33/min
3	932	s	0 above −35 dBFS	—

Stratified exact test p = 2.8e-16 all 15 pairs went the same way. A first scoring gave 2 events at drive 3 (p = 6.3e-14, 14 of 15 pairs). Both fell 4 s after the run's exit had already restored drive 2, and were credited to the last window only because that window closed at the end of the recording.

The soaks at drive 3:

Table 181. H.4 Was it the clocks? What cut the pops

Condition	Window	Isolated pops
220 Hz tone at −30 dBFS over A2DP, A32 volume 47, zero-data floor on	45.1 minutes	1 (0.022/min), about 150 times fewer than at drive 2. A shower in the house during the run produced no false events.
Constant word `0xFF000000`, A2DP streaming (the strongest rate amplifier found)	90.0 minutes	0, and one stall (39 ms) with no event near it

The 90-minute capture ran at 47 997 samples/s and lost 0.011 % of the time, which at drive 2′s rate would hide 0.03 of a pop. Its upper bound is about 0.03/min: not proof of a cure.

Drive 3 became the start-up default, read back as 3 from all three pads after a cold power cycle.

What it is. It cut the pops about 150 times in the measured conditions. The story offered for it is a hypothesis; no oscilloscope has ever seen an edge. It runs like this. GPIO0 drives two converter clock inputs down a harness stub. The DAC's fallback clock recovery is off while an external SCK is present. A stiffer edge stops a marginal sampling instant latching the wrong bit. The data rate leaves 354 ns per bit, so the question is edge quality, not timing. It is a margin fix, not a root cause: it says the DAC was latching wrong bits on a marginal edge, not why the edge was marginal. The root cause was never found.

It cannot stop MCLK by accident. GPIO0 is an RTC-capable pad, and the framework writes only the pad's drive field; the pad's function select is a separate bit.

Prior art, found on the web, generic to this DAC and the ESP32 (no complete search was made for this exact arrangement, one ESP32 master clock feeding both converters):

- TI's answer to a "click noise" case on a sibling DAC was to ground its system clock so the internal PLL runs from BCK (TI E2E 1052863). That is how this radio was wired before the clock-wiring change, when the pops were harder.
- TI publishes no data-to-bit-clock setup and hold figure for this DAC family (TI E2E 1542019). At a 354 ns bit period timing is not the issue; edge quality is.
- Nobody else reported pops that survive a constant word, or fixed pops with `gpio_set_drive_capability()`.

Two other changes of the same night, neither measured as a pop fix:

- The Bluetooth transmit ceiling of 0 dBm was added as a mitigation. Its effect on the pops was never measured.
- The state message's 4 Hz query of the controller's transmit power was removed after an A32 reboot was blamed on it. The blame was later withdrawn: the uptime read had been the S3′s, both boards had restarted

within a minute, and the set had been power-cycled. The cause of that reboot was never established the removal was a precaution, not a proven fix.

H.5 What did drive 3 cost?

The trade-off as heard. At drive 3 the radio path was heard as distorted, like people lightly talking through kazoos. At drive 2 the radio sounded right, but the pops came back hard, like full-on popcorn, more than ever before. Both were reproduced by changing `sys.clkdrv` and nothing else. It was the first result pointing at the clock lines shared by both converters rather than at either part: the setter moves all three pads together, which is why it had looked like one knob.

Drive 3 never eliminated the pops. They had been heard at drive 3, on BT, with the tube radio in or not, and both sources popped alike. Every measured number was from the Bluetooth path only because that is what had been streamed: a limit of the evidence, not a property of the fault. The drive-3 soak had Bluetooth streaming, so no drive-3 measurement with Bluetooth idle exists. A hardware item suspected on the master-clock line had always been fitted (Hardware Bible, chapter 4 (p. 46)), so it could not explain any change.

The burst. At drive 2, RADIO source, A32 volume 8, the recorder counted 932 pops in 5.1 minutes (183/min), 55 times the earlier drive-2 rate. None coincided with a stall; the median size was −14.7 dBFS, and 879 of the 932 fell in one 10 dB band (−20 to −10 dBFS), unlike the three tight families of the earlier evening. Five conditions differed from the earlier runs and none was isolated. The volume of 8 did not hide the pops; they were plainly audible. Decoupling capacitors had been fitted the day before (Hardware Bible, chapter 4); they may be part of the cause. Untested.

Two tests designed and never run:

- The split drive (`sys.clkpair`, through `setClockDriveSplit()`): drive MCLK and the BCK/LRCK pair apart. The prediction was that MCLK at 2 with BCK/LRCK at 3 (`23`) would give both good halves, because the DAC latches its data on BCK edges while the ADC's conversion timing derives from MCLK; `32` tests the opposite assumption. It is in the firmware, never run.
- The volume swing: alternate the A32 volume between 47 and 94 while a tone streams, and see whether the pops move with the tone (an error proportional to the sample) or stay put (a fixed wrong code at the DAC).

H.6 Where it stands

The hardware changed, not the firmware. The audio cables were changed (Hardware Bible, chapters 6 and 10), and even before those changes no pops had been heard since ferrite cores were added. No firmware changed for it, and no instrumented pop count exists after it.

The decision. Drive 3 stays the start-up default, and drive 2 will not be tried again: drive 3 works with no apparent consequences, and there is nothing to gain from drive 2. `sys.clkdrv` stays in the firmware so that drive 2 against 3 can be tested again, or drive 2 put back, without a reflash. `sys.clkpair` stays in the code, never run.

H.7 What is still open

- The root cause of the pops.
- The 183/min burst, and whether the decoupling capacitors played a part.
- The "kazoo" distortion, judged by ear only. No recording of it exists, so it and the pops were never put on one scale, and it has not been measured since the cable changes.
- Why the A32 rebooted during the hunt.
- A drive-3 measurement with Bluetooth idle, the split drive, the volume swing, and the Bluetooth transmit-power A/B: none was run.
- No waveform has ever been seen. There was no oscilloscope; distortion was judged by ear.
- The firmware cannot see a DAC-side clock error. No telemetry exists for it.

The lesson on method. Three theories died in the first round alone, each to a measurement and not an argument, and each had been stated more confidently than the evidence allowed. A measurement consistent with a hypothesis is not one that discriminates between hypotheses.

H.8 How to count pops

If the pops come back, this is the method that worked here.

The recorder.

- A microphone recorder about a foot from the radio, streaming continuous 48 kHz audio over USB, with its own clock logged beside its sample count: a dropped chunk is a splice, and a splice looks like a click.
- Read it in bulk with a large driver buffer (1 MB here). The first attempt ran at 38 kHz with 18 lost chunks. A good capture runs at 47 99x of 48 000 samples/s.
- Turn the microphone gain down far enough not to clip a foot from a speaker.

The detector.

- The 1 ms RMS of a high-passed signal (a 10-tap filter).
- An absolute threshold of −35 dBFS, with −45 dBFS as a secondary.
- 50 ms dead time.
- An isolation score: the event's level minus the median over ±200 ms. Real pops scored 25–58 dB; crackle, speech and handling scored below 15.
- Have a listener's ear confirm clips of each size of event before trusting the detector.
- Do not use a shape filter ("falls 10 dB in 5 ms"): pops ring, and such a filter kept only 29–61 % of real ones.

- If a heat gun is used as a trigger, keep it away from the microphone.

The design of a run.

- One-minute alternation, 15 pairs or more, both arms in the same recording, a source actually streaming, and a stratified per-pair exact test.
- Never compare totals, or one long block per arm: the rate changes by the hour (whole mornings of nothing, then bursts).
- A silent link sends exact zeros and hides pops, so use `sys.dctest 1` to make them visible whatever plays.
- Decide the rule before the run (here: fewer than 6 events is inconclusive).
- Close the last window at the end of the run, not at the end of the recording.
- If events cluster (a crackle storm), the per-pair test is invalid: do not cite its p-value. Two storms once crossed a music-against-DC alternation and made its p-value (9.94e-28) invalid: 199 of its 218 music events came from the storms, so the events clustered and the test's independence did not hold.
- Check the stall counter and the timestamps to tell a stall click (step-shaped; it correlates with a recorded switch step) from a pop (an impulse).
- A reboot or an update pops by itself, so check the flash log before counting a pop "on its own".

Traps that cost real time here.

- Portal sessions expire after 5 idle minutes. A long script must log in before every portal call, or its commands are silently refused. A whole 30-minute A-B-A soak was lost that way, and only the pad readback showed it.
- A script stamps a switch after its command and readback, so the real switch lands about 1.2 s before the stamp.
- Never edit a shell script while a copy of it runs: the shell re-reads the file and the running copy dies.
- A build-and-flash chain must test the build's own exit code. `pio run | grep` returns grep's status and would flash a failed build.
- A cleanup trap that restores a setting must also exit, or the script carries on toggling with no recorder.
- Two readers of a read-and-reset counter corrupt each other. A 40-minute "dropout" log was entirely the A32 console `s` draining the peak meter.

Rates to beat, each with its window:

Table 182. H.8 How to count pops

Condition	Window	Pops
Drive 2, 220 Hz tone over A2DP	936 s	52 (3.33/min)
Drive 3, same run	932 s	0 above −35 dBFS
Drive 3, tone over A2DP	45.1 minutes	1
Drive 3, constant word, A2DP streaming	90.0 minutes	0
Drive 2, RADIO, A32 volume 8	5.1 minutes	932

Say the window and the bound, never the bare word “zero”.

Appendix I. Pins as the firmware uses them

This appendix is for anyone who changes the firmware, ports it to other boards, or checks a pin with a meter or a scope. It lists, for both boards, every pin the firmware drives or reads, the name the firmware gives it, and what the firmware does with it: input or output, any internal pull-up it switches on, and the level it treats as active. It says nothing about what a pin is wired to. For wiring, see the Hardware Bible, chapter 4 (section 4.2 (p. 47) for the main board, section 4.3 (p. 50) for the audio board), and its chapter 10 (p. 256) for the cables.

All the names come from one file, `include/pins.h`, compiled into both programs. It is the firmware’s pin authority: a pin moves there and nowhere else. The peripherals behind these pins (timers, PWM channels, buses) are listed in chapter 4 (p. 46), section 4.12.4 (p. 64).

I.1 Main board (S3)

Table 183. I.1 Main board (S3)

GPIO	Firmware name	Direction, as set	What the firmware does with it	Chapter
1	`S3_AMP_SENSE`	input, plain (no internal pull)	Read on every pass of the main loop. Low = amplifier on. A change counts after 50 ms.	4, section 4.5 (p. 52)
2	`S3_DIGIT_2`	output	Digit select for the tens of minutes. High = digit on. In the low output register.	4, section 4.6 (p. 53)
5	`S3_INDEX_HALL`	input, plain (no internal pull)	The needle’s index. Low = index magnet present.	5, section 5.8 (p. 83)
6	`S3_SEG_A`	output	Segment A. Driven high during a digit’s lit window when the segment is in that digit’s pattern; high lights the segment. In the low output register.	4, section 4.6 (p. 53)
7	`S3_SEG_B`	output	Segment B, as segment A.	4, section 4.6 (p. 53)
8	`S3_SEG_F`	output	Segment F, as segment A.	4, section 4.6 (p. 53)
9	`S3_STEPPER_I1`	output, PWM	Needle motor phase; LEDC channels 2–5 at 20 kHz serve the four phases.	5, section 5.4 (p. 78)
10	`S3_STEPPER_I2`	output, PWM	Needle motor phase, as above.	5, section 5.4 (p. 78)
11	`S3_STEPPER_I3`	output, PWM	Needle motor phase, as above.	5, section 5.4 (p. 78)
12	`S3_STEPPER_I4`	output, PWM	Needle motor phase, as above.	5, section 5.4 (p. 78)
13	`S3_UART_TX`	output, serial	The link to the A32: sends, on `Serial2` at 921 600 baud, pin given explicitly.	9, section 9.5 (p. 231)

GPIO	Firmware name	Direction, as set	What the firmware does with it	Chapter
14	`S3_UART_RX`	input, serial	The link to the A32: receives.	9, section 9.5 (p. 231)
15	`S3_SEG_C`	output	Segment C, as segment A.	4, section 4.6 (p. 53)
16	`S3_SEG_D`	output	Segment D, as segment A.	4, section 4.6 (p. 53)
17	`S3_SEG_E`	output	Segment E, as segment A.	4, section 4.6 (p. 53)
18	`S3_SEG_G`	output	Segment G, as segment A.	4, section 4.6 (p. 53)
21	`S3_PANEL_PWM`	output, PWM	The panel lamps: LEDC channel 1, 1 kHz, 8-bit duty. Higher duty = brighter.	4, section 4.7 (p. 55)
38	`S3_AS5600_SDA`	I2C data	Bus `Wire` (I2C 0), 100 kHz: the tuning-angle sensor (AS5600) at address 0x36.	5
39	`S3_AS5600_SCL`	I2C clock	As above.	5
40	`S3_DIGIT_3`	output	Digit select for the units of hours. High = digit on. In the high output register.	4, section 4.6 (p. 53)
41	`S3_DIGIT_4`	output	Digit select for the tens of hours (the leftmost digit). High = digit on. In the high output register.	4, section 4.6 (p. 53)
42	`S3_DIGIT_1`	output	Digit select for the units of minutes (the rightmost digit). High = digit on. In the high output register.	4, section 4.6 (p. 53)
47	`S3_RDA_SDA`	I2C data	Bus `Wire1` (I2C 1), 100 kHz: the RDA5807M. Printed by console `R`.	6, section 6.6 (p. 128)
48	`S3_RDA_SCL`	I2C clock	As above.	6, section 6.6 (p. 128)

The rule the display depends on. All seven segment pins stay below GPIO 32, because the display interrupt clears segments through the low output register in one write; digit pins may be anywhere (chapter 4 (p. 46), section 4.17 (p. 70)).

Other values in the same file: `S3_I2C_HZ` = 100 000, the clock of both I2C buses; `AS5600_ADDR` = 0x36. `RDA5807_ADDR` (0x11) is defined but unused: the tuner's driver carries its own address constant (chapter 6 (p. 120), section 6.6 (p. 128)).

What no S3 pin does. No S3 pin drives a fan, the display's colon, the blue Bluetooth lamp, or anything on the A32's side. GPIO 4 is not used; `pins.h` calls it "the only spare".

Pins the S3 firmware leaves alone, as `pins.h` lists them. A new function must take a pin outside these, and outside those already used.

Table 184. I.1 Main board (S3)

GPIO	Why
35, 36, 37	Taken by the octal PSRAM of an R8 module, per the module's datasheet.
0, 3, 45, 46	Strapping pins, read by the chip at reset.
19, 20	Native USB: the console.
43, 44	The chip's first UART, kept free.

I.2 Audio board (A32)

Table 185. I.2 Audio board (A32)

GPIO	Firmware name	Direction as set	What the firmware does with it	Chapter
0	A32_PCM1802_MCLK	output	The I2S master clock, 11.2896 MHz (256 × 44 100), for both converters. Forced: the classic ESP32 can put the I2S master clock only on GPIO 0, 1 or 3, and 1 and 3 are its console. Never detached from the I2S peripheral. Pad drive 3 at start-up.	10
4	A32_PCM5102_DIN	output	I2S data out, to the DAC. Its pad drive can be changed from the portal (sys.dindrv, 0 to 3).	10
17	A32_I2S_LRCK	output	The shared I2S frame clock (LRCK), 44.1 kHz. Pad drive 3 at start-up.	10
18	A32_I2S_BCK	output	The shared I2S bit clock (BCK), 2.8224 MHz. Pad drive 3 at start-up.	10
19	A32_PCM1802_DOUT	input	I2S data in, from the ADC.	10
21	A32_RTC_SCL	I2C clock	Bus Wire, 100 kHz (A32_I2C_HZ): the battery clock (DS3231) at address 0x68. Starting the bus switches on the ESP32's internal pull-ups.	9, section 9.9 (p. 240)
22	A32_RTC_SDA	I2C data	As above.	9, section 9.9 (p. 240)
25	A32_BT_BUTTON	input, internal pull-up on	The Bluetooth pair button. Low = pressed. Debounced in firmware: a press counts on release, after being held more than 30 ms.	10
26	A32_UART_RX	input, serial	The link to the S3: receives, on Serial2 at 921 600 baud, pin given explicitly.	9, section 9.5 (p. 231)
27	A32_UART_TX	output, serial	The link to the S3: sends.	9, section 9.5 (p. 231)
33	A32_BT_LED	output, PWM	The blue Bluetooth lamp: LEDC channel 0, 2 kHz, 8-bit. Driven active low: the firmware pulls the pin low to light the lamp. It drives it fully on for 300 ms of every 600 ms in the LINK state, at the top of every LOOK breath, and in both FOUND flashes (the lamp's states are in chapter 10 (p. 256)).	10
35	A32_VOLUME_POT	input, analogue	The volume knob, read every 50 ms with the ESP32's own ADC at its widest input range (11 dB attenuation): two throw-away conversions, then the median of five.	10
36	A32_MODE_ADC	input, analogue	The source selector, read every 50 ms: three throw-away conversions (this pin reads high for a while after another channel of the ESP32's own ADC has been sampled), then the median of five. Raw below 424 = AUX, 2471 and above = RADIO, anything between = BT.	10

Other values in the same file: A32_I2C_HZ = 100 000; the selector thresholds A32_MODE_THRESH_AUX_BT = 424 and A32_MODE_THRESH_BT_RADIO = 2471, which the S3's console s also prints. DS3231_ADDR (0x68) and AT24C32_ADDR (0x57) are defined but unused: the battery clock's driver carries its own address constant.

Why the link's pins are given explicitly. The classic ESP32's default pins for Serial2 are GPIO 16 and 17, and GPIO 17 is the I2S frame clock. A serial port

opened without its pins would silently take it (chapter 9 (p. 224), section 9.14.2).

What no A32 pin does. No A32 pin drives the DAC's hardware mute: the mute is software only, and GPIO 16 is unused (chapter 10 (p. 256)).

Pins the A32 firmware leaves alone or does not use, as `pins.h` lists them:

Table 186. I.2 Audio board (A32)

GPIO	Status
2, 5, 12, 15	Strapping pins: left alone.
6 to 11	The flash chip: left alone.
34 to 39	Inputs only, with no internal pull-ups or pull-downs; the volume knob (35) and the selector (36) use two of them.
13, 14, 23, 32	Unused.
16	Unused. GPIO 16 and 17 are free only on a module without PSRAM: a PSRAM module takes them, and GPIO 17 is the I2S frame clock.

Index

"... would add the least coverage", or a sample replaced: 142
"ABANDONED ... check its connector": 142
"flash" on the page for a main-board file for the wrong chip: 292
"MARKS REFUSED (...)", red message: 143
"Measuring pass speed" says less than it does. The row (reap: 294
"NOTHING FOUND ... nudge it": 142
"pairing window open", and nothing opens. Pair a new device : 299
"pairing window open", but nothing opens: 291
"QUADRATIC REFUSED" or "the marks are degenerate": 143
"ROLLED BACK" stays in every start-up report of both boards : 300
"stopped at a soft limit - the band runs past it; aborted": 110
"tuner ends NOT measured" on every surface: 143
"TUNING MARKS REFUSED - the file announced 0 and 0 parsed", : 292
- an earlier update was ROLLED BACK: 40, 242
- an earlier update was ROLLED BACK at every start-up; the s: 292
- an earlier update was ROLLED BACK at the end of the image : 10
1017 or 0879: 11
12-hour or 24-hour, not as wanted: 11
8888, or dark while playing: 11
[A32] settings NOT saved - the flash write failed; retrying: 172, 244
[FAIL] proto self-test - this image will NOT be confirmed: 10, 40
[FAIL] proto self-test - this image will NOT be confirmed; .: 215
[FAIL] the access point will not start ...: 213
[WARN] A32 silent and the link pill red, yet the sound plays: 243, 292
[WARN] A32 silent. Clock and needle keep running.: 65
[WARN] A32 silent. Clock and needle keep running. or [WARN] : 243
[WARN] A32 silent., then the same as above: 174
[WARN] A32 silent.; link SILENT; AUDIO BOARD SENDS NO STATE;: 292
[WARN] index not found in the expected direction ...: 109
[WARN] settings NOT saved - the flash write failed.; Save an: 172
[WARN] the A32 restarted - handshaking again.: 65
[WARN] the DS3231 has no valid time yet. at each reading; pi: 244
[WARN] the DS3231 has no valid time yet. every minute, pill : 66
[WARN] wifi dropped.; the access point appears: 213
A blank board's access point cannot be joined: 213
A blank board's access point cannot be joined; the portal sl: 291

A blank board's access point transmits at 2 dBm. The ladder : 296
A broken audio-board image on trial says the wrong thing. An: 300
A build reports success but the exit code is not 0: 40
A change gone after a main-board update: 174
A change made during the A32's trial is gone after a restart: 280
A click at the start of an audio-board update when the fade-: 301
A click at the start of every audio-board update; a click wi: 291
A click; stalls rises: 279
A direct write of the position can race a pending correction: 294
A good new program is gone after a power cut or a reboot: 243
A home after a crash, watchdog or software restart: 111
A main-board upload stops part-way: 214
A measurement REFUSED, DISCARDED, ABANDONED or NOTHING FOUND: 291
A new main-board program runs but never confirms: 215
A new S3 program misbehaves, still on trial, portal reachabl: 40
A new S3 program never confirms: 40
A new S3 program ran healthily for a minute, confirmed itsel: 66
A one-way link failure leaves the audio board on a stale amp: 298
A phone connects while the radio is off or on another source: 280
A pop: 281
A portal jog stutters: 291
A portal reboot refused: 110
A portal-raised access point is confirmed on the console onl: 296
A reboot during a trial: 110
A relay that fails mid-image sends no abort. The audio board: 299
A relayed A32 program crashes or hangs at start-up: 40
A relayed A32 program runs but never completes a handshake: 41
A restart and a home; s shows the reset reason TASK WATCHDOG: 111
A restart; the boot report shows reset reason 6: 242
A restore did not bring back the accounts or the WiFi networ: 12
A setting back at its old value after a power cut: 172
A settings upload with a warning hides the count: the page s: 296
A settings write can land while the needle moves. "Still" is: 295
A sweep stops halfway at power-up: 109
A warning at start-up, and defaults: 173
A wrong-way first search after a software restart: 110
A32: still on trial - retry in 1 min: 40
A32: still on trial - retry in 1 min (err 0): 243
A32_BT_BUTTON: 360
A32_BT_LED: 360
A32_I2C_HZ: 360
A32_I2S_BCK: 360
A32_I2S_LRCK: 251, 360
A32_MODE_ADC: 360
A32_MODE_THRESH_AUX_BT: 270, 360
A32_MODE_THRESH_BT_RADIO: 270, 360
A32_PCM1802_DOUT: 360
A32_PCM1802_MCLK: 360
A32_PCM5102_DIN: 360
A32_RTC_SCL: 360
A32_RTC_SDA: 360
A32_UART_RX: 360
A32_UART_TX: 360
A32_VOLUME_POT: 360
ABANDONED, "hold it still": 142
About 3.3 s before anything starts. At every start-up the ma: 302
ACTIONS: 208
After an A32 update the radio stays silent, and no handshake: 242
After W, the old time comes back: 66
An audio edit lost just before the audio board fell silent; : 292
An audio edit made just before the audio board falls silent : 299
An audio-board image built with the wrong PROTO_VERSION conf: 298
An audio-board upload stops part-way: 214
An S3 update fails without a restart: 40
An unreadable tuneUsed line. It is skipped with no note, and: 296
An update fails with the updater's error 8 (magic byte): 242
An upload is refused for the wrong chip: 40
An upload refused (wrong chip, on trial, audio board not ans: 292
An upload stops halfway: 40
AP_SSID: 321
AS5600_ADDR: 359
AT24C32_ADDR: 360
Audio and Bluetooth settings back at their defaults after an: 174
Audio changes gone after a power cut, or an

audio-board rest: 172
Audio rows show n/a, or edits are refused with "the audio bo: 12
Audio rows shown as n/a; audio-board changes do not hold: 174
Audio settings back at their defaults after an update: 280, 292
Audio settings back to defaults after an audio-board update: 12
Audio-board console changes are undone. The volume law (t, T: 300
AUTO_GAP_MS: 141
AUTO_STILL_MS: 141
AUTO_TRIES_MAX: 102
autoConnect is still in the protocol, stored by the audio bo: 299
BAND_BUDGET: 102
BAND_MARGIN_HS: 102, 107
Blank digits: 11
BLOCK: 262, 345
BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both: 292
BOARDS RUN DIFFERENT PROTOCOL VERSIONS - flash both; console: 174
BT: the disconnect did not land - still connected; the lamp : 280
BT_CON: 273, 289
BT_DROP_TIMEOUT_MS: 289, 345
BT_FOUND: 273, 289
BT_LINK: 273, 286, 299
BT_LOOK: 273, 299, 345
BT_OFF: 273
BT_ON: 273
BT_STDBY: 273, 345
CFG_SPURS: 140, 141, 151, 183, 319
Changes gone after a power cut just after a main-board updat: 172
Clock a little off: 66
Clock blank, lamps dark, needle still: 66
CONFIRM_AFTER_HELLO_MS: 319, 340
CONFIRM_AFTER_MS: 45, 64, 319, 340
console busy: 215
CORRECT_MAX_HS: 89, 102, 107, 108, 117, 118
crc mismatch or end failed: 243
CTRL_US: 102
DEAD_TICKS: 64, 70
DEFAULT_PASS: 323
DEFAULT_USER: 323
DEVICE_NAME: 345
DISCARDED: 142
DISCARDED after an angle sensor outage: 142
DISCARDED, "press measure again" or "it will retry by itself: 142
Drop one sample also drops hand marks. It accepts slots 0 to: 297
Drop one sample removed a hand mark: 291
DRV_FULL: 79, 308
DRV_HALF: 79
DRV_LEDC_BITS: 102
DRV_LEDC_FREQ_HZ: 102
DRV_MICRO: 79
DRV_WAVE: 79
DS3231_ADDR: 360
Eastern North American time after W: 291
ENV_ONE: 345
ESP_PWR_LVL_N0: 345
ESP_PWR_LVL_N12: 342
EST5EDT: 188, 220, 333
Every A32 update starts with a click: 280
Every measurement refused; the RDA state says "NOT FOUND on : 142
Every new sample off by the same amount: 143
Every reading off by the same amount: 143
Every reading off by the same amount; one sample with a larg: 291
FINE_PER_EREV: 102
FINE_PER_HALFSTEP: 102
FINE_POINTS: 141
Fixed features appear at multiples of 2.8224 MHz. One suspec: 293
Fixed features gone after start-up, or a settings file's lis: 143
FONT: 64
FORCE_HOLD_MS: 197, 321
Full volume or silence after a change to the audio interface: 291
FW_COMMIT: 24, 37, 39, 72, 227, 236, 340
FW_VERSION: 24, 37, 38, 39, 44, 72, 227, 236, 340
HALFSTEPS_PER_REV: 103, 104, 117, 308
HI10: 141, 152
HOME_BACKOFF_BUDGET_HS: 86, 102
HOME_BUDGET_HS: 86, 102, 107
HOME_REAPPROACH_BUDGET_HS: 86, 102
HTTP 409 the audio board is not answering - its settings can: 244
HTTP_MAX_DATA_WAIT: 207, 220
HUNT_NETMOVE_MAX: 97, 102
HUNT_STEPS_MIN: 97, 102
HUNT_WINDOW_MS: 97, 102
i calls normal powers "LOW". It adds *** LOW - the transmitt: 295
i says OWN ACCESS POINT while joining; B, o or O say "saved": 292
I2S_CHANNEL_FMT_RIGHT_LEFT: 260
I2S_COMM_FORMAT_STAND_I2S: 260
I2S_NUM_0: 260
IDLE_CHECK_MS: 91, 103
IDLE_MS: 323

IDLE_SOCKET_MS: 207, 323
IDX_DEBOUNCE_K: 83, 102, 104, 107, 108, 117
image confirm FAILED (err N) - retrying: 41, 243
IN_CAP: 211, 324
IRAM_ATTR: 70
JOIN_MS: 196, 321
K_BOOL: 180
K_ENUM: 180
K_FLOAT: 180
K_INT: 180
link SILENT on one or both boards: 243
LIT_MAX: 64, 71
LO_WINDOW_HI10: 293
LOG_CAP: 211, 324
LOST_AFTER: 141
MAJOR: 39
MAX_BAD: 323
MAX_SESS: 323
MAX_USERS: 323
MEM_MAGIC: 96, 102, 118
MEM_POS_SANE_HS: 96, 103, 107
MIGRATED: 186
MSG_ADC_CLOCK: 247, 342, 347
MSG_BT_CMD: 341
MSG_BT_FORGET: 341
MSG_BT_LOOK: 286, 299, 341
MSG_BT_PAIR: 299
MSG_BT_TX: 342, 347
MSG_CAL_POT: 341
MSG_CFG: 226, 304, 342
MSG_CLK_DRIVE: 342, 347
MSG_CLK_PAIR: 236, 343, 347
MSG_DC_WORD: 342, 347
MSG_DIN_DRIVE: 342, 347
MSG_GET_CFG: 300, 342
MSG_GET_TIME: 341
MSG_HELLO: 341
MSG_HELLO_ACK: 308, 341
MSG_LOG: 342
MSG_OTA_ABORT: 239, 342
MSG_OTA_BEGIN: 44, 239, 240, 250, 251, 301, 340, 342
MSG_OTA_DATA: 239, 340, 342
MSG_OTA_END: 239, 340, 342
MSG_OTA_STATUS: 239, 342
MSG_PING: 70, 221, 341
MSG_PONG: 341
MSG_REBOOT: 341
MSG_SD_DELAY: 247, 342, 347
MSG_SET_AUDIO: 181, 226, 304, 341
MSG_SET_BT: 181, 226, 341
MSG_SET_SYS: 226, 229, 341
MSG_SET_TIME: 226, 341
MSG_SIGN_TAIL: 342, 347
MSG_STATE: 226, 228, 342
MSG_TEST_DC: 342, 347
MSG_TIME: 226, 342
MSG_USE_POT: 342, 347
MSG_ZERO_FLOOR: 342, 347
Music at the wrong pitch: 279
Mute off after a restart: 12
NEEDLE FAULT after an automatic re-home: 109
NEEDLE FAULT at power-up or after Home: 109
Needle frozen, clock display dark: 110
Needle frozen, or the portal dead, for good: 111
Needle holds still while you tune: 110
NEEDLE HUNTING on the portal: 110
Needle limits back at ±300: 12
NEEDLE OFF - re-index pending (try n/3), with n rising: 109
NEEDLE OFF on the portal: 109
NEEDLE OFF, NEEDLE RE-INDEXING, NEEDLE FAULT, NEEDLE HUNTING: 291
Needle pinned at one end while tuning: 110
Needle resumes after an update that did not finish: 110
No "unsaved" sign on the page, for example during the trial : 296
no answer from the radio.: 214
No captive portal: ... line on the console: 213
No live figure for the audio board's free memory. The one fi: 301
No lock around the settings across the two cores. Known race: 295
No network time without the audio board. If the audio board : 298
No radio or Bluetooth sound; rx 0 with wrong-version climbin: 244
No warning for a needle gain that will hunt. The start-up ch: 294
not rebooting - <why>: 214
NOT saved yet - this firmware is on trial after an update ...: 10
Nothing: 109, 174, 175, 245
Nothing at first: 143
Nothing at first; console s shows a large residual on one sa: 142
Nothing: the audio no longer mutes during a measurement: 143
Nothing; crc counts one more: 243
NTP_FRESH_MS: 64, 322
OFF: 229
Off by minutes: 11
Off by whole hours: 11
Off the house network, Ambersong appears; a phone joins but : 291

one key at a time - each character is its own command: 215
one key at a time ..., console busy; the display freezes and: 291
One pop at every audio-board restart or update: 291
OTA_BEGIN: 287
OWN_A32_AUDIO: 180, 185
OWN_A32_BT: 180, 185
OWN_S3: 180, 184
P3: 342
Peak or ring readings missing on the portal after the audio : 291
Pill BATTERY CLOCK LOST ITS TIME - check its battery: 11
Pill BATTERY CLOCK NOT ANSWERING: 11, 66, 244
PING: 298
POLL_MAX: 324
POLL_MIN: 324
Pops: 291
POT_DEADBAND: 319, 346
PROBE_GAP_MS: 141
proto self-test FAILED - muted, BT closed, awaiting OTA ever: 242
proto self-test FAILED - muted, BT closed, awaiting OTA; sil: 292
PROTO_BAUD: 339
PROTO_MAX_PAYLOAD: 39, 319, 339, 343
PROTO_SILENCE_MS: 339, 346
PROTO_SOF0: 338, 339
PROTO_SOF1: 338, 339
PROTO_STATE_MS: 339, 346
PROTO_VERSION: 23, 37, 39, 43, 44, 185, 236, 250, 253, 286, 297, 299, 304, 306, 338, 339
PROTO_VERSION_LEN: 39, 339
PROVISIONAL_LIMIT_HS: 94, 103, 107, 108, 118, 319
Q16: 257, 268, 284, 345
Q24: 257, 268, 281, 284, 345
R2_MONO: 308
R8: 359
Rare drops or underruns in a long stream: 279
RDA5807_ADDR: 359
Re-index answers "re-indexing" but homes; steps lost after r: 291
Re-index answers "re-indexing" when it homes. On a needle wi: 294
Re-index on a needle with no zero answers "re-indexing": 109
Readings shared between tasks without a lock. A status line : 293
Readout mode 1 shows a stale frequency. With the readout on : 302
Reboot the main board answers not rebooting - <why>: 174
Reboot the main board shows only "rebooting". On trial the r: 296
REFUSED: 142
REFUSED notes, or a block refused: 173
REFUSED, "a different signal": 142
REFUSED, "Move the dial 0.3 MHz": 142
REIDX_STILL_MS: 91, 103
RETRY_MS: 196, 321
RING_FRAMES: 262, 345
RTC_NOINIT_ATTR: 96
s shows *** TASK WATCHDOG ***: 66
s shows BROWNOUT: 66
s shows INTERRUPT WATCHDOG: 66
s shows PANIC: 66
S3: 3, 4, 5, 6, 16, 17, 18, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 55, 56, 57, 58, 59, 60, 61, 62, 65, 66, 67, 68, 69, 70, 71, 72, 73, 100, 119, 153, 155, 186, 187, 189, 190, 191, 192, 193, 194, 195, 197, 199, 204, 207, 209, 210, 211, 215, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 262, 269, 270, 273, 275, 276, 277, 279, 280, 282, 283, 298, 306, 309, 312, 314, 315, 316, 318, 321, 323, 324, 327, 328, 333, 336, 338, 339, 340, 341, 342, 343, 344, 346, 350, 354, 359, 360
S3_AMP_SENSE: 358
S3_AS5600_SCL: 75, 359
S3_AS5600_SDA: 75, 359
S3_DIGIT_1: 359
S3_DIGIT_2: 358
S3_DIGIT_3: 359
S3_DIGIT_4: 359
S3_I2C_HZ: 359
S3_INDEX_HALL: 75, 358
S3_PANEL_PWM: 359
S3_RDA_SCL: 359
S3_RDA_SDA: 359
S3_SEG_A: 358
S3_SEG_B: 358
S3_SEG_C: 359
S3_SEG_D: 359
S3_SEG_E: 359
S3_SEG_F: 358
S3_SEG_G: 359
S3_STEPPER_I1: 75, 358
S3_STEPPER_I2: 358
S3_STEPPER_I3: 358
S3_STEPPER_I4: 75, 358
S3_UART_RX: 359
S3_UART_TX: 358

SAMPLE_RATE: 345
Samples are stored to 100 kHz. The fine pass measures in 50 : 293
Samples stored up to 50 kHz high; the fine-pass list shows a: 291
Saving the Network card always rejoins, 3 s later, even for : 296
SEAT_MARGIN: 103, 108, 141
SET_SYS: 298
Settings: 292
Settings changed during a trial are gone: 41
Settings changed during the A32's trial are gone: 244
SETTINGS LOCKED - newer version stored: 12
SETTINGS LOCKED - newer version stored; Download refused: 173
SETTINGS LOCKED after a deliberate downgrade: 41
SETTINGS LOCKED; a setting back at its old value after a pow: 292
settings NOT saved - the flash write failed; retrying: 280
Settings that make no sense after a start-up, such as the ne: 173
SETTINGS_MAGIC: 72, 318
SETTINGS_VERSION: 37, 70, 72, 118, 151, 182, 184, 318
SF32: 180
SI16: 180
SI32: 180
Sign-in refused with a wait: 214
Silence for up to 20 s after an update stopped mid-transfer: 280
Silence from start-up; [FAIL] on the A32's USB console: 279
Silence while everything else works: the audio task is not w: 301
Silence, lamp dark, Bluetooth closed: 279
Silence, or the radio at full volume: 279
Silence; the link, the portal and the lamp work: 279
SILENT: 4, 232, 333
Silent, invisible to phones; proto self-test FAILED - muted,: 280
SLOT_TICKS: 64
SMP_AUTO_MARGIN: 141
SMP_MAX_DISAGREE10: 141
SMP_MAX_DRIFT: 108, 141, 307
SMP_MIN_MARGIN: 141
SMP_TRUST_AFTER: 141
Soft limits back at ±300 hs: 110
Soft limits back at ±300 hs; the console asks for a recalibr: 173
SOFT LIMITS REFUSED (a..b does not bracket the index) - kept: 173
Some values of an upload present, others not, after a power : 173
Sound and Bluetooth: 291
SU16: 180
SU8: 180
SUCCESS: 43
sys.quiet casts its number unchecked. Nothing in practice: a: 297
TAB_AUDIO: 180
TAB_BT: 180
TAB_SYSTEM: 180
that image is not for the audio board (wrong chip) - nothing: 242
the A32 did not confirm the update - check its version once : 40, 243
The A32 falls asleep for up to about 80 s, then wakes by its: 243
The A32 goes quiet while a console prompt waits: 66
the A32 is not answering: 40
the A32's STATE frame does not match this build. Flash BOTH : 65, 244
The administrator's password is lost: 214
the audio board is not answering - its settings cannot be ch: 174, 215, 281
the audio board is not answering ... (HTTP 409); n/a in a do: 292
The audio board says crc ok, [PASS] clock pads, or [ZDD] DAC: 292
The audio board's help line omits k and g. Both keys work (A: 299
The audio board's s steals the portal's readings. It reads t: 301
The audio board's version is on no portal page. It appears o: 299
The audio board's version on no portal page: 292
The battery clock is wrong from the year 2100. The audio boa: 300
The battery clock wrong from the year 2100: 291
The battery clock's temperature is sent and not used. Every : 300
The clock and the display: 291
The clock at the wrong brightness after - or = with the ampl: 66
The clock blank or --:--; a battery-clock pill; the old time: 291
The clock blank with a working network, the audio board abse: 291
The console's - and = (or +) bypass the fade. They change br: 302
The display freezes, the sound mutes: 215
The dump D is only partly a settings file. Its first lines c: 296

The fine-pass list does not mark a broadcast. A point reject: 293
The Firmware card says the needle "parks"; two builds cannot: 292
The first B after a start-up gives 11 dBm. B keeps its own p: 295
The first search after a power cut or brownout goes up and a: 109
The fixed-feature list empty: 173
The frequency readout shows a leading zero below 100 MHz (08: 302
The gap message names the tuner's ends ("AS5600 gap - angle : 293
The image line adds an earlier update was ROLLED BACK: 215
The join fails and the access point is not seen: 213
The knob maps wrongly: 279
The link between the boards: 292
The machine behaves wrongly after an upload: 173
The needle: 290
The needle frozen and the clock dark: 291
The needle holds still while you tune: 291
The needle holds still while you tune; the portal's Now is b: 143
The needle never moves; s shows *** LEDC REFUSED ***: 111
The needle pinned at one end; the first search at power-up g: 291
The Network card's save rejoins; the access point confirmed : 291
the network name is empty: 213
The network, the portal and the console: 291
The new program misbehaves after image confirmed: 10
The new program misbehaves, still on trial: 10
The page reported a failure: 10
The page reports a failure, yet the board restarted: 40
The page says the update failed, but it worked: 214
The pair button blinks the lamp, but no phone finds the radi: 280
The phone joins Ambersong but no page opens: 213
The portal dead but the radio answers a ping; sign-in refuse: 291
The portal dead while the radio, clock and needle work: 291
The portal is dead but the radio answers ping: 213
The portal is dead, but the radio, clock and needle work: 66
The portal is gone for up to 10 minutes after WiFi off: 215
The portal is slow while the needle moves: 214
The portal is starved while the needle moves. The step task : 295
The portal's needle buttons run on the portal task (core 0) : 294
The previous program is back after Reboot the main board (th: 214
The radio goes home on its own: 213
The radio is off the house network; Ambersong appears: 212
The relay stops mid-stream; the sound stays off until the A3: 40
The save before a commanded restart is tried once. When the : 300
The settings download is refused: 215
The sign-in form comes back: 214
The stall click. The audio task is sometimes frozen for 23 t: 301
The top of the dial (107.9 MHz) cannot be measured: 291
The top of the dial cannot be measured. The firmware sweeps : 293
The upload is refused at once: 214
The upload stops partway; the sound stays off until the A32 : 243
The volume does not follow the knob's travel: 279
The wrong brightness after - or =; 0879; a frequency shown w: 291
this firmware is still on trial - retry in a minute: 215
this firmware is still on trial - retry in a minute (HTTP 40: 10, 40
TICK_US: 64
TRACK_ACC_ALPHA: 88, 103
TRACK_DEADBAND_HS: 88, 102
Trouble with a new S3 program while it is on trial: 66
TUNE_DEADBAND: 103, 141
TUNE_MARKS: 140, 151, 319
Tuning and the dial: 291
Tuning wrong after a long angle-sensor outage, or from start: 110
Two main-board update messages. An upload refused during the: 296
TX_LADDER: 198, 322
Typed soft limits rejected: 110
Updates and phones refused (memory): 291
Updates and versions: 292
Updates refused (the updater's error 0), phones cannot conne: 279
upVmax accepts speeds that stall the

needle: up to 4000 half: 294
Version stamps cannot always be ordered.
The stamp comes fro: 302
WDT_TIMEOUT_S: 64, 340
WIFI_ALL_CHANNEL_SCAN: 195
WIFI_AP_STA: 195
WIFI_CONNECT_AP_BY_SIGNAL: 195
WIFI_MODE_NULL: 196, 221
WIFI_PS_NONE: 196, 322
WIFI_STA: 195
Within 2 s the sound fades out, the phone is dropped, the bl: 243
Wrong or missing sound after sys.dindrv 0 or 1: 279
Wrong pitch; rare drops; the volume does not follow the knob: 291
wrong-version N on s, pill BOARDS RUN DIFFERENT PROTOCOL VER: 65
y echoes the WiFi password, onto USB and into the console ri: 297
y shows the password; i says *** LOW ...; the first B gives : 291
ZDD_FRAMES: 345
ZDD_LSB: 345, 348
ZDD_STALL_US: 345
—: 66